

PULSECHAIN RESEARCH

The pDAI stabilization thesis

Observations of PulseChain activity

Complete edition with 73 research appendices
4 October 2026

Sole intended purpose: observing, documenting and interpreting activity on PulseChain.

This report does not provide financial, investment, trading, legal, tax or any other advice. Descriptions of possible mechanisms are research hypotheses, not recommendations or instructions to act.

Aave and Portal · private L2 delivery · stablecoin issuers · Curve and Balancer · supply management · distributed arbitrage · replenishment

The favorable purpose assumption is stated in the opening. Observed capabilities, proposed connections and operating requirements remain distinct. Dated checkpoints apply; this is a research synthesis, not a verified integrated deployment.

Contents

Main report sections and every research appendix are listed below. Each appendix also has its own section contents. Page numbers refer to the numbered PDF pages and are clickable.

Purpose and scope	5
The thesis	5
1. Why pDAI could organize the restored network	6
2. How capital reaches the local markets	6
3. Aave and Portal can advance local claims	7
4. New collateral can reach existing Curve coins	8
5. Common accounting directs connected market arbitrage	9
6. Build support gradually and keep pDAI circulating	9
7. Patient financing needs settlement and collectible income	11
8. Define the benefit to PulseChain participants	13
9. Connect each interpretation to its next milestone	13
Appendix Contracts and local route evidence	14
Direct delivery and optional conversion routes	16
A proposed dollar support benchmark	17
Historical pool coverage and supply management	18
Confirmed governance changes and attribution limits	19
Financing families and native-asset conditions	19
Aave V3 Portal contracts and changes required	20
Origin OUSD	21
Reserve eUSD	21
mStable mUSD	21
FEI and the Maker routes after shutdown	21
Other issuers and shorter routes	23
StakeWise sETH2 and the minting permission	23
Existing lending links and the local Lido branch	24
Basket and vault routes already connect StakeWise assets	24
Silo governance can provide control without changing the owner Safe	25
Synthetix sUSD: bridge output and local collateral recycling	25
Evidence coverage for the complete thesis	27
Appendix index	28

Research appendices

Appendix 1: Two recorded calls to the 86ff contract	31
---	----

Appendix 2: Copied Aave governance: public ownership, upgrade and messaging history	33
Appendix 3: Across and UMA control on PulseChain: a bounded historical audit	39
Appendix 4: Copied Arbitrum gateway mint inventory on PulseChain.....	44
Appendix 5: Copied Arbitrum bridge on PulseChain: recorded activity and token compatibility	50
Appendix 6: Examined PulseChain addresses: established roles and bounded leads	58
Appendix 7: Conditional cross-chain asset representation: May 2023 models	60
Appendix 8: Four copied Curve metapools: bounded historical configuration review	67
Appendix 9: Copied Aave debt repayment: bounded public transaction audit.....	70
Appendix 10: Copied Arbitrum gateway discovery and historical permission audit	75
Appendix 11: Copied Arbitrum escrow gateways: post-fork withdrawal receipt census	79
Appendix 12: PulseChain GRT gateway: mint transaction and allowance reconciliation.....	82
Appendix 13: Native-L2 assets and L1 representations: the May 2023 bridge baseline.....	86
Appendix 14: Across HubPool custody and Aave Portal settlement.....	92
Appendix 15: Native Arbitrum assets around May 2023: a documentary baseline for pARB.....	97
Appendix 16: Balancer Aave wrapper pools: balances, limits and public transaction evidence	102
Appendix 17: Portal, waDAI and repayment: accounting of the proposed loop	108
Appendix 18: Reserve eUSD: bridged collateral and issuance of the existing pool token	115
Appendix 19: Reserve eUSD Main and timelock: recorded governance history.....	121
Appendix 20: Post-fork history of specialized copied Arbitrum gateways.....	124
Appendix 21: Two examined wallets: recorded Aave debt, collateral and repayment rights.....	128
Appendix 22: Withdrawal event coverage and historical supply reconciliation	134
Appendix 23: PulseChain Arbitrum gateway withdrawals: historical execution audit.....	136
Appendix 24: Aave receipts registered in copied Balancer pools	139
Appendix 25: Executed arbitrage and Maker routes after shutdown	142
Appendix 26: Assertion acceptance, source balances and transferable claims.....	149
Appendix 27: Atropa liquidity connections and routes into stability pools	154
Appendix 28: Canonical bridge flows, distributed accumulation and intent.....	158
Appendix 29: PulseChain thesis evidence and authority	161
Appendix 30: Review of the complete pDAI white paper	175
Appendix 31: Coordinated issuance and connected-pool correction	177
Appendix 32: PulseChain bridge, governance and DeFi evidence audit	181
Appendix 33: How source-chain assets could support pDAI demand on PulseChain	202
Appendix 34: Deferred Portal settlement and scheduled pDAI accumulation	221
Appendix 35: Equal-value stablecoin routes: executed issuance, redemption and funding	227
Appendix 36: PulseChain funding conversions and rebasing pool reserves	234
Appendix 37: How bridges, stablecoin issuers and pools can form one pDAI support system	240
Appendix 38: L2 collateral, local stablecoin issuance and the Curve network	260
Appendix 39: StakeWise and Lido routes through PulseChain lending and governance.....	275
Appendix 40: Fifty-million-pDAI execution and complete reinvestment.....	281

Appendix 41: Local pDAI-to-PLS exit evidence and publication boundaries	284
Appendix 42: Local PLS financing, settlement and bounded pDAI support	289
Appendix 43: Finite pDAI exits into PLS: a fee-inclusive support model	299
Appendix 44: Maker adapter state and the remaining pDAI routes	304
Appendix 45: Inventory-supported local pricing and a proposed LP exit	307
Appendix 46: Optimism relays and Across governance on PulseChain	310
Appendix 47: pDAI: profitable pool cycles and coordinated liquidity	315
Appendix 48: PulseChain Portal backing, aDAI index and local arbitrage	321
Appendix 49: Existing aTokens, new Portal issuance and validator access	326
Appendix 50: Validation of the restorative pDAI thesis	330
Appendix 51: End-to-end routes: issuance, arbitrage and settlement	333
Appendix 52: PulseChain Safe governance and rollup controls	345
Appendix 53: Skeptical economic review — independent first round	350
Appendix 54: Skeptical economic review — second round	356
Appendix 55: Skeptical evidence review, round 1	361
Appendix 56: Skeptical evidence review, round 2	366
Appendix 57: Skeptical protocol review — round 1	371
Appendix 58: Skeptical protocol review — round 2	376
Appendix 59: Two-round skeptical review of the PulseChain mechanism thesis	381
Appendix 60: Stability bounds, two-sided correction and adverse tests	388
Appendix 61: Ten-pass review: method, findings and claim boundaries	394
Appendix 62: Routes and operating conditions for a stable pDAI	398
Appendix 63: StakeWise proxy changes and Curve asset routes	406
Appendix 64: StakeWise vaults, Index Coop and reward authority on PulseChain	411
Appendix 65: pDAI accumulation, income and the stability feedback	416
Appendix 66: sUSD: existing pool connections and issuer-specific bridge minting	422
Appendix 67: Synthetix control, copied Optimism messaging and purchased-stable custody	429
Appendix 68: Synthetix councils and their copied PulseChain authority	436
Appendix 69: Synthetix integrations and observed research-wallet activity	440
Appendix 70: The integrated pDAI accumulation-and-peg thesis	447
Appendix 71: PulseChain thesis: three-round comprehension and evidence study	451
Appendix 72: Twelve-point follow-up: local PLS exits and financing conditions	454
Appendix 73: Completeness and clarity of the pDAI white paper	458
Working archive and evidence directory	462

The pDAI stabilization thesis

4 October 2026. Research white paper. Blockchain investigation is read-only. Contract observations retain their dated checkpoints.

Purpose and scope

The sole intended purpose of this report is to observe, document and interpret activity on PulseChain. It does not provide financial, investment, trading, legal, tax or any other advice.

It does not recommend buying, selling, lending, staking or otherwise using any asset, protocol or proposed route. References to a \$1 target, financing, arbitrage, governance changes and possible outcomes are observational research hypotheses, not price forecasts, guarantees or instructions to act. Evidence and interpretations retain their dated checkpoints and stated assumptions. This purpose applies to the entire report, its historical appendices and its supporting files.

The thesis

This paper examines a proposed reconstruction of PulseChain's inherited financial infrastructure around pDAI. Its working assumption is coordinated intent to benefit pDAI and PulseChain. Under that assumption, governance changes, receipt issuance, stablecoin conversions and distributed arbitrage serve complementary purposes: mobilize capital, manage supply, build usable liquidity and finance continued support.

The strategy combines private-L2 funding or instructions with permitted local operations. Existing token addresses preserve connections to existing markets; credit can separate initial financing from later settlement; gradual purchases build positions; target-price sales return pDAI to circulation and recover payment capital; outside activity can generate income for replenishment. A funded two-sided pDAI dollar policy is the service this network aims to deliver. Its corrective rule is to buy below the target, meet demand with target-price offers, and restore the inputs consumed; an independent executable dollar reference determines payment amounts.

Its comparison rule is **one normalized eligible pStable = one accounting unit, U**. Receipts and LP shares first convert to their underlying claim values. Common U directs relative arbitrage; a \$1 policy additionally selects payment quantities tied to an independently executable dollar value.

Capital and execution can span apparently unrelated wallets and productive positions, with spendable inputs arriving when needed. A central treasury, visible accumulator or persistent local stockpile is not assumed.

Evidence category	Meaning in this paper
Observed	Specific local governance changes, issuance/conversions and two profitable closed trades at saved checkpoints.
Interpretation	Those capabilities form a coordinated reconstruction and pDAI-support strategy. Beneficial intent and its complete private-L2 connection remain assumptions.

Evidence category	Meaning in this paper
Operating requirement	Each selected route needs effective permissions, accepted exact tokens, timely funded payments and reconciled replenishment. The proposed dollar benchmark is a research test.

1. Why pDAI could organize the restored network

pDAI is useful in this construction because several paths already meet at its exact local identity. It is a 3pool base coin, FEI's conversion asset, the input to a registered Synthetix DAI wrapper, and the output of remaining Maker savings-credit conversion. Aave receipts and native-token markets add further connections. The [route evidence \(Appendix 35\)](#) records these distinct paths; ordinary Maker PSM conversion is shut down.

The proposed strategic choice is to make pDAI a commonly used settlement and pricing asset across those paths. Acquiring it provides potential sale inventory and inputs for compatible conversions. pUSDC and pUSDT remain important companion assets for base-pool liquidity and routing. The evidence establishes connections; choosing pDAI as the organizing asset is the thesis, rather than a consequence of pool membership alone.

Component	Proposed job
Private L2	Source positions, authenticated delivery or coordinated instructions.
Aave and Portal	Lending claims, debt restructuring and destination credit before backing.
Stablecoin issuers	Convert accepted collateral into existing market-facing tokens.
Curve and Balancer	Exchange assets and distribute liquidity through connected markets.
Authorized supply controllers and rebasing mechanisms	Adjust eligible supply under their specific rules.
Native-asset positions	Provide usable proceeds or attributable income.
Canonical PulseChain-Ethereum bridge	External capital entry and exit.
Distributed execution wallets	Purchases, conversions, support and replenishment.
Dollar reference and funded bids/offers	Set executable dollar-linked terms for the supported orders.

These are complementary roles and alternative branches. A useful subset can operate the strategy. The [network map \(Appendix 37\)](#) and [issuer audit \(Appendix 38\)](#) retain the token and authority details; common control of every issuer is not a premise established by this research.

2. How capital reaches the local markets

Every selected route must perform five jobs: **source funding, destination acceptance, funded conversion, price-sensitive trading, and replenishment.**

Private L2 asset or receipt OR permitted local financing
 ↓ accepted delivery or local issuance
 Existing local pool coin OR accepted aToken/wrapper
 ↓ funded issuer / Curve / Balancer / other market
 pDAI, neighboring stablecoins or PLS across participating wallets
 ↓ gradual purchases and liquidity allocation
 Usable inputs for bids, offers and connected-market conversions
 ↓ received capital / opposite flows / net earnings
 Restored capacity and settled obligations

Authorized L2 instructions can coordinate local execution separately.

Destination route	Required connection
Direct delivery	The bridge returns the exact token already accepted by a local issuer or market; escrow, minter or relayer funding permits delivery.
Receipt adaptation	A representation enters a compatible local issuer, which values it and creates its existing pool coin.

Source and destination addresses need not match. The final accepted local token matters. One proposed receipt route is **L2 vault receipt → authenticated representation → Reserve collateral plugin and active basket → existing Reserve eUSD → eUSD/FRAXBP → funded pUSDC → pDAI or PLS conversion**. Plugin registration, basket selection, refresh, pricing and issuance must work. Existing basket components cannot simply be replaced by supplying one new token.

Origin OUSD and mUSD connect through 3CRV and have different collateral rules. Existing accepted inputs may avoid changes; a new receipt needs compatible onboarding or an upgrade. Direct delivery of existing pDAI can supply offers or eligible reserves without an issuer detour. FEI and Synthetix conversions are optional branches when they improve net output or distribution. FEI receives a worked example because its local one-for-one conversion is clearly demonstrated; it links FEI to pDAI units, rather than independently anchoring either to dollars. The saved Synthetix evidence also connects native PLS purchases of pDAI, wrapper issuance of sUSD and funded market outputs; these mixed-position flows are not one isolated profitable cycle or fresh L2 delivery.

Each arrow needs its asset, effective executor, payer, obligations and timing. Attestations authenticate state or delivery; issuance creates claims; a funded market supplies the paid asset. A successful local cycle validates local circulation. Fresh L2 financing additionally needs its accepted delivery and source obligations; instructions can coordinate spending without supplying assets. Copied Arbitrum authority does not confer Optimism/Synthetix permissions. The [control audit \(Appendix 52\)](#) preserves those distinctions.

3. Aave and Portal can advance local claims

Aave is the lending and receipt layer; Portal is its V3 destination-credit branch. Ordinary supply deposits underlying and creates its aToken. Borrowing draws available reserve cash and creates debt. Portal lets an authorized Bridge caller issue the recognized destination aToken before matching underlying arrives. Source authentication belongs to the adapter; Aave checks local permission and reserve validation. This is the credit concept in [Aave's Portal proposal](#).

Authenticated adapter with local Bridge permission

↓

Aave V3 mintUnbacked → existing destination aToken

└ compatible wrapper → funded market → pStables → pDAI purchases

└ borrower-owned matching receipt → repayWithATokens → debt falls

Later: matching underlying + supplied fee → backUnbacked → principal falls

This branch separates early financing from later backing. Selling compatible receipts obtains market assets; matching-receipt repayment can remove debt-based withdrawal constraints when account execution, collateral ownership and available cash permit. Each of 86ff's debts requires its own asset, including LINK and CRV. Burned repayment receipts cannot also fund trading, and debt reduction leaves Portal principal outstanding.

The liquidity index changes receipt accounting rather than multiplying new principal: issuing 100 current aDAI does not produce 1,000 current aDAI at an index near ten. V3 Portal receipts are distinct from V2 receipts. The [wrapper census \(Appendix 24\)](#) identifies V3 DAI/USDC/USDT/WETH wrappers and 17 direct Aave linear pools across V2 and V3.

At the saved checkpoints, all 17 unbacked caps were zero. Activation needs Bridge permission, positive allowance, ordinary supply headroom, compatible wrappers and funded output markets. These are specific implementation milestones; raising one cap is insufficient. The [Portal audit \(Appendix 48\)](#) retains the gates and authority requirements. The timing and settlement policy appear below.

4. New collateral can reach existing Curve coins

The proposed change is usually behind a coin Curve already accepts. An authorized issuer accepts authenticated collateral and issues its existing pool token. This can change a pool's economic exposure while preserving its coin slots.

Change	Required authority and effect
Change an issuer's accepted collateral	Effective issuer configuration or upgrade authority permits acceptance, valuation and issuance. Its existing OUSD, eUSD, mUSD or other token enters Curve.
Upgrade an existing pool token	Its effective proxy administrator installs compatible logic, preserving storage and working balance/transfer behavior while implementing issuance and collateral rules.
Replace Curve coin slots or trading logic	Requires that exact pool's change mechanism. The two inspected sETH2/stETH pools are fixed implementation clones with no coin-replacement setter. Token governance or a factory deployment setting cannot replace those slots; a new pool requires separate liquidity.

L2 receipt → authenticated representation → authorized local issuer

↓

existing token at existing address

↓

unchanged Curve coin slot

StakeWise demonstrates local implementation change and historical pool acceptance: custom sETH2 minting was followed by a deposit of about 210.41 million sETH2 into an existing Curve pair. The source/L2 connection remains open. A later incompatible implementation broke token reads and pool quotes at the saved checkpoint; compatible restoration is an identifiable next milestone.

Several assets could back one issuer coin; Curve still sees one coin. Unrelated bridged tokens sent directly to the pool do not fill its slot. The inspected sETH2/stETH implementation has no external Arbitrum oracle input. Issuer valuation must separately handle collateral and claim units. The [matched implementation study \(Appendix 63\)](#) retains the evidence and limits.

5. Common accounting directs connected market arbitrage

The policy admits specific tokens, normalizations, conversion rights, limits and suspension conditions. Common U compares opportunities; actual fees, basket rules, slippage and available payment assets determine execution. Funded conversions transmit support through the eligible connected markets in each usable direction. Fees, capacity and delays determine the achievable relative-price bands; one working mint does not establish the reverse route or every AMM quote.

Buy a cheap eligible coin, convert through an accepting issuer and obtain a coin scarce elsewhere. Or buy a cheap issuer coin and redeem its funded underlying. Authorized issuance or removal of supply can extend those operations. Under the restoration interpretation, the objective is to improve connected liquidity while acquiring useful assets; each individual trade need not profit.

Curve connects OUSD, mUSD and other metacoins to pDAI/pUSDC/pUSDT through 3CRV; Reserve eUSD uses FRAXBP. Balancer and lending venues provide further eligible paths. Shared base reserves count once: multiple issuer routes using the same 3pool or FRAXBP cash are not independently funded alternatives. Rebalancing one venue must account for the payment resources and obligations left in the others. Selling newly issued tokens into a pool consumes counterassets; reverse trades, funded redemption or contraction, or new inputs must restore the needed capacity.

FEI has a demonstrated one-for-one pDAI conversion. Historical mUSD and post-shutdown sDAI cycles closed with **200,053.199510 PLS** and **110.687385 PLS** after gas. These usable proceeds could finance later operations; their subsequent deployment is a separate connection to establish. Profit and direct price support are distinct jobs: the actual sDAI cycle ended by selling pDAI and slightly lowering its closing pool's PLS/pDAI ratio. Under coordinated intent, its proceeds could support a different venue. The policy must assess complete closing flows. The [closed-trade reconciliation \(Appendix 25\)](#) records the actual endpoints.

Supply buffers differ by implementation. The checked PWRD/STBT pools use live balances; OUSD, UZD and Lybra have different treatment. Rebases, minting, borrowing and income require separate resource accounting. The [rebase audit \(Appendix 36\)](#) preserves these distinctions.

6. Build support gradually and keep pDAI circulating

The proposed construction has five phases. They can overlap; selected routes become usable before the whole network is commissioned.

Phase	Proposed action and observable milestone
Activate routes	Establish effective permissions, compatible token behavior, accepted collateral and funded conversions.
Accumulate gradually	Repeated purchases build useful pDAI and neighboring-stable positions across wallets and protocols.
Build execution depth	Fund appropriate pool sides and conversion markets; measure paid output at the supported sizes.
Operate the price policy	Deliver timely purchases and sales at quantities tied to the chosen dollar reference; recover payment resources and route pDAI into measured reserve shortages.
Extend economic use	Enable usable entry, spending, borrowing and exits; reinvest attributable net income into continued support.

The acquisition assumption is a consistent allocation every ten seconds while pDAI is below target and the route is funded. The 50bn allocation enters over time. At 8,640 daily slots, 10,000 input-token units per slot takes about **579 days** to deploy 50bn; 1,000 takes about **16 years**, before pauses. Input units are not verified dollars. Splitting orders reduces individual impact; total impact still depends on replenishment and market depth.

Acquiring a finite seller cohort can reduce its remaining inventory. Liquidity allocation and lending can also return pDAI to circulation through purchases, withdrawals and borrow-and-sell activity. The strategy therefore needs circulation rules, rather than treating acquired supply as permanently unavailable.

Operating decision	Required specification
Reference and payment	Independently executable dollar valuation and exact funded payment asset.
Execution	Supported sizes, all-in bid/offer band and settlement deadline.
Capacity	Cumulative flows, source outages and reference/liquidity shocks covered.
Replenishment	Incoming assets, opposite trades, withdrawals or net earnings, with delivery deadlines and obligations counted once.

If eligible holders can reach funded bids and offers at the declared net terms, sizes and deadlines, those completed orders receive the declared band. This is the conditional deterministic result. It concerns the accessible funded service, including routes through LPs, rather than every finite pool's standalone quote. Receipt surplus or dominant ownership alone does not supply its premises.

The appendix proposes 1,000-pDAI orders within an all-in $\pm 1\%$ band over 24 hours, with declared flow limits and an Ethereum-origin USDC payment candidate. This is a research benchmark, not an observed operator promise. Copied \$1 oracle marks do not establish the payment's dollar value. PLS, ETH or BTC can provide variable-price payment assets if quantities track an independent executable reference.

Target-price sales are the replenishment phase of the acquisition strategy. Buyers receive pDAI while the network receives payment assets that can fund subsequent support. The policy also routes available pDAI into measured shortages across connected pools, lending reserves and compatible issuers. Releasing supply at the target can serve demand and contain a premium while restoring useful circulation.

A funded offer at the target also creates a distribution incentive: where another venue pays a premium on the same executable valuation basis, traders can buy pDAI at the target and sell it into that venue, narrowing the premium if the spread covers costs. This lets outside arbitrage participate in the refill.

```

Acquired or validly issued pDAI available through permitted positions
├─ target-price sale to users → payment capital recovers
├─ routed sale into 3pool → DAI reserve rises; output asset falls
├─ 3pool liquidity deposit → DAI reserve rises; 3CRV is issued
└─ compatible lending/issuer supply → its pDAI cash rises

Recovered payment assets + new capital + collectible net earnings
      ↓
    further support and replenishment

```

Here, 3pool holds pDAI/pUSDC/pUSDT; **3CRV is its LP token**. A routed pDAI sale into 3pool restores its DAI side by withdrawing another base asset. A pDAI liquidity deposit adds DAI and obtains 3CRV under the pool's rules; those claims can connect to accepting metapools. These exchange/deposit directions follow [Curve's 3pool logic](#); the local token identities retain their saved checkpoints. The network must budget the other base assets and shared reserves alongside the pDAI refill.

A direct sale to a user restores the seller's payment resources. Its further allocation determines which pool or lending reserve is refilled. Conversely, a user buying pDAI from 3pool removes DAI while adding the input stablecoin. The proposed network closes that loop by supplying pDAI where routed demand removes it and directing recovered resources to the next support requirement. The aim is repeated redistribution across the ecosystem, with each shortage and funded refill recorded.

The sampled Aave DAI reserve had zero pDAI cash. A valid ordinary supply can add cash while creating its aToken claim, making that refill a measurable destination objective. It does not clear separate Portal principal. Gross sale proceeds recycle capital; only realized net margins or fees count as income. Proceeds allocated to backing, earning positions or other venues also compete with payment capacity for recirculated pDAI. One shared resource-and-deadline budget must cover the chosen commitments across wallets. Renewed selling still requires timely inputs, and lending can return pDAI through withdrawals or borrow-and-sell activity. More aTokens than pTokens provides support through permitted funded conversions. V2 execution changes with size; a price touch does not specify a lasting service.

Universal fixed Ethereum redemption is not required for the bounded local policy. External access still needs valid representations and funded paths. Proposed Ethereum pDAI/ETH or pDAI/WBTC LPs have variable exchange rates and require separate deployment, reserves and bridge evidence.

7. Patient financing needs settlement and collectible income

Clock	Saved condition
Administration	Aave newly queued actions: at least one day; Origin: two days; Reserve voting plus three-day timelock; mStable reported upgrade delay: seven days. Effective controllers matter.
Source confirmation	pARB: 45,818 PulseChain blocks, about 5.30 days at ten-second blocks, subject to progress/challenges. Copied Optimism: seven days.

Clock	Saved condition
Across reimbursement	Bundle liveness: 7,200 seconds; total reimbursement also depends on routes, rights and funding.
Purchases	Ten-second cadence once inputs arrive; each trade need not await a fresh bridge round trip.
Portal backing	No universal per-mint maturity in inspected code; outstanding principal and exact-underlying settlement remain.

These separate clocks explain the proposed long preparation and accumulation period. A funded relayer can advance destination assets before reimbursement. Portal exposure can persist for years in state, consuming capacity while withdrawals need cash. Integration terms can impose deadlines and fees. Eventual [backUnbacked](#) requires an authorized Bridge caller, matching underlying, allowance and supplied fee. Ordinary supply or borrower debt repayment does not clear this principal. The [deferred-financing study \(Appendix 34\)](#) retains all twelve examined debts and the settlement rules.

The favorable feedback is:

```

Gradual acquisitions + funded liquidity
      ↓
More reliable exchange and useful credit
      ↓ possible adoption
Outside payments, trading and borrowing
      ↓ collectible net income
Reinvestment + incoming capital + target-price sales
      ↓
Timely support inputs and eventual backing
      ↗ restored capacity

```

The proposed reinvestment policy directs all attributable net earnings to support after costs, losses and required obligations. Outside trading fees, paid borrower interest and realized rewards can contribute when the network owns the entitlement. Fees between controlled accounts cancel; principal and gross turnover are not income. LP shares sent to an inaccessible burn address provide no usable fee entitlement; ordinary liquidity withdrawal burns shares while returning assets under the pool's rules. Pledged collateral cannot also be free payment capacity, and nested receipts count the same underlying once.

PLS/HEX/PLSX/INC positions need actual entitlement and realizable proceeds. Direct native collateral in Origin or Reserve requires compatible strategies/plugins and valuation; proceeds can instead buy existing accepted collateral. Funding can remain in productive positions and arrive just in time for execution.

The [accumulation study \(Appendix 65\)](#) explains the feedback. The [liquidity-growth study \(Appendix 40\)](#) models depth and reinvestment under explicit hypothetical inputs. Neither establishes actual future demand or income. As parity improves, arbitrage spreads can shrink; perpetual arbitrage profit is not required. Recycled capital, opposite flows, new funded inputs and collectible net income can maintain the service within its budget. Retiring 86ff's debt also removes its future interest contribution; the plan cannot count future interest from that retired debt. Saved gates—zero Portal caps, broken sETH2 logic, disabled ordinary Maker PSM and finite issuer cash—identify the changes or alternative routes needed before their capacity contributes.

8. Define the benefit to PulseChain participants

The proposed public benefit is reliable exchange and productive use across the network. Acquisition and permissions serve that outcome. Its success measures differ by participant:

Participant	Proposed benefit and measurement
pDAI holders	Reliable execution: paid output, supported size, costs and waiting time.
Borrowers	Usable credit: available reserve cash, debt terms, repayment and withdrawal rights.
Liquidity providers	Attributable outside fees alongside inventory risk, losses and withdrawal access.
Stablecoin users and issuers	Working conversions, pDAI replenishment into measured shortages, balanced resources and reconciled redemption claims.
Native-asset participants	Additional productive use, realized rewards or funded demand; native appreciation is not assumed.
New users	Usable entry, settlement and exit paths at actual costs and sizes.

Moving resources between venues can finance a broader reconstruction. Its benefit must appear in the resulting depth, claims, coverage and access of affected participants; changing one group's position does not establish a gain for every holder.

The canonical **PulseChain-Ethereum bridge** is the external-capital boundary for this interpretation. Copied Across and private-L2 gateways are separate objects. A proposed supporting observation is rising PLS per pDAI and persistent canonical bridge assets over three years while PLS declines; a matched time series remains to be reconstructed.

Stable asset quantities despite completed releases require offsetting inputs or reconciled changes. Stable dollar TVL alone can reflect revaluation. Replacement inflows may restore backing while creating replacement claims. The useful pattern to measure is capital arrival followed by distributed net acquisition, usable liquidity, economic activity and replenishment. PLS privacy may obscure lineage, but active deployment and relevant use remain unverified. The [intent assessment \(Appendix 28\)](#) retains independent inflow, recycling and other explanations; persistent vulnerable code alone does not imply inevitable zero value.

9. Connect each interpretation to its next milestone

The working purpose organizes the research; each selected route still needs its own evidence.

Observation or open field	Proposed role	Next milestone
Local authority and token changes	Prepare compatible interfaces	Effective execution rights and restored behavior for the selected route.
Funded local conversions and closed trades	Mobilize resources	Follow exact proceeds into acquisitions or support, then verify post-use refill.

Observation or open field	Proposed role	Next milestone
Private-L2 funding branch	Deliver assets or instructions	Fresh authenticated output, accepted destination token, executor and payer.
Distributed purchases and bridge persistence	Build positions and replenish capital	Reconcile net acquisitions, deposits/releases, claims and valuation changes.
Dollar policy and useful circulation	Provide the public service	Repeated timely execution on both sides, recovered payment assets, pDAI refills at depleted venues and usable entry/exit.

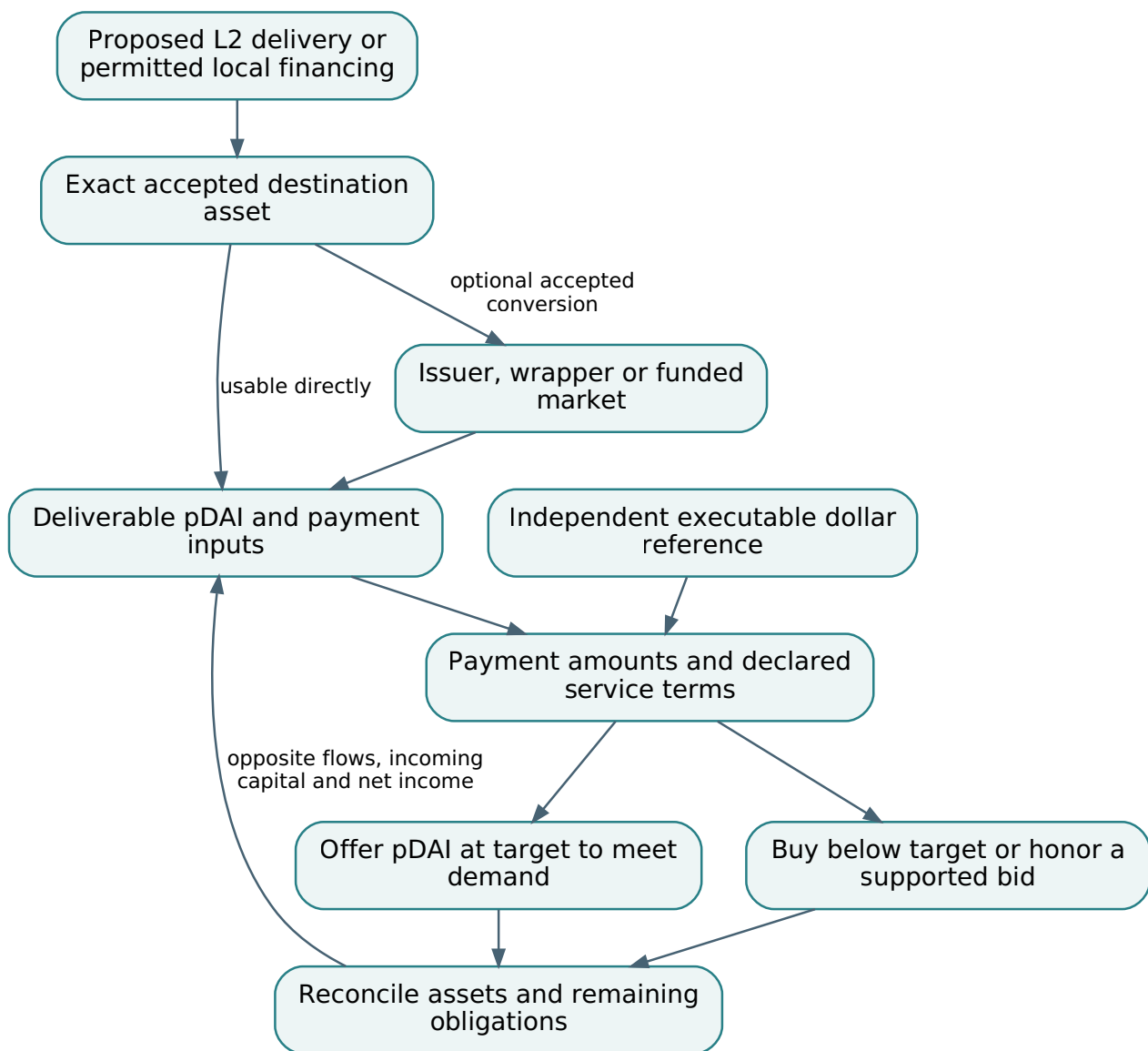
A passing route establishes its tested capacity. Benchmark failures exclude that specified service at the tested conditions; one route failure does not eliminate every alternative. Under restorative intent, useful progress would increasingly connect permissions to funded activity, activity to retained resources, and resources to reliable public exchange. The exact contracts, authority findings, proposed branches and reproduction references follow.

Appendix Contracts and local route evidence

The inherited Curve catalogue is dated 10 May 2023. Bridge and selected authority comparisons include block **27,681,116**; the StakeWise/Lido/Index Coop studies use block **27,690,565**. The funding/rebase resource pin is **27,691,754**, with its additional destination-event query ending at **27,692,266**. The equal-value route resource pin is **27,698,092**. Debt and other studies retain their own earlier pins. These observations establish specific historical facts and must not be combined as a single current balance sheet.

The added arbitrage and Maker adapter checkpoint is **27,699,029**, hash `0x0d0c82f00bdd0048866b1a00f99e362a9c2a853d2d70d21cd11d76640e925ff3`, at **3 October 2026, 01:29:45 UTC**. Its trade reconciliation covers a new savings transaction and the retained earlier mUSD transaction. A focused Maker adapter recheck at **27,707,510**, hash `0x8a99d5ac2c73089e006d50765bbfaa5d92e6fb523fb6d5384a2c100794cfb63b`, at **4 October 2026, 01:39:15 UTC**, updates selected live flags, identities, custody and End allocation; it does not refresh the other balances.

Here, L1 means PulseChain and L2 means the proposed pARB system. These labels do not imply automatic redemption on Ethereum. Source assets and destination pDAI/pUSDC/pUSDT are different chain-specific tokens. Source receipts, bridge outputs and local issuer outputs also need separate identities.



Proposed route: every arrow needs its asset, permissions and usable output. An operator submits the transactions.

For the proposed private-L2 route, first check whether its bridge returns the existing PulseChain pool coin. This gateway discussion is separate from the canonical PulseChain-Ethereum capital boundary. DAI/USDC/USDT here denote copied local tokens, not an automatic Ethereum-backed claim.

DAI, USDC and USDT have configured counterparts with different L2 addresses that return the existing local token. Three sampled pairs—PL2, HGOLD and MCB—have identical numeric addresses on both chains. Identical addresses do not merge their contracts or supplies.

Standard and custom gateways ordinarily release escrow. At the new checkpoint, the selected DAI, USDT, OUSD, eUSD, mUSD and FRAX gateway balances were zero; USDC had 1,000,000 raw units. Mapping and a calculated L2 address do not prove source deployment or withdrawal capacity. ARB and GRT have separate token-specific mint branches. See the [exact bridge table \(Appendix 37\)](#).

If the output is a new representation, the system could offer it as collateral to a local issuer. For example, **if Origin accepts and values a bridged OUSD representation**, it could mint the existing local OUSD token. Curve receives its existing coin. This adapter route becomes useful when direct return is unavailable or insufficient.

L2 OUSD/eUSD/mUSD issuers are proposed source deployments, not established inherited Arbitrum issuers. Historical source alternatives include Aave receipts and Curve or GMX positions; their present pARB state and bridge mappings are also unverified.

After a local stablecoin sale, the system holds purchased pDAI and the pool or buyer holds the issued coin. Any collateral, redemption and bridge obligations remain. Count backing once and exclude pledged collateral from free support inventory. Creating nested claims does not create independent resources.

Pairing the bridged asset in an LP does not automatically make it acceptable issuer collateral. An LP share is a claim on a trading pool's assets. If the issuer accepts one, it must account for its underlying assets, changing exchange rate, withdrawal rights and liquidity. A lending or staking receipt needs the same treatment.

Four checks govern the route: usable source assets, correct bridge output, functioning issuer acceptance, and pool assets available to pay. Matching symbols are insufficient.

Direct delivery and optional conversion routes

This register separates direct deployment from optional conversions. Its FEI branch is a worked example with unusually clear local evidence; it is not required for the strategy. Synthetix, compatible baskets, native-funded purchases and other functioning routes can perform complementary jobs. This construction is separate from the longer new-receipt/Reserve branch and does not assert installed private-L2 delivery.

Arrow	Asset, authority, payer and status
Source → local pDAI	Private source deployment, asset rights, mapping, effective messenger/executor, delivery payer and latency remain unknown. Required PulseChain output is existing pDAI 0xb175474e89094c44da98b954eedeac495271d0f.
Delivered pDAI → direct use	Available exact pDAI can serve offers, funded market trades or eligible pool/lending deposits without FEI. Dollar-valued payment assets for buy-side commitments need separate coverage.
pDAI → sUSD, optional	The existing Synthetix DAI wrapper accepts pDAI and issues sUSD under its fees, cap and validation; historical receipts demonstrate this local branch. Its deposits back synth claims, and its markets need funded output. Details appear in the Synthetix section below.
pDAI → FEI, optional	Public holder deposit into 0x7842186cdd11270c4af8c0a99a5e0589c7f249ce mints existing FEI 0x956f47f50a910163d8bf957cf5846d573e7f87ca, subject to token permission and pause state. Deposited pDAI funds the module; FEI holders' claims remain. This local conversion is demonstrated.
FEI → market output	Existing FEI/3CRV 0x06cb22615ba53e60d67bf6c341a0fd5e718e1655 pays its available counterassets through a selected funded route. Exact output, size, costs and resulting reserves must be recorded; a view quote is not combined execution proof.

Arrow	Asset, authority, payer and status
Proceeds → acquisition/support	Permitted distributed executors must obtain exact pDAI or payment inputs by each deadline. Conversion into the benchmark Ethereum-origin USDC, or independent funding of it, remains open; copied pUSDC does not automatically fill that role.
Used capacity → restored capacity	Name the consumed token, net incoming payer/source, feasible conversion, remaining claims and due time. FEI redemptions consume module pDAI; new pDAI deposits restore its cash while creating FEI claims. Market output capacity needs its own replenishment.

The [equal-value route study \(Appendix 35\)](#) supplies the local module/pool evidence. A complete source-and-service deployment cannot be claimed while the open rows remain unverified.

A proposed dollar support benchmark

This benchmark makes the missing policy testable. Its choices belong to this research; they are not attributed to the wallet group, deployed as contracts or certified as available capacity.

Item	Proposed test term
Payment candidate	Ethereum-origin USDC represented on PulseChain; candidate token 0x15d38573d2feeb82e7ad5187ab8c1d52810b1f07. Mapping, custody/claim coverage and actual Ethereum release must be verified. Copied pUSDC 0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48 is a separate token on PulseChain.
Dollar reference	Independently executable USDC/USD value at the relevant payment size, with payment quantity adjusted as it changes. If onward conversion is unverified, report reference-valued execution rather than established dollar purchasing power.
Order and net band	Exactly 1,000 pDAI per tested buy or sell; net holder cost/proceeds within 1% of \$1 per pDAI, including gas, fees and any committed subsidy.
Horizon and flow	24 hours; 100 eligible 1,000-pDAI requests per side, with orders serialized and shared funding commitments counted once. Predeclare the schedule: 50 sells, 50 buys, 50 sells, 50 buys, every six minutes. Record attempted, accepted, refused and expired volume. Refusing an eligible test request does not remove it from the completion denominator.
Execution	Record quote block b and binding acceptance event. The quote expires at the end of b+2; an accepted order must settle by then at the committed terms. Expiry before acceptance creates no payment promise but remains a failed eligible service request. Non-settlement after acceptance is a failure. Measure the net outcome using the independent executable reference at settlement; transaction inclusion is a measured condition, not guaranteed by this paper.

Item	Proposed test term
Replenishment	Within one hour of each execution, reconcile consumed payment inputs and restore deliverable capacity for another 1,000-pDAI order on both sides, plus all already accepted obligations. Maintain an exact-resource/deadline ledger for the remaining predeclared requests; even after the final request, measure next-order capacity so refill cannot become vacuous. Accepted orders retain their shorter execution deadlines. Opposite-flow receipts count only after arrival; do not assume them when testing one-sided sequences.
Reference stress	Include an observed or separately labelled offline 1% decline in the payment reference with adjusted amounts. Larger moves, source outages and greater flow remain outside this test's stated capacity, and cannot be advertised as covered.
Shortfall	Stop issuing new promises when reference, delivery or capacity requirements fail; still record failures of previously accepted obligations. A later restored quote cannot repair completed holder proceeds.

A focused getter checkpoint at PulseChain block **27,699,585**, hash `0xd8f0bc5ca2e16f0660c45d5595a6c45712befe5a6de8f8a5830cd9aacc433995`, confirms the candidate token's name "USD Coin from Ethereum," six decimals, and owner/bridge contract `0x4fd0aaa7506f3d9cb8274bdb946ec42a1b8751ef`. The mediator maps it to foreign token `0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48` and counterpart `0x1715a3e4a142d8b698131108995174f37aeba10d`. These are configured identities, not audited Ethereum custody or a demonstrated release. The same numeric A0b address on PulseChain is copied pUSDC; chain identity is essential. Raw getters and verified-source ABI references are retained in the completeness review batch.

Ethereum settlement has its own measured latency and coverage test; it is not promised within the local quote interval. The benchmark tests one finite support service, not the full acquisition campaign or every future run. A selected source route still needs its exact asset, accepted output, executor, payer and obligation record. Until those and actual capacity pass, the executable deployment remains incomplete.

Historical pool coverage and supply management

The inherited census contains **618 unique pools** from eight responsive registry roots: 176 have exact 3CRV as a direct coin and 64 have exact FRAXBP. The later destination census has 619 pools. The examined Reserve eUSD pool belongs to FRAXBP. Other bases, lending pools, crypto pools and custom venues require their own paths. The [historical catalogue \(Appendix 37\)](#) retains all 381 returned token identities, failed underlying reads and experimental entries; registry membership does not establish usable inventory.

FRAX AMOs and DOLA Feds are particularly relevant: their designs deploy authorized supply into venues and can remove supply. Yield-rebasing candidates include OUSD, UZD, PWRD, Lybra's separate eUSD, STBT and ARTH.usd. Their exact pool treatment matters. The sampled OUSD/3CRV account is **nonrebasing**, and the sampled `0xb40b...9f1c` UZD pool is **rigid**, excluding elastic balances. Reserve eUSD and mUSD use issuance/redemption; they are not automatic pool-balance rebases. The [supply-buffer analysis \(Appendix 37\)](#) separates these mechanisms.

The later [exact implementation and funding audit \(Appendix 36\)](#) verifies live-balance accounting in five PWRD/STBT pools and stored-balance accounting in both Lybra eUSD CryptoSwap pools. It also traces sUSD into received pUSDT and native PLS. Token addresses, transactions, current resource limits and offline reproduction are preserved there.

Confirmed governance changes and attribution limits

The confirmed postfork changes are Across HubPool ownership and its Safe module ownership, now held by `0xfeca7291cebe75789ffa707754db259187b1d140` and `0x98c89980d1eae247ada3636ae27648f78e66e121`. HubPool's transfer executed after an undisputed UMA assertion and a **five-day challenge interval**. Its recorded rules excluded HubPool calls; successful on-chain settlement does not prove a valid Snapshot vote. Later owner-only configuration registers chain 88888. The October 3 trace audit shows its July relay moving local Hub assets to a wallet through a custom adapter; it does not establish the private network's identity or fresh L2 delivery. A June root execution separately transferred about 1.347 million pDAI, 10.700 million pUSDC and 173,269 pUSDT into the local Spoke, followed by successful refunds and market sales. At the saved checkpoint, HubPool is paused. The [governance audit \(Appendix 46\)](#) and [equal-value route study \(Appendix 35\)](#) distinguish authority from actual funding provenance.

The [expanded audit \(Appendix 52\)](#) also verifies a Reality-module execution that reduced a copied GnosisDAO Safe's threshold to one and a nested execution that did the same to its treasury Safe. Copied Arbitrum timelock executions transferred key proxy administration to externally owned addresses; its minimum delay is now one second. This materially strengthens the evidence for local governance changes. It does not establish common control of every address, Synthetix's protocolDAO or Optimism's proposer. The copied Optimism BondManager still admits only its registered proposer.

Origin, Reserve, Silo, mStable and every stablecoin issuer have **not** been proved to share one controller. In the comparison, 70 generic authority getter values match the inherited state, while 446 are unresolved. Acquired voting control could leave those addresses unchanged. The [family table \(Appendix 37\)](#) records 74 candidate groups, their tokens and authority paths, including unresolved identities.

Financing families and native-asset conditions

Route	What it can contribute	Condition that matters
L2 issuer or yield position → bridge → local issuer	Existing local stablecoins for market operations	Source, bridge, collateral acceptance and valuation work end to end
Existing accepted local collateral → issuer	Local supply without the L2 detour	Owned eligible collateral and functioning issuance
Authorized local minting or credit	Additional local coins	Effective authority, limits and resulting obligations
Portal deposit claims (aTokens) → wrapper → market	Market assets obtained by selling claims	Roles, caps, wrapper acceptance and funded buyers or pool output; unbacked liability remains
Ordinary lending or collateralized borrowing	Available market assets or released owned collateral	Reserve cash, valid collateral and debt service
Escrow release or supported bridge liquidity	Relocates assets already available elsewhere	Exact mapping, escrow or relayer inventory and settlement
Token-specific bridge minting	Supply under a special token's gateway rules	That token's authority; not a generic pStable minter

Route	What it can contribute	Condition that matters
Owned treasury, outside capital or market sales	Spendable local or external assets	Ownership, depth and actual proceeds
Outside interest, LP fees, protocol revenue and realized rewards	Income for replenishment or settlement	Collectible net income after costs, losses and obligations

Cheap acquisition, routing arbitrage and reinvestment can improve these branches. A receipt must be redeemable or saleable before it becomes support inventory. Internal fees cancel when controlled accounts are consolidated.

Owned PLS, HEX, PLSX and INC positions require actual staking or fee entitlement. HEX maturities, native staking exits, reward-token sales and LP losses affect timing and value. The sampled Origin vault does not accept those four native tokens, and Reserve eUSD's basket uses four lending receipts. Direct native collateral needs a compatible strategy/plugin, valuation and authority; another Reserve basket is a separate proposal. The [native-yield study \(Appendix 37\)](#) records the remaining checks.

Aave V3 Portal contracts and changes required

The examined PulseChain V3 Pool is `0x87870bca3f3fd6335c3f4ce8392d69350b4fa4e2`; its ACL Manager is `0xc2aacf6553d20d1e9d78e365aaba8032af9c85b0`. Exact V3 aDAI is `0x018008bfb33d285247a21d44e50697654f754e63`, and its static wrapper is `0x098256c06ab24f5655c5506a6488781bd711c14b`. These are destination contracts, not unspecified source-chain receipts.

Required step	Saved authority or condition
Admit a Portal caller	The ACL role administrator must grant Bridge permission to the actual adapter. Saved default ACL administrator: short executor <code>0xee56e2b3d491590b5b31738cc34d5232f378a8d5</code> . Aave does not independently establish that adapter's private-L2 authentication.
Enable reserve issuance	PoolAdmin/RiskAdmin through the configurator must provide positive unbacked allowance and adequate ordinary supply headroom; reserve status and other validation must pass. All 17 sampled unbacked caps were zero.
Execute administrative changes	Saved executor administrator: <code>0xec568ffffba86c094cf06b22134b23074dfe2252c</code> ; nominated pending administrator: <code>0xa30190b96faee008014aa0b7645081fcbf49e6f</code> . Nomination is not acceptance or proven group control. Newly queued actions have at least the saved 86,400-second delay and a grace-period requirement.
Convert issued claims	The exact wrapper must accept the aToken and the output market must pay. The sampled DAI wrapper's supply-headroom calculation reverted, and its Aave reserve had zero pDAI cash. Neither a pure share conversion nor a raised unbacked cap establishes usable payment.
Retire debt or clear backing	86ff must execute its own supported matching-receipt repayment path. Portal backing separately requires Bridge permission, exact underlying, allowance and supplied fee. Aave roles do not grant mint authority over pDAI, pUSDC, pUSDT, LINK or CRV.

The [authority map \(Appendix 29\)](#) and [Portal pricing study \(Appendix 48\)](#) retain the source checks, pinned state and separate wrapper/market limitations. These are required change paths and dated findings, not newly executed changes or current-capacity claims.

Origin OUSD

The sampled vault accepts local DAI, USDC and USDT when collateral, valuation and pause checks pass. Mint valuation is capped at \$1 per normalized token; prices below \$0.998 are rejected.

A new bridged token needs compatible collateral configuration and valuation. A yield share worth more than one underlying unit may require a wrapper, strategy or accounting change to capture the full value. The sampled delegated admin and price provider have unverified source, leaving the complete change path open.

Reserve eUSD

Reserve's collateral plugins and configurable baskets fit the proposed receipt route well. Governance would need a compatible plugin, registration, basket configuration and activation of a sound basket.

The current prime basket contains four local lending receipts. Issuance requires all basket components in the quoted amounts. Adding one token does not replace the others. A single-asset basket or different allocation is a separate proposal. Saved cached health flags coexist with stale collateral feeds; they do not establish successful issuance after refresh.

mStable mUSD

The current basket accepts local sUSD, USDC, DAI and USDT, with weight limits. Corrected view calls quote small existing-basket inputs and reject OUSD/eUSD as invalid assets. A quote does not prove a complete mint succeeds.

Adding bridged collateral appears to require a compatible upgrade: the current implementation has no public basket-token addition function. Migrating a lending integration does not change the accepted token.

The later transaction audit demonstrates actual mints, redemptions and a closed trading cycle. Its September redemptions used integration caches. Remaining cash and the unresolved investment withdrawals are recorded in the [equal-value route study \(Appendix 35\)](#).

FEI and the Maker routes after shutdown

FEI's fixed module is `0x7842186cdd11270c4af8c0a99a5e0589c7f249ce`; it uses existing FEI `0x956f47f50a910163d8bf957cf5846d573e7f87ca` and pDAI. Public mints deposit pDAI at one-for-one accounting; redemptions receive FEI and pay pDAI from finite reserves. Sampled reserves are about 39.093 million pDAI. Existing FEI enters `0x06cb22615ba53e60d67bf6c341a0fd5e718e1655`, its FEI/3CRV pool. Governance can affect mint permission and FEI token operation; it does not set a price parameter inside this fixed module.

DOLA's operator is Inverse timelock `0x926df14a23be491164dcf93f4c468a50ef659d5b`. Its executed proposal installed custom administrator `0x1b65b3ce1e463c6020e684cd8968e4cd330ab053`; that administrator's ultimate controller remains unresolved. The observed DOLA sales obtained existing pUSDC before Maker minted pDAI. Their exact markets and input/output ledger are in the [funding study \(Appendix 35\)](#).

Maker's USDC PSM is 0x89b78cfa322f6c5de0abceecab66aee45393cc5a; Vat is 0x35d1b3f3d7966a1dfe207aa4514c12a259a0492b; DaiJoin is 0x9759a6ac90977b93b58547b4a71c78317f391a28. Shutdown on April 16, 2025 stopped ordinary PSM conversion because both directions use `Vat.frob`, which requires an active Vat. The later adapter pin nevertheless reports `DaiJoin.live = 1` and retained pDAI mint authority.

The USDC adapter ending 2199 is a different contract from DaiJoin. Exact `AuthGemJoin5` is 0x0a59649758aa4d66e25f08dd01271e891fe52199. The October 4 recheck reports **USDC adapter live = 1, DaiJoin live = 1, Vat live = 0**. Thus ordinary PSM conversion remains disabled at Vat, even though both adapters are live. The USDC adapter's authorized deposit checks its live flag; its public withdrawal does not. Even a caged collateral adapter can permit withdrawal against valid internal collateral credit, subject to retained permissions, cash and token-transfer checks. DaiJoin's withdrawal has its own live check. The [Maker adapter recheck \(Appendix 44\)](#) records the exact conditions.

Maker-related branch	Status and resource supplied
Ordinary pUSDC/PSM conversion	Disabled by Vat shutdown; not counted as active acquisition capacity.
Existing savings/internal-credit conversion	Post-shutdown execution demonstrated; consumes the holder's existing credit and can deliver pDAI.
End collateral settlement through the USDC adapter	Low-priority legacy possibility, retained for completeness; processing/payout unproven and the recorded prorated return unattractive under equal-unit parity.
Authority-dependent credit creation	Separate permission path with corresponding debt; usable group-controlled execution remains unproved.

DaiJoin holds 0.1443398 ERC20 pDAI, alongside 128.303 billion internal DAI units recorded against it in Vat. These are different balances. Its `exit` consumes the caller's internal DAI credit and mints pDAI; it does not give an arbitrary caller the large ledger balance. A successful October 3 transaction proves savings-credit conversion through exact sDAI 0x83f20f44975d03b1b09e64809b757c47f942beea, Pot 0x197e90f9fad81970ba7976f33cbd77088e5d7cf7, and DaiJoin. sDAI's sampled claim is about 3.207 million pDAI, backed within Pot's larger internal credit. Pot's savings rate is frozen at no growth. Count the nested claim once.

Maker End is 0x0e2e8f1d1326a4b9633d96222ce399c708b19c28; its USDC join is 0x0a59649758aa4d66e25f08dd01271e891fe52199. The October 4 recheck finds **113,692,361.975557 pUSDC** at the join and **zero internal PSM-USDC-A collateral allocated to End**. Source and getters support a proposed public **process PSM position → pack internal DAI → claim prorated collateral → exit USDC join** sequence. Its recorded position could allocate 113.185 million internal collateral units; one million pDAI would claim about **879.277935 pUSDC** after token rounding at the fixed rate. Processing and payout have not been demonstrated. The 1967 wallet's public rate-initialization call does not prove Maker control.

This is retained for completeness as a low-priority legacy settlement possibility. At the recorded rate, one pDAI yields about 0.00087928 pUSDC. Under the thesis's equal-unit assumption, the output is only **0.08793%** of the input's accounting value, before costs. It is therefore economically unattractive as a parity-support funding route. It could make sense if eligible pDAI were acquired below that pUSDC-equivalent payout after all costs, or for a holder collecting an already packed settlement claim. Neither case establishes an operating peg-financing loop.

Authorized `Vat.suck` has no shutdown check and creates internal credit with corresponding debt. This is a separate possible issuance branch, requiring a usable path through an existing authorized contract. A contract's retained authority does not give every caller arbitrary issuance. The [Maker shutdown and](#)

arbitrage study (Appendix 25) retains exact amounts, permissions, matched runtimes, 709 selected exit logs, the verified latest trace and settlement preconditions. All execution analysis is historical or source arithmetic.

Other issuers and shorter routes

DOLA has authorized minting, XAI has owner-controlled minting, aUSD has authorized minters and ceilings, and FRAX has authorized minting pools. These provide candidate routes when the required authority is actually controlled. Their controller and backing conditions differ.

Liquity V1's LUSD is a counterexample to universal governance takeover: its issuance path is immutable and governance-free. LQTY ownership does not allow bridged stable collateral to be added. Valid native-collateral borrowing or market acquisition may still obtain local LUSD.

The network needs a useful subset of functioning routes. Common control of every issuer has not been established. Exact authorities and changes are in the [issuer study \(Appendix 38\)](#) and [authority map \(Appendix 29\)](#).

StakeWise sETH2 and the minting permission

MINTER_ROLE is permission to create token units, subject to the token code's other checks. It is not collateral or a bridge. A bridge-linked issuer could use that permission to turn authenticated L2 backing into the existing local token that a Curve pool accepts.

StakeWise provides evidence for the local issuance and pool-acceptance parts. On 20 June 2026, its Safe installed custom sETH2 and rETH2 implementations and initialized a8ed as their minter. Three subsequent successful direct calls created **one million, one trillion and two trillion sETH2** for a8ed. Their traces show local issuance and reward-checkpoint calls, without an L2 message or collateral transfer in those transactions. The amounts do not establish ETH or dollar backing. SWISE received a similar minter upgrade on 8 July.

On 2 August, another address deposited about **210.41 million sETH2** into an existing sETH2/stETH Curve pool. The trace proves Curve accepted the unchanged sETH2 address under its custom implementation. It does not trace those deposited units back to the a8ed mints. Subsequent observed swaps paid stETH into the pool and received sETH2; they did not extract stETH for pDAI purchases.

Proposed bridge route:

L2 asset → authenticated bridge or message → compatible local issuer

```
↓ mint existing sETH2
```

funded sETH2/stETH Curve pair

Observed local route:

```
a8ed direct call → custom sETH2 mint
```

separate large sETH2 position → existing Curve pair

Several bridged assets could back one sETH2 receipt if an authorized issuer accepted and accounted for them. Curve would still see one sETH2 coin slot. Its matched implementation prices trades from token balances, amplification and fees; it has no external Arbitrum price-feed input. An oracle could inform a future issuer's conversion policy, but would not independently set this pool's executable price or supply its counterassets.

The upgraded components are **StakeWise token proxies and its separate deposit pool**, rather than Curve's trading-pool implementation. A new compatible issuer could custody PLS, Ethereum-backed ETH or WBTC representations and issue the existing token. Setting one token to redeem for one PLS would

require that funded conversion rule; mint authority or a claimed oracle price alone would not establish it. The exact external representation, bridge backing and redemption rights remain to be proved. Unrelated tokens sent directly to Curve do not become its configured coins.

At the saved checkpoint, the StakeWise route is interrupted. On 14 September, the same Safe explicitly pointed sETH2 to StakeWise **Pool** code. At refreshed block **27,690,169**, that runtime exactly matches the verified Pool implementation, ERC20 getters revert, and both inherited Curve pairs' quotes fail. The two pairs hold zero and about **9.10 local stETH**, respectively. A working token implementation, authenticated bridge route, valuation and funded liquidity are still required. The separate StakeWise deposit pool was also upgraded to a native sweeper and lost its **167.17 million PLS** balance in June. The [StakeWise study \(Appendix 63\)](#) records the exact upgrades, minter calls, recipient, pool identities and evidence limits.

Existing lending links and the local Lido branch

sETH2 is already listed directly in Fuse pools 46 and 149 and in a legacy Silo market. This gives the thesis an existing collateral route, rather than requiring every protocol to add a new token. Fuse's StakeWise market was designed for **sETH2 collateral → FEI loan → FEI/3CRV → local stables**. The loan remains debt. At block **27,690,565**, the sETH2 token and price calls fail, the Fuse FEI market has zero cash, and the sETH2 Silo is paused with no selected price provider or copied-WETH liquidity. Restoring token functionality alone would not fund these loans.

Another branch is **sETH2 → funded exchange → local stETH → wstETH → an accepting lending market**. The checked factories contain seven Uniswap V3 pools and 65 V2 pairs for exact sETH2; Balancer's sole direct sETH2 pool pairs it with an sETH2 basket and has zero balances. A name or listing does not establish usable capacity. Existing stETH/wstETH integrations offer more connections, but cannot directly accept arbitrary sETH2 as their underlying asset.

Local Lido also accepts **native PLS → local stETH**, without a governance takeover. Its deposit allowance refills at 23.4375 PLS per block; recent verified deposits show this branch operates. wstETH is an immutable wrapper for that exact stETH. Aave V3 accepts wstETH and has about **1.65 million copied-WETH units** at its WETH receipt contract, although its sampled stablecoin cash balances are zero. Copied WETH wraps native PLS on PulseChain. These routes can supply local inventory; Ethereum redemption and dollar value require separate evidence.

Basket and vault routes already connect StakeWise assets

Index Coop's copied **dsETH** basket already contains exact **sETH2, wstETH and Rocket Pool rETH**. Rocket Pool rETH is a different token from StakeWise rETH2. An installed extension trades the basket's rETH2 rewards into sETH2 through Uniswap V3. Ordinary issuance requires proportional amounts of all three components; newly minted sETH2 alone cannot mint the whole basket. This supplies an existing integration between staking tokens, reward reinvestment and local exchange liquidity.

Control of dsETH is not confirmed. Its operator Safe remains 2-of-6. Its Orca membership module creates an indirect administrator route to changing signers, but that route ends at an inherited 3-of-5 Safe whose sampled controls are unchanged. Manager replacement also requires the separate methodologist's approval. The sampled 1967, e121 and a8ed wallets hold zero dsETH and have no sampled reinvestment permission. At the saved checkpoint, the basket's sETH2 operations are interrupted and its checked pUSDC exchange pool has zero active liquidity. It cannot be counted there as a demonstrated group-controlled stablecoin funding source.

Opium's sETH2 seller vault adds a route to funded options premiums and settlement obligations. A deployed Babylon strategy connects native staking, rETH2-to-sETH2 exchange and sETH2-to-copied-WETH exchange. The original StakeWise reward oracle and Merkle distributor name the already-

controlled StakeWise Safe as administrator; that permission can alter the local oracle roster. The five current oracle members require at least four reward-update signatures. These local reports are distinct from rollup validation and do not establish external-chain backing. The [vault and authority study \(Appendix 64\)](#) records exact contracts, current limitations and reproduction evidence.

StakeWise **rETH2** is 0x20bc832ca081b91433ff6c17f85701b6e92486c5, distinct from Rocket Pool **rETH** at 0xae78736cd615f374d3085123a210448e74fc6393. Three historical direct calls also minted one million, one trillion and two trillion rETH2. A sampled rETH2/WPLS pair holds about 323,024 WPLS against 1.923 billion rETH2. That provides a finite native-market connection, rather than proof of ETH backing or a one-PLS exchange rate. Minting either StakeWise token does not itself create native PLS or change Rocket Pool governance.

Silo governance can provide control without changing the owner Safe

The legacy SiloGovernor controls a two-day timelock. That timelock manages pauses, deposit limits and selection among registered compatible price providers. It also owns **SILO** and **XAI**. SILO has an owner-only mint function, so a successful governance proposal could mint voting tokens and strengthen subsequent voting control. Repository ownership, new price-provider registration and LTV changes have separate owner permissions.

Current governance requires 100,000 delegated SILO to propose and ten million votes for quorum at the billion-token supply. Voting lasts 21,000 blocks, followed by the two-day timelock: roughly **4.43 days** at ten-second blocks, before execution. Newly minted supply also changes future quorum. Voting-token balances must be delegated at the relevant snapshot.

This is a concrete route for the proposed Silo control. A completed takeover remains unverified: the observed July 2024 takeover proposal is **Defeated**, with about 9.703 million affirmative votes below quorum, and the completed selected post-fork governor/timelock history contains no execution. The sampled wallets are not a complete group census. Unchanged owner addresses alone cannot settle whether voting power has been acquired.

The supplied ShapeShift transactions independently prove the broader governance pattern. e121 executed an optimistic proposal through an enabled Safe module, acquired that module's ownership, returned it to the Safe, and acquired it again in a later event. Successful changes to its dispute window demonstrate configuration authority. That module does not appear in the legacy Silo owner Safe's current module list. The accompanying UMA transaction commits a vote seeking emergency-proposer ownership; its exact oracle request remains unresolved at this checkpoint. A separate Merkle-root update changes native-PLS reward entitlements, without minting or bridging assets. The [lending and governance study \(Appendix 39\)](#) records the exact routes, addresses, transactions and limitations.

Synthetix sUSD: bridge output and local collateral recycling

Existing local sUSD reaches six inherited Curve pools, including DAI/USDC/USDT/sUSD, sUSD/3CRV, sUSD/FRAXBP and an Aave V2 receipt route. It is also accepted by the sampled mUSD basket. MIM connects through both Curve bases and a direct sUSD/UST/MIM pool. Shared bases provide trading paths to OUSD and Reserve eUSD; minting those tokens separately still requires their accepted collateral.

Synthetix's own [historical loop](#) describes DAI on Optimism → wrapper-minted sUSD → bridge to Ethereum → redemption or trading. The inspected copied bridge can ask its Issuer to mint the implementation behind existing local sUSD after messenger/counterpart authentication. The copied Optimism StateCommitmentChain still configures a **604,800-second, seven-day** fraud-proof window. Its batch

counters match the inherited snapshot, and a complete selected postfork event query found no new state batches. Fresh private-L2 issuance therefore still needs evidence of an accepted destination message/root path. A private source RPC does not disprove the design; it prevents source verification.

The [expanded control audit \(Appendix 52\)](#) finds Synthetix's **protocolDAO** owner still a **5-of-13 Safe**; the copied Optimism registry/messenger owner is a separate **5-of-7 Safe**. Sampled public controls match inherited state; private-key custody remains unknown. SNX stakers elect the Spartan Council, but policy representation and multisig execution authority are separate. The [council study \(Appendix 68\)](#) checks the additional council Safes and distinguishes the May 2023 roster from the obsolete copied L1 NFT and unverified current private-L2 membership. Synthetix's authenticated owner relay can administer suitably owned L2 contracts without ownership of the global messenger.

Successful postfork Optimism withdrawals are now verified, including **301,632.643171596086897512 pDAI delivered to Across HubPool**. Their proofs use April/May 2023 commitments already accepted before the fork. They prove local settlement and exact pool-token delivery, while fresh private-L2 commitments remain unverified. The DAI transfer releases escrow; Synthetix's synth bridge has a separate authorized mint path.

There is a demonstrated local recycling route: registered wrapper `0xe01698760ec750f5f1603ce84c148bab99cf1a74` accepts existing pDAI and issues sUSD. **Five successful historical e121 transactions on 4-7 September 2026 deposited about 136,554.716 pDAI and issued 136,213.329 sUSD**. One deposits 30,000 pDAI and returns 29,925 sUSD. The later recorded getter checkpoint has a **20-million-DAI cap, 5.352-million reserves and 14.648-million spare capacity**, with the selected pause/cache predicates clear. Reading those existing receipts performs no mint or simulation.

```
Source synth → authenticated bridge → existing local sUSD
                                     ↓ buy cheap pDAI
                             purchased pDAI inventory
                               ├── retain for peg support
                               ├── eligible LP / earning position
                               └── DAI wrapper → additional local sUSD
                                     ↓ further trading
```

The DAI wrapper's sampled fees are 25 basis points to mint and 5 basis points to burn. Its architecture creates a conversion band against **sUSD**, rather than independently proving a US-dollar price. Deposited pDAI backs redeemable synth claims; it cannot also be counted as freely spendable treasury inventory. The depositing operator need not redeem, but eligible other synth holders may do so. Ordinary wrapper custody does not itself pay deposit interest.

The selected factory has two recognized children: DAI/sUSD and copied-WETH/sETH. The latter has a 20,000-WETH cap and zero sampled spare capacity; older EtherWrapper/native-adapter contracts also exist. No USDC, USDT, mUSD, FRAX or MIM child was found in that factory. Its owner can create additional wrappers, and the owner-controlled SystemSettings can change individual caps and fees. Those actions require actual authority; control by the proposed group has not been proved.

The [integration inventory \(Appendix 69\)](#) connects exchanges, lending, bridges and strategies. It verifies e121's sUSD activity since April 2025 and 1967's Curve/ySUSD/lending activity in 2026. Four local direct-integration addresses retain special exchange terms; the five tested research wallets receive defaults. Sampled sETH/sBTC rates are invalid, so their historical atomic trading routes are not certified as presently working. sUSD can distribute inventory through existing pools without separately changing every issuer.

Ten-second purchases and bridge settlement are separate clocks. A seven-day replenishment interval contains 60,480 purchases: at 1,000 sUSD each, the flow uses 60.48 million sUSD before fees and buffers. A large transfer can settle first and fund gradual spending. The observed local DAI-wrapper capacity is much smaller than that illustration or 50-billion-token issuance. Scaling requires additional accepted capacity or separately funded routes.

The common integration criterion remains the **final PulseChain token accepted by the pool**, regardless of its L2 address. The dedicated [sUSD route study \(Appendix 66\)](#) retains the exact pool and bridge identities; the control audit supplies the later cap, Safe and finality checks.

Evidence coverage for the complete thesis

This index preserves the discussed mechanisms, including unresolved proposals, without treating every historical claim as verified. Dated source studies and frozen evidence retain the historical record. Maintained findings and new evidence are linked below.

Topic	Evidence and scope
Closed arbitrage and pDAI as a central conversion asset	Executed mUSD and post-shutdown sDAI cycles (Appendix 25) ; modeled pool cycles (Appendix 47) , including profitable closing trades that lower a pDAI exit price
DOLA, FEI, mUSD, OUSD, Maker and named-wallet funding	Reconciled issuance and conversion routes (Appendix 35) ; Maker shutdown and savings/End branches above
Every catalogued Curve coin and issuer family	618 inherited pools, 381 token identities and 74 monetary candidate families (Appendix 37) ; issuer collateral and governance gates (Appendix 38) . Common control of every issuer is unproved
Rebases and controlled supply buffers	PWRD/STBT live accounting, OUSD/UZD exclusions, Lybra stored accounting and resource checks (Appendix 36) ; FRAX AMOs, DOLA Feds and receipt issuance in the network catalogue
Exact bridge outputs, escrow and specialized minting	Gateway withdrawal census (Appendix 11) , ARB/GRT/LPT/HGOLD branches (Appendix 20) , GRT mint reconciliation (Appendix 12) and control history (Appendix 10)
Source-side staking and collateral receipts	Historical Arbitrum Aave, Curve and GMX candidates (Appendix 15) , Reserve bridged collateral (Appendix 18) and Reserve governance history (Appendix 19) . Private current L2 deployments remain unverified
Portal caps, wrappers and exact change permissions	Portal pricing and wrapping (Appendix 48) , 17 linear pools and receipt inventory (Appendix 24) , reserve limits (Appendix 16) and authority map (Appendix 29) . Examined ACL Manager: <code>0xc2aacf6553d20d1e9d78e365aaba8032af9c85b0</code>
All examined 86ff debt and deferred backing	Twelve V3 debts and campaign clocks (Appendix 34) , wallet positions (Appendix 21) , Portal accounting (Appendix 17) . Debt repayment and reserve backing are separate
Assertions, validators, delays and messenger control	Assertion acceptance and receipts (Appendix 26) , receipt checks (Appendix 49) , Optimism/Across relays (Appendix 46) . Local optimistic governance is distinct from owning Optimism consensus

Topic	Evidence and scope
Reality SafeSnap, Zodiac, Gnosis and BondManager	Safe modules, threshold changes, nested authority and rollup controls (Appendix 52). An execution in one governed protocol does not establish universal messenger ownership
Synthetix sUSD, wrappers, integrations and councils	sUSD routes (Appendix 66), registered integrations (Appendix 69), storage and control (Appendix 67), elections and council Safes (Appendix 68). Seventeen seats exclude the five Grants seats; current PulseChain representatives are unverified
StakeWise, Lido, Fuse/Rari, Silo, Index and strategies	Token/pool proxy changes and mint traces (Appendix 63), lending and exchange listings (Appendix 39), dsETH, Opium, Babylon and reward authority (Appendix 64). Silo's examined takeover proposal failed; dsETH control remains unproved
Native PLS, HEX, PLSX and INC financing	Local PLS funding (Appendix 42), native-asset evidence (Appendix 41) and the integrated-network native-yield section. Market PLS proceeds, owned emissions and released principal need separate ledgers
Atropa, gradual purchases and inventory growth	Atropa exact pairs and burned LP limits (Appendix 27), coordinated markets (Appendix 31), liquidity growth (Appendix 40) and the deferred campaign. Fifty billion tokens are not assumed to enter an LP at once
Dominant ownership, V2 bands and Ethereum/BTC exits	Managed peg and LP exits (Appendix 45), route design (Appendix 62), stability bounds (Appendix 60). Funded local parity and external purchasing power are separate promises

The [five-analyst consensus study](#) (Appendix 70) reviews the integrated L2, accumulation, privacy and dollar-support interpretation against frozen evidence. It preserves all fifteen assessments, remaining dissent and reproduction references.

Previous multi-round reviews are retained in the [three-round assessment](#) (Appendix 71) and [complete-paper review](#) (Appendix 30). Agreement between reviews is a clarity check; reproducible balances, code, authority and complete execution are the evidence for a mechanism.

The [completeness and clarity assessment](#) (Appendix 73) records the 3 October draft's coverage, research benchmark and remaining proof requirements. Its review agreement applies to that frozen draft; the added Aave/Portal and Curve explanations retain the cited studies' evidence limits.

The [restorative thesis assessment](#) (Appendix 50) validates the current funding, conversion and price-control argument with five analysts, including FEI's optional role and the remaining proof requirements.

Appendix Index

Each appendix contains its complete source Markdown after de-identification and link conversion. Historical text can contain superseded findings; the dated corrections and current main text govern the synthesis. Original embedded hashes identify pre-redaction records. The release manifest identifies the files actually supplied here.

Maintained research studies are printed below. Frozen drafting snapshots, individual model assessments, raw evidence and third-party source copies are indexed in the accompanying working archive. Review rounds were AI-assisted and shared the underlying evidence; they are not a human comprehension study or independent audit.

- Appendix 1: Two recorded calls to the 86ff contract
- Appendix 2: Copied Aave governance: public ownership, upgrade and messaging history
- Appendix 3: Across and UMA control on PulseChain: a bounded historical audit
- Appendix 4: Copied Arbitrum gateway mint inventory on PulseChain
- Appendix 5: Copied Arbitrum bridge on PulseChain: recorded activity and token compatibility
- Appendix 6: Examined PulseChain addresses: established roles and bounded leads
- Appendix 7: Conditional cross-chain asset representation: May 2023 models
- Appendix 8: Four copied Curve metapools: bounded historical configuration review
- Appendix 9: Copied Aave debt repayment: bounded public transaction audit
- Appendix 10: Copied Arbitrum gateway discovery and historical permission audit
- Appendix 11: Copied Arbitrum escrow gateways: post-fork withdrawal receipt census
- Appendix 12: PulseChain GRT gateway: mint transaction and allowance reconciliation
- Appendix 13: Native-L2 assets and L1 representations: the May 2023 bridge baseline
- Appendix 14: Across HubPool custody and Aave Portal settlement
- Appendix 15: Native Arbitrum assets around May 2023: a documentary baseline for pARB
- Appendix 16: Balancer Aave wrapper pools: balances, limits and public transaction evidence
- Appendix 17: Portal, waDAI and repayment: accounting of the proposed loop
- Appendix 18: Reserve eUSD: bridged collateral and issuance of the existing pool token
- Appendix 19: Reserve eUSD Main and timelock: recorded governance history
- Appendix 20: Post-fork history of specialized copied Arbitrum gateways
- Appendix 21: Two examined wallets: recorded Aave debt, collateral and repayment rights
- Appendix 22: Withdrawal event coverage and historical supply reconciliation
- Appendix 23: PulseChain Arbitrum gateway withdrawals: historical execution audit
- Appendix 24: Aave receipts registered in copied Balancer pools
- Appendix 25: Executed arbitrage and Maker routes after shutdown
- Appendix 26: Assertion acceptance, source balances and transferable claims
- Appendix 27: Atropa liquidity connections and routes into stability pools
- Appendix 28: Canonical bridge flows, distributed accumulation and intent
- Appendix 29: PulseChain thesis evidence and authority
- Appendix 30: Review of the complete pDAI white paper
- Appendix 31: Coordinated issuance and connected-pool correction
- Appendix 32: PulseChain bridge, governance and DeFi evidence audit
- Appendix 33: How source-chain assets could support pDAI demand on PulseChain
- Appendix 34: Deferred Portal settlement and scheduled pDAI accumulation
- Appendix 35: Equal-value stablecoin routes: executed issuance, redemption and funding
- Appendix 36: PulseChain funding conversions and rebasing pool reserves
- Appendix 37: How bridges, stablecoin issuers and pools can form one pDAI support system
- Appendix 38: L2 collateral, local stablecoin issuance and the Curve network
- Appendix 39: StakeWise and Lido routes through PulseChain lending and governance
- Appendix 40: Fifty-million-pDAI execution and complete reinvestment

- Appendix 41: Local pDAI-to-PLS exit evidence and publication boundaries
 - Appendix 42: Local PLS financing, settlement and bounded pDAI support
 - Appendix 43: Finite pDAI exits into PLS: a fee-inclusive support model
 - Appendix 44: Maker adapter state and the remaining pDAI routes
 - Appendix 45: Inventory-supported local pricing and a proposed LP exit
 - Appendix 46: Optimism relays and Across governance on PulseChain
 - Appendix 47: pDAI: profitable pool cycles and coordinated liquidity
 - Appendix 48: PulseChain Portal backing, aDAI index and local arbitrage
 - Appendix 49: Existing aTokens, new Portal issuance and validator access
 - Appendix 50: Validation of the restorative pDAI thesis
 - Appendix 51: End-to-end routes: issuance, arbitrage and settlement
 - Appendix 52: PulseChain Safe governance and rollup controls
 - Appendix 53: Skeptical economic review — independent first round
 - Appendix 54: Skeptical economic review — second round
 - Appendix 55: Skeptical evidence review, round 1
 - Appendix 56: Skeptical evidence review, round 2
 - Appendix 57: Skeptical protocol review — round 1
 - Appendix 58: Skeptical protocol review — round 2
 - Appendix 59: Two-round skeptical review of the PulseChain mechanism thesis
 - Appendix 60: Stability bounds, two-sided correction and adverse tests
 - Appendix 61: Ten-pass review: method, findings and claim boundaries
 - Appendix 62: Routes and operating conditions for a stable pDAI
 - Appendix 63: StakeWise proxy changes and Curve asset routes
 - Appendix 64: StakeWise vaults, Index Coop and reward authority on PulseChain
 - Appendix 65: pDAI accumulation, income and the stability feedback
 - Appendix 66: sUSD: existing pool connections and issuer-specific bridge minting
 - Appendix 67: Synthetix control, copied Optimism messaging and purchased-stable custody
 - Appendix 68: Synthetix councils and their copied PulseChain authority
 - Appendix 69: Synthetix integrations and observed research-wallet activity
 - Appendix 70: The integrated pDAI accumulation-and-peg thesis
 - Appendix 71: PulseChain thesis: three-round comprehension and evidence study
 - Appendix 72: Twelve-point follow-up: local PLS exits and financing conditions
 - Appendix 73: Completeness and clarity of the pDAI white paper
-

Appendix 1: Two recorded calls to the 86ff contract

Contents of this appendix

Finding.....	31
FdbEE transaction: allowance observation, no token transfer.....	31
b157 transaction: successful wrapper, reverted internal call.....	32
Evidence and limits	32

Source record: docs/studies/analysis/parb_86ff_withdrawal_observations_20260929.md.

Two recorded calls to the 86ff contract

Research date: 29 September 2026. Scope: a bounded, read-only review of two existing transactions involving `0xe6df4ca976f3fec8da89efec332dc2245f0a86ff`. This addresses what these transactions demonstrate about token withdrawals; it is not a complete audit of the contract or its access controls.

Finding

Neither selected transaction demonstrates an asset withdrawal. The FdbEE transaction successfully reads a cETH allowance. The b157 transaction has a successful outer receipt but a reverted internal delegated call. Treating either outer success status as proof that tokens were withdrawn would misdescribe the execution evidence.

These results do not prove that 86ff never enabled withdrawals in other transactions. They establish the behavior of these two selected records. The contract has no verified source in the preceding discovery batch, so this note does not infer public access, ownership or arbitrary-token withdrawal rights from a function selector or the presence of multiple callers.

FdbEE transaction: allowance observation, no token transfer

Transaction `0x57ef...dacd` occurred at block **26,687,406, 2 June 2026, 19:17:45 UTC**, hash `0x4349dbf0c25a6741c9dee03c84b05c5002f12dece39d373e54d7572fc33bdd99`. The sender was `0xfdbbee32bcf68bb0ced033200373384a619c71967`; the destination was 86ff. Its receipt reports success and contains no logs.

The historical trace contains only two frames: the outer call to 86ff and an inner **static call** to `0x4ddc2d193948926d02f9b1fe9e1daa0718270ed5`. The latter contract reports symbol **cETH** at this block. The inner call reads the allowance from 86ff to FdbEE and returns the maximum unsigned 256-bit value.

This supports a specific positive observation: the token contract reported that allowance for those two addresses during the transaction. It does not show when or how the allowance was established. No `approve`, `transfer`, `transferFrom`, token redemption or native-currency payment appears in this execution trace. There is no observed withdrawal amount to report.

A token allowance is also not a record that FdbEE owns 86ff, owns cETH's underlying lending protocol, or controls every asset held by 86ff. Each is a separate claim. This review does not convert the observed allowance into a claim about the authorization rules of every 86ff operation.

b157 transaction: successful wrapper, reverted internal call

Transaction [0x2b6f...8de5](#) occurred at block **25,967,897**, **8 March 2026, 01:41:15 UTC**, hash [0x8bd832c4fe1d3d1ce7d9c762714be1414f1bb87478e3964acfe14b1f4afb609a](#). The sender was [0xb1573ff4421bf7f0b5d18aa9723f01a79930c68e](#); the destination was 86ff. Its receipt reports success and contains one log emitted by 86ff.

The trace again contains two frames. The second is a **delegatecall from 86ff to itself**, which reports **Reverted** and has no subcalls. The outer frame nevertheless completes successfully. Consequently, the receipt's overall success must not be applied to the failed internal operation.

The recorded input contains the copied WETH address, but the actual trace does not reach that token contract. An address appearing in input bytes is not an executed token call. There is no observed WETH transfer, deposit, withdrawal, native-currency payment or Aave interaction in this transaction. The 86ff log has not been assigned a source-verified event name; it is not treated as an ERC-20 Transfer event or proof of an asset movement.

Evidence and limits

The observations use `eth_getTransactionReceipt`, `trace_transaction`, `eth_getBlockByNumber` and a token-symbol getter through <https://rpc.pulsechain.com>. Block numbers and hashes are specified individually above. These are one-provider observations, not an independently verified state proof. No transaction, simulation, impersonation, constructed withdrawal call or state change was performed.

Raw evidence is preserved under ignored `artifacts/parb-wallet-debt-positions-20260929/86ff_observations/`; each sender's receipt, historical trace and block metadata, plus the cETH symbol read. The selected hashes came from the preceding recent-transaction discovery record; that limited discovery list is not an exhaustive contract history.

Any stronger claim that anyone can withdraw any token would require evidence of the actual authorization logic and the relevant successful asset movements. These two records establish an allowance read and a caught internal failure, not that universal withdrawal claim. They likewise do not establish debt-token ownership, repayment, profit allocation or the owner of 86ff.

Appendix 2: Copied Aave governance: public ownership, upgrade and messaging history

Contents of this appendix

Findings and scope	33
The July 2023 V2 ownership change and upgrades	34
Proposal 225 and the examined BAD9 wallet	34
The examined FdbEE32 wallet and August 2026 governance actions	34
What proposal 218 actually records	35
Proposal 218's intended action: list MAI on Aave V3 Optimism	35
Proposal 214: a real stkAAVE upgrade, with an unset GHO hook	36
What the V3 evidence does and does not show	37
Governance participation versus administrative control	37
Methods, evidence and limits	38

Source record: docs/studies/analysis/parb_aave_history_20260929.md.

Copied Aave governance: public ownership, upgrade and messaging history

Research date: 29 September 2026. This is a retrospective study of publicly recorded PulseChain transactions and contract state. No transactions, impersonation or local state-changing experiments were performed for this study.

Findings and scope

There is direct evidence of post-fork changes to copied Aave infrastructure. In July 2023, governance proposal 224 transferred ownership of the Aave V2 addresses provider to `0xa30190b96faee0080144aa0b7645081fcbf49e6f`. The same address submitted two subsequent V2 implementation upgrades. In August 2026, other wallets queued existing proposals, one proposal emitted an outbound governance message through the copied Optimism messenger, and another upgraded stkAAVE to an implementation supporting a GHO debt-token hook.

These are distinct findings with distinct implications. The V2 provider ownership transfer is evidence of a particular administrative change. Publicly queueing or executing an already-approved proposal is not, by itself, evidence that the caller owns governance. A sent message is not proof that its destination executed. A GHO-compatible staking implementation is not proof that a GHO debt token has been connected. No selected post-fork Aave V3 Portal events were found in the examined range, and all 17 reserve caps in the subsequent wrapper census are zero.

The principal snapshot is PulseChain chain 369, block **27,659,393**, hash `0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275`. The event census spans blocks **17,233,000–27,659,393**, excluding inherited Ethereum history through block 17,232,999. Timestamps below are UTC. The evidence does not identify a real person controlling any address or establish common ownership among the wallets.

The July 2023 V2 ownership change and upgrades

The copied governance contract is `0xec568fffb86c094cf06b22134b23074dfe2252c`; its short executor is `0xee56e2b3d491590b5b31738cc34d5232f378a8d5`. The relevant V2 provider and pool are respectively `0xb53c1a33016b2dc2ff3653530bff1848a515c8c5` and `0x7d2768de32b0b80b7a3454c06bdac94a69ddc7a9`.

Wallet `0xa30190b96faee0080144aa0b7645081fcbf49e6f` created proposal 224 on **27 June 2023 16:04:35**, block **17,638,713**, and queued it on **30 June 2023 18:11:15**, block **17,665,133**. The proposal's decoded actions specified two ownership transfers to that address and a short-executor pending administrator change. [Creation transaction](#), [queue transaction](#).

The successful execution on **1 July 2023 18:11:35**, block **17,673,565**, contains an `OwnershipTransferred` event from the V2 provider, naming the short executor as previous owner and `0xa301...9e6f` as new owner. The same receipt contains another ownership transfer at `0x3d569673daa0575c936c7c67c4e6aeda69cc630c`, a short-executor pending-administrator event, and `ProposalExecuted(224, ...)`. A pending-administrator event is not an administrator-acceptance event and must not be described as one. [Execution receipt](#).

The same wallet then submitted two successful transactions directly to the V2 provider, each producing a V2 pool `Upgraded` event:

UTC time, 1 July 2023	Block	New implementation	Transaction
18:19:15	17,673,611	<code>0xf3497ef6894da33b3f360f589cd6cfe5230f5984</code>	Receipt
18:27:25	17,673,660	<code>0xf30e1f577bea90a61f82ef136fbf26082947ea6d</code>	Receipt

The second implementation remains in the sampled V2 implementation slot at the principal block. This establishes a specific sequence of provider ownership and pool code changes. It does not establish a V3 upgrade, Portal activation or the purpose and full behavior of the replacement V2 code. The earlier study's reverting V2 reserve reads cannot be treated as evidence of successful debt repayment or arbitrage functionality without separately inspecting that code.

Proposal 225 and the examined BAD9 wallet

At **18:14:25 on 1 July 2023**, between the ownership transfer and the first pool upgrade, `0xa301...9e6f` created proposal **225**. Its creation event lists six actions: four `changeAdmin(address)` calls, a `governance transferOwnership(address)` call, and an executor `setPendingAdmin(address)` call, all naming `0xa301...9e6f` as the proposed recipient. The targets are the copied AAVE, `stkAAVE`, `ABPT` and `stkABPT` proxies, governance, and the long executor. These are proposed actions; a creation event does not prove they took effect. [Proposal creation](#).

The examined wallet `0xbad9dff1026e59ea4c39cc3398aa5eb6f6f229ac` successfully queued proposal 225 on **11 October 2024 11:36:45**, block **21,638,432**. The recorded event names it as `initiatorQueueing`. No proposal-225 execution event appears in the bounded governance census through the principal block. The queue transaction therefore establishes participation in that proposal's lifecycle, not the completed six transfers or ownership by the queueing wallet. [Queue receipt](#).

The examined FdbEE32 wallet and August 2026 governance actions

Wallet `0xfdbee32bcf68bb0ced033200373384a619c71967` queued three proposals on **3 August 2026**:

UTC time	Proposal	Block	Transaction
13:16:05	219	27,194,469	Receipt
13:22:25	214	27,194,504	Receipt
14:38:45	218	27,194,946	Receipt

The same wallet executed proposal **218** on **5 August 2026 13:48:05**, block **27,211,293**. No execution of proposal 219 appears in this bounded census. Proposal 214 was executed by a different address, described below.

What proposal 218 actually records

The successful receipt contains a `ProposalExecuted(218, ...)` event, the short executor's transaction event, a `SentMessage` event from `0x25ace71c97b33cc4729cf772ae268934f7ab5fa1`, and a `TransactionEnqueued` event from `0x5e4e65926ba2746755eb562121fac00d24e9dd2`. The latter has verified source named `CanonicalTransactionChain`; the messenger is the copied Optimism resolved-delegate proxy. This is the copied Optimism messaging route, not evidence of an Arbitrum bridge withdrawal. [Public execution receipt](#), [queue contract source](#), [messenger proxy source](#).

Decoding the already-recorded `SentMessage` data gives:

Field	Recorded value
Message sender	Short executor <code>0xee56e2b3d491590b5b31738cc34d5232f378a8d5</code>
Destination target	<code>0x7d9103572be58ffe99dc390e8246f02dcae6f611</code>
Nonce	348,126
Gas-limit field	1,920,000
Message method	<code>queue(address[],uint256[],string[],bytes[],bool[])</code>
Queued target	<code>0x1720d605289a189c5fbd360ca2307db5c6d3fdfd</code>
Queued action	<code>execute()</code> , zero native value, empty argument bytes, <code>delegatecall</code> flag true

The corresponding enqueue data contains the `relayMessage` envelope. The event topics and nested selectors were checked by Keccak signature hashes, and dynamic arrays were decoded from the receipt's existing bytes. No transaction was constructed. The receipt proves submission of this governance message on PulseChain. It does not prove a receiving rollup exists, that the message was relayed, or that the queued target's payload executed on another chain. It also does not contain an Aave Portal mint or stablecoin bridge-mint event. The payload adapter itself was not explorer-verified in the source request, so its complete implementation is not inferred solely from these decoded fields.

Proposal 218's intended action: list MAI on Aave V3 Optimism

The message can now be identified beyond its selector. Its complete **580-byte inner queue message is byte-for-byte identical** to the message recorded in the [original proposal-218 Seatbelt report](#). Both identify the same destination executor, target payload, zero native value, `execute()` signature, empty

arguments and delegatecall flag. The outer message nonces differ: 348,126 in the observed PulseChain receipt and 348,777 in the original report. This is equality of the inner instruction, not equality of the complete cross-chain envelope or proof of receiving-chain execution.

[Aave's original proposal page](#) identifies its purpose as adding MAI to Aave V3 on Optimism. It was created on 8 May 2023 at block 17,216,247, before the PulseChain snapshot; voting ended at block 17,242,647, after that snapshot. Its Ethereum execution on 13 May 2023 is separate from the PulseChain execution on 5 August 2026. This supports an inherited proposal being acted on years later, rather than a newly authored PulseChain proposal with a different instruction. It does not explain the 2026 caller's motive.

The [published original listing payload](#) provides the intended reserve configuration:

Setting	Original listing instruction
Asset	MAI, 0xdfa46478f9e5ea86d57387849598dbfb2e964b02 on Optimism
Price feed	0x73a3919a69efcd5b19df8348c6740bb1446f5ed0
Supply cap	7,600,000 MAI
Borrow cap	2,500,000 MAI
Isolation debt ceiling	1,900,000 in the protocol's dollar-denominated accounting
Collateral	Enabled in isolation mode; 75% LTV; 80% liquidation threshold
Liquidation bonus / protocol fee	5% / 10%
Reserve factor	20%
Borrowing	Enabled; stable-rate mode disabled
Flash loans	Disabled for the new asset

The original [Optimism subreport](#) uses that same executor/payload address and records the corresponding listing configuration in its original-network analysis. These sources establish the historical intended payload; they do not verify bytecode or storage on an unidentified PulseChain-linked Optimism execution network.

In plain language, the message requests: queue the specified MAI listing payload for the destination Aave governance executor. It is not a validator attestation of trillions of stablecoins, a collateral deposit, or a Portal mint request. A received queue instruction would still require the receiving executor's checks and subsequent execution. A successful PulseChain send does not itself reach Ethereum's canonical Optimism network.

Comparison evidence is preserved in `proposal218_message_comparison.json`; original report and payload snapshots are in the same ignored evidence batch. No original-network simulations were rerun. The analysis compares recorded bytes and published historical documents only.

Proposal 214: a real stkAAVE upgrade, with an unset GHO hook

Wallet 0x63f4ce5786754ff56adc54b1d48cb46953ffb817 executed proposal **214** on **10 August 2026 13:36:45**, block **27,252,708**. The transaction succeeded and contains both the proposal execution event and stkAAVE's Upgraded event. [Execution receipt](#).

Implementation-slot reads immediately before and at that block independently show the transition:

Block	stkAAVE implementation
27,252,707	0xe2e8badc5d50f8a6188577b89f50701cde2d4e19
27,252,708	0xaa9faa887bce5182c39f68ac46c43f36723c395b
27,659,393	0xaa9faa887bce5182c39f68ac46c43f36723c395b

The verified proposal payload at `0xe427fcbd54169136391cfedf68e96abb13da87a0` explains the receipt. Its `execute()` moves stkAAVE's proxy administrator, deploys a `StakedAaveV3` implementation, and upgrades and initializes the proxy. The receipt names the prior administrator `0x79426a1c24b2978d90d7a5070a46c65b07bc4299` and new administrator `0x86c3ffee349a7cfff7ca88c449717b1b133bf517`. Initialization assigns the slashing, cooldown and claim-helper roles to the short executor; its configuration includes 30% maximum slashing and a 20-day cooldown. These changes are staking-system changes, not a stablecoin reserve being supplied or repaid. [Verified proposal payload and dependencies](#).

The payload's included `StakedAaveV3` source returns revision **5** and includes the public `ghoDebtToken` hook. This supports the observation that a GHO-compatible implementation was introduced. However, the verified payload does not call a GHO debt-token setter. At the principal snapshot, `ghoDebtToken()` returns the zero address, the canonical GHO address has no code, and its V3 reserve configuration is zero, as recorded in the earlier fixed-block `gho.json` reads. No `GHODebtTokenChanged` event was found in the selected post-fork census. The documented conclusion is **GHO hook support introduced, connection not established at the sampled address and block**. A separately deployed custom GHO would need its own identified contract evidence.

What the V3 evidence does and does not show

The V3 pool at `0x87870bca3f3fd6335c3f4ce8392d69350b4fa4e2` reports the same implementation, `0xf1cd4193bbc1ad4a23e833170f49d60f3d35a621`, at block 17,233,000, before and after the stkAAVE upgrade, and at the principal block. No V3 pool `Upgraded`, `MintUnbacked` OR `BackUnbacked` events were returned over the searched post-fork interval. The selected configurator census likewise returned no upgrade, unbacked-cap or supply-cap changes; the ACL census returned no role grants, revocations or role-administrator changes; the V3 provider census returned no ownership transfer.

The previously recorded state shows DAI, USDC and USDT unbacked caps and outstanding unbacked balances all zero. The [wrapper census \(Appendix 24\)](#) subsequently checked all 17 listed reserve configurations and found their unbacked caps zero. Taken together, these observations do not support a claim that Portal issuance or settlement through this exact V3 pool has already driven the observed wrapper trades. This is a bounded negative finding about specific addresses, event signatures and blocks. It is not an exhaustive search of every possible alternative deployment, every reserve action or all repay/borrow activity. Unchanged implementation pointers alone do not establish that every storage value is unchanged.

Governance participation versus administrative control

The verified governance implementation exposes `queue(proposalId)` and `execute(proposalId)` as external methods without an owner-only restriction. Queueing requires the proposal to be in the succeeded state. Execution requires the queued state and invokes the configured executor, whose timelock rules also apply. Their emitted initiator fields identify `msg.sender`. [Verified governance source](#).

Accordingly, BAD9's queue of 225 and FdbEE32's queues and execution are genuine public actions, but do not alone establish their voting power, proposal authorship, executor ownership or control of all downstream protocols. In contrast, proposal 224's successful ownership-transfer event and the

subsequent provider upgrade transactions establish a concrete administrative change at the specified V2 provider. The distinction is derived from the contract and receipts, not from assumptions about the wallets' motives.

Methods, evidence and limits

The underlying census used `eth_getLogs` over the explicit interval and saved block headers, originating transactions and successful receipts for matching events. Both event datasets report no query errors. Implementation checkpoints use the EIP-1967 implementation storage slot. This research pass decoded already saved receipts and retrieved verified source through the public explorer. The event and calldata signature checks use the read-only `web3_sha3` method.

Evidence is retained under ignored `artifacts/parb-aave-history-20260929/`: `events.json`, `governance_events.json`, `implementation_checkpoints.json`, `stk_upgrade_transaction.json`, **verified source responses**, `proposal218_signatures.json`, and `proposal218_decoded.json`. The earlier `artifacts/parb-validation-20260928/gho.json` supplies the aligned GHO reads. The maintained study records findings and reproduction references rather than embedding bulk logs or copied source.

This is RPC/explorer evidence, not independently verified consensus state. Event absence is limited to the searched signatures and contracts. A complete claim about remote execution requires receiving-chain evidence; a claim about motive requires evidence beyond transaction mechanics. The chronology establishes real V2 ownership changes, V2 and stkAAVE upgrades, and governance messaging, while leaving those larger causal claims unproved.

Appendix 3: Across and UMA control on PulseChain: a bounded historical audit

Contents of this appendix

Findings	39
Scope and historical boundary	40
Control map at the inherited and endpoint blocks	40
HubPool ownership changed through an oracle-mediated Safe execution	41
The optimistic-governance module changed owner to e121	41
The UMA dependencies are not one interchangeable oracle	41
GovernorV2 proposals and the limits of the ownership claim	42
Reproduction and interpretation boundary	42

Source record: docs/studies/analysis/parb_across_control_20260929.md.

Across and UMA control on PulseChain: a bounded historical audit

Research date: 29 September 2026. This study uses public, read-only contract state, logs, receipts, historical execution traces and published source. It examines the claim that the examined wallets acquired control of copied Across and UMA components. It does not attribute addresses to people or assume that ownership of one component establishes ownership of an entire protocol.

Findings

The subsequent [proposal and relay audit \(Appendix 46\)](#) matches the HubPool ownership proposal to the execution below by both proposal hash and UMA assertion ID. The undisputed assertion settled after five days, although its embedded rules excluded HubPool calls. These records establish on-chain execution, not a valid off-chain Snapshot vote. The later owner-only chain-88888 configuration is also verified there.

Two control changes are directly established. Across's copied **HubPool** changed owner to `0xfeca7291cebe75789ffa707754db259187b1d140` on 19 June 2026. An **OptimisticGovernor module attached to the Across Safe** changed owner to the examined e121 wallet, `0x98c89980d1eae247ada3636ae27648f78e66e121`, on 19 August 2026. Successful receipts, actual execution traces and endpoint owner getters agree on both changes.

The module used UMA's OptimisticOracleV3 while executing each change through the Safe. This is a specific, observable connection between Across governance, UMA oracle infrastructure and the named e121 address. It does not mean e121 became owner of the OptimisticOracleV3 contract or of UMA's Finder, VotingV2 or GovernorV2. Those sampled ownership/role addresses remained unchanged through the endpoint. Two GovernorV2 proposals were submitted by e121; proposal submission is distinct from execution and ownership transfer.

HubPool is an Across component, so “Across and HubPool” should not be counted as two independent protocol acquisitions. The bounded results support a component-level control map, not a finding that all Across and UMA authority was transferred to one operator. The supplied `0xb8698cffffdaaf26ca21fe8aa560c18cc2b2f5996` address was not a direct new owner in the verified changes below; no shared human ownership is inferred from other research about that address.

Scope and historical boundary

Endpoint: chain **369**, block **27,659,393**, hash

0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275. The inherited comparison is block **17,232,999**. Post-fork log searches cover **17,233,000-27,659,393**, inclusive. The endpoint block was re-read and matched. All observations come from <https://rpc.pulsechain.com>, so these are one-provider observations rather than independent consensus verification.

Across addresses were discovered from the historical deployment records at commit

a231a21df4ee58378d3f0fc8148daf5bc4791e3c: [HubPool deployment](#) and [AcrossConfigStore deployment](#). The HubPool deployment names its Finder dependency, which was then checked using the live endpoint getter. This avoids treating unrelated UMA versions as interchangeable.

Control map at the inherited and endpoint blocks

Component	Address	Inherited owner/ controller	Endpoint owner/ controller
Across HubPool	0xc186fa914353c44b2e33ebe05f21846f1048beda	Across Safe below	0xfeca7291cebe75789ffa707754db259187b1d140
AcrossConfigStore	0x3b03509645713718b78951126e0a6de6f10043f5	Across Safe	Across Safe, unchanged
Across Safe	0xb524735356985d2f267fa010d681f061dff03715	Safe account; distinct signer/module authority	Safe account; examined execution path uses its module
Across optimistic-governance module	0x8692b776d1ff0664177c90465038056dc64f8991	Across Safe	e121
UMA Finder	0x40f941e48a552bf496b154af6bf55725f18d77c3	GovernorV2 below	GovernorV2, unchanged
UMA GovernorV2 owner role	0x7b292034084a41b9d441b71b6e3557edd0463fa8	GovernorV2 itself	GovernorV2 itself, unchanged
UMA VotingV2	0x004395edb43efca9885cedad51ec9faf93bd34ac	GovernorV2	GovernorV2, unchanged
UMA OptimisticOracleV3	0xfb55f43fb9f48f63f9269db7dde3bbbe1ebdc0de	GovernorV2	GovernorV2, unchanged
UMA emergency-proposer contract	0x91f1804acaf87c2d34a34a70be1bb16bb85d6748	GovernorV2	GovernorV2, unchanged

The Safe's address is the owner of ConfigStore, but a Safe's capabilities are not described completely by an `owner()` field. The recorded traces below demonstrate module-mediated execution. An unchanged ConfigStore owner address therefore should not be presented as proof that its surrounding governance path is unchanged. Conversely, ownership of the module is not itself a record that every target configuration was changed.

The endpoint HubPool owner `0xfeca...d140` has no deployed bytecode at that block. That observation describes the account at the endpoint; it does not identify who controls it. The endpoint Safe's `getOwners()` returns five signer addresses. Those signers are not sufficient on their own to describe the module execution demonstrated in the receipts.

HubPool ownership changed through an oracle-mediated Safe execution

Transaction [0x6775...04fd](#) succeeded at block **26,822,316**, **19 June 2026, 05:46:05 UTC**, hash `0x6c83c1e7ab758bdd30d7fc8c0e245f49ee70c5ffc077528e21bc11e0f69a61de`. Its HubPool `OwnershipTransferred` event moves ownership from the Across Safe to `0xfeca7291cebe75789ffa707754db259187b1d140`; that address is also the transaction sender and the endpoint owner.

The historical trace shows a call to the module at `0x8692...8991`, delegated execution through implementation `0x28cebfe94a03dbca9d17143e9d2bd1155dc26d5d`, interaction with `OptimisticOracleV3`, a call through the Safe, and the final HubPool ownership transfer. The receipt records the state-changing event; the trace supplies the execution path. These are observations of an existing successful transaction, not a hypothetical administrative capability.

The [historical HubPool source](#) assigns its owner multiple administrative functions, including pool routes, cross-chain contract configuration and other parameters. Identifying that administrative owner is relevant to interpreting configuration history. It does not imply arbitrary mint authority over each token the HubPool holds.

The optimistic-governance module changed owner to e121

Transaction [0x703f...bc61](#) succeeded at block **27,330,450**, **19 August 2026, 23:39:05 UTC**, hash `0x0c6a25938fb01f1e3319423a742857f9901a9a16fbf0abacca8bfc47389258a`. Its sender is e121. The module emits `OwnershipTransferred` from the Across Safe to e121, matching the endpoint `owner()` result.

The historical trace follows the module, `OptimisticOracleV3` and the Safe, then returns to the module for its ownership transfer. Thus the change is more specific than an address merely appearing in a proposal or event payload. The verified implementation is [OptimisticGovernor](#). Its owner controls settings such as the proposal rules, liveness, collateral/ bond parameters and escalation-manager configuration. Its proposal execution logic separately interacts with `OptimisticOracleV3` and the avatar/target execution mechanism.

The searched module history returned 35 logs. This study verifies the material ownership changes and their execution paths; it does not claim to have audited every proposal's merits or to have established control of every contract reachable through every module configuration. No occurrence of the supplied `0xb869...5996` address was found in those returned module logs.

The UMA dependencies are not one interchangeable oracle

HubPool's endpoint `finder()` is `0x40f941e48a552bf496b154af6bf55725f18d77c3`, matching its historical deployment. Its historical dispute code resolves the **SkinnyOptimisticOracle** interface. The Safe governance module instead interacted with **OptimisticOracleV3** in the verified transfers. The same Finder registry names several distinct oracle and support contracts:

Finder interface	Address at both comparison blocks
<code>SkinnyOptimisticOracle</code>	<code>0xee3afe347d5c74317041e2618c49534daf887c24</code>
<code>OptimisticOracle</code>	<code>0xc43767f4592df265b4a9f1a398b97ff24f38c6a6</code>
<code>OptimisticOracleV2</code>	<code>0xa0ae6609447e57a42c51b50eae921d701823ffae</code>
<code>OptimisticOracleV3</code>	<code>0xf55f43fb9f48f63f9269db7dde3bbbe1ebdc0de</code>
Oracle	<code>0x004395edb43efca9885cedad51ec9faf93bd34ac</code> — verified as VotingV2

Finder interface	Address at both comparison blocks
Store	0x54f44ea3d2e7aa0ac089c4d8f7c93c27844057bf
IdentifierWhitelist	0xc649d9da4d1362c4daea67573430bd6f945e570
CollateralWhitelist	0xdbf90434df0b98219f87d112f37d74b1d90758c7

Finder ownership is unchanged and its complete post-fork log query returned no events in the bounded interval. Its [verified source](#) separates registry ownership from the individual implementations it returns. The eight sampled mappings agree at the two comparison blocks. This does not assess every interface ever registered with UMA or every oracle configuration.

SkinnyOptimisticOracle has a different interface and control structure from an Ownable application module. It must not be assigned a fictitious `owner()` because another UMA contract has one. OptimisticOracleV3 and VotingV2 are separately verified contracts with owner getters; each reports GovernorV2 at both comparison blocks and returned no ownership-transfer events in the searched interval.

GovernorV2 proposals and the limits of the ownership claim

The [verified GovernorV2 source](#) uses exclusive roles. Its sampled roles are unchanged between the inherited and endpoint blocks:

Role	Holder
Owner, role 0	GovernorV2 itself, 0x7b292034084a41b9d441b71b6e3557edd0463fa8
Proposer, role 1	0x50efac9619225d7fb4703c5872da978849b6e7cc
Emergency proposer, role 2	0x91f1804acaf87c2d34a34a70be1bb16bb85d6748

The complete GovernorV2 log query returned two `NewProposal` events in the searched interval, IDs **194** and **195**. Their successful submission receipts identify e121 as the transaction sender:

- Proposal 194: block **27,331,017**, [transaction 0x6f0d...d8b0](#).
- Proposal 195: block **27,341,471**, [transaction 0x2845...c8a8](#).

No `ProposalExecuted` or role-reset event was returned from GovernorV2 in this interval. The endpoint role readings and the emergency-proposer contract's unchanged owner do not establish completed control transfers corresponding to those submissions. A successful proposal-submission transaction demonstrates submission, not approval or execution of the proposal's contents.

This audit does not determine voting-token concentration, delegated voting power, the outcome of every oracle request, or control of every subsidiary contract. Those are distinct potential mechanisms of influence. It would be incorrect either to call the two submitted proposals proof of whole-UMA ownership, or to claim unchanged owner getters exclude every other form of governance influence.

Reproduction and interpretation boundary

Raw evidence is preserved in the new ignored batch `artifacts/parb-across-control-20260929/`. It includes historical deployment records; inherited/endpoint owner, role and Finder reads; bounded event queries; the two ownership receipts and their `trace_transaction` results; submission receipts; and explorer-verified

source responses. Files retain the exact RPC request and response. Expected absence of an ABI method is distinguished from a returned owner value; a receipt's success is distinguished from completion of a subsequently proposed governance action.

No write transaction, impersonation, message construction, simulated privileged call or configuration change was performed. The findings establish particular control changes and dependencies. They do not establish common human ownership, intent, or a minting capability absent from the controlled token's own contracts.

Appendix 4: Copied Arbitrum gateway mint inventory on PulseChain

Contents of this appendix

Findings	44
Pin, method and census boundary	44
Destination gateway inventory.....	45
ARB: direct bridgeMint, historically executed	46
GRT: verified issuance bounded by an accumulating allowance	46
LPT and HGOLD: explicit limits on confirmation	47
Named stablecoins and Aave receipt tokens	47
What token type does and does not establish	48
Reproduction records.....	48
Follow-up transaction audit	48

Source record: docs/studies/analysis/parb_bridge_mint_inventory_20260929.md.

Copied Arbitrum gateway mint inventory on PulseChain

Research date: 29 September 2026. This is a bounded, read-only census of existing router registrations, destination gateway implementations and token permissions. It interprets recorded transactions; it does not construct transactions or assume that a copied L1 contract proves the existence of a corresponding running L2.

Findings

There is more than one destination issuance model. **ARB has a confirmed direct bridgeMint integration and an observed post-fork mint. GRT has a different, authorized, allowance-limited mint integration and an observed post-fork mint.** LPT has a source-confirmed external-minter branch for escrow shortfalls, but this batch does not verify the current external minter's implementation. HGOLD has a registered gateway whose source calls bridgeMint, but its token source and current token-level permission remain unverified. WETH wraps native currency received by its gateway; this is another form of issuance with different backing.

These findings rule out an ARB-only account of the copied infrastructure. They do not establish generic issuance of existing stablecoins or Aave receipt tokens. In particular, the queried OUSD, eUSD, mUSD, FRAX and three Aave v3 receipt-token addresses select the standard gateway, whose inbound operation transfers existing L1 escrow. Routing to that gateway does not grant it the token's issuer role.

Pin, method and census boundary

All endpoint storage and view reads use block **27,659,393** (0x1a60c81), hash 0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275. The inherited snapshot boundary is block **17,232,999**. Event searches use the public PulseChain RPC's available history from block zero through the endpoint, including inherited Ethereum events, and distinguish post-fork events from that inherited history. Transaction observations additionally retain their own block hashes. No public state was changed.

Router 0x72ce9c846789fdb6fc1f34ac4ad25dd9ef7031ef returned **167 GatewaySet events covering 129 distinct L1 token addresses**. The last event per token was reconciled against the router's endpoint getGateway result. This produces 98 standard-gateway routes (including five default fallbacks), 20 custom-gateway routes, four disabled routes, and seven single-token specialized routes. There are **nine nonzero destination gateway addresses** in this census. Seven named assets were also queried, producing 136 endpoint token reads in total.

This is complete for the returned router event set, not for every ERC20 on PulseChain. The standard gateway's default makes unregistered addresses resolve to a gateway too. Such resolution does not prove a historical deposit, usable counterpart token, available escrow, or successful withdrawal. Direct gateway registrations that are never selected by this router, arbitrary future contracts, other routers and unrelated issuer mechanisms are outside this census. Full registration logs were additionally obtained for the ARB and HGOLD reverse-like gateways; each returned one registered token pair.

Implementation slots were read at the pinned endpoint rather than assuming the May 2023 implementation still applied. Verified explorer source was retrieved for each identified implementation or direct gateway. A zero EIP-1967 slot alone is not treated as proof that an arbitrary contract has no other indirection; source availability and observed call paths determine the narrower conclusions.

Destination gateway inventory

Gateway	Endpoint implementation / direct source	Registered token scope in router census	Incoming accounting
0xa3a7b6f88361f48403514059f1f16c8e78d60eec	0xb4299a1f5f26ff6a98b7ba35572290c359fde900, L1ERC20Gateway	98 effective routes	Transfer existing L1 escrow
0xcee284f754e854890e311e3280b767f80797180d	0xc8d26ab9e132c79140b3376a0ac7932e4680aa45, L1CustomGateway	20 routes	Transfer existing L1 escrow; custom registration is not reverse minting
0x01cdc91b0a9ba741903aa3699bf4ce31d6c5cc06	0xd41ca6a1d034d178c196dfa916f22f7d1a1b8222, L1GraphTokenGateway	GRT	Transfer escrow; mint an allowed shortfall into escrow first
0xd92023e9d9911199a6711321d1277285e6d4e2db	0x4b8e9b3f253e68837bf719997b1eeb9e8f1960e2, L1WethGateway	WETH	Deposit native currency into WETH, then transfer WETH
0xd3b5b60020504bc3489d6949d545893982ba3011	Direct verified L1DaiGateway	DAI	Transfer from designated L1 escrow
0x6142f1c8bbf02e6a6bd074e8d564c9a5420a0676	Direct verified L1LPTGateway	LPT	Transfer available escrow; call an external minter's bridgeMint for any shortfall
0x0f25c1dc2a9922304f2eac71dca9b07e310e8e5a	0xc4e3ff0b5b106f88fc64c43031be8b076ee9f21c, L1ERC20TokenGateway	wstETH	Transfer existing L1 token; token, counterpart and withdrawal checks apply
0xbbce8aa77782f13d4202a230d978f361b011db27	0x4e8c24617c4592895d52ac6df29d9647e9d01bb5, L1ForceOnlyReverseCustomGateway	ARB	Call the registered L1 token's bridgeMint

Gateway	Endpoint implementation / direct source	Registered token scope in router census	Incoming accounting
0x04bdbe0bc4edb62ea12070435ee67782ba292948	Direct verified CustomL1Gateway	HGOLD	Source calls the registered token's bridgeMint; token permission unverified

Primary source records are the explorer's verified contracts for the [standard gateway implementation](#), [custom gateway implementation](#), [GRT implementation](#), [WETH implementation](#), [DAI gateway](#), [LPT gateway](#), [wstETH implementation](#), [ARB implementation](#) and [HGOLD gateway](#). The saved API responses preserve the actual source retrieved, including inherited base contracts; the explorer links alone are not substitutes for the pinned reads.

ARB: direct bridgeMint, historically executed

L1 ARB is 0xb50721bcf8d664c30412cfbc6cf7a15145234ad1. The reverse gateway's single TokenSet event pairs it with L2 address 0x912ce59144191c1204e64559fe8253a0e49e6548 at block 16,840,312, [transaction 0x5cc80a28....](#) Router selection followed at block 16,840,532, [transaction 0x5c3e3b15....](#) Both are inherited registrations, not post-fork changes.

The token's verified implementation authorizes its configured `arbOneGateway` to call `bridgeMint` and `bridgeBurn`; the prior pinned audit verified that `getter` points to this reverse gateway. The successful post-fork mint in [transaction 0xeebae5bc...](#), block 21,469,490, is established by receipt, trace and supply change in [the bridge observations \(Appendix 5\)](#).

This proves exact-token destination issuance. The same study records malformed accepted source metadata and does not claim independently verified L2 escrow. Successful destination execution and economic source backing are separate facts.

GRT: verified issuance bounded by an accumulating allowance

L1 GRT is 0xc944e90c64b2c07662a292be6244bdf05cda44a7. Its specialized router registration is inherited block 16,149,151, [transaction 0x844a8a3a....](#) The gateway verifies the token identity and accepted counterpart. If an incoming amount exceeds escrow, `_mintFromL2` checks its accumulated allowance, increments `totalMintedFromL2`, calls `GraphToken.mint(escrow, shortfall)`, and then transfers the complete incoming amount from escrow to its recipient. This is a GRT-specific issuance mechanism, not the ARB `bridgeMint` token interface.

At the endpoint, `GraphToken.isMinter(gateway)` returns true and the gateway's `paused()` returns false. Its recorded state is:

Quantity	GRT units
Total minted through this allowance	62,875,936.675361278837595998
Cumulative allowance at block 27,659,393	64,230,816.8555
Difference between allowance and recorded minted amount	1,354,880.180138721162404002
Allowance accrual per block under current settings	6.0365

The allowance difference is an accounting ceiling at the pinned block, not a promise of executable future transfers or a dollar value. Escrow transfers, authentication and other token checks still apply. Future configuration is not assumed fixed.

Exactly one `TokensMintedFromL2` event was returned in the searched history: [transaction 0xf7ed5701...](#), block **27,434,949**, hash `0xabc7a6608cbb7515daaa1464f02cf94fc162b9f89c8c7a0a2859d140c575dd4bb`, **1 September 2026, 13:42:05 UTC**. The receipt succeeds. Its trace follows the outbox, bridge, GRT gateway and currently identified gateway implementation, then calls `GraphToken.mint` and `GraphToken.transferFrom`. The token emits a zero-address mint to escrow `0x36aff7001294dae4c2ed4fdefc478a00de77f090` for **62,875,936.675361278837595998 GRT**, matching the gateway event. The escrow then transfers the withdrawal amount to `0xa0b347117a878da9942fa5c31b87c773a2969913`.

Total supply between the preceding block and this block rises by precisely the same mint amount. The external transaction sender is `0xfbba783afda7b41ebb7473a914f0b6eb9f9139f3`. No common human ownership is inferred from these addresses. This batch proves destination issuance; it does not independently authenticate an L2 lock, source burn or source state history.

LPT and HGOLD: explicit limits on confirmation

LPT, `0x58b6a8a3302369daec383334672404ee733ab239`, selects its dedicated gateway through inherited block 14,207,632, [transaction 0x994651c7....](#) The gateway's verified `finalizeInboundTransfer` transfers available LPT from `l1LPTescrow`, then calls `minter.bridgeMint` for any shortfall. Endpoint getters identify `ESCROW 0x6a23f4940bd5ba117da261f98aae51a8bffa210a` and `minter 0x8dddb96cf36ac8860f1de5c7c4698fd499fab405`. The token's owner getter returns that same minter. Verified `LivepeerToken` source restricts its ordinary mint method to its owner and additionally checks that minting is not finished.

However, the configured minter source was unavailable in this batch. Its getter identifies controller `0xf96d54e490317c557a967abfa5d6e33006be69b3`, but that alone does not prove which gateway callers it accepts. This study therefore classifies LPT as an identified issuance branch with an unresolved current external-minter permission check, not a newly demonstrated successful bridge mint.

HGOLD (Hashgold) is `0x445164d9e038e1dae0989e4b6ac745b78d26f181`. The gateway's sole `TokenSet` event maps the same numeric address on both sides; router registration occurs in the same inherited block 16,971,420, [transaction 0x24371101....](#) The verified gateway overrides inbound escrow handling to invoke the token's `bridgeMint` and outbound handling to invoke `bridgeBurn`.

The token's endpoint name and symbol getters identify Hashgold/HGOLD, but the explorer returned no verified token source. Its EIP-1967 slot was zero, and speculative gateway getters reverted; neither result establishes issuer permissions. The zero-address Transfer search returned 85 inherited mint logs and no post-fork mint logs through the endpoint. Inspection of the last such inherited transaction shows a direct token call, not a bridge call. Those mint logs therefore do not prove a successful HGOLD `bridgeMint` integration. It remains a registered source-supported gateway path with unverified token-level behavior.

Named stablecoins and Aave receipt tokens

Exact L1 token queried	Address	Endpoint route
OUSD	<code>0x2a8e1e676ec238d8a992307b495b45b3feaa5e86</code>	Standard escrow gateway
mUSD	<code>0xe2f2a5c287993345a840db3b0845fbc70f5935a5</code>	Standard escrow gateway
eUSD	<code>0xa0d69e286b938e21cbf7e51d71f6a4c8918f482f</code>	Standard escrow gateway
FRAX	<code>0x853d955acef822db058eb8505911ed77f175b99e</code>	Standard escrow gateway

Exact L1 token queried	Address	Endpoint route
Ethereum Aave v3 aDAI	0x018008bfb33d285247a21d44e50697654f754e63	Standard escrow gateway
Ethereum Aave v3 aUSDC	0x98c23e9d8f34fefb1b7bd6a91b7ff122f4e16f5c	Standard escrow gateway
Ethereum Aave v3 aUSDT	0x23878914efe38d27c4d67ab83ed1b93a74d4086a	Standard escrow gateway

These are exact-address routing observations, not a census of all stablecoins, all Aave deployments, all Balancer wrappers, or every similarly named token. This batch does not inspect governance actions in Origin, Reserve or Morpho. Nor does it assess pool liquidity or establish that routing to a standard gateway makes a rebasing receipt safe to bridge. Aave Portal's pool-authorized issuance is a separate accounting mechanism; it is not conferred by router registration.

What token type does and does not establish

An ordinary transferable ERC20, including an ERC20 called a PRC20 on PulseChain, can be a custody asset for an appropriately implemented reverse bridge. ERC4626 vault shares also expose ERC20 balances and can represent source collateral that remains invested. The destination representation has the redemption rights its actual implementation grants. Rebasing receipts require compatible accounting; locking a rebasing balance does not automatically synchronize destination shares. ERC721 and ERC1155 require different custody and message semantics and are not made compatible with these ERC20 gateways by registration alone.

Three checks must remain distinct: **routing/registration**, **authenticated bridge execution**, and **the exact destination token's issuance permission**. A fourth is the economic question of independently evidenced source backing. The ARB and GRT destination examples answer the first three for particular recorded transactions. They do not replace the fourth or transfer permissions to unrelated stablecoins. The [historical bridge model study \(Appendix 13\)](#) explains native-L2 custody and L1 representation accounting in more detail.

Reproduction records

Raw observations are preserved under `ignored artifacts/parb-bridge-mint-inventory-20260929/`. The batch includes router event logs, the 129-token event inventory, 136 endpoint routing reads, reverse-gateway registrations, nine endpoint implementation reads and verified source responses, token permission reads, GRT mint logs/receipt/trace/block/supply comparison, and HGOLD mint-log and sampled-transaction observations. The manifest records the endpoint and file hashes. Earlier ARB transaction and token evidence remains in the separate historical bridge batches cited by the companion study.

This study's strongest new result is an actual **GRT-specific, authorized and allowance-limited destination issuance**, alongside ARB's reverse-token issuance. No evidence collected here makes the named stablecoin or Aave-token routes operate as those specialized minting integrations.

Follow-up transaction audit

The [GRT reconciliation \(Appendix 12\)](#) establishes that the September withdrawal combined 1 existing GRT with the recorded mint, and that its allowance parameters and minter permission were inherited. Its historical source design burns withdrawn GRT on L2. The [escrow receipt census \(Appendix 11\)](#) screens 102 additional withdrawal events across four gateway addresses and identifies a retired LPT gateway,

expanding historical gateway-address coverage to ten. The [specialized history \(Appendix 20\)](#) classifies the recorded LPT withdrawal as an escrow release, with no minter call and no block-to-block supply increase. Source capability and executed issuance remain separate findings.

Appendix 5: Copied Arbitrum bridge on PulseChain: recorded activity and token compatibility

Contents of this appendix

Findings	50
Evidence boundary	51
Recorded control changes	51
Recorded escrow withdrawals	52
ARB: an observed destination mint through the reverse gateway	52
Source-message provenance of the ARB mint	53
Public MIM transaction and historical call traces	54
Destination routing actually observed	55
What a reverse lock-and-mint bridge represents	55
Token standards and receipts	55
Interpretation and reproduction references	56

Source record: docs/studies/analysis/parb_bridge_observations_20260929.md.

Copied Arbitrum bridge on PulseChain: recorded activity and token compatibility

Research date: 29 September 2026. This study is retrospective and read-only. It examines existing state, logs, receipts and published contract source. It does not perform simulated privileged calls, impersonation or transactions.

Findings

The copied Arbitrum contracts have a demonstrable post-fork transaction history. The bridge proxy administrator changed owners twice, the custom gateway changed implementation and changed back, and the rollup recorded node confirmations. Several custom-gateway withdrawals have corresponding stablecoin transfers out of escrow in successful receipts. These observations establish activity and specific control changes. They do not identify the humans behind addresses or establish that the withdrawn assets originated from new native-L2 deposits.

The more specific claim about **ARB** has direct transaction support: on 21 September 2024, the copied ARB token minted to a recipient during a successful outbox/gateway transaction. The mint, withdrawal finalization and outbox execution occur in the same receipt. This proves the destination ARB mint occurred; the source escrow balance has not been independently established.

A native-L2 lock/L1 mint bridge is a real contract pattern. In the reviewed Arbitrum reverse-custom implementation, L2 tokens are transferred into escrow and the L1 gateway invokes the destination token's `bridgeMint`. This can represent transferable ERC-20-compatible tokens, including suitable vault shares. It is not a universal permission to mint every existing L1 token. The observed mappings for OUSD, mUSD, eUSD and FRAX point to the ordinary standard gateway, not a reverse-mint gateway.

Evidence boundary

All state conclusions use chain **369**, block **27,659,393**, hash

0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275. Historical log searches run from block **17,233,000** through that endpoint. This boundary follows the inherited Ethereum snapshot at block 17,232,999 reported by the [PulseChain mainnet repository](#). Thus these observations distinguish post-fork activity from inherited Ethereum logs. Block timestamps below are UTC.

The source is <https://rpc.pulsechain.com>; responses are saved under ignored artifacts/parb-bridge-observations-20260929/. The endpoint block hash was re-read and matched. Logs carry their own block/transaction hashes. Receipts for the specifically cited ownership changes, upgrades and withdrawals report success. This is one-RPC evidence, not independent consensus verification.

The named-contract search is bounded: router, standard/custom/DAI gateways, ARB gateway, proxy administrator and rollup were queried across the stated range. A query returning no logs does not prove that no relevant transaction occurred anywhere, that no unlogged storage mutation occurred, or that a distinct gateway does not exist. No actual pARB execution RPC or complete source deployment catalogue has been established. An L1 counterpart address is not proof of a running source contract at that address.

Recorded control changes

The bridge proxy administrator is 0x9ad46fac0cf7f790e5be05a0f15223935a0c0ada. Its two returned `OwnershipTransferred(address,address)` events show this sequence:

Time	Block	Observed change	Transaction
2025-02-24 15:50:45	22,794,544	0x3fffbadaf827559da092217e474760e2b2c3cedd → 0xb8698cfffdaaf26ca21fe8aa560c18cc2b2f5996	Receipt
2025-03-01 20:30:15	22,838,294	0xb8698cfffdaaf26ca21fe8aa560c18cc2b2f5996 → 0x353e1d055ae5352d1b8c9d3f0ffef1e3a4b96294	Receipt

The custom gateway 0xcee284f754e854890e311e3280b767f80797180d emitted two `Upgraded(address)` events on 1 March 2025:

Time	Block	Implementation in event	Transaction
20:10:55	22,838,182	0x0750201bf8875a6fdea2ff1a70743c58bdebaf2d	Receipt
20:29:25	22,838,289	0xc8d26ab9e132c79140b3376a0ac7932e4680aa45	Receipt

Both upgrade transactions have sender 0xb8698cfffdaaf26ca21fe8aa560c18cc2b2f5996. The second implementation is also the pinned endpoint's implementation. Consequently, reading only the endpoint would miss an intervening implementation change. The temporary implementation's complete behavior and all intermediate storage effects are not established by these events alone. They are evidence of an upgrade sequence, not evidence of a particular mint mechanism or beneficial/harmful intent.

The router returned no logs in the searched post-fork interval. Its owner and the custom gateway's owner at the endpoint remain `0x3ffffbadaf827559da092217e474760e2b2c3cedd`, a separate role from proxy-admin ownership. Treating these different authority layers as one owner would hide the observed distinction.

Recorded escrow withdrawals

Four named-stable withdrawals have both a custom-gateway `WithdrawalFinalized` event and a token `Transfer` from the gateway in the same successful receipt. Six decimal places are used for the copied USDC and USDT contracts.

Time	Block	Token units transferred from gateway	Recipient	Transaction
2023-05-27 22:41:15	17,375,533	1,310,883,939.04768 1 USDC	0xcb5cf20b3e413b6984 ae2c685a1ec151ad3736 e2	Receipt
2023-06-13 10:56:45	17,516,880	491,009,317.461778 USDT	0xcb5cf20b3e413b6984 ae2c685a1ec151ad3736 e2	Receipt
2024-08-14 19:11:25	21,150,681	6,000,000 USDC	0x42543853a72edb6f1f f9bddab0cf4676187ec2 8a	Receipt
2024-08-23 15:20:45	21,225,821	900,000 USDT	0x42543853a72edb6f1f f9bddab0cf4676187ec2 8a	Receipt

These are transfers from existing escrow, not zero-address mint events. Their token-unit sizes do not establish dollar value. The custom gateway's two other returned withdrawal events occur in [one transaction at block 27,434,939](#) and identify token addresses `0xc08512927d12348f6620a698105e1baac6ecd911` and `0x58b6a8a3302369daec383334672404ee733ab239`. Their source positions and token metadata have not been classified in this study.

The copied rollup returned **265** `NodeConfirmed` **events** during the searched interval. The last records node **5355**, block **27,434,764**, transaction `0x029d...83ea`, consistent with the endpoint's `latestConfirmed()` value. Confirmation is a specific L1 observation. The event does not disclose every source account or prove the thesis's asserted trillion-token deposits. Neither node confirmations nor the observed withdrawals, by themselves, demonstrate native-L2 receipt locking followed by minting an existing L1 OUSD, mUSD, eUSD or aToken.

ARB: an observed destination mint through the reverse gateway

The copied L1 ARB token is `0xb50721bcf8d664c30412cfbc6cf7a15145234ad1`. At the endpoint, the router selects gateway `0xbbce8aa77782f13d4202a230d978f361b011db27`, whose reported counterpart is `0xcad7828a19b363a2b44717afb1786b5196974d8e`. These are distinct from the standard and custom escrow gateways used by the named stablecoins.

The gateway's post-fork log query returns one `WithdrawalFinalized` event: [transaction 0xeebae5...1968a](#), block **21,469,490**, **2024-09-21 11:15:05 UTC**, block hash `0xbbc9b58dce7934e35e967f4b96f8810c8d47625ffe6490eb4b1cffdb195f60`. Its receipt reports success and contains, in order:

1. An outbox execution event identifying the ARB gateway and its counterpart.

2. An ARB Transfer from the zero address to `0x42543853a72edb6f1ff9bddab0cf4676187ec28a`.
3. A `WithdrawalFinalized` event emitted by the ARB gateway for the same token, recipient and amount.
4. A bridge call event naming the outbox and ARB gateway.

The transaction sender is the recipient above; its direct target is the copied outbox `0x0b9857ae2d4a3dbe74ffe1d7df045bb7f96e4840`. This receipt supports the claim that ARB was minted on PulseChain through the copied Arbitrum messaging and gateway path. It does not establish an independently observed source escrow deposit or prove that every destination claim is backed by a redeemable source position. This distinction remains necessary even when the L1 transaction succeeds.

The token reports 18 decimals. `totalSupply()` at block 21,469,489 is **6,903,225.677772492555339246 ARB**. At block 21,469,490 it is **972,406,600,390,402,296,078,733,787,224.655675065021335672 ARB**. The increase is exactly the receipt's zero-address transfer amount: **972,406,600,390,402,296,078,726,883,998.977902572465996426 ARB**. This reconciles an actual token supply increase, not merely an event bearing a bridge-related name. The size alone says nothing about redeemability or value. The preceding block's hash is `0x03b8e9e8a7329b97e1a050628b156a530e013f3259a05109bb15b8ef939c0e75`. The token implementation at the mint block matches the endpoint implementation identified below.

Published [L1ArbitrumToken source](#) contains `bridgeMint` and `bridgeBurn` restricted to `arb0neGateway`. This is an explicit token-level integration, explaining why the ARB example cannot be generalized to an arbitrary existing ERC-20 or existing Aave receipt. The destination token implementation read at the endpoint is `0xad0c361ef902a7d9851ca7dcc85535da2d3c6fc7`; the ARB gateway implementation is `0x4e8c24617c4592895d52ac6df29d9647e9d01bb5`. The token's `arb0neGateway()` getter matches the router-selected gateway. Explorer-verified implementations identify [L1ArbitrumToken](#) and [L1ForceOnlyReverseCustomGateway](#). The latter disables ordinary token self-registration. Its reverse-gateway inheritance implements destination minting, but its existence is not a permissionless mint service for every token standard or every pool coin.

Source-message provenance of the ARB mint

A further bounded audit decodes the metadata in the already recorded ARB transaction using the `executeTransaction` parameter order in the published [Outbox implementation](#). The transaction supplies counterpart sender `0xcad7828a19b363a2b44717afb1786b5196974d8e`, destination `0xbbce8aa77782f13d4202a230d978f361b011db27`, and message index 86. The successful historical trace's L2-sender getter agrees with that counterpart.

Its `l2Block`, `l1Block` and `l2Timestamp` fields all contain the identical integer **1072187306136427181113590999908369971006187625255**, whose hexadecimal representation is the destination gateway address. These fields therefore do not provide plausible source block numbers or a calendar timestamp. Comparing the raw integer numerically with the fork block or Unix time would misleadingly label the message as far-future; it cannot establish an actual post-fork source history. This differs from the MIM transaction below, whose metadata describes ordinary pre-fork block/time values.

The destination execution date, 21 September 2024, is independently established by its PulseChain block. The claim that the corresponding source deposit or lock occurred before or after the fork is **not established** by the ARB message's supplied metadata. A successful destination mint is not a substitute for a separately observed source lock.

The latest `NodeConfirmed` event in the existing rollup inventory before this mint is node **5253**, block **21,462,914**, [transaction 0x6127...dd00](#). This establishes temporal ordering only. The existing receipt and call trace do not directly name that node as the mint's message provenance, and this audit does not assert a root-to-message linkage from proximity. It has not reconstructed or replayed any message.

Decoded fields and the five nearest prior confirmation events are preserved separately in `artifacts/parb-arb-provenance-20260929/recorded_metadata.json`, referencing the previously saved transaction hash, block hash and raw input. This audit performed only interpretation of existing read-only evidence; it made no configuration changes or simulated calls.

Public MIM transaction and historical call traces

The supplied transaction `0xa2e5...227d` is successful at block **27,256,598, 2026-08-11 00:51:35 UTC**, hash `0x030fa534e9ffaa60decec678e56d76f627d84ad7dbfb545101d01fdc06620887`. Its sender and token recipient are the now-specified e121 wallet, `0x98c89980d1eae247ada3636ae27648f78e66e121`. The token `0x99d8a9c45b2eca8864373a26d1459e3dff1e17f3` reports symbol **MIM** and 18 decimals at this transaction's block. The receipt records **152.747067838559905107 MIM** transferred from the standard gateway `0xa3a7b6f88361f48403514059f1f16c8e78d60eec` to e121, alongside its withdrawal finalization and outbox execution events. The token transfer originates from the gateway, not the zero address.

The public RPC's `trace_transaction` method returned the actual historical execution trees for both this transaction and the ARB mint. These sharpen the receipt-only distinction:

Recorded transaction	Observed application call path	Token operation
MIM, 11 August 2026	e121 → outbox → bridge → standard gateway → MIM	Standard ERC-20 transfer, paid from existing gateway balance
ARB, 21 September 2024	0x4254...c28a → outbox → bridge → reverse gateway → ARB	bridgeMint, followed by token implementation execution and mint event

The MIM trace contains 15 frames including proxy delegation and authentication reads. Its gateway queries the inbox's bridge, the bridge's active outbox and the outbox's L2 sender; the latter returns counterpart `0x09e9222e96e7b4ae2a407b98d48e330053351eee`. The ARB trace contains 16 frames; its corresponding L2 sender read returns `0xcad7828a19b363a2b44717afb1786b5196974d8e`. The ARB token call is actually `bridgeMint`, not an inference based only on the transfer event's appearance. Neither transaction's call tree contains an Aave, Balancer or Curve call.

The existing MIM transaction's decoded outbox message metadata references L2 block **85,474,848**, L1 block **17,151,178**, and L2 timestamp **2023-04-29 10:32:06 UTC**. That metadata predates the inherited Ethereum snapshot. The transaction is therefore consistent with later execution of an inherited withdrawal message; it is not evidence of a new post-fork pARB deposit or staking position. The source block and its underlying receipt have not been independently obtained, so this conclusion does not promote the message's supplied metadata into a separately verified L2 history.

These two cases demonstrate different destination mechanics in actual records. MIM supplies an exact link between the named e121 wallet and the copied Arbitrum bridge. ARB supplies an exact existing-token mint through its dedicated reverse integration. Neither transaction, considered alone, establishes the broader Aave receipt → Balancer → DAI sequence or the source lock that would back every newly issued representation. No real-person attribution follows from either sender address.

The public Otter trace pages could not be read in this environment (HTTP 403); `debug_traceTransaction` was unavailable on the RPC. The successful `trace_transaction` responses provide the execution evidence instead. These provider limitations were not interpreted as empty or failed transactions. New raw results are kept separately under `artifacts/parb-bridge-leads-20260929/`, particularly `mim_parity_trace.json`, `arb_parity_trace.json`, the transaction/receipt pair, and MIM block/metadata reads. No live or locally simulated transaction was sent.

Destination routing actually observed

The router is `0x72ce9c846789fdb6fc1f34ac4ad25dd9ef7031ef`. Pinned `getGateway(token)` reads from the preceding study show:

Existing L1 asset	Selected gateway	Interpretation of identified route
DAI	<code>0xd3b5b60020504bc3489d6949d545893982ba3011</code>	Dedicated DAI gateway; not the generic reverse gateway
USDC, USDT	<code>0xcee284f754e854890e311e3280b767f80797180d</code>	Custom gateway; observed withdrawals transferred escrowed tokens
OUUSD, mUSD, eUSD, FRAX	<code>0xa3a7b6f88361f48403514059f1f16c8e78d60eec</code>	Standard ERC-20 escrow gateway
ARB	<code>0xbbce8aa77782f13d4202a230d978f361b011db27</code>	Separate gateway with recorded destination ARB mint

The standard gateway's balances of those four stablecoins were zero at the endpoint; the custom gateway's USDC balance was one USDC. A standard fallback mapping does not prove a source token has been deployed or registered, nor that a destination mint function exists. No positive reverse-gateway deployment and registration for these named existing pool coins has been demonstrated in this bounded investigation.

What a reverse lock-and-mint bridge represents

The historical [L2 reverse custom gateway](#) uses `safeTransferFrom` and measures the change in its own token balance. The historical [L1 reverse custom gateway](#) uses `bridgeMint` for the incoming destination representation and `bridgeBurn` for its outgoing return. The L2 return operation transfers escrowed tokens out. This establishes the lock/mint and burn/release direction from executable source. The files' header comments contain a contradictory direction description; the function bodies, rather than that comment, support this interpretation.

For ordinary non-rebasing shares, the relevant backing unit can be the locked **share**, rather than an amount of underlying cash released on L1. A minted L1 share representation can be backed by a locked L2 share without L1 underlying escrow. That fact still does not give it the local vault's share-token identity or local underlying redemption rights. A source share, a new destination representation, and an existing destination vault share are three distinct contracts unless an actual integration establishes the intended relationship.

The [L1 gateway authentication](#) checks both the bridge caller and the outbox-reported counterpart sender. Separately, the destination token must permit the gateway's mint/burn operations. Bridge authority and token issuer authority are different checks. A new representation may implement the required interface; an existing pool coin does not acquire it simply by sharing a name, address on another chain, or an ERC-20 interface.

Token standards and receipts

"PRC-20" in this context describes ERC-20-style tokens on PulseChain; the name does not add cross-chain issuance authority. The tokenized-vault standard relevant to the "4621" reference is **ERC-4626**. This study does not identify an ERC-4621 bridge class.

Token kind	Compatibility with the reviewed native-L2 escrow operation	What must not be inferred
Ordinary transferable ERC-20 / PRC-20	Can supply balance, allowance and transfer behavior used by the escrow	ERC-20 does not define bridgeMint, bridgeBurn or cross-chain identity
ERC-4626 vault shares	Shares implement ERC-20, so ordinary transferable shares can be escrowed as shares	A remote representation is not automatically a share in an existing L1 vault; local asset() and redemption are separate
Static Aave wrappers, cTokens, many y/yv vault shares	ERC-20-compatible share form is a potential escrow unit, subject to actual transfer behavior	Same symbols or share rates do not create mint authority over the exact existing destination receipt
Rebasing aTokens/OUUSD and similar balances	Transfers may be compatible, but escrow balances can change while claims remain outstanding	The reviewed deposit balance-delta measurement does not itself synchronize later rebases across chains
Fee-on-transfer tokens	Source deposit code measures actual received balance, rather than assuming requested amount arrived	Return-transfer fees, burn behavior and post-deposit rebases are not resolved by that one measurement
ERC-777-style tokens with ERC-20 compatibility	May expose the required ERC-20 methods, but transfer hooks add contract-specific behavior	Interface overlap alone does not establish suitability or deployed support
ERC-721 NFTs	Not handled by this fungible-token amount-based gateway	NFT ownership/token IDs require NFT-aware representation and custody semantics
ERC-1155 multi-token assets	Not handled by this ERC-20 gateway	Per-ID balances, batch operations and receiver requirements differ
Native PLS/ETH-like balances	Not ERC-20 assets; wrapped forms are distinct token contracts	The generic token gateway is not itself the native-currency bridge
Nontransferable receipts or debt positions	Cannot generally be escrowed using ordinary allowance/transfer semantics	A position's existence or valuation does not make it a transferable backing token

The primary standards are [ERC-20](#), [ERC-4626](#), [ERC-721](#), and [ERC-1155](#). ERC-4626 requires ERC-20 share behavior but permits nontransferable implementations to revert transfers; therefore even the ERC-4626 label alone is insufficient to demonstrate escrow compatibility. ERC-4626 asset/share conversions also do not imply immediate redemption liquidity or external-dollar backing.

Interpretation and reproduction references

The public trail supports the proposition that copied infrastructure continued to process recorded actions and underwent specific administrator and implementation changes, including the documented destination ARB mint. It presently does not connect those events into the asserted complete source-deposit → native receipt lock → exact L1 stablecoin mint → pool purchase sequence. The difference is evidentiary: the source contracts, locked positions and authenticated messages for that specific sequence have not been identified.

The returned logs are saved as `router_rpc_logs.json`, `custom_rpc_logs.json`, `proxyadmin_rpc_logs.json` and `rollup_rpc_logs.json`; each stores the exact request and response. `event_receipts.json` and `event_receipts_retry.json` store transaction, receipt, block and event-signature reads. Some initial requests timed out; the successful retries are kept separately rather than overwriting failures. Full event

inventories and source copies remain in ignored artifacts, while this note retains only material findings. The ARB case additionally uses `arb_gateway_logs.json`, `arb_mint_receipt.json`, `arb_mint_tx.json`, `arb_mint_block.json`, `arb_mint_supply_before.json`, `arb_mint_supply_after.json`, `arb_token_gateway.json` and the two verified implementation source responses. The unfiltered ARB mint-event query returned an unreadable response; the successful gateway log query and exact receipt establish the cited event without treating that failed query as an empty result. `standard_rpc_logs.json` contains 93 returned withdrawal events and `dai_gateway_rpc_logs.json` contains two. These wider event inventories have not each been promoted to a receipt-level interpretation in this bounded note.

Reproduction uses `eth_getLogs` with the stated address and inclusive block range, `eth_getTransactionReceipt` for the cited hashes, and `eth_getBlockByNumber` for each event block and the endpoint. Event signatures were checked with `web3_sha3`, not guessed from a display label. Endpoint mapping and balance calls are preserved in `artifacts/parb-validation-20260928/contracts.json` and can be repeated with the existing read-only `Research/scripts/probe_pulsechain_routes.py` scanner. No transaction-writing method was called in this study.

Appendix 6: Examined PulseChain addresses: established roles and bounded leads

Contents of this appendix

Address register	58
Transactions with strong protocol evidence.....	59
Additional leads and interpretation limits.....	59
Evidence preservation	59

Source record: docs/studies/analysis/parb_core_addresses_20260929.md.

Examined PulseChain addresses: established roles and bounded leads

Research date: 29 September 2026. All reads are retrospective. State uses PulseChain chain 369, block 27,659,393, hash 0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275.

Address register

Seven public addresses were examined. Six return empty runtime bytecode at the endpoint; the seventh is the previously identified copied Arbitrum rollup contract. Empty current code is an observation, not a claim about an address's entire historical behavior or owner.

Examined address	Established role or current evidence
0x98c89980d1eae247ada3636ae27648f78e66e121	Sender and recipient of the examined MIM escrow withdrawal; trace and receipt verified
0xfdbee32bcf68bb0ced033200373384a619c71967	Queued Aave proposals 219, 214 and 218; executed 218, producing an outbound Optimism governance message
0xc000000be68420c4be47ee7004d0c69dfe57fd34	Empty runtime code and nonzero account nonce; no protocol-control role established by this pass
0x48fc8b22ae5319786a8e25ef49b1b49942000000	Explorer's sampled recent transactions target 0x0c4576ca1c365868e162554af8e385dc3e7c66d9, whose published proxy source is named 0gvStakingProxy; actual staking action and authority require implementation-specific interpretation
0x6e3bf3c7bd91144d694621298cb8da370479a7bb	Sampled recent transactions include token-approval selectors; no administrator transfer or bridge-mint role established by these calls
0xbad9dff1026e59ea4c39cc3398aa5eb6f6f229ac	Queued Aave proposal 225; sampled later transactions interact with Reserve asset-adaptor contracts
0x5ef0d09d1e6204141b4d37530808ed19f60fba35	Copied Arbitrum rollup proxy; 265 returned post-fork node-confirmation events in the bridge census

The first, second, sixth and seventh entries connect directly to already examined bridge/governance evidence. The remaining addresses are useful leads, but this study does not promote their presence in a list into proof of common control. Account nonces include inherited activity; they are not counts of post-fork governance transactions.

Transactions with strong protocol evidence

The [e121 MIM withdrawal](#) transfers 152.747067838559905107 MIM from the standard gateway to e121. Its historical trace uses token `transfer`, and its accepted source-message metadata predates the fork. The [bridge study \(Appendix 5\)](#) records the full trace, exact addresses, dates and comparison with the distinct ARB `bridgeMint` transaction.

The [BAD9 proposal-225 queue](#) and [FdbEE proposal-218 execution](#) are successful public actions. Queue and execution entry points are publicly callable subject to proposal state and timelock rules. They do not alone prove that the submitter owns the executor or authored the proposal. Proposal 218 emits a message through copied Optimism contracts; receiving-chain execution is not established. See the [Aave study \(Appendix 2\)](#).

Additional leads and interpretation limits

The bounded explorer sample requested the latest 30 ordinary transactions per address, restricted to blocks 17,233,000–27,659,393. It is not a complete transaction, token-transfer or internal-call history. A returned empty list is not a conclusion that an address was inactive. Twelve selected transactions were also checked against successful RPC receipts and matching block headers; source classifications below refer to published explorer code, not an assumption that every proxy has retained the same implementation throughout history.

The `48Fc...` sample includes [a call to the OGV staking proxy](#). This introduces a concrete Origin-associated lead. It is not itself evidence of OUSD bridge minting or Origin governance ownership. The proxy's [published source](#) identifies the contract family; the implementation and specific action would need separate historical reconciliation.

BAD9's sampled transactions at block 26,768,729 target three contracts whose published names are `RTokenAsset`, `Asset` and `Asset`: `0x6e3b6b31c910253fef7314b4247823bf18d174d9`, `0xd5cc2875bbc53afbcc41bf04e7ba37f2894cbfa1`, and `0x2c312da96f98a5b7822270f69afd2d7ae8e748dc`. An [example receipt](#) anchors the lead. All three recorded inputs use selector `0xf8ac93e8`, matching `refresh()` in their published interfaces and a read-only signature hash check. Refreshing an asset adapter should not be described as minting an `RToken` without the corresponding issuance and collateral-transfer evidence.

The `FdbEE...` sample also includes calls to a published `SmartYield` contract at `0x4b8d90d68f26def303dcb6cfc9b63a1aaec15840`. These are additional protocol interactions, not evidence of an Arbitrum source deployment. No claim of a completed stablecoin bridge cycle follows from their coexistence with Aave transactions.

Evidence preservation

Read-only address code/nonce results and bounded explorer responses are in `artifacts/parb-core-addresses-20260929/address_reads.json`. Published target sources and selected receipt/header confirmations are preserved in that same ignored batch directory. These records complement the earlier frozen bridge and Aave event batches. No public transaction or local state-changing simulation was submitted during this review.

Appendix 7: Conditional cross-chain asset representation: May 2023 models

Contents of this appendix

Principal conclusion	60
Historical source boundary	60
Across V2: liquidity delivery, not a universal token minter.....	61
Replenishment on both chains and deposits remaining on L2.....	61
L2 capital supporting an L1 DeFi action.....	62
Observed control and L1 inventory	64
Omnibridge: a historical reverse representation model.....	64
Polygon PoS mintable assets: a distinct burn/mint example.....	65
A conditional proposition for the thesis	65
How the evidence strengthens the paper.....	65

Source record: docs/studies/analysis/parb_crosschain_plausibility_20260929.md.

Conditional cross-chain asset representation: May 2023 models

Research date: 29 September 2026. This study asks what the historical contract models permit under explicit assumptions. It does not require a presently available pARB RPC, assert that a source chain is operating, or submit transactions. Historical PulseChain observations remain separate evidence.

Principal conclusion

A source-native transferable token or vault receipt can remain locked on its origin network while an authorized representation is minted on a destination network. Destination underlying escrow is not a universal requirement. Historical Arbitrum reverse gateways and Omnibridge reverse-mode code provide concrete examples; Polygon's mintable-token predicate provides a related child-burn / root-mint model. These are distinct from ordinary return withdrawals that release pre-existing destination escrow.

This establishes a technically plausible building block for the thesis. It does not establish that the resulting representation equals an already existing destination aToken, OUSD, mUSD, eUSD or FRAX token. That identity and its issuance permission are separate contract properties. A destination pool must accept the actual token address received, and have the assets necessary for the contemplated output.

Historical source boundary

Sources were selected at or before **10 May 2023, 22:36:11 UTC**, the inherited Ethereum snapshot boundary. A repository revision before this date demonstrates availability of that code, not universal deployment of every feature. The [Arbitrum/Optimism historical study \(Appendix 13\)](#) adds direct inherited-block reads for ARB's existing reverse integration. Current documentation is used only for terminology or address discovery; later Across intents versions, Optimism Bedrock, Gnosis USDS/ transmuter changes, and later asset deployments are not retroactively inserted into the baseline.

Across V2: liquidity delivery, not a universal token minter

The pre-cutoff source revision is [a231a21df4ee58378d3f0fc8148daf5bc4791e3c](#), 5 May 2023. Its SpokePool accepts deposits on enabled routes and transfers the origin token into the spoke. Normal fast fills transfer destination tokens supplied by the relayer; supported slow-fill/refund flows use available protocol inventory. Hub/spoke accounting and optimistic bundle validation coordinate repayment and rebalancing. This lets funds become available before canonical settlement, but still requires destination capital.

The [historical SpokePool](#) uses destination-token transfers for fills. The [HubPool](#) mints its own LP shares when liquidity providers deposit L1 underlying. Those LP shares are not newly issued USDC, DAI, OUSD or Aave aTokens.

The [Arbitrum SpokePool](#) uses a mapped token counterpart and the Arbitrum gateway router for funds returned to the HubPool. Thus Across can be a transport/liquidity component of a broader system, but does not eliminate the underlying bridge's token rules. Controlling a settlement component does not cause an unrelated token contract to acquire a mint method or create absent destination cash.

Replenishment on both chains and deposits remaining on L2

Yes: Across V2's historical design can replenish spoke inventory as well as L1 hub inventory. Source-chain user deposits add tokens to that chain's SpokePool. Positive hub-to-spoke rebalance amounts deliver tokens from the HubPool through a configured adapter. In the other direction, relayer-refund settlement can include an amount to return through the chain-specific bridge. Ordinary L1 LP deposits also replenish the HubPool. The source contracts show both directions; this is not an inference from modern Across documentation.

A normal fast fill uses a relayer's destination assets. Thus the recipient can receive existing L1 tokens while the original deposit remains in the L2 SpokePool awaiting settlement. The relayer selects a repayment chain, subject to the supported route and valid settlement accounting. Repayment on L2 can leave the assets on that network, but transfers them out of the spoke to the relayer. Repayment on L1 consumes L1 inventory instead. Neither operation creates an unrestricted new stablecoin balance from a remote deposit alone.

Ignoring fees, a conceptual 100-unit example illustrates the first case:

Stage	L2 SpokePool	Relayer's L1 inventory	Recipient's L1 inventory	Relayer's L2 inventory
Before deposit	0	100	0	0
Source user deposits 100	100	100	0	0
Relayer fills on L1	100	0	100	0
Valid refund pays relayer on L2	0	0	100	100

The source user spent 100 on L2; the recipient received 100 on L1; the relayer exchanged the location of its inventory. No canonical bridge transfer of those 100 units is necessary in this abstract case if the supported settlement route permits the L2 refund. This is a conditional liquidity exchange, not a record of an observed PulseChain bundle.

Across's bundle structure includes net-send amounts and running balances rather than requiring every individual deposit to move through a canonical bridge immediately. Opposing flows and local refund needs can therefore reduce gross cross-chain rebalancing. A continuously replenished spoke may keep a positive balance while L1 recipients continue receiving fills. A persistent pool balance does not mean the same deposit remains exclusively reserved for its original depositor: obligations are pooled and settled across flows.

For a one-direction stream, finite L1 relayer/protocol inventory eventually needs replacement. Replenishing only the L2 spoke does not itself replenish L1 cash. Continued L1 delivery needs incoming L1 funds, offsetting business, completed bridge returns or another explicitly funded arrangement. If an ordinary canonical return is blocked by missing L1 escrow, Across's advance does not remove that settlement problem. Local L2 refunds can change which party bears inventory exposure, but they do not grant destination mint authority.

This is the precise sense in which a “reverse lock and release” analogy is useful: source custody can coexist with destination payment funded elsewhere. It is not a generic permanent lock/L1-representation-mint function. In the historical code, source `deposit` does not automatically mint L1 LP shares. The HubPool's ordinary LP issuance requires an L1 underlying deposit, and receipt valuation involving utilized reserves should not be confused with immediately spendable L1 cash.

Source anchors are the pinned HubPool's root-bundle rebalance logic, the pinned SpokePool's fill/refund logic, and the Arbitrum spoke's canonical-return adapter, linked above. These are contract capability statements, not claims that all corresponding copied routes, actors and balances are currently operational.

L2 capital supporting an L1 DeFi action

Across can economically connect a source deposit to a destination DeFi action: The recipient spends the destination tokens provided by a relayer or an inventory-funded slow fill. Compatible local calls can be composed in one L1 transaction. The source deposit, destination execution and later settlement remain separate transactions across chains. Local atomic execution does not make the complete cross-chain transaction atomic.

For example, assuming a supported route and ignoring fees, a 100-unit L2 deposit can support a relayer delivering 100 existing units on L1. A recipient integration could supply those units to Aave, receiving aTokens, or exchange them in a Curve pool. Aave must accept that exact underlying and permit supply; Curve must accept that exact input and have available output liquidity. The L2 deposit remains part of the bridge's settlement obligations. These are conditional applications of ordinary funded transfers, not evidence of a historically executed PulseChain route or a claim of profitable arbitrage.

There is a material historical deployment distinction:

Evidence	Finding	Implication
May 5 repository <code>SpokePool.sol</code>	Funded fill transfers tokens, then calls <code>handleAcrossMessage</code> on a contract recipient when a message is present	This source supports destination receive-and-execute composition
Same revision's <code>deployments/mainnet/Ethereum_SpokePool.json</code>	Recorded deployment has an older <code>deposit/fillRelay</code> ABI without a message argument	Repository capability cannot be assumed to exist in that deployed contract

Evidence	Finding	Implication
Copied Ethereum spoke 0x4d9079bb4165aeb4084c526a32695dcfd2f77381	Explorer-published source has ordinary transfers and no recipient message callback; runtime bytecode identical at inherited block 17,232,999 and endpoint 27,659,393	The inspected copied spoke does not establish the proposed native callback route
HubPool-configured spoke 0x5c7bcd6e7de5423a257d81b442095a1a6ced35c5, inherited implementation 0xa667498f46457548f1d3ad557340b95fdb290148	Verified source contains <code>handleAcrossMessage</code> and message-bearing fills	Native callback capability existed in this proxy's fork baseline
Configured spoke's endpoint implementation 0x788e49c4e2b3414777d6671d7411951e81d35808	A post-fork upgrade is recorded; replacement source is not explorer-verified	Complete current callback and application behavior is not established by the inherited source

The configured-spoke discovery corrects the earlier overextension of the older spoke's limitation. The first deployment inspected was not the spoke selected by the sampled HubPool mapping. Source-backed conclusions must follow the exact proxy and implementation at the specified block. See the [configured-spoke audit \(Appendix 14\)](#) and the [inherited implementation source](#).

The deployed-spoke finding uses the [historical deployment artifact](#) and [explorer-published contract source](#). The published contract is a directly deployed, constructor-based contract; its sampled EIP-1967 implementation slots are zero. The complete runtime is 17,282 bytes at both comparison blocks, with SHA-256 `9a42b4065ba8533d92f7de422d2fe944a609444e543dbb4bb3a4445ff4b9e4b9`. This is an endpoint comparison and source inspection, not an independent compiler reproduction or an exhaustive discovery of additional spoke deployments.

The proposed source's callback follows the funded transfer and is not wrapped in an outer `try/catch`. If its failure propagates, the destination transaction reverts, including its transfer; receiver logic could instead handle an inner failure itself. An ordinary ERC-20 transfer in the older deployed spoke does not automatically invoke a receiving contract's arbitrary DeFi logic. An additional authorized L1 coordinator could compose compatible fill and receiver calls, but no such deployed integration or historical execution is established by this study. A native callback and external transaction composition are different integration cases.

Further inspection of the published deployed source resolves another possible shortcut: its inherited `MultiCaller.multicall` uses `address(this).delegatecall` for each item. It batches methods on the spoke itself; it is not an arbitrary external-target executor for Aave or Curve. Its presence therefore does not replace a recipient callback or the separate authorized L1 integration described above. The ordinary `fillRelay` is public and is not limited to externally owned accounts, so the reviewed interface does not categorically exclude composition by a contract. That compatibility is narrower than proof that a specific funded coordinator exists or that an entire route has executed successfully.

Consequently, local arbitrage can be atomic if the local contracts and funding permit it; a profitable cross-chain arbitrage round trip is not guaranteed. Any L1 flash loan must be repaid using funds available before that L1 transaction ends. A future L2 reimbursement cannot itself repay it inside that transaction. The research does not assume an oracle quote or a remote balance makes absent L1 tokens available for transfer.

Observed control and L1 inventory

The [Across/UMA control audit \(Appendix 3\)](#) establishes a HubPool ownership transfer to `0xfeca...d140` on 19 June 2026 and an Across OptimisticGovernor module ownership transfer to `e121` on 19 August 2026. These are relevant governance changes. The sampled UMA infrastructure did not show corresponding ownership transfers, and common human ownership remains unproved. Administrative control alone does not supply destination tokens.

At the fixed endpoint, the HubPool's actual copied DAI, USDC and USDT balances and its stored liquid reserves are all zero. Its stored utilized reserves are respectively 1,656,490.776260091143953778 DAI, 16,210,612.259813 USDC and 500,442.221859 USDT. Utilized-reserve accounting is not immediately spendable cash or independent proof that remote receivables are collectible. These reads do not establish the balances of all spokes, external relayers or prospective funders, and do not rule out funded delivery from those other sources.

Reproduction evidence is preserved in the ignored batch `artifacts/parb-across-accounting-20260929/`: historical deployment artifacts, `reserve_reads.json`, `lp_inventory.json`, `spoke_implementations.json`, `spoke_17232999.json`, `spoke_27659393.json`, and `spoke_source.json`. State reads use the same single public RPC and fixed endpoint as the main paper. No state-changing transactions or route simulations were submitted.

Omnibridge: a historical reverse representation model

Pre-cutoff source revision `c814f686487c50462b132b9691fd77cc2de237d3`, 6 September 2021 contains reverse-mode support in both Home and Foreign mediators. Gnosis is a separate chain, not an Ethereum rollup; Omnibridge uses the Arbitrary Message Bridge's authentication model. The same representation analysis can still be applied to a hypothetical copied counterpart.

The [shared historical mediator logic](#) distinguishes source-native assets from already-bridged representations. A source-native deposit increases accounted escrow and requests a mapped representation on the other side. A representation returning to its origin is burned, allowing origin escrow to be released. The destination can deploy a bridge-controlled token representation when none exists. Consequently, a token originating on the Gnosis side can conceptually be escrowed there and represented on the Ethereum/PulseChain side without pre-existing L1 underlying escrow, where the reverse-mode mediator configuration supports it.

The [Foreign mediator](#) retains execution-limit checks and uses the shared distinction between release and mint. A custom existing representation must support the expected authorized mint/burn behavior. Standard representation tokens are ERC-677-compatible, which includes ordinary ERC-20 transfer behavior. This compatibility does not automatically map a newly created representation into an existing Curve or Balancer pool.

Fixed-balance vault shares provide clearer escrow units than raw rebasing balances. Interest, transfer fees, rebases, rounding and redemption restrictions still need asset-specific accounting. This study establishes the historical model; it does not establish the exact reverse-mode activation, mapping or issuer authority for every copied PulseChain token.

The separate xDAI bridge must not be conflated with this generic model. Its DAI-origin return path releases DAI backing; it is not a general right to mint arbitrary destination DAI. Current Gnosis documentation contains later changes that do not form part of this May 2023 analysis.

Polygon PoS mintable assets: a distinct burn/mint example

The pre-cutoff revision [ece4e54546a4e075f3a03b2699bc6bd92a5bc065](#), 3 April 2023 contains `MintableERC20Predicate`. Its authenticated exit model processes a child-token burn and, if root-token inventory is insufficient, invokes the root token's authorized mint before paying the withdrawal. Ordinary root-origin ERC-20 escrow predicates are a different path. Polygon PoS is also not interchangeable with an optimistic-rollup security model.

This is another historical example where an authorized destination issuance path need not start with enough root escrow. The root token must actually be mintable by that predicate and have the appropriate mapping. Source burn, message/exit validation and destination issuance remain necessary accounting links. The existence of a generic mintable predicate does not make all existing DAI or USDC contracts mintable by it. [Historical predicate source](#).

A conditional proposition for the thesis

Let a source protocol issue q transferable receipt shares after a valid local deposit. Assume the shares are genuinely locked, the bridge recognizes that lock under its specified trust model, destination claims are issued only once, and redemption extinguishes the corresponding claim before releasing those shares. The bridge can then issue a representation backed by those source shares while the underlying remains in the source protocol. No second L1 copy of that underlying is logically necessary for this representation model.

If the issued destination token is the exact token accepted by a destination pool, and that pool's rules and available balances permit output $y > 0$ DAI, then trading that representation for y DAI is mechanically possible. The counterparty/pool receives the remote-backed claim and the trader receives existing DAI. This is exchange of claims against an existing inventory; it is not proof that a new externally redeemable dollar was created.

For an ordinary L1 Aave `aToken`, Portal supplies a different accounting link: it creates the destination reserve's `aToken` while recording outstanding unbacked issuance. Until destination underlying arrives, source escrow alone does not remove that local unbacked balance. A wrapper can hold the correct ordinary `aToken`, but wrapper compatibility does not establish completed Portal settlement. These two accounting models should not be merged into one claim of universally fully backed L1 Aave liquidity.

This proposition supports **conditional possibility**, including purchases of `pDAI` when the stated acceptance and liquidity conditions hold. It does not prove the required token integrations already exist, identify an operator, or guarantee scale, profitability, permanent liquidity replenishment or a \$1 external-value peg.

How the evidence strengthens the paper

The appropriate framework has three categories: historical mechanism available, observed copied deployment or transaction, and explicit scenario assumption. A missing source RPC belongs in the third category for source execution; it is not a reason to reject the mathematical possibility. Conversely, broad administrator control cannot be used to quietly replace a missing mechanism with arbitrary hypothetical code and call it an inherited capability.

The strong theoretical case specifies the receipt actually locked, the exact representation actually minted, its existing pool acceptance, conserved share accounting, and finite output inventory. The audit already contributes actual ARB destination minting, Aave wrapper acceptance, `e121` `mUSD`-to-`DAI` purchases, and examined-wallet repayments. These support individual links without pretending they constitute one historically verified end-to-end transaction.

Source snapshots and cutoff metadata are preserved in ignored `artifacts/parb-historical-models-20260929/`. No new contract deployments, transactions, privileged simulations or transaction payloads were produced.

Inherited-block presence reads also found code at the copied Foreign Omnibridge proxy `0x88ad09518695c6c3712ac10a214be5109a655671`, whose implementation getter returned `0x8eb3b7d8498a6716904577b2579e1c313d48e347`, and the Polygon mintable predicate proxy `0x9923263fa127b3d1484cfd649df8f1831c2a74e4`, returning `0x0f92d459b20d21f6bf9e02056ea9165d3f78ba62`. The explorer publishes `ForeignOmnibridge` source for the former implementation, including representation handling. It did not return verified source for the latter implementation. These reads establish inherited L1 infrastructure presence, not every source counterpart, asset mapping or effective permission. Records are in `copied_bridge_presence.json` and the `implementation-source` responses.

Appendix 8: Four copied Curve metapools: bounded historical configuration review

Contents of this appendix

Configuration-event finding	67
Implementation and coverage limits	67
Examined-wallet exchange and liquidity history	68
e121: actual mUSD-to-DAI and FRAX-to-USDT transfers.....	68
Reproduction	69

Source record: docs/studies/analysis/parb_curve_history_20260929.md.

Four copied Curve metapools: bounded historical configuration review

Research date: 29 September 2026. Read-only evidence on PulseChain chain 369, blocks 17,233,000–27,659,393. Endpoint hash: 0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275.

Configuration-event finding

No selected post-fork amplification, fee or pool-administrator events were returned for these four named pools. The collection completed without RPC query errors, and its endpoint block hash matched on recheck.

Pool	Address
OUSD / 3CRV	0x87650d7bbfc3a9f10587d7778206671719d9910d
mUSD / 3CRV	0x8474ddb9e98f5aa3179b3b3f5942d724afcdec9f6
eUSD / FRAXBP	0xaeda92e6a3b1028edc139a4ae56ec881f3064d4f
FRAX / 3CRV	0xd632f22692fac7611d2aa1c0d552930d43caed3b

The selected signatures were RampA(uint256,uint256,uint256,uint256), StopRampA(uint256,uint256), CommitNewFee(uint256,uint256,uint256), NewFee(uint256,uint256), CommitNewAdmin(uint256,address) and NewAdmin(address). The result does not establish that no economic rebalancing occurred: swaps and liquidity additions/removals can change balances without any of these configuration events. It also does not exclude changes in upstream factory ownership, token implementations, base pools or unlogged storage.

Implementation and coverage limits

At the endpoint, OUSD/3CRV and FRAX/3CRV contain minimal-proxy bytecode pointing to 0x5f890841f657d90e081babdb532a05996af79fe6. The [published implementation ABI](#) contains all six searched configuration-event signatures and the standard exchange/liquidity event families used in the wallet check.

eUSD/FRAXBP points to 0x33bb0e62d5e8c688e645dd46dfb48cd613250067; the explorer did not provide a verified ABI for that implementation. The mUSD pool contains full runtime bytecode, but its ABI request likewise returned no ABI. Accordingly, queries of those two addresses are signature-family searches, not a claim that all their possible configuration-event interfaces were audited. An unavailable explorer ABI is not evidence that a pool does not function.

Examined-wallet exchange and liquidity history

A separate indexed-actor query found **820 events in 820 successful transactions**, involving **five** examined wallets. Every matched event was checked for exact inclusion in its successful receipt. An additional sample of 19 earliest/latest transactions per pool/actor was reconciled with originating transactions and block headers. The selected event families were the standard `TokenExchange`, `TokenExchangeUnderlying`, `AddLiquidity`, `RemoveLiquidity`, `RemoveLiquidityOne` and `RemoveLiquidityImbalance` signatures. Counts are:

Pool	Matched events
OUSD / 3CRV	19
mUSD / 3CRV	17
eUSD / FRAXBP	2
FRAX / 3CRV	782

Across these pools, the actors are c000 (690 events), 48Fc (116), e121 (11), BAD9 (2), and 6e3B (1). Full addresses are in the [address register \(Appendix 6\)](#). No selected indexed-actor matches were returned for FdbEE or the rollup address. Of the 820 events, 601 are underlying exchanges, 209 direct pool-coin exchanges, six one-coin liquidity removals and four liquidity additions.

This is direct evidence of participation in the named markets, even though the selected configuration-event search returned zero. It is not a complete wallet activity inventory: a router or intermediary may appear as the event's indexed buyer/provider instead of the examined address. Missing such activity is a known limitation. Conversely, participation does not alone identify common ownership, coordinated intent or native L2 funding.

e121: actual mUSD-to-DAI and FRAX-to-USDT transfers

At block **27,264,036**, e121 exchanged **13,489.157023144354307302 mUSD** for **564.501482257993107424 DAI**. The successful receipt includes both the mUSD transfer from e121 to the mUSD pool and the exact DAI transfer from that pool to e121. This establishes an actual stablecoin-to-pDAI purchase by the named wallet, rather than merely a quote or a theoretical pool connection. [Transaction](#).

At block **27,068,777**, the same wallet exchanged **150,000 FRAX** for **154,598.175347 USDT**. The receipt's FRAX input and USDT output transfers match the exchange event. [Transaction](#). Five FRAX-to-USDT exchange events from that wallet over blocks 27,068,777–27,068,922 total **731,000 FRAX input** and **753,387.520878 USDT output**. These are gross flows in copied-token units, not a dollar valuation or a complete net inventory change. A sixth nearby FRAX-pool event runs in the other direction, USDC into FRAX.

The economic source of the spent mUSD and FRAX has not been reconciled to an L2 deposit or authorized bridge mint. These receipts prove the destination trading leg; they do not close the source-to-destination causal chain.

Reproduction

The maintained manifest is `Research/experiments/parb_curve_history_20260929.json`, run by `Research/scripts/collect_pulsechain_events.py`. Results, endpoint bytecode and source/ABI responses are preserved under `ignored artifacts/parb-curve-history-20260929/`. This is a bounded public-history study; no calls that change chain state or hypothetical privileged simulations were performed.

The full wallet event/receipt collection is `wallet_events.json`; the separate saved log index is `wallet_event_index.json`, and sampled header checks are in `representative_wallet_receipts.json`. Both event collections agree on the 820 matches. The additional index was saved while full receipt verification was in progress; the original verification subsequently completed successfully.

Appendix 9: Copied Aave debt repayment: bounded public transaction audit

Contents of this appendix

Findings	70
Scope and method	70
Aggregate results by asset	71
Representative receipts and debt reconciliation.....	72
Why a large debt-token Mint can accompany a tiny repayment.....	72
Links to the seven examined addresses.....	72
Interpretation limits.....	73
Evidence preservation and reproduction	74

Source record: docs/studies/analysis/parb_debt_history_20260929.md.

Copied Aave debt repayment: bounded public transaction audit

Research date: 29 September 2026. This study reads public historical logs, receipts, block headers, reserve metadata and verified source. It submits no transactions and performs no local simulations.

Findings

There are genuine post-fork Aave V3 repayments, including material copied-USDT and copied-USDC payments by the examined addresses. Across the examined pool and interval, standard Repay events report **2,925,550.790450 USDT** and **407,302.146331 USDC**. Representative large repayments have matching underlying transfers into the reserve's aToken contract and reconciled debt-token events. Three examined wallets occur as indexed repayers. Some payments explicitly reduce another examined wallet's debt.

The same audit finds only **444 raw wei of DAI**, or **0.000000000000000444 DAI**, in four standard DAI repayment events. One such 111-wei payment triggers an approximately 4.805-million-DAI debt-token Mint event reflecting accrued interest. That large accounting event is not a payment of 4.805 million DAI. The distinction is material when interpreting explorer transfer views.

No standard Aave V2 Repay events were returned over the same interval. Its implementation was replaced after the fork with explorer-unverified code, so this finding cannot exclude a nonstandard debt adjustment at that address. The V3 totals measure gross recorded repayment, not net system debt decline, retained reserve cash, dollar value or a proven bridge-funded purchase loop.

Scope and method

The pools are copied Aave V2 `0x7d2768de32b0b80b7a3454c06bdac94a69ddc7a9` and copied Aave V3 `0x87870bca3f3fd6335c3f4ce8392d69350b4fa4e2`, on PulseChain chain 369. The explicit post-fork range is **17,233,000-27,659,393**. The ending block hash is `0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275`, rechecked after collection. The ending block timestamp is **28 September 2026 08:22:35 UTC**. This is not a search of other Aave deployments or all token transfers on PulseChain.

The verified original V2 and V3 implementations establish different signatures:

- V2: `Repay(address,address,address,uint256)`.
- V3: `Repay(address,address,address,uint256,bool)`.

The three indexed addresses are reserve underlying, debt beneficiary and repayer. V3's final field indicates repayment using aTokens. The direct `eth_getLogs` requests succeeded without returned errors: **zero V2 events, and 383 V3 events across 71 distinct transactions**. All returned V3 events have `useATokens = false`. Token decimals and symbols were read at the pinned block. For a representative largest event in each asset, historical `getReserveData` at that event's block identifies the actual aToken and debt-token addresses. The associated receipt, transaction and block header were retrieved and checked. [Original V2 implementation source](#), [V3 implementation and dependency source](#).

Aggregate results by asset

Amounts are whole units of the indicated copied token, not external-dollar valuations. They must not be added across assets as though each unit had the same value.

Underlying asset	Decimals	Repay events	Gross amount	Largest event
USDT	6	21	2,925,550.790450	290,000
USDC	6	17	407,302.146331	100,000
DAI	18	4	0.000000000000000004 44	0.000000000000000001 11
WETH	18	325	86.003562721157877 676	75.003562721157874 144
rETH	18	4	160.26234508299442 1006	84.082619990302430 868
cbETH	18	12	373.31381001062072 2431	55

Token addresses are respectively:

- USDT: `0xdac17f958d2ee523a2206206994597c13d831ec7`.
- USDC: `0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48`.
- DAI: `0x6b175474e89094c44da98b954eedeac495271d0f`.
- WETH: `0xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2`.
- rETH: `0xae78736cd615f374d3085123a210448e74fc6393`.
- cbETH: `0xbe9895146f7af43049ca1c1ae358b0541ea49704`.

The event count needs context: **321 of the 325 WETH events are eleven wei each**, and another is one wei. The remaining WETH payments are 10, 1 and 75.003562721157874144 whole tokens. A high count therefore does not imply a similarly large paid amount. The returned events span **9 July 2023-25 November 2025**; no standard V3 repayment event occurs in the searched later interval through the principal block. The observed payments consequently do not show a new repayment wave following the August 2026 stkAAVE upgrade.

Representative receipts and debt reconciliation

All six representative receipts are successful, contain the queried repayment log at the expected block hash, and contain an underlying transfer of the exact repaid amount from the indexed repayer to the historically identified aToken contract.

Asset	UTC date	Amount	Receipt
USDT	19 October 2023	290,000	Transaction
USDC	19 October 2023	100,000	Transaction
DAI	26 August 2024	111 wei	Transaction
WETH	25 November 2025	75.003562721157874144	Transaction
rETH	22 March 2025	84.082619990302430868	Transaction
cbETH	8 January 2024	55	Transaction

V3's verified `BorrowLogic.executeRepay` calls the relevant debt token's `burn`, updates reserve accounting, and, for these underlying-funded payments, transfers underlying to the aToken contract before emitting `Repay`. Debt-token event values reflect interest crystallization as well as the current payment. In the representative USDT receipt, the variable-debt `Burn` value is **289,853.874610 USDT**, with `balanceIncrease` **146.125390 USDT**. Their sum is the actual **290,000 USDT** paid. Similarly, the representative USDC `burn` value **99,949.802554** plus accrued balance increase **50.197446** equals the **100,000 USDC** payment. The WETH, rETH and cbETH representatives reconcile in the same way. This establishes more than an isolated ERC-20 transfer label. [Verified repayment and scaled-balance interfaces](#).

Why a large debt-token Mint can accompany a tiny repayment

In the DAI representative, examined wallet `0x48fc8b22ae5319786a8e25ef49b1b49942000000` pays **111 raw wei DAI** on behalf of examined wallet `0x6e3bf3c7bd91144d694621298cb8da370479a7bb`. The underlying DAI transfer into aDAI is exactly 111 wei. The variable-debt token at `0xcfc8d0c70c850859266f5c338b38f9d663181c314` emits a `Mint` with:

```
Accrued balance increase: 4,805,376.907763045679338591 DAI
Debt-token Mint value:    4,805,376.907763045679338480 DAI
Difference:                0.000000000000000111 DAI
```

The interface explicitly documents that a `burn` call can emit a `Mint` when accrued interest exceeds the amount repaid. The receipt exactly reconciles to that case. The approximately 4.805-million amount is crystallized debt interest net of a tiny payment; it is not 4.805 million underlying DAI transferred or debt paid off. Counting debt-token `Mint`/`Burn`/`Transfer` values as stablecoin repayments without this reconciliation would misclassify the transaction. [Receipt](#).

Links to the seven examined addresses

Three examined addresses occur in the indexed repayment fields. Totals below group by the event's **repayer**, not the top-level transaction sender and not the debt beneficiary:

Examined repayer	Asset	Events	Gross amount
0x6e3bf3c7bd91144d69462129 8cb8da370479a7bb	USDT	9	2,161,000
Same address	USDC	16	380,000
0xc000000be68420c4be47ee70 04d0c69dfe57fd34	USDT	9	705,000
0x48fc8b22ae5319786a8e25ef 49b1b49942000000	cbETH	3	42.014212052571746658
Same address	WETH	4	1.0000000000000000033
Same address	USDT	1	10
Same address	DAI	2	0.0000000000000000222

One receipt confirms c000... repaying **200,000 USDT** on 19 October 2023. Another confirms 6e3b... paying **1,000 USDT on behalf of c000...** on the same date. A third confirms 48fc... paying **10 USDT on behalf of c000...** on 16 August 2024. These are specific on-chain financial relationships, stronger than merely interacting with the same application. They do not establish common private-key ownership or a named person's identity. [c000 payment](#), [6e3b on-behalf payment](#), [48fc on-behalf payment](#).

Viewed by beneficiary, c000 has **706,010 USDT** repaid, including its own 705,000 plus the two payments above. Wallet 6e3b has **2,160,000 USDT**, **380,000 USDC** and **444 wei DAI** repaid on its debt positions. Beneficiary and payer totals describe the same events from different perspectives and must not be added together.

No indexed repayer or beneficiary matches were found for examined e121

0x98c89980d1eae247ada3636ae27648f78e66e121, FdbEE32 0xfdbee32bcf68bb0ced033200373384a619c71967, BAD9 0xbad9dfff1026e59ea4c39cc3398aa5eb6f6f229ac, or the copied rollup 0x5ef0d09d1e6204141b4d37530808ed19f60fba35. This limited result does not exclude funding, contract-mediated or unrelated protocol activity by them.

Interpretation limits

The observed payments validate part of the debt-repayment claim, but not a claim that large quantities of DAI have already been paid into these pools. Gross repayments do not equal the decrease in outstanding debt between two dates: new borrowing, accrued interest, liquidations, debt-mode conversions and other debt changes must also be reconciled. Nor do gross repayments equal cash permanently retained by Aave; other actions may withdraw or borrow the underlying later, including within the same transaction. The rETH representative, for example, contains additional underlying inflows and outflows, so its repay amount cannot be treated as the transaction's net reserve-cash increase.

The absence of standard V2 repayment events is especially bounded. The original V2 implementation at the start of the interval is explorer-verified, but the two July 2023 replacement implementations are not. Their upgrade history is documented in the [Aave governance study \(Appendix 2\)](#). Standard event absence cannot rule out storage writes or alternative interfaces in replacement code. Conversely, one cannot infer that such unobserved writes actually happened.

This audit excludes liquidation-specific debt settlement and other event types from the Repay total. It does not reconstruct all borrowing or prove an external dollar valuation. No funding provenance from pARB, Portal issuance or a Curve rebalance has been demonstrated by these repayment receipts. That would require matching the relevant earlier transfers and protocol events, rather than attributing a mechanism from the fact that debt was repaid.

Evidence preservation and reproduction

The explicit batch is ignored `artifacts/parb-debt-history-20260929/. requests.json` records the pinned header, signature hashes and full-range `eth_getLogs` requests. `V2_repay_logs.json` and `V3_repay_logs.json` retain the responses. `largest_receipt_evidence.json` contains metadata calls, historical reserve reads and receipts for the six representatives; `supplied_wallet_receipts.json` contains the three additional wallet examples, boundary timestamps and ending-block hash recheck. `repayment_summary.json` records exact raw totals, decimal-normalized totals, representative receipt references, underlying-transfer checks and debt-event reconciliations. Verified-source responses are retained alongside them.

Reproduction consists of replaying these read-only RPC requests at the stated blocks, grouping each Repay log by indexed reserve, and dividing raw amounts by the corresponding 10^{decimals} . Separate every receipt by transaction hash and log index to avoid duplication. The selected receipt checks establish representative execution; this pass did not fetch all 71 transaction receipts. The aggregate counts and totals derive from the RPC's complete returned event list, not a claim of independent consensus verification or an exhaustive whole-ecosystem debt audit.

Appendix 10: Copied Arbitrum gateway discovery and historical permission audit

Contents of this appendix

Findings	75
Historical gateway discovery	75
GRT: the actual permission grant	76
Transaction-time versus endpoint state	76
The temporary custom-gateway implementation	77
Reproduction and query limitations	77

Source record: docs/studies/analysis/parb_gateway_control_history_20260929.md.

Copied Arbitrum gateway discovery and historical permission audit

Research date: 29 September 2026. This study checks historical gateway discovery and permissions at the recorded ARB and GRT issuance dates. All operations were public reads. The endpoint is PulseChain block **27,659,393**, hash `0xc5e582cd902f4bffa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275`. The inherited snapshot is **17,232,999**; post-fork history begins at 17,233,000.

Findings

GRT's minter permission and the implementation that later executed its mint were installed before the fork, in the same April 2023 transaction. The selected GRT ownership, minter and proxy-event histories return no post-fork changes through the endpoint. ARB's configured authorized gateway also matches at the inherited snapshot, immediately before its recorded mint, and endpoint. These findings establish inherited issuance permissions for the recorded mints; they do not establish human ownership of the contracts or source-chain backing.

Equal implementation snapshots do not exclude temporary upgrades. The custom gateway changed implementation and restored its previous implementation on 1 March 2025. Historical execution must therefore be checked for each transaction. No selected custom-gateway withdrawal falls in that interval.

Historical gateway discovery

The prior router census contains 167 `GatewaySet` logs covering 129 token addresses and ten distinct nonzero, non-sentinel gateway addresses over its history. Nine remain selected in the endpoint inventory; the tenth is the retired LPT gateway. This update additionally checks the router's exact verified `DefaultGatewayUpdated(address)` event, rather than assuming token-specific registration events include default changes.

The full-range default query returned **one receipt-verified event**, at block **13,469,475, 22 October 2021 20:08:00 UTC**, setting `0xa3a7b6f88361f48403514059f1f16c8e78d60eec` as the default. That address is already in the inventory. [Default-setting transaction](#).

The default getter returns that same gateway at all four sampled blocks below. This adds no eleventh gateway. It closes the selected default-change-event gap, not a universal discovery problem: other routers, direct gateway integrations and contracts absent from the searched event history remain outside this census. The verified router initializer assigns its initial default without this event; therefore event enumeration alone is not claimed to reconstruct every initialization state. The inherited getter anchors the post-fork interval.

GRT: the actual permission grant

At block **17,018,223, 10 April 2023 13:54:59 UTC**, the same successful transaction records:

- GraphProxy ImplementationUpdated from 0x2bc65e92b68560851c225459a31df6617448ec31 to 0xd41ca6a1d034d178c196dfa916f22f7d1a1b8222.
- Clearance of the pending implementation.
- GraphToken MinterAdded for gateway 0x01cdc91b0a9ba741903aa3699bf4ce31d6c5cc06.

Recorded grant and implementation activation.

GraphToken's verified source makes `mint` conditional on `isMinter(caller)`. Its permission events are `MinterAdded(address)` and `MinterRemoved(address)`. The GRT proxy uses `ImplementationUpdated(address,address)`, `PendingImplementationUpdated(address,address)` and `AdminUpdated(address,address)`; looking only for OpenZeppelin's `Upgraded(address)` would miss its history. [GraphToken source](#), [GraphProxy source](#).

The specialized-control batch returned **30 receipt-verified events**, all inherited: eleven selected GRT-token ownership/minter events, seven GRT proxy events, six WETH proxy events, two wstETH proxy events, two ARB gateway proxy events and two ARB token proxy events. The other selected specialized gateway addresses returned no events for the queried proxy signatures. There were no query errors in this completed batch. Zero results at direct contracts do not prove those contracts have no other control mechanism.

This complements the existing [GRT allowance reconciliation \(Appendix 12\)](#): The allowance update is separately recorded on 18 April 2023, and the later September 2026 issuance used the inherited permission and allowance formula. The allowance is a token quantity, not an independent measurement of source collateral or a dollar valuation.

Transaction-time versus endpoint state

Getter-only reads were made at four independently pinned blocks:

Snapshot	Block
Inherited	17,232,999
Immediately before the recorded ARB mint block	21,469,489
Immediately before the recorded GRT mint block	27,434,948
Audit endpoint	27,659,393

All twelve sampled EIP-1967 implementation slots match across these points: The router, ten identified gateways and the ARB token. Four direct gateway contracts have zero in that slot; zero alone is not proof that no other form of indirection exists. ARB's `arb0neGateway()` consistently returns 0xbbce8aa77782f13d4202a230d978f361b011db27; GRT's `isMinter(0x01cdc91b0a9ba741903aa3699bf4ce31d6c5cc06)`

consistently returns true. All reads completed without RPC errors. The actual recorded issuance traces provide the stronger evidence of the implementation used within each mint transaction; predecessor-block reads alone would miss a same-block change.

The temporary custom-gateway implementation

The completed **post-fork** router, standard-gateway and custom-gateway proxy and default-event search returned exactly two receipt-verified events, both at custom gateway

0xcee284f754e854890e311e3280b767f80797180d:

Date UTC	Block	Recorded implementation
1 March 2025 20:10:55	22,838,182	0x0750201bf8875a6fdea2ff1a70743c58bdebaf2d
1 March 2025 20:29:25	22,838,289	Restored 0xc8d26ab9e132c79140b3376a0ac7932e4680aa45

Upgrade transaction, restoration transaction.

The explorer returned no verified source for the temporary implementation. The six custom-gateway withdrawal events in the existing census occur outside this 107-block separation. This does not exclude other calls during the interval. The changed implementation is not classified as an issuer, and no mint is inferred from the upgrade event itself. The finding demonstrates why endpoint code cannot substitute for historical traces.

Reproduction and query limitations

Maintained manifests are [parb_gateway_control_history_20260929.json](#), [parb_gateway_control_events_20260929.json](#), [parb_gateway_specialized_controls_20260929.json](#) and [parb_gateway_control_getters_20260929.json](#). They preserve the requested scope, narrowed event queries and historical getter specification. No collector or production application code changed.

Run event manifests with `Research/scripts/collect_pulsechain_events.py` and a new output path; run the getter manifest with `Research/scripts/probe_pulsechain_routes.py --calls` at the four listed blocks. Generated range-specific execution manifests, raw responses, sources and block hashes are retained under ignored `artifacts/parb-gateway-control-history-20260929/`, with SHA256SUMS.

Some initial full-range router/standard/custom event queries returned truncated JSON. Those failures are preserved and are **not** counted as empty results. The full-range specialized query, full-range default query and complete post-fork router/standard/custom query subsequently succeeded with receipt and block-hash verification. Partitioned inherited router controls succeeded for blocks 0–13,999,999. One bounded finer follow-up then successfully covered 15,000,000–17,232,999. Its 14,000,000–14,999,999 query still failed with truncated JSON; this is the **remaining inherited-control history gap**. No repeated broad query was attempted in that follow-up. Accordingly this study does not claim a complete inherited router implementation chronology. The independently successful post-fork query covers the entire post-fork interval.

The finer pass adds four receipt-verified inherited implementation events: The router, standard gateway and custom gateway upgraded together at block 15,302,774 on **8 August 2022 17:02:04 UTC**; the router upgraded again at block 15,926,608 on **8 November 2022 16:47:59 UTC**. The August standard/custom implementations match their inherited and endpoint values; the November router implementation matches its inherited and endpoint value. The two later inherited partitions returned zero selected events without query errors.

The follow-up uses four explicit manifests, `Research/experiments/parb_gateway_inherited_controls_0_20260929.json` through `parb_gateway_inherited_controls_3_20260929.json`. Fresh evidence, collector output and the exact failed query diagnostic are preserved under `artifacts/parb-gateway-control-inherited-followup-20260929/` with their own SHA-256 manifest. The original batch and its 53-file hash manifest were preserved.

The white paper can strengthen its claims about inherited GRT issuance authority and historical address coverage. It must retain the distinctions between token registration, implementation history, successful destination issuance and independently verified source backing. No new token-specific mint class was identified by this control-history audit.

Appendix 11: Copied Arbitrum escrow gateways: post-fork withdrawal receipt census

Contents of this appendix

Scope and findings	79
A token can have withdrawals through more than one gateway	80
Split-transfer reconciliation	80
Historical gateway discovery	80
Method and reproduction	80

Source record: docs/studies/analysis/parb_gateway_withdrawal_census_20260929.md.

Copied Arbitrum escrow gateways: post-fork withdrawal receipt census

Research date: 29 September 2026. This study audits actual returned gateway withdrawal events and their successful transaction receipts. It does not create transactions or simulate changes to contract state.

Scope and findings

The selected range is PulseChain block **17,233,000 through 27,659,393**, with endpoint hash `0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275`. Four gateways returned **102**

WithdrawalFinalized events across 74 transactions, involving 90 distinct token addresses. Every event was reconciled to its successful receipt and matching block header. The RPC reported no query errors.

Gateway	Address	Returned withdrawal events
Standard	0xa3a7b6f88361f48403514059f1f16c8e78d60eec	93
Custom	0xcee284f754e854890e311e3280b767f80797180d	6
DAI	0xd3b5b60020504bc3489d6949d545893982ba3011	2
wstETH	0x0f25c1dc2a9922304f2eac71dca9b07e310e8e5a	1

The complete receipts contain **no standard ERC-20 Transfer event from the zero address**. For 101 withdrawals, the same-token receipt has an exact transfer matching the gateway's amount and recipient. Their senders are the standard or custom gateway, designated DAI escrow, or wstETH gateway, as applicable. The remaining withdrawal has two outgoing token transfers whose sum exactly matches the gateway amount, as described below.

This is a receipt-level screen, not a universal proof that every token's supply was unchanged. Tokens may use nonstandard issuance events, rebasing or modified implementations. Historical traces and token-specific state reconciliation are needed to classify a positive issuance candidate conclusively. Endpoint source alone cannot establish what a proxy executed at every historical block.

A token can have withdrawals through more than one gateway

The GRT specialized-gateway mint does not make every GRT withdrawal a mint. The standard gateway also emitted a GRT withdrawal on **10 November 2024**, block **21,896,488**, for **642,162.424364481063167735 GRT**, paid to `0x42543853a72edb6f1ff9bddab0cf4676187ec28a`. Its matching Transfer comes from the standard gateway, and its receipt contains no zero-address Transfer. [Recorded GRT standard-gateway transaction](#).

The custom gateway also emitted an LPT withdrawal for **2 LPT**, paid to `0xa0b347117a878da9942fa5c31b87c773a2969913`, on **1 September 2026**, block **27,434,939**. The matching transfer comes from that custom gateway. The transaction contains two selected withdrawal events; the other token's event is counted separately in the census. [Recorded LPT custom-gateway transaction](#).

These observations are why an audit should search gateway execution history, not only the router's current preferred mapping for each token. They do not establish that those transactions share a human controller or that one financed the other.

Split-transfer reconciliation

At token `0x45804880de22913dfe09f4980848ece6ecbaf78`, the recorded gateway amount is **10,333,968,436,222,304,303 raw units**. The receipt contains two transfers from the standard gateway: one to the withdrawal recipient and one to `0x38699d04656ff537ef8671b6b595402ebdbdf6f4`. Their raw amounts sum exactly to the gateway amount. Therefore the recipient-only exact-amount check fails, while the gross transfer reconciliation succeeds. This study does not infer new issuance from that difference or independently establish the second recipient's purpose. [Split-transfer receipt](#).

Historical gateway discovery

Re-reading all 167 saved router registration logs identifies one additional historical gateway beyond the nine effective gateway addresses in the endpoint inventory: retired LPT gateway `0xd82c27966ebb293b2d8646d8eacb293bc260698e`. Its registration predates the fork. The explorer identifies verified `L1LPTGateway` source. Zero-address default routing and the address-one disabled sentinel are not counted as gateways. The separate specialized-gateway study checks this historical address too.

This yields **ten identified gateway addresses across the returned registration history**, compared with nine effective endpoint addresses. Combining this study with the [specialized-gateway history \(Appendix 20\)](#) and [GRT withdrawal history \(Appendix 12\)](#) yields **106 returned withdrawal events in 78 distinct transactions, involving 91 token addresses** across those ten gateways. It remains bounded to this router and the available logs, not every bridge deployment on PulseChain.

Method and reproduction

The new explicit manifest is `Research/experiments/parb_escrow_gateway_withdrawals_20260929.json`. Run `Research/scripts/collect_pulsechain_events.py --manifest` with that file and a fresh `--output` path. The collector verifies chain 369, the endpoint header, successful receipts and inclusion of each returned event.

The event signature is `WithdrawalFinalized(address,address,address,uint256,uint256)`, verified in each gateway's retained source/interface. Token identity and amount are decoded from the event data; recipient is decoded from its indexed topic. The receipt screen compares each event against same-token

Transfer logs and separately checks all receipt Transfer logs for a zero sender. A mint event in the same receipt would be a candidate, not proof of a causal gateway mint: the trace would still need to connect the gateway call to token issuance.

Raw records, decoded receipt screening, historical-gateway source/registration and SHA-256 hashes are retained under ignored artifacts/parb-escrow-gateway-withdrawals-20260929/. The earlier source and router inventory remain frozen in their original batch. Provider history completeness, unqueried event signatures and other routers remain limitations. No production code changed; production tests are not applicable to this study.

Appendix 12: PulseChain GRT gateway: mint transaction and allowance reconciliation

Contents of this appendix

Recorded transaction.....	82
The allowance was inherited and accumulated with block height.....	83
Both post-fork GRT withdrawals	84
Escrow transfer ledger and the origin of the remaining 1 GRT	84
Historical source design: burn on L2, then release or mint on L1	85
Effect on the main paper	85
Reproduction references.....	85

Source record: docs/studies/analysis/parb_grt_mint_reconciliation_20260929.md.

PulseChain GRT gateway: mint transaction and allowance reconciliation

This read-only audit establishes how the GRT destination gateway funded a completed withdrawal on PulseChain. It distinguishes the destination evidence from the historical source-chain design. The bounded event census ends at block **27,659,393**, hash

0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275. It makes no finding about participants' motives, human identities or an independently operating pARB network.

Recorded transaction

Transaction 0xf7ed5701901cef056395582eab068f1a2e75cdfcf717301c16e89a05ff885ae9 succeeded at block **27,434,949, 1 September 2026 13:42:05 UTC**. Its block hash is

0xbc7a6608cbb7515daaa1464f02cf94fc162b9f89c8c7a0a2859d140c575dd4bb.

Item	Observed value
Transaction sender	0xfbba783afda7b41ebb7473a914f0b6eb9f9139f3
Transaction target / outbox	0x0b9857ae2d4a3dbe74ffe1d7df045bb7f96e4840
Bridge	0x8315177ab297ba92a06054ce80a67ed4dbd7ed3a
GRT gateway	0x01cdc91b0a9ba741903aa3699bf4ce31d6c5cc06
Gateway implementation actually called	0xd41ca6a1d034d178c196dfa916f22f7d1a1b8222
GRT token	0xc944e90c64b2c07662a292be6244bdf05cda44a7
GRT escrow	0x36aff7001294dae4c2ed4fdefc478a00de77f090
Recipient and event's source-account field	0xa0b347117a878da9942fa5c31b87c773a2969913
Withdrawal amount	62,875,937.675361278837595998 GRT
Existing escrow contribution	1 GRT

Item	Observed value
Newly minted shortfall	62,875,936.675361278837595998 GRT
Escrow balance after transaction block	0 GRT

The actual trace, rather than only present implementation storage, identifies the implementation used for this transaction. The gateway reads the escrow's balance, calls `GraphToken.mint(escrow, shortfall)`, and then calls `transferFrom(escrow, recipient, withdrawalAmount)`. The receipt contains both the zero-address mint transfer and the full outgoing escrow transfer. The previously retained total-supply comparison matches the shortfall exactly.

The accounting reconciliation is therefore:

1 existing GRT + 62,875,936.675361278837595998 minted GRT = 62,875,937.675361278837595998 delivered GRT.

The trace's source-sender getter returns configured counterpart `0x65e1a5e8946e7e87d9774f5288f41c30a99fd302`. This establishes the destination checks and calls that succeeded; it does not independently prove the origin of the message or a source-chain burn.

The allowance was inherited and accumulated with block height

The complete selected configuration-event query, block 0 through the audit endpoint, returned six receipt-verified events with no query errors: four initial address-setting events, one allowance update, and the one mint event. There were **no post-fork selected configuration events**. This statement is bounded to those addresses, signatures, range and RPC results; it is not an exhaustive audit of all ways proxy or contract state could change.

The allowance update is [transaction 0x44d5c8a62f2a790f2d7f8238c66df042d500e1b429ffb0bc2f9724009817907d](#), block **17,071,882, 18 April 2023 06:45:23 UTC**, before the fork. Its emitted parameters are:

- Accumulated allowance snapshot: **0 GRT**.
- Accrual rate: **6.0365 GRT per block**.
- Effective starting block: **17,018,986**. This differs from the transaction's block.

Getter-only reads at inherited block 17,232,999, transaction-predecessor block 27,434,948 and transaction block 27,434,949 return the same three parameters. `GraphToken.isMinter(gateway)` returns true at all three snapshots. The inherited and predecessor cumulative minted amounts are zero; the transaction-block value equals the recorded shortfall.

For transaction block $b = 27,434,949$, the verified gateway formula gives:

$\text{allowance}(b) = 0 + 6.0365 \times (27,434,949 - 17,018,986) = 62,875,960.6495 \text{ GRT}$.

Subtracting the transaction's cumulative minted amount leaves **23.974138721162404002 GRT** of unused allowance at that block. The later endpoint allowance of **64,230,816.8555 GRT** leaves **1,354,880.180138721162404002 GRT**. These are GRT quantities under the recorded accounting rule, not dollar amounts or a guarantee of another withdrawal.

This evidence supports **use of inherited mint permission and block-accrued capacity**. It does not support saying that a post-fork actor increased the GRT mint rate. The block-based formula does not itself observe actual source-chain rewards or custody.

Both post-fork GRT withdrawals

The gateway's `WithdrawalFinalized(address,address,address,uint256,uint256)` query covers blocks **17,233,000–27,659,393**. It returned exactly **two events**, with receipt inclusion, successful status and block-hash consistency verified and no query errors.

Date UTC	Transaction	Amount and disposition
8 January 2025 06:18:05	0xd35b3809e5551a1972b518cd38baf093663edddcac26b8c9792ac1d0d7b71093	30,490,952.885900226731295138 GRT transferred from escrow to 0xb8698cffffdaaf26ca21fe8aa560c18cc2b2f5996 ; no GRT mint transfer in this receipt.
1 September 2026 13:42:05	0xf7ed5701901cef056395582eab068f1a2e75cdfcf717301c16e89a05ff885ae9	62,875,937.675361278837595998 GRT delivered, funded by 1 existing GRT and the new mint described above.

The January withdrawal amount equals the escrow balance read at the inherited snapshot. Equality does not by itself prove that no intermediate deposits or withdrawals occurred. The two recipient addresses are factual observations; common ownership is not established by these events.

Escrow transfer ledger and the origin of the remaining 1 GRT

A follow-up query of all post-fork GRT Transfer logs into and out of the exact escrow returned two incoming and two outgoing events through the same endpoint. The first incoming event was not a mint. On **8 January 2025 at 05:13:55 UTC**, wallet [0xb8698cffffdaaf26ca21fe8aa560c18cc2b2f5996](#) called the GRT token and transferred **1 GRT** into escrow in [transaction 0xa9848625...](#), block **22,391,830**. Receipt success, event inclusion and block/transaction hashes were verified. Historical balance getters reconcile the sequence:

Point in the observed history	Escrow GRT
Block 22,391,829, before the deposit	30,490,952.885900226731295138
Block 22,391,830, after the 1-GRT deposit	30,490,953.885900226731295138
Block 22,392,214, before the January withdrawal	30,490,953.885900226731295138
Block 22,392,215, after the January withdrawal	1
Immediately before the September mint, as read in its trace	1
After the September mint and full outgoing transfer	0

Thus the observed escrow ledger reconciles from its inherited balance through the two withdrawals: the January deposit added 1 GRT, the January withdrawal left that amount, and the September transaction combined it with newly minted GRT. The two incoming/outgoing log queries returned no other post-fork escrow transfers in this range. These are address-level token movements, not evidence of the depositor's purpose or ownership of the September recipient.

The new manifests are `Research/experiments/parb_grt_escrow_ledger_20260929.json` and `Research/experiments/parb_grt_escrow_reconciliation_20260929.json`. They preserve exact read-only request parameters; raw responses, receipt, headers, transaction and historical balance reads are under ignored artifacts/parb-grt-escrow-ledger-20260929/, with a SHA-256 manifest.

Historical source design: burn on L2, then release or mint on L1

The official [GIP-0037](#) predates the fork and describes L2 indexing-reward issuance plus bounded L1 gateway minting when withdrawals exceed escrow. Its allowance follows linear block-based accrual. This is a protocol-specific accommodation for L2-native GRT issuance, not a generic permission to mint arbitrary tokens.

The official repository's [L2 gateway at commit 68f70769803e03db6da78d8b89e1af23aad303cd](#), dated **5 May 2023**, calls `bridgeBurn` on the sending account during `outboundTransfer`. That source describes **burn on L2 followed by escrow release and, if necessary, authorized shortfall minting on L1**. It is not a design in which the very same withdrawable GRT remains locked or invested on L2 while being issued again on L1.

The historical official source explains the mechanism; it is not a verified pARB runtime or source-chain transaction. No independently accessible pARB source-state record was supplied for this transaction. The audited destination mint stands independently of that missing source observation.

Effect on the main paper

GRT is a second observed destination-issuance case beyond ARB, using a different accounting mechanism. The paper should state that a lack of sufficient L1 escrow does not universally prevent withdrawal: this GRT gateway filled a real shortfall using pre-existing token mint authority, within its allowance.

The finding does not establish that every gateway mints, that a stablecoin gateway can use GRT's permission, that a source position remains invested, or that issued GRT was subsequently accepted as collateral or exchanged into pDAI. Each of those would require its own address-specific transaction evidence. The factual contribution is an executed, quantitatively reconciled example of a specialized issuer-aware bridge.

Reproduction references

The maintained manifests are:

- [parb_grt_allowance_events_20260929.json](#).
- [parb_grt_withdrawal_events_20260929.json](#).
- [parb_grt_historical_getters_20260929.json](#).

Run the event manifests with `Research/scripts/collect_pulsechain_events.py --manifest <manifest> --output <new-output>`. Run the getter manifest with `Research/scripts/probe_pulsechain_routes.py --calls <manifest> --block <hex-block> --output <new-output>` at `0x106f467`, `0x1a29fc4` and `0x1a29fc5`. Both collectors refuse to overwrite an existing output.

New raw results, historical official source and arithmetic reconciliation are retained under ignored `artifacts/parb-grt-mint-reconciliation-20260929/`, with a SHA-256 manifest linking the prior transaction evidence. The broader source and routing census is documented in [\[parb_bridge_mint_inventory_20260929.md \(Appendix 4\) \(Appendix 4\)\]\(#appendix-04\)](#). No state-changing call or transaction was made.

Appendix 13: Native-L2 assets and L1 representations: the May 2023 bridge baseline

Contents of this appendix

Architectural finding	86
Historical cut and evidence	87
The two Arbitrum accounting models	87
ARB was integrated before the fork	88
Which token classes fit the native-L2 custody model?	88
Exact existing pool coins and application permissions	89
Validator control, source backing and registration	90
Legacy Optimism as a comparison	90
What an L1 fork preserves	90
Reproduction record	91

Source record: docs/studies/analysis/parb_historical_bridge_models_20260929.md.

Native-L2 assets and L1 representations: the May 2023 bridge baseline

Research date: 29 September 2026. Scope: conditional architecture and historical source review, supported by read-only observations of inherited contract state. This note assumes the hypothetical source rollup and its source execution state are supplied. The absence of a public pARB RPC is a limitation on verifying history, not a reason to stop evaluating that architecture.

Architectural finding

The native-L2 lock/L1 mint mechanism was available before the PulseChain fork. Arbitrum's reverse gateways preserve native tokens in L2 escrow and issue an authorized representation on L1. L1 escrow of the represented underlying is not required for this model. ARB is a concrete integration of that architecture whose L1 gateway, token and registration existed in the inherited snapshot.

The model is distinct from the ordinary return of an L1-origin asset. In that model, an L2 representation is burned and existing L1 escrow is released. The same messaging system can support different asset-accounting models because the destination gateway and token determine the accounting operation. An accepted message does not itself decide whether a token is minted or transferred.

For the wider thesis, a transferable source vault receipt can be the asset held in escrow. Its underlying may remain invested in the source protocol. This establishes conditional architectural plausibility for a representation backed by locked source receipts. It does not establish that every existing L1 stablecoin or vault receipt recognizes that representation as its own token.

Historical cut and evidence

The source cutoff is **10 May 2023, 22:36:11 UTC**, corresponding to inherited Ethereum block **17,232,999**. The block returned by the public PulseChain RPC has hash `0xe5667f9cafb115b1dcfc36b0b569a5601f01dfa000698140a1298ef4f87e2e37`. Reads in this note use that inherited block or explicitly identified earlier event blocks. They do not substitute 2026 state for the May 2023 baseline.

Repository commit queries were bounded by that precise cutoff:

Repository	Pinned commit	Commit time, UTC
OffchainLabs/token-bridge-contracts	64cfba62f6dc1ce3b89869db341956f298810299	10 May 2023, 09:13:51
OffchainLabs/nitro-contracts	3d90a211ff1a0b0bb3fcede103d3d375b636a211	10 May 2023, 15:52:43
ArbitrumFoundation/governance	3d8e6fbf4319d0517d4b47ea980961a06fa5f7aa	10 May 2023, 21:03:15
ethereum-optimism/optimism	9196ae3f83e5727e284700dda978d4e624799554	10 May 2023, 21:47:56

Source availability at a date is not proof that every repository contract was deployed at that date. Deployed ARB facts below therefore use historical state and receipts as well as published source. The Optimism comparison deliberately uses the legacy `packages/contracts` bridge, not Bedrock code that was also present in the contemporaneous repository.

The two Arbitrum accounting models

Asset origin and direction	Source operation	Destination operation	Where backing remains
Ordinary L1-origin token, L1 → L2	Transfer L1 token into gateway escrow	Mint L2 representation	L1 gateway
Ordinary L1-origin token, L2 → L1	Burn L2 representation	Transfer existing L1 token from escrow	Escrow decreases as the claim is retired
Native-L2 token, L2 → L1 through reverse gateway	Transfer L2 token into gateway escrow	Mint authorized L1 representation	L2 gateway
Native-L2 representation returning L1 → L2	Burn L1 representation	Transfer native token out of L2 escrow	Source escrow decreases as the representation is retired

In the historical [L1 base gateway](#), the ordinary incoming escrow operation transfers the L1 token to its recipient. The [L1 reverse override](#) instead invokes the destination token's `bridgeMint`; its return operation invokes `bridgeBurn`. The [L2 reverse override](#) transfers source tokens into custody and measures the balance actually received. For the return, it transfers tokens out of custody. These function bodies establish the direction despite an inconsistent direction comment in the historical source headers.

For a fixed-supply, non-rebasing source asset, let s be source token supply, ϵ the amount locked for this representation, and d the outstanding destination representation in equivalent units. At a settled checkpoint in the simple one-to-one model, with no pending claims, $d = \epsilon$. Source circulating units are $s - \epsilon$; adding destination units produces $(s - \epsilon) + d = s$. A destination mint increases that chain's token

supply while the source lock removes the matching amount from independent circulation. Adding the full token supplies on both chains would count escrowed source tokens twice. Multiple destinations require the total claims against an escrow allocation to be reconciled together.

During asynchronous transfers, committed escrow also covers pending issuance rights and pending return rights. In a lossless fixed-unit model without donations, fees or unrelated allocations, $E = D + I + R$, where I and R are those exclusive pending claims. Count each claim once as its processing status changes. This is a conservation model, not a claim that all these terms are exposed by existing contract getters. It explains why escrow and circulating representations need not be equal while a lock/mint or burn/release is in flight.

This is representation backed by custody, rather than new independent source collateral. The identity and redemption rights of the represented source token remain essential: custody of one receipt does not make it redeemable as any other receipt or as an external dollar.

ARB was integrated before the fork

The reverse gateway `0xbbce8aa77782f13d4202a230d978f361b011db27` was created in [transaction 0x1070...ee25](#), block **16,840,302, 16 March 2023, 12:07:11 UTC**. Its successful receipt identifies the gateway as the newly created contract and records implementation `0x4e8c24617c4592895d52ac6df29d9647e9d01bb5`.

At block **16,840,312, 12:09:11 UTC** that day, [transaction 0x5cc8...6dae](#) records registration connecting L1 ARB `0xb50721bcf8d664c30412cfbc6cf7a15145234ad1` with L2 ARB `0x912ce59144191c1204e64559fe8253a0e49e6548`. These are inherited Ethereum events, not new PulseChain governance actions.

At inherited block 17,232,999, both token and gateway have bytecode, the router's ARB mapping selects this gateway, the token's `arbOneGateway()` returns this gateway, and the gateway's implementation is still `0x4e8c...1bb5`. Its verified implementation is `L1ForceOnlyReverseCustomGateway`. The [historical force-only source](#) disables the ordinary token self-registration entry point. This is a governed/registered integration, not an open mint facility for every fungible token.

The pre-cutoff [L1 ARB token](#) explicitly restricts `bridgeMint` and `bridgeBurn` to `arbOneGateway`. Its [L2 counterpart](#) is a distinct source-chain contract. The later recorded PulseChain ARB mint in the companion [bridge observations \(Appendix 5\)](#) therefore uses a destination capability already present in the inherited architecture. That later mint does not establish the actual source collateral position; this historical section establishes when the integration existed.

Which token classes fit the native-L2 custody model?

The relevant distinction is transferable units and their accounting behavior, not whether the token is called a stablecoin, governance token or staking receipt.

Source class	Architectural assessment
Ordinary transferable ERC-20 / PRC-20	Fits the balance/allowance/transfer custody interface. Destination mint authority is a separate requirement.
Transferable ERC-4626 shares	Fits as shares. The represented property is a claim on source vault shares, whose source assets can remain invested.
Static Aave receipt wrappers, cTokens and conventional transferable vault shares	Often fit as fixed share units, subject to their actual implementation. Redemption follows the source receipt's rules.

Source class	Architectural assessment
Rebasing receipts, including direct aTokens or OUSD-style balances	Transferability alone is insufficient to specify the claim. A fixed destination amount does not automatically track later source rebases or interest.
Fee-on-transfer assets	The reviewed source gateway measures actual received amount. That does not alone account for fees on eventual release or unrelated balance changes.
Nontransferable vault shares or debt positions	Do not fit ordinary escrow transfer semantics merely because they expose balance-like information.
ERC-721 / ERC-1155	Require NFT or multi-token custody and representation semantics; the reviewed ERC-20 reverse gateway does not supply those interfaces.
Native currency	Requires its native-currency bridge model, or a separately wrapped fungible token.

The relevant vault standard is [ERC-4626](#), not an identified “4621” bridge category. ERC-4626 shares implement ERC-20, but the standard permits nontransferable implementations to revert transfers. “PRC-20” does not itself add remote mint permissions to the [ERC-20 interface](#).

A static source wrapper can express interest as an increasing source conversion rate while the bridged share count stays fixed. A rebasing receipt can change the escrow's token-unit balance after deposit. These are different claim models. The generic reverse gateway does not by itself reconcile every possible rate, fee, rounding convention or rebase across two separately executing protocols.

Exact existing pool coins and application permissions

Three identities must remain separate: a native source receipt, a destination representation of that receipt, and an existing destination protocol's receipt. A new destination representation of source aDAI may be backed by source aDAI without being the aDAI contract already accepted by a destination Balancer wrapper. Equal symbols, equal decimal counts and matching numerical addresses on different chains do not establish that relationship.

The necessary architecture has three separate permission layers:

1. Messaging recognizes the source message and counterpart contract.
2. The gateway recognizes the token pair and representation rule.
3. The exact destination token authorizes its issuer, and the downstream pool accepts that exact token address.

ARB has explicit token-level bridge mint/burn support. An ordinary ERC-4626 `mint` operation instead receives the vault's underlying asset to issue vault shares; the method name does not mean arbitrary bridge issuance. Aave Portal is a further, application-specific destination issuance mechanism with its own bridge role, reserve accounting and settlement rules. Generic reverse bridging does not automatically satisfy Portal settlement or issue an existing Aave aToken. Under assumed application integration, these can be conditional routes; they should not be conflated with the unmodified generic gateway behavior.

With source execution and the relevant token integration stipulated, it is architecturally coherent for invested source receipts to remain locked while their exact authorized destination representation is traded. That conditional statement is the positive result. It does not need a public source RPC to be logically evaluated. Which integrations were actually deployed, and which specific source deposits occurred, remain separate historical questions.

Validator control, source backing and registration

The audit does not establish abandonment of a pARB execution network or exclusive validator control by the examined wallets. For conditional analysis these can be assumptions. The absence of a public source endpoint is not itself proof of either claim.

Optimistic confirmation and genuine source execution are different evidence. Without an effective honest challenger, the security premise that invalid claims are contested is lost. Acceptance by the parent contracts is consequently not independent proof that the claimed source collateral exists. Nor does a validator role automatically grant token/application administration. An assumed source environment with actual, usable balances and correctly configured native applications is a stronger premise than an accepted assertion alone.

Even granting that stronger premise, L1 contract behavior remains separate. An authenticated message cannot by itself change a token's mint permission, turn a standard transfer into issuance, or make one token address equivalent to another pool coin. A mintable reverse representation requires its destination issuance path; an ordinary return requires transferable destination inventory. The ARB implementation provides a concrete example of pre-existing authorized issuance, not a universal stablecoin issuer.

A formal new registration transaction is not universally necessary. Integrations can be inherited, registration can be supplied by a bridge administrator or factory, and standard routes can use predefined counterpart derivation. Which checks apply must be read from the actual entry point rather than inferred from the presence of a function named `register`. This distinction does not remove the exact destination token's transfer or issuance rules. Likewise, storing a token mapping is not itself a grant of mint authority by that token.

Source reserve solvency in copied-token units can be stipulated consistently. It does not establish external-dollar backing, automatic acceptance by L1 Aave or Curve, or separate backing for multiple claims that reuse one collateral allocation. An L1 bridge can create a new representation without the original issuer managing it, where the bridge design supports this; existing L1 pools still need to accept that representation's exact address.

The main paper's source-state-control section applies these distinctions to ARB, ordinary escrow release, factory-created representations, Aave Portal, OUSD/eUSD/mUSD and funded Across delivery. This is a source/accounting review; no new state-changing experiments or public transactions were performed.

Legacy Optimism as a comparison

The May 2023 legacy [L2 StandardBridge](#) burns the L2 representation on withdrawal and identifies its paired L1 token. The corresponding [L1 StandardBridge](#) authenticates the counterpart messenger, reduces the recorded deposit amount for the token pair, and transfers existing L1 tokens to the recipient.

That standard return is not the Arbitrum native-L2 reverse model. A valid withdrawal message does not make the legacy Optimism L1 standard bridge mint an arbitrary existing L1 token when its corresponding escrow/deposit accounting cannot support the withdrawal. The existence of more general cross-domain messaging does not change the standard bridge's programmed token operation. This comparison concerns the reviewed legacy standard contracts, not every custom bridge or later Optimism version.

What an L1 fork preserves

Copying Ethereum state preserves the L1 contracts, storage, token mappings, escrows and recorded rollup commitments at the snapshot. It does not copy the separate L2 execution database simply because an L1 contract contains its root or counterpart address. These are different layers of the system.

For this theoretical exercise, source-chain execution, balances and operating protocols may be explicit assumptions. Under those assumptions, the analysis can assess token custody, authorization, representation and redemption without requiring a public source endpoint. A historical assertion that a particular source deposit occurred would require additional source or message evidence. Keeping those claim types separate allows positive architectural findings without presenting assumptions as observed transactions.

Reproduction record

The new batch is `artifacts/parb-historical-bridge-models-20260929/`. It contains the cutoff commit queries, source manifest and source caches, inherited-block metadata/code/getter responses, bounded gateway logs, and the deployment and registration receipts/blocks. Historical gateway log coverage is block 16,600,000 through 17,232,999. The selected deployment receipt directly reports the gateway's creation address; its date is not inferred merely from the first log returned by that bounded query.

RPC observations use `eth_getCode`, `eth_getStorageAt`, `getter eth_call`, `eth_getLogs`, `eth_getBlockByNumber` and `eth_getTransactionReceipt` against `https://rpc.pulsechain.com`. They are one-provider observations. No transaction payloads, privileged state changes or simulations were produced. Raw data and source copies remain in ignored artifacts rather than this maintained study.

Appendix 14: Across HubPool custody and Aave Portal settlement

Contents of this appendix

Finding.....	92
Source and observation boundary.....	92
What each transfer actually does.....	93
Aave supply is not Portal backing.....	94
Observed adapters and the boundary of the standard interface.....	94
Configured SpokePool: historical callback and a later implementation change.....	95
Implication for the proposed loop.....	95

Source record: docs/studies/analysis/parb_hub_aave_accounting_20260929.md.

Across HubPool custody and Aave Portal settlement

Research date: 29 September 2026. This is a read-only assessment of the proposed flow “purchased pDAI → Across HubPool → Aave → backUnbacked.” It distinguishes the deployed interfaces and their accounting from a separately authorized integration that might connect them.

Finding

The copied HubPool's standard liquidity operations do not automatically lend DAI into Aave or settle an Aave Portal obligation. `addLiquidity` takes actual DAI into HubPool and issues an Across LP claim. `backUnbacked` takes actual DAI into Aave and reduces its previously issued unbacked balance. These are distinct cash movements with different outstanding claims.

A separately integrated arrangement could connect the systems, but its cash movement and treatment of Across LP claims would need to be specified. The observed HubPool includes governed adapter calls, so absence of an Aave-named entry point is not proof that every possible custom integration is impossible. It is evidence that the proposed lending step is not supplied automatically by the standard operations reviewed here.

Source and observation boundary

HubPool is `0xc186fa914353c44b2e33ebe05f21846f1048beda`. Its [explorer-verified deployed source](#) and ABI were reviewed alongside the May 2023 [repository source](#). The versions have some source differences, including the default identifier; the relevant liquidity, reserve-sync and adapter accounting operations support the conclusions below. Repository availability is not substituted for deployed behavior where the verified source is available.

Endpoint reads use PulseChain block **27,659,393**, hash

`0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275`. Actual cash and recorded reserve results are preserved in the companion batch `artifacts/parb-across-accounting-20260929/reserve_reads.json`.

Token	Actual HubPool token balance	Recorded liquid reserves	Recorded utilized reserves
DAI	0	0	1,656,490.776260091143953778
USDC	0	0	16,210,612.259813
USDT	0	0	500,442.221859

Recorded utilized reserves are accounting entries, not presently spendable token balances in HubPool. These observations do not establish the value or recoverability of every remote claim. They also do not preclude fresh pDAI being transferred into HubPool later.

What each transfer actually does

Operation	Custody change	Claim/accounting change	Does this settle Portal?
HubPool addLiquidity	Transfers underlying from provider to HubPool	Increases liquid reserves and issues Across LP tokens at the pool exchange rate	No
Direct token transfer to HubPool	Increases HubPool's actual token balance	Does not itself issue Across LP tokens or invoke Aave	No
HubPool sync following an excess balance	No further transfer	Increases recorded liquid reserves and reduces utilized reserves by the same excess, subject to bond accounting	No
HubPool removeLiquidity	Transfers available underlying from HubPool to LP redeemer	Burns the LP claim and reduces liquid reserves	No
Aave ordinary supply	Transfers underlying into the Aave reserve	Creates or increases an aToken claim for the beneficiary	Does not reduce an existing Portal unbacked balance
Aave backUnbacked	Transfers underlying into the Aave reserve	Reduces existing unbacked principal; accounts separately for any fee	Yes, for the actual backing amount accepted

In deployed HubPool `addLiquidity`, exchange-rate synchronization occurs before the new deposit, then the contract issues LP shares and receives the underlying. In `removeLiquidity`, the LP claim is burned and the actual underlying is paid out; available liquid reserves and token cash constrain that payment.

An unsolicited transfer is different. `_sync` treats excess underlying balance as cash returning to the pool: it raises liquid reserves and decreases utilized reserves. Ignoring fees and other concurrent changes, their sum does not increase from this reclassification alone. A direct transfer therefore cannot be treated simultaneously as a newly issued Across LP position, a new independent receivable, and a completed Aave settlement.

Aave supply is not Portal backing

The historical [Aave Pool interface](#) and [BridgeLogic](#) separate ordinary supply from settlement of previously issued unbacked aTokens. `backUnbacked` uses the lesser of the requested amount and outstanding unbacked principal, reduces that outstanding balance, and transfers underlying plus any specified fee into the aToken contract. It is restricted through Aave's bridge authorization. It does not mint an additional aToken principal position for the party paying the backing.

Consequently, lending 100 DAI to Aave through ordinary supply can create a 100-DAI-denominated aToken position, but it does not also discharge a separate 100-DAI Portal obligation. Paying 100 DAI through `backUnbacked` can discharge that obligation, but it does not simultaneously give its payer another fresh 100-DAI aToken claim. The same distinction applies when the funding originated from a purchase, an Across LP redemption, or some other transfer.

If Across LP claims remain outstanding while their supporting cash is committed to settle someone else's already-issued aToken liability, the Across balance sheet needs a corresponding real asset or an explicit extinguishment of its claim. Merely labelling the movement “lending” does not establish either. An Across LP token is not automatically an aToken, and an Aave balance is not automatically included in HubPool's ordinary reserve calculation.

Observed adapters and the boundary of the standard interface

The endpoint's `crossChainContracts` reads identify:

Stored chain key	Adapter	SpokePool
1	0x527e872a5c3f0c7c24fe33f2593cfb890a285084	0x5c7bcd6e7de5423a257d81b442095a1a6ced35c5
42161	0x29528780e29abb8af95a5e5a125b94766987543f	0xe35e9842fcea96570b734083f4a58e8f7c5f2a
369	Zero	Zero

The two nonzero adapter addresses match the pinned historical deployment records for [Ethereum_Adapter](#) and [Arbitrum_Adapter](#). These are stored keys in copied state; the key 1 does not itself change the execution chain back to Ethereum or prove a live remote connection.

The historical [Ethereum adapter](#) is a same-chain pass-through: its token operation transfers tokens to the designated recipient, and its message operation calls its designated target. The [Arbitrum adapter](#) uses the canonical bridge/router and messaging interfaces. These source paths do not define a native Aave lending strategy or automatically invoke Portal settlement. This is an assessment of the identified historical adapters, not every possible adapter contract.

HubPool executes adapter code through `delegatecall` in the HubPool context. Standard positive pool-rebalance sends reduce liquid reserves and increase utilized reserves as assets are sent toward the configured spoke. The base exchange-rate calculation uses liquid reserves, utilized reserves and undistributed LP fees; it does not independently discover or value arbitrary Aave investments. Governed adapter composition is therefore a meaningful extension point, but its existence is not evidence that the hypothesized HubPool-to-Aave investment and claim reconciliation already occurred.

The HubPool ABI contains neither an Aave supply/deposit method nor a Portal `backUnbacked` method. Its `multicall` batches supported HubPool operations; that name is not evidence of a general Aave lending service. No actual HubPool→Aave lending or Portal-settlement transaction was identified in this bounded source/accounting review. It was not an exhaustive search of every HubPool transaction or every separately configured adapter.

Configured SpokePool: historical callback and a later implementation change

The configured key-1 SpokePool is `0x5c7bcd6e7de5423a257d81b442095a1a6ced35c5`. It must not be confused with the older `0x4d9079bb4165aeb4084c526a32695dcfd2f77381` deployment inspected elsewhere. The older address's ABI does not establish this configured proxy's callback capabilities.

At inherited block 17,232,999, the configured proxy's implementation is `0xa667498f46457548f1d3ad557340b95fdb290148`. Its [verified Ethereum_SpokePool implementation and included SpokePool source](#) **do contain destination message callbacks**. The deposit and fill interfaces include message bytes. Following token payout, a contract recipient with a nonempty message can receive `handleAcrossMessage` with the token, amount, whether the fill is complete, the relayer and the message. The self-relay branch returns earlier without the transfer or callback.

This corrects any broader conclusion that the configured copied SpokePool lacked callbacks at the fork. The historical callback supports conditional destination composition with a compatible recipient. It does not make Aave's Pool itself an Across message handler or establish that an actual recipient already supplies assets to Aave or performs authorized Portal settlement. The verified inherited ABI has no Aave-specific lending/backing entry point.

At the endpoint the proxy instead points to `0x788e49c4e2b3414777d6671d7411951e81d35808`. [Transaction 0x4c07...7ee4](#) successfully upgraded it at block **26,912,634, 30 June 2026, 04:52:25 UTC**, block hash `0xc8f90908ec94d2608cb6e88eeabc9538f21767014f38b2abe7187b916131b859`. The sender was HubPool's observed owner `0xfeca7291cebe75789ffa707754db259187b1d140`. The actual trace runs through HubPool, its Ethereum adapter and the configured SpokePool's upgrade operation. It uses stored chain key 1 on PulseChain.

The old implementation has 22,759 bytes of runtime code at the inherited block; the endpoint implementation has 2,191 bytes of runtime code at the endpoint. The explorer returns **no verified source or ABI** for the new implementation. Code size alone does not identify its functionality. This study therefore does not claim that the current implementation retains the inherited callback, removes it, or adds Aave methods. The historical capability and the later unverified implementation are separate findings.

Likewise, the zero mapping under key 369 is an observation about one stored configuration key, not proof that every possible Across operation stops on PulseChain. The successful key-1 administrative trace demonstrates that at least that configured same-chain message path operated there. Whether a particular source/destination liquidity route is consistent with the deployed chain-ID rules requires that route's own evidence; changing the label does not by itself create liquidity or an Aave integration.

The new observations are preserved separately under `artifacts/parb-mapped-spoke-20260929/`: inherited/endpoint implementation and code reads, the proxy upgrade log, successful receipt and trace, block metadata, and both explorer source responses. All were read-only.

Implication for the proposed loop

Buying existing pDAI and moving it into HubPool can replenish Across cash. Moving that pDAI into Aave can instead fund an Aave supply position or settle Portal, according to the actual method and authorization used. A complete ledger must identify which claim is created, retained or discharged at each movement. No individual unit can remain spendable HubPool cash and also be cash already delivered to Aave at the same instant.

The missing link in the standard-interface description is an observed or explicitly stipulated integration that moves the assets and reconciles those claims. This is compatible with conditional architectural analysis; it is not filled simply by assuming that an Across deposit is automatically lent into Aave or that ordinary Aave supply equals `backUnbacked`.

New source and adapter-read evidence is preserved under ignored `artifacts/parb-hub-aave-accounting-20260929/`. Public access was read-only. No simulated transactions, privileged calls or configuration changes were performed.

Appendix 15: Native Arbitrum assets around May 2023: a documentary baseline for pARB

Contents of this appendix

Findings	97
Evidence boundary	98
Aave V3: a concrete native Arbitrum deposit baseline	98
The two Arbitrum USDC identities	99
Other historically documented native receipt classes	99
Curve liquidity-pool shares	99
GMX GLP and staked receipt layers	100
Named protocols that need a different historical assumption	100
Implications for a lock-and-mint interpretation	101
Evidence preservation	101

Source record: docs/studies/analysis/parb_native_assets_baseline_20260929.md.

Native Arbitrum assets around May 2023: a documentary baseline for pARB

Research date: 29 September 2026. This study uses dated protocol repositories, deployment registries and issuer announcements. It does not assume an available pARB RPC, perform simulations or submit transactions. It identifies historical source-side candidates for the theoretical mechanism; it does not claim that those contracts currently execute on an independently observed pARB chain.

Findings

The proposed source-side operation has genuine historical building blocks. Aave V3 had a documented Arbitrum deployment with DAI, bridged USDC and USDT reserves before May 2023. Depositing supported assets could create the corresponding Arbitrum aTokens. Curve also documented Arbitrum-native stablecoin liquidity-pool shares, and GMX documented GLP issuance against supported pool assets. Those receipt classes could represent assets held or invested by source-side contracts.

It is nevertheless inaccurate to assume that every named Ethereum application already had the same native Arbitrum deposit facility. The reviewed historical Origin, mStable and Reserve registries do not establish native Arbitrum OUSD, mUSD or eUSD issuance. Compound's own announcement places activation of its Arbitrum USDC market on **15 May 2023**, after the 11 May baseline. The later Compound III market is also not the same receipt model as Ethereum's Compound V2 cDAI/cUSDC/cUSDT markets.

An assumption that other applications were subsequently deployed or repurposed on pARB remains a legitimate theoretical extension. It is a different claim from identifying applications inherited from the actual May 2023 Arbitrum environment. Neither source-side existence nor receipt transferability grants permission to mint the exact existing L1 token accepted by a Curve or Balancer pool.

Evidence boundary

The main inventory comes from repositories maintained by the relevant protocols, pinned to commits available around the May 2023 cutoff. Aave's selected address book commit is dated **26 April 2023**. Its nine listed reserves are a documented baseline, not a newly measured, exhaustive list at every moment on 11 May. Curve's Arbitrum registry was last changed on **21 April 2023**, and the file in its 11 May snapshot is byte-identical. GMX's selected commit is dated **10 May 2023**. Repository publication establishes documented deployments and mechanisms; it does not independently prove a historical balance, cap, pause flag or oracle value.

An attempted read of the public original-Arbitrum RPC returned HTTP 403. This study therefore makes no new source-chain archival-state assertion. The absence of a pARB endpoint does not prevent conditional analysis: the paper can explicitly assume a running source chain with specified applications and balances. What it cannot do is promote that assumption into an observed transaction history.

Aave V3: a concrete native Arbitrum deposit baseline

The historical Aave address book identifies:

Component	Arbitrum address
Pool	0x794a61358d6845594f94dc1db02a252b5b4814ad
Addresses provider	0xa97684ead0e402dc232d5a977953df7ecbab3cddb
Configurator	0x8145edddf43f50276641b55bd3ad95944510021e
Oracle	0xb56c2f0b653b2e0b10c9b928c8580ac5df02c7c7
ACL manager	0xa72636cbcaa8f5ff95b2cc47f3cdee83f3294a0b
Static aToken factory	0x90b1255a76e847cc92d41c295ded5bf2d4f24b3d

The same record lists nine underlying/aToken pairs:

Underlying	Underlying address on Arbitrum	Source aToken address
DAI	0xda10009cbd5d07dd0cecc66161fc93d7c9000da1	0x82e64f49ed5ec1bc6e43dad4fc8af9bb3a2312ee
LINK	0xf97f4df75117a78c1a5a0dbb814af92458539fb4	0x191c10aa4af7c30e871e70c95db0e4eb77237530
Bridged USDC	0xff970a61a04b1ca14834a43f5de4533ebddb5cc8	0x625e7708f30ca75bfd92586e17077590c60eb4cd
WBTC	0x2f2a2543b76a4166549f7aab2e75bef0aefc5b0f	0x078f358208685046a11c85e8ad32895ded33a249
WETH	0x82af49447d8a07e3bd95bd0d56f35241523fbab1	0xe50fa9b3c56ffb159cb0fca61f5c9d750e8128c8
USDT	0xfd086bc7cd5c481dcc9c85ebe478a1c0b69fcb9	0x6ab707aca953edaeefbc4fd23ba73294241490620
AAVE	0xba5ddd1f9d7f570dc94a51479a000e3bc967196	0xf329e36c7bf6e5e86ce2150875a84ce77f477375

Underlying	Underlying address on Arbitrum	Source aToken address
EURS	0xd22a58f79e9481d1a88e00c343885a588b34b68b	0x6d80113e533a2c0fe82eabd35f1875dcea89ea97
wstETH	0x5979d7b546e38e414f7e9822514be443a4800529	0x513c7e3a9c69ca3e22550ef58ac1c0088e918fff

Source: [Aave Arbitrum address book](#), commit 61e8d02, 26 April 2023.

These are specific listed inputs, not interchangeable stablecoin labels. A DAI deposit uses the listed Arbitrum DAI contract and creates the listed Arbitrum aDAI. It does not create the PulseChain copy of Ethereum V3 aDAI at 0x018008bfb33d285247a21d44e50697654f754e63. Mapping source claims into that destination receipt is a separate application/bridge integration. Pool listing also does not establish acceptance of an arbitrary deposit size: active/frozen state, supply caps, successful transfers and the configured reserve rules still govern the source deposit. A documented static-aToken factory is not evidence that every possible wrapper was deployed or bridged.

The two Arbitrum USDC identities

The pre-cutoff USDC address above is the Ethereum-bridged asset, subsequently called **USDC.e**. Circle announced native Arbitrum USDC on **8 June 2023**, at 0xaf88d065e77c8cc2239327c5edb3a432268e5831. That later native token must not be inserted into a May 2023 deployment inventory by using a current address book. “Native Aave market accepting bridged USDC” means the lending application runs on Arbitrum; it does not mean Circle issued that underlying natively there. [Circle's dated issuer announcement](#).

Other historically documented native receipt classes

Curve liquidity-pool shares

Curve's pre-cutoff Arbitrum registry identifies a direct stablecoin example:

Item	Recorded Arbitrum deployment
2pool swap contract and LP token	0x7f90122bf0700f9e7e1f688fe926940e8839f353
Input assets	Bridged USDC 0xff970a61a04b1ca14834a43f5de4533ebddb5cc8; USDT 0xfd086bc7cd5c481dcc9c85ebe478a1c0b69fcb9
Recorded input decimals	6 and 6
Gauge	0xce5f24b7a95e9c8a7df4b54e911b4a3dc8cdaf6f

This supplies a historically documented alternative to a lending receipt: liquidity provision creates claims on a stablecoin pool. The LP token can have an economic claim on source-side assets while being a distinct token from either deposited stablecoin. A bridge holding that LP claim would not thereby gain the right to mint Ethereum's 3CRV or FRAXBP token.

The same historical registry also contains Arbitrum tricrypto, EURS/2CRV, WBTC/renBTC and ETH/wstETH pools. For example, tricrypto's LP token is 0x8e0b8c8bb9db49a46697f3a5bb8a308e744821d2, distinct from its swap contract 0x960ea3e3c7fb317332d990873d354e18d7645590. EURS/2CRV is already a nested receipt construction: its second pool coin is the Arbitrum 2pool LP. These examples broaden the asset classes without treating each nesting level as additional independent collateral. [Curve Arbitrum registry](#), commit 573da2e, 21 April 2023.

GMX GLP and staked receipt layers

GMX's pre-cutoff Arbitrum configuration identifies GLP at [0x4277f8f2c384827b5273592ff7cebd9f2c1ac258](#), a manager at [0x321f653eed006ad1c29d174e17d96351bde22649](#), and separate fee/staked trackers. Its Arbitrum asset configuration includes DAI, bridged USDC and USDT. The liquidity manager transfers supplied assets into the vault and issues GLP using the vault's valuation and existing GLP supply. The reward router can then stake the GLP through tracker layers. This is a native pool-share mechanism, not a fixed-dollar stablecoin issuer. [Historical Arbitrum addresses, asset configuration, GLP manager](#).

GLP/staked-GLP transfer rules, handlers, cooldowns and redemption conditions matter to whether a particular receipt can actually be held by an escrow. The presence of ERC-20-like methods does not guarantee unrestricted transfers. No exact existing L1 GLP representation or accepting PulseChain pool is proved by this source-side example. It establishes an additional plausible source receipt family, not a completed L2-to-L1 path.

Named protocols that need a different historical assumption

Family	Historical evidence reviewed	Consequence for the May 2023 pARB hypothesis
Origin OUSD / woUSD	Origin's dated repository contains Ethereum mainnet OUSD/vault deployments and collateral strategy addresses.	Does not establish a native Arbitrum OUSD vault accepting the three assumed pStables. A later/custom pARB deployment is a separate assumption.
mStable mUSD / savings shares	The historical token registry distinguishes Ethereum mUSD from Polygon mUSD and lists test networks.	Polygon deployment is not Arbitrum deployment. No native Arbitrum mUSD issuer is identified in that registry.
Reserve eUSD	Historical configuration identifies eUSD and its Aave/Compound collateral under Ethereum chain ID 1; plugin addresses are explicitly mainnet.	The existence of Ethereum eUSD does not establish the same basket, plugins and issuer natively on Arbitrum.
Compound V2 cTokens	Canonical cDAI/cUSDC/cUSDT in the reviewed collateral registries are Ethereum contracts.	Do not substitute them for native Arbitrum lending receipts merely because the protocol code can run on an EVM.
Compound III Comet	Compound's announcement records Arbitrum USDC initialization proposal 160 executing 15 May 2023.	Live activation postdates the 11 May baseline; it can support a later-state hypothesis, not an already-live May 11 market claim.
Yearn vault shares	No dated Arbitrum vault deployment was verified in this bounded pass.	Remains a candidate family, not an established May 2023 native source deployment here.

The underlying primary records are [Origin's May 2023 addresses](#), [mStable's historical token registry](#), [Reserve's May 2023 chain configuration](#), and [Compound's dated launch report](#). Absence from these bounded registries is not proof that no third party ever deployed similar code. It is a reason not to assert the particular historical deployment without evidence.

Compound III also needs a mechanism distinction. Its market has a base asset that suppliers can earn interest on; other supported assets serve as collateral for borrowing that base asset. The initial Arbitrum market's base asset was bridged USDC. That does not supply a native cDAI, cUSDT and cUSDC market for each of the three inputs. Collateral deposits are not automatically separate transferable yield receipts equivalent to Compound V2 cTokens. [Compound III's primary documentation](#).

Implications for a lock-and-mint interpretation

The relevant categories are transferable asset units, lending claims, vault shares, LP shares and restricted/staked positions. ERC-20 compatibility describes basic fungible-token behavior; ERC-4626 adds a standardized single-asset vault share interface. Neither standard alone supplies cross-chain authentication, remote redemption, or mint authority over an existing L1 token. A non-ERC-4626 receipt such as an LP share is not excluded merely because it uses a different accounting interface. Conversely, a compatible interface does not make all receipts equivalent in collateral value or transferability.

For a conditional model, it is coherent to assume that an identified source receipt is held in escrow while a destination token represents that claim. The unit of account must match the receipt's behavior: interest-accruing aToken units, fixed vault shares with a changing exchange rate, or pro-rata LP shares. The model must treat independent source redemption and outstanding destination claims consistently. Naming several wrappers does not multiply the collateral represented by a single locked underlying position.

The bridge's output identity remains decisive. A newly issued L1 representation of Arbitrum aDAI, GLP or 2pool shares is not automatically the existing L1 aDAI, waDAI, OUSD, mUSD, eUSD or 3CRV already accepted by a pool. Authorized minting of those particular outputs, or an independently funded conversion into them, is a separate condition. For Aave Portal, source escrow and destination `mintUnbacked/backUnbacked` accounting are separate mechanisms, as described in the main study.

Thus the theoretical source-receipt concept is plausible for more than one historical Arbitrum asset class. The strongest documented stablecoin examples are the three Aave reserves and Curve's USDC/USDT LP, with GMX as an additional pool-share family. Extending the model to native OUSD, mUSD, eUSD or post-cutoff Comet requires clearly naming the extra deployment/version assumption. This preserves the distinction between mechanism feasibility and historical fact without making a source RPC a prerequisite for theoretical reasoning.

Evidence preservation

Generated sources and GitHub commit metadata are in `ignored artifacts/parb-native-assets-baseline-20260929/`, with `source_manifest.json` recording the selected commit dates. Repository trees document the bounded deployment searches; source files preserve the exact address registries read. The April 21 and May 11 Curve registry copies were compared and match.

The official Circle and Compound announcements were available through browser retrieval; direct local downloads returned HTTP 403. Their dates and claims are cited directly above. No unavailable pARB state or blocked archival response is treated as positive deployment evidence. The inventory is deliberately bounded and does not claim every native Arbitrum protocol, every accepted token, or every L1 market capable of accepting a future representation.

Appendix 16: Balancer Aave wrapper pools: balances, limits and public transaction evidence

Contents of this appendix

Findings	102
Contracts and accounting	102
The meaning of the limit	103
Precision and wrapper rates	104
How different balance changes affect capacity	104
Historical configuration and trades	105
Evidence and reproducibility	106

Source record: docs/studies/analysis/parb_pool_limits_20260929.md.

Balancer Aave wrapper pools: balances, limits and public transaction evidence

Research date: 29 September 2026. This study uses public contract reads, historical logs, transaction receipts and verified source. It performs no transactions or local state-changing simulations. Its principal state is PulseChain chain 369, block **27,659,393**, hash

0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275.

Findings

The existing V3 Aave wrapper pools do not have a universal fixed quota of wa-tokens they can accept. Their capacity depends on the requested output, registered pool balances, wrapper exchange rates, fee targets, token precision and available pool shares. Adding actual main-token liquidity through a recorded pool operation can increase the amount exchangeable for wrapped tokens. Adding wrapped tokens alone does not create additional main-token cash.

There is also direct public evidence that these wrappers have traded on PulseChain. A successful June 2026 transaction exchanged waDAI for DAI, and a September 2026 transaction exchanged waUSDC for USDC. These transactions establish acceptance of the exact wrapper addresses. They do not establish that the input receipts came from Aave Portal or any pARB bridge.

The sampled fee and target configuration is inherited from before the fork. Queries found no subsequent target or fee events for these three pools through the principal block. Their low main balances therefore must not be described as proof of a newly imposed fixed deposit cap or a documented governance takeover.

Contracts and accounting

All three pools use Balancer V2 Vault 0xba12222222228d8ba445958a75a0704d566bf2c8. Here “V3 Aave” describes the lending receipts, not Balancer protocol V3. The DAI pool's verified contract is ERC4626LinearPool, inheriting LinearPool and LinearMath.

Main asset	Linear pool	Wrapped asset
DAI	0x6667c6fa9f2b3fc1cc8d85320b62703d938e4385	0x098256c06ab24f5655c5506a6488781bd711c14b
USDC	0xcbfa4532d8b2ade2c261d3dd5ef2a2284f792692	0x57d20c946a7a3812a7225b881cdcd8431d23431c
USDT	0xa1697f9af0875b63ddc472d6eebada8c1fab8568	0xa7e0e66f38b8ad8343cff67118c1f33e827d1455

The Vault separately records `cash` and `managed` for every token in each pool. `cash` is held by the Vault for that pool; `managed` records assets outside it under the registered asset manager. Their sum supplies the pricing balance. A swap paying out a token must also have sufficient corresponding cash. The Vault's aggregate ERC-20 balance includes other pools and is not the balance available to this pool. A direct ERC-20 transfer to the shared Vault has no pool identifier and does not itself change a pool's recorded allocation. The verified Vault updates the relevant accounting during swaps, joins, exits and authorized asset-manager balance operations. [Verified Vault source](#).

Fresh `getPoolTokenInfo` reads at the principal block give:

Main asset	Registered cash, whole tokens	Managed amount	Last balance-change block
DAI	10.055365963093764759	0	26,908,905
USDC	63.044960	0	27,458,661
USDT	9.999901	0	27,251,196

An aggregate balance or a very large preminted BPT balance cannot substitute for these amounts. BPT is the pool share token. The implementation initially premints $2^{112} - 1$ raw BPT units into the pool; circulating or “virtual” supply is total supply less the BPT held in the pool. Uncirculated shares are not additional underlying stablecoins. [Verified DAI pool source and dependencies](#).

The meaning of the limit

LinearMath prices swaps using a nominal main balance. Let m be the main balance, L and U the lower and upper targets, and f the fractional fee. Express all balances in the same normalized main-token units. Ignoring integer rounding for explanation:

$$\begin{aligned}
 N(M) &= M - f(L-M) && \text{when } M < L \\
 &M && \text{when } L \leq M \leq U \\
 &M - f(M-U) && \text{when } M > U
 \end{aligned}$$

For wrapped-to-main exact-input swaps, the normalized main-equivalent input w is subtracted from $N(M)$. Checked subtraction requires $w \leq N(M)$; the result is converted back into a real main balance. In the below-target region the corresponding main balance at nominal zero is approximately:

$$\begin{aligned}
 M_{\text{floor}} &= f * L / (1 + f) \\
 \text{main-token output headroom} &= M - M_{\text{floor}}
 \end{aligned}$$

This is an accounting floor for this swap direction and configuration, not a universal reserve requirement or a cross-pool liquidity limit. Actual results include fixed-point rounding, token decimals and Vault cash checks. The verified source uses downward rounding for nominal fees and normalized inputs, downward rounding for output, and upward rounding for required inputs in exact-output swaps. [LinearMath and scaling functions in verified source](#).

All three sampled pools report $L = 1,000,000$, $U = 5,000,000$, and $f = 0.00001$ (0.001%). Thus the approximate floor is **9.999900001 main tokens**. The DAI pool's corresponding output headroom is approximately **0.055465962 DAI**. The USDC pool's is approximately **53.045060 USDC**. USDT is within approximately one millionth of a token of the floor; six-decimal discretization is material at that scale. These are balances at the recorded block, not perpetual limits.

The prior study's one-aDAI failure and smaller successful local trades are consistent with this formula, but those simulations are not public historical transactions and were not rerun here.

The upper target is the top of the desired zero-fee balance range. It is **not** a hard maximum quantity of wrapped tokens or total liquidity. Above that range, the alternate fee branch applies. Conversely, a main balance below the lower target is not an automatic total trading halt; the remaining nominal balance still matters.

Precision and wrapper rates

DAI has 18 decimals; USDC and USDT have six. The pool upscales raw balances to 18-decimal normalized units. Wrapped balances additionally incorporate the wrapper's main-equivalent rate. The ERC-4626 pool obtains that rate by calling `convertToAssets` with an amount scaled according to both tokens' decimals. Consequently one wa-token is not generally one underlying token, nor can a raw USDC amount be compared directly with a raw DAI amount.

At the sampled block the rate getters return approximately **9.9253150471** for waDAI, **6.7812647985** for waUSDC and **9.3397383592** for waUSDT, expressed in main units per whole wrapped token. These are local wrapper conversions, not external-dollar prices or attestations of remote collateral. They can change with the wrapper's underlying accounting. Historical trades must use their historical rates rather than the later snapshot's values.

How different balance changes affect capacity

Observed or supported operation	Consequence
Main tokens enter the pool and are recorded to its balance	Main inventory and nominal main balance can increase; later wrapped-to-main output capacity can increase, subject to the resulting state.
Wrapped tokens enter in exchange for main tokens	Main tokens leave; that direction consumes main output headroom.
Wrapped tokens enter in exchange for BPT	Wrapped backing and circulating shares can increase, while main cash remains unchanged; this does not by itself restore main output headroom.
Main tokens enter in exchange for BPT	Adds main inventory and distributes pool shares; the shares represent claims rather than extra stablecoin cash.
BPT enters in exchange for main tokens	Main tokens leave; returning pool shares does not create replacement main assets.

Observed or supported operation	Consequence
Registered asset manager redeems wrapped holdings and accounts for actual main liquidity	Can move liquidity between wrapped and main forms, subject to wrapper redeemability and valid balance accounting.
Main or wrapped tokens are transferred directly to the shared Vault address	Does not automatically credit a particular pool.

For this linear-pool implementation, ordinary post-initialization `joinPool` and `exitPool` hooks are unimplemented and revert. What users may call adding or removing liquidity is normally implemented as a swap between main/wrapped tokens and the pool's preminted BPT. Initialization and recovery-mode exits are distinct cases. General statements about Balancer joins must therefore be qualified for these exact contracts. The pool's three possible token pairs are main/wrapped, main/BPT and wrapped/BPT. [Verified LinearPool swap and join hooks.](#)

Governed targets and fees are configurable, but not unrestricted. Targets must be whole normalized main-token units, ordered lower $\leq$ upper, with upper $\leq$ **4,294,967,295** whole main units. The public target setter also requires the current main balance to lie inside **both the current and proposed ranges**. The fee setter requires the current balance to lie inside the current range. Fees are bounded between 0.0001% and 10%. These state checks remain relevant even when an account has administrative permission. All three sampled main balances are below their existing lower target.

The owner getter returns `0xba1ba1ba1ba1ba1ba1ba1ba1ba1ba1ba1ba1b`, which the source expressly defines as the **delegated-owner sentinel**. It is not evidence of a private wallet holding ownership. Such actions are evaluated through the Vault Authorizer, reported as `0x6048a8c631fb7e77eca533cf9c29784e482391e7`. Main and wrapped token identities are immutable in this implementation. The Vault separately limits raw per-pool cash, managed and total balances to $2^{112} - 1$. None of these structural bounds is the small DAI headroom measured above.

Historical configuration and trades

The following DAI configuration events are present in successful receipts, but precede inherited Ethereum block 17,232,999. They must be attributed to the pre-fork history, not a later PulseChain intervention:

UTC date	Block	Event	Transaction
29 April 2023 14:49:23	17,152,449	Targets set to 0 / 6,000,000	Receipt
29 April 2023 15:58:59	17,152,788	Targets set to 1,000,000 / 5,000,000	Receipt
6 May 2023 12:26:59	17,201,543	Fee set to 0.00001	Receipt

Explorer queries for target and fee events across all three pools from block 17,233,000 through 27,659,393 returned no logs. A separate direct RPC `eth_getLogs` query confirmed the empty DAI result. This is bounded evidence about these event types and addresses, not proof that nothing else in their systems changed.

The main-token `lastChangeBlock` values lead to genuine post-fork transactions:

UTC date	Block	Publicly recorded swap	Transaction
29 June 2026 17:57:25	26,908,905	0.618019167905589973 waDAI → 5.200713616007117708 DAI	Receipt
4 September 2026 09:38:45	27,458,661	6.679668 waUSDC → 43.641283 USDC	Receipt

Both transaction receipts report success. Their top-level senders are respectively `0xe9608d7eec311931e0b69312f1fbfb55e0ea816` and `0x7e093ba1474b79481f9b87d66c99a819f25e82e2`. A top-level sender is not necessarily the ultimate beneficiary or source of the wrapped assets. No attribution to a named person or coordinated organization follows from these transactions alone.

A third successful transaction, on **10 August 2026 09:13:15 UTC**, block **27,251,196**, contains four USDT-pool swaps in sequence:

1. 26,235,296.082510 USDT → 3,074,846.802700 waUSDT.
2. 24,204,240.738844603002366863 BPT → 26,235,296.079992 USDT.
3. 0.002323348248838548 BPT → 0.002518 USDT.
4. 3,074,846.802700 waUSDT → 26,235,093.729348557560349311 BPT.

The USDT inflow equals the two USDT outflows in those swap events, so large gross turnover is compatible with the tiny ending main balance. This is direct evidence that main liquidity can enter and leave within one transaction and that BPT participates in linear-pool liquidity accounting. It does not establish creation of net new USDT, a bridge deposit, or a particular the profit. The top-level sender is `0x4bc27518f155f18f4519304335dc4e087daca6cf`. [Successful transaction and logs](#).

These observations show why a current-state capacity calculation and a historical transaction study answer different questions. Today's small residual balance does not bound yesterday's gross trading volume. Equally, a large historical gross amount does not establish independently available present liquidity.

Evidence and reproducibility

The method starts from the principal block's `getPoolTokenInfo` result, reads Vault logs for each returned `lastChangeBlock` and pool ID, and obtains each transaction, receipt and block header. Swap topics identify pool ID and token addresses; amounts are decoded using their respective decimals. Successful receipts and matching block hashes substantiate the quoted events. No transaction payload was constructed or submitted.

Generated evidence is under `ignored artifacts/parb-pool-limits-20260929/`: `vault_reads.json`, `last_balance_changes.json`, `config_receipts_and_block.json`, per-pool target/fee query responses, `vault_verified_source.json`, and extracted verified dependencies. The existing `artifacts/parb-validation-20260928/linear_limits.json` and `linear_v3_verified_source.json` provide the aligned target/rate and pool source evidence. The principal block hash was checked again in this research pass.

The broad historical DAI Swap explorer request timed out; this study therefore does not claim an exhaustive swap catalogue. Its narrower last-change-block RPC queries succeeded. DAI `PoolBalanceChanged` and `PoolBalanceManaged` explorer queries over the post-fork interval returned no logs, which does not imply inactivity: The confirmed linear-pool activity is represented by Swap events. USDC/USDT configuration absence was checked with the explorer, while DAI was also checked through direct RPC. These sources are not independent consensus verification.

The primary verified sources explain arithmetic and supported interfaces; the public transactions establish selected historical executions. Neither alone establishes the provenance of every input receipt, a working pARB Portal path, or economic backing outside these recorded contracts.

Appendix 17: Portal, waDAI and repayment: accounting of the proposed loop

Contents of this appendix

Finding.....	108
Contracts and evidence.....	108
Separate the balances before interpreting the transactions	109
What each proposed stage changes.....	110
Issuance and wrapping.....	110
Selling the wrapper	110
Portal settlement.....	110
Full versus partial settlement	111
What the earlier local experiments actually demonstrate.....	111
Third-party debt repayment and replenishment.....	112
Current-state and economic limits.....	112
Reproduction references.....	113

Source record: docs/studies/analysis/parb_portal_loop_accounting_20260929.md.

Portal, waDAI and repayment: accounting of the proposed loop

Research date: 29 September 2026. This study interprets verified source and existing recorded evidence. It performs no transactions, simulations or configuration changes. Historical PulseChain findings use chain 369, block **27,659,393**, hash 0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275.

Finding

The proposed sequence can transfer existing DAI from a Balancer pool into an Aave reserve while leaving newly issued aToken claims outstanding. Wrapping and trading those claims does not create the DAI used for settlement. Full settlement requires enough actual destination DAI to cover the newly recorded unbacked amount, plus any fee actually paid. If swap proceeds are smaller, a residual unbacked balance remains unless funded from another source.

Ordinary borrower debt and Portal unbacked issuance are separate balances. Paying off a borrow position can replenish reserve cash but does not automatically settle Portal or release its outstanding-issuance capacity. Third-party repayments can therefore improve liquidity without creating new protocol equity: cash replaces an existing loan claim. The same DAI cannot simultaneously be spent on ordinary repayment, Portal settlement, a separate escrow, and freely available price-support inventory.

These are conditional accounting conclusions. The sampled DAI/USDC/USDT Portal caps are zero; the bounded public event census found no Portal mint/backing events at this V3 pool after the fork. Earlier modified-local-fork experiments are demonstrations under supplied assumptions, not observed public executions.

Contracts and evidence

The relevant existing destination contracts are:

Component	PulseChain address
Aave V3 pool	0x87870bca3f3fd6335c3f4ce8392d69350b4fa4e2
Its sampled implementation	0xf1cd4193bbc1ad4a23e833170f49d60f3d35a621
DAI	0x6b175474e89094c44da98b954eedeac495271d0f
V3 aDAI	0x018008bfb33d285247a21d44e50697654f754e63
waDAI vault	0x098256c06ab24f5655c5506a6488781bd711c14b
waDAI/DAI linear pool	0x6667c6fa9f2b3fc1cc8d85320b62703d938e4385

The pool's verified source includes `BridgeLogic`, `BorrowLogic` and `ValidationLogic`. The waDAI implementation is an `ATokenVault` with an ERC-4626 interface and a receipt-deposit extension. This analysis uses those specific implementations rather than assuming that every contract called a bridge, wrapper or staking vault has the same rules. [Verified V3 source and dependencies](#), [verified wrapper implementation](#).

Separate the balances before interpreting the transactions

Use the following quantities for one underlying reserve, expressed in comparable underlying units at the relevant accounting time:

- c : actual underlying cash held by the reserve's `aToken` contract.
- b : ordinary borrower debt, including the applicable stable/variable interest.
- u : the reserve's recorded Portal unbacked amount.
- a : outstanding `aToken` claims, with treasury and interest accounting considered separately when needed.
- h : the actor's freely spendable underlying balance.

The table below describes principal changes, excluding intervening interest, rounding, treasury accrual and explicit fees. It is a change ledger, not a claim that four getter values always satisfy an exact global balance-sheet identity.

Operation of size x	Cash c	Ordinary debt b	Unbacked u	<code>aToken</code> claims a
Ordinary underlying supply	$+x$	—	—	$+x$
Ordinary borrowing	$-x$	$+x$	—	—
Underlying repayment of a borrow	$+x$	$-x$	—	—
Repayment using <code>aTokens</code>	—	$-x$	—	$-x$
Portal unbacked issuance	—	—	$+x$	$+x$
Portal backing of recorded issuance	$+x$	—	$-x$	—
Withdrawal against <code>aToken</code> claims	$-x$	—	—	$-x$

These distinctions follow from the pool's operation-specific libraries. Portal issuance does not create the recipient's ordinary variable- or stable-debt token. Instead, it increases the reserve-level unbacked accounting and issues the ordinary aToken. Conversely, backing that issuance does not burn the aToken already held by the wrapper, trader or pool. It supplies cash to support the existing claim. [BridgeLogic](#) and [BorrowLogic](#).

Zero reserve cash alone is not proof of zero reserve assets: outstanding loans can remain. Equally, a recorded loan claim is not immediately available cash for withdrawal. A cross-chain escrow, if assumed to exist, adds another asset/custody ledger; it is not automatically included in the destination Aave cash getter.

What each proposed stage changes

Issuance and wrapping

`mintUnbacked` issues the existing reserve aToken and increases `u`; it does not receive the destination underlying. It is restricted to the bridge role and subject to reserve supply validation and the reserve's unbacked cap. The sampled contract applies active/frozen/paused checks and the supply-cap check as part of that validation. This describes existing rules, not a recommendation to change them.

Depositing those aTokens into the existing waDAI vault transfers the aTokens into the wrapper and creates wrapper shares. At the protocol level, this moves the holder of the Aave claim; it does not create another independent underlying allocation. The wrapper's `totalAssets` is its aToken balance less claimable fees. Share conversion depends on this balance and share supply. "Static" describes the share representation, not a fixed one-share/one-DAI ratio, a guarantee of cash redemption, or a fixed dollar price.

The receipt-deposit path accepts aTokens directly. It is distinct from a wrapper deposit path that first supplies underlying to Aave. The direct receipt path therefore does not by itself fix the destination cash shortfall created by unbacked issuance. [ATokenVault source](#).

Selling the wrapper

A successful waDAI-to-DAI swap delivers DAI already available to the specific Balancer pool and leaves that pool holding additional wrapper claims. It does not cause the DAI token's total supply to increase or, by itself, transfer cash into Aave. The amount received depends on normalized wrapper shares, pool balances, fees and rounding; it cannot be inferred from the aToken symbol.

The pool study found a below-target nominal-balance constraint in the sampled linear pool, with approximately **0.055466 DAI** of output headroom at the principal snapshot. That is state-dependent output capacity, not a fixed lifetime issuance quota. Adding main liquidity through recorded pool accounting can increase it. Simply adding wrapper claims does not create new main-token inventory. See the [pool accounting study \(Appendix 16\)](#).

Portal settlement

The deployed `backUnbacked` path calculates the principal accepted as the smaller of the requested amount and current `u`. It decreases `u` by that principal, transfers actual underlying plus the supplied fee into the aToken contract, and distributes fee accounting between liquidity providers and protocol treasury. It is also bridge-role restricted.

The fee is a separate supplied amount. The bridge protocol fee parameter allocates part of that fee to the protocol; it is not, in this implementation, an automatic percentage charge on the backing principal. A zero fee was used in the earlier local experiments. Neither an oracle value nor remote escrow substitutes for the underlying transfer required by this settlement path. [Verified settlement code](#).

Full versus partial settlement

Let a cycle issue M underlying-equivalent $aToken$ units, obtain Y actual DAI from selling the corresponding wrapper shares, pay backing principal s , and pay settlement fee F . Ignoring other transactions and gas-token balances:

$$\begin{aligned}\text{Unbacked after} &= \text{Unbacked before} + M - S \\ \text{Actor DAI after} &= \text{Actor DAI before} + Y - S - F\end{aligned}$$

Settling the new issuance in full requires $s = M$, assuming the earlier unbacked balance is unchanged and separately accounted for. If the only funding is the sale proceeds, full settlement requires $Y \geq M + F$. If $Y < M + F$, the difference must come from another source or remain unsettled. A premium market price could in principle leave a trading surplus; a discount, fees or slippage can leave a deficit. This is an economic condition to measure, not a guaranteed result of wrapping.

If some purchased DAI is instead used for ordinary borrower repayment R , the spending ledger becomes:

$$\text{Actor DAI after} = \text{Actor DAI before} + Y - S - F - R$$

The principal R reduces the chosen borrow debt, not u . The choice to repay a borrow therefore does not settle the same Portal obligation a second time. An actor can make both payments if it has sufficient funding; their amounts cannot both be credited against one unit of cash spent.

Partial backing can release part of the unbacked cap, but repeated small deficits accumulate in u . A large cap is room for outstanding issuance, not a source of the missing settlement principal. Similarly, ordinary supply or repayment can raise cash without changing u or its remaining cap headroom.

What the earlier local experiments actually demonstrate

The old tests were conducted on a deliberately modified isolated fork with assumed permissions and altered configuration. They did not execute a `pARB` source deposit or message. Their locally generated transaction hashes are not evidence of public PulseChain execution. This study only reads their saved results; it does not rerun them.

The `local_portal_small.json` run reports two 0.01-DAI issuance cycles:

Item	Cycle 1	Cycle 2
aDAI issued in underlying units	0.01	0.01
Existing DAI obtained and used for backing	0.009999901128271614	0.009999902030089731
Cumulative remaining unbacked	0.000000098871728386	0.000000196841638655

The run's `passed` status means its assertions succeeded. It does **not** mean the issuance was fully settled. After two cycles it records 0.02 units issued, 0.019999803158361345 DAI delivered for backing, and a nonzero unbacked remainder. The actor spent the acquired underlying on settlement; it did not retain that same DAI as free inventory.

A separate `local_portal_fixed_time.json` run has slightly different accrued interest/timing results: backing amounts **0.009999900451908697** and **0.009999900902817507**, and cumulative unbacked **0.000000198645273796**. These are separate experiments, not conflicting values for one public transaction. Wrapper conversion, ongoing aToken interest and small residual aToken balances also mean that the units being compared must be explicit. Neither run removes its residual obligation by rounding it to zero.

The existing Curve round-trip experiment likewise consumes trading costs and leaves residual unbacked. It demonstrates a conditional destination sequence at the tested amount, not additional underlying creation or completed cross-chain backing. The appropriate evidentiary category is **historical local feasibility experiment**, not **observed on-chain operating loop**.

Third-party debt repayment and replenishment

A third party can repay a supported borrow on behalf of the debt beneficiary, subject to the repayment rules. For an underlying-funded payment, the payer's cash becomes Aave cash and the beneficiary's recorded debt is reduced. The protocol has exchanged a loan receivable for cash. That improves liquidity but does not create an additional independent asset of the same value. The payer has supplied the resources.

Such repayment can make previously illiquid aToken claims redeemable, subject to withdrawal rules and competing uses of the reserve. If an AMM or another holder can realize its wrapped claims into underlying, its available main liquidity may change. However, Aave cash is not automatically credited to a Balancer pool. There must be an actual, correctly accounted movement between the relevant contracts. The same point applies if a lender supplies fresh cash: that funding creates a new supply claim rather than free protocol equity.

Repayment using aTokens has a different effect: it extinguishes the caller's eligible supply claim and borrow debt without bringing in new underlying cash. It therefore should not be counted as cash replenishment. In either repayment mode, debt interest and scaled-balance events must be reconciled. A debt-token Mint event during repayment can represent interest crystallization rather than new underlying or a large payment. See the [public repayment study \(Appendix 9\)](#).

The documented public repayments provide examples of genuine externally funded debt service. They do not establish that a subsequent Portal issue, wrapper sale and backing payment were performed with the same funds. The linkage is a separate historical claim requiring transaction-level evidence.

Current-state and economic limits

At the principal snapshot, sampled DAI, USDC and USDT unbacked caps are **zero** and recorded unbacked amounts are **zero**. The bounded post-fork V3 event audit found no `MintUnbacked` OR `BackUnbacked` events. Existing source code supporting Portal and real wrapper trades therefore does not establish that this Portal loop has already operated. The old experiments' changed conditions must remain explicit when evaluating conditional feasibility.

A fresh getter-only batch at that same block confirms zero underlying cash for all three aToken contracts while their ordinary debt-token supplies remain large:

Reserve	aToken total supply	Configured supply cap	Variable-debt total supply
DAI	398,790,715.764328012268 937017	338,000,000	529,550,309.026518945728 832490
USDC	1,069,800,960.539750	1,760,000,000	1,281,493,069.377475
USDT	720,907,170.978766	200,000,000	911,981,635.742381

Stable-debt total supplies are zero. All three reserves report active, not frozen and not paused. These flags alone do not establish that a new supply or unbacked issuance passes the other checks. In particular, `validateSupply` checks:

```
(scaled aToken supply + accrued-to-treasury scaled amount)
  × next liquidity index + proposed new amount <= supply cap in underlying units
```

The actual code uses ray fixed-point multiplication and token-decimal scaling. The DAI and USDT displayed aToken supplies already exceed their respective caps before adding the treasury component or a new amount. Thus the supply-cap condition provides an additional barrier to new issuance at the sampled state, separate from the zero unbacked cap. USDC's displayed supply is below its cap, but the full predicate still includes treasury accrual and the proposed amount; that one comparison does not establish general issuance availability.

Interest can carry outstanding aToken or debt balances above a cap that restricts new activity. Above-cap balances therefore do not by themselves prove that a recent transaction bypassed the cap. Similarly, the large debt-token supplies are accrued accounting balances, not evidence of equally large new borrowing, cash currently held, recoverable dollar value, or available Portal capacity. The difference between debt totals and aToken totals is not an audited surplus: treasury claims, interest accounting, collection prospects and the wider reserve history have not been fully reconciled here.

A completed accounting cycle can relocate DAI from an AMM into Aave. It leaves the AMM holding claims instead of the DAI it paid out and leaves Aave with cash supporting aToken claims. It does not burn DAI total supply. The acquired DAI is also not permanently inaccessible: reserve borrowing and withdrawal rules still govern its subsequent use. If remote collateral remains locked, its custody, associated destination claims and eventual release remain a separate cross-chain balance sheet.

Economic support for the overall system can come from paid fees, realized investment income, external collateral, borrower repayments, contributed capital or favorable market prices. Each is an identifiable source or redistribution of value. Repeating receipt issuance, wrapping and sale alone is not proof of new equity, unlimited liquidity or an externally funded dollar peg.

Reproduction references

The source basis is the saved explorer response `artifacts/parb-debt-history-20260929/v3_implementation_source.json`, including the actual `BridgeLogic`, `BorrowLogic`, `ValidationLogic` and `debt` interfaces; and `artifacts/parb-validation-20260928/wav3_impl_source.json` for the wrapper. The pool and public debt studies linked above identify their separate read-only evidence batches. Public Portal-event absence and implementation checkpoints are recorded in `artifacts/parb-aave-history-20260929/`. The fresh same-block reserve totals and flags are recorded in `artifacts/parb-loop-accounting-20260929/reserves.json` and `reserve_summary.json`, with the read-only call manifest at `Research/experiments/parb_loop_reserve_accounting_20260929.json`.

The old experimental values come specifically from `artifacts/parb-validation-20260928/local_portal_small.json`, `local_portal_fixed_time.json` and `local_portal_curve.json`. Reading those records does not change their evidentiary status: they are modified-fork tests, not transactions found on the public chain. No source copies, bulk results or new state-changing experiment are added to this study.

Appendix 18: Reserve eUSD: bridged collateral and issuance of the existing pool token

Contents of this appendix

Principal findings	115
Exact contracts, versions and snapshots	115
Current registry and basket	116
Why a bridged token needs a collateral adapter	118
Basket governance and actual ownership evidence	118
Same-token issuance and conservation of backing	119
What this resolves, and what remains conditional	119
Reproduction and retained evidence	120

Source record: docs/studies/analysis/parb_reserve_bridged_collateral_20260929.md.

Reserve eUSD: bridged collateral and issuance of the existing pool token

Research date: 29 September 2026. This study examines a conditional architecture against the deployed copied Reserve v2 contracts. All new chain calls are getters; no governance changes, token issuance, administrative payloads or state-changing simulations were performed.

Principal findings

The copied eUSD issuer, Main, AssetRegistry and BasketHandler identify themselves as version 2.0.0. The complete registered token/plugin mapping and prime basket match at the inherited fork boundary and endpoint snapshot. The active basket contains saUSDC, cUSDC, saUSDT and cUSDT at the exact addresses recorded below. The existing Curve pool lists the same eUSD contract minted by this issuer.

The deployed issuer accepts the quantities required by its active basket; registration alone does not make an asset an active basket component. Its verified source distinguishes asset refresh, basket status, throttle capacity, issuance and collateral transfer. The cached basket status and stale price-feed observations below do not establish successful issuance at the endpoint.

These reads do not show newly bridged collateral installed in this eUSD basket. Direct e121 role checks are reported separately from the additional [Main and timelock governance-history audit \(Appendix 19\)](#). No finding about human ownership or participant intent follows from these reads.

Exact contracts, versions and snapshots

The endpoint is PulseChain chain **369**, block **27,659,393**, hash

0xc5e582cd902f4bff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275, **28 September 2026 08:22:35 UTC**.

Historical comparison uses the final inherited Ethereum block **17,232,999**, hash

0xe5667f9cafb115b1dcfc36b0b569a5601f01dfa000698140a1298ef4f87e2e37. Each batch checks its chain and rechecks its block header.

Component	Proxy address	Endpoint implementation
eUSD RToken	0xa0d69e286b938e21cbf7e51d71f6a4c8918f482f	0xecbba78d8cd654dfc582cb7fcf31d8a2a0b7a6cc
Main	0x7697ae4def3c3cd52493ba3a6f57fc6d8c59108a	0x143c35bfe04720394ebd18abeca83ea9d8bede2f
AssetRegistry	0x9b85ac04a09c8c813c37de9b3d563c2d3f936162	0x5a004f70b2450e909b4048050c585549ab8afeb8
BasketHandler	0x6d309297dddfea104a6e89a132e2f05ce3828e07	0x0ef3c7fe9c1cf7352d52e2867652b7547defdbe5

All four endpoint `version()` getters return **2.0.0**. The implementation slots were read at the endpoint and verified explorer source was retrieved for those exact addresses. Modern upstream deployment lists and the earlier convenience file containing Solidity 0.8.28 are not evidence for these deployed v2 semantics. [Verified RToken implementation](#), [verified Main implementation](#), [verified AssetRegistry implementation](#), [verified BasketHandler implementation](#).

The relevant BackingManager is `0xf014fef41ccb703975827c8569a3f0940cfd80a4`. Earlier same-block pool reads identify this exact eUSD address as coin zero of Curve pool `0xaeda92e6a3b1028edc139a4ae56ec881f3064d4f`; coin one is FRAXBP. Thus issuance by this RToken proxy preserves the token identity already recognized by the pool. Acceptance as a pool coin is not a guarantee of any particular swap size, price or underlying FRAX/USDC inventory. [Curve pool study \(Appendix 8\)](#).

Current registry and basket

`getRegistry()` returns the same fourteen ERC-20 addresses **and their plugin addresses** at both sampled blocks. The complete `getPrimeBasket()` response also matches exactly. These two snapshots establish unchanged endpoints; they do not exclude temporary intermediate changes or establish control of any wallet.

Registered symbol	Exact token	In active four-asset basket
eUSD	0xa0d69e286b938e21cbf7e51d71f6a4c8918f482f	No
RSR	0x320623b8e4ff03373931769a31fc52a4e78b5d70	No
stkAAVE	0x4da27a545c0c5b758a6ba100e3a049001de870f5	No
COMP	0xc00e94cb662c3520282e6f5717214004a7f26888	No
saUSDC	0x8f471832c6d35f2a51606a60f482bcfae055d986	No
saUSDT	0x21fe646d1ed0733336f2d4d9b2fe67790a6099d9	Yes
cUSDC	0x39aa39c021dfbae8fac545936693ac917d5e7563	Yes
cUSDT	0xf650c3d88d12db855b8bf7d11be6c55a4e07dcc9	Yes

Registered symbol	Exact token	In active four-asset basket
USDC	0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48	No
USDT	0xdac17f958d2ee523a2206206994597c13d831ec7	No
DAI	0x6b175474e89094c44da98b954eedeac495271d0f	No
USDP	0x8e870d67f660d95d5be530380d0ec0bd388289e1	No
TUSD	0x00000000000085d4780b73119b644ae5ecd22b376	No
saUSDC	0x60c384e226b120d93f3e0f4c502957b2b9c32b15	Yes

Two different addresses return the symbol saUSDC. Only 0x60c384...2b15 is in the active basket. The shared name is not evidence of fungibility or membership. Registered reward, governance and other assets are not automatically basket collateral. In particular, registering an asset can support trading or reward accounting without making it an issuance deposit.

The four active collateral plugins are:

Basket token	Plugin	Token decimals	Target amount per basket unit
saUSDC 0x60c384...2b15	0xe5a1da41af2919a43dac3ea22c2bdd230a3e19f5	6	0.25 USD
cUSDC 0x39aa39...7563	0x8a01936b12bcbeec394ed497600ede41d409a83f	8	0.25 USD
saUSDT 0x21fe64...99d9	0x7fdb32980861cc63751a0aea5a5b3ecb5119acd	6	0.25 USD
cUSDT 0xf650c3...dcc9	0x69bd37b82794d64dc0c8c9652a6151f8954fd378	8	0.25 USD

Each plugin reports `isCollateral=true`, `targetName=USD`, and `targetPerRef=10^18`. The basket nonce is three and its timestamp is **5 April 2023 07:22:35 UTC**. Basket status and all four plugin statuses return zero, the v2 SOUND enum, and `fullyCollateralized()` returns true. Main reports neither paused nor frozen.

Those getters must be read with the pricing evidence. All four plugin `price()` responses are **(0, 2^192-1)**, not a usable narrow price estimate. The USDC plugins reference feed 0x8fffffd4afb6115b954bd326cbe7b4ba576818f6; the USDT plugins reference 0x3e7d1eab13ad0104d2750b8863b489d65364e32d. Their eight-decimal feed answers are 99,983,300 and 100,074,527, respectively, but their `updatedAt` values are **1,683,705,647** and **1,683,732,947**, both 10 May 2023. Each plugin's `oracleTimeout()` is **86,460 seconds**. The endpoint timestamp is years later. This is direct evidence of stale feed timestamps and uninformative current price intervals, despite the recorded SOUND statuses.

Issuance calls `assetRegistry.refresh()` before checking the basket. Therefore the cached status, the basket quote and `fullyCollateralized()` do not prove a successful current issuance. No state-changing refresh or issuance was simulated here. The core implementation sources are verified; availability of exact plugin source is different: explorer lookups for all four plugin addresses returned address-only records without

verified source. Their successful getters therefore support the stated observations, while this study does not claim a complete source-level proof of each plugin's next refresh behavior. A returned status alone does not establish economic backing or successful future refresh.

Why a bridged token needs a collateral adapter

The verified registry maps an exact ERC-20 address to an `IAsset` adapter. `toColl` additionally requires that adapter's `isCollateral()` return true. The adapter supplies the token address and decimals, price bounds, collateral status, target denomination and reference-unit conversions. Registration calls its refresh logic; a label such as PRC-20 or ERC-4626 does not supply those semantics on its own. A token can have a standard transfer interface and still have incompatible collateral behavior.

The basket's target quantities are denominated in **target units per basket unit**, not arbitrary counts of bridged shares. A collateral adapter's `targetPerRef` relates target units to reference units; `refPerTok` relates reference units to each whole collateral token. Basket quantities and token decimals then determine the actual deposited raw integer amounts, with rounding. For yield receipts and bridge representations, these conversions must reflect the particular claim and its redemption/valuation semantics. Oracle control alone does not provide the missing underlying asset or validate those claims. [Asset interfaces and basket conversion code](#).

A governance-approved adapter and basket configuration could therefore represent a particular L1 bridge receipt backed by escrowed L2 collateral. A mintable bridge representation and an ERC-4626 receipt are candidate asset forms, not universal acceptance classes. The bridge must first deliver a token on the issuer's chain; Reserve then evaluates and takes custody of that exact token. If a receipt merely references the same collateral already promised to other independent claims, registering it does not cure duplicated backing.

Basket governance and actual ownership evidence

In the verified v2 components, the governance modifier checks `main.hasRole(OWNER, caller)`. `OWNER` is the right-padded bytes32 ASCII string `OWNER`, not its hash. Main's public role getters use the `_ROLE` suffix. The `OWNER` role is its own administrator and is also administrator of `PAUSER`, `SHORT_FREEZER` and `LONG_FREEZER`. The actual `getRoleAdmin` reads confirm that hierarchy. `register`, `collateral-plugin` replacement and prime-basket changes use this governance boundary; UUPS component upgrade authorization also uses it. [Main Auth and component authorization source](#).

The examined e121 address, `0x98c89980d1eae247ada3636ae27648f78e66e121`, returns **false** for all four roles. The same bounded checks return false for the modern upstream documented timelock `0x7bea807798313fe8f557780dbd6b829c1e3ad560` and governor `0xf4a9288d5deb0eae987e5926795094bf6f4662f8`. Those latter addresses are candidates from upstream documentation, not established PulseChain governance owners. The initial collateral batch did not enumerate role holders. The subsequent [governance audit \(Appendix 19\)](#) identifies timelock `0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c` as the sole `OWNER` in the returned Main role history and confirms the endpoint getter. Its proposer is Governor Alexios. Neither direct e121 role checks nor unchanged Main ownership establish who controls voting power in that governor.

Registry eligibility, configured prime basket and active reference basket are separate states. The verified source makes setting the prime configuration a governance action; an active-basket switch has its own conditions. A single-token basket is structurally possible because the configuration requires a nonempty valid collateral list, not a minimum of two constituents. It does not follow that one arbitrary token can replace the current four-component deposit without a valid configuration and active-basket transition.

Same-token issuance and conservation of backing

The v2 RToken's ordinary `issueTo` path requires a positive amount, an unpaused and unfrozen system, refreshed assets, a SOUND basket, issuance-throttle capacity and its exchange-rate invariant. For existing supply s , requested raw eUSD amount m and current required basket units b , it computes the additional basket requirement as the upward-rounded $b \times m / s$. It asks the active basket for a deposit quote rounded upward, increases required basket units, mints eUSD to the recipient and transfers **every required basket token** from the issuer to BackingManager. A failed transfer reverts the entire transaction, including the mint. When supply is zero, its initial basket relationship is handled by a separate branch. [Exact `issueTo` source](#).

Consequently, a bridged asset included with a partial weight is only one part of the required basket. The issuer must also supply the other constituents. If an authorized, valid active basket instead consists solely of that asset, ordinary issuance can use it alone. This changes the collateral composition of the same RToken; it does not give the bridge unrestricted mint authority over eUSD. The separate `mint` function in this source is restricted to BackingManager and is not a public arbitrary-collateral deposit function.

At the endpoint, eUSD total supply and `basketsNeeded` both equal **13,223,010.934604392856634411**, in their respective 18-decimal units. An exact one-basket quote using the v2 **CEIL enum value 2** returns 0.229775 saUSDC, 10.94460233 cUSDC, 0.224353 saUSDT and 11.20233014 cUSDT. These are receipt-token amounts reflecting their reference conversion rates, not four equal counts of USDC/USDT. The first inventory quote used ROUND enum value 1 only to identify the basket; the retained CEIL quote is the issuance-relevant rounding convention. A quote is a read-only amount calculation, not an execution success proof.

`issuanceAvailable()` returns **1,000,000 eUSD**. The configured throttle uses an absolute rate of 1,000,000 eUSD and a supply fraction of **2.5%**, with an hourly limit equal to the larger of those two quantities. It is replenishing issuance-rate capacity, not a lifetime one-million-token maximum, and it does not waive backing or status checks. The source has governance-bounded throttle configuration, but this study makes no assumption that e121 can change it. [Throttle library and issuer source](#).

Changing the configured basket does not transport new backing into BackingManager. Existing eUSD holders retain claims under the protocol while holdings and required composition may diverge. Rebalancing, collateral status, redemption and any recapitalization rules remain relevant. An intended new basket must therefore be distinguished from proof that it is funded and redeemable. The theoretical representation held by Reserve is a claim on the escrowed source asset; those layers do not constitute two independent pools of free collateral. Selling eUSD transfers the claim to its purchaser, while the backing commitment continues.

What this resolves, and what remains conditional

There is a coherent positive architecture: a bridge supplies a recognized L1 collateral representation, Reserve accepts that representation as valid basket collateral, and the existing eUSD issuer creates the same token already listed in Curve. The exact token-identity obstacle is resolved at the issuer boundary, not by a shared symbol or by having a bridge call eUSD mint directly.

For the actual sampled deployment, the registered mapping and prime basket remain the inherited configuration, direct e121 governance authority is absent, and present pricing evidence does not establish usable refreshed issuance. A specific bridged collateral contract, valid adapter, authorized configuration, actual backing custody, successful issuance receipt and subsequent public trade would be needed to turn the conditional architecture into an observed path. None of those missing facts can be replaced by the assertion that the destination pool lists eUSD. The conditional architecture is separate from the observed transaction record.

Reproduction and retained evidence

The explicit getter manifests are the six files named `Research/experiments/parb_reserve_collateral_{metadata,roles,plugins,inventory,inherited,feeds}_20260929.json`. Use `Research/scripts/probe_pulsechain_routes.py` with the explicit `--calls` manifest, matching `--block` and a fresh output path. The inherited manifest uses `0x106f467`; all other manifests use `0x1a60c81`. Do not substitute the runner's default experiment mode. Existing outputs are preserved.

Raw responses, verified core sources, source-query results and decoded summary are under ignored `artifacts/parb-reserve-collateral-20260929/`. The initial attempts to call `OWNER()` and similarly abbreviated role getters revert; verified source identifies the correct `_ROLE()` getter names and the later batch records their successful results. No missing or reverted result is treated as a zero-valued state. Source-fetch failures are distinct from successful on-chain plugin getters. No production application behavior changed, so no production test suite was run for this study.

Appendix 19: Reserve eUSD Main and timelock: recorded governance history

Contents of this appendix

Findings	121
Scope and pinned chain	121
Main ownership and role history	122
Owning timelock and proposer	123
Reproduction and limits	123

Source record: docs/studies/analysis/parb_reserve_governance_history_20260929.md.

Reserve eUSD Main and timelock: recorded governance history

Research date: 29 September 2026. This is a read-only audit of returned public role and timelock events, transaction receipts, verified source and fixed-block getters. It does not identify human wallet owners or assess intentions.

Findings

The sampled Reserve eUSD Main contract has **one OWNER in the reconstructed event history**, `0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c`. The endpoint `hasRole` getter confirms that role. The address has verified `TimelockController` source. Its ownership was established on **23 February 2023**, before the PulseChain fork. The Main role query returned 37 events from three inherited transactions and **no postfork RoleGranted or RoleRevoked event**.

The owning timelock's selected event query returned 70 events across 14 transactions. All were inherited: 37 scheduled calls, 25 executed calls, five role grants, one revocation and two minimum-delay changes. It returned **no postfork scheduled call, executed call, role grant, role revocation or delay change** through the endpoint. These observations do not show a postfork eUSD governance execution in this Main/timelock pair.

The timelock's verified proposer is `0x7e880d8bd9c9612d6a9759f96acd23df4a4650e6`, whose getter returns the name **Governor Alexios** and identifies this same timelock. Unchanged Main ownership and the absence of returned postfork timelock events do **not** exclude changed voting control of that governor. Governor proposal, vote, delegation and token ownership histories were not enumerated in this bounded study.

Scope and pinned chain

The chain is PulseChain **369**. Both event queries cover block zero through **27,659,393**, hash `0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275`, **28 September 2026 08:22:35 UTC**. The inherited-state comparison uses block **17,232,999**, hash `0xe5667f9cafb115b1dcfc36b0b569a5601f01dfa000698140a1298ef4f87e2e37`.

Each returned event was checked for inclusion in a successful receipt and for consistent transaction, receipt and block-header hashes. Both event queries reported zero RPC errors. The endpoint header was rechecked after collection. An empty postfork result remains scoped to the selected addresses and event signatures and the RPC's historical completeness; it is not proof about every Reserve deployment or every form of governance influence.

Main ownership and role history

Main is 0x7697ae4def3c3cd52493ba3a6f57fc6d8c59108a. Its OWNER role is the right-padded bytes32 ASCII string OWNER. The three recorded transactions are:

Date and time, UTC	Block	Main events	Recorded role effect
23 February 2023 00:45:23	16,687,620	9 grants, 4 revocations	Initial setup; OWNER ultimately held by 0x24d0...ed8e and 0x9494...e8c0 at this transaction's end
23 February 2023 00:53:11	16,687,659	8 grants, 5 revocations	OWNER granted to timelock 0xc8ee...6a2c; the two preceding OWNER roles revoked
23 February 2023 02:26:59	16,688,121	8 grants, 3 revocations	Freezer and pauser assignments changed; no OWNER change

The supporting receipts are [initial setup](#), [timelock ownership assignment](#), and [freezer/pauser assignments](#).

All eleven accounts appearing in Main role events were checked against all four roles at both the inherited boundary and endpoint: **44 hasRole responses per snapshot**. The two sets of responses are identical and match replay of the returned event history. The resulting current memberships are:

Role	Addresses with true endpoint getter
OWNER	0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c
SHORT_FREEZER	0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c; 0x4986164f2fb9bb898b19382b1e835cd9c81eba46; 0x4bad40e0d92ebc2620a0d5aff7887c7f2e67fdd8; 0xe45c3179b135288dd8e1e3c20eafebb2b2e7d771; 0x0d88776dd9a654cfe9c67b5b1d9ce2fddd815a34
LONG_FREEZER	0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c; 0x0173e2965c1aec9d395eb14e3407ef95c2e1a47d
PAUSER	0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c; 0x7f9ffa8dea49647725ca6ce621e03aa20401ff63; 0x4bad40e0d92ebc2620a0d5aff7887c7f2e67fdd8; 0xe45c3179b135288dd8e1e3c20eafebb2b2e7d771; 0x0d88776dd9a654cfe9c67b5b1d9ce2fddd815a34

The earlier [Reserve collateral study \(Appendix 18\)](#) separately records false direct role getters for the examined e121 account. This expanded history identifies the actual owning contract; it does not turn that earlier direct-role result into a finding about indirect voting control.

Owning timelock and proposer

Explorer source identifies `0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c` as `TimelockController`. Its role constants and all five distinct role/account pairs returned in its grant/revocation history were checked at the endpoint. Every check reconciles with the returned event history. [Verified owning timelock source](#).

Role or getter	Endpoint observation
<code>TIMELOCK_ADMIN_ROLE</code>	Timelock itself is true; setup contract <code>0x24d0aaac80a457be7843c59d45a1b90fbb02ed8e</code> is false
<code>PROPOSER_ROLE</code>	<code>0x7e880d8bd9c9612d6a9759f96acd23df4a4650e6</code> is true
<code>CANCELLER_ROLE</code>	<code>0x576ca79e46171a2b3e26f13a4334940ebcd72164</code> is true
<code>EXECUTOR_ROLE</code>	Zero address is true, the verified source's open-execution convention for otherwise eligible scheduled calls
<code>getMinDelay</code>	259,200 seconds, or three days
<code>Proposer name()</code>	Governor Alexios
<code>Proposer timelock()</code>	<code>0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c</code>
<code>Proposer token()</code>	<code>0x18ba6e33ceb80f077deb9260c9111e62f21ae7b8</code>

The latest event in the selected timelock history occurred on **13 April 2023 15:58:59 UTC**, block **17,039,497**, in [transaction 0x0018...6567a](#). The 25 inherited `CallExecuted` events target Main eleven times, AssetRegistry six times, BasketHandler three times, the governor twice, and the timelock, BackingManager and `0x90eb22a31b69c29c34162e0e9278cc0617aa2b50` once each. These are counts of recorded executed calls, not counts of proposals or proof of later activity. This study does not decode every inherited operation's business purpose.

Reproduction and limits

Explicit event manifests:

- `Research/experiments/parb_reserve_governance_history_20260929.json`
- `Research/experiments/parb_reserve_timelock_history_20260929.json`

Run these through `Research/scripts/collect_pulsechain_events.py` using `--manifest` and a fresh `--output`. Explicit getter manifests are `parb_reserve_governance_holders_20260929.json`, `parb_reserve_governance_holders_inherited_20260929.json` and `parb_reserve_timelock_holders_20260929.json` under `Research/experiments/`. Use `Research/scripts/probe_pulsechain_routes.py --calls` with the matching explicit block, rather than its default experiment mode.

Raw RPC records, source, summary and SHA-256 hashes are preserved under ignored `artifacts/parb-reserve-governance-history-20260929/`. The first timelock getter attempt failed at network connection before producing an evidence file; a subsequent identical read completed. No failed call was treated as a false role.

The study establishes recorded role assignments and selected governance executions for this eUSD instance. It does not establish absence of changed voting power, common wallet ownership, activity at another Reserve instance, or a running Reserve deployment on an unqueried pARB chain. No transactions, administrative actions or state-changing simulations were performed. Production code was not changed and production tests were not applicable.

Appendix 20: Post-fork history of specialized copied Arbitrum gateways

Contents of this appendix

Findings	124
Pin, coverage and method.....	125
LPT: an executed escrow release	125
ARB: previously identified issuance independently reconciled	125
Bounded negatives and recovered query failures	126
Consequence for the white paper.....	126
Reproduction references.....	126

Source record: docs/studies/analysis/parb_specialized_mint_history_20260929.md.

Post-fork history of specialized copied Arbitrum gateways

Research date: 29 September 2026. This study audits recorded PulseChain transactions and public read-only state. It neither constructs transactions nor infers participants' motives or independently verified source-chain backing.

Findings

The selected specialized gateways produced **two post-fork withdrawal events**: one LPT escrow release and the previously identified ARB issuance. The LPT transaction is an additional observed bridge execution, **not another mint**. No withdrawal event was returned for HGOLD, WETH, or the retired LPT gateway.

Together with the separately reconciled GRT transaction, the currently confirmed specialized destination issuance cases remain **ARB and GRT**. Source code also identifies LPT and HGOLD issuance branches, but source capability and historical execution must be reported separately.

Token / gateway	Returned post-fork withdrawals	What the transaction evidence establishes
ARB, 0xbbce8aa77782f13d4202a230d978f361b 011db27	1	Exact-token bridgeMint; zero-address Transfer and matching supply increase
LPT, 0x6142f1c8bbf02e6a6bd074e8d564c9a54 20a0676	1	Existing escrow transferred; no minter call; unchanged total supply
LPT retired gateway, 0xd82c27966ebb293b2d8646d8eacb293bc 260698e	0	No selected withdrawal event returned
HGOLD, 0x04bdbe0bc4edb62ea12070435ee67782b a292948	0	No selected withdrawal event or token zero-address Transfer returned

Token / gateway	Returned post-fork withdrawals	What the transaction evidence establishes
WETH, 0xd92023e9d9911199a6711321d1277285e 6d4e2db	0	No selected withdrawal event or WETH Deposit to this gateway returned

GRT's separately confirmed shortfall mint is described in the [gateway inventory \(Appendix 4\)](#). Its event and transaction were not counted again in the five-gateway search above.

Pin, coverage and method

Chain: PulseChain 369. Search interval: **17,233,000 through 27,659,393** inclusive, after the inherited Ethereum boundary. Endpoint hash: 0xc5e582cd902f4bff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275.

The event signature is `WithdrawalFinalized(address,address,address,uint256,uint256)`, taken from the retained verified gateway ABIs. For each returned event the collector checked receipt inclusion, receipt success and agreement among log, receipt, transaction and block-header hashes. It checked the pinned endpoint again after collection. Both gateway batches completed with **zero query errors**. Every returned withdrawal was examined, rather than sampling a subset.

Each observed withdrawal then received an independent transaction trace and historical `totalSupply()` reads at the preceding block and transaction block. Token zero-address Transfer searches provide a separate cross-check for ARB, HGOLD and LPT. WETH uses its `Deposit(address,uint256)` event, restricted to the selected gateway as the indexed recipient: treating every WETH deposit as a bridge mint would conflate unrelated native-currency wrapping with bridging.

This is a census of the named gateways and signatures within the stated range. It is not a chain-wide census of all bridges, tokens, issuance interfaces or unlogged actions. Returned history depends on the public RPC's completeness. The retired gateway was included because inspecting only currently selected router destinations can miss earlier integrations.

LPT: an executed escrow release

[Transaction 0x377c6bd1...](#) executed at block **21,402,997, 13 September 2024, 13:57:55 UTC**, hash 0x5488cda163f2cf7883c4aa3da3e71b79d2f0f9af61ff93910b3386e13963b9ba.

The receipt records **10,459,514.129760763228599483 LPT** moving from escrow 0x6a23f4940bd5ba117da261f98aae51a8bffa210a to 0x42543853a72edb6f1ff9bddab0cf4676187ec28a. The transaction sender is that same recipient address. The gateway emits `WithdrawalFinalized` for the amount.

The retained 13-entry trace passes through the outbox and bridge into the LPT gateway. The gateway calls the LPT token's `balanceOf` and `transferFrom`. **It does not call the external minter**. No zero-address mint appears in this receipt. Total supply is **24,918,514.377941353891894504 LPT** both before and after the block.

The verified gateway source has two branches. When escrow covers the incoming amount it transfers existing tokens; only a shortfall invokes the external minter's `bridgeMint`. This observed transaction executed the first branch. Accordingly, it does not resolve the separate question of whether the configured minter would accept that gateway's issuance call under other circumstances.

ARB: previously identified issuance independently reconciled

[Transaction 0xeebae5bc...](#) executed at block **21,469,490, 21 September 2024, 11:15:05 UTC**, hash 0xbbc9b58dce7934e35e967f4b96f8810c8d47625ffe6490eb4b1cfffdb9e195f60.

The 16-entry trace includes the reverse gateway calling the exact L1 ARB token's `bridgeMint`. The receipt contains a zero-address Transfer to `0x42543853a72edb6f1ff9bddab0cf4676187ec28a`. Its amount and the historical supply increase both equal **972,406,600,390,402,296,078,726,883,998.977902572465996426 ARB**.

The independent token-event search, completed as eleven disjoint block ranges, returned exactly this one post-fork zero-address Transfer. This confirms the previous destination-issuance finding. It does not independently establish source-chain collateral or a dollar valuation for the minted quantity.

The LPT and ARB transactions share the same externally visible sender/recipient address. This is an address-level relationship, not proof of a particular human owner or a common purpose.

Bounded negatives and recovered query failures

HGOLD returned no zero-address token Transfer and no selected gateway withdrawal for the full post-fork interval. WETH returned no selected gateway withdrawal and no Deposit with this gateway as recipient. The retired LPT gateway returned no selected withdrawal. These negative observations do not establish that the contracts are incapable of execution or that no other route exists.

Some independent token-log requests returned truncated JSON, literally `{"jsonrpc": "2.0", "id": 1`, rather than a valid result or structured RPC error. The original failures were retained and failed ranges were partitioned and retried. ARB's complete interval was recovered with million-block partitions. LPT required progressively smaller partitions, with the final 100,000-block failed interval recovered as ten successful 10,000-block queries. **No unresolved range gap remains in these selected token-log searches.**

The full post-fork LPT token search returned zero zero-address Transfers. This is a negative observation for that event signature and address, not proof that every possible form of supply accounting must emit that event. Independently, both LPT gateway-withdrawal searches completed successfully for the entire interval, and the sole returned withdrawal was reconciled as an escrow release. Failed responses were never treated as empty results.

Consequence for the white paper

The evidence supports a destination-accounting taxonomy grounded in actual transactions: **escrow release (LPT), direct token issuance (ARB), and authorized shortfall issuance (GRT)**. Native-currency wrapping is a separate source-defined WETH behavior for which this batch found no post-fork gateway execution.

An event named `WithdrawalFinalized` establishes neither new issuance nor its absence by itself. Classification requires the token movements, executed call path and relevant supply/accounting changes. Likewise, source containing a mint branch is insufficient evidence that the branch was used. This distinction allows the paper to acknowledge multiple specialized issuance mechanisms without attributing them to ordinary stablecoin routes or claiming unobserved backing.

Reproduction references

Gateway queries use the existing read-only collector: `Research/scripts/collect_pulsechain_events.py`, with manifests `Research/experiments/parb_specialized_mint_history_20260929.json` and `Research/experiments/parb_retired_lpt_history_20260929.json`. The independent token/trace/supply scope is recorded in `Research/experiments/parb_specialized_token_issuance_history_20260929.json`.

Raw request/response records, receipts, block headers, traces, historical supply reads and unsuccessful responses are retained under ignored artifacts/parb-specialized-mint-history-20260929/, with a SHA-256 file manifest. Gateway verified sources are preserved in the prior gateway-inventory batch; the retired gateway source is in the escrow-gateway-withdrawals batch. No public state changes, transaction simulations or signing were used in this study.

Appendix 21: Two examined wallets: recorded Aave debt, collateral and repayment rights

Contents of this appendix

Findings	128
Scope, provenance and decoding	129
Aggregate account data	129
Reserve-specific positions	129
0xe6df4ca976f3fec8da89efec332dc2245f0a86ff	130
0x6e3bf3c7bd91144d694621298cb8da370479a7bb	130
Collateral availability and repayment meaning	131
Existing post-fork repayment evidence	131
Implication for the proposed Portal and arbitrage interpretation	132
Reproduction references	132

Source record: docs/studies/analysis/parb_wallet_debt_positions_20260929.md.

Two examined wallets: recorded Aave debt, collateral and repayment rights

Research date: 29 September 2026. This is a getter-only public-state audit with retrospective interpretation of an existing repayment-log census. It submits no transactions, constructs no repayment or liquidation transaction, and performs no state-changing simulation.

Findings

The examined address `0xe6Df4Ca976F3feC8DA89EfEc332dC2245F0a86Ff` does have very large recorded debt in the sampled copied Aave V3 pool. Its variable-debt balances include approximately 136.369 million DAI, 260.160 million USDC, 186.361 million USDT and 43.154 million WBTC. The WBTC liability dominates its aggregate oracle valuation. Its sole nonzero aToken holding among the 17 reserves is approximately 105,219.941 aWETH, enabled as collateral.

The second address, `0x6e3BF3c7bd91144D694621298cB8dA370479A7bb`, has recorded DAI, USDC and USDT debt and holds aWETH/acbETH collateral. It also appears in the earlier public repayment census. Neither address has a healthy borrow position under the pool's sampled oracle accounting: both health factors are far below one. These are on-chain accounting facts, not market valuations or evidence identifying a human owner.

Repaying another account's ordinary borrow reduces that account's debt. It does not by itself transfer its collateral, ownership, borrowing rights or remaining aTokens to the payer. Portal backing and ordinary borrower repayment remain separate accounting operations. A profit claim requires an independently established right to receive assets and realizable proceeds exceeding every funding cost; the existence of a large debt balance establishes neither.

Scope, provenance and decoding

All new chain reads use PulseChain chain **369**, block **27,659,393**, hash

0xc5e582cd902f4bffa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275, timestamp **28 September 2026**

08:22:35 UTC. The header was rechecked after each batch. The V3 pool is

0x87870bca3f3fd6335c3f4ce8392d69350b4fa4e2. The reserve list comes from the same-block recorded

getReservesList census, and getReserveData identifies each reserve's aToken, stable-debt token, variable-

debt token and reserve ID. Balances are actual balanceOf results on those contracts; they are not inferred from explorer labels or transfers.

The 17 reserves examined are WETH, wstETH, WBTC, USDC, DAI, LINK, AAVE, cbETH, USDT, rETH, LUSD, CRV, MKR, SNX, BAL, UNI and LDO. Their underlying decimals and symbols were read directly. MKR uses a bytes32 symbol response, while the other sampled responses use the ABI dynamic-string representation. WBTC has eight decimals, USDC/USDT six, and the remaining reserves eighteen. The collateral flags are decoded from getUserConfiguration, using $\text{bit } 2 \times \text{reserveId} + 1$; the borrowing flag uses $\text{bit } 2 \times \text{reserveId}$. Both wallets report eMode zero.

The provider's returned oracle is 0x54586be62e3c3580375ae3723c145253060ca0c2. Its BASE_CURRENCY is the zero address and its BASE_CURRENCY_UNIT is **100,000,000**. Account-data values below are divided by that unit; health factors are divided by 10^{18} . A zero-address base convention does not provide external dollar redemption or establish that copied token market prices agree with the oracle. Feed freshness and external market depth were not audited in this bounded batch.

As a consistency check, both aggregate debt values match exactly the sum of $\text{floor}(\text{variableDebtRaw} \times \text{oraclePriceRaw} / 10^{\text{underlyingDecimals}})$ across reserves. The enabled-collateral holdings similarly reconcile exactly to reported total collateral. This independently ties the account totals to the individual receipt/debt-token reads. All 34 stable-debt balance reads return zero.

Aggregate account data

Value	e6Df...86Ff	6e3B...A7bb
Collateral, oracle base units	193,537,665.46258272	10,684,216.39305430
Debt, oracle base units	1,234,186,253,416.83358896	82,673,116.84503121
Available borrowing, oracle base units	0	0
Current liquidation threshold, basis points	8,250	8,249
LTV, basis points	8,000	7,999
Health factor	0.000129371538181202	0.106605513847397546

These interest-bearing liabilities are not a measure of tokens currently held by the borrower, the amount originally borrowed, or tokens newly minted in a single transaction. An origination chronology and interest-index decomposition would be needed to establish how each balance developed. This audit does not infer that the 43.154-million-WBTC debt corresponds to that many WBTC transferred at origination.

Reserve-specific positions

Only nonzero positions are printed below; each omitted asset had zero aToken, stable-debt and variable-debt balances for that wallet in the complete census. The aToken values are underlying-denominated Aave claims, not static wa-wrapper share quantities. Every stable-debt balance is zero. Receipt/debt contract addresses and raw integer amounts remain in the retained batch.

0xe6df4ca976f3fec8da89efec332dc2245f0a86ff

Asset	aToken balance, underlying units	Enabled collateral	Variable debt, underlying units
WETH	105219.940524291683911174	Yes	0
WBTC	0	No	43153535.13073855
USDC	0	No	260160228.034081
DAI	0	No	136368622.244154345895913053
LINK	0	No	3781427041.257192622440086794
cbETH	0	No	0.0000000000000001435
USDT	0	No	186361487.52217
CRV	0	No	22643463699.94498872590361026
MKR	0	No	2142427.95051214236040429
SNX	0	No	949827.511833239489551679
BAL	0	No	27743215.961357915553012945
UNI	0	No	10120349.82238671373623663
LDO	0	No	23080060.878290878526466789

0x6e3bf3c7bd91144d694621298cb8da370479a7bb

Asset	aToken balance, underlying units	Enabled collateral	Variable debt, underlying units
WETH	5807.947654522616321512	Yes	0
USDC	0	No	7708932.009274
DAI	0	No	50786176.522950319448694821
cbETH	0.679997896010046071	Yes	0
USDT	0	No	24156631.336535

Collateral availability and repayment meaning

Actual WETH cash in its aToken contract is **1,649,712.575569873755878469 WETH**; actual cbETH cash is **42,200.252108978475003934 cbETH**. Both exceed the respective combined aToken holdings of these two wallets at this snapshot. Thus a blanket claim that all collateral reserves have no cash would be incorrect. Conversely, DAI, USDC and USDT cash is zero. Reserve cash and a wallet's right to withdraw it are separate conditions.

The verified `V3 SupplyLogic.executeWithdraw` burns the caller's aTokens, transfers underlying to the selected recipient and applies the collateral health-factor/LTV validation when borrowing remains. The burn and transfer are within the same transaction: a later validation failure reverts the transaction.

`SupplyLogic.executeFinalizeTransfer` also subjects a collateral aToken transfer to health validation when the sender has borrowing. Calling a generic token withdrawal function on a separate contract does not remove these Aave checks. [Verified V3 source and dependencies](#).

Repayment can improve health and, if sufficiently large under the applicable prices and parameters, make some collateral withdrawable by its rightful holder. The exact amount depends on the entire remaining debt/collateral position, the asset being withdrawn and current validation rules. Reducing debt by a small amount does not automatically free the same amount of collateral, especially for these severely undercollateralized oracle-accounting positions.

In the verified `BorrowLogic.executeRepay`, underlying repayment transfers the payer's actual underlying to the reserve aToken contract and burns the beneficiary's selected debt. It does not transfer the beneficiary's collateral to the payer. A distinct liquidation operation has distinct rules and effects; it must not be described as ordinary `repay(onBehalfOf)` ownership acquisition. Credit delegation likewise authorizes borrowing subject to its own rules; it is not an ordinary ERC-20 transfer of an existing debt balance. [Verified BorrowLogic and SupplyLogic](#).

The representative DAI variable-debt token is `0xc8f8d0c70c850859266f5c338b38f9d663181c314`. Its implementation slot at the pinned block identifies `0xac725cb59d16c81061bdea61041a8a5e73da9ec6`. The verified `VariableDebtToken` implementation explicitly reverts with `OPERATION_NOT_SUPPORTED` in `transfer`, `transferFrom`, `approve`, `allowance` and `allowance adjustment` methods. Its balance records debt attributed to an account; it is not a freely transferable asset that a caller can take from a contract and thereby acquire collateral ownership. [Verified variable-debt implementation](#).

The DAI stable-debt token's sampled implementation is `0x15c5620dffac7c7366eed66c20ad222d1ed57`; the explorer returned an address-only record without verified source. This study therefore does not claim an equally verified current implementation-level transfer proof for that contract. Both wallets' stable-debt holdings are zero. Nor does the representative DAI source check constitute a full implementation census of all 34 debt-token contracts; the complete balance census and the source-check scope are distinct.

The further claim that `e6Df...86Ff` is a contract with a publicly callable token-withdrawal facility requires separate bytecode/source and access-control evidence. These balance getters neither establish nor refute that behavior. Even if such a facility exists, it does not by itself override transfer restrictions of the requested token or collateral validation in Aave. Any claim that a caller receives valuable assets after paying the contract's debt must establish that entitlement separately.

Existing post-fork repayment evidence

The existing V3 census covers blocks **17,233,000–27,659,393**, using the standard `Repay(address,address,address,uint256,bool)` signature. Its indexed addresses distinguish reserve, beneficiary and repayer. No new broad log query was needed for this wallet audit.

No returned standard Repay event names e6Df...86Ff as either beneficiary or repayer. This is compatible with its present debt and does not imply zero debt, no original borrowing, or no other debt-changing operation. For 6e3B...A7bb, 29 returned logs match at least one of those roles:

Role of 6e3B...A7bb	Asset	Events	Gross recorded amount
Repayer	USDT	9	2,161,000
Repayer	USDC	16	380,000
Debt beneficiary	USDT	8	2,160,000
Debt beneficiary	USDC	16	380,000
Debt beneficiary	DAI	4	0.000000000000000444

The role totals overlap and must not be added as independent payments. The USDT difference includes a **1,000-USDT** payment by 6e3B...A7bb for another account, c000...fd34. It demonstrates third-party repayment, not an associated collateral-ownership transfer. [Verified example receipt](#).

The DAI total is 444 raw wei. One 111-wei payment on behalf of 6e3B...A7bb generated a debt-token Mint event of approximately 4.805 million DAI because previously accrued interest exceeded the tiny repayment. The mint-event magnitude is not the amount paid. The earlier study reconciles interest, transfer and debt events in the receipt. [Example DAI receipt, repayment-history study \(Appendix 9\)](#).

Both bounded V2 `getUserAccountData` calls at 0x7d2768de32b0b80b7a3454c06bdac94a69ddc7a9 revert. The prior study records post-fork implementation changes at that pool. These failed getters establish unavailable standard-interface results, not zero V2 positions. This study makes no V2 position valuation.

Implication for the proposed Portal and arbitrage interpretation

These results strengthen the factual premise that some examined addresses have large outstanding borrow liabilities and collateral claims. They do not prove that those liabilities were acquired by transferring debt tokens, that a payer can obtain the collateral, or that the claims have realizable external-dollar value.

Ordinary repayment replaces a reserve loan claim with actual reserve cash; Portal backing reduces reserve-level unbacked issuance. Paying one balance does not automatically settle the other. A newly issued aToken sold to a pool leaves that pool holding an Aave claim. A subsequent repayment of a third party's borrow may add reserve cash, but its economic cost and the separately outstanding Portal balance remain in the accounting. The same token proceeds cannot fund both commitments twice.

A profitable arbitrage is therefore conditional on realizable relative prices, full settlement costs, fees, inventory, and an enforceable entitlement to the assets eventually received. Neither a large debt getter nor a public repayment function proves those conditions. The read-only findings support precise accounting analysis; they do not establish a completed self-funding cycle. [Portal accounting study \(Appendix 17\)](#).

Reproduction references

The maintained explicit getter manifests are `Research/experiments/parb_wallet_debt_metadata_20260929.json` and `Research/experiments/parb_wallet_debt_balances_20260929.json`, with the bounded source-provenance lookup in `Research/experiments/parb_wallet_debt_implementations_20260929.json`. Run each with `Research/scripts/probe_pulsechain_routes.py`, `explicit --block 0x1a60c81, --calls` pointing to that manifest and a fresh `--output` path. The explicit manifests contain getters only; this study does not use the runner's default experiment mode. Existing result paths are preserved.

Ignored raw evidence is under `artifacts/parb-wallet-debt-positions-20260929/`: `metadata.json`, `balances.json`, `decoded_summary.json` and `repayment_matches.json`. The latter derives only from the already recorded `artifacts/parb-debt-history-20260929/V3_repay_logs.json`. The `implementation/source` records supporting token-transfer semantics, where available, are retained in the same batch. Exact raw results, addresses, header hashes and errors remain available there; this study prints the decision-relevant positions rather than embedding source or the entire response catalogue.

Appendix 22: Withdrawal event coverage and historical supply reconciliation

Contents of this appendix

Event coverage	134
Supply reconciliation	134
Reproduction	135

Source record: docs/studies/analysis/parb_withdrawal_evidence_checks_20260929.md.

Withdrawal event coverage and historical supply reconciliation

Research date: 29 September 2026. This read-only study checks two parts of the existing copied-Arbitrum withdrawal audit: partitioned event coverage and before/after token supply. It uses the same fixed PulseChain endpoint, block **27,659,393**, hash

0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275.

Event coverage

The input is the four retained event batches described in the [escrow gateway census \(Appendix 11\)](#), [specialized gateway history \(Appendix 20\)](#), and [GRT reconciliation \(Appendix 12\)](#). These collectively contain 106 receipt-verified WithdrawalFinalized events in 78 transactions, covering ten identified historical gateway addresses and 91 token addresses.

For every gateway, this batch independently queried the same withdrawal signature in two adjacent, nonoverlapping ranges:

- Blocks 17,233,000 through 22,446,196.
- Blocks 22,446,197 through 27,659,393.

All **20 requests** returned valid results. The combined event identities match the prior full-range queries exactly: **no missing or additional events**. Identity includes emitter address, transaction hash, log index and block hash. An empty result for a partition was used only after a valid RPC response. This checks consistency across query partitioning; it does not independently validate the provider's chain history or cover different event signatures.

Supply reconciliation

The 106 events produce 106 distinct token-and-block pairs. The batch queried `totalSupply()` for each token at its withdrawal block and the preceding block: **212 successful, 32-byte return values**, with no reverts or missing results.

Result	Number of token/block pairs
Identical before/after supply	104
Positive supply difference	2
Negative supply difference	0

Result	Number of token/block pairs
Missing, reverted or malformed result	0

The two positive differences are:

Token	Block	Increase in whole-token units
ARB	21,469,490	972,406,600,390,402,296,078,726,88 3,998.977902572465996426
GRT	27,434,949	62,875,936.675361278837595998

Both use 18 decimal places and match the previously identified transaction's mint event and historical call trace exactly. No other sampled withdrawal block shows a net supply increase for its withdrawn token.

A block-level difference alone does not attribute a supply change to one transaction: unrelated transactions in that block can change the same token. Similarly, zero net change alone cannot exclude offsetting issuance and burning. The separate [full trace audit \(Appendix 23\)](#) examines all 78 actual transactions and classifies their observed gateway operations. Together, these checks strengthen the distinction between transfer-only withdrawals and the two observed mint executions. Neither check proves the unobserved source-chain burn or collateral history.

Reproduction

The maintained manifest is `Research/experiments/parb_withdrawal_evidence_checks_20260929.json`. Run:

```
python Research/scripts/check_pulsechain_withdrawal_evidence.py \
--manifest Research/experiments/parb_withdrawal_evidence_checks_20260929.json \
--output artifacts/parb-withdrawal-evidence-checks-new-run
```

The script accepts only `eth_chainId`, `eth_getBlockByNumber`, `eth_getLogs` and `eth_call`, uses no state overrides, verifies chain 369 and the endpoint header, and rechecks the endpoint after collection. It refuses to reuse an existing output directory and retains transport errors separately from RPC results. Raw requests/responses, decoded comparisons and file hashes are in ignored `artifacts/parb-withdrawal-evidence-checks-20260929/`. The successful live read-only run, arithmetic assertions, hash checks and Python compilation validated this research script. Production application code was not changed.

Appendix 23: PulseChain Arbitrum gateway withdrawals: historical execution audit

Contents of this appendix

Finding and scope.....	136
Executed gateway behavior	136
Implementations observed during execution	137
Cases requiring more than a selector count.....	137
Reproduction and retained evidence	138

Source record: docs/studies/analysis/parb_withdrawal_trace_audit_20260929.md.

PulseChain Arbitrum gateway withdrawals: historical execution audit

Finding and scope

Historical execution traces were obtained for **all 78 transactions containing the 106 withdrawals in the existing event census**. All 106 events match exactly one successful gateway finalization call by gateway address, token address, recipient and raw amount. There are no trace-fetch failures, unmatched events, trace identity mismatches or failed trace frames in this batch.

The executed paths comprise **104 transfer-only withdrawals, one ARB bridgeMint, and one GRT mint followed by transfer**. The GRT and ARB transactions remain the only demonstrated destination issuance cases within this finite census. This extends the earlier receipt screen into complete historical execution coverage; it does not establish completeness for other bridges, unidentified gateways, different event signatures or later transactions.

The inherited fork boundary is block **17,232,999**. The event interval is **17,233,000–27,659,393**, ending at block hash `0xc5e582cd902f4bffa3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275` on **28 September 2026 at 08:22:35 UTC**. The event catalogues cover ten gateway addresses. Seven contain withdrawals; HGOLD, WETH and the retired LPT gateway returned no matching withdrawals in the preceding event searches and therefore provide no execution frames for this trace batch.

Executed gateway behavior

Gateway family	Withdrawals	Actual token calls used for delivery
Standard ERC-20	93	93 transfer(recipient, amount) calls
Custom ERC-20	6	6 transfer(recipient, amount) calls
DAI	2	2 transferFrom(escrow, recipient, amount) calls
wstETH	1	1 transfer(recipient, amount) call

Gateway family	Withdrawals	Actual token calls used for delivery
LPT	1	1 <code>transferFrom(escrow, recipient, amount)</code> call; no external minter call
ARB	1	1 <code>bridgeMint(recipient, amount)</code> call
GRT	2	2 <code>transferFrom(escrow, recipient, amount)</code> calls; one also calls <code>mint(escrow, shortfall)</code>

The **105 transfer calls** consist of 100 `transfer` calls and five `transferFrom` calls. Every one has recipient and amount arguments exactly matching its withdrawal event. Of these calls, 103 return ABI-encoded `true` and two return empty data; none returns `false`. This establishes the calls, their successful execution and matching declared amounts. Balance and supply reconciliation is a separate check, particularly for tokens whose transfer semantics might differ from ordinary ERC-20 accounting.

The classification counts the initial call into a token once. An implementation `delegatecall` carrying the same selector is not counted as a second transfer or mint. Calls elsewhere in a transaction are not attributed to a gateway withdrawal merely because they occur in the same receipt.

Implementations observed during execution

These addresses come from the actual historical `delegatecall` frames, not from present-day proxy storage. Each listed proxy used the same implementation throughout the withdrawals observed for that gateway in this batch.

Gateway	Implementation actually executed
Standard <code>0xa3a7b6f88361f48403514059f1f16c8e78d60eec</code>	<code>0xb4299a1f5f26ff6a98b7ba35572290c359fde900</code>
Custom <code>0xcee284f754e854890e311e3280b767f80797180d</code>	<code>0xc8d26ab9e132c79140b3376a0ac7932e4680aa45</code>
wstETH <code>0x0f25c1dc2a9922304f2eac71dca9b07e310e8e5a</code>	<code>0xc4e3ff0b5b106f88fc64c43031be8b076ee9f21c</code>
ARB <code>0xbbce8aa77782f13d4202a230d978f361b011db27</code>	<code>0x4e8c24617c4592895d52ac6df29d9647e9d01bb5</code>
GRT <code>0x01cdc91b0a9ba741903aa3699bf4ce31d6c5cc06</code>	<code>0xd41ca6a1d034d178c196dfa916f22f7d1a1b8222</code>

DAI gateway `0xd3b5b60020504bc3489d6949d545893982ba3011` and LPT gateway `0x6142f1c8bbf02e6a6bd074e8d564c9a5420a0676` execute directly in these transactions. This observation about gateway execution does not mean that every downstream contract or token also executes directly.

Cases requiring more than a selector count

The [GRT mint transaction](#) contains both a mint and a transfer. Its mint is **62,875,936.675361278837595998 GRT**, while delivery is **62,875,937.675361278837595998 GRT**. Existing escrow supplies the difference of one GRT. Counting the full delivery as minted would overstate issuance. The [separate GRT reconciliation \(Appendix 12\)](#) contains the allowance, supply and receipt evidence.

The [January 2025 GRT withdrawal](#) uses the same gateway implementation but calls only `balanceOf` and `transferFrom` on GRT. It contains no GRT mint call. The existence of a gateway's mint capability cannot determine which funding branch a particular withdrawal actually used.

The [LPT withdrawal](#) similarly calls `balanceOf` and `transferFrom` on LPT without calling its external minter. The [ARB withdrawal](#) instead calls the exact destination token's `bridgeMint`, with no escrow transfer branch in that execution.

Two standard-gateway withdrawals, involving [sUSD](#) and [SNX](#), contain an additional call back into the token proxy with selector `0x907dff97`. The proxies' verified ABIs identify this function as `_emit(bytes,uint256,bytes32,bytes32,bytes32,bytes32)`. The recorded arguments specify the ERC-20 Transfer event topic, the standard gateway as sender, the withdrawal recipient and the transferred amount. These are token-proxy event emission calls, not additional mint calls. Their callers differ from the gateway, and their positions are nested under the already identified transfer. Sources: [sUSD proxy](#), [SNX proxy](#).

Reproduction and retained evidence

Run from the repository root:

```
python Research/scripts/audit_pulsechain_withdrawal_traces.py Research/experiments/
parb_withdrawal_trace_audit_20260929.json
```

The manifest names four existing event catalogues. It also identifies four retained historical traces for reuse: ARB, LPT, the examined MIM withdrawal and the September GRT mint. The other **74 traces** were fetched using `trace_transaction` from <https://rpc.pulsechain.com>, with a maximum of four concurrent requests. Existing trace files are reused on rerun. No transaction simulation, signing, state change or transaction submission occurs.

Raw request/response records, source responses, event-level classifications and a SHA-256 evidence manifest are retained under ignored `artifacts/parb-withdrawal-trace-audit-20260929/.audit.json`. `audit.json` preserves every event's gateway call path, exact-token calls, return data, gateway and token delegate targets, transfer source and argument comparisons. Each trace frame's transaction hash, block number and block hash is checked against the event. Error responses remain explicit errors rather than being interpreted as empty traces or absence of activity.

These traces establish destination execution. They do not independently establish the corresponding pARB source-chain transaction, source collateral or source-token burn. They also do not establish subsequent use of the issued tokens as collateral or in purchases of pDAI; those are separate transaction tracing questions.

Appendix 24: Aave receipts registered in copied Balancer pools

Contents of this appendix

Finding and coverage	139
Exact wrapper identities	139
Registered pool addresses	140
V3 reserve coverage and Portal status	141
Reproducibility and limits	141

Source record: docs/studies/analysis/parb_wrapper_inventory_20260929.md.

Aave receipts registered in copied Balancer pools

Research date: 29 September 2026. Read-only inventory at PulseChain chain 369, block 27,659,393, hash 0xc5e582cd902f4bffa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275.

Finding and coverage

The bounded factory census contains **17 direct Aave linear pools, nine distinct wrappers and six underlying assets**. Four wrappers refer to the sampled Aave V3 pool: DAI, USDC, USDT and WETH. Five refer to Aave V2: DAI, USDC, USDT, FRAX and TUSD. This is an inventory of exact registered token addresses, not a claim that all wrappers are presently redeemable or that their inputs came through Portal.

Nine historical canonical Aave/ERC-4626 factory families returned 38 pool creation events. All returned creations precede the PulseChain fork boundary; the latest is block 17,231,795. Their presence is inherited infrastructure, not proof of post-fork deployment. Other sampled pools use Sturdy, Morpho, Idle or mock receipts; an Aave-shaped interface alone does not make them direct claims on the sampled Aave pool. Custom factories outside this list remain outside coverage. The census does not enumerate native deployments on an unidentified pARB execution network.

Exact wrapper identities

The wrapper's aToken getter, the aToken's pool/underlying getters and Balancer Vault registration agree for each entry. V2 wrapper symbols can resemble the aToken symbol itself; names alone are insufficient to identify a wrapper.

Market	Underlying	Wrapper	Exact aToken	Registered pools
V2	DAI	0x02d60b84491589974263d922d9cc7a3152618ef6	0x028171bca77440897b824ca71d1c56cac55b68a3	3
V2	FRAX	0x3efceb450fb799f67cd97cdc27a0f4cd7b02b63b	0xd4937682df3c8aef4fe912a96a74121c0829e664	1

Market	Underlying	Wrapper	Exact aToken	Registered pools
V2	TUSD	0x236f4a95361e03267329a558bdd224b1f95f1377	0x101cc05f4a51c0319f570d5e146a8c625198e636	2
V2	USDC	0xd093fa4fb80d09bb30817fdcd442d4d02ed3e5de	0xbcca60bb61934080951369a648fb03df4f96263c	4
V2	USDT	0xf8fd466f12e236f4c96f7cce6c79eadb819abf58	0x3ed3b47dd13ec9a98b44e6204a523e766b225811	3
V3	DAI	0x098256c06ab24f5655c5506a6488781bd711c14b	0x018008bfb33d285247a21d44e50697654f754e63	1
V3	USDC	0x57d20c946a7a3812a7225b881cdcd8431d23431c	0x98c23e9d8f34fefb1b7bd6a91b7ff122f4e16f5c	1
V3	USDT	0xa7e0e66f38b8ad8343cfcf67118c1f33e827d1455	0x23878914efe38d27c4d67ab83ed1b93a74d4086a	1
V3	WETH	0x59463bb67ddd04fe58ed291ba36c26d99a39fb6c	0x4d5f47fa6a74757f35c14fd3a6ef8e3c9bc514e8	1

V2 pool: 0x7d2768de32b0b80b7a3454c06bdac94a69ddc7a9. V3 pool: 0x87870bca3f3fd6335c3f4ce8392d69350b4fa4e2. V3 wrappers expose ERC-4626-style asset conversion and refer to the exact V3 receipt. V2 static wrappers expose their own interface, including `ASSET()` and `LENDING_POOL()`; a reverting `asset()` call does not disprove their identity. The V2 pool's observed implementation changes require separate behavioral analysis before assuming ordinary legacy redemption still works.

Registered pool addresses

V2 DAI — wrapper 0x02d60b84491589974263d922d9cc7a3152618ef6:

- 0xa3823e50f20982656557a4a6a9c06ba5467ae908
- 0x804cdb9116a10bb78768d3252355a1b18067bf8f
- 0xae37d54ae477268b9997d4161b96b8200755935c

V2 FRAX — wrapper 0x3efceb450fb799f67cd97cdc27a0f4cd7b02b63b:

- 0x6a1eb2e9b45e772f55bd9a34659a04b6f75da687

V2 TUSD — wrapper 0x236f4a95361e03267329a558bdd224b1f95f1377:

- 0x454ed96955d04d2f5cdd05e0fd1c77975bfe5307
- 0x3bb22fc9033b802f2ac47c18885f63476f158afc

V2 USDC — wrapper 0xd093fa4fb80d09bb30817fdcd442d4d02ed3e5de:

- 0x652d486b80c461c397b0d95612a404da936f3db3
- 0x9210f1204b5a24742eba12f710636d76240df3d0
- 0x3cdae4f12a67ba563499e102f309c73213cb241c

- 0x82698aecc9e28e9bb27608bd52cf57f704bd1b83

V2 USDT — wrapper 0xf8fd466f12e236f4c96f7cce6c79eadb819abf58:

- 0xe6bcc79f328eec93d4ec8f7ed35534d9ab549faa
- 0x2bbf681cc4eb09218bee85ea2a5d3d13fa40fc0c
- 0x2f4eb100552ef93840d5adc30560e5513dfffacb

V3 DAI — wrapper 0x098256c06ab24f5655c5506a6488781bd711c14b:

- 0x6667c6fa9f2b3fc1cc8d85320b62703d938e4385

V3 USDC — wrapper 0x57d20c946a7a3812a7225b881cdcd8431d23431c:

- 0xcbfa4532d8b2ade2c261d3dd5ef2a2284f792692

V3 USDT — wrapper 0xa7e0e66f38b8ad8343cff67118c1f33e827d1455:

- 0xa1697f9af0875b63ddc472d6eebada8c1fab8568

V3 WETH — wrapper 0x59463bb67ddd04fe58ed291ba36c26d99a39fbc6:

- 0x60d604890feaa0b5460b28a424407c24fe89374a

V3 reserve coverage and Portal status

The sampled V3 reserve list contains WETH, wstETH, WBTC, USDC, DAI, LINK, AAVE, cbETH, USDT, rETH, LUSD, CRV, MKR, SNX, BAL, UNI and LDO. All 17 configurations encode an unbacked mint cap of **zero** at the pinned block. DAI, USDC, USDT and WETH have direct V3 wrappers in this census; no direct V3 wrapper was found for the other 13 within the nine surveyed factories. This is a bounded negative finding, not proof that no such wrapper exists anywhere.

Portal belongs to V3. A V2 wrapper does not accept a V3 aToken merely because both correspond to DAI or USDC. The matched V3 wrappers are structurally relevant to Portal because Portal issues their ordinary aTokens, but the observed zero caps and historical event search do not establish live Portal issuance. The [Aave history study \(Appendix 2\)](#) supplies that distinction.

Reproducibility and limits

Maintained call manifests are `Research/experiments/parb_factory_pools_20260929.json`, `parb_factory_wrappers_20260929.json`, `parb_wrapper_matches_20260929.json` and `parb_wrapper_registered_20260929.json`. They run with the read-only `Research/scripts/probe_pulsechain_routes.py` collector. Recorded responses, creation receipts and source references are under ignored `artifacts/parb-wrapper-inventory-20260929/`. Root review reconciled the nine wrapper links and all 17 Vault registrations directly from those responses and decoded the 17 cap fields; this is not a second independent RPC observation.

Factory coverage: 20211208-aave-linear-pool, 20220404-erc4626-linear-pool-v2, 20220425-unbutton-aave-linear-pool, 20220817-aave-rebalanced-linear-pool, 20221207-aave-rebalanced-linear-pool-v3, 20230206-aave-rebalanced-linear-pool-v4, 20230206-erc4626-linear-pool-v3, 20230409-erc4626-linear-pool-v4 and 20230410-aave-linear-pool-v5. Factory catalogue provenance is [Balancer deployments commit b55dc2b](#). The asset/share interface is defined by [ERC-4626](#). The deployed V3 DAI wrapper implementation is available from the [PulseChain source endpoint](#).

Public records were collected and the checks described above completed. A further automated review did not complete. The inventory therefore does not have a completed independent review.

Appendix 25: Executed arbitrage and Maker routes after shutdown

Contents of this appendix

What a closed loop proves.....	142
The mUSD trade closes in native PLS	143
A new Maker savings loop executed after shutdown	143
DOLA issuance and Maker conversion are separate	144
What the large DaiJoin balance means.....	144
Savings capacity and normalization	145
Maker USDC collateral settlement is a third route.....	145
StakeWise and real external collateral	146
pDAI connects the loops and PLS supplies purchasing capital.....	146
Next evidence and reproduction.....	147

Source record: docs/studies/analysis/pulsechain_arbitrage_and_maker_shutdown_20261003.md.

Executed arbitrage and Maker routes after shutdown

3 October 2026. Read-only PulseChain research. This study verifies closed trading cycles, separates their capacity from continuing financing, and resolves which Maker routes survived emergency shutdown. No transactions, signatures, execution simulations or state overrides were submitted.

Two historical transactions close in their starting currency with positive proceeds after transaction gas. The new example uses market-bought sDAI and Maker savings redemption after shutdown. Maker's ordinary USDC PSM remains disabled. Its savings redemption and separate collateral settlement are different routes with different resources.

What a closed loop proves

A trading loop starts with an asset, follows funded conversions, and returns to that same asset. Intermediate inventory must be accounted for, including share burns that do not emit ordinary token transfers. Borrowing principal, continuing debt, fees and gas must also be included.

For owned starting capital, the relevant result is:

Net result = closing asset received – starting asset spent – costs
– any new liabilities attributable to the operation.

That result can be checked exactly for an executed transaction. The next transaction faces the reserves and issuer capacity left by the first. Closing the trader's inventory does not restore all pools and issuers to their starting balances.

This distinction allows a precise deterministic thesis: **given specified balances, permissions, contract rules, trade size and execution order, the conversion quantities are determined.** A recurring parity policy additionally needs usable conversions and funded bids/offers after each operation. Arbitrage normally consumes a price discrepancy; perpetual profit is not required once parity is restored. New demand, opposite conversions, owned inventory or received funding may replenish capacity.

The mUSD trade closes in native PLS

The [September 30 transaction](#) funded a temporary constructor with **26,704.810084659172875544 PLS**. It bought pUSDC, bought mUSD through Curve, redeemed part into four underlying stablecoins, sold those coins and sold the remaining mUSD through other markets.

The extended reconciliation now checks **all nine ERC20 identities** involved at the trading address. Each has a zero getter balance before its acquisition and a zero final getter balance. All corresponding transfer inputs and outputs match. These tokens include mUSD, sUSD, pUSDC, pDAI, pUSDT, aUSD, BUSD, RAI and FEI; this accounting does not assume their raw units have equal value. The trace contains swaps, approvals and cached mUSD redemptions, rather than a borrowing leg. The constructor returns empty code.

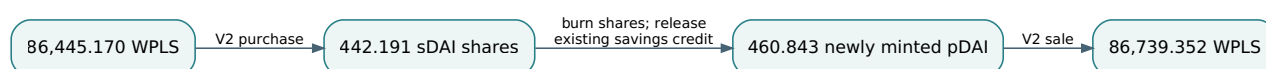
The recorded native payments account for the transaction's funding and proceeds. It returned **228,173.113310542683909513 PLS**. Transaction gas was **1,415.103716302431089681 PLS**. The resulting net native gain is **200,053.199509581079944288 PLS**.

This is an executed local arbitrage cycle. It does not establish dollar profit or group ownership of the trader. Its redemptions consumed issuer caches; the later small cache snapshot still makes replenishment the important capacity question. The [earlier route study \(Appendix 35\)](#) retains the detailed redemption quantities and contrasting lending-funded mUSD transaction, which left Compound debt.

A new Maker savings loop executed after shutdown

The latest selected Daijoin exit is [transaction 4c86](#), block **27,698,875, 3 October 2026 at 01:02:55 UTC**. The trader contract is `0xc6db262c884aafe11863e9feff91906c098339da`; the transaction sender is `0x61131e45c2ac7bcfd908d4d55656c8ff10b6bea9`.

Leg	Executed quantity
Wrapped PLS spent in a V2 market	86,445.169623647461048320 WPLS
Existing sDAI bought	442.190946786411992251 shares
Shares redeemed and burned	442.190946786411992251 shares
Existing pDAI minted through Daijoin	460.843244053775807041 pDAI
pDAI sold in another V2 market	460.843244053775807041 pDAI
Wrapped PLS received	86,739.352117403540056575 WPLS



Gross gain was **294.182493756079008255 WPLS**. Gas was **183.495108481921589452 PLS**, leaving **110.687385274157418803 PLS** after attributing the entire transaction gas to this operation. WPLS is the native PulseChain wrapper at 0xa1077a294dde1b09bb078844df40758a5d0f9a27. This is local PLS arithmetic, not an Ethereum ETH or US-dollar result. The WPLS gain stayed in the trading contract; the sender paid gas. The combined operation calculation does not establish common human ownership.

All bought sDAI shares are accounted for by its `Withdraw` event and burn path. SavingsDai's implementation reduces balances and supply without emitting a `Transfer-to-zero` for that burn. A ledger using only ERC20 `Transfer` events would incorrectly report leftover shares. All minted pDAI was sold. The transaction added no native principal and contains no observed borrowing leg.

The internal sequence is especially useful:

1. sDAI calls `Pot.drip`; Pot calls `Vat.suck` with **zero** additional credit.
2. `Pot.exit` releases the existing internal DAI credit belonging to the redeemed shares.
3. `DaiJoin.exit` transfers the caller's internal credit into DaiJoin and mints the corresponding pDAI.
4. The trader sells that pDAI for WPLS.

There is no `Vat.frob` in this route. Thus the observed mint is the conversion of an existing savings claim into ERC20 pDAI. It is not a new collateral loan or continuing post-shutdown savings yield.

The selected historical search found **709 DaiJoin exit logs** in contiguous ranges from block 23,225,242 through 27,699,029, with no query errors. The latest log was independently matched to its successful receipt, header and 42-call historical trace. The other 708 logs were not individually receipt-verified in this batch. This is an exit census, not a census of every possible pDAI mint.

DOLA issuance and Maker conversion are separate

The [executed October 2023 funding route](#) (Appendix 35) was:

Inverse governance mints DOLA at its existing token address

- Balancer and Curve pay existing pUSDC for DOLA
- Maker USDC PSM receives pUSDC collateral
- Vat records collateral and corresponding internal DAI debt
- DaiJoin mints existing pDAI.

Maker did not mint the DOLA. DOLA was not Maker's accepted collateral in this route. The **liquidity-pool legs** converted DOLA into pUSDC; the **Maker PSM leg** converted the accepted pUSDC into newly minted pDAI. The recorded pUSDC deposit and pDAI output were both **17,100,665.759301** normalized units. Other INV trades occurred in that transaction and belong in any full economic assessment.

After shutdown, trading DOLA for **already existing pDAI** through funded markets remains a separate possible route. Its usable output must be quoted and reconciled at the chosen size. The old `DOLA → pUSDC → ordinary Maker PSM` route fails at `Vat.frob` because `Vat.live` is zero. Zero PSM fees, available collateral tokens or an enabled DaiJoin do not remove that check.

What the large DaiJoin balance means

At block **27,699,029**, DaiJoin has:

Balance	Normalized quantity	Meaning
<code>pDAI.balanceOf(DaiJoin)</code>	0.1443398 pDAI	Direct ERC20 tokens held at the address
<code>Vat.dai(DaiJoin)</code>	128,302,945,566.5903780737557 02841 internal DAI units	Maker ledger credit at that address, normalized from 45 decimals

`DaiJoin.exit` does not hand out the second balance to arbitrary callers. It moves **the caller's** internal credit into DaiJoin, then mints ERC20 pDAI. The caller needs sufficient credit and the required Vat permission. `DaiJoin.join` performs the reverse conversion by burning pDAI and crediting internal DAI. The [primary adapter source](#) explains these operations; the saved deployed runtime exactly matches Ethereum's adapter runtime.

Current `DaiJoin.live` = 1 and its pDAI mint permission is retained. This explains the observed post-shutdown mint. It does not turn DaiJoin's ledger balance into a treasury available for fresh arbitrary issuance. Maker's `Vat.move` also checks the source account's authorization; being a Vat administrator does not itself bypass that transfer permission.

Existing authorized `Vat.suck` code can create internal credit and corresponding unbacked debt without a `live` check. That is a separate authority-dependent issuance path. The retained Pot remains a Vat-authorized contract, but its inspected public rate is **exactly 1 RAY** with `Pot.live` = 0; its observed drip creates zero new credit. End's Vat authorization also does not give every caller arbitrary access to `suck`. Effective control must include an actual callable path through an existing authorized contract. No group-controlled arbitrary post-shutdown issuance path was proved here.

Savings capacity and normalization

The pinned Pot holds about **106.980497 million internal DAI units**, covering its recorded savings claims at the sampled conversion rate. sDAI's own recorded claim is **3,207,016.56999736475838344 pDAI**, represented by about **3.077215 million shares**. These are nested claims on the same savings inventory; do not add them as two resources.

One sDAI share converts to **1.042181544879916519242831081 pDAI** at the checkpoint. It is a savings receipt, so the thesis's one-U comparison must normalize that exchange rate. Its unchanged accumulated premium is not proof of new interest after shutdown. Redemption reduces both its shares and the corresponding Pot credit.

The inspected source also provides the reverse deposit path, using DaiJoin and Pot credit transfers. Both directions require their token, permission and timing checks. This batch verifies actual redemption and sale; it does not claim to have executed or simulated a fresh deposit.

Maker USDC collateral settlement is a third route

The USDC join physically holds **113,692,361.582588 pUSDC**, while End has **zero currently allocated internal PSM-USDC-A collateral**. The PSM position still records **113,184,637.972615** collateral and debt units. These are different accounting locations.

Matched source and getter inputs support this proposed sequence:

```
End.skim processes the existing PSM position into End
→ holder converts pDAI into internal DAI using DaiJoin.join
→ holder authorizes and packs that credit into End
```


- End.cash allocates the configured prorated USDC claim
- USDC GemJoin.exit releases actual pUSDC against the holder's credit.

skim, pack and cash are public functions with accounting conditions. Skim arithmetic would allocate **113,184,637.972615** internal collateral units from this recorded position, assuming the inspected state remains unchanged. End and the USDC join retain the relevant Vat permissions. The join's exit is public but debits the caller's internal collateral; it does not expose all physical USDC to any caller. The sampled USDC pause/blacklist gates permit transfers. This analysis uses arithmetic and source inspection, without calling these execution functions.

The already fixed rate is **0.000879277935257435670774324 pUSDC per pDAI**. For a one-million-pDAI claim, the calculated transferable output is **879.277935 pUSDC** after rounding to six decimals, assuming sufficient allocated collateral and all remaining checks. This is a prorated shutdown claim, not one-U conversion or a dollar guarantee.

The user-identified 1967 wallet publicly called End.flow to establish that rate; its current packed balance is zero. That call proves preparation of the public settlement rate, not ownership of Maker governance or an executed USDC claim. The earlier study's selected Cash history contains no PSM-USDC-A payout. Actual processing and payout remain the next evidence for this branch. See the [primary USDC adapter](#) and [End source](#).

StakeWise and real external collateral

The [StakeWise study \(Appendix 63\)](#) proved replacement token logic and large local issuance into an existing Curve identity. It did not prove a changed Curve coin list. The refreshed getter still shows sETH2's incompatible Pool implementation; its supply getter reverts, and the examined pair still holds about 9.10 local stETH. That Curve implementation has fixed initialized coin identities and no Arbitrum price-feed hook.

An authorized compatible issuer could accept PLS or authenticated Ethereum ETH/WBTC representations, lock collateral and mint the existing sETH2 claim. Multiple collateral types would sit **behind one accepted token identity**. Directly sending an unrelated token into Curve does not make it its sETH2 coin. A proposed one-PLS conversion needs an actual mint/redemption rule and funded PLS reserves; local Lido already supplies a separately demonstrated PLS-to-stETH issuance branch under its limits.

StakeWise rETH2 and Rocket Pool rETH are separate contracts. Replacing StakeWise rETH2 does not control Rocket Pool rETH. Backing either claim with externally redeemable ETH/BTC requires proof of the exact bridge representation, external backing, issuer custody and withdrawal rights. Holding a copied token or pairing it with another asset does not establish those rights. Such real collateral could contribute independent resources to the coordinated network; the particular StakeWise collateral route remains a proposal.

pDAI connects the loops and PLS supplies purchasing capital

pDAI is a central conversion asset across FEI, the Synthetix DAI wrapper, Maker savings and Curve's shared bases. This supports the description of a fulcrum. The network also has routes centered on pUSDC and FRAXBP; pDAI's role is important without requiring every route to pass through it.

The new savings loop gives a measured distinction between arbitrage and accumulation. Its closing pool, 0xae8429918fdbf9a5867e3243697637dc56aa76a1, receives **460.843244 pDAI** and pays **86,739.352117 WPLS**. The trace's prior reserve getter and receipt's Sync event reconcile exactly. Its reserve-ratio price falls from **188.7666525414** to **188.7657455456 PLS per pDAI**, about **0.000480485%**, while the trading operation profits. This is one pool's ratio movement, not a statement about the aggregate market or dollar price. A separate operation could use the WPLS gain to buy and retain pDAI.

For a support campaign, the native-asset ledger should be:

```
Free purchasing PLS = owned liquid PLS + received rewards
                     + released owned principal + net market proceeds
                     - gas - debt service - other committed payments.
```

Received native emissions are distinct from ERC20 issuance. WPLS wraps existing native PLS; StakeWise token mints create receipt units. Selling those units into a funded market obtains that market's existing PLS. The three-trillion-token mint history is not three trillion newly created native PLS. Total chain-wide emissions are also different from the rewards actually owned and received by the proposed operator.

All free operator-owned PLS could be assigned to purchases under an explicit allocation policy. Actual output would follow the funded market curve, fees, available pDAI and the chosen target. The operator would retain only the resources left after those purchases for future bids and obligations. This is a financing option, rather than proof of control over all PLS or of a completed accumulation campaign.

The next discriminating evidence is **PLS source → spendable wallet or treasury → actual pDAI purchase → retained pDAI or funded support inventory**. Classify each flow as buying, selling or merely passing through pDAI. A changing ratio alone cannot distinguish swaps from liquidity changes, token supply effects or a moving external reference. Opposing conversions can replenish an issuer; conservation of the consolidated resources still applies.

Next evidence and reproduction

For continuing parity, follow the resources **after** a closed trade: refreshed quotes, issuer cash, outstanding claims, two-way conversion and an actual capacity-restoration event. For mUSD, that means cache replenishment or funded lending withdrawals. For Maker, follow sDAI/Pot credit and the two market reserves, and separately seek an actual USDC End processing/payout. For the L2 hypothesis, require a fresh authenticated destination delivery with its source obligations. For StakeWise, require working ERC20 logic and an actual collateral-to-token-to-funded-output transaction.

The new capacity checkpoint is chain **369**, block **27,699,029**, hash

`0x0d0c82f00bdd0048866b1a00f99e362a9c2a853d2d70d21cd11d76640e925ff3`, **3 October 2026 at 01:29:45 UTC**.

Historical trades retain their own checkpoints. Raw new evidence is in `artifacts/pulsechain-arbitrage-and-shutdown-20261003/`; earlier mUSD evidence remains in its original frozen batch.

`Research/experiments/pulsechain_arbitrage_and_shutdown_20261003.json`, `pulsechain_maker_shutdown_claims_20261003.json`, `pulsechain_sdai_shutdown_capacity_20261003.json` and `pulsechain_maker_usdc_exit_gates_20261003.json` contain explicit getter manifests. The separate `pulsechain_daijoin_shutdown_history_20261003.py` collects only the specified logs, headers, receipt and historical trace. Four adapter/savings runtimes were compared byte-for-byte against their Ethereum deployments; verified source records are retained with the requests.

Reproduce the **132 offline consistency checks** with:

```
python Research/scripts/analyze_pulsechain_arbitrage_and_shutdown.py \
  --input artifacts/pulsechain-arbitrage-and-shutdown-20261003 \
  --prior artifacts/pulsechain-equal-value-routes-20261003 \
  --output /tmp/pulsechain-arbitrage-and-shutdown-verified.json
```

The authoritative result is `verified-loops-and-price-effect.json`. The evidence inventory preserves hashes of raw files, reproduction inputs and reused evidence. These checks reconcile RPC evidence; they are not a cryptographic light-client proof or a complete wallet-ownership audit.

Appendix 26: Assertion acceptance, source balances and transferable claims

Contents of this appendix

What can be proved	149
Reproduction baseline	149
Contract argument	150
Why impossibility of challenge is not established	150
Four capabilities, not one	151
ERC20, vault shares and NFTs.....	151
Application to the thesis.....	152

Source record: docs/studies/analysis/pulsechain_assertions_and_receipts_20260930.md.

Assertion acceptance, source balances and transferable claims

30 September 2026. Source-level analysis supporting the coordinated pDAI proposal.

What can be proved

For the reviewed historical Nitro implementation, an assertion that passes the parent contract's admission and confirmation predicates can be confirmed without the parent re-executing its claimed L2 computation. If the assertion is invalid and no effective honest challenge prevents confirmation, the parent can accept an incorrect execution result. Its associated message root can then become an accepted Outbox root. This is a concrete consequence of optimistic verification.

The proposition has two independent premises: the ordinary contract predicates are satisfied, and no effective challenge or other intervention blocks acceptance. The absence of a challenge alone does not satisfy staking, ancestry, inbox, deadline or pause checks. A rival live stake can prevent confirmation even before a fraud-proof game finishes. Conversely, the mere existence of challenge code does not ensure that someone will use it successfully and in time.

This is stronger than saying validator control is irrelevant. It is narrower than saying validator keys confer unrestricted mint authority over every L1 token. Incorrect message acceptance can compromise a gateway's assumptions; the gateway still executes its own destination code.

Reproduction baseline

The historical cutoff is Ethereum block 17,232,999, 10 May 2023 at 22:36:11 UTC. The reviewed Nitro source is commit `3d90a211ff1a0b0bb3fceb103d3d375b636a211`, before that cutoff. It is an upstream code baseline, not a bytecode-equivalence certificate for every PulseChain proxy implementation at every later block. It does not import modern BoLD behavior into a 2023 fork. The earlier [historical bridge study \(Appendix 13\)](#) records the cutoff and repository selection.

The release preserves eight downloaded source documents and their SHA-256 hashes in `artifacts/pulsechain-coordinated-mechanism-20260930/sources/manifest.json`. The current Nitro white paper is supporting architectural documentation; the pinned contracts determine the historical function analysis.

Contract argument

Stage	Reviewed function	Enforced condition and implication
Participation	<code>onlyValidator, _newStake</code>	Admission requires an allowed validator or disabled whitelist, adequate stake and eligible staker status.
Assertion admission	<code>stakeOnNewNode</code>	Requires a stake on the prior confirmed ancestry, minimum timing, nonempty assertion and required inbox progress.
State commitment	<code>createNewNode</code>	Checks predecessor state hash, terminal machine status, inbox bounds and commitment consistency. It records the proposed after-state; it does not execute the complete claimed transition.
Confirmation	<code>confirmNextNode</code>	Requires elapsed deadlines, the correct predecessor and live staking conditions excluding unresolved competing live stakes.
Message publication	<code>confirmNode</code>	Matches the supplied block hash and send root to the stored commitment, then calls <code>outbox.updateSendRoot</code> .
Message execution	<code>Outbox.recordOutputAsSpent, executeTransactionImpl</code>	Checks membership in an accepted root and replay protection; supplies the asserted L2 sender context to the destination call.
Destination transfer	<code>L1ArbitrumGateway.onlyCounterpartGateway</code> and token-specific transfer logic	The bridge and counterpart sender must match. Escrow transfers need available tokens; reverse minting needs a token that recognizes that gateway's mint authority.

Primary sources: [RollupUserLogic](#), [RollupCore](#), [RollupLib](#), [Outbox](#), [L1ArbitrumGateway](#).

The decisive logical distinction is commitment versus correctness. A consistent hash commits the asserter to a claim. It does not establish that the claim follows from valid execution. Optimistic fraud-proof participation supplies that missing check. If this defense fails, a well-formed false claim may survive. The [Nitro white paper](#), sections 1.1, 2.2 and 5–6, states the honest-participant assumption and separates deterministic execution from confirmation to the parent chain.

Why impossibility of challenge is not established

The same pinned `RollupUserLogic` contains two public functions that can disable the validator whitelist: `removeWhitelistAfterFork` checks a chain-ID change against the implementation's immutable deployment chain ID; `removeWhitelistAfterValidatorAfk` checks elapsed inactivity. The latter uses the latest node creation block, the confirmation period and an additional 45,818 blocks. Block counts must not be converted to PulseChain time using an Ethereum block-time assumption.

If an inherited implementation retains an Ethereum deployment chain ID and the fork changes `block.chainid`, the fork predicate can matter directly. A current implementation replacement, proxy routing or different deployed version can change this assessment. No state-changing call was made in this study.

Therefore a proof that PulseChain challenges are impossible needs the deployed implementation and routing at the chosen block, actual whitelist state, applicable escape functions, challenge dependencies, staking access and transaction inclusion conditions. A quiet validator set, unavailable public RPC or absence of observed disputes does not settle those questions. This study proves the conditional acceptance mechanism, not permanent exclusive control in deployment.

Four capabilities, not one

1. **Claiming a result:** an operator can submit a claimed L2 result if admitted.
2. **Having the parent accept it:** the confirmation and challenge conditions apply.
3. **Using a source balance in applications:** valid execution requires token permissions, or a separately assumed modified source execution environment. Honest Nitro nodes deterministically computing the original rules do not acquire spendable balances merely because an incorrect parent assertion won.
4. **Receiving a destination asset:** an accepted message still reaches a particular receiver whose code determines transfers, minting, limits and failures.

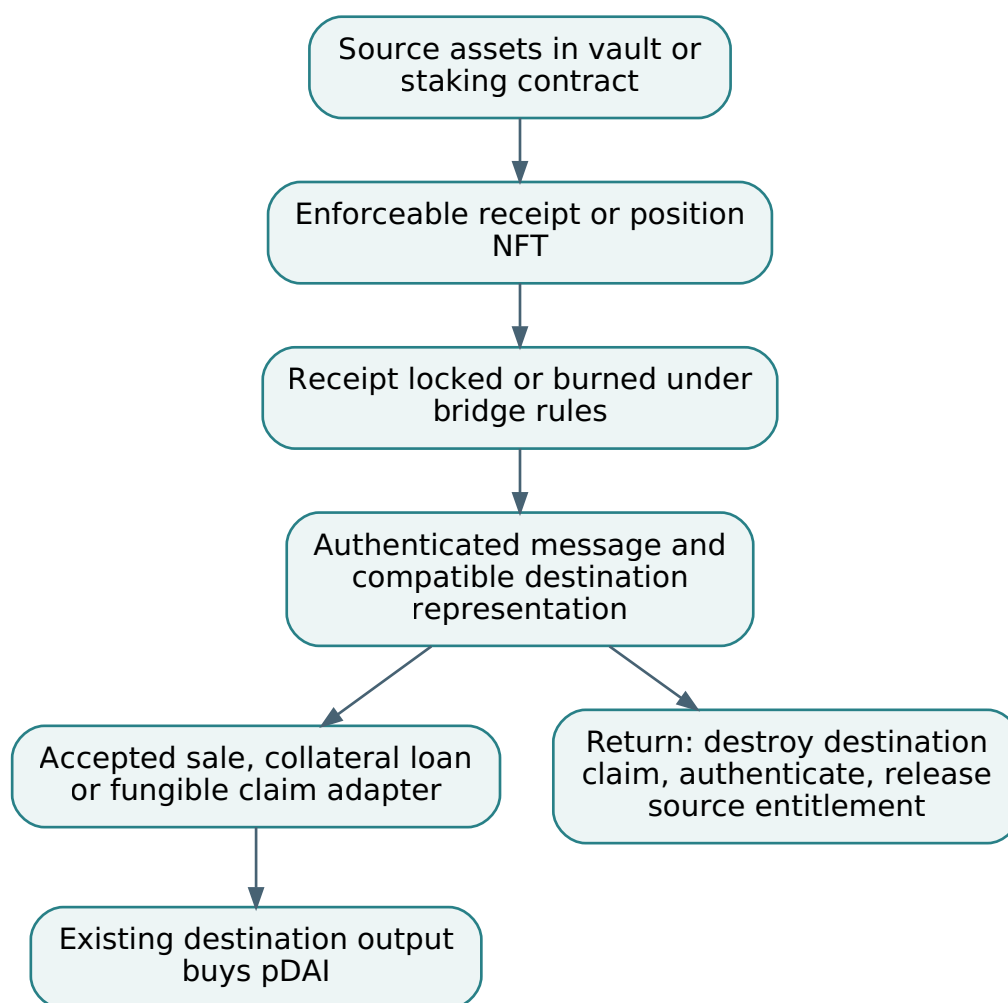
An accepted false message root is economically consequential even without an honest source application recognizing the claimed transaction. It should be described as a failure of the message-authentication assumption, not proof that ordinary source lending, staking and token issuance all executed successfully. The proposal may instead assume valid authorized source issuance and compatible receipts; that route does not depend on a false assertion.

ERC20, vault shares and NFTs

ERC20 assets can be bridged through suitable contracts. The historical `L1ReverseCustomGateway.inboundEscrowTransfer` calls `bridgeMint` on its mapped destination token. That is a concrete source-native representation mechanism, not a universal minter for arbitrary tokens already held by Curve pools. The function bodies establish direction; a contradictory comment in this source must not override them. [Pinned reverse gateway](#).

A vault or staking receipt can represent assets that remain invested on the source chain. Many such shares are ERC20s: [ERC-4626](#) standardizes tokenized vault shares using that interface. A position can instead be represented by an [ERC-721 NFT](#) or an ERC-1155 claim when the application defines the entitlement and a compatible bridge preserves exclusive ownership. NFT metadata alone does not enforce custody, withdrawal or cross-chain redemption.

The correct claim path is:



An NFT cannot be passed directly as the input coin of an ordinary fungible Curve pool. A buyer, lender, authorized redemption mechanism or fungible adapter must convert its economic entitlement into the exact accepted input token. Fractionalizing a position changes its interface, not the underlying assets or their withdrawal limits. The owner cannot sell an exclusive receipt and also count its underlying as freely available collateral elsewhere.

Section 3.2.5 of the Nitro white paper explicitly discusses NFT bridging and source-native tokens as possible bridge extensions. It does not specify a deployed pARB-PulseChain NFT liquid-staking integration. No such complete route has been established by this evidence set. Architectural support is documented; the particular contracts, ownership, custody and funded conversion are the remaining implementation evidence.

Application to the thesis

The strongest defensible proposal grants coordinated source operation, extensive issuance capacity and compatible destination acceptance, then traces the assets received and sold. Trillions of nominal source units can be an assumed input without assigning those units a dollar value. The capability argument explains why accepted claims could reach destination markets. The [network experiment \(Appendix 31\)](#) tests what renewed accepted inputs do to those markets after arrival.

Common beneficial ownership, ecosystem-aligned incentives and sustained cooperation remain explicit scenario premises where not evidenced. Neither arbitrary assertion capacity nor an NFT interface proves those premises.

Appendix 27: Atropa liquidity connections and routes into stability pools

Contents of this appendix

Direct findings	154
Observed trading and a large-route estimate.....	155
How the web can contribute to the proposal.....	155
Evidence and reproduction.....	157

Source record: docs/studies/analysis/pulsechain_atropa_liquidity_20260930.md.

Atropa liquidity connections and routes into stability pools

30 September 2026. Read-only study at PulseChain block **27,673,362**, hash
0x5c20cef7102df1d2519b28fbca0a884876ed59ecb9c01fd6b4ff033e9a1f5886.

Direct findings

Atropa token 0xcc78a0acdf847a2c1714d2a925bb4477df5d48a6 is connected to the copied DAI, USDC and WETH contracts through substantial PulseX reserves. The sampled V1 pools have most of their LP supply at the conventional dead address 0x00000000000000000000000000000000dead.

Pair	Pool address	Reserves in actual token units	LP supply at dead address
pDAI/ATROPA	0x5ef7aac0de4f2012cb36730da140025b113fada4	5,318,250,505.567230 pDAI; 259,558,327.341311 ATROPA	93.762001%
pUSDC/ATROPA	0xc636bfe0bae34824380b4e26bc34e4614e55e483	4,039,674,044.005196 pUSDC; 87,461,884.133066 ATROPA	96.442287%
pWETH/ATROPA	0x8f61d8fd20aef7f15019986ba277aeaf75b5062e	1,504,957,955.592714 pWETH; 397,854.331383 ATROPA	87.677239%

These are token counts, not dollars or claims on corresponding Ethereum assets. pWETH here is the copied 0xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2 contract. pUSDC is 0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48; pDAI is 0x6b175474e89094c44da98b954eedeac495271d0f. Token identity must not be replaced by a same-symbol externally bridged asset.

The balances substantiate extensive LP placement at a conventional burn sink, not complete burning of every share. They do not identify a common operator or prove who controls the remaining LP supply. One thousand raw LP units sit at the zero address in each sampled pair as minimum liquidity. Other pairs have different ownership: the sampled V2 Atropa pairs report zero LP balance at the dead address. Do not describe the entire graph as uniformly burned.

The two factories examined were `0x1715a3e4a142d8b698131108995174f37aeba10d` and `0x29ea7545def87022badc76323f373ea1e707c523`. All ten unordered pair combinations among pDAI, pUSDC, pWETH, ATROPA and WPLS returned a nonzero pair on each factory. The 20 resulting pairs are a bounded graph, not an exhaustive Atropa ecosystem census. Pair existence is not a minimum-liquidity guarantee.

Observed trading and a large-route estimate

A successful public transaction at block **18,108,015** traverses WPLS, pDAI, TEDDY BEAR, ATROPA, pDAI and WPLS, including the same V1 ATROPA/pDAI pool. This establishes historical use of the connection, not present profitability or linkage to a particular Curve deposit. [Transaction](#).

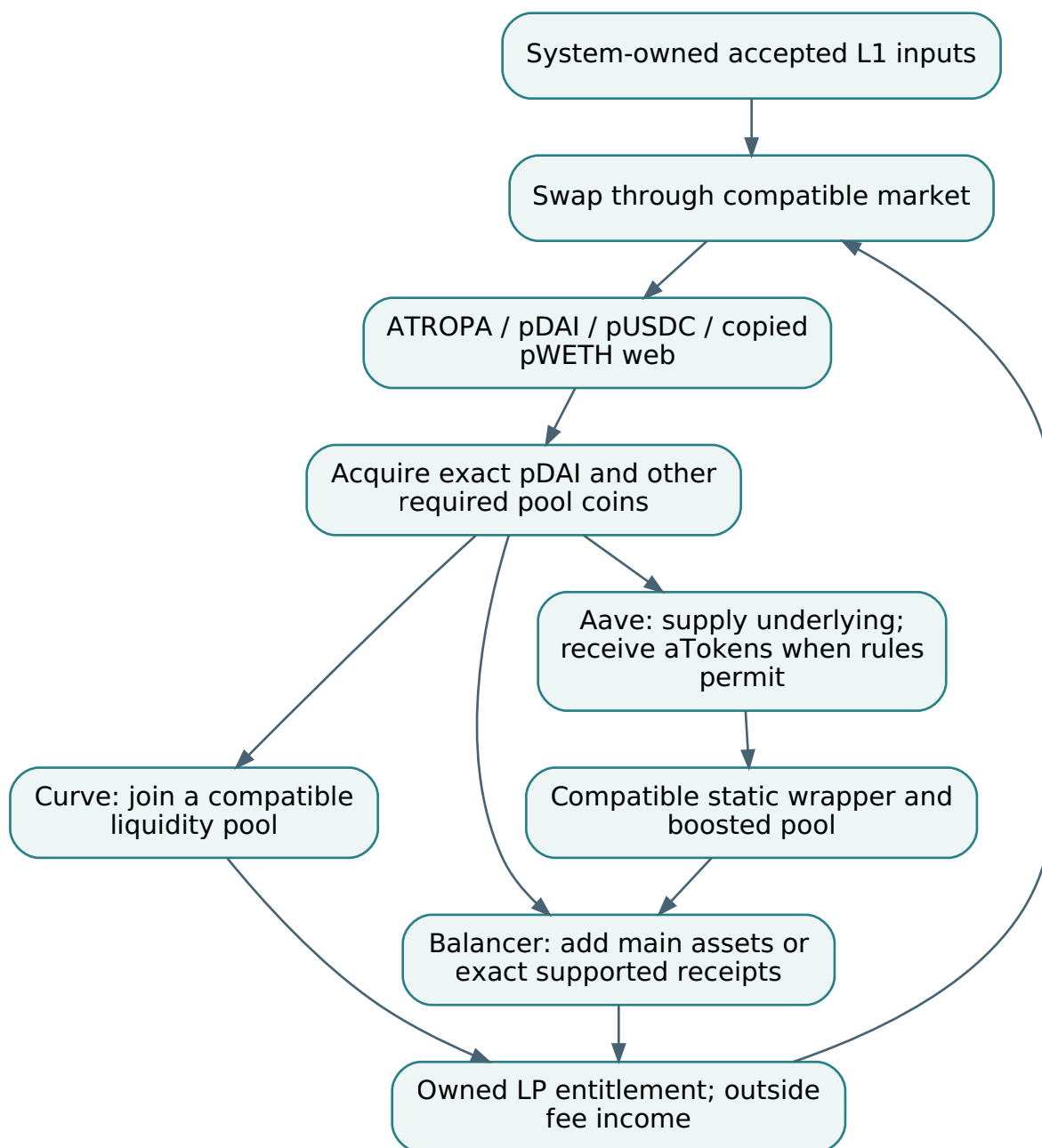
The published V1 pair source checks the constant-product invariant after a 29/10,000 input adjustment, corresponding to a 0.29% swap fee. Applying that formula to the pinned reserves gives the illustrative route:

```
50,000,000 pDAI
→ approximately 2,410,586.080356 ATROPA in the pDAI/ATROPA pool
→ approximately 108,047,516.402627 pUSDC in the ATROPA/pUSDC pool.
```

This reserve calculation precedes token-unit rounding, gas, MEV and token transfer-behavior checks. It is not a router execution simulation or a dollar quote. The surprising output ratio expresses the current relative token prices: it does not establish that pUSDC is worth one dollar or that a seller has received \$108 million. [Published pair source](#).

How the web can contribute to the proposal

The relevant operation is an exchange and subsequent deposit, rather than withdrawal of burned LP shares:



Buying pDAI from an Atropa pool transfers pDAI to the buyer and leaves the pool holding its accepted payment asset. Depositing that pDAI into a Curve pool moves existing inventory into a different market and creates the corresponding LP entitlement. No pDAI is created by the movement. If a round trip between the web and Curve produces a surplus after costs and restores its starting capital, that surplus can be reinvested. Buying an asset and holding it until a hoped-for re-peg is instead an inventory position until the closing trade occurs.

The existing burned LP positions may provide persistent venues because their associated shares are not normally available for withdrawal. Their reserves remain exchangeable and can become skewed or depleted on one side. The system can earn profits trading against those pools; it cannot collect their dead LP holders' fee entitlement as its own income. New system-owned LP positions have separate ownership and withdrawal rights.

ATROPA and copied pWETH are not automatically supported inputs to a stablecoin Curve pool. The exact destination coin must first be acquired. Every conversion changes reserves in the source web. Coordinating Curve, Balancer and Aave thus needs a consolidated inventory budget, not an assumption that each venue has received an independent copy of the same assets.

Evidence and reproduction

The fixed-block batches are `Research/experiments/pulsechain_liquidity_discovery_20260930.json`, `pulsechain_atropa_pairs_20260930.json`, and `pulsechain_atropa_reserves_20260930.json`. Reproduce each through `Research/scripts/probe_pulsechain_routes.py --block 0x1a64312 --calls MANIFEST --output FRESH_PATH`. The collector records requests and responses and rechecks the block hash.

The raw reads, parsed pair summaries, explorer transaction transfers, verified source and formula estimate are preserved under `artifacts/pulsechain-liquidity-expansion-20260930/`. Public explorer data supplied the discovery lead; pinned RPC getters establish reserves and LP balances. The evidence uses one public RPC and is not independent consensus verification.

Appendix 28: Canonical bridge flows, distributed accumulation and intent

Contents of this appendix

The strongest interpretation	158
Stocks and flows resolve the apparent contradiction	159
Does persistent exploitation imply zero?	159
What would strengthen the accumulation interpretation?	160
Review and preservation	160

Source record: docs/studies/analysis/pulsechain_bridge_flows_and_intent_20261003.md.

Canonical bridge flows, distributed accumulation and intent

3 October 2026. Focused five-analyst review of the bridge-persistence observation, following the integrated architecture assessment. Blockchain work remains read-only.

The observation has **meaningful conditional merit**. Persistent canonical bridge asset quantities despite completed withdrawals would require replacement inputs or other reconciled balance changes. That weakens a closed, unreplenished-drain explanation. A coordinated extract-redeploy-refill strategy is consequently a serious hypothesis. Stable bridge balances alone do not establish its controller, net capital accumulation or pDAI peg purpose.

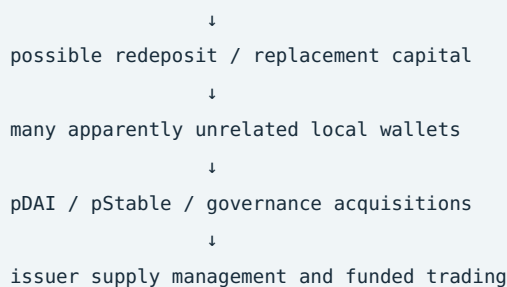
The user identifies the canonical **PulseChain-Ethereum bridge** as the capital boundary. Copied Across HubPool and the private-L2 branch are different objects. The reported three-year native-price decline, persistent bridge assets and substantial departures have not been independently reconstructed as a matched time series in the saved research. the sole-channel premise defines this review's perimeter rather than a verified census of every capital route.

The corrected operating model is **many apparently unrelated wallets with capital supplied just in time**. No central treasury, visible accumulator, identifiable cluster or persistent local stockpile is assumed. Completed execution still requires exact spendable inputs before or within the supported trade. Capacity is assessed across the permitted paths, with obligations and competing commitments counted once.

The strongest interpretation

Selected governance and issuance powers could turn copied-protocol opportunities into spendable PLS or stablecoins. Some PLS could be exchanged for genuinely bridged assets and withdrawn. External capital could subsequently return and fund acquisitions through different wallets. Other routes could fund local trades directly from source receipts or existing markets. The same network could acquire pDAI, neighboring stables, voting power and issuer collateral while public activity looks dispersed.

```
Local permitted issuance / arbitrage
      ↓
Spendable PLS or stables
      ↓
Optional sale for genuine bridged assets → Ethereum withdrawal
```



These are candidate connections, not a demonstrated closed canonical cycle. An unrelated depositor can supply the replacement capital. Acquisitions can be speculative or profit-seeking without a dollar policy. “Extraction” and stabilization are not mutually exclusive: monetizing one venue could finance support elsewhere. The relevant question is the net resource effect and subsequent use.

A local receipt sale obtaining PLS does not itself remove Ethereum escrow. A further exchange for a valid bridge claim and completed release is another leg. Minting copied pDAI is different from obtaining an Ethereum-backed canonical claim. The [saved closed trades \(Appendix 25\)](#) and [funding conversions \(Appendix 36\)](#) prove local operations, not that additional exit-and-refill sequence.

Stocks and flows resolve the apparent contradiction

For each exact Ethereum-origin asset in the canonical custody perimeter:

Closing quantity = opening quantity + deposits – completed releases + reconciled other changes.

Other changes require classification: direct transfers, fees, losses, asset migrations or balance-changing token behavior. Outstanding destination claims, pending messages and payout obligations need a separate ledger. A deposit replacing withdrawn backing can create a replacement liability rather than surplus capital.

If quantities are stable while verified releases occur, offsetting quantities must enter or another measured change must explain the balance. That is the observation's strongest implication. If only aggregate dollar TVL stays flat, appreciation of ETH or other assets can offset withdrawals; substitution between assets can also mask one reserve's decline. Compare exact token quantities and independently valued flows, not just a dashboard total.

Gross turnover can be large when the same capital repeatedly exits and returns. Replacement inflow is therefore different from positive net external commitment. Stable bridge quantities can coexist with changing ownership, local liquidity depletion or increased claims. Native PLS price and market capitalization have different denominators from bridge backing: bridged capital can remain while buyers prefer other assets or native selling overwhelms native demand.

Does persistent exploitation imply zero?

Not as a logical necessity. A death spiral is plausible when recurrent sales or losses outrun renewed demand and replenishment, valuable trading reserves decline, and confidence or useful activity collapse. Persistent vulnerable code is only one part of that chain.

Realized extraction can be limited by funded counterassets; a once-profitable opportunity can disappear even if its mint permission remains. Losses may be concentrated in particular pools. Independent demand, replacement capital, market making and surviving uses can sustain other activity. Those are counterexamples to inevitability, not a forecast or guaranteed positive floor. A falling price is compatible with an adverse extraction scenario without establishing it.

The bridge observation accordingly weakens **unopposed depletion**, while leaving **extraction subsidized by new entrants**, ordinary inventory recycling and coordinated accumulation as competing explanations.

What would strengthen the accumulation interpretation?

The most useful measurement combines two layers:

1. A matched per-asset canonical ledger of custody, destination claims, finalized gross deposits/releases, pending obligations, other adjustments and independent price revaluation. This establishes actual replacement, net commitments and coverage.
2. Aggregate local flows around funding arrivals: exact pDAI and neighboring-stable buys/sells, issuance/burns, issuer custody, LP changes and funded executions across wallets. Distinguish net acquisition from transfer churn, pledged backing and liquidity removal.

Repeated arrivals followed by distributed pDAI acquisitions and compatible protocol operations would support the economic pattern. Execution permissions, authenticated instructions or committed funding could provide separate coordination evidence without identifying people. Timing and matching amounts alone also fit market makers.

For **peg intent**, the additional discriminator is recurring two-sided execution that responds to an independent dollar reference, including after adverse moves, with funding available on time and capacity restored. Refill plus rising relative price is stronger evidence of resource rotation or accumulation than of a specified \$1 policy. Independent arbitrageurs can collectively create some of these effects; measured behavior and common control remain distinct claims.

Review and preservation

The mechanism, economics, attribution, routes and clarity analysts evaluated the same frozen intent input and the subsequent distributed-wallet correction. They used saved evidence and conditional reasoning, without new blockchain calls or historical-series claims. The separate experiment record is `Research/experiments/pulsechain_bridge_intent_review_20261003.json`; the five Markdown/JSON reports, inputs, findings and hashes are in `ignored artifacts/pulsechain-intent-review-20261003/`.

This focused review does not alter the fifteen frozen assessments or ballots of the [three-round architecture study \(Appendix 70\)](#). The [maintained white paper \(main paper\)](#) now uses distributed execution capacity and incorporates the canonical-bridge intent test. The original private recovery records, private continuity notes and research ZIP remain preserved.

Appendix 29: PulseChain thesis evidence and authority

Contents of this appendix

Reading the evidence.....	161
Named contracts and accounts	162
Changes and cooperation required.....	163
Supply headroom at the actual mint	165
Exact examples of token delivery	165
Proof map for the mechanism	166
Issuer-collateral extension.....	170
Complete route and operating design	171
Integrated network extension, 1 October 2026	172
Remaining proof requirements	173

Source record: docs/studies/analysis/pulsechain_claims_and_authority_20260930.md.

PulseChain thesis evidence and authority

30 September 2026. This map separates observed contract state, published source behavior, conditional arithmetic, scenario assumptions and unresolved execution. It does not identify private-key owners, certify economic guarantees or authorize any blockchain changes. All new chain requests in this review are getters, contract-code reads or storage reads.

Reading the evidence

Later evidence in the [1 October Safe/rollup audit \(Appendix 52\)](#) identifies the Arbitrum timelock and security-council executor roles and verifies externally owned address control of the UpgradeExecutor and key bridge proxy admins. Statements below about unresolved role holders describe this older snapshot's verification status; the later audit supplies the extended authority map without attributing private-key ownership.

The main observation snapshot is PulseChain block **27,675,130**, hash `0xdd12f23ad571882b63fd33c5d3909a45a05fe6f0a3c2b273cb7e1e6b0917954`, at **30 September 2026 05:20:55 UTC**. Separate older observations retain their own block numbers. A contract address observed on PulseChain refers to that chain, even when the address also exists on Ethereum. Published verified source supports contract interpretation; this review has not independently compiled all sources and matched every deployed bytecode dependency.

Raw getter records are in `artifacts/pulsechain-deferred-campaign-20260930/` and the new `artifacts/pulsechain-three-round-review-20260930/`. Each request records its target, calldata, pinned block and RPC response. The three new authority manifests are under `Research/experiments/` with stems `pulsechain_review_authority`, `pulsechain_review_authority_details` and `pulsechain_review_owner_storage`, all dated 20260930. Source interpretations cite the published `sourceCode` contents, not imaginary line numbers in JSON.

Named contracts and accounts

Name	Full PulseChain address	What is established
Aave V3 pool	0x87870bca3f3fd6335c3f4ce8392d69350b4fa4e2	Examined destination reserves and debt.
Pool implementation	0xf1cd4193bbc1ad4a23e833170f49d60f3d35a621	Implementation slot at the snapshot; published source saved.
Linked Portal library	0x57572c9e795f4b6a748efbeab7e0a1b9996a0a24	Pool-linked BridgeLogic; code and published source saved.
Addresses provider	0x2f39d218133afab8f2b819b1066c7e434ad94e9e	Getter identifies ACL, configurator and ACL administrator.
ACL Manager	0xc2aacf6553d20d1e9d78e365aaba8032af9c85b0	Checks roles; manages grants under role administrators.
Pool Configurator	0x64b761d848206f447fe2dd461b0c635ec39ebb27	Provider getter; cap-setting entry points in saved source.
Short executor	0xee56e2b3d491590b5b31738cc34d5232f378a8d5	Provider owner and ACL admin; has PoolAdmin and default ACL admin roles, lacks Bridge at the snapshot.
Current executor administrator	0xec568fffb86c094cf06b22134b23074dfe2252c	getAdmin() identifies the governance contract.
Nominated executor administrator	0xa30190b96faee0080144aa0b7645081fcbf49e6f	getPendingAdmin(); no contract code at the snapshot; directly lacks PoolAdmin.
Examined borrower called 86ff	0xe6df4ca976f3fec8da89efec332dc2245f0a86ff	Contract code; twelve nonzero V3 debts and an owner() response.
Borrower reported owner	0x37c843419218430619464fbea890f609537fabd5	owner() response and empty contract code at the snapshot; account execution rights still need verified source or traces.
Copied rollup	0x5ef0d09d1e6204141b4d37530808ed19f60fba35	Confirmation period and owner getter.
Rollup reported owner	0x3fffbadaf827559da092217e474760e2b2c3cedd	Published TransparentUpgradeableProxy source; implementation slot identifies 0x86f0cf42ad673b3d666d103e009ec142d1298a17 and proxy admin slot 0xb8698cfffdaaf26ca21fe8aa560c18cc2b2f5996. A controlling signer is not established by these slots.
Across HubPool	0xc186fa914353c44b2e33ebe05f21846f1048beda	Owner, liveness and copied chain-key configuration.
HubPool reported owner	0xfeca7291cebe75789ffa707754db259187b1d140	Owner getter; no contract code at the snapshot; identity and common control unproved.
Configured key 1 SpokePool	0x5c7bcd6e7de5423a257d81b442095a1a6ced35c5	Hub configuration; recorded upgrade and current implementation below.

Name	Full PulseChain address	What is established
Spoke implementation	0x788e49c4e2b3414777d6671d7411951e81d35808	Recorded upgrade; implementation behavior remains incompletely verified.

Getter evidence for the first group is `metadata-rpc.json` and `routes-rpc.json` in the deferred batch. New default-admin, pending-role, code and owner checks are `authority-rpc.json` and `authority-details-rpc.json` in the review batch. The bridge role's administrator is `bytes32(0)`, and the short executor holds that default role. These establish one available contract authority path; they are not an exhaustive census of every role holder.

Changes and cooperation required

Required capability	Contract or account that matters	Proven authority and remaining condition
Control executor actions	Short executor	Existing administrator is governance. Under the saved executor source, the nominated address can call <code>acceptAdmin()</code> without a queued delay if it can sign as that address and state remains unchanged. Acceptance has not occurred in the sampled state.
Change reserve supply and unbacked caps	Pool Configurator	Published implementation accepts <code>RiskAdmin</code> or <code>PoolAdmin</code> ; short executor is an observed <code>PoolAdmin</code> . Actions through its <code>queueTransaction</code> and <code>executeTransaction</code> require its administrator, a queued action, at least the current 86,400-second delay and execution within the grace period.
Authorize Portal issuer	ACL Manager	<code>addBridge</code> delegates to role grant checks. Default role administers Bridge at the snapshot and short executor holds it. The actual adapter address and secure integration are unspecified. Nomination itself grants no direct roles; the pending address directly lacks <code>PoolAdmin</code> in the observed query. Its direct Bridge membership was not queried.
Issue receipts successfully	Pool and authorized adapter	Positive unbacked caps, sufficient supply headroom, active, unfrozen and unpaused reserve and all applicable validation are needed. Headroom must be recomputed at execution using current indexed supply and treasury accounting.
Back Portal exposure	Pool and authorized Bridge caller	<code>backUnbacked</code> also requires Bridge permission, exact underlying, token allowance and supplied fee. An ordinary supply does not reduce unbacked exposure; after role revocation another authorized caller may be needed.

Required capability	Contract or account that matters	Proven authority and remaining condition
Mint underlying as a possible settlement source	Each exact underlying token's issuer controls	Separate token-specific permissions and issuance conditions would be needed for pDAI, pUSDC, pUSDT or other underlying assets. Aave administration and the Portal Bridge role do not grant them. Rollup or Across ownership likewise does not establish mint authority over unrelated tokens. The complete required issuer-control map remains unresolved.
Convert receipts at campaign scale	Exact wrapper and funded output markets	Matching token address, wrapper limits, liquidity and executable output are required. Administrative cap changes do not fund the output markets. Complete routes for every debt token are not established.
Repay and withdraw 86ff collateral	Borrower contract	repayWithATokens cancels the calling account's own matching debt. Reported owner cooperation alone does not prove a callable batch or withdrawal path. Borrower source response was address-only.
Continue pARB messages or change its administration	Rollup and owner contract	Accepted assertions require admission, timing and challenge conditions. Owner getter points to a proxy with an observed UpgradeExecutor implementation. Published source requires EXECUTOR_ROLE for execute and ADMIN_ROLE for ordinary role grants. Execute delegates to a payload running with the owner proxy's authority and storage, so an authorized payload can also alter role storage. Actual holders and their signing authority need further tracing. Rollup ownership does not itself confer mint authority over arbitrary destination tokens.
Deliver proposed Across metacoins	Hub, applicable adapter/spoke and funded relayer	Hub owner is identified, but chain-key and route configuration, current spoke behavior, relayer funding and exact asset mappings need a complete integration. Copied Hub key 369 has zero adapter and spoke at the snapshot.
Execute local support	Proposed keeper and funded market positions	Keeper account, oracle/reference, supported sizes, response rules and withdrawable LP entitlement are unspecified. Anyone can trade in a compatible public pool; this does not give ownership of burned LP shares.

Required capability	Contract or account that matters	Proven authority and remaining condition
Supply Ethereum exit liquidity	Proposed bridge representation and external LPs	Ethereum token identity, legitimate bridge path, actual ETH/WBTC funding and LP commitments are unresolved. These are a proposed separate exit, not an existing verified capability.

Rollup owner proxy slots are recorded in `owner-storage-rpc.json`; the published UpgradeExecutor source is `rollupOwnerImplementation-source.json` in the review batch. Published source for the proxy-admin address was unavailable. This identifies role requirements without pretending its executor-role holders or upgrade controller are known.

The executor logic is preserved at `artifacts/parb-local-pls-protocol-conditions-20260929/v3_authority_source.json`. Its `acceptAdmin` checks only the pending caller, while its queue and execution functions check the current admin, timestamp and queued action. Its minimum delay is 86,400 seconds. Configurator and ACL sources are `configurator_source.json` and `acl_source.json` in that batch. Provider ownership, ACL grant authority and reserve configuration rights are different checks and have been kept separate.

Supply headroom at the actual mint

The published validation requires an ordinary supply cap of zero, meaning unlimited, or the integer ray-math condition:

```
rayMul(scaledTotalSupply + accruedToTreasury_after_updateState,
      nextLiquidityIndex) + mintAmount <= supplyCap * 10^decimals
```

Portal separately requires existing unbacked plus new principal to fit its positive unbacked cap. Receipt burns after a mint do not repair an earlier failed check. Current configuration, index, accrual and proposed staged allocations must be checked at execution; a snapshot minimum is not a safe permanent cap recommendation. All seventeen observed reserves were active, unfrozen and unpaused. Source: `ValidationLogic.validateSupply` and `BridgeLogic.executeMintUnbacked` in the saved pool and linked-library sources.

Exact examples of token delivery

The documented local route uses pDAI `0x6b175474e89094c44da98b954eedeac495271d0f`, V3 aDAI `0x018008bfb33d285247a21d44e50697654f754e63`, waDAI `0x098256c06ab24f5655c5506a6488781bd711c14b` and Balancer linear pool `0x6667c6fa9f2b3fc1cc8d85320b62703d938e4385`. Existing local trades do not establish newly delivered source receipts or current campaign-size funding. [Receipt study \(Appendix 49\)](#).

The historical ARB mint example uses destination ARB `0xb50721bcf8d664c30412cfbc6cf7a15145234ad1`, source counterpart `0x912ce59144191c1204e64559fe8253a0e49e6548` and authorized reverse gateway `0xbbce8aa77782f13d4202a230d978f361b011db27`. This is a token-specific mint, not pDAI issuance or evidence of a funded ARB-to-pDAI conversion at scale. The examined stablecoin and aToken router mappings use escrow routes instead. [Gateway inventory \(Appendix 4\)](#).

Proof map for the mechanism

ID	Assertion	Classification and supporting evidence
P01	Common coordinated control and ecosystem-preserving incentives	Scenario premise; addresses and role getters do not prove common private-key ownership or behavior.
P02	Repeated source inputs can finance repeated purchases	Conditional mechanism: each accepted delivery must yield usable destination inputs. Assertion study (Appendix 26) and network model (Appendix 31) .
P03	Unchallenged invalid assertions can publish an incorrect accepted root	Pinned historical Nitro source capability under admission, ancestry, timing and staking predicates. No permanently unchallengeable deployment or actual invalid assertion is proved. Assertion study (Appendix 26) .
P04	Disabled whitelist does not exclude every new validator	Observation at block 27,673,362, current user implementation modifier and raw records. It does not prove a working challenge or sufficient data. Validator study (Appendix 49) .
P05	Accepted messages can release escrow or mint mapped tokens	Source capability of the particular gateway and mapped destination permissions. It is not universal token minting. Route atlas (Appendix 51) .
P06	NFT or vault shares can carry exclusive position rights	Standards and application-dependent capability. A funded buyer, loan, redemption or adapter must convert them into accepted market inputs; no complete deployed NFT route is proved. Assertion study (Appendix 26) .
P07	Existing destination aToken conversions have succeeded	Public transaction observations with exact contracts and amounts. They do not prove preceding pARB issuance. Receipt study (Appendix 49) .
P08	Zero unbacked cap blocks positive Portal issuance at the check	Deployed getter plus linked BridgeLogic cap predicate. Existing transfers and funded ordinary supply are distinct. Deferred study (Appendix 34) .
P09	The observed seventeen unbacked caps are zero	Same-block raw reserve configurations, decoded by the offline census. Deferred study (Appendix 34) .
P10	Supply and wrapper gates also matter	Published supply validation; waDAThis review checked-subtraction failure observed separately at block 27,673,486. Pricing study (Appendix 48) .

ID	Assertion	Classification and supporting evidence
P11	Receipt indexing does not multiply new principal	Published scaled mint accounting and exact index getters. Existing balance already includes index. Pricing study (Appendix 48) .
P12	The examined Portal has no automatic backing deadline	Inspected deployed linked library contains no maturity timestamp or auto-backing call. Integration obligations can differ. Deferred study (Appendix 34) .
P13	Outstanding nominal Portal principal consumes headroom until backing	Source predicate and accounting. Principal itself is not indexed, though aToken claims can accrue. Deferred study (Appendix 34) .
P14	Per-reserve unbacked-cap encoding is 2^{36} minus 1 whole units	Source bit field, conditional bound before implementation change. Deferred study (Appendix 34) .
P15	The sampled borrower has twelve nonzero V3 debts	Same-block balances over all seventeen enumerated reserves, reconciled exactly to aggregate oracle debt. V2 getter failure remains unresolved. Deferred study (Appendix 34) .
P16	Matching caller-owned aTokens can cancel its debt	Pool/BorrowLogic source capability. New Portal exposure remains; lawful account execution and collateral rights are additional requirements. Borrower rights study (Appendix 21) .
P17	Available collateral is approximately 105,220.503 aWETH, redeemable for WETH only through valid withdrawal	Same-block holding and cash observations; full repayment, withdrawal checks and owner rights are separate. Deferred study (Appendix 34) .
P18	Repaying debt removes that borrower's interest contribution	Source accounting and debt-share observation, conditional economic consequence. It does not establish future rates in every reserve. Deferred study (Appendix 34) .
P19	Outside earnings can fund later backing	Conditional arithmetic on exclusively owned realizable assets. Normalized 5% examples are assumptions, not observed yield. Deferred study (Appendix 34) .

ID	Assertion	Classification and supporting evidence
P20	More aToken units than pTokens alone does not prove buyback coverage	Conversion-output arithmetic; net executable purchasing power must cover inventory sellable within the supported horizon and competing commitments. Revalue when the payment asset's dollar price, output capacity or sellable inventory changes. Managed peg study (Appendix 45) .
P21	Accessible funded bids and asks can constrain an executable band	Conditional market policy for declared size, time, costs and execution. A fluctuating payment asset requires changing payment quantities against an independent dollar reference. No deployed guaranteed controller is proved. Managed peg study (Appendix 45) .
P22	Purchases reduce former holders' sellable stock but do not fix an AMM curve	Conditional ownership analysis plus standard V2 invariant. Resale creates sellable float again; assets deposited into owned LPs remain available to traders. Outside pool assets do not become system-owned merely through trading. Managed peg study (Appendix 45) .
P23	Ten-second scheduling gives 8,640 eligible slots per day	Arithmetic; actual inclusion, duty cycle and acquired output vary. Deferred study (Appendix 34) .
P24	Splitting alone does not reset constant-product slippage	Offline arithmetic with no intervening flows; fees can make split output lower. Deferred study (Appendix 34) .
P25	Corrective arbitrage can transmit intervention between pools	Conditional closing paths after costs and liquidity constraints; toy network experiment is not a live system census. Network study (Appendix 31) .
P26	A V2 pool has variable execution despite deterministic calculation	Standard source invariant and rational example. It does not prove every deployed fork's fee or token behavior. Managed peg study (Appendix 45) .
P27	Atropa pools hold large nominal inventories with extensively burned LP shares	Pinned raw reserves and dead-address LP balances. Symbols and amounts are not dollar backing; burned LP positions cannot be withdrawn by the operator. Atropa study (Appendix 27) .
P28	3CRV connects 3pool to OUSD, mUSD and FRAX; eUSD uses FRAXBP	Exact coin and base-pool getters at the main snapshot. Deferred study (Appendix 34) .

ID	Assertion	Classification and supporting evidence
P29	Examined Across metacoin routes are not established	Correct Hub getter returns zero on nine examined mappings; historical Spoke getter reverts, which proves neither enabled nor disabled status. Deferred study (Appendix 34) .
P30	Internal fees and nested receipts cannot be counted as independent income or principal	Consolidated ownership accounting. Collected outside fees require the actual earning entitlement. Managed peg study (Appendix 45) .
P31	LP locking prevents provider withdrawal, while swaps can remove counterassets	Standard V2 mint/burn/swap rights, conditional to the actual lock mechanism. Managed peg study (Appendix 45) .
P32	An LP exit can work without an issuer's fixed redemption promise	Proposed market execution from actual LP reserves; bridge and funded Ethereum exit remain unverified. Managed peg study (Appendix 45) .
P33	A fixed dollar pDAI can have a changing ETH ratio	Identity $\text{USD/pDAI} = \text{ETH/pDAI} \times \text{USD/ETH}$. Dollar reference and arbitrage access are required assumptions. Managed peg study (Appendix 45) .
P35	A declared StableSwap depth can execute a 50m sale near target while the next quote is lower	Hypothetical amplification, reserves, fee and U value; numerical model independently reproduced. Liquidity study (Appendix 40) .
P36	Repeated modeled support can depress neighboring assets	Eight constructed shocks correct pDAI to 0.995 while U falls to 0.694741; no claim about actual market future. Network study (Appendix 31) .
P34	All clocks must remain distinct	Observed executor seconds, rollout blocks and Hub liveness, plus proposed campaign interval and separate backing horizon. Deferred study (Appendix 34) .
P37	Canonical GH0 operation was not established at the sampled PulseChain address	At block 27,659,393, saved gh0.json reports no code at canonical GH0 and an unset stkAAVE debt-token hook. A custom deployment elsewhere is not excluded. Aave history (Appendix 2) .
P38	The surveyed receipt routes include three V3 stablecoin wrappers, one V3 WETH wrapper and five V2 wrappers	Bounded factory census with exact aToken, wrapper and pool getters. Registration does not establish current usable conversion or campaign-scale capacity; V2 and V3 inputs are distinct. Wrapper inventory (Appendix 24) .

ID	Assertion	Classification and supporting evidence
P39	A fixed accounting oracle is distinct from independent dollar measurement and payment	Original Ethereum GHO listing deliberately installs a fixed \$1 feed; the copied DAI feed has a May 2023 update timestamp. The economic distinction does not establish a chosen payment asset or operating PulseChain GHO. Original GHO listing, pricing evidence (Appendix 48) .

Issuer-collateral extension

These claims concern the subsequent issuer-collateral revision, not the frozen three-round review.

ID	Claim	Evidence and limit
P40	An issuer can convert bridged collateral into its existing local stablecoin	Conditional on exact collateral acceptance, valuation, permissions and mint checks; no end-to-end route executed. Issuer study (Appendix 38) .
P41	Existing Curve coin membership need not change after issuance of that exact coin	Direct coin/base-pool getters for all 619 census pools; issuer minting and pool trading are separate. Issuer study (Appendix 38) .
P42	Reserve is designed for compatible collateral plugins	Verified v2 source and current registry/basket reads; registration alone is insufficient and refreshed issuance remains unresolved. Reserve audit (Appendix 18) , issuer extension (Appendix 38) .
P43	Current Origin collateral is DAI, USDT and USDC; mint price is capped at 1 and threshold is .998	Fixed-block getters and active verified VaultCore; delegated admin/oracle source is unverified and full mint execution was not tested. Issuer study (Appendix 38) .
P44	Current mUSD accepts its existing basket in small view quotes, but new collateral needs another path	Verified MusdV3 exposes no public asset-addition setter; 100-unit quotes succeed for three inputs, 10m fail weight limits, OUSD/eUSD inputs fail. Quotes are not executed mints. Issuer study (Appendix 38) .
P45	mStable governor is a sampled three-of-five Safe	Getter/storage/Nexus evidence identifies addresses and reported admin delay; signer keys and complete proxy-admin execution are not proved. Issuer study (Appendix 38) .
P46	Selected stable tokens allow authorized local supply without first accepting a new bridge receipt	Verified DOLA/XAI/aIUSD/FRAX token sources; role holders, owning-controller execution, caps and backing require separate proof. Issuer study (Appendix 38) .

ID	Claim	Evidence and limit
P47	Every returned pool in eight responsive roots was retained	1,236 listings reconcile to 619 unique addresses; all direct arrays decoded, twelve underlying reads failed, 34 pools have incomplete balance reads. Unregistered pools are outside scope. Issuer study (Appendix 38) .
P48	All stablecoins cannot be assumed to have interchangeable governance powers	Immutable LUSD issuer addresses, renounced BorrowerOperations owner, centralized minter roles and distinct identities; common human control is not established. Existing native-collateral routes are not excluded. Issuer study (Appendix 38) .
P49	Candidate L2 yield protocols do not imply existing pARB deployments or automatic L1 acceptance	Historical baseline plus primary documentation; pARB RPC/ deployment inventory unavailable. Issuer study (Appendix 38) .

Complete route and operating design

These statements belong to the expanded white paper. They do not convert its proposed controller into deployed behavior or its historical evidence into current operational capacity.

ID	Claim	Evidence and limit
P50	The paper enumerates all accounting route families identified in the saved research	Fourteen financing variants and seven market/maintenance variants, with overlaps explained. This is bounded coverage of the research, not all possible deployments. Route design (Appendix 62) .
P51	Support needs payment inventory for downward pressure and owned pDAI for upward pressure	Conditional two-sided inventory accounting. Depth, costs, ordering, size, horizon and future outside supply determine usable capacity. Managed peg (Appendix 45) , route design (Appendix 62) .
P52	New minting of another stablecoin does not automatically provide pDAI supply	Token-specific issuer permissions and address identity; owned pDAI sales, a valid pDAI mint path or market acquisition are separate. No new pDAI mint authority was demonstrated. Issuer study (Appendix 38) .
P53	Immediately deliverable inventory differs from encumbered or delayed earning positions	Proposed support/settlement buffer policy and consolidated ownership accounting. No selected reserve amount, future yield or permanent coverage has been certified. Route design (Appendix 62) .

ID	Claim	Evidence and limit
P54	A more stable design needs a declared metric and comparable constraints	Proposed measurement of size-dependent executable prices, time, outages, liquidity and obligations. No globally optimal design was proved. Route design (Appendix 62) .
P55	Review agreement can improve the document but cannot prove the system flawless	reviews assess declared evidence, clarity and scope; missing deployments, permissions and funding remain independent proof requirements. Route design (Appendix 62) .

Integrated network extension, 1 October 2026

The [integrated network study \(Appendix 37\)](#) fixes the historical membership cutoff and separates configured identities, supply behavior and effective control. The new findings do not extend the previous edition's review approvals to this revision.

Claim	Evidence and boundary
P56	Eight historical registry roots yield 618 pools and 381 token identities at inherited block 17,232,999. Ethereum archive reads use the verified matching inherited header; 12 underlying-coin reads remain unresolved. This is bounded registry membership, not current pool funding or every custom deployment.
P57	A bridge can return an existing L1 pool coin when its L2 address differs. Current DAI/USDC/USDT mappings and saved exact gateway implementations establish the identity/release design. Source deployment, authenticated messages and available escrow remain separate requirements.
P58	Three sampled configured pairs have identical L1/L2 numeric addresses: PL2, HGOLD and MCB. Current pair getters plus historical token labels; different chains still have separate contracts and supply. HGOLD token mint permissions remain unresolved.
P59	Standard/custom gateways do not generically mint local stables. Selected current DAI/USDT/OUUSD/eUSD/mUSD/FRAX gateway token balances are zero; USDC has 1,000,000 raw units. Other venues or future deposits may contribute separately; a failed read is not zero.
P60	FRAX AMOs and DOLA Feds are candidate venue supply buffers; rebasing tokens and collateral issuance have distinct mechanisms. Primary architecture descriptions do not prove installed local controllers, common control or present capacity.

Claim	Evidence and boundary
P61	The sampled OUSD/3CRV account is nonrebasing; the sampled UZD 0xb40b...9f1c pool is rigid. Current eligibility getters and inherited/source accounting; other UZD versions' failed eligibility reads remain unresolved. Pool accounting needs separate proof before any proposed rebase integration.
P62	No resolved sampled generic authority comparison differs from inherited state: 70 matching values, 446 unresolved records, plus 18 matching controller/storage/delay comparisons. This neither confirms universal takeover nor excludes changed voting control. Some new monetary candidates lack issuer authority reads.
P63	Across HubPool and Safe module owners changed in verified postfork transactions; current getters agree. Two authority changes within Across do not prove UMA ownership, common private-key ownership or intent to peg pDAI. Aave's sampled nominee remains pending.
P64	PLS/HEX/PLSX/INC earning positions could contribute realized support resources. Proposed branch; no generic Origin/Reserve acceptance of these assets is proved. New plugins/strategies require compatible authority and valuation; existing accepted stable collateral is a shorter alternative.
P65	Connected issuer conversions, supply controllers and funded venue trades could narrow stablecoin market differences. Conditional route arithmetic and inventory accounting; frozen \$1 markings do not fund AMM bids, and relative convergence does not establish external dollar purchasing power.
P66	Historical final canonical crvUSD address has no code at the inherited boundary; earlier crvUSD labels remain unresolved versions. Successful archive code getter, not inference from later docs. The inherited Mochi USDM identity is distinct from later Mountain USDM.

Remaining proof requirements

The [sUSD extension](#) (Appendix 66) adds P67–P69: six inherited Curve pool connections and sampled mUSD acceptance (P67); verified copied Synthetix bridge source with messenger/counterpart guards, enabled transfer setting and the Issuer mapping to the existing sUSD implementation at separate block 27,683,120 (P68); and the distinction between that Optimism-specific issuer path and pARB's zero-inventory sampled escrow branch (P69). None establishes common governance control, a verified current source chain or an executed end-to-end transfer.

Its financing extension adds P70–P72: approved collateral wrappers and the explicit historical Optimism DAI → sUSD → Ethereum bridge/redeem-or-trade precedent in SCCP-264/SCCP-2011 (P70); the version boundary between later Arbitrum V3 native/spot tokens and inherited PulseChain V2 sUSD (P71); and the original bridge announcement's seven-day withdrawal wait, equivalent to 60,480 ten-second intervals (P72). The primary documents and connected metapool rows are frozen in `artifacts/pulsechain-susd-financing-20261001/`. Neither upstream proposal status nor timing proves a deployed pARB source, available copied wrapper capacity, accepted cross-chain messaging or the copied route's delay.

The [control and storage audit \(Appendix 67\)](#) adds P73–P77. The current Synthetix 5-of-13 Safe, Optimism 5-of-7 Safe and nested 2-of-2 Safe retain inherited sampled control values; selected runtime comparisons match, without proving private-key possession (P73). The copied Optimism fraud-proof window is 604,800 seconds and its selected postfork state-batch census is empty (P74). The recognized local DAI/sUSD wrapper has a 20-million-DAI cap, 14.648-million-DAI sampled spare capacity and unchanged positive fees (P75). Caps, fees and creation of new factory wrappers are owner-controlled capabilities; authority by the proposed group is unproved (P76). Depositing bought pDAI creates collateral for eligible synth redemption rather than an exclusive freely spendable treasury reserve or interest-paying deposit (P77). The exact earlier and later checkpoints, successful selected control census and collection limitations are preserved separately.

The contract-level proof establishes boundaries and available authority paths. An operational proof still needs the exact source assets, adapter addresses, destination conversion capacity, funded support inventory, account execution rights, keeper policy, dollar reference and end-to-end accounting. Outside income is one funding option, not a required premise for every purchase. All assumptions must stay visible when the main narrative is simplified.

A flawless perpetual dollar peg, complete private-key control map and funded external exit are not established by this evidence. The narrower proposal can be proved conditionally once the missing inputs and policy are specified.

Appendix 30: Review of the complete pDAI white paper

Contents of this appendix

Method	175
Findings	176
Reproduction and publication checks	176

Source record: docs/studies/analysis/pulsechain_complete_whitepaper_review_20260930.md.

Review of the complete pDAI white paper

30 September 2026. This study records review of the expanded coordinated-financing and pDAI-maintenance proposal. All five reviews accepted the final frozen document within its declared saved-research scope, with no material blockers identified. This is agreement on the document, not certification of deployment, funding, optimality or flawlessness.

Method

Five agent reviews assessed the paper in three rounds: clarity, protocol, economics, bridges and reproduction. Each received the same frozen thesis and route register for its round. The second and third rounds also included frozen claims maps. reviews were instructed to read independently, avoid other reviews' reports, identify exact defects, distinguish document validity from operational readiness, and preserve simplicity. The author integrated the first-round findings and then condensed the explanation for the final round.

These are model-agent reviews with shared research context and informed repeat readers. They are not five independent human experts, statistical samples, calibrated error probabilities or empirical novice testing. The clarity review used novice and informed-reader role-plays to test the explanation; those are simulated assessments. “Stochastic consensus” here means convergence among separately reasoned model reviews, without a statistical guarantee of correctness.

Round	Frozen main words	Review outcome
1	2,921	Economic logic accepted; four roles requested targeted clarifications before acceptance.
2	3,425	Five document-level passes; no material blockers identified.
3	3,215	Five document-level passes after simplification; no material blockers identified.

The final route register contains fourteen financing variants and seven market/maintenance variants. These are overlapping accounting categories, not twenty-one independent funding sources. Coverage means all route families identified in this saved research, not every possible future application or unregistered pool.

Findings

The final [white paper \(main paper\)](#) explains source positions, bridge representations, issuer collateral acceptance, existing local coin output, Curve trading and separately funded buying/selling inventories. It keeps the L2 issuer route central while preserving Portal, debt retirement, gradual purchases, deferred settlement, outside income, Atropa and proposed external exits.

Review resolved ambiguity about the unselected payment asset, chain-specific token identities, source issuer deployment assumptions, conditional diagrams, pledged backing, wallet versus pool-accessible selling supply, Portal method permissions and administrative clocks. The final [route register \(Appendix 62\)](#) retains exact conditions, authority references, all saved route families, measurement terms and fresh-directory reproduction guidance. The [claims map \(Appendix 29\)](#) distinguishes verified observations, source capabilities and proposed policy.

Remaining proof requirements are substantive: source deployments and usable balances, exact bridge mappings, effective contract/account control, compatible collateral acceptance, deliverable support inventory, a selected reference and measured two-way capacity. The uncompleted historical local-protocol-conditions batch remains marked incomplete. Review agreement does not close these gaps.

Reproduction and publication checks

Frozen inputs and all fifteen actual reports are in `artifacts/pulsechain-complete-whitepaper-20260930`; the explicit experiment manifest is `Research/experiments/pulsechain_complete_whitepaper_20260930.json`. Input and report hashes identify exactly what was reviewed. The maintained final thesis, route register and claims map match their round-three frozen inputs.

This edition changes prose and research records only. Seventeen scoped script/test files match the preceding issuer release's original hashes. That code-changing revision passed 61 research tests and 82 subtests. Tests were not rerun for this writing-only edition; local checks verified links, numerical examples, frozen text identities and unchanged prior archives. Diagram assessment was source inspection, not renderer validation.

The separate release is `artifacts/pulsechain-anonymous-complete-whitepaper-release-20260930`. Its builder validates appendix links and preserves public-chain identifiers. The batch's `release-verification.json` records final ZIP CRC, every manifest file hash, export equality, required evidence and private-recovery exclusion. Earlier releases and the private recovery records remain preserved. No new RPC calls, signing, broadcasts, permission changes or state-changing simulations occurred in this synthesis and review edition.

Appendix 31: Coordinated issuance and connected-pool correction

Contents of this appendix

Question and scope	177
Method	177
End-to-end results	178
Repeated selling and renewal.....	178
Inventory asymmetry	179
Reflexivity and LP accounting.....	179
Reproduction and validation.....	179

Source record: docs/studies/analysis/pulsechain_coordinated_network_20260930.md.

Coordinated issuance and connected-pool correction

30 September 2026. Reproducible hypothetical market experiment.

Question and scope

Can a controller with no initial destination treasury assets use renewed, accepted source claims to acquire pDAI and induce profitable correction through connected pools? Yes, in this constructed market. The experiment grants the source issuance and representation premises and tests their market consequences. It neither deploys those capabilities nor establishes them on PulseChain.

The earlier [standing-facility experiment \(Appendix 60\)](#) tests a fixed-inventory comparator. Its exhaustion result does not by itself evaluate a network that continually introduces accepted source assets. This experiment gives that network its proposed additional input and tracks where the resulting inventory goes.

Method

Four constant-product pools charge 0.3% of input; fees remain inside reserves. $\mathfrak{d}$ denotes pDAI, $\mathfrak{p}$ PLS, $\mathfrak{u}$ a surrounding fungible asset, and $\mathfrak{x}$ an accepted destination representation of a source asset or receipt. No backing valuation or guaranteed dollar price is assigned to $\mathfrak{X}$. $\mathfrak{U}$ begins at a one-dollar marginal PLS price but has no independent redemption promise. This makes propagation of pressure into surrounding assets visible.

Pool	Initial reserves
X/U	10,000,000 X and 10,000,000 U
D/U	10,000,000 D and 10,000,000 U
D/P	1,000,000 D and 10,000,000,000 P
U/P	10,000,000 U and 100,000,000,000 P

The fixed external reference is \$0.0001 per PLS. Controller balances start at zero. Outside traders hold the amounts needed for their specified shocks; the arbitrageur has owned PLS capital. These are existing assets, not assets created by the controller. LP ownership and withdrawals are not simulated.

For input q , input reserve x , output reserve y and fee f , the pool pays $y(1-f)q / [x+(1-f)q]$, then adds the entire input and subtracts the output. All amounts use Decimal precision 60. No floating-point numbers enter the model.

For a sale shock, the controller issues source X , locks it and receives one destination X per locked source unit. It sells $x \rightarrow u \rightarrow D$, retaining D . A separate trader chooses the more profitable of $P \rightarrow D \rightarrow U \rightarrow P$ and its reverse, optimizes trade size up to its budget, and acts only if gross profit exceeds 3,000 PLS gas. A binary search selects controller input sufficient to restore the D/P marginal price to at least \$0.995 after that correction, within a per-round issuance limit of one trillion X .

For a buy shock, the controller sells retained D into D/U . The trader's reverse cycle can then lower D/P toward \$1.005. The controller never mints D in this experiment. Insufficient D inventory therefore remains a meaningful test.

Each round assumes that any required bridge delivery has completed. The model does not shorten a challenge window. Controller gas, bridge fees and losses during latency are excluded, so this is a gross capacity experiment, not a net profit forecast. Only arbitrageur profit includes the stated gas charge.

End-to-end results

An outside sale of 200,000 D receives **1,662,497,915.624479 PLS** and moves the instantaneous marginal price from \$1 to \$0.694792. The original seller's average execution is about \$0.831249; later correction does not change that payment.

The controller issues and bridges **458,545.820869 X** , receives **437,183.459185 U** , and buys **417,666.978491 D** . The trader starts its corrective cycle with **1,642,425,206.936962 PLS**, earning **404,756,426.740876 PLS** after the assumed gas charge. D/P reaches \$0.995.

The controller has acquired more D than the outside seller supplied because it also acquired inventory previously held by pools. No tokens are missing: source issuance equals source escrow and newly issued destination X , while every non- X token is conserved across all destination accounts.

Next, an outside buyer spends **2 billion PLS**, receiving **167,029.366446 D** , and raises the marginal price to \$1.433043. The controller sells **354,543.328273 D** into D/U , receiving **356,785.113193 U** . The reverse arbitrage earns **367,247,096.815829 PLS** after modeled gas and lowers D/P to \$1.005. The controller ends with **63,123.650218 D** and **356,785.113193 U** .

These retained assets are holdings, not a demonstrated net profit. The X claim and the resulting risk held by its purchasers remain part of the consolidated account. No X redemption liability is silently settled or assigned a zero cost.

A fourth scenario extends this sequence with another 200,000- D sale. The controller reuses the **356,785.113193 U** recovered during buying, adds **106,688.977595 newly issued X** through X/U , and acquires another **429,624.257935 D** . Corrective arbitrage again returns D/P to \$0.995. This demonstrates reinvestment of recovered inventory. It is distinct from earning new outside fees, which this model does not allocate to treasury ownership.

Repeated selling and renewal

Eight consecutive 200,000- D sales are each corrected to \$0.995 in this model. The controller accumulates **3,265,728.565660 D** , issuing a cumulative **6,769,533.095641 X** . The last correction requires **1,512,055.592012 X** , compared with **458,545.820869 X** initially.

This is a successful repeated intervention under the chosen assumptions, with a measurable transfer of exposure. X/U now holds **16,769,533.095641 X** and **5,972,141.013973 U**, instead of 10 million of each. U/P's marginal dollar price falls to **\$0.694741**. The controller has maintained the chosen pDAI price by changing the surrounding network; it has not kept every asset at \$1.

This distinction matters for a design that expects FRAX, pUSDC, pUSDT and other assets to support one another. If each must also retain a dollar reference, their own corrective sources and final output inventories need to be specified. Internal parity alone leaves the common dollar price undetermined. Additional bridges can add accepted inputs or reach additional reserves; their existence does not, by itself, establish replenishment of every spent output asset.

For a constant-product X/U pool, ignoring fees for the bound, finite input gives $U_{out} = R_U X_{in} / (R_X + X_{in}) < R_U$. Renewing X can continue purchases while conversion remains available, but cannot extract more U than the pool owns without incoming U. A network can refill a particular pool from another; the consolidated account must identify where that U originated and who now holds X.

Inventory asymmetry

A separate buying shock begins before any controller accumulation. Market arbitrage lowers D/P from \$1.439280 to **\$1.065651**, but cannot attain the specified upper target. The controller has no D to sell. This case demonstrates the need for prior inventory or an additional exact-D delivery route. It does not test every possible network: authorized D issuance, releasable D escrow or available lending would be additional explicit capabilities with their own balances and obligations.

Reflexivity and LP accounting

The experiment models accumulation directly. It does not model ownership of LP shares, reinvestment, endogenous confidence or borrower demand. Those channels are accounted for in the [accumulation study \(Appendix 65\)](#): outside fees and interest can replenish controlled assets after costs, whereas internal transfers do not create consolidated earnings.

The same swap equations apply whether withdrawable LP shares are held by the operator, outside holders or an inaccessible address. Those arrangements differ in who can withdraw principal and realize fees. Irrecoverably held shares can prevent the corresponding liquidity withdrawal; swaps still alter the reserves. Thus burned LP ownership does not supply a fixed reserve composition or a spendable fee claim for the controller.

Reproduction and validation

Run from the repository root, selecting a fresh output path:

```
python Research/scripts/model_pulsechain_coordinated_network.py \
--manifest Research/experiments/pulsechain_coordinated_network_20260930.json \
--output artifacts/pulsechain-coordinated-mechanism-20260930/network-results.json
```

The saved output records manifest and script hashes, assumptions, each shock, controller input, arbitrage route and profit, pool reserves, treasury inventory, and conservation residuals. Its maximum per-token conservation residual is below $1e-35$ units; the observed maximum is $1e-48$ across these scenarios. Source issuance, escrow and destination issuance match exactly in Decimal accounting. Source and destination claims are kept separate rather than counted as two independent assets. Outputs are reproducible byte for byte.

An independent 90-digit replay gives a maximum pool-reserve difference below $4e-49$ units. Writing a complete cycle as $Aq/(B+Cq)$ gives the unconstrained profit-maximizing input $(\sqrt{A/B}-B)/C$, clipped to zero and the trader's budget. The numerical optimizer's inputs differ from this closed-form solution by less than 0.000003 PLS across the tested cycles. The independent replay uses no imports from the model script. Results are recorded in `artifacts/pulsechain-coordinated-mechanism-20260930/validation.json`.

The optimizer uses the concavity of compositions of positive constant-product swap curves. The validation checks the closed-form cycle optimum independently, all token conservation identities, both directional effects, eight renewed corrections, the unsatisfied inventory case and original seller execution. It tests a declared mechanism, not live profitability or perfect stability.

Appendix 32: PulseChain bridge, governance and DeFi evidence audit

Contents of this appendix

Findings	182
Scope and evidence method	182
The recorded sequence	183
What the two examined bridge transactions prove	184
ARB: destination minting is directly observed	184
MIM: e121 received an escrow release	185
Router census and additional destination mint evidence	185
GRT: a second executed destination issuance	186
What the broader transaction census adds	187
Results of the five audit checks	187
Evidentiary effect of GRT	189
Bridge and Portal accounting in inspected source	189
Token standards and eligibility	189
Across: replenishing both sides without confusing delivery and minting	190
Aave and HubPool contract accounting	191
Supply, loan repayment and Portal backing are different operations	191
Fixed-block reserve observations	192
Aave control, Portal configuration and stkAAVE	193
The Aave-Balancer wrapper inventory	193
Pool capacity and what actual swaps show	194
Reserve eUSD: deployed basket and issuance state	195
Reserve Main ownership and timelock history	196
Other stablecoins, native deposits and Curve connectivity	196
FRAX configuration and recorded debt repayments	197
The 86ff and 6e3B debt positions	198
A debt token is not a transferable creditor claim	198
Examined addresses and attribution	199
Coverage of the original ten research subjects	199
Provenance of earlier experiments	200
Research record	201

Source record: docs/studies/analysis/pulsechain_dai_evidence_audit_20260929.md.

PulseChain bridge, governance and DeFi evidence audit

Evidence companion to the [conditional mechanism report \(main paper\)](#).

Research updated 29 September 2026. This paper reports public transaction observations, fixed-block contract state and behavior identified in deployed source. Its conclusions concern the audited addresses and stated search ranges. It does not attribute motives, legal character or human ownership to the activity.

Findings

The recorded PulseChain history includes gateway-mediated **ARB and GRT mints**, standard-gateway escrow releases, Aave governance executions and upgrades, Across ownership and implementation changes, wrapper trades, Curve exchanges and Aave debt repayments. Each is an independently evidenced observation. The collected records do not yet connect them into one reconciled source-deposit, destination-issuance, purchase and settlement history.

The gateway census covers 129 token addresses appearing in the selected router's returned registration logs and nine destination gateways. Their accounting differs: some transfer escrow, some invoke token-specific issuance, and WETH wraps received native currency. The queried existing OUSD, eUSD, mUSD, FRAX and three Aave V3 stable receipt addresses select the standard escrow gateway.

Reserve's copied eUSD deployment is version 2.0.0. Its fourteen registered token/plugin pairs and prime basket match between the inherited snapshot and the audit endpoint. Its active basket contains four lending receipts. Cached basket status, current plugin price reads and oracle timestamps are reported separately; a successful present-day issuance has not been established by these getter calls. Governance authority is investigated through roles and recorded changes, rather than inferred from unrelated protocol control.

Earlier conditional mechanism studies remain separately identified among the companion studies. Hypothetical balances, assumed governance ownership and local experiments are not evidence that public-chain transactions occurred.

Scope and evidence method

"L1" means PulseChain, chain 369. "pARB" means a proposed execution network settling to its copied Arbitrum contracts; it does not identify the operating Ethereum Arbitrum network. The prefix "p" distinguishes copied assets from issuer-supported Ethereum balances. For readability, tables use familiar token symbols, but all state and transaction findings concern PulseChain.

The [PulseChain mainnet README](#) identifies inherited Ethereum state through block **17,232,999**, dated 10 May 2023 at 22:36:11 UTC. Historical searches start at **17,233,000**. Copying Ethereum state copied its L1 bridge and governance contracts; it did not by itself copy each rollup's separate execution database. Across is a bridge system, rather than an additional L2 execution database.

The principal endpoint is block **27,659,393**, hash

0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275, on 28 September 2026. Research completed on the following day still uses this fixed endpoint. Reads use <https://rpc.pulsechain.com>; explorer data and published source supplement them. Cited historical events were reconciled with successful receipts and block hashes. The two examined bridge transactions also have historical `trace_transaction` results. Otter's examined trace pages were inaccessible to the research clients; the RPC supplied the actual traces.

This is reproducible evidence from one RPC and related explorer services, not independently verified consensus or source-chain state proofs. Negative event findings apply only to the selected addresses, event signatures, range and provider completeness. An endpoint getter cannot exclude a temporary historical change, and an event cannot establish a human operator's identity.

Companion studies hold detailed evidence without reproducing bulk RPC data:

- Bridge history, token compatibility and supplied traces (Appendix 5).
- Router census and token-specific mint evidence (Appendix 4).
- GRT transaction, inherited allowance and source-design reconciliation (Appendix 12).
- Escrow-gateway withdrawal receipt census (Appendix 11).
- All 78 historical transaction traces (Appendix 23).
- Partitioned event coverage and 212 historical supply reads (Appendix 22).
- Historical default gateways, implementations and issuer permissions (Appendix 10).
- Specialized-gateway withdrawal and issuance history (Appendix 20).
- Reserve eUSD deployed contracts, registry, basket and issuance accounting (Appendix 18).
- Reserve Main ownership, timelock and recorded governance history (Appendix 19).
- Aave governance and implementation history (Appendix 2).
- Exact Aave wrapper and Balancer pool inventory (Appendix 24).
- Balancer capacity, configuration and historical swaps (Appendix 16).
- Aave repayment history and examined-wallet matches (Appendix 9).
- Curve configuration and examined-wallet event review (Appendix 8).
- May 2023 Arbitrum and legacy Optimism bridge models (Appendix 13).
- Native Arbitrum asset baseline (Appendix 15).
- Across, Omnibridge, Polygon and the conditional representation argument (Appendix 7).
- Across and UMA component control history (Appendix 3).
- HubPool custody and Aave settlement accounting (Appendix 14).
- Portal, wrapper sales and borrower repayment accounting (Appendix 17).
- 86ff and 6e3B debt and collateral positions (Appendix 21).
- Two recorded 86ff transactions and withdrawal-claim limits (Appendix 1).

The recorded sequence

Date, UTC	Observation	Evidentiary significance
27 May / 13 June 2023	Custom-gateway receipts transfer about 1.311 billion USDC and 491 million USDT from escrow	Large actual escrow withdrawals; not new stablecoin minting
1 July 2023	Aave proposal 224 executes; V2 address-provider ownership passes to 0xa30190b96faee0080144aa0b7645081fcbf49e6f; two V2 pool upgrades follow	Specific post-fork control and implementation changes
21 September 2024	ARB is minted through its reverse gateway	An existing destination token has a functioning gateway-authorized mint path
11 October 2024	Supplied BAD9... wallet queues Aave proposal 225	Governance participation, not by itself protocol ownership
8 January 2025	GRT specialized gateway transfers 30,490,952.885900226731295138 GRT from escrow to 0xb869...5996	Recorded escrow release; receipt contains no GRT mint

Date, UTC	Observation	Evidentiary significance
24 February / 1 March 2025	Bridge ProxyAdmin ownership changes twice; custom gateway upgrades and changes back	Endpoint state alone misses intervening administration
19 June 2026	Across HubPool ownership transfers to 0xfeca...d140	Specific post-fork administrative control change
29 June 2026	Recorded waDAI swap pays 5.200713616007117708 DAI	Existing destination pool accepts that exact wrapper
30 June 2026	0xfeca...d140 upgrades HubPool's configured L1 SpokePool through its Ethereum-adapter path	Actual copied-spoke implementation change; replacement behavior requires separate analysis
3-5 August 2026	Supplied FdbEE... wallet queues proposals and executes 218	Actual governance activity; proposal 218 emits an Optimism-side governance message
10 August 2026	Proposal 214 executes the stkAAVE upgrade	Upgrade is real; an operational GH0 deployment is a separate question
11 August 2026	Examined e121 executes a 152.747067838559905107 MIM withdrawal	Standard-gateway escrow release, with pre-fork source-message metadata
19 August 2026	Across OptimisticGovernor module ownership transfers to e121	Specific governance-module authority change
1 September 2026	GRT gateway mints 62,875,936.675361278837595998 GRT into escrow and delivers tokens	Receipt, historical trace and total-supply change establish destination issuance
4 September 2026	Recorded waUSDC swap pays 43.641283 USDC	Another exact wrapper-to-underlying public exchange

Proposal 218's inner message is now matched byte-for-byte to the original 2023 Aave instruction to list **MAI on Aave V3 Optimism**, with a 7.6-million MAI supply cap, 2.5-million borrow cap and 1.9-million isolation debt ceiling. The proposal predates the fork; the observed PulseChain send occurred in 2026. This identifies what was communicated without establishing a destination execution or a Portal mint. The [Aave study \(Appendix 2\)](#) records the message comparison and original primary sources.

The companion studies link the receipts and record their full hashes. Chronology alone does not demonstrate that different senders share ownership or purpose.

What the two examined bridge transactions prove

ARB: destination minting is directly observed

The [21 September 2024 transaction](#) executed at block **21,469,490**. Its historical trace follows the copied outbox, bridge, reverse gateway and ARB token's `bridgeMint`. Its successful receipt contains the corresponding zero-address `Transfer`, gateway withdrawal and outbox execution events.

Component	Address
L1 ARB token	0xb50721bcf8d664c30412cfbc6cf7a15145234ad1
ARB reverse gateway	0xbbc8aa77782f13d4202a230d978f361b011db27

Component	Address
Gateway counterpart recorded on L1	0xcad7828a19b363a2b44717afb1786b5196974d8e
Mint recipient	0x42543853a72edb6f1ff9bddab0cf4676187ec28a

The mint was **972,406,600,390,402,296,078,726,883,998.977902572465996426 ARB**, using the token's 18 decimals. The total-supply difference between the preceding block and the execution block exactly matches that amount. This is token units, not dollar value. Published implementations identify `L1ArbitrumToken` and `L1ForceOnlyReverseCustomGateway`; the token's `arbOneGateway()` matches the gateway. The force-only variant restricts ordinary self-registration.

This positively supports the destination-mint claim. The receipt and trace do not independently prove an equal source balance was locked, that it remains locked, or that source redemption and destination burning reconcile. The source-side runtime and escrow history have not been supplied or verified. A further historical decode found that the ARB message's recorded source-block and timestamp fields do not describe a plausible source chronology. They therefore supply no independent evidence of a native source deposit before or after the fork. The preceding rollup confirmation is only a temporal neighbor; a direct message/root link was not established. This reinforces the distinction between the proved destination mint and unproved source backing.

MIM: e121 received an escrow release

The [11 August 2026 transaction](#) at block **27,256,598** has sender and recipient `0x98c89980d1eae247ada3636ae27648f78e66e121`. The trace follows `outbox` → `bridge` → `standard gateway` → `MIM.transfer`. It transfers **152.747067838559905107 MIM** from `0xa3a7b6f88361f48403514059f1f16c8e78d60eec` to e121. The MIM token is `0x99d8a9c45b2eca8864373a26d1459e3dff1e17f3`.

The accepted message metadata references L2 block **85,474,848**, L1 block **17,151,178**, and **29 April 2023, 10:32:06 UTC**. That is consistent with later execution of an inherited withdrawal. It is not independent verification of source execution, but it does not evidence a new post-fork native pARB deposit. Neither supplied transaction contains Aave, Balancer or Curve calls.

Router census and additional destination mint evidence

Router `0x72ce9c846789fdb6fc1f34ac4ad25dd9ef7031ef` returned **167 GatewaySet events covering 129 distinct token addresses** through the fixed endpoint. Each latest registration was reconciled with an endpoint routing getter. The resulting inventory contains **nine nonzero destination gateways**:

Route family	Token count in census	Deployed incoming accounting / evidence
Standard ERC-20 gateway	98, including five default fallbacks	Transfers existing L1 escrow
Custom gateway	20	Transfers existing L1 escrow; custom routing is not itself mint authorization
DAI gateway	1	Transfers designated L1 escrow
wstETH gateway	1	Transfers existing L1 tokens
ARB reverse gateway	1	Calls token <code>bridgeMint</code> ; a post-fork mint is directly observed

Route family	Token count in census	Deployed incoming accounting / evidence
GRT gateway	1	Mints an allowance-bounded escrow shortfall; a post-fork mint is directly observed
LPT gateway	1	Calls an external minter for an escrow shortfall; current external-minter caller authorization remains unverified
HGOLD gateway	1	Verified gateway source calls bridgeMint; token-level authorization remains unverified
WETH gateway	1	Wraps received native currency and transfers WETH
Disabled routing	4	No enabled gateway in the returned mapping

This census is bounded by the selected router's returned event history. Default routing also applies to tokens without explicit registrations. It is not a claim to enumerate every token on PulseChain or every independent bridge. Seven additional exact-address reads place OUSD, mUSD, eUSD, FRAX and the copied Ethereum V3 aDAI, aUSDC and aUSDT on the standard escrow route. This does not establish available escrow or a functioning source counterpart. The [gateway inventory \(Appendix 4\)](#) records all addresses, implementations, registrations and token-permission limitations.

GRT: a second executed destination issuance

[Transaction 0xf7ed5701...](#) executed at block **27,434,949**, on **1 September 2026 at 13:42:05 UTC**. The historical trace follows outbox → bridge → GRT gateway → GraphToken.mint → GraphToken.transferFrom. Its successful receipt records a mint of **62,875,936.675361278837595998 GRT** to escrow `0x36aff7001294dae4c2ed4fdefc478a00de77f090`. The total-supply change from the preceding block matches that amount exactly. The trace reads **1 GRT already in escrow** before minting; the escrow then transfers **62,875,937.675361278837595998 GRT** to `0xa0b347117a878da9942fa5c31b87c773a2969913`. Thus the delivered amount is the existing escrow contribution plus the minted shortfall.

The token is `0xc944e90c64b2c07662a292be6244bdf05cda44a7`; its gateway is `0x01cdc91b0a9ba741903aa3699bf4ce31d6c5cc06`. At the endpoint the token's `isMinter(gateway)` returns true and the gateway reports unpaused. The gateway's recorded cumulative mint allowance is **64,230,816.8555 GRT**, and its recorded minted amount is **62,875,936.675361278837595998 GRT**. Their difference is **1,354,880.180138721162404002 GRT**. The configured allowance accrual is **6.0365 GRT per block**. These are token-accounting quantities, not dollar values or a guarantee that a future transaction will execute.

The allowance was configured before the fork, on **18 April 2023**, with a zero allowance snapshot, effective start block **17,018,986**, and **6.0365 GRT per block**. Historical getters at the inherited boundary, transaction-predecessor block and transaction block agree on these parameters and the gateway's minter permission. The selected configuration-event census contains no post-fork change. At the mint block, accrued allowance was **62,875,960.6495 GRT**; after this mint, **23.974138721162404002 GRT** remained. This is evidence of use of inherited, block-accrued capacity, rather than evidence that someone increased the rate after the fork.

The GRT gateway returned two post-fork withdrawal events. The other is an [8 January 2025 escrow release](#) of **30,490,952.885900226731295138 GRT** to `0xb8698cfffdaaf26ca21fe8aa560c18cc2b2f5996`. Its receipt contains no GRT mint. This identifies a recorded interaction with the previously discussed address; it does not establish common ownership with the September transaction's parties.

The historical source design is **L2 burn followed by L1 escrow release or shortfall minting**. Official pre-fork GIP-0037 introduced the minting allowance for L2-native indexing rewards, and the dated L2 gateway source calls `bridgeBurn` during withdrawal. It is not evidence that the same transferred GRT remains locked in an L2 investment position. The [GRT reconciliation study \(Appendix 12\)](#) provides the official proposal, pinned source, exact arithmetic and historical getter records. It does not claim independently observed pARB source execution.

GRT uses its specialized `mint` permission and allowance accounting, whereas ARB uses its registered reverse-token `bridgeMint` interface. Both destination mints are observed. Neither receipt independently establishes source custody, source execution history or the human identity of the transaction sender.

What the broader transaction census adds

Combining the four escrow-family gateways, the GRT gateway and the five current or retired specialized gateways produces **106 returned withdrawal events across 78 transactions and 91 token addresses**. All ten identified historical gateway addresses were queried over blocks 17,233,000–27,659,393. ARB and GRT remain the two confirmed destination-mint cases within this selected history. The selected gateway queries and recovered token-log partitions have no unresolved query gaps; provider completeness and unqueried mechanisms remain separate limitations.

A separate post-fork search of the standard, custom, DAI and wstETH gateways returned **102 withdrawal events across 74 transactions and 90 token addresses**. Their complete receipts contain no zero-address ERC-20 Transfer. For 101 events, a same-token transfer exactly matches the gateway amount and recipient; the remaining event's two outgoing transfers sum to the gateway's gross amount. This screen identifies no additional mint candidate, but nonstandard issuance or rebasing requires token-specific analysis rather than reliance on this event pattern alone. The [receipt census \(Appendix 11\)](#) records methods, scope and the split-transfer reconciliation.

Historical router registration logs identify **ten gateway addresses** in total, including one retired LPT gateway, compared with nine effective endpoint gateways. Current preferred routing alone also misses actual withdrawals: GRT has a standard-gateway escrow transfer and LPT has a custom-gateway escrow transfer in the returned history. Auditing only each token's current preferred gateway would omit those transactions.

The LPT specialized gateway executed a withdrawal of **10,459,514.129760763228599483 LPT** on **13 September 2024**, in [transaction 0x377c6bd1....](#) Its trace uses existing escrow, contains no external-minter call, and its block-to-block total supply is unchanged. A mint branch exists in the gateway source, but this transaction did not execute it. The [specialized history \(Appendix 20\)](#) records the separate ARB, LPT, HGOLD, WETH and retired-gateway searches and their limits.

Results of the five audit checks

The follow-up review checks the five proposed evidence areas against the same finite post-fork census. It does not infer source-chain history from successful destination execution.

Audit area	Completed evidence check	Result
Historical gateway discovery	Token-specific registration history plus historical default-gateway events, including retired LPT gateway	Ten identified historical gateway addresses; the returned historical default is the already included standard gateway
Event coverage	Twenty independently partitioned log queries across all ten gateways	Exact reproduction of all 106 events, with no missing or additional identities and no unresolved query errors
Actual execution	Historical traces for all 78 transactions; exact gateway/token/recipient/amount matching for every withdrawal	104 transfer-only withdrawals, one ARB bridgeMint, one GRT mint plus transfer; no trace-fetch gaps or failed frames
Supply and escrow accounting	212 historical totalSupply reads for 106 token/block pairs, plus GRT escrow ledger and historical balance reads	104 unchanged supply comparisons; only the known ARB and GRT mints increase supply; all calls return valid results
Historical permissions and implementation	Historical state, contract-specific upgrade/permission events, and implementation addresses actually executed in traces	GRT's mint permission and executed implementation were installed before the fork; ARB/GRT authorization observations match their recorded mint calls

For 105 transfer-based deliveries, the gateway's transfer-call recipient and amount exactly match the withdrawal event. There are 100 `transfer` calls and five `transferFrom` calls; 103 return true and two return empty data, with no false returns. Token-specific receipt accounting remains relevant: the earlier split-transfer case has two outgoing token movements matching the gross amount. Historical token-proxy event-emission callbacks in SNX and sUSD were identified as such, rather than counted as additional issuance.

The supply comparisons concern whole blocks; they do not alone attribute a change to one transaction or rule out offsetting mint/burn activity. The actual traces and receipts supply that transaction-level context. The observed ARB and GRT increases match their mint calls and events. Current endpoint bytecode was not substituted for the implementations shown in historical execution traces.

The control history also confirms a temporary custom-gateway implementation change on **1 March 2025**, from 20:10:55 to 20:29:25 UTC, before the previous implementation was restored. None of the six returned custom-gateway withdrawal events lies in that block interval, and their actual traces use the restored implementation. Equal implementation values at widely separated snapshots therefore do not imply that no intermediate upgrade occurred. The absence of selected withdrawal events during that interval does not exclude other calls.

GRT's gateway implementation update and minter grant occurred together on **10 April 2023 at 13:54:59 UTC**, block **17,018,223**, in [transaction 0x7130fe8c....](#). The later allowance configuration is the separately recorded 18 April 2023 transaction. These establish inherited execution machinery and authorization; they do not establish a post-fork acquisition of those permissions.

The origin of the existing 1 GRT is now also recorded. On **8 January 2025 at 05:13:55 UTC**, `0xb8698cffffdaaf26ca21fe8aa560c18cc2b2f5996` transferred 1 GRT to escrow in [transaction 0xa9848625....](#). Its later January withdrawal left 1 GRT. The full returned escrow-transfer ledger and historical balance reads reconcile this remainder with the 1 GRT used in September. This connects observed token movements across the two dates without inferring why the deposit was made or who controls another wallet.

One earlier implementation-history query remains incomplete: selected router/standard/custom control events for inherited blocks **14,000,000 through 14,999,999** returned malformed JSON even after narrowing the range. Successful adjacent partitions establish later inherited implementation changes, but do not fill that gap. The complete post-fork control query, all 78 withdrawal traces, all 212 supply reads and the separately verified April 2023 GRT grant are unaffected. This audit does not claim an uninterrupted pre-fork implementation chronology.

These checks strengthen the destination audit. Their remaining scope limits are the selected router/gateway discovery, event signatures, provider history, and audit endpoint. They do not independently demonstrate a pARB source burn, remote collateral custody, or subsequent use of issued GRT to purchase pDAI. Detailed methods and reproduction records are in the [trace study \(Appendix 23\)](#), [coverage and supply study \(Appendix 22\)](#), [control history \(Appendix 10\)](#), and [GRT escrow reconciliation \(Appendix 12\)](#).

Evidentiary effect of GRT

The observed GRT shortfall mint establishes that insufficient L1 token escrow is **not a universal bar to destination issuance**: a particular deployed gateway used its existing token-specific mint permission to supply the shortfall. That result is stronger than merely locating a mint function in source code. It also establishes that ARB is not the only observed destination mint model.

The conclusion remains token-specific. This transaction does not establish mint permission for the existing DAI, USDC, USDT, eUSD, OUSD or Aave receipt contracts; it does not demonstrate continued source investment; and it does not trace issued GRT into pDAI purchases or issuer collateral. Those are distinct transaction and accounting questions. The main paper therefore records an executed bridge issuance mechanism without treating it as a completed stablecoin funding or price-stabilization history.

Bridge and Portal accounting in inspected source

An ordinary L1-origin bridge escrows tokens on L1 and releases that inventory when an accepted return message arrives. The MIM trace exemplifies release. A native-L2 reverse bridge can lock tokens on L2 and mint a destination representation through a specifically authorized L1 token. The ARB trace establishes that destination leg, while source backing remains unverified. Token-level burn/mint systems form a third pattern: they require their own message validation and destination issuance permissions.

Aave Portal is separate application accounting. `V3 mintUnbacked` creates the reserve's ordinary destination `aToken` and increases its unbacked balance. `backUnbacked` transfers actual destination underlying into that reserve and reduces the outstanding amount. Source collateral can support a bridge's obligation, but it does not perform that destination transfer. This follows [Aave's BridgeLogic](#).

In underlying units, $U_{\text{after}} = U_{\text{before}} + \text{issuance} - \text{settlement}$. A purchased pDAI used for settlement becomes reserve cash. It cannot simultaneously be counted as a separate bridge-escrow asset and free treasury inventory. Similarly, a source vault receipt and the underlying it represents are one collateral allocation, not two independent reserves.

Token standards and eligibility

Asset form	Compatibility with a lock/representation model	Additional issue
ERC-20 / PRC-20 fungible token	Transferable balances can be escrowed	The existing destination token must authorize issuance; PRC-20 is not a universal mint permission

Asset form	Compatibility with a lock/ representation model	Additional issue
ERC-4626 vault share	Shares are ERC-20 balances and can be escrowed if transferable	Shares and underlying units differ; conversion, fees and redemption limits matter
Static Aave wrapper or cToken	A transferable receipt can be represented	Exact market identity and exchange-rate accounting must reconcile
Rebasing aToken or OUSD	Requires accounting appropriate to changing balances	A fixed token amount may not represent a fixed share of collateral over time
Curve LP token / Balancer BPT	Transferable pool shares can be represented	Their value and redeemability depend on pool assets; wrapping adds no new backing
Fee-on-transfer token	Nominal requested amount may differ from received amount	Escrow credit must follow the actual accounting behavior
ERC-721 / ERC-1155	Different identifiers and transfer interfaces	The reviewed fungible gateway is not a generic NFT gateway
Nontransferable position	Cannot ordinarily be locked by a standard ERC-20 transfer	Requires a different representation mechanism, if the protocol permits one

The relevant vault standard is [ERC-4626](#). Neither it nor [ERC-20](#) mandates a bridge mint hook. A newly issued representation also has a new identity unless it uses an already integrated token contract; existing pools accept exact addresses.

Across: replenishing both sides without confusing delivery and minting

The May 2023 Across V2 model supports funding on both sides. User deposits replenish an origin SpokePool; hub-to-spoke bridge transfers can replenish L2 inventory; L1 LP deposits and completed spoke-to-hub returns can replenish L1 inventory. Fast fills use the relayer's existing destination tokens, while validated bundles govern refunds and net rebalancing. These capabilities are present in the pinned historical sources, rather than inferred from a modern Across version.

Consequently, a source deposit can remain in the L2 spoke while a recipient already has an L1 payment. If a supported settlement pays the relayer on L2, the assets can stay on that network while ownership moves to the relayer; they no longer remain in that spoke as the original deposit. Opposing flows and pooled settlement can keep aggregate spoke balances replenished and reduce canonical bridge transfers. The same units cannot be simultaneously counted as freely withdrawable by the depositor and available to satisfy unrelated refunds.

These source-defined settlement operations do not themselves mint L1 stablecoins or LP shares against arbitrary L2 deposits. One-direction L1 payouts still consume destination capital, which needs actual replacement through incoming funds, offsets, bridge settlement or separately funded credit. Empty canonical L1 escrow is not repaired solely by accumulating more source-side deposits. The [Across analysis \(Appendix 7\)](#) contains a 100-unit balance example and the exact source references.

An L1 recipient can use delivered tokens in Aave or Curve, subject to the actual token, permissions and local liquidity. The destination fill and DeFi action can be composed within one L1 transaction through a suitable integration. The preceding L2 deposit and subsequent cross-chain settlement remain separate.

Thus L2 capital can support an L1 trade economically, while the actual trade spends destination capital. A future source-chain refund cannot repay a flash loan that must settle before the destination transaction ends.

The configured L1 spoke at `0x5c7bcd6e7de5423a257d81b442095a1a6ced35c5` has a verified inherited implementation, `0xa667498f46457548f1d3ad557340b95fdb290148`, that includes `handleAcrossMessage` and message-bearing fill interfaces. Thus recipient callback capability was present in this proxy's fork baseline, rather than merely available in a repository. A compatible receiver could use that callback for a funded destination DeFi action. [Inherited implementation source](#).

The configured proxy was upgraded on **30 June 2026, 04:52:25 UTC**, at block **26,912,634**, by `0xfeca...d140`, to `0x788e49c4e2b3414777d6671d7411951e81d35808`. Its source is not explorer-verified, so the inherited source does not establish the replacement's complete behavior or its integration with Aave. The [HubPool assessment \(Appendix 14\)](#) records the configuration and upgrade evidence. An earlier version of this paper examined the distinct older spoke `0x4d9079bb4165aeb4084c526a32695dcfd2f77381`, which lacks the callback. Applying that older deployment's limitation to the configured proxy was incorrect; this paragraph corrects that conclusion.

The [successful upgrade transaction](#) traces HubPool → Ethereum adapter → configured SpokePool. The implementation's runtime changes from 22,759 bytes in the inherited implementation to 2,191 bytes in the replacement. Code size alone does not determine its purpose or whether it retains any particular function.

The older contract's `multicall` only delegates calls back to itself and is not a general external-target executor. That fact remains true for the inspected older code, but does not negate the configured proxy's inherited callback. Native callbacks and external transaction composition are distinct mechanisms; neither alone establishes a historical funded Aave/Curve execution or profit.

The [control audit \(Appendix 3\)](#) does establish material changes: HubPool ownership moved to `0xfeca...d140` on 19 June 2026, and the Across Safe's OptimisticGovernor module moved to `e121` on 19 August 2026. Historical traces show module/oracle/Safe execution. This supports a specific governance connection, without proving ownership of all UMA components or a common human controller. At the fixed endpoint, actual HubPool balances and stored liquid reserves for copied DAI, USDC and USDT are all zero. Positive utilized-reserve entries are accounting amounts, not destination cash. Other spoke and relayer inventories were not exhaustively audited by these reads.

Aave and HubPool contract accounting

The [HubPool assessment \(Appendix 14\)](#) checks the explorer-published deployed source and endpoint adapter mappings. Configured chain keys 1 and 42161 identify historical Ethereum and Arbitrum adapters; key 369 has zero adapter and spoke addresses. The recorded spoke upgrade used the key-1 same-chain adapter successfully on PulseChain. The identified standard adapters do not themselves implement an Aave investment strategy.

Supply, loan repayment and Portal backing are different operations

Let c be actual underlying cash held for an Aave reserve, A its aToken claims, B ordinary borrower debt and U its Portal unbacked amount. The simplified principal changes below omit interest, treasury accrual and rounding; all amounts are in equivalent units of the same underlying asset.

Operation	Cash C	aToken claims A	Borrower debt B	Portal amount U
Ordinary supply of x	$+x$	$+x$	unchanged	unchanged

Operation	Cash C	aToken claims A	Borrower debt B	Portal amount U
Ordinary loan repayment of x using underlying	+x	unchanged	-x	unchanged
Repayment of x using existing aTokens	unchanged	-x	-x	unchanged
Portal issuance of x	unchanged	+x	unchanged	+x
Portal backing of x principal	+x	unchanged	unchanged	-x
Ordinary borrow of x	-x	unchanged	+x	unchanged

These operations affect different ledger entries. Ordinary supply adds cash together with a new withdrawal claim; Portal backing settles an already-created claim without issuing another aToken for that payment. Ordinary repayment exchanges a borrower receivable for cash. It does not settle u or mint a reward to the repayer. Repayment with aTokens reduces two claims without adding cash.

The asset must also match. DAI can settle DAI Portal principal. Selling USDC or USDT wrappers for DAI does not allow DAI to directly settle USDC or USDT Portal principal: that would require the relevant underlying to be obtained separately. An oracle's valuation does not make these balances interchangeable inside the reserve's transfer and accounting functions.

The [Portal accounting study \(Appendix 17\)](#) ties these changes to the deployed pool's BridgeLogic, BorrowLogic and ValidationLogic, and the actual waDAI implementation. Portal backing is bridge-role restricted. Any supplied backing fee is separately allocated by the implementation; it must not be assumed to be a fixed percentage automatically charged on principal.

Ordinary repayment on behalf of another account does not transfer that account's collateral to the payer. Liquidation is a separate operation. The recorded repayments establish payment activity; their source has not been reconciled to Portal issuance or native pARB deposits.

Fixed-block reserve observations

A new getter-only batch at block **27,659,393** returned:

Copied reserve	Actual underlying cash	Portal unbacked U	Portal mint cap	Supply cap	aToken total supply, rounded
DAI	0	0	0	338,000,000	398,790,715.764 328
USDC	0	0	0	1,760,000,000	1,069,800,960.53 9750
USDT	0	0	0	200,000,000	720,907,170.978 766

All three report active, not frozen and not paused. Those flags do not override zero Portal caps, supply-cap checks or absent cash. Reported debt-token totals are receivables including accrued interest, not available underlying. The additional fact that DAI and USDT aToken totals exceed their configured supply caps matters independently of the zero Portal caps; a historical cap is not a guarantee that accruing claims remain below it.

In the reviewed implementation, Portal issuance calls supply validation, whose cap calculation includes aToken supply and treasury accrual. DAI and USDT already exceed their caps before adding treasury accrual or any new issuance. This is an additional deterministic barrier at the sampled state. USDC's lower displayed supply does not by itself establish that all its supply checks pass.

Reproduction uses the explicit getter manifest `Research/experiments/parb_loop_reserve_accounting_20260929.json` and the existing read-only collector. Requests and responses are preserved in `artifacts/parb-loop-accounting-20260929/reserves.json`; decoded summaries are in `reserve_summary.json`. The header hash was checked before and after the batch. No state overrides or simulated state-changing calls were used.

Aave control, Portal configuration and stkAAVE

Aave V2, V3 and stkAAVE must be treated separately. V2's provider ownership transfer in [proposal 224's execution](#) is direct control evidence. Its subsequent implementation changes are also recorded. The endpoint V2 implementation differs from the inherited one, and sampled ordinary reserve/provider reads revert. These facts do not establish that the altered implementation retains ordinary V2 behavior or has repaid debt.

The sampled V3 pool is `0x87870bca3f3fd6335c3f4ce8392d69350b4fa4e2`. Its implementation is unchanged at the checked fork and endpoint states. Selected post-fork searches returned no pool `MintUnbacked` or `BackUnbacked` events, no pool/configurator upgrades, no configurator unbacked/supply-cap changes and no selected ACL-role/provider-ownership changes. All **17** listed reserves have **zero** unbacked-mint caps at the endpoint. V2 ownership evidence cannot substitute for V3 bridge-role and configuration evidence.

The reviewed V3 implementation stores the unbacked cap in a 36-bit field: **68,719,476,735 whole units per reserve**, not across all stablecoins. This is a configurable maximum for outstanding unbacked issuance, not current enabled capacity or a lifetime issuance total. Three such maxima sum to 206,158,430,205 nominal units. The endpoint's actual caps remain zero, with separate supply caps and reserve checks. See [ReserveConfiguration](#). The fact that this numerical maximum exceeds the copied DAI supply of about 44.363 billion does not establish available purchasing power or pool cash.

stkAAVE's [10 August 2026 upgrade](#) is real and reports revision 5 at the endpoint. Its `ghoDebtToken()` remains zero. The sampled canonical GHO address, `0x40d16fc0246ad3160ccc09b8d0d3a2cd28ae6c2f`, has no bytecode or listed reserve there. A GHO-related interface is therefore present without evidence here of an operational canonical GHO system. A custom GHO elsewhere is not excluded. GHO facilitator capacity, collateral and debt accounting would be separate facts from the stkAAVE upgrade.

The Aave-Balancer wrapper inventory

The nine-factory census found **38 pools**, of which **17** have direct Aave wrapper links: nine wrappers across six underlying assets. Their returned creation events are pre-fork. The full exact aToken and pool addresses are in the [inventory study \(Appendix 24\)](#).

Aave market	Underlying	Wrapper address	Pools
V2	DAI	<code>0x02d60b84491589974263d922d9cc7a3152618ef6</code>	3
V2	USDC	<code>0xd093fa4fb80d09bb30817fdd442d4d02ed3e5de</code>	4
V2	USDT	<code>0xf8fd466f12e236f4c96f7ccea6c79eadb819abf58</code>	3
V2	FRAX	<code>0x3efceb450fb799f67cd97cdc27a0f4cd7b02b63b</code>	1

Aave market	Underlying	Wrapper address	Pools
V2	TUSD	0x236f4a95361e03267329a558 bdd224b1f95f1377	2
V3	DAI	0x098256c06ab24f5655c5506a 6488781bd711c14b	1
V3	USDC	0x57d20c946a7a3812a7225b88 1cdcd8431d23431c	1
V3	USDT	0xa7e0e66f38b8ad8343cff671 18c1f33e827d1455	1
V3	WETH	0x59463bb67ddd04fe58ed291b a36c26d99a39fbc6	1

Thus the V3 waDAI observation is correct. These are Balancer linear pools using Aave V3 receipts; “V3” does not mean Balancer protocol V3. A V2 DAI wrapper and a V3 DAI wrapper hold different aTokens. They are not interchangeable inputs. No direct V3 wrapper was found within this census for the remaining 13 V3 reserves: wstETH, WBTC, LINK, AAVE, cbETH, rETH, LUSD, CRV, MKR, SNX, BAL, UNI and LDO. Custom factories remain outside the census.

V3 vault share conversion uses local aToken balances, shares and fees. An oracle can affect collateral valuation or other issuance checks, but does not grant mint authority or certify remote escrow. A reported conversion rate, immediately redeemable underlying, pool-tradable output and external dollar value are separate quantities.

Pool capacity and what actual swaps show

The [29 June waDAI transaction](#) exchanged **0.618019167905589973 waDAI for 5.200713616007117708 DAI**. The [4 September waUSDC transaction](#) exchanged **6.679668 waUSDC for 43.641283 USDC**. These establish wrapper acceptance and output payments. They do not establish that the input receipts were minted through Portal or backed by newly locked PARB assets.

The limits are dynamic, not a fixed quota of wa-tokens that may enter forever. At the endpoint, the three sampled stable linear pools have:

Main asset	Registered main cash	Lower / upper target, normalized whole units
DAI	10.055365963093764759	1,000,000 / 5,000,000
USDC	63.044960	1,000,000 / 5,000,000
USDT	9.999901	1,000,000 / 5,000,000

With fee $f = 0.00001$, below-target nominal main balance is $N(M) = M - f(L - M)$. In the wrapped-to-main direction the nominal balance cannot become negative. The corresponding main floor is approximately $fL/(1+f) = 9.999900001$ normalized tokens. Ignoring final integer rounding, this leaves about **0.055465962 DAI** or **53.045060 USDC** for that direction at the endpoint. It is a snapshot bound for a specific route, not a universal bound on transaction volume.

A [10 August USDT-pool transaction](#) recorded **26,235,296.082510 USDT gross input and exactly equal gross output** across its constituent pool swaps. Its large throughput and small ending cash are compatible because balances changed during the transaction. The gross amount is not newly created net USDT.

Accounted additions of main assets can expand output capacity. More wrapper inventory alone cannot replenish main cash. Sending tokens directly to the Vault does not necessarily credit the selected pool's accounting. These linear implementations use swaps against preminted BPT for ordinary liquidity operations; their ordinary post-initialization join/exit hooks revert. Preminted BPT is share accounting, not underlying cash.

Targets and fees are configurable subject to authorization and balance-range checks; the upper target is not an absolute inventory cap. Main/wrapped token identities are immutable in the reviewed implementation. The owner sentinel `0xba1ba1ba1ba1ba1ba1ba1ba1ba1ba1ba1ba1b` delegates authorization and is not an identified person's wallet. The detailed study records structural bounds and rounding. Returned target/fee events for the DAI pool predate the fork; no post-fork target/fee events were found for the three sampled pools.

Reserve eUSD: deployed basket and issuance state

The audited RToken is `0xa0d69e286b938e21cbf7e51d71f6a4c8918f482f`, Main is `0x7697ae4def3c3cd52493ba3a6f57fc6d8c59108a`, and BackingManager is `0xf014fef41ccb703975827c8569a3f0940cfd80a4`. RToken, Main, AssetRegistry and BasketHandler all return version **2.0.0**. Their endpoint implementations were resolved and verified source was retrieved for those exact implementations. The [Reserve study \(Appendix 18\)](#) records each proxy, implementation and getter batch.

`getRegistry()` returns the same fourteen exact token/plugin pairs at inherited block 17,232,999 and endpoint 27,659,393. `getPrimeBasket()` also matches exactly. This establishes matching endpoints; it does not exclude intermediate changes. The active basket has these components, each with target weight **0.25 USD per basket unit**. These weights are protocol target units, not independent evidence of a market dollar price or equal counts of tokens.

Active receipt	Exact token address
saUSDC	<code>0x60c384e226b120d93f3e0f4c502957b2b9c32b15</code>
cUSDC	<code>0x39aa39c021dfbae8fac545936693ac917d5e7563</code>
saUSDT	<code>0x21fe646d1ed0733336f2d4d9b2fe67790a6099d9</code>
cUSDT	<code>0xf650c3d88d12db855b8bf7d11be6c55a4e07dcc9</code>

The registry contains another token named saUSDC at `0x8f471832c6d35f2a51606a60f482bcfae055d986`; it is not the active saUSDC component. DAI, USDC and USDT are registered but are not these four active basket entries. Symbol matching is insufficient to identify accepted collateral.

The cached basket status is **SOUND**, `fullyCollateralized()` returns true, and Main reports neither paused nor frozen. Separately, all four collateral plugins return the price interval **(0, 2¹⁹² - 1)**. The queried USDC and USDT feed updates date to **10 May 2023**, with an `oracleTimeout` of **86,460 seconds**. Issuance refreshes assets before checking basket status. Consequently these cached results and amount quotes do not establish successful current issuance. Exact plugin source was not explorer-verified in this batch, so the next refresh outcome is not asserted solely from the observed stale feeds.

Total supply and `basketsNeeded` each read **13,223,010.934604392856634411** in their respective units. `issuanceAvailable()` reads **1,000,000 eUSD**. The configured hourly throttle takes the larger of one million eUSD and 2.5% of supply; it is replenishing rate capacity rather than a lifetime supply limit. Other issuance requirements still apply.

The deployed issuer's `issueTo` calculates the required active-basket quantities, mints eUSD and transfers every required collateral component to BackingManager within the same atomic transaction. Failed transfers revert issuance. The existing Curve pool `0xaeda92e6a3b1028edc139a4ae56ec881f3064d4f` lists this exact eUSD address. These are facts about issuer accounting and pool identity; this batch has not identified a public issuance funded by newly bridged pARB collateral or a native pARB Reserve deployment.

Reserve Main ownership and timelock history

The Main OWNER is timelock `0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c`. The returned Main role history contains **37 events in three transactions**, all on **23 February 2023**, before the fork. All 44 role/account combinations from that history were checked at both the inherited and endpoint blocks; they match the reconstructed memberships and are identical between snapshots. The [ownership-assignment receipt](#) records the timelock receiving OWNER and the preceding owners' roles being revoked.

The owning timelock's selected history contains **70 events across 14 transactions**, all inherited: 37 scheduled calls, 25 executed calls, five role grants, one revocation and two delay changes. No post-fork event was returned for these selected signatures through the audit endpoint. The timelock's proposer is `0x7e880d8bd9c9612d6a9759f96acd23df4a4650e6`, whose getters return **Governor Alexios**, this same timelock, and voting token `0x18ba6e33ceb80f077deb9260c9111e62f21ae7b8`. The timelock delay is three days.

Direct e121 role checks return false for OWNER, PAUSER, SHORT_FREEZER and LONG_FREEZER. These findings identify this deployment's governance contracts and recorded actions; they do not exclude changes in voting control of Governor Alexios. Proposal, vote, delegation and voting-token ownership histories were not enumerated in this bounded audit. Nor do these results cover every Reserve instance. The [governance study \(Appendix 19\)](#) records all role memberships, event signatures, receipts and limitations.

Other stablecoins, native deposits and Curve connectivity

There is no universal “stake stablecoins” operation. Aave requires listed underlying assets; Origin uses supported vault collateral; mStable uses basket and weight checks; Reserve uses its particular collateral basket. Compound, Yearn, savings vaults and LP pools create different receipts with different redemption conditions. No complete native pARB deployment catalogue has been verified. L1 copies alone do not establish native source operation.

Named asset	Sampled destination facts	Cross-chain conclusion
OUSD	Origin vault supports DAI, USDC and USDT; token issuance is vault-controlled	Existing token's standard-gateway mapping does not establish bridge minting
mUSD	Basket contains DAI, USDC, USDT and sUSD; earlier large basket quotes hit weight limits	Native source issuance and an authorized destination mint remain unverified
eUSD	Version 2.0.0 basket uses saUSDC, cUSDC, saUSDT and cUSDT; endpoint issuance availability was 1 million	Direct deposit of any arbitrary stable is not the sampled basket rule
FRAX	Governed minting, AMO and Ferry contracts exist	Minting authority, collateral checks and transport inventory are distinct
MIM	Examined e121 transaction proves a standard-gateway withdrawal	Evidence is escrow release, not native-L2 lock/L1 mint

Named asset	Sampled destination facts	Cross-chain conclusion
cTokens / vault shares / LP tokens	Transferable receipt families can be represented in suitable integrations	Exact existing destination token permissions and pool acceptance remain asset-specific

The observed router maps OUSD, mUSD, eUSD and FRAX to the standard gateway, whose balances of those four tokens were zero at the endpoint. ARB's separate reverse gateway cannot be generalized to them. Other escrows are not literally all empty: the sampled USDC custom gateway held one USDC.

Existing Curve connections include OUSD/3CRV `0x87650d7bbfc3a9f10587d7778206671719d9910d`, mUSD/3CRV `0x8474ddb98f5aa3179b3b3f5942d724afcd9cf6`, eUSD/FRAXBP `0xaeda92e6a3b1028edc139a4ae56ec881f3064d4f`, and FRAX/3CRV `0xd632f22692fac7611d2aa1c0d552930d43caed3b`. They establish candidate trading connectivity into FRAX, USDC and DAI. The presence of a route is weaker evidence than a successful composed transaction, and several paths share base-pool liquidity. Independent quotes cannot be added as independent available reserves.

Earlier fixed-state quotes and local experiments are described below with their proper provenance. A new wallet-event census provides stronger destination evidence: **820 selected exchange/liquidity events across 820 successful transactions** involve five examined wallets in these four pools. In particular, e121 spent **13,489.157023144354307302 mUSD** to receive **564.501482257993107424 DAI** at block 27,264,036, with both actual transfers in the [receipt](#). Five e121 FRAX-to-USDT swaps also total 731,000 FRAX input and 753,387.520878 USDT output. The funding origin is not connected to L2 issuance by these events. This proves destination trading activity while leaving the cross-chain funding claim unresolved.

The two examined bridge traces do not demonstrate Curve rebalancing. A bounded search of the four named pools returned no selected post-fork amplification, fee or pool-administrator events. That result does not exclude rebalancing by swaps or liquidity movements, upstream factory changes, or unlogged state changes; ABI coverage is incomplete for two pools. A complete historical cash-flow reconciliation remains unestablished. The [Curve study \(Appendix 8\)](#) records the scope.

FRAX configuration and recorded debt repayments

The sampled FRAX AMO minter reports a 700-million FRAX cap, about 517.490 million net minted, and a 78% minimum collateral-ratio parameter. The sampled Fraxlend AMO holds about 3.730 million FRAX and Ferry about 5.598 million. These are endpoint accounting observations, not unrestricted available issuance. Ferry's recorded target chain 42161 and unpaused state do not establish an operating pARB counterpart. The [Ferry implementation](#) transports inventory; the [AMO minter](#) has separate governed issuance rules.

Replenishing FRAX does not automatically replenish USDC or DAI paid out by the next pool. Every claimed replenishment needs an identifiable incoming asset and source. Burning LP ownership does not burn the pool's underlying assets; swapping them out transfers assets to a recipient. Movement into an Aave reserve, a Curve pool, a bridge escrow or a treasury has different ownership and liquidity consequences. The current evidence does not reconcile those destinations into the proposed system-wide accumulation ledger.

A new bounded repayment census found **383 V3 Repay events**, with gross amounts of **2,925,550.790450 USDT, 407,302.146331 USDC**, 444 wei of DAI, 86.003562721157877676 WETH, 160.262345082994421006 rETH and 373.313810010620722431 cbETH. These asset units are not aggregated as dollars. The supplied `6e3B...` wallet appears as repayer for 2,161,000 USDT and 380,000 USDC; `c000...` for 705,000 USDT; `48Fc...` for smaller multi-asset amounts. Representative largest receipts contain matching underlying transfers and debt-token accounting events. These observations substantiate actual repayment activity and wallet participation.

The 383 repayment events span 71 transactions dated 9 July 2023 through 25 November 2025. None in the searched range establishes a repayment wave following the August 2026 stkAAVE upgrade. A receipt also links a 111-wei DAI payment to about 4.805 million DAI of interest crystallization; that accounting amount is not DAI paid.

Gross repayment turnover is not a before/after net debt reduction: new borrowing, interest and liquidations also matter. The funding source has not been linked to native pARB locking or Portal issuance. No standard V2 Repay events were returned; its altered implementation prevents treating that absence as proof that no nonstandard debt mutation occurred. The [repayment study \(Appendix 9\)](#) gives exact amounts, receipts and the distinction between debt burning and interest-related mint events.

The collected evidence does not include a complete historical intervention ledger or a demonstrated external-dollar redemption commitment for pDAI.

The 86ff and 6e3B debt positions

The [additional wallet audit \(Appendix 21\)](#) reads all 17 V3 reserves for `0xe6df4ca976f3fec8da89efec332dc2245f0a86ff` and `0x6e3bf3c7bd91144d694621298cb8da370479a7bb` at the same fixed block. Both have substantial recorded variable debt. Stable-debt balances are zero across the inspected reserves. Representative underlying-denominated balances are shown below; the 86ff account also owes other assets.

Recorded position	86ff	6e3B
DAI variable debt	136,368,622.244154345895913053	50,786,176.522950319448694821
USDC variable debt	260,160,228.034081	7,708,932.009274
USDT variable debt	186,361,487.522170	24,156,631.336535
WBTC variable debt	43,153,535.13073855	0
aWETH collateral claim	105,219.940524291683911174	5,807.947654522616321512
acbETH collateral claim	0	0.679997896010046071
Health factor	0.000129371538181202	0.106605513847397546

WETH is the only supplied collateral reserve for 86ff in this census. The WBTC debt dominates its oracle-valued debt total; this total must not be described as that many units of pDAI or as an equivalent market dollar amount. The reserve's oracle reports prices in a base unit of 100,000,000. Dividing account-data values by that unit yields approximately 1.234 trillion units of oracle-accounted debt for 86ff and 82.673 million for 6e3B. These are protocol valuation outputs, not verified external redemption values.

Both accounts have zero reported available borrowing. Their health factors are below 1, so ordinary removal or transfer of collateral aTokens remains subject to restrictive health checks. Collateral availability is not the same as free accessibility: the WETH and cbETH reserves have actual underlying cash at this snapshot, but that does not waive either borrower's collateral constraints. The sampled V2 `getUserAccountData` calls revert for both addresses; this does not establish zero V2 positions or extend the V3 findings to changed V2 code.

A debt token is not a transferable creditor claim

An Aave aToken represents a supplied-asset claim. An Aave variable- or stable-debt token instead records what its account owes. The fixed-block DAI variable-debt implementation is

`0xac725cb59d16c81061bdea61041a8a5e73da9ec6`; its [verified source](#) explicitly rejects `transfer`, `transferFrom`, `approve`

and allowance as unsupported. A generic withdrawal therefore cannot transfer that debt to the caller. The sampled DAI stable-debt implementation was not explorer-verified; both wallets' sampled stable-debt balances are zero. This study does not claim to have separately verified every receipt-token implementation from one example.

Repaying a debt on behalf of 86ff burns or reduces the corresponding debt balance belonging to 86ff. It does not give the payer ownership of 86ff, ownership of its aWETH, or an exclusive right to a later withdrawal. If a permissionless withdrawal function exists, its own rules and Aave's collateral checks determine whether a particular asset can leave. Paying toward debt is not, by itself, acquisition of that right. Aave liquidation is a separate operation; ordinary repayment must not be assigned liquidation's collateral transfer semantics.

The description of 86ff as permitting public token withdrawals is a specific additional contract claim. The explorer source lookup returned no verified source or ABI; full- and partial-match Sourcify metadata lookups also returned no match. Its runtime is present and measures 20,578 bytes. These observations do not independently establish the permissions or recipient rules of every withdrawal path. Recorded withdrawals can substantiate particular paths without proving every caller can withdraw every token.

A [bounded review of two existing transactions \(Appendix 1\)](#) did not find a withdrawal. The successful [2 June 2026 FdbEE transaction](#) reads cETH's existing allowance from 86ff to FdbEE, returning the maximum unsigned integer. It does not grant that allowance or transfer tokens. The [8 March 2026 transaction](#) has a successful outer receipt but a reverted internal self-delegatecall and no token transfer. Neither trace includes an Aave call. These bounded findings do not disprove every possible public withdrawal path; they establish that these two successful outer receipts are not withdrawal or repayment evidence. An allowance for cETH is also a different permission from ownership of Aave collateral or transferability of an Aave variable-debt token.

The significance for the thesis is narrower but real: paying supported debts can replenish reserve cash, and sufficiently improving a borrower's position can permit collateral use under the rules. The entire balance sheet matters. Repaying only the stablecoin debts does not erase 86ff's WBTC and other debts, and repayment financed by Portal receipt sales leaves the Portal obligation to be settled separately. The size of a debt balance is neither a profit amount nor evidence that the whole proposed cycle has been implemented.

Examined addresses and attribution

The full e121 address is now identified. Two other examined addresses intersect with receipt-verified Aave governance history, and the final address is the copied rollup contract itself. Their presently established roles are recorded in the [address evidence note \(Appendix 6\)](#).

A common transaction destination, interacting with the same protocol, or submitting an executable governance proposal does not establish common key ownership. This study does not attribute these addresses to Richard Heart or a particular organization. The audit makes no finding about the participants' motives.

Coverage of the original ten research subjects

Subject	Finding established	Stronger claim still unresolved
1. Control and governance	Specific bridge changes, V2 control transfer, Across module/HubPool ownership changes and configured-spoke upgrade	Unified ownership of every relevant system

Subject	Finding established	Stronger claim still unresolved
2. L2 collateral / L1 issuance	ARB and GRT destination mints traced; MIM escrow release traced; nine gateways classified	Independently verified source locks backing the claimed stable issuance
3. Limits	V3 maximum is per reserve; all sampled actual unbacked caps are zero	Currently enabled tens-of-billions Portal capacity
4. V3 waDAI	Exact wrapper, aToken and pool linkage verified	Historical Portal origin for traded wrappers
5. Native assets	Nine direct wrappers and 17 destination pools inventoried; asset requirements distinguished	Complete native pARB protocol and reserve catalogue
6. Oracles / fungibility	Local share conversion and mint authority are different mechanisms	Remote backing and external-dollar redemption
7. OUSD, mUSD, eUSD, GHO	Named token routes checked; eUSD v2 registry/basket endpoints match; stkAAVE upgrade proved	Native-source-to-existing-L1-token mint paths for these assets
8. FRAX replenishment	Separate AMO issuance, lending and Ferry functions identified	Closed cash-flow replenishment across all downstream assets
9. Accumulation and repayment	Escrow withdrawals, wrapper trades, V3 repayments and the 86ff/6e3B debt positions established	Source funding, net debt decline, repayment profit and reconciled DAI accumulation
10. Peg evidence	No complete intervention or external-dollar redemption record established in collected evidence	A demonstrated, sustainable \$1 external-value guarantee

All ten subjects are addressed. “Addressed” does not mean all ten stronger claims were confirmed; the evidence supports different conclusions for each.

Provenance of earlier experiments

The earlier edition included isolated local Anvil experiments under assumed Aave administrator control. They locally enabled issuance and executed small DAI mint/wrap/swap/settlement cycles. Another destination-only experiment traversed eUSD → FRAX → USDC → DAI and left a settlement shortfall. These were conditional execution demonstrations with modified local configuration, not public PulseChain transactions and not verified source-L2 deposits. Their outputs and experiment identities are preserved; they are excluded from this paper’s historical transaction trail. No such transaction experiments were run for this retrospective update.

Review of the saved small DAI experiment confirms that its reported *passed* status did not mean full settlement. Two 0.01-DAI issues produced a total 0.019999803158361345 DAI for backing, leaving 0.000000196841638655 DAI of Portal unbacked principal. A separate fixed-time experiment also left a nonzero remainder. These examples demonstrate why successful individual calls and a successful test label are insufficient proof of a fully settled, profitable loop.

Earlier read-only quote probes likewise establish only their sampled call results. In particular, the nominal 250-billion-input quotes did not show 250-billion payouts, and the local route did not establish remote backing. This edition relies on the public transactions above for its positive historical findings.

Research record

Maintained read-only collectors are `Research/scripts/probe_pulsechain_routes.py` and `Research/scripts/collect_pulsechain_events.py`. Explicit experiment manifests live under `Research/experiments/`; raw results remain under ignored `artifacts/parb-*` directories. Historical collections preserve request parameters, responses, event signatures, receipts and block hashes. Companion studies identify the relevant batches and source revisions. No public-chain transactions, governance votes or configuration changes were submitted.

Appendix 33: How source-chain assets could support pDAI demand on PulseChain

Contents of this appendix

A conditional mechanism report, with a PLS-denominated price objective	202
Abstract	203
1. Define the outcome before discussing the machinery	203
2. The premises of the theoretical system	205
3. The principal path: invested assets become a marketable claim	205
Exact token identity matters	206
What “backed” means in this argument	207
A complete stipulated financing ledger	207
Other financing families can also close their ledgers	208
4. The alternative transport and financing paths	209
Arbitrum reverse gateways: the receipt can stay invested	209
GRT and LPT: token-specific issuance can replace missing escrow	210
Aave Portal: temporary local issuance with a backing obligation	210
Across: financing delivery through existing destination capital	211
Gnosis, Polygon and ordinary return bridges	211
Other protocol families and downstream liquidity	211
5. How the paths converge on pDAI demand	212
6. A finite-input example of reaching the PLS-valued target	212
An executable quantity, rather than only a marginal quote	213
Trading fees and thirty successive local exits	213
7. From reaching a price to sustaining it	214
Appreciation can reduce the PLS requirement and enable reserve rebuilding	215
8. What the audit contributes to this possibility argument	216
9. Could governance remove the roadblocks?	217
10. What would strengthen or falsify a particular composition	219
Conclusion	219
Evidence and source index	220

Source record: docs/studies/analysis/pulsechain_dai_mechanism_analysis_20260929.md.

How source-chain assets could support pDAI demand on PulseChain

A conditional mechanism report, with a PLS-denominated price objective

29 September 2026. Theoretical synthesis based on historical contract models and a read-only PulseChain audit. The full [evidence audit \(Appendix 32\)](#) is maintained separately, with transaction references, addresses and reproduction records. No transactions, deployments or governance changes were made for this report. The numerical examples are mathematical models, not blockchain

simulations. Deployment observations refer to PulseChain chain 369, block **27,659,393** (28 September 2026), hash `0xc5e582cd902f4bfff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275`. They are not claims about every later state. A [two-round skeptical review \(Appendix 59\)](#) separates the surviving conditional result from stronger claims the report does not prove. The subsequent [twelve-point follow-up \(Appendix 72\)](#) adds local-exit evidence and bounded financial models, and identifies the unfinished detailed permission review separately from completed findings.

Abstract

There are coherent conditional paths by which source-chain assets could support purchases of existing pDAI on PulseChain. A transferable receipt for invested source assets can be locked on its source chain while an authorized representation circulates on PulseChain. That representation can support buying power through a market sale, an eligible collateralized issuer, or a funded loan. Other paths use source-token burning and destination minting, temporary Aave Portal issuance, or Across delivery funded by existing destination liquidity. These paths have different backing and settlement requirements, but can converge on purchases of pDAI through local markets.

This report examines possibility rather than requiring an observed end-to-end operation. It treats source-network operation, specified administrative control and necessary integrations as explicit scenario assumptions. The audit supplies concrete examples of destination issuance, escrow withdrawals, governance activity, wrapper acceptance and trading. It does not establish any participant's future plan. The report makes no attribution of motives or human wallet ownership.

The target is **a PulseChain exchange price at which pDAI purchases \$1 worth of PLS**, using a stated PLS dollar reference and trade size. Ethereum redemption is not a requirement. A finite-input market example demonstrates that this target is mathematically reachable in a specified market. Whether the proposed funding paths can supply the required buying power in actual markets remains a condition, not a forecast. Reaching the target and maintaining it are separate propositions.

The holder's exit is a **local exchange of pDAI for PLS on PulseChain**. No bridge to Ethereum or external-dollar redemption forms part of that outcome. A fee-inclusive extension now demonstrates thirty successive target-valued local exits under an explicitly funded, bounded scenario, alongside cases where a smaller budget or changed PLS reference fails. Those are conditional mathematical results rather than measurements of the deployed network.

The report establishes three limited results: historical mechanisms exist; a stipulated receipt-sale ledger can finance a purchase without double-counting assets; and sufficient input can reach the target in a specified market model. It does **not** derive that funding from validator control or nominal copied balances. A complete route using the named deployed contracts at the audited snapshot remains unestablished. The broader claim that control and copied collateral alone ensure the target does not follow from the conditional result.

1. Define the outcome before discussing the machinery

Let $s(t)$ be the chosen dollar reference for one PLS at time t . Let $o(q,t)$ be the net PLS obtained by selling q pDAI through the specified PulseChain market route, after its trading fees and price impact. Then:

$$\text{pDAI execution value}(q,t) = s(t) \times o(q,t) / q.$$

The proposed target is an execution value of at least one dollar for the stated quantity. PLS may be exchanged through its one-for-one wrapped form where the route supports wrapping and unwrapping. Network costs should be reported separately if the objective is proceeds after all transaction costs.

Here, “redemption into PLS” describes a local market exit. It does not assert that pDAI's token contract guarantees a fixed PLS payout. The relevant test is whether the selected funded route actually delivers the stated amount of PLS.

Cash flow	Required for the local exit?	Role in the analysis
Holder sells pDAI for PLS, possibly receiving and unwrapping WPLS	Yes	The measured outcome
A financing protocol settles its own debt, Portal issuance or collateral claims	Only where the selected financing route creates that obligation	Determines available funding and later local liquidity
A representation holder returns a claim to its source chain	No	Optional claim lifecycle; relevant if backing or settlement assumptions rely on it
Holder bridges assets to Ethereum or cashes out into external dollars	No	Outside the target and the support model

A source-return restriction therefore does not automatically prevent a local pDAI-to-PLS swap. It can still affect how much a counterparty pays for a receipt, or whether a financing arrangement remains solvent. Those are separate tests.

The [local wrapper evidence \(Appendix 41\)](#) supports the native-unit equivalence for these exact assets:

Local exit asset	Identity	Conversion basis
PLS	PulseChain native currency	The unit used in the local-exit model
Canonical WPLS	0xa1077a294dde1b09bb078844df40758a5d0f9a27	Deposit/withdrawal exchanges equal native PLS units
Inherited WETH	0xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2	Its unchanged WETH9 logic pays native PLS on PulseChain

Thus a compatible local route can end in any of these forms, normalized into PLS units before conversion costs. Their token addresses remain distinct, and an arbitrary bridged external-ETH token using the WETH label is not covered by this result. Wrapped claims and the native currency backing them are the same allocation of assets, not independent reserves that can be added together.

This definition needs a disclosed PLS dollar reference independent of the pDAI market being evaluated, but does not require pDAI redemption on Ethereum. A copied USDC symbol or an oracle value of one is not automatically that reference. A marginal pool price, an executable quote for 1,000 pDAI, and the proceeds from selling billions of pDAI are different measurements. A report claiming the target should identify which it means.

Because PLS itself has a variable dollar price, a fixed pDAI/PLS ratio does not necessarily maintain a fixed dollar value. The necessary target ratio changes with $s(t)$. Applying a reference price to a PLS quantity is a valuation convention. It does not establish that the entire PLS output could be exchanged for that many external dollars. Any claim of external liquidation proceeds would additionally need the reference venue's executable depth; that is outside this local target.

2. The premises of the theoretical system

The conditional system considered here assumes:

1. An operating source environment, such as the proposed pARB, with the stated usable asset balances and functioning source applications.
2. Effective authority over the particular bridge, messaging and application roles required by the selected path. Common administration is a scenario premise, not a conclusion about the identified wallets.
3. An authorized destination token integration and, where needed, a compatible collateral adapter, issuer, lender or market.
4. Actual destination purchasing liquidity, including the relevant pool outputs or a funded counterparty. Nominal token quantities are not substituted for executable purchasing power.
5. Consistent accounting for backing, outstanding claims, fees and settlement.

Copying Ethereum's L1 state did not itself copy every L2 execution database. The source environment is therefore an additional premise. For theoretical analysis, the absence of a public pARB RPC does not prevent evaluating that premise; it limits the historical claims that can be made about it.

Validator and messenger authority governs acceptance of cross-chain statements. Token issuer permissions govern destination issuance. Application governance governs eligible assets and protocol settings. These are distinct roles, even when the scenario assigns them to one group. An accepted statement alone is not an asset with market value. The scenario needs an executable funded exchange or another explicit funding basis, not merely an asserted balance. A funded pool or lender can mechanically accept a weakly backed claim and release existing assets. Independent collateral value is not a universal technical prerequisite for that finite transaction. Its effect on solvency and later redemptions must be considered separately.

Authority over upgradeable components allows analysis of additional supported configurations. Such configurations are identified as assumptions rather than inherited capabilities already in operation. Immutable contracts and unrelated token issuer permissions are not made controllable by ownership of nearby proxies. This keeps the argument more precise than assuming arbitrary new code can do anything desired.

Three configuration categories should not be conflated: **the observed deployed state, existing code under different authorized parameters**, and **new integrations or changed implementations**. A path needing a new collateral adapter or token mapping belongs to the third category until its exact interface is specified. Historical code availability does not by itself put it in the first category. This report does not use arbitrary replacement code as proof that the inherited infrastructure already supplies a complete route.

The required control assumption also affects the credibility of claims. An authority able to change source balances or custody rules may affect dilution and redemption expectations. Control establishes neither a market haircut nor a guaranteed price; funded acceptance and any assumed constraints on later changes must be specified separately.

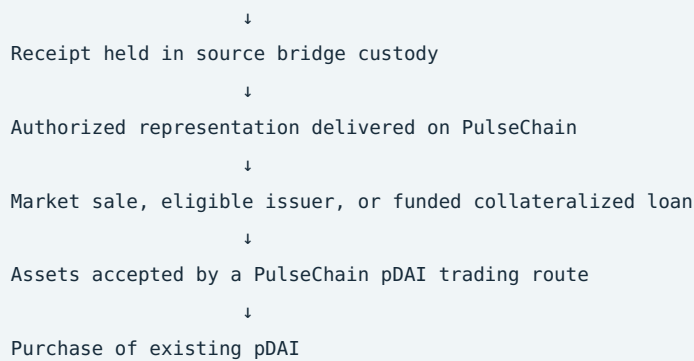
3. The principal path: invested assets become a marketable claim

The central intuition has a coherent accounting form:

Source assets invested in a functioning protocol

↓

Transferable receipt representing that position



In this model the bridge holds the **receipt**, while its underlying assets can remain invested in the source protocol. The destination representation is a claim on that receipt. Neither the source underlying nor the same receipt can also remain independently spendable by the original depositor.

Transferable ERC-20/PRC-20 tokens, transferable ERC-4626 vault shares, cTokens and static lending receipts are relevant asset forms. Rebasng receipts need compatible share/rate accounting; their changing balances are not automatically synchronized across chains. Nontransferable positions and NFT interfaces do not fit an ordinary fungible-token gateway merely by sharing a descriptive label. ERC-4626 itself permits nontransferable shares, and its conversion views are not guarantees of executable redemption. The actual share transfer and withdrawal rules matter. Its normal asset-funded `mint` method is not a general bridge mint permission. [ERC-4626 specification](#).

Three destination choices can turn this representation into purchasing power:

- **Direct sale.** A market accepts the exact representation and exchanges pDAI, PLS or an intermediate asset for it. The buyer or pool acquires the claim.
- **Local collateralized issuance.** An existing issuer accepts the representation as eligible backing and creates its own token. A pool already listing that issuer's token can accept it without treating the bridge representation as the same token.
- **Funded borrowing.** A lender accepts the representation as collateral and lends existing destination assets. Those assets can fund a purchase, while the borrower retains a repayment obligation.

None of these choices requires Aave Portal. Their common economic requirement is that the delivered claim obtains actual buying power on PulseChain.

Exact token identity matters

A representation of source aDAI is not automatically the existing PulseChain Aave aDAI contract. Similarly, a bridged token called eUSD is not automatically the eUSD address already accepted by a Curve pool. Shared symbols, decimals or numerical addresses across chains do not establish interchangeability.

The local-issuer route provides a separate way to obtain an existing pool coin: The bridge delivers collateral x , and the local issuer creates token y against eligible x . x and y remain different assets. Reserve eUSD supplies a concrete basket-based architecture for this distinction. Its collateral registration, active basket, valuation, issuance limits and actual transfers must all fit the hypothetical configuration. In a multi-asset basket, supplying one component does not replace the others.

The audit found an existing eUSD issuer and a Curve pool accepting that exact eUSD token. It did not find a newly installed pARB receipt in the sampled eUSD basket. The additional integration is an explicit condition of this path. [Reserve source and state study \(Appendix 18\)](#).

What “backed” means in this argument

Custody coverage means the representations correspond to exclusive source receipts. **Source solvency** concerns whether those receipts' underlying claims are recoverable against the source protocol's assets and liabilities. **Redemption liquidity** concerns what can actually be withdrawn, and when. **Purchasing capacity** is the quantity of destination assets that an executable sale or funded loan delivers. None of these measures alone establishes all the others. A trillion units can be fully escrowed and receive no funded bid.

A collateral oracle can affect what a configured protocol accepts. It cannot by itself create the output assets paid by that protocol. Conversely, a protocol can spend its existing inventory against a declared collateral value even if later liquidation would recover less. The finite purchase can succeed while the funder bears a shortfall. That distinction is necessary for a neutral assessment of technical possibility and economic sustainability.

A complete stipulated financing ledger

To connect receipt transport to the market example, assume a source investor owns **1,000 transferable shares** in a functioning source vault, representing its claim on **1,000 underlying units**. The source investment's units and value are not assumed to equal dollars. Assume an authorized one-to-one representation, and a destination counterparty with existing PLS that accepts all 1,000 claims for the exact buying budget $\mathfrak{B}$ in section 6's finite-sale example: approximately **2,167,281,613.012984 PLS**. This is a stipulated bid, not a quote observed for any audited receipt. The idealized ledger excludes transaction costs; real execution needs separately funded costs and applicable timing.

Participant	Before	After custody, representation sale and pDAI purchase
Source vault	1,000 underlying units; 1,000 share claims outstanding	Same underlying and share obligations, ignoring yield or loss
Source investor / destination purchaser	Owens 1,000 source shares	Owens approximately 684,271.838699 purchased pDAI; has sold the represented claim
Source custodian	No allocated shares for this position	Holds 1,000 shares committed to the outstanding destination claim
Destination representation system	No outstanding claim for this position	1,000 representation units outstanding against those shares; no extra independent underlying
Funded destination counterparty	At least $\mathfrak{B}$ spendable PLS	Holds 1,000 representations; PLS balance falls by $\mathfrak{B}$
Illustrative pDAI/PLS AMM	Section 6's initial reserves	PLS increases by $\mathfrak{B}$; pDAI decreases by the purchaser's received amount

The source investor first commits the shares, receives the representation, sells it for $\mathfrak{B}$, and then purchases pDAI. No participant spends a balance before receiving it. Source assets stay invested because the counterparty acquires the exclusive claim. The counterparty could be a buyer, or a funded market whose rules produce the stipulated total output. The bridge creates the representation; the counterparty supplies the PLS. This demonstrates consistent conditional composition, without establishing that an audited deployment provides that bid or implements every required interface.

A borrowing variant retains a borrower liability and lender receivable; a local issuer variant retains the issuer's collateral/redemption obligations and still requires a funded market for its output. Neither variant can count the same receipt, debt proceeds and underlying as independent capital. A complete deployment-specific proof would replace each stipulated interface and bid with the exact contract behavior, amounts and executable market curve.

Other financing families can also close their ledgers

The [local financing study \(Appendix 42\)](#) supplies complete illustrative ledgers for the main alternatives. These establish accounting consistency under explicit acceptance and funding assumptions, not generic compatibility between arbitrary receipts and the named deployments.

Family	Source of local purchasing assets	Position remaining after the purchase
Receipt sale	A buyer or market pays existing local assets for the exclusive representation	Buyer holds the source claim; seller holds purchased pDAI
Collateralized borrowing	A funded lender advances local assets against the accepted representation	Borrower holds pDAI and owes the loan; collateral secures the lender's receivable
Eligible local issuance	An issuer accepts required collateral; a funded market buys its issued token	Issuer holds collateral and owes its token's rights; market holds the issued token
Portal receipt sale and backing	A market pays pDAI for compatible local wrapper claims; a separate contribution funds any shortfall	Aave holds backing pDAI, the market holds receipt claims, and the operator retains only any actual net surplus
Across-style funded delivery	Relayer supplies existing local tokens and is reimbursed in the configured source asset	Source ownership changes to the relayer; the local inventory used for delivery requires replenishment

These are different ways to mobilize or relocate assets. A source receipt can remain invested in the sale, borrowing or issuance examples; in the Across example that remains possible only if the configured source asset is an admissible investment receipt. The same source allocation cannot remain freely spendable by the original holder after being sold, pledged or used as backing.

For example, a funded lender can advance 700 PLS against an accepted 100-unit representation. Spending those 700 PLS creates a local purchase while leaving a 700-PLS principal debt. Its future payment does not have to occur before the purchase, but belongs in the support horizon when due. Another example issues 100 local issuer tokens against eligible collateral and sells them for 700 PLS; the output market supplies those PLS and inherits the issued claim. These are stipulated quantities, not dollar valuations or observed offers.

For Portal, issuing 100 pDAI-equivalent claims and selling their wrapper for 99.9 pDAI can be fully settled with a separately funded 0.1-pDAI contribution when the backing fee is zero. The operator then holds no free pDAI from that cycle; the market holds claims and Aave holds 100 pDAI backing them. This closes the accounting instead of leaving the shortfall implicit. Each stage must also pass its own post-operation deposit limits and other validation conditions.

These financing parties may have later settlement or redemption obligations. The **pDAI holder's local PLS exit does not require performing those obligations or bridging out**. They matter to how much funding remains available and whether the financing system can repeat its operations.

4. The alternative transport and financing paths

These are families of mechanisms, not a claim that every named route is presently configured on PulseChain. The historical baseline is the May 2023 fork boundary; later protocol features are not silently added to it.

Path	What reaches PulseChain	What happens to the source asset	Essential condition
Arbitrum reverse gateway	Authorized representation of a native source token or receipt	Locked in source custody	Exact integration and a destination market or collateral use
GRT-style specialized gateway	Existing destination token, funded by escrow plus an authorized mint shortfall	Historical design burns the source tokens withdrawn	Token-specific mint permission, allowance and accepted message
LPT-style gateway	Existing destination token from escrow and, if necessary, an external minter	Source exit follows its token-specific bridge rules	Effective minter integration; capability differs from observed execution
Aave Portal	Existing local aToken, potentially wrapped into its compatible wa-token	Source funding arrangements are separate from the local aToken ledger	Enabled issuance, market liquidity and actual local backing settlement
Across	Existing destination tokens delivered by a relayer or protocol inventory	Deposit enters settlement; repayment may occur on the source chain	Funded destination delivery and collectible settlement
Gnosis Omnibridge reverse representation	Mapped or factory-created representation	Native source token or compatible receipt is locked	Operating counterpart, authorization, limits and downstream acceptance
Polygon PoS mintable predicate	Mapped root token, using inventory and authorized shortfall minting	Child token is burned under the bridge model	Correct mapping, authenticated exit and root-token mint authority
Ordinary Arbitrum or legacy Optimism return	Existing destination token released from escrow	Source representation is burned	Sufficient destination escrow and valid withdrawal accounting
Token-level burn/mint integration	Exact token authorized by its own issuer/message system	Source units are burned	A verified token-specific integration; generic messaging is insufficient

Arbitrum reverse gateways: the receipt can stay invested

The historical reverse-gateway model locks native source tokens and mints their authorized L1 representation. For a transferable vault receipt, this preserves source investment while changing who can circulate the claim. In a simple one-to-one, non-rebasing model, outstanding representations must correspond to the receipts committed to them. The same escrow allocation cannot independently back multiple unrestricted claims.

At settled checkpoints, representation supply can equal its allocated escrow. During asynchronous transfers, account for exclusive pending issuance and return rights as well. In a simple lossless model without donations, rebasing, fees or other allocations:

committed escrow = outstanding representations + pending issuance rights + pending return rights.

Each right is counted once as it changes status; this is an accounting model, not a claim that every term is exposed by an existing getter. Multiple destinations must be reconciled against the same allocation. If its holder chooses a source return, the representation is retired before the corresponding source receipt is released. This is not a prerequisite for a pDAI holder's local PLS exit. The receipt holder can then seek source redemption under that vault's limits, queues and solvency. The receipt's current asset conversion rate is separate from its bridged share count; a destination oracle does not automatically track or enforce that rate.

Historical reverse-gateway code and ARB's inherited integration demonstrate the architecture. The recorded PulseChain ARB transaction demonstrates destination minting. Applying the architecture to a different receipt still requires that receipt's own integration and destination acceptance. [Historical source and accounting \(Appendix 13\)](#).

GRT and LPT: token-specific issuance can replace missing escrow

GRT is the clearest executed example of a destination escrow shortfall being filled by issuance. The September 2026 transaction combined 1 existing GRT with 62,875,936.675361278837595998 newly minted GRT. Its mint authority and allowance parameters were inherited; the allowance accumulated with block height.

The historical GRT model burns withdrawn source GRT and can mint an allowed shortfall on L1. It therefore supports a distinct part of the thesis: sufficient pre-existing L1 escrow is not universally necessary for delivery of an integrated token. It does not demonstrate continued investment of the same withdrawn GRT. [Official historical GIP-0037, PulseChain transaction reconciliation \(Appendix 12\)](#).

LPT's gateway source includes an external-minter shortfall branch, but the recorded LPT withdrawal used existing escrow. HGOLD's gateway contains a bridge-mint call, while current token-level permission remains unresolved and the selected post-fork search found no execution. These are additional capability categories, not additional demonstrated mint transactions. [Specialized gateway study \(Appendix 20\)](#).

Aave Portal: temporary local issuance with a backing obligation

Portal can create the ordinary destination aToken before destination underlying arrives, subject to bridge authorization and reserve checks. A compatible static wrapper can accept that aToken, and a market accepting the exact wrapper may exchange existing pDAI for it. This can move pDAI from an AMM into Aave's backing flow while leaving the market holding wrapper shares whose underlying redemption remains subject to Aave cash and withdrawal rules.

Portal has an additional accounting obligation: the destination reserve's unbacked amount must be settled with its actual underlying asset. Source collateral can support the funder's obligation, but it does not itself become cash inside destination Aave. Ordinary supply and ordinary loan repayment do not settle Portal's unbacked balance.

For a DAI-reserve Portal cycle, if M units of pDAI principal produce Q pDAI from selling the compatible wrapper, backing principal plus an actual backing fee F solely from those proceeds requires $Q \geq M + F$. A shortfall requires separate funding. After backing, the transferred pDAI is reserve cash supporting outstanding claims; it is not also free treasury inventory. Backing USDC or USDT issuance requires that reserve's underlying, not simply the DAI obtained from a different sale.

This path can affect liquidity distribution and trading prices. It is not, by itself, an unlimited source of net assets or permanently locked pDAI. The audited stablecoin Portal caps were zero, so enabled operation is an additional scenario condition rather than an observed configuration. There is also an independent supply-cap barrier: sampled DAI aToken supply was about **398.79 million** against a **338 million** cap,

and USDT supply about **720.91 million** against a **200 million** cap. The deployed Portal path applies supply validation as well as the unbacked cap. Enabling Portal capacity alone would not establish those paths' availability under otherwise unchanged state. [Deployed-code accounting and wrapper study \(Appendix 17\)](#).

Across: financing delivery through existing destination capital

Across can connect source capital to an L1 purchase without requiring the same units to complete a canonical bridge transfer first. A source deposit supports a relayer delivering existing PulseChain tokens. The recipient may then trade or supply those tokens locally, while later settlement repays the relayer.

If repayment occurs on the source chain, assets can remain on that network while the relayer's inventory changes location. The same source deposit does not remain an unrestricted claim of the original depositor after repayment. Source spoke replenishment, destination hub replenishment and relayer inventory are separate balances.

Supported callbacks can combine destination delivery with a local DeFi action. That local transaction can be atomic; the entire cross-chain sequence is not. Continuous one-direction delivery needs replenished destination capital, offsetting flows or another funded credit arrangement. The current copied spoke's replacement implementation was not source-verified in the audit, so its inherited callback behavior is not assumed unchanged. [Historical Across model and balance example \(Appendix 7\)](#), [actual HubPool/spoke observations \(Appendix 14\)](#).

Gnosis, Polygon and ordinary return bridges

Gnosis Omnibridge's historical reverse mode supports source-native custody and destination representations, including representation creation through its factory model. This can accommodate a transferable source receipt in a compatible integration. Gnosis is a separate chain, not an Arbitrum rollup. The separate xDAI bridge's DAI-origin return releases its backing and is not the generic Omnibridge reverse-mint mechanism.

Polygon PoS's historical mintable predicate supplies a different source-burn / root-mint model. It can use root inventory and mint a shortfall when authorized for that root token. Ordinary escrow predicates do not acquire that permission merely by sharing Polygon's messaging infrastructure.

The audit established inherited PulseChain-side presence for an Omnibridge proxy and a Polygon mintable-predicate proxy. It did not establish every operational counterpart or token mapping; the sampled Polygon implementation also lacked verified explorer source. Their conditional roles are anchored in historical code, with deployment-specific behavior left as an assumption. [Historical sources and presence reads \(Appendix 7\)](#).

Ordinary Arbitrum and legacy Optimism returns remain useful when destination escrow is funded. They provide existing assets, not a general shortfall-mint facility. Their contribution is transport of available purchasing assets.

Other protocol families and downstream liquidity

Reserve issuance and receipt-backed borrowing are destination uses of collateral, not separate bridge types. Origin, mStable and other issuers have their own supported assets and valuation rules; lending systems, including different Aave or Morpho designs, have their own eligibility and funding rules. The report does not assign those names generic acceptance of an arbitrary pARB receipt. No new Origin/Morpho-specific collateral route is established by this synthesis.

FRAX AMO issuance or lending can contribute FRAX where their own conditions permit it. Ferry transports balances. Supplying FRAX to one pool does not itself replace USDC, DAI or PLS paid out by the next pool. Similarly, a stkAAVE upgrade does not by itself establish an operating GHO issuer; the sampled GHO linkage was not operationally established. These may be additional integrations in a scenario, rather than presumed sources of unlimited cash.

Authenticated messengers and AMBs can connect compatible applications, but the destination receiver determines whether a message transfers, mints or records something. A message is not itself a token or purchasing asset. New custom systems are possible in principle, but cannot be catalogued as already inherited protocol capabilities without specifying their accounting and authorization.

5. How the paths converge on pDAI demand

Once a participant has an accepted destination asset, an available route might exchange it directly for pDAI or pass through FRAX and USDC first. The eUSD/FRAXBP, FRAX/base-pool and other Curve connections supply possible market links; actual mUSD-to-DAI exchanges and Aave-wrapper trades supply observations of individual local links. This supports the composition argument without claiming the bridge mints financed those observed trades.

Purchases increase the input asset held by the market and decrease its pDAI inventory. AMM prices respond to the resulting balances. An acquired pDAI can be retained, supplied, escrowed or used to settle an obligation, but those choices have different effects on later availability. Aave deposits and bridge escrow are commitments with release conditions, not necessarily permanent burns.

Finite output liquidity remains relevant even when mint capacity is large. A pool limit can mean available cash, a pricing constraint, an issuance cap, or a replenishing rate limit. These are not interchangeable. Adding appropriate liquidity may increase capacity, but also changes inventory and the price path; adding more of the token being sold does not conjure the missing output asset. [Balancer limits and observed swaps \(Appendix 16\)](#).

The circulating supply need not all be purchased to raise a market price. Conversely, a numerical issuance ceiling larger than total pDAI supply does not prove sufficient buying power. A receipt's conversion rate, market discount, fees, competing pools and newly offered pDAI determine the relevant quantities. Multiple routes sharing one base pool share its inventory; their individual quotes cannot be added as independent reserves.

6. A finite-input example of reaching the PLS-valued target

This example is constructed to demonstrate possibility. It is **not a measurement of a PulseChain pool, a current PLS price, or an estimate of the actual cost**. Assume a fee-free constant-product market, no other trading, and a fixed PLS dollar reference of \$0.0001. Let x be pDAI reserves, y PLS reserves and $k = x \times y$. Its marginal pDAI value is $s \times y / x$.

Quantity	Initial state	After the illustrative purchase
pDAI reserves	1,000,000	316,227.766017
PLS reserves	1,000,000,000	3,162,277,660.168379
Marginal pDAI value at the fixed reference	\$0.10	\$1.00

For target τ , the required terminal reserves are:

$$y' = \sqrt{k \times \tau / s} \text{ and } x' = \sqrt{k \times s / \tau}.$$

The input is $y' - y$: **2,162,277,660.168379 PLS**, worth **\$216,227.766017** at the assumed reference. The buyer receives **683,772.233983 pDAI**. This demonstrates a finite purchase raising the specified pool's marginal price from ten cents to one dollar without purchasing its entire pDAI inventory. The constant-product model is standard AMM mathematics; this calculation is specific to the hypothetical inputs above. [AMM mechanism reference](#).

An executable quantity, rather than only a marginal quote

For a subsequent sale of q pDAI, average proceeds under this fee-free model are $s \times y' / (x' + q)$ dollars per pDAI. Setting that average to target τ gives:

$$y' = [T \times q + \sqrt{(T^2 \times q^2 + 4 \times s \times T \times k)}] / (2 \times s).$$

For $q = 1,000$ pDAI, an illustrative input of **2,167,281,613.012984 PLS**, valued at **\$216,728.161301**, produces a state where selling those 1,000 pDAI returns **10,000,000 PLS**, worth **\$1,000** at the fixed reference. The sale itself moves the pool price; this is not a promise that successive holders receive the same execution value. Immediately after that sale, the modeled marginal reference price is about **\$0.996842718387**. The construction achieves the stated finite execution; it does not defend the market against subsequent sales.

Fees, competing markets, arbitrage, changes in the PLS reference and new selling all change these figures. The example supplies a constructive market-level possibility result. Its connection to the bridge thesis is conditional: one or more financing paths must deliver enough **actual net purchasing assets**, not merely a token face value equal to the displayed budget. Direct pDAI purchase routes need not convert through PLS; their own executable curves replace this example's curve.

The explicit mathematical batch is `Research/experiments/parb_pls_price_existence_20260929.json`; calculated values are retained under ignored artifacts/`parb-pls-price-existence-20260929/`. Both price equations were checked using high-precision decimal arithmetic.

Trading fees and thirty successive local exits

The new [local PLS support study \(Appendix 43\)](#) adds input fees retained in a constant-product pool and repeated seller flows. The illustrative starting reserves and PLS reference are unchanged. With a 0.3% input fee in both directions, the initial purchase required to fund a 1,000-pDAI exit at the target is approximately **2,175,295,195.262100 PLS**. The fee is an example parameter, not an asserted PulseChain pool fee.

Give the operator an additional **400 million PLS** of support inventory and assume thirty successive sales of **1,000 pDAI**. Before each sale, the operator restores the finite-sale quote when its available inventory covers the required purchase. Under the fixed reference and no other flows, all thirty holders receive **10 million PLS each**. The intervening 29 purchases consume about **290,871,239.738011 PLS**, leaving **109,128,760.261989 PLS** of the additional support inventory. There is no fresh restoration after the final sale.

Illustrative scenario, after the separately funded initial purchase	Target exits out of 30	First below-target exit
400 million support PLS; 1,000 pDAI per sale; fixed reference	30	None
No further support PLS	1	2
100 million support PLS	10	11

Illustrative scenario, after the separately funded initial purchase	Target exits out of 30	First below-target exit
400 million support PLS, but PLS reference halves at sale 16	15	16
400 million support PLS, but each sale is 2,000 pDAI	20	21

This provides a constructive **bounded local-support result**. It does not require Ethereum redemption or an external-dollar payout. It does require the stipulated initial purchase and support inventory; the financing paths must provide those assets after their own immediate obligations. The model includes trading fees, conserves pDAI and PLS at every step, and never spends a negative treasury balance. Gas is separately funded; other trades, LP changes, loans and their maturities, and competition for transaction ordering are excluded. Consequently, it establishes the specified sequence, not a continuous peg under arbitrary flows. The study records the full equations, manifest and reproduction command; the previous fee-free batch is preserved unchanged.

7. From reaching a price to sustaining it

For a specified market state and seller-flow scenario, let c be the actual input assets required to reach the target and b the purchasing assets available from all selected financing paths, after immediate costs and reserved settlement funding. Express b and c in the same spendable input asset after required conversions, including conversion fees and price impact; do not add face values of different claims. A funded route with $b \geq c$ can support that modeled intervention if all intervening contract operations are executable. This is a conditional sufficiency statement, not a measured inequality for the audited network.

The sequence must also be funded at every intermediate step, in each required asset. A positive final balance does not finance an earlier payment. This is a cash-feasibility condition, distinct from the economic solvency of collateral. Cross-chain settlement delays cannot be treated as a single atomic transaction. Route budgets sharing collateral, lender cash, relayer inventory or base-pool liquidity cannot be added independently. Quotes must reflect the state left by preceding trades, not mutually incompatible uses of the same initial balances.

Do not subtract every long-term loan principal as if it were payable immediately: credit can finance a current purchase while its debt remains outstanding. But that liability must remain in the system's balance sheet. A claim of a repeatable, self-financing process additionally needs resources for eventual repayment, redemption, fees, losses and reserve replenishment.

Below the target, funded buying can support demand. Above it, selling retained pDAI can add supply and rebuild reserves. This feedback policy can be described without assuming a particular operator has adopted it. It does not guarantee a durable peg: selling pressure can exceed the available intervention budget, and the assets backing that budget can change value.

A bounded support claim needs a disclosed time horizon, execution quantity, permitted price band, seller and redemption flows, available purchasing inventory, debt maturities, and replenishment assumptions. An above-target sale also requires available pDAI inventory. Fees, realized income, new contributed capital or profitable trades may replenish reserves; merely specifying a buy/sell rule does not establish those inflows. No finite budget guarantees the target under arbitrary unbounded selling pressure.

Source collateral valued solely by the hoped-for pDAI price creates a circular funding argument. Such financing may execute under a protocol's declared values, but it does not independently establish recoverable support capital. Price support is also not automatically arbitrage: profit needs an executable favorable conversion or price difference after costs. A desired one-dollar price is not itself a redemption or resale opportunity.

Debt repayment can restore Aave withdrawal liquidity and allow borrowers to withdraw released collateral. Those assets may be sold again. A rigorous support scenario includes those flows instead of counting all repayment as permanently removed market supply. Competing markets can likewise supply cheaper pDAI to sell into a supported pool. The cost of changing one pool need not equal the cost of supporting an accessible network of markets.

Debt can also create a conditional discounted-claim benefit. If an eligible borrower can buy 100 units of the relevant aToken for 90 underlying units and use those claims to discharge 100 units of its own debt, its balance sheet improves by the difference before costs. This is a benefit from the claim's discount and an existing repayment right; it adds no new reserve cash and does not transfer another borrower's collateral to an unrelated payer. The [financing study \(Appendix 42\)](#) distinguishes this from underlying-funded repayment and Portal backing. No such profit is attributed to the examined wallets without the corresponding evidence.

Appreciation can reduce the PLS requirement and enable reserve rebuilding

The proposed rise in PLS value is a useful **conditional scenario**. If the independent PLS reference rises from \$0.0001 to \$0.00015, the PLS needed to deliver \$1 falls from 10,000 to approximately 6,666.67 per pDAI; at \$0.0002, it falls to 5,000. The same native-unit relationship applies to the exact native wrappers identified in section 1. No Ethereum redemption is involved.

A fixed pDAI/PLS ratio would then give pDAI a higher dollar-reference price. Maintaining the target instead may permit an operator to sell retained pDAI for PLS, reducing that ratio and rebuilding its PLS inventory. This requires unencumbered pDAI, a funded pool and accounting for the sale's impact and fees. Appreciation changes reference values; it does not itself create PLS units.

A separate numerical checkpoint starts after the fifteenth successful exit in the 0.3%-fee support scenario. The operator holds approximately 698,378.135367 pDAI and 259,579,401.553066 PLS. Under alternative assumed reference increases:

Reference change	Operator pDAI sold	PLS recovered	Resulting operator PLS inventory
+50%	70,153.484465	572,788,016.391764	832,367,417.944829
+100%	130,138.650678	920,191,644.708543	1,179,771,046.261608

Both sales fit the modeled pDAI inventory and leave the next 1,000-pDAI holder sale at exactly \$1 of reference-valued PLS per pDAI. These are alternative branches from the same checkpoint; their proceeds must not be added together. The PLS comes from the pool in exchange for the operator's pDAI, rather than appearing through a valuation change. All balances reconcile with retained fees.

This supplies a concrete sell-above-target counterpart to the funded buying example. It is one inventory-constrained rebalancing checkpoint, not proof of continuous stability or a causal prediction that a pDAI peg increases PLS value. The initial PLS accumulation, competing flows and market-wide reference-price formation remain outside this model. Equations and reproducible batches are in the [support study \(Appendix 43\)](#).

Source collateral, its bridge representation and a stablecoin issued against that representation form a chain of claims. They are not three independent allocations of capital. The same pDAI cannot simultaneously settle Portal, repay unrelated borrowing, remain in another escrow and fund another purchase. A source receipt may continue earning yield, but future yield is not current liquid PLS unless someone provides financing against it.

8. What the audit contributes to this possibility argument

The historical record is a foundation for identifying mechanisms, not a requirement that the entire hypothetical system already have operated.

Established finding	Contribution to the theory	Scenario condition not supplied by that finding
ARB and GRT destination mint executions	Destination issuance despite absent or insufficient token escrow is possible for specific integrations	Other tokens' permissions and source backing
104 transfer-only withdrawals alongside those two mint cases	Existing escrow transport and specialized issuance coexist	A universal mint mechanism for all stables
Existing Aave aTokens, compatible wrappers and recorded swaps	Exact local receipts can participate in markets	Enabled Portal capacity and adequate settlement funding
Existing eUSD issuer and matching Curve token identity	A local issuer can supply its own existing pool coin	Eligible bridged collateral installed in its active basket
Across control changes and historical funded-delivery architecture	A liquidity provider can connect source deposits with local actions	Actual destination inventory and functioning current integrations
Historical reverse-mode Omnibridge and mintable Polygon code, with copied L1 presence	Additional custody/issuance architecture families existed	Complete operational counterparts and token mappings
Recorded mUSD-to-DAI trades and examined-wallet repayments	Local exchanges and debt settlement actually occur	Their financing by the proposed source-chain system
Recorded pDAI-to-WPLS trade followed by aggregate native PLS payout	An actual local holder-exit component exists	A \$1 value, current executable quote or connection to the proposed source financing

At the fixed audit endpoint, the three sampled stablecoin Portal caps were zero; HubPool's sampled DAI/USDC/USDT cash balances were zero; eUSD's prime basket and registry matched the inherited snapshots. Sampled Reserve price feeds retained May 2023 updates and collateral plugins returned uninformative price intervals; cached SOUND status alone did not establish successful current issuance. These facts identify conditions a different operational scenario would have to satisfy. They do not mathematically exclude the conditional architectures.

Candidate path	Material limitation at the audited state or in the reviewed evidence
Aave Portal → exact V3 wrapper	Zero stablecoin Portal caps; independent DAI/USDT supply-cap barriers; roles, amount, swap liquidity and settlement must all fit
Sampled waDAI linear pool	Approximately 0.055466 DAI output headroom at the sampled state; a state-dependent constraint, not a lifetime ceiling

Candidate path	Material limitation at the audited state or in the reviewed evidence
Across funded delivery	Zero sampled HubPool stablecoin cash does not measure every relayer's capital; actual funded delivery and replenishment remain unestablished
Reserve eUSD collateral route	No sampled pARB receipt in the active basket; stale feeds and uninformative price intervals; registration alone is insufficient
Named stables and receipts through an ordinary gateway	Their default escrow route does not inherit ARB/GRT mint authority; an empty escrow cannot supply its transfer output
Reverse or specialized mint routes	Specific historical interfaces exist; the exact proposed source receipt, destination identity and downstream funding remain route-specific conditions

The recorded ARB withdrawal's purported source block/timestamp values are not plausible chronology. Its destination mint remains an observed action; those fields do not establish a real source deposit. More generally, an accepted message from a path under the assumed common control is not independent proof of that path's source assertion. The GRT trace similarly proves destination issuance, not an independently audited source investment.

The full audit covers 106 selected withdrawals across ten identified historical gateways, all 78 associated transaction traces and 212 historical supply reads. Its limitations, including one pre-fork upgrade-history query gap, remain in the [evidence companion \(Appendix 32\)](#). No end-to-end source-deposit-to-pDAI-purchase execution or participant intent is asserted here.

The added local-exit audit verifies a branch of [transaction 0x520d978f...](#) on 23 January 2025: **782.592868262568217956 pDAI** was exchanged for **264,588.142395152594479041 WPLS** through pool `0x947b4633e32e0c7f2c76753b43f008480715416d`. The aggregated transaction subsequently unwrapped combined WPLS and paid native PLS. Other branches contributed to the aggregate payout, which cannot be attributed wholly to pDAI.

At the principal endpoint that pool reported approximately **18.087 million pDAI** and **3.444 billion WPLS** in reserves. Its published swap logic applies a 0.29% input fee, distinct from the illustrative model's 0.3%. These are fixed historical observations, not live quotes or an assessed dollar value. They establish concrete local-exit infrastructure; the small waDAI linear-pool limit does not measure all pDAI-to-PLS liquidity. The [local evidence study \(Appendix 41\)](#) retains receipt, trace, reserve, source and wrapper-conversion references.

9. Could governance remove the roadblocks?

Yes, some identified roadblocks are conceivably remediable under effective governance. A zero cap or absent integration at the audited snapshot is not proof of permanent impossibility. Under the scenario's assumption that the operators hold the relevant authority, they may be able to change parameters, authorize compatible integrations or commit additional resources. The useful test is the scope of that authority and the resulting contract behavior, rather than an inference from how meticulous the operators appear to be.

Roadblock	Conceivable authorized mitigation	What remains to establish
Portal unbacked cap and reserve supply cap	Change each relevant setting through its authorized configuration process	Effective V3 authority, implementation-specific bounds, executable amount and all remaining reserve checks
Missing bridge mapping or token integration	Authorize a compatible mapping/integration where the contracts permit it	Exact destination mint authority, source accounting and receiving-token compatibility; mapping alone is insufficient
Ineligible issuer collateral or stale valuation inputs	Adjust eligible collateral, active basket or supported valuation integration where governance permits	Actual basket transfers, conversion rules, valuation behavior, and a funded market for the issued token
Existing pool rejects the delivered representation	Use a compatible existing route or establish a separately funded compatible market	An immutable pool's coin addresses do not change merely because a neighboring protocol changes; new pools need actual output inventory
Insufficient destination inventory	Supply treasury/LP capital, collect repayments, arrange funded credit, or obtain offsetting flows	Actual quantities, counterparties, timing, costs and any new claims against that funding
Settlement or redemption shortfall	Commit additional assets or a funded financing arrangement	Each required asset transfer and outstanding obligation still reconciles; one payment cannot satisfy two independent claims
An implementation lacks necessary behavior	A permitted upgrade or new compatible component could change the hypothetical system	This is a changed implementation, not proof the inherited deployment already had that behavior; immutable or separately controlled contracts remain distinct

If authority extends to the **exact token issuer**, authorized issuance may also increase the spendable units of that token. It would therefore be too strong to say governance can never add token balances. However, changing a lender's limits or oracle marks alone does not create an unrelated output asset. Issuing more units does not guarantee the executable PLS value required for the target. Application governance also does not imply control of native PLS issuance or consensus rules; that would be a different system assumption.

Governance feasibility includes the required voting power, quorum, proposal and execution permissions, timelocks, implementation bounds and successful execution. A declared common owner does not substitute for all those conditions. The existing control evidence is narrower: Aave V2 changes do not prove V3 ACL control; a stkAAVE upgrade does not establish GHO; Across changes do not establish control of all UMA dependencies; and the Reserve role studies do not establish the named group's effective voting control. These are evidence limits, not contradictions of the explicit common-control assumption in this theoretical exercise.

Consequently, the present-state barriers should be read as **requirements to resolve**, not permanent disproofs. A successfully reconfigured and funded system could support a finite purchase. Whether it can repeat the operation solvently and defend the target remains a separate balance-sheet and market question. Governance is a potentially important enabling mechanism within the theory, not a substitute for that analysis.

10. What would strengthen or falsify a particular composition

The conditional picture becomes more informative when its interfaces and economic limits can fail specific tests. These are proof obligations, not instructions to change any deployed system.

Required specification	Failure that defeats that particular proposed route
Exact source asset, transferable receipt and realizable rights	The receipt cannot be transferred, represents a different asset, or has conflicting claims
Source custody/burn, message identity and return accounting	The same source allocation backs incompatible claims or the required return cannot release it
Exact destination token and effective issuer permission	A valid message reaches a receiver unable to issue the required token
Exact downstream pool coin, wrapper or active collateral basket	Shared names or registration do not satisfy the actual receiving interface
Funded output at the required amount	The accepted claim receives too little pDAI, PLS or intermediate buying asset
Compatible parameters, oracles, caps and timing	Any required contract predicate fails at the relevant execution state
Complete sequential cash and liability ledger	A step spends unavailable funds, shared inventory is counted twice, or required immediate settlement remains unpaid
Defined execution target and market coverage	The input budget does not reach the target for the stated quantity across the specified routes
Bounded seller/redemption and funding scenario	Required interventions or maturities exceed spendable reserves and replenishment within that scenario

For example, a trillion correctly locked source units with no funded destination bid defeats control-and-custody sufficiency. A Portal sale at par followed by full principal backing leaves no free pDAI profit before costs. A finite local inventory consumed without replenishment defeats unlimited Across delivery. These counterexamples refute the stronger universal claims; they do not refute every conditional funded purchase.

A theoretical witness can stipulate missing facts, as section 3 does. A claim about deployed executability instead needs the exact predicates at compatible states; a historical claim needs linked events and transfers. A sustained-price claim additionally needs the bounded dynamic scenario. Those proof levels cannot substitute for one another.

Conclusion

The narrowed conditional mechanism is coherent. Source receipts can preserve invested assets while an authorized destination claim is monetized; specialized token issuance, temporary protocol credit and funded bridge delivery provide additional paths. Their outputs can converge on purchases of pDAI. A sufficiently funded market intervention can reach a stated PLS-valued target in a specified market, as the finite-input example demonstrates.

The extended model supports a bounded sequence of fee-paying local exits and an inventory-constrained sell-above-target example under assumed PLS appreciation. Native PLS, canonical WPLS and the exact inherited WETH contract can be evaluated through their native-unit conversion paths. None of these local outcomes requires the pDAI holder to bridge to Ethereum. Appreciation is an explicit scenario; its occurrence as a consequence of the peg is not established by the model.

The stipulated financing ledger shows where purchasing assets come from and where each claim ends up. It does not show that nominal copied collateral or validator control generates those assets. Funded mechanical acceptance can permit a finite purchase even when collateral is economically weak; that fact does not establish solvent repetition or a durable peg.

The report therefore supports conditional architectural possibility and a specified market-level existence example. It does not establish a complete route executable in the audited deployed configuration, that the named wallets adopted this mechanism as their plan, or that pDAI will reach or maintain the target. Those stronger conclusions require the additional proof obligations above.

Evidence and source index

- [Complete read-only PulseChain evidence audit \(Appendix 32\).](#)
 - [Historical Arbitrum and legacy Optimism bridge models \(Appendix 13\).](#)
 - [Across, Omnibridge and Polygon historical models \(Appendix 7\).](#)
 - [GRT mint, allowance and escrow reconciliation \(Appendix 12\).](#)
 - [Aave Portal and local backing accounting \(Appendix 17\).](#)
 - [Exact Aave-Balancer wrapper inventory \(Appendix 24\).](#)
 - [Pool capacity and actual swaps \(Appendix 16\).](#)
 - [Reserve collateral eligibility and issuance \(Appendix 18\).](#)
 - [Across HubPool and configured-spoke accounting \(Appendix 14\).](#)
 - [Curve identities and observed exchanges \(Appendix 8\).](#)
 - [Local financing and complete illustrative settlement ledgers \(Appendix 42\).](#)
 - [Fee-inclusive local support and appreciation models \(Appendix 43\).](#)
 - [Historical local exits, native wrappers and selected evidence \(Appendix 41\).](#)
-

Appendix 34: Deferred Portal settlement and scheduled pDAI accumulation

Contents of this appendix

Evidence and assumptions	221
Complete examined 86ff V3 debt	221
Atomic repayment and continuing Portal exposure	222
Caps and authority	223
Five independent clocks	223
Earning over years	224
Consistent buying every ten seconds	225
Across delivery and Curve rebalancing	225
Reproduction and limits	226

Source record: docs/studies/analysis/pulsechain_deferred_campaign_20260930.md.

Deferred Portal settlement and scheduled pDAI accumulation

30 September 2026. This study examines a coordinated PulseChain campaign that repays matching borrower debts, buys in small scheduled amounts, operates connected Curve liquidity and funds eventual backing from collected income. It establishes the complete examined V3 debt census and several code and configuration constraints. It does not establish an operating end-to-end campaign, the source chain's balances, common ownership or a durable dollar peg.

Evidence and assumptions

All fresh chain reads use chain **369**, block **27,675,130**, hash

0xdd12f23ad571882b63fd333c5d3909a45a05fe6f0a3c2b273cb7e1e6b0917954, timestamp **30 September 2026**

05:20:55 UTC. Header rechecks agree after each batch. Read-only `eth_call` queries collect balances, configurations, routes and quotes; code and implementation storage are read without overrides. No transaction was signed, broadcast, or simulated as a state-changing call. The strategy tests are view calls with hypothetical accounting arguments.

The scenario assumes coordinated control, authorized destination bridge issuance, sufficient supply and unbacked caps, usable account execution, market acceptance and sufficient route liquidity. Trading budgets are 44bn DAI, 60bn USDC and 50bn USDT units, separate from the issuance allocated to borrower repayment. These are not dollar valuations. Multi-year tolerance, net earnings, trade amounts and an independent dollar reference are modeling assumptions. Across metacoins delivery is not established by these observations.

Complete examined 86ff V3 debt

Account: 0xe6df4ca976f3fec8da89efec332dc2245f0a86ff. The current `getReservesList` returns the same 17 assets as the earlier census. Each current `getReserveData` identifies its debt and receipt contracts; all balance queries use those contracts at the same block. USDC and USDT use six decimals, WBTC eight and the

other listed assets eighteen. All 17 stable-debt balances are zero. The individually decoded debts reconcile exactly to the account's aggregate debt under its sampled oracle prices. Twelve variable-debt balances are nonzero:

Underlying asset	Variable debt in token units, rounded except cbETH
pDAI	136,922,803.834140
pUSDC	261,016,418.599713
pUSDT	187,090,022.959893
pWBTC	43,836,589.082845
LINK	3,841,442,884.551383
CRV	23,014,632,202.924069
MKR	2,166,349.981530
SNX	954,043.355941
BAL	27,993,956.678406
UNI	10,167,590.111045
LDO	23,187,809.019895
cbETH	0.000000000000001435

WETH, wstETH, AAVE, rETH and LUSD have zero debt in this account. WETH is its sole nonzero aToken holding, approximately **105,220.502652 aWETH**. Sampled WETH underlying cash exceeds that holding. Exact raw integers, all omitted zero positions, reserve totals and actual cash are retained in `model.json`. The V2 account getter still reverts; no V2 zero-debt finding is inferred.

The debt balances include accrued interest and are not original borrowed cash, recoverable collateral or evidence of equivalent underlying issuance. Debt in different assets cannot be summed without a specified valuation. Inherited oracle valuations do not establish executable dollars.

Atomic repayment and continuing Portal exposure

The current pool proxy still identifies implementation `0xf1cd4193bbc1ad4a23e833170f49d60f3d35a621`. Fresh explorer-verified source lists its linked BridgeLogic at `0x57572c9e795f4b6a748efbeab7e0a1b9996a0a24`. That library's verified source matches the pool's included BridgeLogic source. Current runtime reads for both are retained. Its mint operation invokes ordinary supply validation, increments `aggregate.reserve.unbacked`, checks the cap, updates rates and mints the recognized destination aToken. [Verified pool source, verified linked library](#).

Assume an authorized bridge issues each matching V3 receipt to 86ff and that 86ff can legitimately make its own repayment calls. `repayWithATokens` reduces the matching borrower debt and burns its receipts without supplying underlying cash. It does not reduce `reserve.unbacked`. This method is for the calling account, whereas ordinary underlying repayment can name a beneficiary. The existence of a token-transfer facility does not prove arbitrary Aave execution.

The twelve repayments can be considered as one destination batch, conditional on account support, validation, runtime interest, scaled-balance rounding and gas. The cbETH remainder must be reconciled even though economically tiny; an aToken mint of too few raw units can fail scaled-unit rounding. A successful all-debt repayment removes debt-based withdrawal constraints but does not give the payer ownership of collateral. No such batch was executed or simulated here.

For each asset X, the proposed debt restructuring is:

```
authorized destination mint of aX
-> caller burns matching aX against its X debt
-> borrower X debt falls; reserve cash is unchanged
-> equal Portal X principal remains to be backed
```

Separate receipts sold through wrappers cannot also be burned in repayment. Allocations and retained claims must be consolidated across controlled accounts. The previous [repayment and rights audit \(Appendix 21\)](#) and [Portal ledger \(Appendix 17\)](#) explain these rights.

Caps and authority

All 17 current unbacked caps are zero, so positive Portal issuance fails under the observed configuration. Ordinary supply caps apply to issuance as well; raising the unbacked cap alone is insufficient. Every nonzero debt plus its separate campaign allocation fits the standard 36-bit unbacked-cap encoding maximum of 68,719,476,735 whole underlying units in its own reserve. For DAI, USDC and USDT the combined new amounts are approximately 44.137bn, 60.261bn and 50.187bn. Other bridges and prior unbacked amounts share that reserve's headroom. Encoding feasibility is not actual mint authorization or liquidity.

The fresh provider returns ACL Manager `0xc2aacf6553d20d1e9d78e365aaba8032af9c85b0`. Its reported ACL admin and provider owner are short executor `0xee56e2b3d491590b5b31738cc34d5232f378a8d5`. `isPoolAdmin(shortExecutor)` returns true. Executor `getAdmin` remains governance `0xec568fffb86c094cf06b22134b23074dfe2252c`; `getPendingAdmin` remains `0xa30190b96faee0080144aa0b7645081fcbf49e6f`. Delay and minimum delay both return 86,400 seconds. The standard setter cannot set the delay below its minimum; once the operating permissions exist, individual market operations do not each require this administrative delay.

Five independent clocks

Clock	Finding and significance
Administrative activation	Current executor delay 86,400 seconds; acceptance and actual authority are separate.
pARB assertion confirmation	<code>confirmPeriodBlocks</code> returns 45,818; elapsed time also depends on chain progress and challenges.
Across bundle liveness	Current HubPool liveness returns 7,200 seconds. This is not total reimbursement latency or a Portal maturity.
Trading campaign	Proposed eligible ten-second observation slots; pauses, block inclusion and execution are additional constraints.
Eventual Aave backing	Inspected BridgeLogic contains no per-mint maturity or automatic collection; exact integration terms remain separate.

Unbacked principal is increased by mints and reduced by explicit backing; this entry has no independent interest-index update. Receipt balances can still accrue liquidity-index interest. The backing library accepts a fee argument and allocates it; it does not encode a universal time-based penalty or enforce payment at fourteen days. An integrating bridge can impose fees and deadlines. Original Portal documentation describes temporary advances, not an entitlement to permanent funding. [Portal design](#).

An unchanged exposure may therefore persist in state for years. That is an implementation possibility, not a demonstrated operating arrangement. It continues consuming capacity; suppliers still hold claims, withdrawals still require cash, and governance or market acceptance may change. A bridge message or an ordinary Aave deposit does not discharge this aggregate obligation.

Earning over years

Removing borrower debt stops its future interest contribution. 86ff accounts for 100% of sampled LDO debt, about 99.9946% of UNI debt, 99.7430% of SNX debt, 91.1897% of LINK debt and 45.2012% of CRV debt. It is not the sole borrower in every reserve. Remaining collectible external debts can still support income; the repayment itself does not manufacture it.

The configured DAI strategy lacks explorer-verified source. Instead, three view calls test its actual current response with controlled input arguments: 100 underlying units of hypothetical cash, a 10% reserve factor, zero stable debt, then varying variable debt and unbacked amount. They return:

Variable debt	Unbacked	Liquidity rate per year	Variable borrow rate per year
100	0	1.125%	2.5%
100	1,000	0.1875%	2.5%
0	1,000	0%	0%

No reserves or balances were modified. These results establish behavior for the tested arguments, not a full strategy source audit, a guarantee of income collection, or rates for every other asset. They agree with the inherited architecture in which unbacked supply dilutes the liquidity utilization ratio.

[Architecture reference](#).

LP fees, external borrowing payments and realizable rewards can be additional funding streams if the system owns the corresponding earning claim. Internal fees cancel on consolidation. Burned LP positions provide no withdrawable system entitlement; eligible owned LP positions may earn staking incentives. Reward sales require market depth and can affect reward prices. Lending interest booked against uncollectible borrowers is not spendable fee cash.

For a fixed obligation M and exclusively owned productive assets A , assuming constant net collectible annual yield y and reinvestment, assets grow as $A(t) = A(0) * (1 + y)^t$. Spendable assets, issuer claims and backing obligations must be tracked in the actual asset denominations. The normalized model assumes one asset and no losses, new issuance, additional outside claims or term changes.

With $M=100$ and $A(0)=80$, 5% yield reaches 100 total assets in 4.57 years. Consuming those assets leaves zero capital in this modeled portfolio. Requiring income alone to pay 100 while preserving the original 80 instead needs assets of 180 and takes 16.62 years. At 10%, the corresponding times are 2.34 and 8.51 years. Zero yield never fills a shortfall. These are sensitivity examples, not PulseChain yield forecasts. A price rise in an asset owed can enlarge the cost of any missing backing; the portfolio benefits from its net owned exposure.

Partial income-funded backing is also possible and restores some headroom earlier, but reduces the income left for reinvestment. That competing allocation is not simulated by the simple endpoint examples. Interim withdrawal demands, realized losses and the tradeoff with peg-support reserves remain decisive.

Consistent buying every ten seconds

One eligible slot deploys a fixed input amount; the acquired pDAI varies with the quote. At 8,640 slots per day, spending a 50bn-input-unit budget at 1,000, 10,000 or 100,000 units per slot takes 5,787.04, 578.70 or 57.87 active days. Half-duty eligibility doubles calendar time. The policy pauses at the target, adverse quotes, insufficient input inventory or required liquidity reserves. This is a schedule, not proof of executable depth or a dollar anchor.

Time slicing reduces the size of one intervention and permits intervening flows. It does not erase cumulative impact on unreplenished reserves. The offline example starts with one million units on each side of a constant-product pool and spends one million input units. With zero fees, one trade and 100 trades both acquire 500,000 output units. At a 0.29% fee, one trade acquires 499,273.947223 units and 100 trades acquire 498,997.538504. No sellers or arbitrage replenish that example. Favorable intervening flow or added liquidity, after costs, is what can improve realized campaign execution.

Outside sellers replenish the desired token while taking payment assets. Arbitrage can redirect demand between venues but may also trade against a predictable buyer. Additional keeper transactions require gas; scheduled observations need not all result in submitted transactions. The actual policy needs complete route quotes and a consolidated liquidity/settlement budget.

Across delivery and Curve rebalancing

Fresh getters identify 3pool's coins as pDAI/pUSDC/pUSDT, and FRAXBP's as FRAX/pUSDC. Metapool coin identities are:

Market	Contract	Direct pool coins
OUSD	0x87650d7bbfc3a9f10587d7778206671719d9910d	OUSD and 3CRV
mUSD	0x8474ddb98f5aa3179b3b3f5942d724afcdec9f6	mUSD and 3CRV
FRAX	0xd632f22692fac7611d2aa1c0d552930d43caed3b	FRAX and 3CRV
eUSD	0xaeda92e6a3b1028edc139a4ae56ec881f3064d4f	eUSD and FRAXBP

3CRV is 0x6c3f90f043a72fa612cbac8115ee7e52bde6e490 and FRAXBP LP is 0x3175df0976dfa876431c2e9ee6bc45b65d3473cc. eUSD therefore shares pUSDC and FRAX connections rather than a direct 3CRV base. OUSD/FRAX proxy getters return the meaningful ABI word with trailing zero padding; this is retained verbatim and decoded only for the declared single-word result. Fixed-block underlying quote calls also return amounts, without proving large-scale swaps, allowances, execution or source funding. The historical [receipt study \(Appendix 8\)](#) establishes actual destination trades separately.

Underlying exchanges can transform delivered OUSD or mUSD into exact base coins. Adding base-coin liquidity creates 3CRV claims; staking or reusing these claims does not duplicate base-pool principal. Buying pUSDC and pUSDT need not buy pDAI directly, but can alter relative prices and induce corrective

cycles. Their direction must be evaluated through the complete closing trades. Fresh source for 3pool and the OUSD/FRAX metapool implementation is retained; modern documentation is architectural context, not a replacement deployment. [Verified metapool source](#).

For OUSD, mUSD and eUSD, the current HubPool's exact historical `poolRebalanceRoute(uint256,address)` getter returns zero under all sampled keys 1, 42161 and 369. The configured key-1 SpokePool still points to `0x788e49c4e2b3414777d6671d7411951e81d35808`; its old `enabledDepositRoutes` interface reverts in all nine probes. Both HubPool and that SpokePool hold zero of each examined metacoin. These observations do not inventory relayers, other spokes or every possible route. An initial probe using a different route signature is retained but excluded from route findings; the corrected manifest uses the explorer-verified HubPool ABI.

The assumed Across leg consequently needs an established exact-token route with funded relayers, or a new integration that supplies compatible local issuance/conversion. Receiving a same-symbol wrapped token is insufficient. The destination recipient must separately swap or add liquidity; delivery does not automatically rebalance Curve. Relayers' reimbursement and outstanding LP claims remain obligations to reconcile even under coordinated control. [Copied deployment and upgrade audit \(Appendix 14\)](#), [historical Across financing model](#).

Reproduction and limits

The maintained manifests are the `pulsechain_deferred_campaign` JSON files under `Research/experiments`, with explicit metadata, debt, routes, bridge and rates reads. Each getter collection runs through `Research/scripts/probe_pulsechain_routes.py` with `--block 0x1a649fa` and a fresh output destination. Keep snapshot identities and historical results. The new batch directory is `artifacts/pulsechain-deferred-campaign-20260930/`. It retains raw RPC records, block rechecks, explorer source responses, `source-index.json` and the offline model.

Reproduce the offline findings with:

```
python Research/scripts/model_pulsechain_deferred_campaign.py \
--manifest Research/experiments/pulsechain_deferred_campaign_20260930.json \
--output artifacts/pulsechain-deferred-campaign-reproduction/model.json
```

The model records manifest, script and evidence hashes. Its representative behavior test checks cumulative fixed-pool impact, zero-income funding limits, preservation of productive principal and schedule arithmetic. Production code is untouched. The full research suite is run with `./scripts/test.sh --research`. Strategy source verification, 86ff execution authority, large executable wrapper capacity, complete Across metacoin routes, actual collected yields and robust dollar-price control remain unresolved. The mechanism is a coherent conditional financing proposal; a multi-year operating result has not been demonstrated.

Appendix 35: Equal-value stablecoin routes: executed issuance, redemption and funding

Contents of this appendix

Why the common value matters	228
mUSD: recent executed market-to-issuer arbitrage	228
mUSD also connects to lending-funded issuance	229
FEI: an existing fixed-parity conversion with substantial cash.....	230
Origin and Inverse: two distinct issuance models.....	230
DOLA → Balancer/Curve → pUSDC → existing pDAI	231
Across: substantial local inventory movement, followed by market sales	232
Most valuable next evidence	232
Evidence and reproduction	233

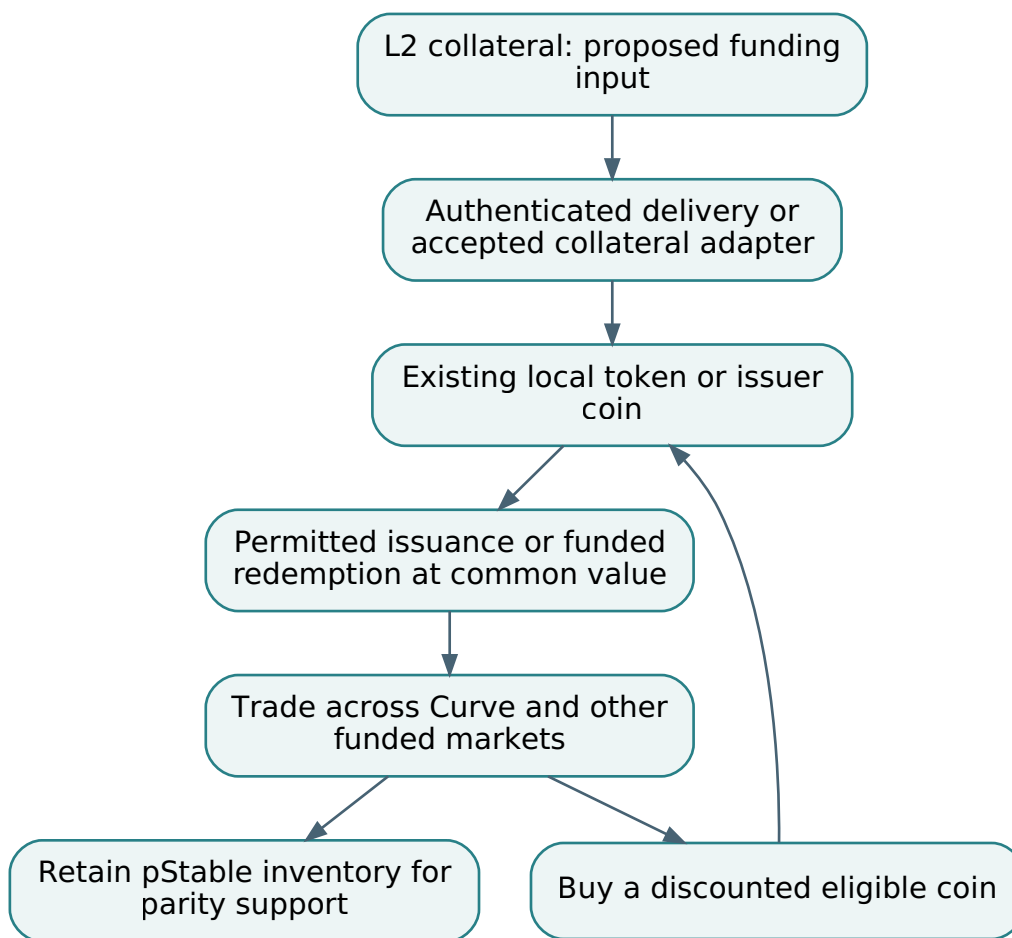
Source record: docs/studies/analysis/pulsechain_equal_value_routes_20261003.md.

Equal-value stablecoin routes: executed issuance, redemption and funding

The working thesis assigns **one common value U to one whole unit of every eligible pStable**: pDAI, pUSDC, pUSDT and the admitted issuer coins. This is the system's proposed accounting and parity rule. Observed market quotes can differ; those differences provide the proposed arbitrage opportunities. Receipt tokens, LP shares and foreign-currency assets first require their own exchange-rate normalization. U becomes a dollar promise only with a funded dollar-reference exit.

The October 3 search establishes additional functioning local connections. Most notably, a September 30 transaction bought discounted mUSD through Curve, redeemed part of it for all four basket stables and converted the proceeds into native PLS. FEI has a simpler, funded one-for-one pDAI conversion. An Inverse governance proposal actually minted a large amount of the existing DOLA coin. Across's changed administration moved substantial local stablecoin inventory into market trades. None of these transactions establishes fresh private-L2 collateral delivery.

Why the common value matters



A protocol accepting two basket coins near one-for-one can connect markets that currently price those coins differently. For example, buy cheap pUSDC, obtain an admitted issuer coin, redeem for a scarce pDAI, and sell or retain that pDAI. Conversely, buy discounted issuer coins and redeem their reserves. Each operation must satisfy the actual basket, mint, redemption and liquidity rules. Shared accounting gives the comparison rule; the route supplies the executable quantities.

These routes can redistribute existing capital, realize market differences and finance inventory. Lending-funded operations also create liabilities. A common valuation does not make those liabilities disappear or replenish assets already paid out. A recurring L2 branch must show its accepted destination token and funding input, rather than only a private-chain label.

mUSD: recent executed market-to-issuer arbitrage

At block **27,677,491**, on **30 September 2026 at 12:05:35 UTC**, [this transaction](#) deployed a temporary trading contract and performed the following operations:

Operation	Receipt-confirmed quantity
Pay pUSDC into the mUSD/3CRV Curve route	332.580073 pUSDC
Receive existing mUSD	3,514.021960458029333023 mUSD
Burn mUSD across four redemptions	896.975403751798273907 mUSD

Operation	Receipt-confirmed quantity
Receive sUSD	145.369503792975478359 sUSD
Receive pUSDC	143.443901 pUSDC
Receive pDAI	155.079448251736530091 pDAI
Receive pUSDT	451.543364 pUSDT

The remaining mUSD was sold through other local pools; the receipt accounts for the full acquired mUSD quantity. Initial native funding was **26,704.810084659172875544 PLS**. The contract returned **228,173.113310542683909513 PLS** to the sender. Subtracting initial funding and **1,415.103716302431089681 PLS** of transaction gas leaves **200,053.199509581079944288 PLS of net native proceeds**. Twelve sampled token balances at the trading address were zero in the preceding block. This establishes this transaction's local cash flows; it does not establish recurring capacity, external-dollar profit or the trader's connection to the hypothesized wallet group.

The exact issuer is `0xe2f2a5c287993345a840db3b0845fbc70f5935a5`; its Curve metapool is `0x8474ddb98f5aa3179b3b3f5942d724afcddec9f6`. The receipt and historical trace show the four underlying payments coming from mStable's existing Aave and Compound integration contracts. **The September redemptions used cached underlying; they did not call the lending pools.** [mStable's contract documentation](#) describes the upstream basket and integration interfaces; this finding comes from the saved PulseChain transaction.

At the October 3 resource pin, the integration contracts' directly held cash is only about **2.951588 sUSD, 2.912429 pUSDC, 3.148679 pDAI and 9.167871 pUSDT**. The basket records larger investment claims, but sampled Aave reserve-data and aToken balance getters revert. These observations do not prove that every withdrawal is impossible. They do prevent counting the nominal basket as currently withdrawable cash without further evidence. Current quotes still show a large market-versus-redemption difference; quotes do not test underlying withdrawals.

The complete selected postfork mStable event search returned **72 receipt-verified events: 29 Minted, four MintedMulti and 39 Redeemed**. The four September 30 redemptions are the latest in that search.

mUSD also connects to lending-funded issuance

[An October 24, 2024 transaction](#), sent by `0x48fc8b22ae5319786a8e25ef49b1b49942000000`, used a Balancer flash-loan callback and transferred **1,000,000 pUSDC** into mStable's Compound integration. It minted **998,634.756111707391994433 mUSD**, borrowed **963,458.121028 pUSDC** from Compound's cUSDC market, redeemed portions of mUSD for underlying stables and sold the remainder through Curve. The funding contract received its million-pUSDC principal back.

This proves a real **Balancer funding → mStable collateral deposit → Compound borrowing → issuer redemption/Curve sale** connection. The continuing Compound debt is part of the financing ledger. The stablecoin outputs alone are not net income. The same sender also executed older Across root bundles; that address-level overlap does not establish the sender's identity or coordinated control of both protocols.

FEI: an existing fixed-parity conversion with substantial cash

The **SimpleFeiDaiPSM** at `0x7842186cdd11270c4af8c0a99a5e0589c7f249ce` accepts the existing pDAI address and issues existing FEI at `0x956f47f50a910163d8bf957cf5846d573e7f87ca`. Its verified source transfers one pDAI in for one FEI, or receives one FEI and pays one pDAI out. This module's conversion does not consult an oracle. Redeemed FEI is held for a later burn rather than burned immediately.

One saved mint deposited **2,115.690961647546400768 pDAI** and issued exactly that many FEI. A May 24, 2026 redemption returned **162,213.391725678525310538 pDAI** for the same quantity of FEI. A June 15 redemption similarly paid **5,365.6583048387362816 pDAI**. The full selected postfork module search returned **103 Mint and 214 Redeem events**, all receipt-verified without query gaps.

The May 24 redeemer is the user-identified **1967 wallet**, `0xfdbec32bcf68bb0ced033200373384a619c71967`. Three minutes earlier, it exchanged **179,876.007186616736577400 TRIBE** for exactly the subsequently redeemed FEI quantity through the existing V2 pair `0x9928e4046d7c6513326ccea028cd3e7a91c7590a`. TRIBE's exact address is `0xc7283b66eb1eb5fb86327f08e1b5816b0720212b`; current getters confirm the identity and 18 decimals. This is a demonstrated **TRIBE → funded FEI market → fixed-parity pDAI redemption** route used by a named research wallet. It does not establish the wallet's original TRIBE funding or profit after valuing that input.

At the resource pin, the PSM contains **39,092,598.332549153109816030 pDAI**, remains an authorized FEI minter, and its mint/redeem flags and FEI's pause flag are false. This is a concrete funded local redemption branch. It has finite pDAI inventory; public mints add that inventory, while redemptions consume it. Those pDAI tokens also back other FEI claims and cannot all be assigned exclusively to the proposed campaign.

The existing FEI/3CRV metapool is `0x06cb22615ba53e60d67bf6c341a0fd5e718e1655`. Its sampled 100-FEI quotes return about **100.201370 pDAI, 223.956170 pUSDC or 229.315149 pUSDT**. These are separate output quantities at a fixed snapshot, not dollar prices or a proven combined arbitrage. Compare the acquisition cost and all legs at the same trade size. The existing FEI identity already fits the pool; ordinary issuance and trading do not require a Curve coin-list change.

The shortest L2-linked candidate here is **authenticated delivery of existing pDAI → FEI PSM → existing FEI/3CRV pool**. A differently addressed bridge representation needs an accepted conversion first. Across's checked chain-88888 token route does not currently establish that delivery.

Origin and Inverse: two distinct issuance models

A saved Origin mint transferred **900 pUSDT** from the caller to the Origin vault at `0xe75d77b1865ae93c7eaa3040b038d7aa7bc02f70` and minted **900 existing OUSD** at `0x2a8e1e676ec238d8a992307b495b45b3feaa5e86`. Its historical trace includes the vault's oracle consultation. This verifies ordinary accepted-collateral issuance. It does not show acceptance of an arbitrary new L2 receipt. Nine OUSD zero-address issuance events were receipt-verified in the broad search.

Inverse proposal 109 executed through GovernorMills `0xbecb6bb0aa4ab551966a7e4b97cec74bb359bf6` and timelock `0x926df14a23be491164dcf93f4c468a50ef659d5b`. It minted **10 quadrillion DOLA** at the existing address `0x865377367054516e17014ccded1e7d814edc9ce4` to `0x2eec75d3b370a95b7198c4daadb198c9509df785`, alongside INV/DBR actions. Its last action transferred timelock administration to `0x1b65b3ce1e463c6020e684cd8968e4cd330ab053`, which then accepted administration. Current getters confirm proposal state Executed, that administrator, and the timelock as DOLA operator. The operator may mint even when its separate minter flag is false.

This establishes an executed governance-authorized local supply route and an administrator transfer. It does not identify the ultimate controller of the custom administrator or link that controller to the other named wallets. DOLA quantities become usable funding when a compatible market pays counterassets.

DOLA → Balancer/Curve → pUSDC → existing pDAI

The large recipient transferred 120 million DOLA to one contract and withdrew it again, then sent 120 million DOLA to `0xba020c1a5fa7dda6ed1a02f41fbde5325d8fcf73`. [A subsequent transaction on October 31, 2023](#), called by that same recipient, performed this complete funding chain:

Leg	Actual flow
150 Balancer sales, each 80,000 DOLA	12,000,000 DOLA → 1,024,995.605256 pUSDC
DOLA/FRAXBP Curve underlying swap	100,000,000 DOLA → 16,075,670.154045 pUSDC
Maker USDC PSM collateral deposit	17,100,665.759301 pUSDC
Maker Daijoin issuance to the trading contract	17,100,665.759301 pDAI

The 150 smaller sales executed inside one transaction; they are not evidence of a ten-second schedule. The same transaction also used INV, including purchases of DOLA, so consolidated economics must include that input. The table reconciles the specified DOLA-derived pUSDC proceeds and their exact conversion into pDAI; it is not a profit statement.

```
Authorized DOLA mint
    ↓
Balancer DOLA/pUSDC + Curve DOLA/FRAXBP
    ↓
Existing pUSDC
    ↓ Maker PSM: accepted collateral at one-for-one accounting
Existing pDAI
```

The Curve pool is `0xe57180685e3348589e9521aa53af0bcd497e884d`; it reaches pUSDC through the FRAX/pUSDC base pool. The Maker PSM is `0x89b78cfa322f6c5de0abceecab66aee45393cc5a`, its USDC join is `0x0a59649758aa4d66e25f08dd01271e891fe52199`, and Daijoin is `0x9759a6ac90977b93b58547b4a71c78317f391a28`. The trace shows `sellGem`, the Vat collateral/debt accounting and Daijoin's mint of the existing pDAI address. Maker records the PSM debt against deposited pUSDC; this is distinct from the separate 86ff borrower and Aave Portal backing. [The primary PSM reference](#) explains the upstream conversion.

This large route executed historically, but its normal Maker PSM leg is currently disabled. The resource pin reports `Vat.live = 0`; verified Vat code requires `live = 1` for `frob`, which both PSM directions use. Zero current `tin/tout` fees and nominal debt-ceiling room do not bypass shutdown. Maker's configured End reports shutdown at **April 16, 2025, 06:27:15 UTC**. Its USDC settlement rate is about **0.000879277935257436 pUSDC per pDAI**, a separate prorated settlement rule rather than the proposed equal-unit rule. Actual settlement access and remaining collateral require their own accounting. Administrative issuance is also a separate permission path; it is not established by ordinary PSM access. None of the six sampled operator/trading addresses is a Vat ward.

The **1967 wallet also initialized that PSM-USDC-A settlement rate** through [this June 2026 End.flow transaction](#). This function is public; calling it does not prove governance ownership. The complete selected End history contains 90 receipt-verified shutdown/settlement events, including nine Cash events, but none of those Cash events is for PSM-USDC-A. Current End has **zero allocated internal USDC collateral**, while the USDC join holds about **113.692 million pUSDC** and the PSM position remains recorded. Physical tokens at the join are not automatically available to an End claimant. The next concrete settlement check is whether collateral processing can move the PSM position into End, followed by funded `pack/cash` and join withdrawal. This is a proposed accounting route; it was not executed or simulated in this research.

Across: substantial local inventory movement, followed by market sales

After a June 19, 2026 emergency deletion, `0xa3414934e9220ad7d81c64449842084d2641c5a2` proposed a new root bundle at **05:51:25 UTC**. Execution at **08:38:15 UTC** sent the following from HubPool

`0xc186fa914353c44b2e33ebe05f21846f1048beda` to local SpokePool `0x5c7bcd6e7de5423a257d81b442095a1a6ced35c5`:

Token	Hub-to-Spoke quantity
pDAI	1,347,161.201028105891530236
pUSDC	10,699,577.784957
pUSDT	173,268.617415

Under the thesis's common-value assumption these sum to **12,220,007.603400105891530236 U**.

This is not an independently established US-dollar amount. Copied WETH and WBTC were also transferred. Copied WETH at `0xc02a...6cc2` wraps native PLS on this fork.

The trace uses the verified **Ethereum_Adapter**, a same-chain pass-through. The event's inherited chain label is 1, while actual Spoke refund leaves specify chain 369. Subsequent successful refund claims paid the proposer **1,347,311.207536981046850455 pDAI, 10,704,238.399741 pUSDC and 177,473.841197 pUSDT**, plus other assets. Refund amounts exceed the new Hub payments; existing Spoke inventory contributes too. Count the transfer and refund as consecutive uses of the same assets, not two funding additions.

Within minutes the recipient sold the pDAI and substantial pUSDC/pUSDT through local market routes. Selected receipts show payment in the token `0x02dcd04e3f455d838cd1249292c58f3b79e3c3c`, whose symbol is WETH. This is a different address from copied WETH; these trades alone do not prove its external redemption.

The June 30 chain-88888 administrative relay also executed local calls. After a local Spoke implementation upgrade, [the July 2 chain-88888 relay](#) delegated to custom adapter `0x3233f8c0db384a2a1d6a307de41a804dfe411eb6`. Its 15-call trace moved local Hub assets directly to `0x83fa95ce3671d4f40d9ada7105b245ac911cbbd8`, including **1,817,316.581867 pUSDC and 289,925.729137970519362629 pDAI**. That wallet then sold assets through local markets. The custom adapter has no verified source in this batch; conclusions are limited to observed calls.

These are real funding-disposition routes. They do not show a fresh L2 deposit, collateral mint or authenticated remote delivery. At the October pin, HubPool is paused; thirteen checked chain-88888 token mappings, including the examined stablecoins, return zero. [Across's upstream overview](#) describes liquidity-funded relayer repayment; the copied fork's actual traces determine the findings above.

Most valuable next evidence

The new priority is **replenishment of a demonstrated conversion**. For mUSD, find an actual post-September cache refill or funded withdrawal from its Aave/Compound investments. That distinguishes an ongoing common-value conversion from a short opportunity using the remaining cache. For the L2 thesis, seek a fresh authenticated delivery of an admitted local token, then reconcile its FEI, mUSD or sUSD conversion and resulting pStable inventory. FEI provides a particularly simple destination conversion to test because its parity rule and liquid pDAI inventory are explicit.

The DOLA proceeds now identify a large executed route and its current Maker shutdown boundary. Trace current DOLA market capacity and the custom Inverse timelock administrator next; inspect Maker's settlement claims separately from its disabled PSM. For the 1967 wallet, trace the origin of the TRIBE that financed its demonstrated FEI redemption. These investigations can establish further supply-control

and funding connections without assuming that every protocol belongs to one group. A resource-backed rebase could replenish owned reserves, but its net realization and pool accounting still require the checks in the [funding and rebases study \(Appendix 36\)](#).

Evidence and reproduction

Current resource reads use chain **369**, block **27,698,092**, hash `0x95395f560a8ca8c49e9032534e7ac3285b183786ec8d731c5eadf81e0a876a52`, timestamp **2 October 2026, 22:48:55 UTC** (3 October in Melbourne). Historical transactions retain their own headers. The prior-inventory check uses block 27,677,490. Do not combine different snapshots into a contemporaneous consolidated balance sheet.

Raw RPC records, receipts, headers, historical traces and verified source downloads are retained under `artifacts/pulsechain-equal-value-routes-20261003/`. Explicit manifests are in `Research/experiments/pulsechain*_20261003.json`. Queries used getters, code/storage reads, logs, receipts and historical traces only. No transactions, setter simulations or state overrides were submitted.

The broad ten-token mint search initially had **21 query errors and four verification gaps**. The DOLA and MIM missing ranges were repaired with 50,000-block requests; each returned zero events without gaps. FRAX and canonical LUSD remain incomplete in that broad search. Failed queries never establish absence. The complete mStable, FEI-module and selected Across searches independently establish their stated scopes.

Reproduce the offline reconciliation using:

```
python Research/scripts/analyze_pulsechain_equal_value_routes.py \
  --input artifacts/pulsechain-equal-value-routes-20261003 \
  --output /tmp/pulsechain-equal-value-verified.json
```

The verifier passed **2,111 offline checks** of receipt/header/trace consistency, event inclusion, exact issued and redeemed amounts, native proceeds, retained debt, permissions, snapshots, the DOLA-to-Maker route, the 1967 wallet's funding and local Across asset flows. The current result is `verified-routes-final.json`; the SHA-256 inventory preserves the evidence used. These are consistency checks of RPC evidence, not a cryptographic light-client proof or a complete ownership audit.

Appendix 36: PulseChain funding conversions and rebasing pool reserves

Contents of this appendix

Finding and scope.....	234
Follow the funding into actual market assets	234
Which rebases actually reach the trading reserves?	236
Rebasing capacity is not current income	237
Highest-value next evidence	237
Reproduction	238

Source record: docs/studies/analysis/pulsechain_funding_and_rebases_20261002.md.

PulseChain funding conversions and rebasing pool reserves

Finding and scope

A funded local conversion route is demonstrated. The e121 wallet bought pDAI with native PLS, issued existing Synthetix sUSD, used inherited dHEDGE portfolios, withdrew sUSD and other synths, sold sUSD through the existing Curve pool for pUSDT, and converted the pUSDT into native PLS. The legs are observed transactions involving mixed inventory. They are not an isolated investment-profit calculation or a fresh private-L2 funding proof.

Rebasing can materially change reserves in some inherited Curve pools. The PWRD and STBT pools examined below use an implementation that prices from live ERC20 balances, less recorded admin fees. Other pools handle balances differently, and the token itself may exclude the pool from rebasing. Current yield availability must be established separately.

All investigation was read-only: public receipts, historical traces, logs, runtime code, storage and explicit pure/view getters. No transaction, setter simulation, state override or signature was used. The source pARB RPC remains unavailable.

Current pool and resource getters are pinned to PulseChain chain **369**, block **27,691,754**, hash `0x37e17b339cdd71627ed1ed3da1cef538790401685438094920b82ceac396a`, **2026-10-02 04:47:55 UTC**. The additional destination-event census ends at block **27,692,266**, hash `0xaa2d9472094a89fce7f49279040ed6f831e108af14a8bba346ae6ebdf183087f`. Historical receipts retain their own blocks and hashes.

Follow the funding into actual market assets

The actor is `0x98c89980d1eae247ada3636ae27648f78e66e121`. Exact pool-facing assets are pDAI `0x6b175474e89094c44da98b954eedeac495271d0f`, sUSD `0x57ab1ec28d129707052df4df418d58a2d46d5f51`, and pUSDT `0xdac17f958d2ee523a2206206994597c13d831ec7`.

Owned native PLS
| observed market purchases
▼



The September 7 [purchase transaction](#) supplied **10,101,010 PLS** to the PulseX router and received **48,483.983138861573080045 pDAI** from two market paths. The subsequent [wrapper mint](#) deposited **48,483.98 pDAI** and issued **48,362.77005 sUSD** after the 25-basis-point mint fee. This is an observed local entry requiring owned input capital.

Across five September wrapper transactions, the prior [Synthetix integration study](#) (Appendix 69) reconciles **136,554.715815649876218232 pDAI deposited** to **136,213.329026110751527686 sUSD issued**. At the current pin the wrapper cap is **20 million pDAI**, and its pDAI balance is **5,354,411.586932067826653098**; approximately **14.6456 million pDAI** below that cap. Other checks and successful execution are still required; this is one capacity measurement.

The repeated deposit contracts are inherited dHEDGE portfolios, including `0x907feb27f8cc5b003db7e62dfc2f9b01ce3fadd6` and `0xd076b9865feb49a43aa38c06b0432df6b6cbca9e`. Receipt burns and underlying transfers show that withdrawals return portfolio assets, including sETH or sBTC alongside sUSD. The project's [historical contract source](#) explains mixed-asset withdrawals; its current repository text is architectural context rather than proof of an exact deployed implementation match. The historical traces and receipt transfers are the evidence for the observed actions.

On September 8, both sUSD withdrawals were followed by Curve sales and then spending of exactly the pUSDT received:

Withdrawal and sale	sUSD sold	pUSDT received, then spent	Native PLS received from subsequent swaps
First Curve sale , followed by native exit	83,081.493093379804930146	190,820.895994	15,902,820.236239012814777489
Second Curve sale , followed by native exit	53,857.118893515352358917	118,769.952612	9,883,099.449811970863908131

Combined: **309,590.848606 pUSDT** became **25,785,919.686050983678685620 PLS** received by the wallet. The four sale/exit transactions cost **7,528.349405514803925374 PLS** in gas. Earlier purchases, wrapper fees, portfolio deposits, other withdrawn assets and remaining claims must also enter any profitability calculation. The original 10.101-million-PLS purchase does not account for all assets later withdrawn or sold.

The native exits route through several markets, including pUSDC, pDAI and Atropa transfers. That supports the integrated-market interpretation. An intermediate purchase followed by immediate resale is not retained peg-defence inventory. These particular transactions convert inventory back to PLS; they do not establish a permanent \$1 bid or the operator's motive.

The existing sUSD Curve pool is `0xa5407eae9ba41422680e2e00537571bcc53efbfd`. Its current view quotes return **2,318.024207 pUSDT for 1,000 sUSD, 23,083.972283 for 10,000, and 221,933.602676 for 100,000**. These are local relative prices, not external dollars. Average proceeds decline with size. Current quotes neither reproduce the September executions nor prove a complete profitable cycle.

Which rebases actually reach the trading reserves?

Two independent questions govern the result: does the token change this pool's balance, and does the pool use that balance when pricing and paying trades?

Token and exact identity	Pool result at the current pin	Implication
OUSD <code>0x2a8e...5e86</code>	Pool <code>0x8765...910d</code> has rebase state zero and nonzero fixed credits per token	Its OUSD balance is excluded from yield rebases. Eligible treasury-held OUSD could still receive yield separately.
UZD <code>0xb40b...9f1c</code>	Pool <code>0x6893...de3a</code> is a rigid address	This version's elastic accounting does not increase that pool's balance. The other UZD versions need their own tests.
PWRD <code>0xf0a9...b86b</code>	Both catalogued pools use live-balance accounting; <code>0xbcb9...4105</code> recognizes about 955,348.400863 PWRD, while <code>0x2de8...e1a5</code> is empty	A changing PWRD balance enters the active pool's reserves. Controller-reported assets and realized yield require separate verification.
STBT <code>0x5308...b44a</code>	All three catalogued pools use the same live-balance implementation; <code>0x892d...690b</code> recognizes about 3,740,776.432441 STBT	Interest distributions can change recognized reserves. Permissioned transfers, issuer authority and independent asset backing remain relevant.
Lybra V1 eUSD <code>0x97de...a371</code>	Both catalogued pools use a CryptoSwap implementation with stored balances and a later balance-synchronization path	A rebase should not be assumed to affect quotes immediately. The active <code>0x880f...a5e3</code> pool quotes only 0.002875 pUSDC for one eUSD at this pin.
ARTH.usd <code>0x973f...9a71</code>	Its checked pool holds about 71.613946 wrapper units, with about 60.331027 recorded as trading balance	Inspect its GMU conversion and pool accounting before counting growth as spendable inventory. It is distinct from ARTH.

The PWRD/STBT clones point to `0x55aa9bf126bcabf0bdc17fa9e39ec9239e1ce7a9`. Its PulseChain runtime matches the [Ethereum explorer's published deployed bytecode](#) exactly. Its verified `_balances()` reads each token's `balanceOf(pool)` minus recorded `admin_balances`; pricing and liquidity calculations use that result. The sampled getters reconcile those quantities. Curve's [rebasings metapool source](#) illustrates this architecture. A positive rebase can increase LP-owned trading reserves, alter relative quotes and permit arbitrage that redistributes counter-assets; a negative rebase can reduce those reserves. Neither direction supplies external purchasing power by itself.

Lybra's clones point to `0xa85461afc2deec01bda23b5cd267d51f765fba10`, also an exact runtime match to the [Ethereum published code](#). This is a two-asset **CryptoSwap**, rather than the rebasing StableSwap metapool implementation. Its invariant and exchange calculations use stored balances. `_claim_admin_fees()` includes a live-balance synchronization step. Establish which successful call reaches that step and how quotes change around an actual rebase before budgeting this route as an automatic buffer.

The OUSD pool's matched destination implementation at `0x5f890841f657d90e081babdb532a05996af79fe6` stores trading balances and reports live excess as admin balance. The current OUSD implementation is `0x33db8d52d65f75e4cdda1b02463760c9561a2aa1`. Its opt-in acts on **msg.sender**; a governance wallet calling it opts in that wallet, not the Curve pool. Current Origin documentation describing remote governance opt-in must not be imported into this older implementation. A token change or compatible new venue may be necessary for different treatment.

UZD's [verified token source](#) permits `removeRigidAddress(pool)` only to `DEFAULT_ADMIN_ROLE`. Removing the exclusion is a distinct permission from operating a rebalancer. Pool recognition still needs testing after any change. No such action was performed here.

Reserve eUSD and mUSD expand or contract through issuance/redemption. FRAX AMOs and DOLA Feds manage permitted supply. Those mechanisms should remain separate from holder-balance rebases, static receipt exchange-rate growth and ordinary swaps.

Rebasing capacity is not current income

PWRD's verified token calculates holder balances from a factor derived from controller-reported assets. STBT's verified implementation permits issuer-authorized positive or negative interest distributions, with interval and ratio limits. These are specific supply mechanisms, not proof of common group ownership or newly generated underlying income.

STBT's `lastDistributeTime` remains **2023-05-10 09:01:11 UTC**. This recorded state does not support assuming current continuous treasury-interest accrual on the fork.

Lybra's code makes the funding leg particularly clear: `excessIncomeDistribution` receives eUSD, directs the fee portion to its service-fee pool, burns excess payer shares and transfers surplus stETH to the payer. Reducing shares redistributes claims to remaining eUSD holders. The stETH input must exceed recorded deposits; it is not created by the distribution itself.

At the pin, Lybra holds **2,964.739596779408217723 stETH** against **2,964.739596779408217375** recorded deposits: only **348 wei** of excess. Its last-report timestamp is **2024-09-06 08:13:35 UTC**. This branch currently supplies no material surplus stETH to that distribution route. The fee ledger and accumulated fees also require accounting. This does not rule out future local staking or a different funded strategy; it identifies the resource that must change.

There are two useful rebase designs. An eligible treasury account can receive genuine net yield, sell the incremental inventory and use the proceeds to support cheap pDAI. Alternatively, a compatible pool can recognize balance growth directly, allowing prices and arbitrage to redistribute liquidity. The latter benefits the relevant LP claims; count only the system-owned share, and do not count the same yield again as a separate treasury asset. Positive rebase supply can also pressure the rebasing coin's price, and losses can shrink support capacity.

Highest-value next evidence

The added query of the configured Synthetix destination bridge covers blocks **27,683,121–27,692,266**, with zero query errors and no `FinalizeSynthTransfer` or `InitiateSynthTransfer` events. This extends the prior observation; it does not prove that every possible alternative bridge or issuance method is inactive.

The most valuable next piece for the **fresh L2 funding thesis** is one successful, independently reproducible PulseChain transaction that follows:

```
New accepted source commitment/message
→ configured messenger 0x25ace71c97b33cc4729cf772ae268934f7ab5fa1
→ correct cross-domain sender 0x136b1ec699c62b0606854056f02dc7bb80482d63
→ bridge 0x39ea01a0298c315d149a490e34b59dbf2ec7e48f.finalizeSynthTransfer
→ authorized Issuer mint into the existing sUSD identity
→ funded conversion and inventory reconciliation
```

This is the configured copied **Optimism/Synthetix** path. Control of copied Arbitrum administration or a separate optimistic arbitration module does not substitute for its messenger and counterpart authentication. A proposed pARB entry requires its own proved gateway or an authorized, compatible adapter/configuration change.

The source commitment must be fresh, rather than reuse an inherited pre-fork withdrawal. Preserve the accepted root/message proof, sender authentication, mint receipt, exact pool token, recipient, market proceeds and source liabilities. The private RPC need not become public to observe a destination settlement, but inaccessible source accounting can leave backing unresolved.

This priority follows three passes: the first found successful local issuance; the second established actual market conversion and native proceeds; the third matched rebasing pool code and checked current resource availability. More integration names would add less than proving the missing replenishment entry. For the multi-year earnings branch, the next proof is separately an actual resource-backed distribution that increases owned, realizable net assets and reconciles the pool's balances and quotes.

The strengthened thesis is therefore: **existing local issuance can feed funded market conversions; compatible rebasing assets can redistribute genuine yield into treasury or LP inventory; fresh authenticated issuance or realized net earnings must establish replenishment at the required scale.** These results establish useful local connections without establishing external-dollar parity or indefinite financing.

Reproduction

Bulk evidence is in ignored `artifacts/pulsechain-funding-and-rebases-20261002/`; earlier mint receipts remain in `artifacts/pulsechain-synthetix-integrations-20261001/`. Explicit manifests are under `Research/experiments/pulsechain_funding_*_20261002.json`, `pulsechain_rebase_resources_20261002.json` and `pulsechain_fresh_synth_delivery_20261002.json`. The event retry covers all four timed-out pUSDT incoming ranges and returned no additional events or query errors. Getter failures and the unavailable debug tracer are preserved; successful historical tracing used `trace_transaction`.

Run the offline reconciliation into a new output path:

```
python Research/scripts/analyze_pulsechain_funding_and_rebases.py \
--root artifacts/pulsechain-funding-and-rebases-20261002 \
--prior artifacts/pulsechain-synthetix-integrations-20261001 \
--output /tmp/pulsechain-funding-reproduction.json
```

The generated report preserves input hashes, fixed snapshots, reconciled asset flows, matched implementations, current quotes and explicit limits. The verifier checks receipts, transaction actors, chronology, burned portfolio receipts, identical amounts sold and spent, native trace payouts, runtime identity, balance accounting, resource state and destination-event coverage. Research verification does not require executing blockchain actions.

Appendix 37: How bridges, stablecoin issuers and pools can form one pDAI support system

Contents of this appendix

The integrated mechanism.....	240
Which date and which pools?	241
Same address has two different meanings.....	242
Which tokens actually provide supply buffers?.....	243
What governance control is actually confirmed?	244
Why coordinated control would be valuable	246
Add native L1 yield as a second defense.....	246
Close the Curve-Aave-Balancer loop.....	247
What would make this an operating system?	248
Reproduction and evidence limits	248
Inherited monetary candidates: every curated family.....	249
Receipts and unresolved monetary identities.....	256

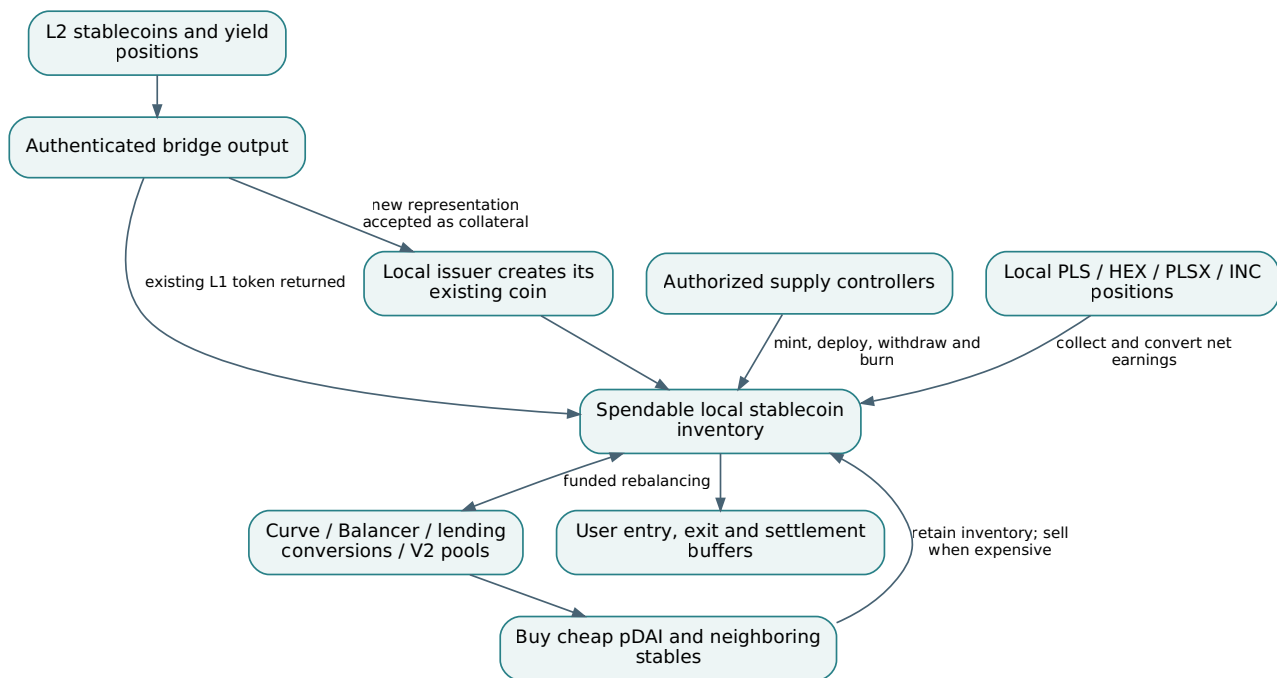
Source record: docs/studies/analysis/pulsechain_integrated_network_20261001.md.

How bridges, stablecoin issuers and pools can form one pDAI support system

1 October 2026. Read-only research. This study extends the [white paper \(main paper\)](#). Protocol names below identify candidate contract families; they do not identify the people controlling their keys.

The integrated mechanism

The strongest version of the proposal combines three sources of usable inventory: bridges that return existing local tokens; issuers that create existing local tokens against accepted collateral or authorized credit; and locally earned proceeds. A shared operator could move that inventory among Curve, Aave, Balancer and ordinary liquidity pools. Supply controllers could replenish an expensive stablecoin or retire an excess one. The operator would buy cheap eligible units, sell expensive units and preserve both sides of its conversion promise.



Every arrow is a separate asset conversion or contract permission. L1 here means PulseChain; L2 means the proposed pARB environment. Its present source deployments were not accessible in this research. Ethereum and ordinary Arbitrum documentation establish upstream designs, not a running pARB deployment.

The important improvement is that **local reissuance is optional when a bridge already returns the token the destination pool accepts**. New collateral adapters matter when the output is a different representation or when existing escrow cannot provide enough inventory.

The subsequent [dedicated sUSD study \(Appendix 66\)](#) adds its six inherited pool connections, mUSD basket acceptance and copied Synthetix issuer-specific synth bridge. It illustrates why the final pool-facing L1 address matters even when source addresses differ, and why issuer minting must be distinguished from pARB escrow release. Its fresh destination observations retain their separate block 27,683,120 identity.

Which date and which pools?

“As at May 2023” is defined here as the inherited snapshot, **10 May 2023 at 22:36:11 UTC**, Ethereum block **17,232,999**, hash `0xe5667f9cafb115b1dcfc36b0b569a5601f01dfa000698140a1298ef4f87e2e37`. This is the PulseChain inherited boundary, not the end of May. We verified that the Ethereum header matches the inherited PulseChain header before reading historical Ethereum state.

Eight previously identified responsive Curve registry roots return **618 unique pools and 381 distinct direct or underlying token identities** at that boundary. Of those pools, **176** have exact 3CRV as a direct coin and **64** have exact FRAXBP. Twelve underlying-coin getters failed; the pools and their direct coins remain in the catalogue. This pass did not measure historical pool balances. Registry membership includes experimental, abandoned and possibly empty pools; it does not establish an active issuer or available cash.

The later PulseChain registry census has **619** pools. Its extra pool is `0x02ac4107893ba767177d69851dcd87bedc63ab22`; its additional cWBTC identity is excluded from the inherited catalogue. Two historical listing reads beyond the historical count failed and were excluded. The historical census cannot be relabelled as a live capacity measurement.

The complete bounded pool list is [pool-catalogue.csv](#). The [all-token identity table](#) retains all 381 tokens, each token's pool addresses and candidate family. Unknown custom or unregistered pools remain outside these eight roots. Consequently, this is complete enumeration within the stated scope, not proof that every possible Curve deployment or monetary issuer has been found. Curve's [MetaRegistry description](#) explains why direct coins, underlying coins and registry overlaps must be distinguished.

Same address has two different meanings

First, the L2 token and L1 token can have **the same numeric address on different chains**. Those are still separate contracts and supplies. Second, a bridge can return **the exact existing L1 address**, even though the L2 address differs. Both can fit existing L1 infrastructure; the second is sufficient for an existing Curve coin.

At PulseChain block **27,681,116**, hash `0xea0b8aa496b70c7ae85abe1b403e79d80b04515a4c698430deb99f3974d863a7`, the checked router/gateway identities include:

Asset	Configured or calculated L2 identity	Existing L1 output	What the saved gateway implementation does
DAI	<code>0xda10009cbd5d07dd0cecc66161fc93d7c9000da1</code>	<code>0x6b175474e89094c44da98b954eedeac495271d0f</code>	Dedicated DAI gateway releases local DAI escrow
Bridged USDC	<code>0xff970a61a04b1ca14834a43f5de4533ebddb5cc8</code>	<code>0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48</code>	Registered custom gateway releases local USDC escrow
USDT	<code>0xfd086bc7cd5c481dcc9c85beb478a1c0b69fcb9</code>	<code>0xdac17f958d2ee523a2206206994597c13d831ec7</code>	Registered custom gateway releases local USDT escrow
OUSD	Calculated <code>0x97c7de436aa87e2db2d7cbd31d71a7b46d762dd9</code>	<code>0x2a8e1e676ec238d8a992307b495b45b3feaa5e86</code>	Standard escrow gateway; source deployment not established
Reserve eUSD	Calculated <code>0x12275dcb9048680c4be40942ea4d92c74c63b844</code>	<code>0xa0d69e286b938e21cbf7e51d71f6a4c8918f482f</code>	Standard escrow gateway; source deployment not established
mUSD	Calculated <code>0x23c6d883baeb519452426abb46fd85edb32940bf</code>	<code>0xe2f2a5c287993345a840db3b0845fbc70f5935a5</code>	Standard escrow gateway; source deployment not established
FRAX	Calculated <code>0x7468a5d8e02245b00e8c0217fce021c70bc51305</code>	<code>0x853d955acef822db058eb8505911ed77f175b99e</code>	Standard escrow gateway; source deployment not established

The standard gateway calculates a deterministic counterpart. It has no successful `L1ToL2Token` getter in these samples. That calculation alone does not prove source code, prior deposits, message availability or token backing. A independently deployed L2 token named OUSD at another address is not automatically this counterpart. The registered USDC is the older bridged USDC identity, not every Arbitrum token called USDC.

Literal identical numeric addresses were found for three configured pairs: Plenny PL2

(`0x3642c0680329ae3e103e2b5ab29ddf4d43cbe5`), Hashgold HGOLD (`0x445164d9e038e1dae0989e4b6ac745b78d26f181`) and MCDEX/MUX MCB (`0x4e352cf164e64adcbad318c3a1e22e9eba4ce42`). Historical L1 metadata supplies their names. Current pair getters agree on the identical addresses. MUX also lists MCB's matching Ethereum/Arbitrum addresses in its [primary token documentation](#). None is evidence of a universal dollar-stable bridge minter.

The checkpoint gateway token balance was zero for DAI, USDT, OUSD, Reserve eUSD, mUSD and FRAX; USDC had **1,000,000 raw units**, ordinarily one USDC at six decimals. These are balances at the selected gateway, not total ecosystem resources or all Across liquidity. An authenticated withdrawal cannot obtain an unlimited amount from an empty escrow. HGOLD's token permissions remain incompletely verified. PL2/MCB use the custom escrow gateway; their same-address configuration does not change its release model.

There are real token-specific mint branches. The saved [bridge inventory \(Appendix 4\)](#) verifies postfork ARB reverse-gateway minting and GRT shortfall minting under a token-specific allowance. LPT has a mint-on-shortfall design whose complete current issuer permissions remain unresolved. Those findings support specialized mint routes; they do not grant the same powers over DAI, USDC or every receipt.

The [136-token bridge identity table](#) retains mapping, calculated counterpart, registration result and raw gateway balance. The source chain was not queried, so these are configured destination identities rather than a list of proven live pARB assets.

Which tokens actually provide supply buffers?

“Supply buffer” has three useful forms. A **rebase** changes eligible holders' balances. **Authorized venue supply management** mints tokens into a pool or withdraws and burns them. **Collateral issuance/redemption** creates and retires tokens against an accepted asset or debt position. All can change available inventory, but they require different actions and obligations.

Candidate	Supply behavior relevant to the thesis	Exact qualification
FRAX	Curve AMOs can mint/deploy FRAX and remove supply through their permitted operations	Strong candidate for active venue inventory management; installed local controllers, limits and collateral accounting must be checked
DOLA	Feds expand by minting/supplying into a venue and contract by withdrawing/burning	Strong candidate for direct Curve/Balancer/lending supply management under actual Fed authority
OUSD	Yield rebase for eligible holders, plus collateral mint/redeem and configured strategies	The sampled OUSD/3CRV pool is nonrebasing; yield does not automatically expand its OUSD balance
UZD	Elastic yield claim, with version-specific rigid-address exclusions	The sampled 0xb40b...9f1c UZD pool is explicitly rigid; its pool balance does not follow the elastic yield calculation
PWRD	Yield-rebasing stable tranche	Yield, loss protection, pool eligibility and copied strategy operation require confirmation
Lybra V1 eUSD	Separate interest-rebasing eUSD design	Address 0x97de...a371, not Reserve eUSD 0xa0d6...482f; rates and inherited controller must be checked
STBT	Permissioned Treasury-yield rebase	Issuer and whitelist restrictions apply; the Ethereum Treasury custody arrangement is not copied by a PulseChain fork

Candidate	Supply behavior relevant to the thesis	Exact qualification
ARTH.usd	Rebasing wrapper around a distinct valuecoin system	Convert using its own target and share rules, rather than assume every raw unit is one dollar
USDN	Historical source monetary/interest design with Ethereum representation	Bridge and wrapper behavior must be audited before asserting that source interest reaches a local Curve balance
Reserve eUSD, mUSD, aUSD, XAI, USX, MAI, MIM, sUSD and others	Basket issuance, debt/credit issuance, authorized minting or debt amortization	Flexible supply is useful, but these are not interchangeable automatic pool rebases

Frax documents the historical [Curve AMO](#). Inverse documents [Fed expansion and contraction](#). These primary descriptions support the supply-controller mechanism; they do not demonstrate the current copied contracts' execution rights or live capacity. Later upstream versions must not be imported into the inherited design without matching deployed code.

The OUSD pool `0x87650d7bbfc3a9f10587d7778206671719d9910d` returns `rebaseState=0` and a nonzero `nonRebasingCreditsPerToken`. The inherited OUSD source uses a separate fixed credits-per-token value for nonrebasing accounts. Contract accounts default to that behavior unless eligible opt-in applies. This agrees with Origin's [rebase documentation](#). Pool membership alone therefore does not establish a yield-rebase buffer.

The verified UZD source at `0xb40b6608b2743e691c9b54ddbdee7bf03cd79f1c` implements separate elastic and rigid balances. Its Curve pool `0x68934f60758243eafaf4d2cfed27bf8010bede3a` returns true from `containRigidAddress` and appears in `rigidAddresses`. The other two inherited UZD versions reverted on those eligibility getters; their behavior remains unresolved. All three sampled daily deposit limits return zero, so unlimited immediate issuance is not established. The [exact verified source response](#) and saved getter responses are retained.

Gro describes [PWRD's rebasing tranche](#), Lybra describes [its own eUSD balance model](#), and Matrixdock documents [STBT's rebase and transfer restrictions](#). These architectures are candidate mechanisms, not current PulseChain yield results. Gro's upstream [wind-down notice](#) also prevents assuming its original yield operation continues unchanged.

A token rebase does not ensure an old pool recognizes increased balances as tradable LP inventory. Inspect the exact pool's stored balances, rate adapters and accounting. Curve's older [registry source](#) distinguishes pool balances from excess token balances treated as admin balances. Newer StableSwap NG explicitly handles rebasing assets, but that later implementation is not automatically installed in the May 2023 pools. Changing an excluded account to rebasing without compatible pool accounting could misallocate yield or break assumptions; it is not a demonstrated peg action.

Modern Mountain Protocol USDM must not be confused with the inherited **Mochi USDM** at `0x31d4eb09a216e181ec8a43ce79226a487d6f0ba9`; Mochi identifies it in [its deployment list](#). The final canonical crvUSD address `0xf939e0a03fb07f59a73314e73794be0e57ac1b4e` returns no code at the inherited boundary. Three earlier crvUSD-labelled tokens remain in the identity catalogue as unresolved versions. Later canonical crvUSD/PegKeeper or GHO designs cannot be silently assumed to operate in that inherited catalogue.

What governance control is actually confirmed?

There are three different questions: which contract has authority; who has enough votes or signer access to make it act; and whether the proposed group controls those people or keys. A token's `owner()` getter usually answers only part of the first question. Acquiring governance tokens can change voting control while every owner address stays unchanged.

The new comparison replays **516 generic authority getter records** across the inherited Ethereum boundary and the PulseChain checkpoint. **70** return the same valid encoded value; **446** are unresolved, empty or reverting. No resolved comparison shows a changed returned value. Another **18** controller/storage/delay comparisons match. Zero owners are retained as returned zero addresses, not assigned to a supposed controller. This is a narrow comparison of sampled getters, not evidence that effective voting control is unchanged. It also leaves many newly curated monetary candidates without issuer-controller verification.

Component	Evidence for control	What can be concluded
Across HubPool	Successful postfork ownership-transfer receipt/trace; fresh owner getter	Owner changed to 0xfeca7291cebe75789ffa707754db259187b1d140 on 19 June 2026
Across Safe OptimisticGovernor module	Successful postfork ownership-transfer receipt/trace; fresh owner getter	Module owner changed to 0x98c89980d1eae247ada3636ae27648f78e66e121 on 19 August 2026; this is not ownership of every UMA contract
Origin OUSD	Governor is timelock 0xc35918cde7233f2dd33fa41ae3cb6ae0e42e0e69f; two-day delay and sampled delegated admin match the inherited state	Administrative structure verified; common-group voting or execution control unconfirmed
Reserve eUSD	Main OWNER is timelock 0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c; proposer Governor Alexios 0x7e880d8bd9c9612d6a9759f96acd23df4a4650e6	Structure verified; saved selected Main/timelock event history has no postfork changes through its older endpoint; voting takeover remains untested
mStable	Nexus governor Safe 0xf6ff1f7fceb2ce6d26687eaab5988b445d0b94a2; three-of-five owners and sampled implementation/admin slots match inherited values	Structure verified; control of threshold signers/modules unconfirmed
Silo/XAI	Saved XAI owner-controller 0xe1f03b7b0ebf84e9b9f62a1db40f1efb8faa7d22; source/execution path unresolved	Candidate mint control, not confirmed takeover
FRAX / DOLA / aUSD	Saved issuer-specific owner/timelock, operator or ADMIN observations in the issuer study	Supply authority candidates identified; common-group controller access unconfirmed
Aave short executor	Current admin 0xec568fffb86c094cf06b22134b23074dfe2252c; pending admin 0xa30190b96faee0080144aa0b7645081fcbf49e6f	Nomination exists; pending is not accepted administration
Curve / Balancer	Existing pool/Vault/registry deployments and ordinary public trading interfaces	Trading access does not prove DAO, gauge, pool-admin or Vault administration takeover
Liquity V1 LUSD	Immutable issuer paths and governance-free architecture	There is no LQTY vote that can install arbitrary bridged collateral in this issuer

Component	Evidence for control	What can be concluded
Centralized issuers	Token-specific copied minter/admin roles	On-chain role control and external issuer redemption obligations are separate; no common takeover verified

Across transaction evidence is in the [control study \(Appendix 3\)](#): [Hub ownership transfer](#) and [module ownership transfer](#). These establish two authority changes within Across, not two independent protocol acquisitions. They do not establish common human ownership, ownership of UMA itself or the intent to support pDAI.

Per-family candidate authorities and all returned token addresses are listed below and in [authority-comparisons.csv](#). A full voting takeover proof would additionally need delegation/checkpoint votes, proposal thresholds/quorum, voting periods, the executable proposal outcome, timelock eligibility and any veto/guardian/upgrade powers. Multisigs require threshold signers and enabled modules. Centralized tokens require their specific mint/upgrade paths. No universal “owns governance” flag represents all these designs.

Why coordinated control would be valuable

As an inference about the proposed design, coordinated control could install receipt adapters, change accepted baskets, maintain rate feeds, authorize AMOs/Feds/minters, direct protocol revenues, and allocate owned liquidity and incentives. That would help connect financing and market operations without changing every Curve pool's coin list. Existing-token issuance and ordinary swaps often require issuer authority rather than Curve governance.

Those are reasons a coordinated operator might seek control. They do not prove why an observed address transfer occurred. The wider common-control hypothesis needs attributable wallet and executed-governance evidence. It is not required to buy or trade an immutable coin such as LUSD through an ordinary permitted route.

Add native L1 yield as a second defense

Native assets can contribute reserves and earnings without the L2 round trip. The shortest candidate route is **owned native position → collected proceeds → accepted local stable collateral → issuer or market → support inventory**. It does not require Origin or Reserve to directly accept the original native token.

Asset	Plausible local position	Conversion and timing to include
PLS	Native staking if the operator actually controls an eligible validator position; funded LPs; wrapped PLS lending where accepted	Staking/custody and exit terms, realized rewards and the market price of PLS; copied WETH that wraps PLS is not Ethereum ETH
HEX	Native HEX stakes or HEX LPs; compatible tokenized-stake positions if deployed	Stake maturity and penalties; token rewards need actual market conversion; holding liquid HEX differs from owning a stake
PLSX	Funded LP/farm or other verified earning position	Holding PLSX alone does not establish fee entitlement; verify the specific position, rewards and withdrawability

Asset	Plausible local position	Conversion and timing to include
INC	Realized farm rewards or an actual earning position	Emissions become support inventory only after a funded sale/conversion; perpetual reward-token demand is not established

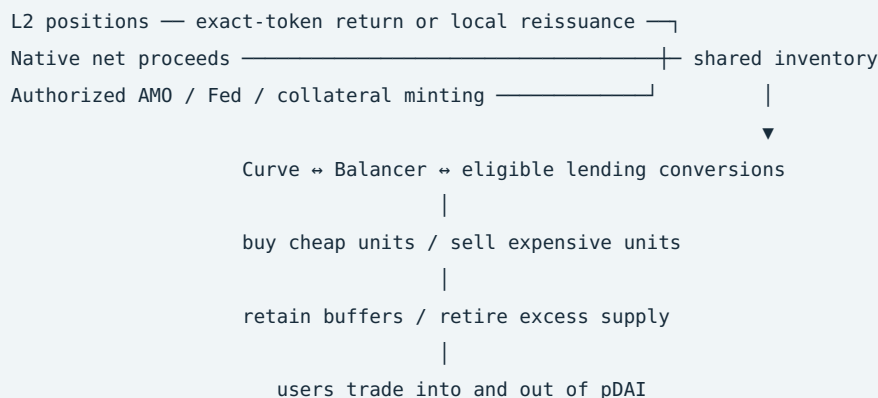
Origin's sampled vault accepts DAI/USDC/USDT, not PLS/HEX/PLSX/INC. Harvesting a native position and converting proceeds into accepted collateral is the easier branch. A new vault strategy, wrapper or collateral type requires a compatible authority and valuation path. The report does not certify native “staking in Origin” as an existing public product.

Reserve eUSD's sampled basket consists of four lending receipts; its RSR staking backstop is a separate function. Adding a native asset or LP claim to that basket requires a compatible collateral plugin, registration, valuation and activated basket. A separately configured RToken can be explored. A native-asset basket may track a volatile index; a \$1 accounting label does not remove its collateral's changing purchasing power. Newer upstream index products cannot be assumed to exist in the copied May 2023 RToken contracts. See the [Reserve collateral audit \(Appendix 18\)](#).

Keep immediately usable support assets outside long-maturity positions. Native receipts used as collateral are encumbered; their principal cannot simultaneously fund exits. Multiple PLS/HEX/PLSX/INC exposures may fall together. Their locally realized value can defend local parity, but an external dollar promise must use current realizable valuations, haircuts and available conversion. Fees from trading only among controlled accounts cancel in consolidated accounting; collectible payments from outside participants are different.

Close the Curve-Aave-Balancer loop

The integrated thesis is **an inventory and supply-management network with arbitrage between its venues**. A local stablecoin accepted as one accounting unit can connect markets. A working issuer conversion can create an actionable trading opportunity; its accounting oracle alone does not make the opportunity executable.



For example, **if** an Origin mint accepts one pDAI for one OUSD and a funded market pays 0.99 pUSDC for that OUSD, buying the pDAI for 0.80 pUSDC can leave 0.19 pUSDC before all fees, costs and risks. The route creates buying pressure on pDAI and selling pressure on OUSD. Its executable size depends on the issuer's current valuation/eligibility and the pool's actual bid. Quotes and balances change as trades execute. The example is conditional arithmetic, not a measured profitable route at the checkpoint.

If all connected ordinary dollar-unit coins have working conversions and available inventory, similar trades can narrow differences throughout their connected markets. FRAX/DOLA-style supply operators could mint/deploy where their own coin is scarce and withdraw/burn where it is excessive. Another issuer could accept accumulated cheap collateral and supply its existing coin. Ordinary LP deposits and withdrawals could redistribute liquidity. Yield proceeds could replenish the buffer. The system need not trade all pools in the same transaction or obtain governance over every coin.

The direction matters. Minting a metacoin and selling it for pDAI supports pDAI but consumes the metapool's counter-assets and increases its metacoin balance. Repeating that sale without a return path can destabilize that issuer's market. A complete loop therefore needs a policy for restoring each venue's useful inventory and meeting issuer redemption/bridge obligations, not just acquiring pDAI once.

Aave is a collateral, debt and available-cash venue; its oracle is not an AMM bid. A funded supply/withdrawal, valid borrowing/repayment or eligible receipt conversion is the route. Balancer has its own swaps, rate-provider/wrapper gates, reserve cash and limits. Curve base tokens such as 3CRV/FRAXBP are LP claims. Their pools can be linked, but they are not interchangeable unlimited one-dollar counters.

Normalize decimals, FX targets and receipt conversion rates before comparing prices. **aToken interest, cToken exchange rates, LP virtual prices, bLUSD appreciation, RAI's redemption target and FPI's index cannot all be treated as one raw token = \$1.** Some assets are genuinely foreign currencies; they can help route liquidity at appropriate exchange rates rather than being forcibly priced at a dollar.

A crucial boundary remains: relative arbitrage can make several local coins trade near one another even when the whole group has weak external dollar purchasing power. To claim actual \$1 purchasing power, name and fund the conversion to an independently priced reference asset. An ETH/BTC pool can provide an exit when funded; its market rate changes with those assets and does not itself define one dollar. Relative convergence, inventory coverage and external dollar conversion are three separate measurements.

What would make this an operating system?

Choose a useful connected subset and give each route an address, authority, available size, full delay and exit operation. Preserve bridge finality/challenge times, governance delays, basket warm-ups and replenishing mint limits, native stake maturities, ten-second purchase scheduling and open Portal/debt obligations. A long financing horizon is compatible with this design; each immediate conversion promise still needs deliverable inventory while slower resources are pending.

Measure both trade directions after fees; count shared 3pool reserves and nested collateral only once; specify issuer supply expansion and contraction; and measure the loss of capacity when a bridge, oracle or issuer fails. The new identity census proves what was registered at the inherited boundary. The current getters prove specific configurations and exclusions. No state-changing operation or end-to-end funded mint/bridge/arbitrage campaign was executed.

Reproduction and evidence limits

Inputs, raw requests/responses, failure records, curation and outputs are in `artifacts/pulsechain-integrated-network-20261001/`; explicit manifests are under `Research/experiments/pulsechain_integrated_*_20261001.json`. `snapshot-headers.json` records the correct boundary and endpoint. An earlier exploratory header request used the wrong boundary and is expressly excluded. PulseChain historical state reads failed at the selected public endpoint, so historical contract reads use Ethereum archive state at the matching inherited header. This is an inherited-state reconstruction, not a later cross-chain equivalence assumption.

The probe requires `--calls` for this study. Use `--chain-id 1 --single-requests --block 0x106f467 --rpc https://eth.drpc.org` for the historical getter manifests, and the recorded chain 369 block for the current manifests with an RPC retaining that state. Some later metadata retries failed because the public endpoint had pruned that current block; failures are preserved and historical names are labelled accordingly. A failed call is neither a zero balance nor proof of absent code. The historical canonical crvUSD code read succeeded and returned `0x`.

Run the two offline reconciliations after the frozen inputs exist:

```
python Research/scripts/analyze_pulsechain_issuer_routes.py --folder artifacts/pulsechain-integrated-network-20261001
python Research/scripts/analyze_pulsechain_integrated_network.py --folder artifacts/pulsechain-integrated-network-20261001 --curation Research/experiments/pulsechain_integrated_protocol_families_20261001.json
```

The probe now supports individual JSON-RPC requests for providers without archive batch support, explicit chain identity, and a research user-agent. Research validation passed **63 tests and 82 subtests**, including unknown-authority preservation, mixed-block rejection and single-request response matching. Production code was not changed.

Inherited monetary candidates: every curated family

The following table covers **74 candidate family/identity groups and 108 token addresses**, including foreign currencies, index targets and explicitly unresolved dollar-labelled identities. It is not “74 confirmed dollar-stable issuers.” Each row’s controller description states the authority to trace. Returned generic addresses are narrow getter observations; they are not a completed effective-governance audit. **Common-group takeover is unconfirmed for every row below.** Verified Across authority changes are separately identified above; Across is not a stablecoin issuer in this table.

Candidate family	Inherited pool token labels	Supply path to verify	Authority to trace / returned token getter
Maker	DAI	Collateral debt and permitted peg modules (primary reference)	MKR voting, Chief/executors; token owner is not sufficient; no resolved generic token authority in this comparison
Circle	EUROC, USDC	Issuer-authorized mint/burn; administrative restrictions (primary reference)	Owner, master minter and proxy admin; no DAO voting ownership; returned masterMinter() 0x02398771fd1db790ef2b656ca3bcb3075f27a72c, masterMinter() 0xe982615d461dd5cd06575bbea87624fda4e3de17, owner() 0x92efac9375e18c9fdac65a439e1a480cd9c09c0, owner() 0xfcb19e6a322b27c06842a71e8c725399f049ae3a
Tether	EURT, USDT, MXNt	Issuer-authorized mint/burn; administrative restrictions (primary reference)	Issuer owner and privilege paths; no DAO voting ownership; returned owner() 0xc6cde7c39eb2f0f0095f41570af89efc2c1ea828

Candidate family	Inherited pool token labels	Supply path to verify	Authority to trace / returned token getter
Paxos	BUSD, USDP	Issuer-authorized mint/burn; two different USDP addresses require identity separation (primary reference)	Paxos roles for Pax Dollar/BUSD; Unit USDP is separately mapped below; returned owner() 0x0644bd0248d5f89e4f6e845a91d15c23591e5d33
TrustToken/True currencies	TUSD, TGBP	Issuer-authorized fiat token supply	Issuer/admin and proxy authority, separately by token; returned owner() 0x00000000000075efbee23fe2de1bd0b7690883cc9, owner() 0x00000000bbcf7700a1b403c9eb666f350707b900
Gemini	GUSD	Issuer-authorized fiat token supply	Custodian/admin controls; no resolved generic token authority in this comparison
Origin	OUSD	Yield rebase plus collateral mint/redeem; contracts can be nonrebasing (primary reference)	Vault governor/timelock, strategies and price provider; OGV voting; returned governor() 0x35918cde7233f2dd33fa41ae3cb6ae0e42e0e69f
Reserve	RSV, eUSD	Collateral basket issuance/redemption; eUSD is not a universal balance rebase (primary reference)	RSV issuer roles; eUSD Main OWNER/timelock and stRSR governor, separately; returned owner() 0xfc82f7d67facea4e93b8501f76ff5003cedccd89
mStable	mUSD	Basket mint/redeem; savings exchange-rate shares	Nexus governor Safe, proxy admin and upgrade delay; no resolved generic token authority in this comparison
Frax	FPI, FRAX	Authorized AMOs and collateral issuance; FPI targets purchasing-power index (primary reference)	FRAX owner/timelock, pools and AMO controllers; FXS/FPIs paths differ; returned owner() 0x6a7efa964cf6d9ab3bc3c47ebddb853a8853c502, timelock_address() 0x8412ebf45bac1b340bbe8f318b928c466c4e39ca
Inverse	DOLA	Feds mint and supply into venues; withdraw and burn (primary reference)	INV governance, Fed chair/operator and token minter rights; no resolved generic token authority in this comparison
Silo	XAI	Authorized credit lines with limits (primary reference)	XAI owner/controller, Silo DAO administration and credit-line permissions; no resolved generic token authority in this comparison

Candidate family	Inherited pool token labels	Supply path to verify	Authority to trace / returned token getter
Alchemix	aUSD	Collateralized debt; authorized minters and ceilings; yield repays debt	ALCX governance/admin and enabled mint ceiling; no resolved generic token authority in this comparison
Liquity V1	LUSD, bLUSD	LUSD immutable native collateral debt; bLUSD is an appreciating Chicken Bonds claim (primary reference)	LUSD has no governance onboarding power; bLUSD contract must be examined separately; returned owner() 0x00
Reflexer	RAI	Collateral debt with floating redemption target (primary reference)	Governance-minimized parameter controllers; not a one-dollar token; no resolved generic token authority in this comparison
Synthetix	sCHF, sKRW, sUSD, sGBP, sEUR, sAUD, sJPY	Collateralized synthetic currencies with rate feeds	SNX collateral plus councils/contracts; token front does not establish effective authority; returned owner() 0xeb3107117fead7de89cd14d463d340a2e6917769
dForce	USX	Collateralized and privileged supply routes (primary reference)	DF governance, vault controllers and mint limits; returned owner() 0x17e66b1e0260c930bfa567ff3ab5c71794279b94
QiDao	MAI	Collateral vault debt and version-specific supply routes (primary reference)	QI governance, vault owners/controllers and bridge routes; returned owner() 0x3182e6856c3b59c39114416075770ec9dc9ff436
Abracadabra	MIM	Collateralized Cauldron borrowing with debt limits (primary reference)	SPELL governance, master contracts and cauldron owners; returned owner() 0x5f0dee98360d8200b20812e174d139a1a633edd2
Bao	bSTBL, BaoUSD	Borrowing and basket claims; bSTBL is not the same raw dollar unit (primary reference)	Bao governance/admin, vault and basket controllers; no resolved generic token authority in this comparison
Metronome	msUSD	Collateralized synthetic debt (primary reference)	MET governance/admin and collateral/mint controllers; no resolved generic token authority in this comparison

Candidate family	Inherited pool token labels	Supply path to verify	Authority to trace / returned token getter
CLever	clevUSD	Yield/debt position and controlled issuance	Issuer, minter/debt ceiling and strategy authority; returned owner() 0xfc08757c505ea28709df66e54870fb6de09f0c5e
Interest Protocol	USDI	Reserve-backed mint/ redeem and interest-accruing supply (primary reference)	Vault controller, governor and collateral/oracle configuration; returned owner() 0x266d1020a84b9e8b0ed320831838152075f8c4ca
Fei	FEI	Protocol-controlled supply and reserve operations	TRIBE governance, Core roles and authorized controllers; no resolved generic token authority in this comparison
Beanstalk	BEAN	Demand-driven algorithmic issuance/debt; multiple legacy versions (primary reference)	Diamond governance/ executor and version-specific contracts; returned owner() 0xc1e088fc1323b20bcbec9bd1b9fc9546db5624c5
Volt	VOLT	Purchasing-power target and protocol reserve supply	Governor/controller and collateral valuation; target is not fixed one-dollar; no resolved generic token authority in this comparison
Ubiquity	uAD	Algorithmic expansion and debt/coupon mechanisms	Manager/admin roles and Ubiquity governance; no resolved generic token authority in this comparison
Gro	PWRD	Yield rebasing stable tranche (primary reference)	Controller/admin, strategy and loss-tranche accounting; no resolved generic token authority in this comparison
Zunami	UZD	Elastic yield claims with rigid-address exclusions; three versioned contracts (primary reference)	Default admin/rebalancer roles and underlying vault valuation; no resolved generic token authority in this comparison
Matrixdock	STBT	Permissioned Treasury-yield rebase (primary reference)	Timelock/controller/issuer/moderator roles and whitelist; copied token does not copy Treasury rights; no resolved generic token authority in this comparison

Candidate family	Inherited pool token labels	Supply path to verify	Authority to trace / returned token getter
Lybra V1	eUSD	Collateralized interest-rebasing eUSD; distinct from Reserve Electronic Dollar (primary reference)	Lybra controller/minter and rate/collateral configuration; no resolved generic token authority in this comparison
MahaDAO	ARTH, ARTH.usd	Valuecoin and rebasing USD wrapper, separately valued	Issuer/controller and wrapper conversion rules; returned owner() 0x6357edbfe5ada570005ceb8fad3139ef5a8863cc
Neutrino	USDN, EURN	Bridged monetary claims; wrapper/rebase propagation needs version audit	Source monetary controller plus bridge/custody authorities; no resolved generic token authority in this comparison
Mochi	USDM	Authorized collateral-debt mint/burn, not Mountain USDM (primary reference)	Mochi Engine and governance/controller; no resolved generic token authority in this comparison
Unit Protocol	USDP	Collateralized stablecoin debt	Unit Protocol vault/manager governance, not Paxos; no resolved generic token authority in this comparison
NAOS	nUSD	Collateralized/credit monetary claim	Issuer and credit/governance controller; no resolved generic token authority in this comparison
OKLink	USDK	Issuer-backed monetary token	Issuer/admin authority; returned owner() 0x46dec2f9093221acab7a806f536f783fea6bda32
HUSD issuer	HUSD	Issuer-authorized monetary token	Issuer/admin authority; returned owner() 0x2bd9caa19a724608bef32dd3c10c0fd1f70b309d
Spice	USDS	Monetary supply/credit design, deployment-specific	Issuer/controller authority; returned owner() 0x9d0bf2632710f48eadff8620fe757bf7093a7ee1
GMO	GYEN, ZUSD	Issuer-authorized fiat supply	GMO issuer/admin; GYEN is JPY; returned owner() 0xfb5dae5bf48df4492b3101f00d844ef1c0268c6d
Terra legacy	UST	Bridged legacy stablecoin claim, two bridge versions	Source rules and bridge authority, not generic local mint governance; returned owner() 0x3ee18b2214aff97000d974cf647e7c347e8fa585, owner() 0x9123077acafb3d743c68418304b2a11566cc1175

Candidate family	Inherited pool token labels	Supply path to verify	Authority to trace / returned token getter
Vader	USDV	Algorithmic/reserve supply, deployment-specific	Vader monetary controller/admin; returned owner() 0xfd9ad7f8b72fc133543cb7cc c2f11c03b81726f9
Archimedes	cealv, lvUSD	Authorized leverage-dollar supply; cealv claim is separate	Owner/minter/leverage controller by deployment; no resolved generic token authority in this comparison
handle.fi	fxEUR, fxUSD	Collateralized synthetic currency debt	FOREX governance and controller/oracle paths; no resolved generic token authority in this comparison
Jarvis	jGBP	Collateralized synthetic GBP	JRT/controller/maintainer and collateral/oracle paths; no resolved generic token authority in this comparison
Iron Bank forex	ibCHF, ibJPY, ibGBP, ibZAR, ibKRW, ibEUR, ibAUD	Synthetic foreign currencies, not ordinary lending shares	Issuer/controller roles and rate feeds; no resolved generic token authority in this comparison
Angle	agEUR	Collateralized euro supply	ANGLE governor/guardian and minter controllers; no resolved generic token authority in this comparison
STASIS	EURS	Issuer-authorized EUR supply	Issuer/admin controls; no resolved generic token authority in this comparison
Monerium	EURe	Issuer-authorized EUR supply	Issuer/admin controls; returned owner() 0x5591a6495d726c7efafb9a9d ad32695d137b4d9f
Membrane	EUROe	Issuer-authorized EUR supply	Issuer/admin controls; no resolved generic token authority in this comparison
VNX	VEUR	Issuer-authorized EUR supply	Issuer/admin controls; returned owner() 0xcdb3867935247049e87c38ea 270edd305d84c9ae
StraitsX	XSGD	Issuer-authorized SGD supply	Issuer/admin controls; returned masterMinter() 0x8c3b0caec968b2e640d96ff0 b4c929d233b25982, owner() 0x6bd9c2b46c17dd1d1ef64c6f e26eb2d630b498d6
DFX	dfxSGD, dfxCAD	Foreign-currency protocol claims; conversion rules required	DFX governance/admin, collateral and mint controllers; no resolved generic token authority in this comparison

Candidate family	Inherited pool token labels	Supply path to verify	Authority to trace / returned token getter
BiLira	TRYB	Issuer-authorized TRY supply	Issuer/admin controls; no resolved generic token authority in this comparison
Blackfridge/poundtoken	GBPT	Issuer-authorized GBP supply	Issuer/admin controls; no resolved generic token authority in this comparison
JPYC	JPYC	Issuer-authorized JPY supply	Issuer/admin controls; no resolved generic token authority in this comparison
CADC issuer	CADC	Issuer-authorized CAD supply	Issuer/admin controls; no resolved generic token authority in this comparison
ZARP	ZARP	Issuer-authorized ZAR supply	Issuer/admin controls; no resolved generic token authority in this comparison
MIMO	PAR	Collateralized EUR stablecoin debt	MIMO governor/admin and vault controllers; no resolved generic token authority in this comparison
FIAT DAO	FIAT	Collateralized debt against fixed-income claims	FIAT DAO controllers/ governance; no resolved generic token authority in this comparison
Empty Set Dollar/Digital Standard	DSU	Collateral conversion monetary unit; exact reserve rule needs deployed audit	Reserve/controller and deployment administrator; no resolved generic token authority in this comparison
xDollar	XIM	Collateralized monetary claim	Issuer/admin and collateral controller; no resolved generic token authority in this comparison
DefiDollar	DUSD	Stable basket / protocol reserve claim	Governance/admin and asset adapters; no resolved generic token authority in this comparison
Celo bridged	cUSD, cEUR	Bridged source monetary tokens	Source monetary governance plus bridge; local copied token rights separate; returned owner() 0x3ee18b2214aff97000d974cf647e7c347e8fa585
Coin98	CUSD	Basket/bridge monetary token, deployment-specific	Issuer/controller and collateral reserve authority; returned owner() 0x1db6ad727ae60d7b4dbec81f79c4bcbcf8759f8

Candidate family	Inherited pool token labels	Supply path to verify	Authority to trace / returned token getter
SORA	XSTUSD, TBCD	Synthetic/bonding-curve monetary claims	Source monetary authority and bridge/controller; returned owner() 0x313416870a4da6f12505a550b67bb73c8e21d5d3
Wasabi	waUSD	Monetary claim, issuer model requires contract audit	Issuer/controller not established; no resolved generic token authority in this comparison
Equal	RUBE	Ruble monetary claim	Issuer/controller not established; no resolved generic token authority in this comparison
Index-labelled CPI	CPI	Index target; exact issuer requires audit	Issuer/controller not established; no resolved generic token authority in this comparison
Titanium Dollar (identity unresolved)	T	Three distinct dollar-labelled contracts; issuer identity unverified	Each issuer/controller unresolved; no resolved generic token authority in this comparison
Adamant Dollar (identity unresolved)	A	Three distinct dollar-labelled contracts; issuer identity unverified	Each issuer/controller unresolved; no resolved generic token authority in this comparison
Vanadium Dollar (identity unresolved)	V	Three distinct dollar-labelled contracts; issuer identity unverified	Each issuer/controller unresolved; no resolved generic token authority in this comparison
Defi Franc	DCHF	CHF-target debt design, exact deployed path requires audit	Issuer/controller not established; returned owner() 0x83737eae72ba7597b36494d723fbf58cafee8a69
Tron DAO Reserve	USDD	Source reserve/monetary issuance with bridge representation	Source issuer plus local bridge/controller authority; no resolved generic token authority in this comparison

Receipts and unresolved monetary identities

These remaining label candidates are retained individually below. Aave aTokens, Compound cTokens, Yearn/iearn shares, Morpho supply vaults, Flux and Iron Bank lending shares, Ruler positions, Curve LPs and Tokemak/Temple receipts are associated infrastructure; their receipt balances are not distinct freely mintable dollar currencies. Ambiguous dollar names and experimental crvUSD versions require issuer identification. No symbol is used as takeover evidence.

Exact token	Label	Identity or interpretation requiring verification
0x028171bca77440897b824ca71d1c56cac55b68a3	aDAI	Aave interest bearing DAI
0x04bc0ab673d88ae9dbc9da2380cb6b79c4bca9ae	yBUSD	iearn BUSD
0x07ac55797d4f43f57ca92a49e65ca582cc287c27	TBT	rebasing TBT
0x0e2ec54fc0b509f445631bf4b91ab8168230c752	LINKUSD	LINKUSD
0x11ebe21e9d7bf541a18e1e3ac94939018ce88f0b	pitchFXS	Pitch FXS
0x16de59092dae5ccf4a1e6439d611fd0653f0bd01	yDAI	iearn DAI
0x1b3c515f58857e141a966b33182f2f3feecc10e9	USK	Usk
0x1be5d71f2da660bfdee8012ddc58d024448a0a59	ycUSDT	Curve (iearn fork) USDT
0x1edf1cf760ebfb6e0b7bd410c2bebb591c7523dd	USDANT	USDANT Stablecoin
0x20658291677a29efddfd0e303f8b23113d837cc7	LUSD	Boosted
0x213ecae6b3cbc0ad976f7d82626546d5b63a71cb	GCD	GCD Stablecoin
0x26ea744e5b887e5205727f55dfbe8685e3b21951	yUSDC	iearn USDC
0x29b41fe7d754b8b43d4060bb43734e436b0b9a33	REUSD	Real Estate USD
0x3175df0976dfa876431c2e9ee6bc45b65d3473cc	crvFRAX	Curve.fi FRAX/USDC
0x36f8d0d0573ae92326827c4a82fe4ce4c244cab6	maDAI	Morpho-Aave Dai Stablecoin Supply Vault
0x39aa39c021dfbae8fac545936693ac917d5e7563	cUSDC	Compound USD Coin
0x3ed3b47dd13ec9a98b44e6204a523e766b225811	aUSDT	Aave interest bearing USDT
0x3fa372155439e9150a99d86a1de93a92f26b00c8	RC_INV_350_DAI_2021_5_31	Ruler Protocol rToken
0x42ef9077d8e79689799673ae588e046f8832cb95	fUSD	fryUSD
0x465a5a630482f3abd6d3b84b39b29b07214d19e5	fUSDC	Flux USDC

Exact token	Label	Identity or interpretation requiring verification
0x466a756e9a7401b5e2444a3fcb3c2c12f bea0a54	PUSd	Stablecoin
0x48759f220ed983db51fa7a8c0d2aab8f3 ce4166a	iUSDT	Iron Bank Tether USD
0x5aa00dce91409b58b6a1338639b9daa63 eb22be7	crvUSD	Curve.Fi USD Stablecoin
0x5d3a536e4d6dbd6114cc1ead35777bab9 48e3643	cDAI	Compound Dai
0x683923db55fead99a79fa01a27eec3cb1 9679cc3	aFEI	Aave interest bearing FEI
0x6bad6a9bcfda3fd60da6834ace5f93b8c fed9598	USV	USDC Savings Vault
0x6c3f90f043a72fa612cbac8115ee7e52b de6e490	3Crv	Curve.fi DAI/USDC/USDT
0x6c5024cd4f8a59110119c56f8933403a5 39555eb	aSUSD	Aave interest bearing SUSD
0x71df9dd3e658f0136c40e2e8ec3988a52 26e9a67	GrapefruitUSD	GrapefruitUSD
0x73a052500105205d3daf004eab301916 da8190f	yTUSD	iearn TUSD
0x76eb2fe28b36b3ee97f3adae0c69606ee db2a37c	iUSDC	Iron Bank USD Coin
0x804c23b6ccfa0a1fae1a81c9964f42ae6 89c790e	BSD	Bahamas Sand Dollar
0x810c4b2a31665ef77b75d2410aceb29f0 f099096	RC_WETH_1650_DAI_2021_4_30	Ruler Protocol rToken
0x823e1b82ce1dc147bbdb25a203f046afa b1ce918	COIL	Coil
0x83f798e925bcd4017eb265844fddabb44 8f1707d	yUSDT	iearn USDT
0x8e595470ed749b85c6f7669de83eae304 c2ec68f	iDAI	Iron Bank Dai Stablecoin
0x94671a3cee8c7a12ea72602978d1bb84e 920efb2	tFRAX	TokemaktFRAX
0x95ecd6caa7e4805fcef2679a9233835 1d24297	crvUSD	Curve.Fi USD Stablecoin
0x9777d7e2b60bb01759d0e2f8be2095df4 44cb07e	ycUSDC	Curve (iearn fork) USDC
0x99d1fa417f94dcd62bfe781a1213c092a 47041bc	ycDAI	Curve (iearn fork) DAI

Exact token	Label	Identity or interpretation requiring verification
0xa5269a8e31b93ff27b887b56720a25f844db0529	maUSDC	Morpho-Aave USD Coin Supply Vault
0xa7e7131a57e44f832cb2de78ade38cad644aac2f	maUSDT	Morpho-Aave Tether USD Supply Vault
0xb556bb40e40f4af34d9eea04d2ff7db2674939bb	RC_xCOVER_100_DAI_2021_6_30	Ruler Protocol rToken
0xbcb8b7fc9197feda75c101fa69d3211b5a30dcd9	xFraxTempleLP	Stax Frax/Temple LP Token
0xbcca60bb61934080951369a648fb03df4f96263c	aUSDC	Aave interest bearing USDC
0xbfef1f07018b3a87fc1e12877038f9616512d587	xUSD1	xUSD1
0xc2cb1040220768554cf699b0d863a3cd4324ce32	yDAI	iearn DAI
0xcf178a685471927e977a0ea8ee555c11b2aa6b7b	rUSD	rUSD
0xd6ad7a6750a7593e092a9b218d66c0a814a3436e	yUSDC	iearn USDC
0xe2ba8693ce7474900a045757fe0efca900f6530b	fDAI	Flux DAI
0xe6354ed5bc4b393a5aad09f21c46e101e692d447	yUSDT	iearn USDT
0xe7d18f2507869280111025f4938376846e8ad11a	RC_xRULER_15_DAI_2021_6_30	Ruler Protocol rToken
0xed279fdd11ca84beef15af5d39bb4d4be23f0ca	LUSD3CRV-f	Curve.fi Factory USD Metapool: Liquity
0xef1385d2b5dc6d14d5fecb86d53cdebefca20fcc	crvUSD	Curve.Fi USD Stablecoin
0xef779cf3d260dbe6177b30ff08b10db591a6dd9c	kUSD	kUSD
0xff709449528b6fb6b88f557f7d93dece33bca78d	ApeUSD	ApeUSD

Appendix 38: L2 collateral, local stablecoin issuance and the Curve network

Contents of this appendix

Finding and scope.....	260
The two source branches and the destination conversion.....	262
The simplest path before governance changes.....	263
Origin: existing OUSD output, new collateral requires compatibility.....	263
Reserve: collateral plugins are a direct match to the hypothesis.....	264
mStable: ordinary accepted inputs work differently from new receipt onboarding.....	265
Other stablecoin families and the easier mint-authority branch.....	266
L2 staking, lending and liquidity candidates.....	268
The complete bounded pool census.....	270
Bridges, delays and the multi-year financing idea.....	271
Economic merits and the limits of the deterministic claim.....	272
Reproduction and proof status.....	273

Source record: docs/studies/analysis/pulsechain_issuer_collateral_network_20260930.md.

L2 collateral, local stablecoin issuance and the Curve network

30 September 2026. This study evaluates an issuer-mediated route from asserted pARB assets to existing PulseChain stablecoins. It examines every pool returned by eight selected responsive Curve registry sources, and audits the principal local issuer branches. It does not certify all possible pARB deployments, every stablecoin's operational mint path, or common human control of their authorities.

Finding and scope

The proposed route is technically meaningful: an authorized source position can become collateral for a destination issuer, which can mint the **existing local token address** recognized by an existing Curve pool. A newly bridged representation need not become a Curve coin. Most of the proposed governance work consequently concerns the issuer, its collateral, valuation and mint permissions.

Reserve's collateral-plugin architecture is especially suitable. Origin requires compatible supported-asset accounting. The current mUSD implementation appears to need an upgrade for a genuinely new collateral address. Some other tokens offer an even shorter authorized-mint branch. None of those architectural observations proves who controls the necessary authority or that source collateral is currently available.

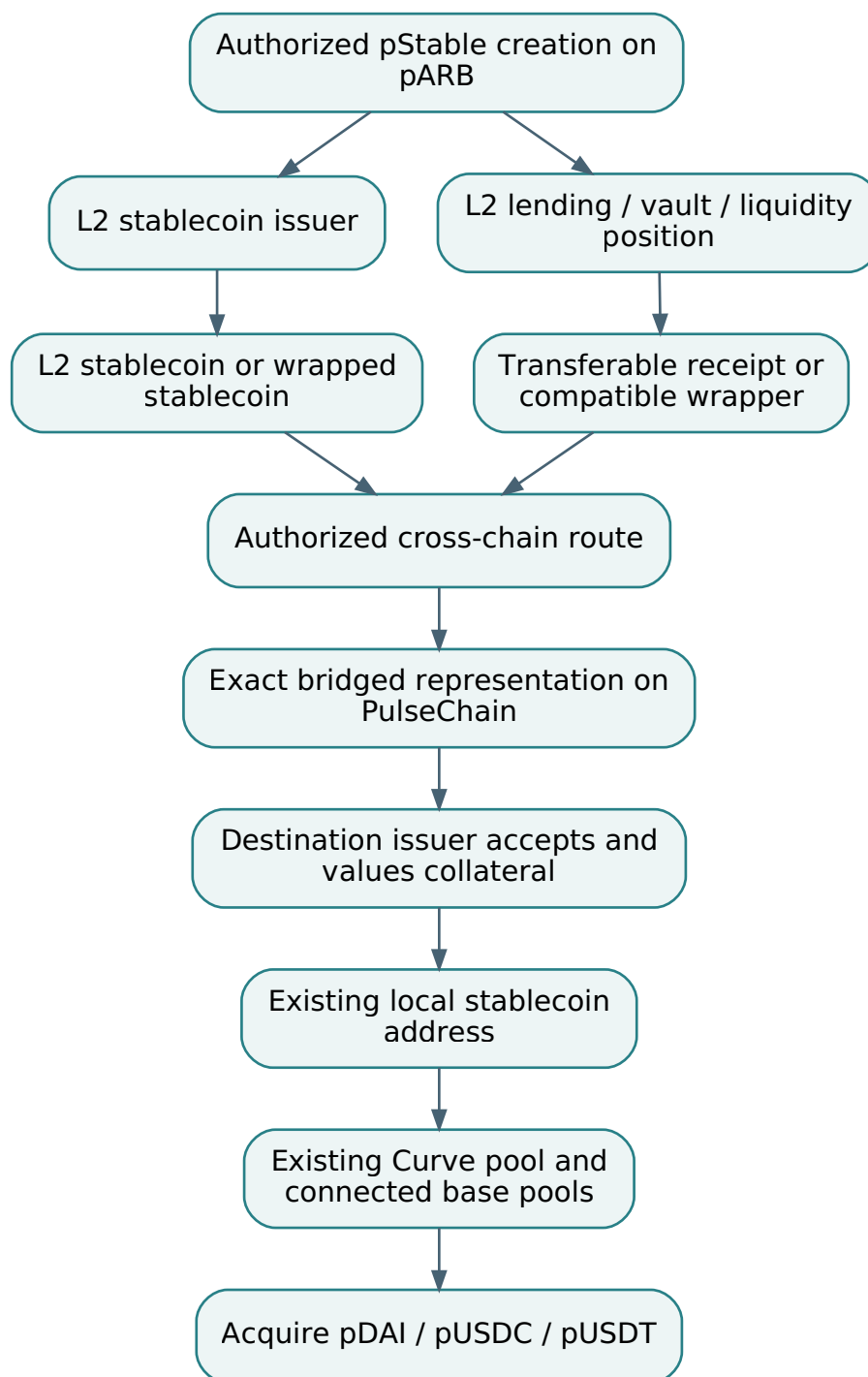
The hypothesis that every Curve stablecoin's governance was acquired is stronger than the mechanism needs. A useful subset can connect several major pools. Literal universal governance control also conflicts with immutable issuer designs such as Liquity V1. Such coins may still be bought, traded, or minted through their existing permitted collateral route.

Fresh chain observations are pinned to PulseChain chain **369**, block **27,675,130**, hash `0xdd12f23ad571882b63fd333c5d3909a45a05fe6f0a3c2b273cb7e1e6b0917954`, timestamp **2026-09-30 05:20:55 UTC**. Each probe rechecks the block header. Source retrieval is separately dated and is not a historical bytecode attestation. Explorer verification is recorded; a verified proxy is not treated as a verified implementation.

All chain operations were getters, view quotes, code or storage reads. No signing, broadcasts, administrative changes, privileged impersonation, storage overrides or state-changing simulations were performed. Positive mintability cannot be established solely by configuration getters or a quote that omits transfers and permissions.

The [maintained paper](#) ([main paper](#)) presents the mechanism simply. This study supplies the distinctions needed to assess it.

The two source branches and the destination conversion



The first branch matches the proposal to mint OUSD, eUSD, mUSD or other stables on L2 from source stable inputs, bridge them, and use their representations to obtain L1 issuance. The second uses lending, vault or LP receipts directly. Neither branch presupposes that a historical Ethereum issuer was deployed natively on historical Arbitrum. Custom pARB deployments are possible assumptions, but need an RPC, chain identity and contract list before their actual state can be inspected.

“e-stables” is retained as a description of source inputs. It has not been assigned a particular Reserve eUSD address, an Ethereum dollar entitlement or a backing model. Those identities materially affect valuation and redeemability.

Three identities must remain distinct:

Object	Example	Required evidence
Source asset	L2 OUSD or aUSDC	Actual source deployment, accepted input, supply mechanism and withdrawal rights
Destination representation	A token representing that source position	Bridge mapping, issuer/minter authorization, supply reconciliation and holder rights
Destination issuer output	Existing local OUSD or eUSD	Exact local token address, authorized issuance and actual issuer acceptance

An L2 OUSD representation does not become local Origin OUSD because the names match. An ordinary canonical withdrawal can release existing L1 escrow, while a custom representation can create a different collateral token. A destination issuer can then convert that new collateral into its own canonical output if its rules allow it.

There is no general requirement to create a new LP before that conversion. An issuer normally receives collateral through its own deposit/mint path. An LP receipt is useful only if that receipt itself is accepted collateral or a permitted strategy position. “Pair it inside the protocol” must specify whether this means a vault deposit, basket membership, a strategy holding, a collateralized loan, or an LP position.

The simplest path before governance changes

First ask whether the bridge can deliver an **already accepted local collateral address**. If it can deliver local DAI, USDC, USDT or a required lending receipt, the corresponding issuer can use its existing rules. There is no need to whitelist a foreign stablecoin merely to mint a local one.

That shortcut may use escrow or relayer cash. It does not establish a limitless supply of local collateral. A bridge withdrawal is bounded by its backing model; a liquidity fill is bounded by spendable inventory and settlement. The benefit is avoiding collateral onboarding and preserving existing issuer accounting.

Next ask whether the destination token already has a controlled authorized minter. DOLA's verified token allows its operator or approved minters to create supply. XAI's verified token permits its owner to mint. aUSD uses administrative whitelist and ceiling controls. FRAX uses authorized pools. If the coordinated-control assumption includes those exact powers, some branches can issue their local Curve coin without first bridging a new receipt.

This is stronger support for a multi-issuer supply thesis, but does not mean all token owners can mint all assets. Source code must establish the role's powers; live reads must identify its holder; contract holders require their own execution path; any ceiling, timelock, pause, collateral or liquidity constraint must be carried forward.

Origin: existing OUSD output, new collateral requires compatibility

The local token is 0x2a8e1e676ec238d8a992307b495b45b3feaa5e86. Its reported vault is 0xe75d77b1865ae93c7eaa3040b038d7aa7bc02f70. The vault's active implementation slot identifies 0x997c35a0bf8e21404ae4379841e0603c957138c3, whose explorer-verified source is VaultCore.

The current asset list contains exactly local DAI, USDT and USDC. Capital operations are unpaused. The mint predicate requires a supported asset, positive deposit, acceptable oracle price and minimum output. The valuation is capped at 1e8, and the minimum acceptable oracle price is 99,800,000. These are contract accounting units, not a test of dollar redeemability. At the pinned block the three sampled oracle prices pass that threshold.

The code mints OUSD and transfers the deposited collateral within one atomic transaction. A failed later transfer or strategy action reverts the transaction. It may rebase and allocate strategies during minting; passing the sampled price predicate is not proof those downstream actions succeed.

For a new L2 receipt, the technical issues are:

- Exact support of its destination address and usable pricing.
- Decimal normalization and whether value is represented by a growing balance or a growing share price.
- Rebase, strategy allocation, redemption and total-value accounting compatibility.
- A bridge redemption rule and source withdrawal route that can support the receipt's stated value.

A non-rebasing share worth 1.20 underlying units would still encounter the core's \$1 mint-price ceiling per normalized token. Correct treatment may need a compatible wrapper, strategy or accounting upgrade. A price below \$0.998 prevents ordinary issuance even if the asset has been added.

The vault governor and token governor are 0x35918cde7233f2dd33fa41ae3cb6ae0e42e0e69f. Its verified Timelock source checks proposer and executor roles; the sampled minimum delay is **172,800 seconds, two days**. The vault delegates administrative calls to 0x1ef0553feb80e6f133cae3092e38f0b23da6452b. That address and the current oracle lack verified source in this explorer sample. The core exposes a governor-only way to replace its admin implementation. Ordinary asset-onboarding behavior of the present admin is therefore unresolved, rather than assumed from modern Origin documentation.

No complete proposer-holder/voting-control census was established here. Activity at an OGV staking proxy is evidence of activity, not proof of proposal authority or common ownership. Modern [Origin contracts](#) describe current upstream development; they are not substituted for the sampled older multi-asset implementation.

The existing destination is OUSD/3CRV at 0x87650d7bbfc3a9f10587d7778206671719d9910d. Successful issuance of that exact OUSD token does not require a new pool coin. Directly sending a different bridged OUSD representation does not provide the same path.

Reserve: collateral plugins are a direct match to the hypothesis

The local Reserve eUSD token is 0xa0d69e286b938e21cbf7e51d71f6a4c8918f482f. Main is 0x7697ae4def3c3cd52493ba3a6f57fc6d8c59108a; AssetRegistry is 0x9b85ac04a09c8c813c37de9b3d563c2d3f936162; BasketHandler is 0xd309297dddfea104a6e89a132e2f05ce3828e07; BackingManager is 0xf014fef41ccb703975827c8569a3f0940cfd80a4.

A bridged receipt can be represented by a collateral plugin, provided the plugin implements the version's interfaces and accurately handles reference value, target units, price, default status and redemption assumptions. The current registry contains **14 registered assets**. Registration is distinct from basket membership.

The configured prime basket is four assets at 0.25 target units each:

Asset	Exact local token
Static Aave USDC receipt	0x60c384e226b120d93f3e0f4c502957b2b9c32b15

Asset	Exact local token
Compound USDC receipt	0x39aa39c021dfbae8fac545936693ac917d5e7563
Static Aave USDT receipt	0x21fe646d1ed0733336f2d4d9b2fe67790a6099d9
Compound USDT receipt	0xf650c3d88d12db855b8bf7d11be6c55a4e07dcc9

A new collateral route generally requires plugin registration, prime-basket selection, activation/refresh of the reference basket and passage of the issuance checks. Merely registering a plugin does not make a deposit issue eUSD. The existing v2 core can support a suitable plugin without necessarily replacing the core implementation.

A single-asset basket is an architectural possibility if governance configures a valid compatible collateral asset. A partial basket allocation requires all remaining components. If 100 target-value units provide one 25% component, issuance of 400 units requires 300 target-value units in the other components as well. It is not fourfold unbacked purchasing power.

Issuance refreshes collateral, requires a sound basket, observes throttles, obtains a rounded-up deposit quote and transfers every required collateral token into BackingManager. The sampled cached basket status is SOUND, fullyCollateralized is true, and Main is unpaused/unfrozen. Older saved plugin reads nevertheless returned unusable price ranges and stale May 2023 feeds. Cached status is not proof that the next refreshing issuance passes. Mutating refresh or issue calls were not simulated.

Main's previously reconstructed OWNER is timelock 0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c; its proposer is GovernorAlexios 0x7e880d8bd9c9612d6a9759f96acd23df4a4650e6. The sampled voting token is 0x18ba6e33ceb80f077deb9260c9111e62f21ae7b8; quorum numerator is 15, voting delay is 14,400 blocks and voting period is 21,600 blocks. The previous timelock read gives three days. Block-based voting durations are not automatically Ethereum's calendar durations on PulseChain. Voting and timelock phases are sequential; permissions are not instantly installed because someone holds tokens.

The previous governance-history pass found no post-fork Main/timelock role changes through its endpoint. That does not exclude changes in effective voting control without a role transfer. It also does not establish the coordinated-control assumption. See the [collateral source audit \(Appendix 18\)](#) and [governance reconstruction \(Appendix 19\)](#).

The destination eUSD pool is 0xaeda92e6a3b1028edc139a4ae56ec881f3064d4f. Its other coin is FRAXBP, **not 3CRV**. Its base pool is 0xdcef968d416a41cdac0ed8702fac8128a64241a2, containing FRAX and USDC. It connects to DAI/USDT through other paths. Reserve eUSD must also be distinguished from the separate token labeled eUSD at 0x97de57ec338ab5d51557da3434828c5dbfada371.

mStable: ordinary accepted inputs work differently from new receipt onboarding

Local mUSD is 0xe2f2a5c287993345a840db3b0845fbc70f5935a5. Its EIP-1967 implementation is 0x15b2838cd28cc353afbe59385db3f366d8945aee, verified as MusdV3. Its sampled four basket inputs are sUSD, USDC, DAI and USDT. The basket is neither failed nor undergoing recollateralization, and the maximum basket size reports ten.

The following **view-only** tests use `getMintOutput` at the pinned block:

Input, 100 whole units	Quoted mUSD, rounded
Local DAI	100.122312
Local USDC	99.600131

Input, 100 whole units	Quoted mUSD, rounded
Local USDT	100.238943

Each input at **10 million whole units** reverts on weight limits. Local OUSD and Reserve eUSD at both tested sizes revert as invalid assets. These tests establish the quote branch's behavior. They do not execute a collateral transfer, run mint's full healthy/permission path or prove backing/liquidity.

The verified source includes a library helper that can add a basket asset. The current token implementation does **not** expose it through an ordinary public collateral-addition function. It exposes minting, weight-limit adjustment, integration migration and other administration. The presence of a helper in a source bundle is not a callable onboarding interface.

Consequently, accepted underlying delivered by the bridge is the simpler route. Accepting an entirely new bridged receipt appears to need a compatible upgrade or a separately constructed route that ends in an existing accepted asset. An integration migration alters where an existing basket asset is invested; it is not an authorization to treat a different ERC20 as that basket asset.

Nexus is 0xafce80b19a8ce13dec0739a1aab7a028d6845eb3. Its governor is Safe 0xf6ff1f7fceb2ce6d26687eaab5988b445d0b94a2, whose sampled threshold is **three of five**:

- 0x284a7f74c73340c08ce413edf44101c23243c66c
- 0x253956aedc059947e700071bc6d74bd8e34fe2ab
- 0x738f25847a2bd6dee11e3ccf438572daa1b413f6
- 0x4f9dfef24e6c5109a7869981f3ffdd865c8546a5
- 0xbe8377398879245a14e1453ffa711f93121c9273

The proxy admin is 0x5c8eb57b44c1c6391fc7a8a0cf44d26896f92386. Its getters point to that Nexus and a **604,800-second, seven-day** upgrade delay, matching Nexus's delay. Its present source is not explorer-verified here, so complete upgrade authorization still requires that controller's code audit. Nexus's governor controls the current implementation's governor-guarded configuration. The Safe's signer list is not proof that one group controls the signers or threshold.

The existing mUSD/3CRV pool is 0x8474ddb98f5aa3179b3b3f5942d724afcddec9f6. No coin-list change is needed if the issuer mints that existing mUSD address. [mStable's primary contract documentation](#) provides upstream context; actual behavior above comes from the pinned local implementation and reads.

Other stablecoin families and the easier mint-authority branch

The catalogue contains local instances of the following important families. This table describes the route type and the unresolved authority or collateral check; it does not assert that their governance was acquired.

Family	Relevant mechanism	Implication for the proposed network
XAI / Silo	XAI token owner can mint; Silo uses credit lines and collateralized lending	Potential direct supply/credit branch; inspect owner-controller execution and credit limits
DOLA / Inverse	Token operator and approved minters; upstream Feds deploy supply into venues	Potential direct local issuance without adding a new Curve coin

Family	Relevant mechanism	Implication for the proposed network
alUSD / Alchemix	Whitelisted minters with ceilings, blacklist and pause checks	A compatible authorized collateral manager may issue; mere ALCX ownership is insufficient
FRAX	Authorized minting pools and governance-controlled administration	A compatible pool/AMO may be simpler than foreign collateral added directly to FRAX
USX / dForce	Stablecoin vault/controller architecture	Potential receipt collateral after exact vault approval and limits; current local implementation unresolved
MAI / QiDao	Collateral-specific vaults and authorized stablecoin issuance	Receipt route is plausible only through an accepted vault and compatible mint authority
MIM / Abracadabra	Authorized minting and Cauldron collateral lending	Distinguish bridge escrow release, MIM supply authority and a funded Cauldron borrowing path
sUSD / Synthetix	Synth issuer/resolver and collateral/debt mechanisms	SNX voting or balances alone do not prove arbitrary collateral or unbounded synth issuance
FEI	Protocol-controlled authorization and backing/market operations	Present issuer permissions, implementation and operational status unresolved
msUSD / Metronome	Collateralized synthetic issuance with collateral factors	Receipt compatibility, oracle and debt capacity must be specified
BaoUSD	Collateralized synthetic borrowing	A receipt requires a compatible collateral vault and local output mint authority
Unit USDP	Distinct collateralized issuer	It is not Paxos USDP; exact contract/controller inspection required
RSV	Older Reserve token/controller system	It is not the eUSD RToken basket; eUSD plugin conclusions do not transfer automatically
LUSD / Liquity V1	Immutable permitted issuer path	No governance power to add an arbitrary new stable receipt; native-collateral route and market acquisition remain alternatives
RAI / Reflexer	ETH/native-asset collateral and an adjustable redemption target	It is not a fixed-\$1 stablecoin; issuer conversion and prices require its own rules
USDC, USDT, Paxos USDP/BUSD, TUSD, GUSD and other centralized families	Specific minter/operator/admin/upgrade powers	Voting-token accumulation does not grant these powers or an Ethereum bank redemption right
Non-dollar tokens	Currency-specific issuers, receipts or bridges	EUR/GBP/CHF/JPY/AUD/SGD assets require FX conversion; one unit is not one dollar

Family	Relevant mechanism	Implication for the proposed network
BEAN, FPI, VOLT and other distinct mechanisms	Protocol-specific issuance or value targets	They cannot inherit the \$1 collateral assumptions of Reserve or Origin

Concrete current authorities from this batch include DOLA operator

0x926df14a23be491164dcf93f4c468a50ef659d5b, XAI owner-controller 0xe1f03b7b0ebf84e9b9f62a1db40f1efb8faa7d22,

FRAX owner address 0xb1748c79709f4ba2dd82834b8c82d4a505003f27 and FRAX timelock

0x8412ebf45bac1b340bbe8f318b928c466c4e39ca. XAI's owner-controller has code but no verified source in the sample; its execution authority is unresolved. An owner-only mint at the token layer is not proof that an external wallet can make the owning contract call it.

USDC reports owner 0xfcb19e6a322b27c06842a71e8c725399f049ae3a and master minter

0xe982615d461dd5cd06575bbea87624fda4e3de17. USDT reports owner 0xc6cde7c39eb2f0f0095f41570af89efc2c1ea828.

alUSD's enumerable ADMIN role has one holder, 0x8392f6669292fa56123f71949b52d883ae57e225; controlling that contract's execution is a separate requirement. Distinct role addresses are not proof of distinct humans or common humans. A generic owner getter can also describe a proxy rather than the effective issuance controller. Every returned authority is preserved with its exact call, including zero, empty and reverting results.

The local LUSD token's verified source fixes BorrowerOperations, TroveManager and StabilityPool as immutable addresses; only its BorrowerOperations can mint. BorrowerOperations is

0x24179cd81c9e782a4096035f7ec97fb8b783e007. The [Liquity V1 documentation](#) explicitly describes an immutable, governance-free design. Its original collateral is native ETH; copied native-value operations on PulseChain use PLS. This does not prove a present operational LUSD loan quote or collateral valuation. It does show why LQTY governance acquisition cannot be the universal onboarding explanation.

Primary architectural references for other branches are [Silo's historical XAI credit-line discussion](#), [Inverse Feds](#), [dForce vaults](#), [QiDao vault terminology](#), [Abracadabra Cauldrons](#), [Metronome collateral factors](#), [Bao borrowing](#) and [Reflexer's governance-minimization guide](#). These establish architecture, not PulseChain ownership or pARB deployment.

L2 staking, lending and liquidity candidates

No accessible pARB RPC or complete pARB deployment inventory was supplied. It is therefore impossible to certify “every live pARB protocol” from the name of the chain. The existing [native-assets baseline \(Appendix 15\)](#) establishes specific historical Arbitrum identities, including Aave V3, Curve and GMX; it does not establish every later protocol on a copied or custom chain.

The candidate network below covers the materially different receipt and issuance architectures. Each proposed local/source instance needs code, accepted token addresses, live deposit/withdrawal predicates, receipt transferability and bridge mapping. A documented Ethereum/Arbitrum product is not evidence that it exists on pARB.

Source family and candidate protocols	Position obtained from suitable stable inputs	Destination issuer requirement
Aave V3	Stable aTokens; static wrappers can avoid rebasing balances	Plugin/wrapper must value the claim and distinguish cash from unbacked amounts
Radiant	Lending rTokens and reward positions	Exact receipt, withdrawal cash and reward rights; no automatic Reserve compatibility

Source family and candidate protocols	Position obtained from suitable stable inputs	Destination issuer requirement
Silo	Lending collateral shares; separate collateral-only and debt shares	Asset-specific share token and solvency/transfer checks; debt shares are not redeemable principal
dForce lending	Interest-bearing supply claims	Exact iToken/controller, exchange rate and available underlying
Compound III	Interest-bearing base-asset balance	Use a compatible claim/wrapper; supplied collateral itself earns no interest in Comet
Fraxlend	fToken shares representing lent assets	Market-specific claim and rate/withdrawal accounting; borrowing is a distinct liability
Curve	Stable LP shares and possibly gauge-staked positions	Full pool composition, virtual-price/withdrawal treatment and stake portability
Uniswap V2, Sushi V2, Camelot V2	Fungible LP shares; farming may custody them	Both reserve legs and withdrawal rights; unfunded pairing is not full collateral
Balancer	BPT/linear/boosted pool shares	Recursive underlying claims and rate providers; shared capital counted once
Uniswap V3, Camelot V3	Range-specific positions, commonly represented by NFTs	A purpose-built wrapper/adaptor, valuation and transferable ownership
Trader Joe/LFJ Liquidity Book	Bin-specific positions	Compatible position wrapper; not assumed to be an ordinary fungible ERC20
Stargate	Stable liquidity-provider claims	Source cash, bridge exposure, withdrawal availability and compatible representation
Beefy	Vault/compounder shares, including LP strategies	Exact strategy, share price and withdrawal path; rewards are not a second principal
Yearn	Yield-vault shares	Version-specific asset/strategy and price-per-share; later products are not fork evidence
Pendle	SY, principal tokens, yield tokens and LP positions	Identify the actual claim; maturity and principal/yield separation matter
GMX	Historical GLP or later market liquidity positions	Mixed-asset basket and trader P&L; do not value as a fixed \$1 stable receipt
Jones, Plutus, Umami and similar managed vaults	Strategy-specific stable or GLP-related shares, where deployed	Additional manager/strategy/lock/redemption risks and exact local deployment proof
Abracadabra, QiDao, Vesta and other collateralized issuers	A minted/borrowed stablecoin if collateral is accepted	Issuance authority, collateral rules and borrowing obligation, not merely staking yield

Source family and candidate protocols	Position obtained from suitable stable inputs	Destination issuer requirement
Origin, Reserve and mStable source instances	Their stablecoins or savings/wrapped claims if custom deployments exist	Explicit source deployment and bridging; not presumed native May-2023 Arbitrum instances
Convex/Stake DAO-style aggregators	Custodied LP/gauge claims or reward rights, where supported	Need a transferable claim to recover principal; reward tokens alone do not suffice

Fraxlend's [primary description](#) distinguishes lending shares from borrowing. Compound III distinguishes interest-bearing base supply from non-interest-bearing collateral. Radiant rTokens, Beefy vaults, Stargate liquidity claims and Pendle's [principal/yield distinction](#) supply the architecture references. Camelot V2 and Uniswap V3 require different position handling.

GMX is particularly time-sensitive: its [current archived V1 documentation](#) says upstream GLP minting has been phased out. Historical GLP capability must not be presented as a currently usable upstream product or an automatically operating pARB deployment. Similar version/date checks apply to new Yearn, Silo, Alchemix and stablecoin products.

Silo's [classic interface](#) distinguishes collateral shares, collateral-only shares and debt shares. It uses the term “bridge assets” for assets connecting isolated lending markets. That terminology does not prove cross-chain transport. XAI token issuance and a Silo's borrowing/liquidity constraints must be tested as separate layers.

Every row can potentially supply a representation for an L1 issuer after suitable integration. **None gives a generic right to deposit its receipt into unchanged Origin, Reserve or mStable.** The most direct candidates are transferable stable lending or vault receipts with understandable withdrawal rights. NFT positions, mixed-risk LPs, leveraged vaults and expiring claims require more adaptation and valuation.

The complete bounded pool census

Curve's provider `0x0000000022d53366457f9d5e68ec105046fc4383` supplied registry addresses. Eight responsive roots were enumerated completely:

Provider ID	Returned pool count
0, main registry	49
3, stable factory	303
5, crypto registry	9
6, crypto factory	251
7, metaregistry	612
8	4
9	4
10	4

Their 1,236 listing entries reconcile to **619 unique addresses**, with zero failed or invalid listing results. The metaregistry overlaps the children; counts are not added as distinct markets. All 619 direct-coin arrays decoded successfully. Twelve underlying-coin reads failed and remain explicitly unresolved; all

base-pool getter outputs were retained. The [primary metaregistry interface](#) explains aggregation, while actual lists here come from PulseChain reads. The sampled metaregistry source itself was not explorer-verified.

The 382 unique direct/underlying token addresses include 98 dollar-stable **label candidates** and 32 non-dollar/other-stable label candidates. Those labels include lending shares and LP tokens; they are not 130 proven stablecoin issuers. Three symbols remain unresolved. Pools can contain anonymous or misleadingly named tokens. The CSV retains every address and never treats a familiar symbol as issuer/backing proof.

There are **176 direct-coin 3CRV pools** and **64 direct-coin FRAXBP pools**. Other stable pools include direct currency pairs, alternative multi-coin bases, lending pools, non-dollar pairs, mixed crypto/stable pools and nested LP positions. Not every Curve pool contains a stablecoin or belongs to the pDAI peg network.

The full census is available as [pool CSV](#), [pool JSON](#) and [token inventory](#). The CSV includes coin identities, underlying identities, registry source, hub classification and raw per-coin balance results. Unknown reads remain unknown. Raw quantities lack decimal normalization and executable-price meaning.

Each pool must be assessed under the same distinctions:

1. **Same existing local coin:** successful issuance can feed ordinary swaps/liquidity without replacing pool coins.
2. **Different bridged token:** current coin membership provides no direct route unless an existing conversion exists.
3. **Base-pool or receipt coin:** the backing inventory and withdrawal route must be traced recursively.
4. **Empty or incomplete balances:** registration is not an executable trade or funded exit.
5. **Mixed/non-dollar assets:** quotes, risk and FX differ from the dollar-stable model.

Corrected balance getters show 350 pools with at least one nonzero coin balance, 235 with every returned coin balance zero, and 34 with incomplete balance reads. Among direct-coin 3CRV pools these counts are 67, 89 and 20; among FRAXBP pools they are 37, 27 and zero. Nonzero inventory is not a successful trade quote or a dollar valuation.

The [issuer-candidate CSV](#) and [JSON](#) retain all 130 label candidates, their pool memberships, explored authority getters, source-verification status where fetched, and unresolved effective control/collateral acceptance. This covers candidates without pretending every symbol is an audited issuer. Unrecognized symbols and remaining token addresses stay in the full token/pool catalogue.

The census preserves all cases. It does not attest that all listed pools are active, liquid, source-audited, permissionless or correctly valued. A source audit and trade-size view quote remain necessary before claiming an individual route is currently executable. Older named-pool transaction evidence remains in the [Curve history study \(Appendix 8\)](#).

Bridges, delays and the multi-year financing idea

Five bridge families remain relevant: ordinary escrow withdrawals, specialized/custom gateways, source-native burn/destination mint systems, Portal receipt financing, and liquidity-fill bridges such as Across. They provide different inventory, token identities, delays and liabilities.

An ordinary Arbitrum gateway can release an existing local token from escrow. It does not automatically mint extra canonical supply. A new collateral representation requires its own authorized mapping/minter or a compatible bridge implementation. The [bridge inventory \(Appendix 4\)](#) preserves 129 registered router mappings plus seven examined extra candidates. Its escrow findings do not refute a separately authorized issuer accepting new bridged collateral.

Across can accelerate supported transfers through liquidity fills, subject to the copied/custom system's routes, relayers, repayment and dispute conditions. Controlling a governance role does not by itself place arbitrary stablecoin or receipt support into its current token/route mappings. A fill delivering canonical local USDC is one branch; minting a new collateral representation is another. The [Across authority study \(Appendix 3\)](#) separates those checks.

The saved copied pARB Rollup confirmation getter returns **45,818 L1 blocks**. This is a block count, not a fixed multi-year settlement date; accepted ancestry, admission, challenges and chain progress affect confirmation. The saved Across Hub liveness getter returns **7,200 seconds, two hours**. That bundle period does not equal a user fill's latency, final reimbursement time or a Portal maturity. Both observations and their raw responses are retained in the [deferred campaign study \(Appendix 34\)](#).

Delay must be stated per route. It includes source issuance, any collateral withdrawal queue or maturity, bridge confirmation/finality, governance installation, issuer throttles/basket warm-up, market accumulation, and eventual debt/backing settlement. Origin's two-day timelock and the reported mStable seven-day upgrade delay are configuration clocks. Reserve's voting clocks are block-based and its sampled timelock is three days. These are not a common bridge waiting period.

For the proposed ten-second campaign, 50bn input units at 10,000 per slot require approximately 579 uninterrupted days. A 1,000-unit allocation extends it to roughly 15.9 years. A pause while pDAI meets the target extends calendar time. Pool replenishment and third-party demand determine whether spaced orders improve overall acquisition; cadence does not create purchasing power or erase aggregate slippage.

A lending/vault receipt can remain invested for years if its actual rules permit it. The earlier Portal source review found aggregate unbacked accounting without a per-mint settlement countdown. Current positive Portal issuance is nevertheless blocked by all 17 observed unbacked caps being zero. New permissions, usable supply caps and authorized bridge execution are prerequisites. Multiple bridges share headroom; later issuance does not erase earlier unbacked amounts.

The complete examined 86ff V3 debt inventory includes twelve nonzero assets. Separate matching receipts can retire its account debts if legitimate account execution is available, but those burned receipts cannot also fund purchases. Portal exposure persists after receipt-based borrower repayment. Debt reduction does not transfer ownership of the borrower's remaining collateral. The [deferred campaign study \(Appendix 34\)](#) retains exact amounts, rates, time scenarios and source predicates.

Economic merits and the limits of the deterministic claim

The strongest version of the thesis is a coordinated collateral-and-issuance network. It can create market-facing local coins, spread acquisition over different pools, retain yield-producing source collateral and accumulate ownership of pStables and their liquidity. It may use existing issuer coin addresses, avoid changes to established pool contracts and give a local unit multiple conversion paths. Direct mint authorities could simplify selected branches further.

The mechanism needs a consolidated balance sheet. If a source position is deposited as backing for local eUSD, the operator cannot simultaneously count that position as free settlement inventory. If eUSD then backs another issued coin, the second issuance creates another claim along the chain, rather than independent unencumbered capital. Genuine fee or interest income can grow net resources; a controlled borrower paying a controlled lender shifts funds within the group. Repeated controlled swaps also incur costs rather than generating outside fee income.

Issuer outputs enter Curve against finite opposing inventory. A metacoin swap buys existing pStables and can move their relative prices; a liquidity deposit supplies inventory and mints an LP claim. An unbalanced deposit may leave the pool more unbalanced. "Rebalancing" should mean a defined improvement in a specified balance, not every addition of a minted token.

A network valuation may recognize 100 source units as 100 target units, or apply a haircut/LTV that permits less issuance. An oracle can assign \$1 to an asserted source asset. That creates a contract-accounting value; redeemable dollars or freely saleable collateral require additional evidence. Discount, liquidity, bridge loss, withdrawal queues and liquidation thresholds can constrain what the issuer's liabilities actually buy.

Ownership concentration improves the operator's ability to supply liquidity and manage circulating float. Acquiring every outside pDAI removes outside pDAI sellers at that moment. Issuing or selling tokens again recreates outside claims. A funded redemption venue can implement a definite exchange rule within its available inventory and permissions. A V2 LP's reserve ratio changes with trades, even when one entity owns the LP. A controlled network can intervene, but a universal price guarantee needs a specified payment asset and funded intervention capacity.

Internal parity among pDAI and locally managed stables is a useful target. Calling it a dollar peg requires an external dollar reference or a defined dollar-deliverable claim. An ETH/WBTC exit can supply market convertibility without fixing dollar proceeds. Frozen copied oracles and a hypothetical fixed GHO accounting price cannot replace that exit inventory. The [managed-peg study \(Appendix 45\)](#) and [portal pricing study \(Appendix 48\)](#) preserve the earlier accounting/market distinction.

The network may still be powerful under the coordinated-control assumption. The appropriate conditional statement is specific: the system can issue through compatible authorized branches, acquire assets at executable quotes, and support a defined conversion within funded capacity. The evidence does not yet establish every source deployment, every controller's effective execution, universal issuer control, an installed cross-chain collateral basket, or unlimited external redemption.

Reproduction and proof status

The explicit batch is `pulsechain-issuer-collateral-20260930`. Manifests are under `Research/experiments/pulsechain_issuer_*_20260930.json`; frozen raw responses, fetched sources and complete catalogues are under the corresponding artifacts directory. The study record is this file. Bulk pool/token rows remain generated outputs, not copied study tables or committed data.

Run the offline reconciliation:

```
python Research/scripts/analyze_pulsechain_issuer_routes.py --folder artifacts/pulsechain-issuer-collateral-20260930
```

For a read-only replay, run the probe with the selected explicit getter manifest and a new output path, for example:

```
python Research/scripts/probe_pulsechain_routes.py --block 0x1a649fa --calls Research/experiments/pulsechain_issuer_route_predicates_corrected_20260930.json --output artifacts/pulsechain-issuer-collateral-replay/route-predicates.json
```

Do not use the probe's default mode: earlier default operations include state-changing simulations outside this task's scope. Every manifest used here has been checked to contain only getters/view calls or read RPC methods. Decimal quantities must be JSON integers; hex strings must start with `0x`.

Two initial balance/quote batches encoded decimal strings incorrectly. Their uncorrected outputs are excluded from findings. Corrected numeric manifests were replayed at the same block, and the probe now rejects malformed string arguments. In particular, early zero-quantity quote reverts are not issuer evidence. The retained evidence note identifies these exclusions.

The checks establish structural route possibilities, current collateral lists, selected controller addresses, fixed-block view predicates and a complete census for the enumerated registries. They do not establish the proposed system's deployment or an executed end-to-end route. No new five-agent review is claimed; the earlier three-round review applies only to its frozen earlier text. Diagrams are source-reviewed and have not been raster-render checked.

Appendix 39: StakeWise and Lido routes through PulseChain lending and governance

Contents of this appendix

Exact assets and routes	275
Where sETH2 is directly listed	276
Lido can supply local stETH without governance control.....	277
Silo control can pass through voting rather than the owner Safe.....	278
What the four supplied transactions establish.....	279
Evidence and reproduction.....	280

Source record: docs/studies/analysis/pulsechain_lido_and_seth2_listings_20261002.md.

StakeWise and Lido routes through PulseChain lending and governance

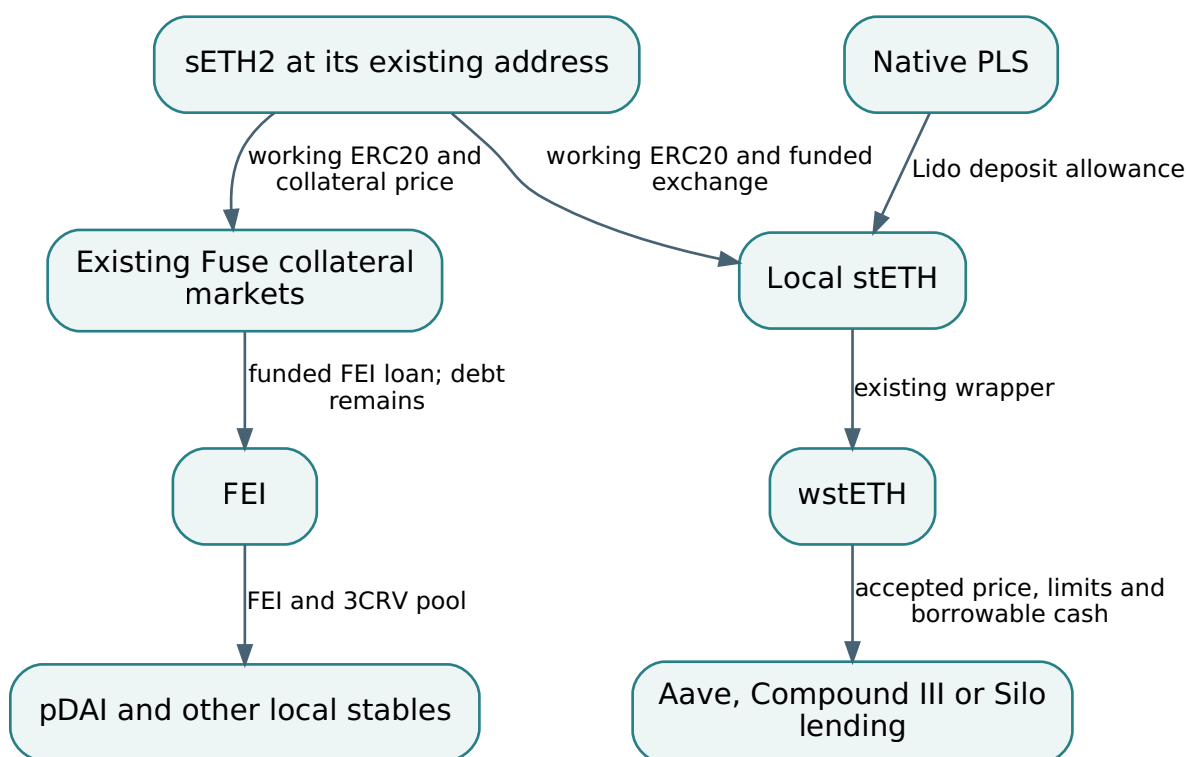
2 October 2026. Read-only research at PulseChain block **27,690,565**, hash

0x8eafcc2cbd9f3c7fd00a499ba4173ba098bc207762d553543f998be5bac03334, timestamp **01:24:15 UTC**. Historical comparisons use the inherited block **17,232,999**. No transactions or setter simulations were submitted.

sETH2 already has direct exchange and lending integrations. Silo also has a voting path to important permissions without taking over its owner Safe. These findings strengthen the proposed network, but do not establish a completed Silo takeover or a working L2 bridge. The current sETH2 implementation fails ERC20 getters, interrupting its routes. Lending additionally needs a working price and actual assets available to borrow.

Exact assets and routes

Asset	PulseChain address	Relevant distinction
sETH2	0xfe2e637202056d30016725477c5da089ab0a043a	Controlled proxy; current implementation is StakeWise Pool code rather than a functioning ERC20 implementation
rETH2	0x20bc832ca081b91433ff6c17f85701b6e92486c5	StakeWise rewards token; reward distribution does not establish collateral listing
stETH	0xae7ab96520de3a18e5e111b5eaab095312d7fe84	The local Lido token paired with sETH2 in the inherited Curve pools
wstETH	0x7f39c581f595b53c5cb19bd0b3f8da6c935e2ca0	Immutable wrapper for that exact stETH, with more lending listings
wsETH2	0x5da21d9e63f1ea13d34e48b7223bcc97e3ecd687	A Set Protocol basket whose current sole component is sETH2; separate from wstETH



These are existing identities and conditional routes. A bridged asset must become the exact accepted token, or pass a separately authorized adapter. No observed sETH2 mint trace establishes that bridge. The [StakeWise proxy study \(Appendix 63\)](#) records the upgrades, minter permissions and direct local issuance.

Where sETH2 is directly listed

Venue checked	Exact sETH2 integrations	Present limitation
Curve	Two inherited sETH2/stETH pools	Quotes fail; counter-token holdings are zero and about 9.10 local stETH
Canonical Uniswap V3 factory	Seven pools, including copied WETH, rETH2, SWISE, wsETH2, wBETH and pDAI	Stored liquidity and counter-token balances do not prove a trade can execute while sETH2 is broken
Uniswap V2 and PulseX factories	65 pairs: 2 Uniswap V2, 12 PulseX V1 and 51 PulseX V2	Many are dust pairs or custom local assets; stored reserves can be stale
Balancer V2 Vault	One current sETH2/wsETH2 pool	Both Vault balances are zero; this is a nested claim, not an independent ETH reserve
Rari Fuse	Collateral markets in pools 46 and 149	sETH2 price and cash getters fail; pool 149's FEI cash is zero
Legacy Silo	Registered sETH2 market	Paused, no selected price provider, no copied-WETH cash

The Uniswap V3 sETH2/copied-WETH pools hold approximately **2,049.41** and **588.57** WETH units on the other side. On PulseChain, copied WETH wraps native PLS; those amounts are not verified Ethereum ETH backing. Two PulseX pairs hold approximately **21.20** and **2.20** local stETH. Counter-token balances are inventory ceilings, not immediately executable output quotes.

Balancer's other links are mainly indirect: the checked Vault has five stETH pools, sixty wstETH pools and seven FEI pools, but no current rETH2 membership among the selected token candidates. The one sETH2 pool is 0x063c624672e390363b25f0c6c68ad9067c34595b. wsETH2's sole component is sETH2 at one unit per basket unit; its manager is 0x6654c5561662f2e229c61cc4671be92336c496b9. Changing basket components requires that basket's permissions and does not automatically change another protocol's collateral policy.

The scoped Fuse census covered **202 directory pools and 789 unique markets**. It found sETH2 in two pools, stETH in five and wstETH in nine, with no exact rETH2 underlying. A directory entry named StakeWise with no markets was not counted as a listing.

The [StakeWise integration proposal](#) explicitly describes depositing sETH2 to borrow FEI in Fuse pool 149. Current getters confirm that market at 0x18f49849d20bc04059fe9d775df9a38cd1f5ec9f, with 75% collateral factor. Its comptroller is 0x5f0ae8b07d1a82be7f557ef93551232749a88ca1. Borrowing sETH2 itself is paused; that flag alone does not prohibit depositing sETH2 to borrow a different asset. The separate FEI market, 0xb1eada44fc85607f837626930afb621c33d0ef2d, has zero cash. A FEI loan would remain debt.

The FEI/3CRV pool 0x06cb22615ba53e60d67bf6c341a0fd5e718e1655 holds about **46,870 FEI** and **1,509,469 3CRV shares** internally. Its underlying quote returns approximately **0.999255 pDAI per FEI** for a one-FEI input. This connects FEI to the proposed stablecoin network, subject to actual execution and liquidity. A 3CRV share is not automatically one dollar.

Lido can supply local stETH without governance control

The current Lido implementation exactly matches its inherited pre-V2 Ethereum implementation. It accepts native PLS deposits and issues local stETH against its native-unit accounting; it does not accept arbitrary sETH2 or an L2 stablecoin through `submit`.

The sampled recent 200-block interval contains **199 receipt-verified deposits totalling 4,687.5 PLS**. The stake limit refills at **23.4375 PLS per block**, up to 150,000 PLS; at the pinned block its available allowance is zero. Thus local issuance continues, but its deposit rate is bounded. Its inherited beacon accounting is not evidence of Ethereum withdrawal rights.

One wstETH represents approximately **1.1232654 stETH**. That wrapper cannot simply substitute sETH2 for its immutable stETH input. Local Curve quotes can differ greatly from inherited accounting: the main PLS/stETH pool quotes about **244.51 PLS per stETH** for one stETH. This suggests a local arbitrage branch to investigate; it does not establish unlimited profit or external-dollar value. Deposit allowance, fees, price impact, inventory and actual transfers govern realized results.

Lido is governable through its inherited LDO/Aragon voting stack. Selected permission managers and implementation addresses match the fork, and the selected post-fork governance history contains no changes or new votes. These checks do not prove group control. They also do not make Lido immutable: sufficient valid voting power could change permissions and implementations.

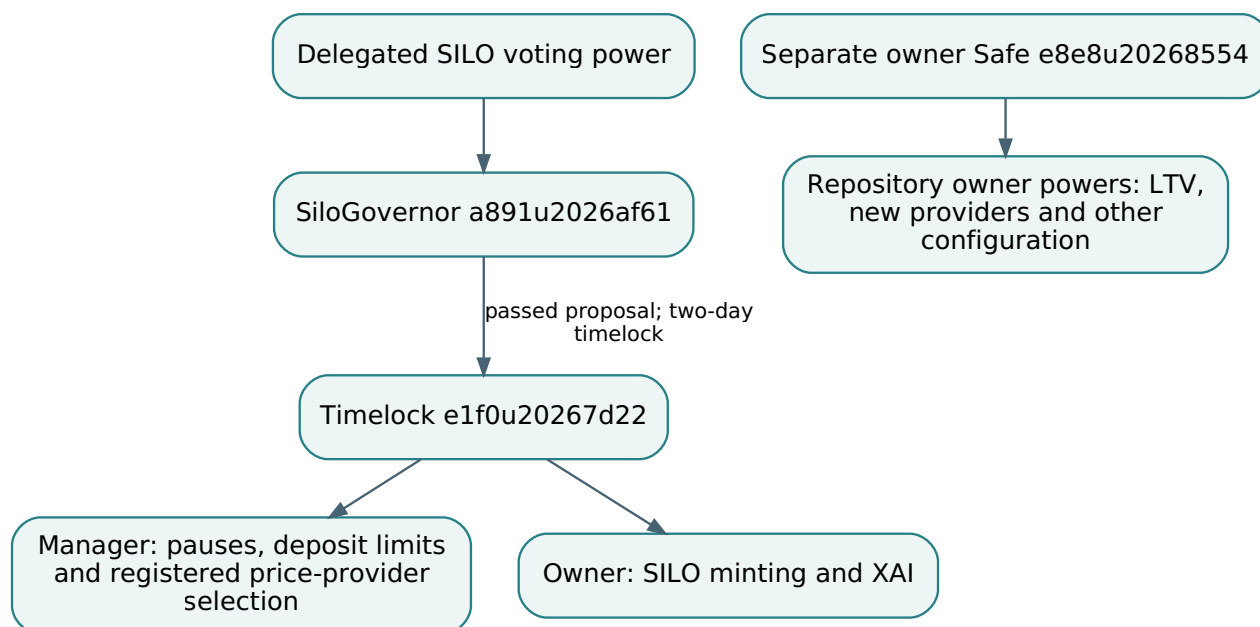
Aave V3 lists wstETH, with ordinary collateral LTV **68.5%** and a **200,000-wstETH supply cap**. Exact sETH2 and stETH are not listed there. Its DAI/USDC/USDT receipt contracts hold zero underlying cash at the checkpoint, but its copied-WETH receipt holds about **1,649,712.62 native-wrapper units**. This is a funded local-asset candidate; successful borrowing still requires all account, price and reserve checks. Inherited dollar oracle values do not establish dollar convertibility.

Compound III's WETH market lists wstETH, but has zero base-WETH cash at the checkpoint. Compound V2's enumerated markets do not list these exact three assets. Current Aave V2 and legacy Euler getter failures leave their current operation unresolved; failed calls were not treated as proof of no listing. The selected Reserve eUSD registry accepts none of these three exact assets. This does not assess every Reserve basket.

The [1inch StakeWise campaign](#) concerned SWISE/1INCH liquidity. It is not evidence that 1inch provides an sETH2 collateral market or that its Ethereum aggregation service executes these PulseChain routes.

Silo control can pass through voting rather than the owner Safe

The actual inherited repository is `0xd998c35b7900b344bbe6555cc11576942cf309d`. Two later Ethereum deployment addresses sampled have no PulseChain code; their absence does not disprove other deployments. Verified runtime matches identify the following legacy authority path:



The governor is `0xa89163f7b2d68a8fba6ca36beed32bd4f3eeaf61`; its timelock is `0xe1f03b7b0ebf84e9b9f62a1db40f1efb8faa7d22`. The timelock manages both the repository and price-provider repository. It owns **SILO** (`0x6f80310ca7f2c654691d1383149fa1a57d8ab1f8`) and **XAI** (`0xd7c9f0e536dc865ae858b0c0453fe76d13c3beac`). SILO's verified source contains an owner-only mint function. Consequently, an initially successful governor proposal could mint more voting tokens and strengthen subsequent voting control. That is a concrete potential control path, not evidence that it occurred.

Current parameters are a 100,000-SILO proposal threshold, one-block voting delay, 21,000-block voting period and 1% quorum: **10 million votes at the present billion-token supply**. The timelock adds **two days**. At ten seconds per block, voting plus timelock takes roughly **4.43 days**, before execution; block-based voting time varies. Minting more SILO also increases later quorum because quorum uses historical total supply. Votes must be delegated at the relevant snapshot; balances alone are insufficient.

The full timelock role/scheduling history from block zero identifies only the governor as remaining proposer and executor, with the timelock itself as remaining admin. No selected timelock schedule or role event occurs after the fork. The repository owner Safe remains 3-of-5 and has no enabled modules in its current paginated listing. Those findings do not identify the holders of its private keys.

The observed [July 2024 proposal](#) tried to transfer six ownership/manager positions to `0x48fc8b22ae5319786a8e25ef49b1b49942000000`. Its current state is **Defeated**: approximately **9.703 million affirmative votes**, below the ten-million quorum, with no opposition. No post-fork governor execution or selected timelock execution was returned in the completed history. The proposal's owner-only repository calls also require the separate owner's authorization; control of manager permissions alone would not authorize them.

The proposer later transferred its SILO away and currently has zero votes. The sampled research wallets hold about **6.274 million SILO at 1967**, undelegated, and **2.377 million at e121**, self-delegated. Their combined balances are below quorum; they are not a complete census of a possible coordinated group. Neither those balances nor the defeated proposal establishes a completed takeover. A successful execution, changed authority, sufficient attributable snapshot voting power, or another governed deployment could establish additional control.

The registered sETH2 Silo is `0x76f7e7b28aee365ad89a780a911a373ad168e4fc`, with 50% collateral LTV. It is paused, its selected price provider is zero, and its copied-WETH liquidity is zero. Governance control could remove manager-side obstacles, but a new price provider's registration and LTV changes are owner-side powers. The wstETH Silo `0x4f5717f1efdec78a960f08871903b394e7ea95ed` is unpaused and has WETH/XAI liquidity, but its current price getter fails. Listing is not sufficient for a usable loan.

What the four supplied transactions establish

Transaction	Verified action	Consequence
30da...2285	On 23 August, e121 executes a settled optimistic proposal through module <code>0x390c57f782e7c31e9cac2fc4fd459bd0bb901c6d</code> ; the copied ShapeShift Safe calls the module's ownership transfer to e121	Completed module ownership transfer using Safe module authority
83ae...a9c0	e121 commits a UMA VotingV2 vote for Admin 195; ancillary text requests emergency-proposer ownership transfer to e121	A vote commitment, not an executed ownership transfer; current exact request has no resolved price
40a9...ecb4	e121 transfers the optimistic module back to the copied ShapeShift Safe	Reverses that module ownership change in this transaction
cb53...42ee	<code>ca509...dee0</code> sets epoch-eight Merkle root on distributor <code>0x1a2e39b54f153dc3582615fbb8544a3bc22d94f</code>	Updates native-PLS reward entitlements; no token mint, bridge relay or Silo authority call

The ShapeShift treasury address is `0x90a48d5cf7343b08da12e067680b4c6dbfe551be`, identified in [ShapeShift's own governance forum](#). On PulseChain it currently has the optimistic module enabled. A subsequent ownership event at block **27,378,842** transfers the module to e121 again, consistent with its current owner getter. Subsequent successful liveness changes include a ten-second window; the current setting is **1,000,001 seconds**, about 11.57 days. Thus the group demonstrably exercised configuration authority over this module. Its owner can change the rules and dispute window, while Safe calls still follow the proposal/assertion execution path. These transactions do not demonstrate that the same module controls Silo.

UMA's emergency-proposer owner still reads GovernorV2, `0x7b292034084a41b9d441b71b6e3557edd0463fa8`. The exact Admin 195 request's `hasPrice` is false; `getPrice` reverts with "Current voting round not ended." The vote commitment does not reveal the vote choice or prove the requested transfer completed.

The rewards distributor's current owner and whitelisted root setter are `0xca50938949a3ccf3960537a01bda9973d6c8dee0`. It lists 43 pool addresses and holds about **245.16 million native PLS**. A Merkle root commits to a claims list; it is not proof of an external-chain asset balance. Adding a pool to this distributor's list does not give it that pool's administrative rights or prove fees were collected from it. Beneficiary amounts and the origin of the funds were not reconstructed here.

Evidence and reproduction

Additional basket and vault evidence is recorded in the [StakeWise vault and Index Coop study \(Appendix 64\)](#). dsETH directly contains sETH2 and has an installed rETH2-to-sETH2 reinvestment extension. Opium supplies an options-collateral connection, while a deployed Babylon strategy joins native staking with those two tokens' exchange routes. The original StakeWise reward oracle and distributor share the controlled StakeWise administrator. dsETH's separate Safe and Orca membership authority match inherited snapshots; group control of that basket remains unproved.

Evidence is saved under `artifacts/pulsechain-lido-lending-20261002/`; explicit batches are under `Research/experiments/`. Getter snapshots preserve block/hash metadata, requests and responses. Event batches preserve the full queried ranges and verify selected logs against successful receipts and block headers. The original Silo price-repository history had eight transport gaps; `silo-governance-retry.json` closes those exact ranges with no errors. Negative conclusions are limited to the selected contracts, signatures, ranges and RPC completeness.

Use `Research/scripts/probe_pulsechain_routes.py --block 0x1a68645 --calls <manifest> --output <new-file>` for the explicit getter batches and `Research/scripts/collect_pulsechain_events.py --manifest <manifest> --output <new-file>` for histories. Existing evidence files are frozen; reproductions require new output names. Both commands retain the chain and pinned endpoint hash. The Balancer census filters exact ABI token words before receipt verification and then checks current Vault membership; excluded logs are retained in raw responses. The unfiltered preliminary Balancer attempt was interrupted and is not a completed evidence run.

The research suite passed **76 tests and 82 subtests** after collector changes. `python Research/scripts/verify_pulsechain_stakewise_lending.py --evidence artifacts/pulsechain-lido-lending-20261002` passed **92 offline evidence checks**, including the four transaction/header/receipt/trace relationships, linked authority facts, listing counts, and completed governance histories. Its output and hashes of 348 evidence files are retained in `verification.json` and `sha256.json`. These checks establish internal consistency; they do not remove trust in the RPC or explorer's verification.

Appendix 40: Fifty-million-pDAI execution and complete reinvestment

Contents of this appendix

Specification	281
Results	281
Liquidity growth and fee recycling	282
Slippage and holding through a re-peg	282
Reproduction and checks	283

Source record: docs/studies/analysis/pulsechain_liquidity_growth_20260930.md.

Fifty-million-pDAI execution and complete reinvestment

30 September 2026. Hypothetical capacity experiment; no live trading.

Specification

The system's proposed policy dedicates every acquired asset and attributable net earning to peg support. All acquired pDAI is deployed into compatible pDAI/U liquidity positions. Required U is added alongside it, and other inventory supports surrounding conversion markets, lending liquidity and operating costs. LP ownership remains usable by the controller unless a separate decision intentionally relinquishes withdrawal rights.

This policy does not require every asset to be deposited into every venue at once. Aave receipts, their wrappers and Balancer LP tokens can express nested claims on the same principal. Reserve accounting records that principal once. Adding pDAI alone increases sell-side availability but is not equivalent to adding the U that pays future pDAI sellers. Balanced or appropriately targeted deposits determine the resulting execution capacity.

The experiment tests a **single 50,000,000-pDAI sale** against liquidity present before the trade. U is initially valued at \$1; a separate \$0.80 scenario tests the difference between internal token parity and dollar execution. Constant product pools charge 0.3% input fees. The two-coin StableSwap model charges 0.04% output fees and defines $A_{nn} = 2A$. These A values are normalized model parameters, not a claim that an arbitrary deployed Curve getter uses the same precision or convention. There is no controller correction during the sale.

Results

Pool rule	Initially on each side	U received for 50m pDAI	Average U per pDAI	Subsequent 1,000-pDAI sale, U per pDAI
Constant product	100m	33,266,599.933267	0.665332	0.443552
Constant product	1bn	47,482,973.758156	0.949659	0.904437
StableSwap A=100	100m	49,655,400.141781	0.993108	0.982574

Pool rule	Initially on each side	U received for 50m pDAI	Average U per pDAI	Subsequent 1,000-pDAI sale, U per pDAI
StableSwap A=100	250m	49,877,212.688687	0.997544	0.995327
StableSwap A=100	500m	49,930,075.410206	0.998602	0.997585
StableSwap A=2000	100m	49,963,362.176383	0.999267	0.998714

A 50m trade is therefore compatible with tight execution under explicit depth, asset-value and pool-rule assumptions. At A=100, 100m per side satisfies an average 99-cent requirement but leaves the subsequent small sale below that band. At 250m per side both measurements exceed 99 cents. Average execution, post-trade marginal conditions and system response time are separate promises.

For a constant-product pool initially holding equal-value reserves R and fee f, the average output ratio for a sale q is $(1-f)R/[R+(1-f)q]$. Requiring at least r gives $R \geq q(1-f)r/[(1-f)-r]$, where $r < 1-f$. For q=50m, f=0.003 and r=0.99, this requires about **7.050214286 billion units per side**. This is an average-execution bound, not a sufficient post-trade quote bound.

With U worth \$0.80, the A=100, 100m-per-side case pays only about **\$39.724320m**, despite returning 49.655400m U. Amplification changes price impact around the assumed exchange relationship; it does not guarantee the external value of U.

Liquidity growth and fee recycling

A second schedule starts with 25m pDAI and 25m U, adding 25m of each per round. These are declared new inventory inputs obtained elsewhere by the system. After three additions, reserves reach 100m each; after eight, 225m each. Each checkpoint runs the same 50m sale independently. Checkpoints are not successive sales whose depleted balances are silently reset.

A separate recycling calculation starts after those eight additions. It grants 40,000 U of **settled outside fee assets**, an 80% system entitlement and 2,000 U of costs/reserves, leaving 30,000 U available. These fees come from outside the tested pool; they are not counted again on top of fees already inside its reserves.

The controller spends about **15,002.005618 U** buying **14,995.994916 pDAI** from the pool, then contributes every acquired pDAI and the remaining **14,997.994382 U** in a proportional LP join. All purchased pDAI is deployed. The final pool holds 225m pDAI and 225.030m U. Its net new pDAI is zero because the purchase and redeposit used the same pool; its net new U is 30,000. Buying pDAI elsewhere and depositing here would instead move pDAI between venues.

That example prevents double counting while demonstrating the intended policy: collected outside income becomes productive liquidity. In a partially owned pool, proportional joining creates new LP claims and swap fees accrue according to actual ownership. Fee entitlement at inaccessible LP addresses is not withdrawable system income. Borrowing produces a liability; gross swap turnover does not itself measure profit.

Slippage and holding through a re-peg

A sale creates a price difference that can reward a subsequent buyer. Immediate arbitrage closes against a different funded price. Buying discounted pDAI and holding until recovery takes price and timing exposure. The system may choose that position because its objective spans many transactions. Realized gains still require a closing sale or a clearly stated valuation basis.

The returning price is financed through actual trades, reserve allocation or outside demand. It does not retroactively improve the initiating seller's execution. Reinvesting profits can deepen future liquidity, but feeding the same assets repeatedly through controlled accounts does not create external earnings. This is why the proposal must measure retained net output and usable depth rather than the number of internal transactions.

Reproduction and checks

Run `Research/scripts/model_pulsechain_liquidity_growth.py` with `--manifest Research/experiments/pulsechain_liquidity_growth_20260930.json` and a fresh `--output` path. Saved results are in `artifacts/pulsechain-liquidity-expansion-20260930/growth-results.json`. The output records manifest/script hashes, 15 capacity cases, nine independent growth checkpoints, fee-recycling balances and the algebraic constant-product bound. The implementation uses Decimal precision 70 and a quadratic solution for the StableSwap output reserve. Tests compare it to a separately implemented integer iterative invariant and check the recycling conservation identities.

Appendix 41: Local pDAI-to-PLS exit evidence and publication boundaries

Contents of this appendix

Finding.....	284
Provenance and fixed endpoint	284
What actually moved.....	285
WPLS is a local native wrapper.....	286
What the endpoint reserves establish	286
Governance qualifications and theory boundaries	287
Reproduction and publication scope.....	287
Clarification: inherited WETH also wraps native PLS	287

Source record: docs/studies/analysis/pulsechain_local_pls_evidence_20260929.md.

Local pDAI-to-PLS exit evidence and publication boundaries

29 September 2026. Point 12 of the strengthened research: evidence boundaries, local exit observations and reproducibility. This study interprets existing transactions and performs fixed-block public reads. No transaction, governance action, state override or simulated state change was submitted.

Here, redemption means exchanging pDAI for PLS on PulseChain. It does not mean bridging out, redeeming an issuer-supported Ethereum stablecoin, or receiving external dollars. Source-bridge return rights remain relevant to claims about source backing, but are not a prerequisite for an ordinary funded local swap.

Finding

An exact local pDAI/WPLS pool and a successful historical pDAI-to-WPLS exchange are now identified. The larger transaction subsequently unwraps its combined WPLS proceeds into native PLS and pays the initiating wallet. This substantiates an actual local exit component, rather than only an illustrative AMM model.

The identified pDAI branch sells **782.592868262568217956 pDAI** for **264,588.142395152594479041 WPLS**. Its output joins proceeds from other branches; the transaction's larger native PLS payout must not be attributed solely to that pDAI input. No external PLS dollar reference was collected, so this observation establishes neither a one-dollar execution value nor future price support.

Provenance and fixed endpoint

The transaction was first found as a lead in the already saved explorer wallet list artifacts/parb-core-addresses-20260929/address_reads.json. Its calldata alone was not taken as execution evidence. This batch retrieved its receipt, historical trace and block, then reconciled actual token/native movements.

Item	Exact identity
Chain	PulseChain, chain ID 369
Copied DAI	0x6b175474e89094c44da98b954eedeac495271d0f
Wrapped native PLS	0xa1077a294dde1b09bb078844df40758a5d0f9a27
Observed pDAI/WPLS pool	0x947b4633e32e0c7f2c76753b43f008480715416d
Pool's endpoint factory getter	0x1715a3e4a142d8b698131108995174f37aeba10d
Historical transaction	0x520d978fba0438d37ac9048f0188bbb9174115f7f2a9abe103fab216a5e0d8ff
Historical block / UTC	22,524,531 / 23 January 2025, 18:49:45
Historical block hash	0x577eb850f3d8bb79385b60b3ef7b9ab487db2d71f3cb36b3ac9e4ec055c81fbe
Sender / final native recipient	0x6e3bf3c7bd91144d694621298cb8da370479a7bb
Principal audit endpoint	27,659,393, 28 September 2026
Endpoint block hash	0xc5e582cd902f4bff3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275

Primary references are the [historical transaction](#), [pool](#) and [WPLS contract](#). Saved RPC request/response records supply the fixed-block and historical detail; explorer pages are not substituted for them.

What actually moved

Within the larger aggregated trade, the execution contract `0x258e0bc6ed656ccb5147383dc02edeab141b6a9a` receives pDAI from a preceding branch and transfers exactly 782.592868262568217956 pDAI to the identified pool. The pool transfers 264,588.142395152594479041 WPLS back to that execution contract. Its Swap event reports the same amounts. Historical pool-reserve reads at the preceding block and the transaction block reconcile exactly with this pDAI increase and WPLS decrease.

Observed quantity	Amount
This branch's pDAI paid into pool	782.592868262568217956
This branch's WPLS output	264,588.142395152594479041
Branch gross WPLS per pDAI	Approximately 338.091686144
Combined WPLS unwrapped across all branches	500,058.645141336259878379
Native PLS paid to initiating wallet	500,032.149752620780691584
Difference paid elsewhere by the execution contract	26.495388715479186795 PLS
Whole transaction's gas cost	2,391.582282068822951736 PLS

The successful historical trace contains WPLS.withdraw, the corresponding native payment from WPLS to the execution contract, and subsequent native payment through `0x6bf228eb7f8ad948d37ded07e595efddfaaf88a6` to the initiating wallet. No failed frame was returned. The transaction's outer native value is zero. The whole transaction's payout less its gas cost is 497,640.567470551957739848 PLS. **That is neither pDAI-branch profit nor the net PLS return attributable solely to its pDAI input:** other assets supplied other branches, and branch-specific cost allocation was not established.

The pool's published PulseXPair source enforces an adjusted-balance invariant with 10000 and an input deduction of 29, corresponding to a 0.29% swap fee in that mechanism. This explains why a reserve ratio is not an execution quote. The recorded branch output already reflects its actual pool execution; no hypothetical fee-free output is substituted for it.

A second saved wallet lead was checked as a direction control: [transaction 0x05e0797a....](#) It includes WPLS-to-pDAI movements and subsequent routing, with no native PLS payment in its returned trace. It is not counted as another pDAI-to-PLS exit merely because both token addresses occur in its calldata.

WPLS is a local native wrapper

The [verified WPLS source response](#) identifies contract WPLS, symbol WPLS, 18 decimals. Deposit credits the caller's WPLS balance by the native value deposited. Withdrawal checks the caller's WPLS balance, subtracts the requested units, and transfers the same amount of native currency to that caller. It has no bridge step or Ethereum-dollar redemption requirement. The historical trace supplies a successful concrete withdrawal in addition to this source-defined behavior.

One raw WPLS unit corresponds to one native PLS wei in these deposit/withdraw operations. Gas and the caller's ability to receive the native payment remain execution considerations. A claim about WPLS equivalence should not silently include transaction fees or guarantee that every receiving contract accepts a native transfer.

WPLS runtime bytes match between the observed trade block and the audit endpoint (2,058 bytes), as do the pool runtime bytes (11,357 bytes). The explorer provides verified source; this study did not independently recompile either contract. The WPLS source's bare receive function can accept native donations without crediting a depositor's WPLS account. Its totalSupply getter reports native contract balance. That getter alone should therefore not be used as a census of all holders' spendable balances.

What the endpoint reserves establish

At the pinned endpoint, token0 and token1 getters identify the exact copied DAI and WPLS addresses above. The pool reports:

Pool reserve	Amount
Copied DAI	18,087,077.898954788899607139
WPLS	3,443,662,680.177700347668184819

These are one pool's reported reserves at a historical fixed block, not a live quote, a network-wide liquidity total or a promise that all that WPLS can be extracted for a finite input. The source enforces output below the relevant reserve and an invariant. A specified-size local exit must account for input, fees, price impact, routing, actual execution state, unwrapping and gas. This batch does not establish a current best route or estimate a dollar price.

This local pool is separate from the earlier sampled Aave linear pools. The small waDAI linear-pool output headroom is a limitation on that receipt-to-DAI leg, not proof that the entire local pDAI-to-WPLS market lacks liquidity. Conversely, observing this funded exit pool does not establish funding of the proposed source receipt or Portal leg.

Governance qualifications and theory boundaries

The local exit observation does not require accepting the claimed common human ownership of bridges, issuers or applications. The historical receipt identifies addresses and actions; it establishes neither who owns the keys nor their plan. Existing audits establish particular Aave v2, stkAAVE, bridge ProxyAdmin and Across control actions. They do not establish every Aave v3 role, all UMA control, Reserve voting power or the ability to change unrelated exact-token issuer permissions. Those bounds remain as recorded in the control studies.

For a future theoretical configuration, effective governance might change some caps, integrations or inventory arrangements. That is different from observing that they changed. The point-12 evidence separates three things: a real 2025 local exit branch; 2026 endpoint pool state; and a hypothetical source-financed system that could use a compatible local exit. None is silently substituted for another. the local-redemption definition removes any need to prove an Ethereum off-ramp for this exit; it does not remove the need to show enough PLS for the stated quantity and price objective.

Reproduction and publication scope

New records are preserved under ignored `artifacts/pulsechain-local-pls-evidence-20260929/`. They include both historical receipts/traces/blocks, endpoint and historical code reads, token identity and reserve getters, published pair/WPLS source responses, exact reconciliation, SHA-256 manifest and a publication file inventory. Requests retain RPC methods, parameters and block tags. Existing maintained read-only collectors are `Research/scripts/probe_pulsechain_routes.py` and `Research/scripts/collect_pulsechain_events.py`.

A manageable publication package has two useful scopes:

- **Local evidence supplement:** approximately 0.43 MB of raw records, or 0.07 MB compressed, covering this local exit and wrapper/pool observations.
- **Reports plus selected representative evidence:** approximately 1.7 MB of reports, local records, representative ARB/GRT mint records, GRT reconciliation, withdrawal supply/coverage records, Aave reserve observations, mathematical example and read-only collectors; approximately 0.44 MB compressed.

Exact file paths, byte counts and hashes are in `publication_inventory.json`; ZIP sizes were measured using deflate level 6. The selection is deliberately **not the complete audit archive**. It supports named representative claims and does not include every source, governance record, Curve census, trace or earlier experiment. A full reproduction of every companion claim requires its separately identified batches. Report text and manifests are much smaller than the full raw catalogue; packaging one must not be described as packaging the other.

The evidence depends on the selected public RPC and related explorer. Matching hashes preserve file identity; they do not independently authenticate the node. No independent header/receipt-root validation or second-node consensus check was performed. A publication should freeze the reports and selected files together, state the audit endpoint and selected scope, and retain links to the precise primary contract and transaction records. Old state and a successful historical exit remain useful evidence without being advertised as a current quote.

Clarification: inherited WETH also wraps native PLS

A separate read-only batch addresses the additional local-exit premise. For the **exact inherited WETH contract** `0xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2` on PulseChain, the proposed native-unit equivalence is supported by its code:

```
1 inherited WETH unit ↔ 1 native PLS unit ↔ 1 canonical WPLS unit,
```

before transaction costs and subject to the relevant transfer/withdrawal calls succeeding. This is mechanical conversion of native units, not an assertion that three different contract identities are interchangeable inputs to a pool.

The [verified inherited WETH source](#) identifies WETH9, with symbol WETH and 18 decimals. Its deposit credits the caller's token balance by `msg.value`. Its withdrawal checks and subtracts the caller's wrapped balance and transfers the same native amount to that caller. On chain 369 the transferred native currency is PLS. The inherited token's name "Wrapped Ether" does not make that local native payment Ethereum ETH.

At the same audit block 27,659,393 and block hash stated above:

Observation	Result
Native PLS held by inherited WETH	33,829,502,874.176104966393612215
Its reported totalSupply, in 18-decimal units	33,829,502,874.176104966393612215
Endpoint runtime size	3,124 bytes
Runtime equality with inherited block 17,232,999	Exact byte equality

The equality between native balance and reported totalSupply is expected because this WETH9 implementation defines totalSupply as its own native balance. It is not an independent enumeration of holder balances. The read demonstrates native assets held in the wrapper; a holder still needs the appropriate wrapped balance and a recipient capable of receiving the native transfer. This batch did not submit or simulate a withdrawal. The source was explorer-verified rather than independently recompiled here.

For theoretical local exits, a route paying this inherited WETH can therefore be evaluated in PLS units through its native withdrawal mechanism; canonical WPLS has its separately verified native wrapper mechanism. A market may still quote them differently because of execution costs, route-specific liquidity or other constraints. Exact receiving-token addresses continue to matter.

Bridged external ETH representations are outside this equivalence. A token at another address that represents ETH held on another chain is not established as a native PLS wrapper merely by using the label WETH. Its own asset and redemption rules require separate evidence. This finding covers native PLS, canonical WPLS at `0xa107...9a27`, and inherited WETH at `0xc02a...6cc2` only; it does not claim every WETH ticker on PulseChain is the same asset.

Wrapped balances and the native currency backing them are one allocation of assets. A model can normalize these three forms into PLS units when conversion is feasible, but must not add a holder's WETH claim and the corresponding PLS inside WETH9 as two independent spending balances. The same qualification applies to WPLS. Native-unit convertibility does not establish a pDAI dollar price or automatically increase the funded output inventory of a particular pool.

Fresh requests, source response, code comparisons, exact balance summary and SHA-256 manifest are retained under `artifacts/pulsechain-inherited-weth-native-20260929/`. The earlier local-exit batch and its manifests were left unchanged. Its publication inventory predates this append; a final publication should include this fresh batch and freeze the updated report separately. No governance or permission analysis was performed for this clarification.

Appendix 42: Local PLS financing, settlement and bounded pDAI support

Contents of this appendix

Resolutions supplied by this study	289
The relevant assets and claims	290
Funded borrowing: the investor retains a pledged claim and a debt	290
Local issuance: an accepted claim becomes the issuer's pool coin	291
Portal: fully backing issuance consumes the purchased pDAI	292
Ordinary debt: two different forms of repayment	292
Across-style delivery: source repayment changes the relayer's inventory	293
Per-step liquidity and consolidated capacity	294
Fee-inclusive local execution model	294
Bounded sequential support and variable PLS value	295
Local exits versus financing-system redemptions	296
Independent numerical cross-check of the six scenarios	296
Extension: local WPLS and forked-WETH exits, with appreciation assumed	296
Independent check: selling retained pDAI after assumed appreciation	297

Source record: docs/studies/analysis/pulsechain_local_pls_financing_20260929.md.

Local PLS financing, settlement and bounded pDAI support

29 September 2026. Accounting companion to the conditional PulseChain report. This is an illustrative model, not a measurement of current markets, a record of execution, or an administrative action plan. The holder's contemplated exit is **selling pDAI for PLS on PulseChain**. Neither Ethereum redemption nor a bridge-out transaction is required for that exit.

Resolutions supplied by this study

Review point	Resolution
1 — Meaning of backing	Distinguish custody coverage, source solvency, redemption timing, funded local exchange and the holder's local PLS exit. None is silently substituted for another.
2 — Origin of financing	Complete illustrative borrowing, issuer and relayer ledgers identify the participant whose existing local assets fund the purchase, plus the asset or claim received in exchange.
6 — Portal and ordinary debt	A fully settled Portal example requires a specific top-up and retains receipt liabilities; ordinary debt repayment is reconciled separately. Neither payment automatically performs the other.
8 — Intermediate cash	A per-asset, per-step inequality requires cash or committed drawable credit before each payment, even when terminal net assets are positive.

Review point	Resolution
9 — Shared capital	Budgets are exclusive allocations of spendable inventory. Shared collateral, LP reserves and relayer cash are consolidated once.
10 — Assets returning to markets	A holder exit, aToken redemption, collateral withdrawal and bridge-receipt return are distinct modeled flows. Local pDAI exits require none of the latter operations.
11 — Bounded support	A fee-inclusive recurrence supports specified local PLS execution for a finite seller sequence and defined reference trajectory, conditional on an explicit reserve budget.

These resolve the conceptual/accounting objections within stated models. They do not turn illustrative funding or configurable interfaces into historical facts about the named wallets.

The relevant assets and claims

Let $\mathbf{r}$ be a transferable source investment receipt and $\mathbf{x}$ its exclusive PulseChain representation. The source investment remains invested when $\mathbf{r}$ changes custodian. Holding $\mathbf{x}$ is not a second independent ownership of $\mathbf{r}$ in addition to an unrestricted source claim held by the depositor.

A source receipt's unit coverage concerns whether its claimed assets or shares exist. Solvency concerns the realizable assets relative to obligations. Liquidity concerns when payments can occur. Local purchasing capacity concerns whether an exact accepted asset receives actual pDAI, PLS or an intermediate token from a funded counterparty. The pDAI holder's exit is a separate local swap.

For a finite transaction, technical acceptance and funded output can suffice even if the incoming claim has weak economic recovery. The accepting market, lender or issuer then retains that exposure. This may change future quotes or create losses, but it is not a reason to declare the initial local purchase mechanically impossible. Conversely, accurate custody alone does not supply the market's PLS output.

The ledgers below use illustrative token counts. **No unit of $\mathbf{R}$, $\mathbf{X}$ or a newly issued token is assumed to equal one dollar.** Figures such as 700 PLS are stipulated exchange amounts, not observed prices. Transaction and operating costs are zero in these small ledgers or paid from separately allocated funds; the quantitative price model below explicitly includes AMM trading fees.

Funded borrowing: the investor retains a pledged claim and a debt

Assume source custody already holds 100 $\mathbf{R}$ and 100 exclusive $\mathbf{X}$ are held by the investor. The lender accepts $\mathbf{X}$ as collateral and has 1,000 PLS of spendable local inventory. It lends 700 PLS against the 100 $\mathbf{X}$. Let $d(700)$ denote the actual pDAI output from spending that 700 PLS through a specified local market.

Participant	Before	After collateral placement, loan and purchase
Source investment	Assets underlying 100 $\mathbf{R}$ remain invested	Unchanged investment, ignoring yield/loss
Source bridge custodian	100 $\mathbf{R}$ allocated to outstanding $\mathbf{X}$	Same 100 $\mathbf{R}$ allocation
Investor/borrower	100 unpledged $\mathbf{X}$; no loan or PLS proceeds	$d(700)$ pDAI; beneficial interest in 100 pledged $\mathbf{X}$; owes 700 PLS principal plus agreed accrual

Participant	Before	After collateral placement, loan and purchase
Lender	1,000 PLS; no loan receivable	300 PLS; 700 PLS loan receivable; holds/enforces pledge of 100 X
pDAI/PLS market	Its initial inventory	Receives 700 PLS and pays d(700) pDAI

The lender's loan receivable and pledged X are not two independent 700-PLS assets. The pledge secures collection of that same debt. The borrower cannot simultaneously treat pledged X as available unencumbered collateral elsewhere unless an explicitly permitted subordinate structure is separately accounted.

This ledger establishes a financed purchase during the loan term. It is not necessary to repay the future principal before spending the borrowed proceeds. For an illustrative maturity of 707 PLS, a terminal scenario can stipulate a funded local sale of some acquired pDAI producing at least 707 PLS; payment of 707 discharges the debt and releases the X pledge. That sale needs its own executable local quote and brings pDAI back to market. An alternative is separately funded cash service while retaining the pDAI. Without either funding source, the loan remains outstanding or enters its agreed default/liquidation process; its accounting cannot be erased by declaring the original collateral “backed.”

No source receipt redemption or Ethereum exit is necessary for the local PLS sale or debt payment. The collateral's exit rights still affect the lender's exposure and hence sustainable credit terms.

Local issuance: an accepted claim becomes the issuer's pool coin

Assume a compatible illustrative issuer accepts all 100 X as the required collateral for issuing 100 Y. This is a stipulated single-collateral model, not a claim that the audited multi-asset Reserve basket accepts X alone. Assume a separate funded market buys the 100 Y for 700 PLS. The investor spends those PLS for d(700) pDAI.

Participant	Before	After issuance, Y sale and pDAI purchase
Source custodian	100 R allocated to X	Same allocation
Investor	100 X	d(700) pDAI; no unrestricted X or Y claim retained
Issuer	No collateral or Y obligations for this position	Holds 100 X; owes the redemption rights of 100 outstanding Y under its stated rules
Y/PLS market	At least 700 available PLS for the modeled exchange	Holds 100 Y; its PLS falls by 700
pDAI/PLS market	Its starting inventory	PLS rises by 700; pDAI falls by d(700)

The market now holds Y, backed in this model by X, which represents source R. This is one claim chain, not three independently spendable capital allocations. A later redemption retires Y against the corresponding collateral entitlement; it cannot both retire the claim and leave the redeemer with an independent extra Y claim. Precise basket, queue and redemption rules remain issuer-specific.

The local holder of acquired pDAI can sell it directly for PLS without redeeming Y or X. The Y market's inventory and issuer obligations nevertheless matter to repeat issuance. More issuance can finance more purchases only while its actual output market supplies the required local buying assets.

Portal: fully backing issuance consumes the purchased pDAI

Consider a permitted DAI-reserve issuance with 100 pDAI-equivalent local aToken claims and a compatible wrapper. The wrapper's own deposit limits must pass at the post-mint state; token compatibility alone does not imply deposit availability. Assume the wrapper sale pays 99.9 pDAI, actual Portal backing fee is zero, and the actor initially contributes 0.1 pDAI. The market starts with 200 pDAI available for the sale. Existing reserve claims and debt are held fixed; the table records changes for the incremental cycle.

Participant/account	Initial incremental position	After mint, wrapper sale and full backing
Operator	0.1 pDAI; no new receipt	0 pDAI; no receipt retained
Wrapper market	200 pDAI allocated to the modeled trade	100.1 pDAI plus wrapper claims representing 100 new aToken principal
Wrapper	No new aTokens or shares	Holds 100 new aToken principal; matching wrapper shares held by the market
Aave reserve cash	No cash from this cycle	+100 pDAI
Aave Portal unbacked amount	0	Rises to 100 at issuance, then returns to 0 at full backing
Aave receipt obligations	No new claims from this cycle	+100 aToken principal claims, held through the wrapper

Conservation is explicit: the initial 200.1 pDAI becomes 100.1 market pDAI plus 100 reserve pDAI. The operator supplies the 0.1 shortfall. No newly minted pDAI or source-chain asset is counted as reserve cash. The underlying can be moved from the market to Aave while the market accepts a redeemable local claim.

In general, issuance M , sale proceeds Q , and actual backing fee F require an extra $\max(0, M + F - Q)$ pDAI for full settlement. If $Q > M + F$, the surplus is actual remaining cash, conditional on that premium being obtainable. Fees credited to existing holders or protocol treasury must be added to their claim accounting rather than treated as unowned surplus.

This is a completed funded relocation of liquidity. It does not independently establish a dollar-reference price increase or an unlimited repeated trade. If eligible receipt holders redeem, the reserve's pDAI can return to their hands and possibly to market. The exact local wrapper can still be useful to a financing composition; its settlement is now explicitly financed instead of silently assumed.

The accounting follows the pinned implementation analysis in [Portal accounting \(Appendix 17\)](#). Whether its caps, roles and market conditions permit the example is a separate configuration question. No privileged action is proposed here.

Ordinary debt: two different forms of repayment

A 100-pDAI underlying repayment changes reserve cash by +100 and ordinary debt by -100. It does not reduce Portal unbacked issuance. A payer on behalf of another borrower does not acquire that borrower's collateral simply by paying. The borrower may become able to withdraw collateral under the applicable rules, which can increase assets available for sale.

A permitted 100-unit repayment with eligible aTokens instead reduces the borrower's debt and burns 100 units of the relevant aToken claim. It adds no underlying reserve cash. The pinned Pool implementation sets the aToken-funded repayment beneficiary to the caller. This is not permissionless acquisition of someone else's debt position through transferable debt tokens.

There is a valid conditional discount mechanism: a borrower whose eligible 100-unit debt can be discharged with 100 eligible aToken units may improve its balance sheet if it can acquire those claims for 90 underlying units, after costs. The difference comes from a discounted claim purchase and the pre-existing debt obligation. It is not 10 units of newly created reserve cash, and does not prove profit for a third party paying someone else's debt. Exact identity, transferability, conversion, debt eligibility and costs determine applicability.

The [pinned Portal/debt study \(Appendix 17\)](#) governs the historical interpretation; the current [Aave Pool documentation](#) corroborates the distinction between underlying and aToken-funded repayment, without replacing the pinned implementation with later protocol behavior.

Across-style delivery: source repayment changes the relayer's inventory

Use an illustrative **compatible configured pair** of source token Ts and local token TI. This does not assume arbitrary source receipts can be mapped into arbitrary existing local assets. Each actual route must satisfy its token mapping and settlement semantics. Fees are zero for this accounting example.

The participant starts with 100 Ts; a relayer starts with 100 TI locally; a separate local market has 700 PLS available to buy those 100 TI.

Participant	Before	After delivery, local trades and source repayment
Source user	100 Ts	No unrestricted Ts; holds $d(700)$ pDAI locally
Source settlement custody	No allocated deposit	Receives 100 Ts, then delivers them to the relayer; no remaining allocation for this settled deposit
Relayer	100 TI locally	100 Ts on source; local TI falls by 100; reimbursement claim is discharged
TI/PLS market	At least 700 available PLS	+100 TI, -700 PLS
pDAI/PLS market	Initial inventory	+700 PLS, $-d(700)$ pDAI

The source capital remains on its source chain, now owned by the relayer. It cannot also remain an unrestricted user asset. If Ts is an admissible investment receipt, its investment may remain invested while receipt ownership changes; that admissibility is a route condition, not a generic property of Across.

This resolves the “reverse replenishment” intuition as an inventory exchange: source capital can reimburse a party that already provided destination capital. Neither the relayer's local inventory nor the local market's PLS automatically replenishes. Opposite flows, additional funded relayers, purchases of local inventory or contributed liquidity can restore them. Those are genuine ways to sustain finite further deliveries, with costs and liabilities recorded.

The route can contain a local atomic delivery-and-trade operation where the configured application supports it. Its later cross-chain reimbursement is not part of that same atomic transaction. The historical architecture and current implementation limitations remain as documented in [Across models \(Appendix 7\)](#) and [HubPool accounting \(Appendix 14\)](#).

Per-step liquidity and consolidated capacity

For actor a , asset j and ordered step n , require:

$$\text{cash}[a,j,n] = \text{initial_cash}[a,j] + \text{cumulative_received}[a,j,n] - \text{cumulative_paid}[a,j,n] \geq 0.$$

Include committed credit only when it is actually drawable by that step and record the matching liability. A terminal surplus cannot finance an earlier unfunded payment. A remote receivable due next week does not count as today's local PLS unless an actual funder advances PLS against it.

To combine routes, allocate explicit slices of each initial pool/treasury balance. If the same 1,000 PLS funds both a relayer and a lender, their combined initial outlay cannot be 2,000 PLS. If earlier trades replenish later inventory, show the actual transfers and reprice subsequent trades at their new pool state. A receipt backing an issuer and securing a lender is one asset with an ordered claim structure, not two independent full-value reserves.

This leaves a positive result: several genuinely independent funded routes can contribute to one intervention budget, and restored balances can support later trades. The condition is conservation and time order, not a restriction to one protocol or a claim that liquidity can never be reused.

Fee-inclusive local execution model

Let a constant-product pool initially contain $x > 0$ pDAI and $y > 0$ PLS. Let $g = 1 - \text{buy_fee}$ and $h = 1 - \text{sale_fee}$, with both in $(0,1]$. Fees stay in the pool; there is no separate removed protocol fee. A gross PLS purchase input $b \geq 0$ leaves:

$$x_1 = x*y / (y + g*b) \text{ and } y_1 = y + b.$$

A subsequent holder sale of $q > 0$ pDAI receives:

$$z = y_1*h*q / (x_1 + h*q) \text{ PLS.}$$

For the disclosed PLS dollar reference $s > 0$, the holder's local execution value is $s*z/q$. No Ethereum redemption is involved. To target $T > 0$, solve:

$$A*b^2 + B*b + C = 0, \text{ where}$$

$$A = s*g$$

$$B = s*y*(1+g) - T*q*g$$

$$C = s*y^2 - T*q*y - T*x*y/h.$$

If the initial sale quote already meets the target, set $b=0$. Otherwise the required nonnegative input is:

$$b = (-B + \sqrt{B^2 - 4*A*C}) / (2*A).$$

This follows by substituting the post-buy reserves into $s*z/q=T$; it accounts for the buy fee retained in the pool rather than incorrectly keeping xy constant after a fee-paying trade. Actual fixed-point implementations need rounding and integer-unit checks beyond these continuous equations.

With $x=1,000,000$, $y=1,000,000,000$, $s=0.0001$, $q=1,000$, $T=1$ and $g=h=0.997$, an independent 80-digit calculation gives:

- Gross initial support purchase: **2,175,295,195.262100018333 PLS**.
- Subsequent holder output: **10,000,000 PLS**, corresponding to the stipulated \$1 reference value per pDAI over that 1,000-pDAI sale.

- After that sale, the next support purchase for the next identical 1,000-pDAI sale is approximately **10,030,042.743282352925 PLS**.

These are invented market inputs, not a cost estimate for real PulseChain pools. They strengthen the previous fee-free example by exhibiting finite cost with nonzero trading fees. The explicit research batch is `Research/experiments/pulsechain_local_pls_support_20260929.json`, evaluated by `Research/scripts/model_local_pls_support.py`; outputs are retained under `ignored artifacts/pulsechain-local-pls-support-20260929/`. This derivation independently checks its formula.

Bounded sequential support and variable PLS value

A useful existence claim is: **a specified finite sequence of local holder sales can receive at least the target reference value when each is preceded by an adequately funded intervention at the modeled state**. It does not require Ethereum token redemption.

For sale event i , use current reserves (x_i, y_i) , gross sale q_i and the stipulated reference s_i . Calculate the nonnegative support purchase b_i above and then execute the mathematical state transitions:

```
x_plus = x_i*y_i / (y_i + g*b_i)
y_plus = y_i + b_i
z_i = y_plus*h*q_i / (x_plus + h*q_i)
x_(i+1) = x_plus + q_i
y_(i+1) = y_plus - z_i.
```

The support treasury pays b_i PLS and acquires $x_i - x_{plus}$ pDAI. External holders deliver q_i pDAI and receive z_i PLS. Each reserve update and each holder exit is local to PulseChain.

For treasury initial PLS R_0 , inflows I_i , compulsory outflows L_i , support inputs b_i and operating costs K_i , require at every prefix n :

$$R_0 + \sum_{i \leq n} (I_i - L_i - b_i - K_i) \geq 0.$$

Treasury pDAI inventory separately equals initial inventory plus its purchased pDAI less its own sales or commitments. Do not count external holder sales as sales of treasury inventory. A treasury sale above target can rebuild PLS only if actual inventory and market outputs permit it; its price impact belongs in the state recurrence.

For fixed s , the required target PLS per pDAI is fixed. For variable s_i , the target changes to T/s_i . The same pDAI/PLS ratio no longer promises a fixed dollar-reference value. A bounded stress scenario can specify a positive minimum PLS reference, a finite number of sales, maximum gross sale sizes, extra costs, debt service and allowed competing flows, then compute the associated budget. An arbitrary downward price shock or unbounded seller flow is outside that bounded claim.

The exact observed sequence model uses known q_i and s_i at each event. A stronger claim protecting any sale up to q_{max} would fund the pre-event quote for q_{max} , recompute after each actual sale and restrict intervening trades. A finite simulation is not proof of every possible event ordering. Support before one sale does not guarantee the same price continuously between events, nor prevent another trade from consuming the quote first. An execution priority or tolerated response delay belongs among the scenario assumptions.

Thus the theory can establish a finite, fee-paying **local PLS exit capacity** under explicit funding and timing. It does not need a claim that all pDAI has Ethereum backing. Its remaining economic burden is local inventory and the obligations of the parties that supplied it, which the ledgers now identify.

Local exits versus financing-system redemptions

A pDAI holder who sells to a local pool receives its PLS; they do not have to redeem an Aave receipt, return a source bridge token, or collect remote vault assets. Those other exit rights are still relevant to the financing parties: a lender may seek collateral, an issuer holder may redeem Y, or a wrapper holder may withdraw pDAI from Aave. Such flows can reduce financing availability or return additional tokens to markets. They are stress inputs to continued support, not additional technical requirements imposed on the pDAI holder's local exit.

This separation is the principal clarification: **a local price-support and exit mechanism can be theoretically complete without any Ethereum redemption mechanism**, while still accounting honestly for the claims and finite capital used to finance it.

Independent numerical cross-check of the six scenarios

A separate calculation recomputed every event in the six 30-event scenarios using 110-digit Decimal arithmetic and a 420-iteration bisection search for the required purchase, rather than reusing its quadratic solver or importing its command script. All 180 event rows matched the saved reserve, payout and support-input values within $3.2e-70$ PLS/token units. Separate cumulative pDAI and PLS conservation checks passed at every event.

Illustrative scenario	Result
Zero fee, additional 400 million PLS support reserve	All 30 exits meet target; 290 million additional PLS spent
0.3% fee, additional 400 million PLS support reserve	All 30 exits meet target; about 290,871,239.738011 additional PLS spent
0.3% fee, no further support reserve	First below-target exit is event 2
0.3% fee, additional 100 million PLS	First below-target exit is event 11
0.3% fee, reference halves at event 16, additional 400 million PLS	First below-target exit is event 16
0.3% fee, 2,000-pDAI sales, additional 400 million PLS	First below-target exit is event 21

Every scenario separately funds the initial price-setting purchase. The support reserve is additional funding, not that same initial purchase counted again. For the 1,000-pDAI sequence, the first seller uses the already prepared quote; 29 further buys prepare the remaining 29 exits. There is no restoration after the last modeled sale. An insufficient reserve triggers no partial support purchase under this particular policy; a different policy would have different outputs and would need a separate calculation.

These results establish a bounded mathematical local-exit witness and show where the same witness fails when its budget or reference assumptions change. They do not calibrate real pool reserves, seller behavior, gas costs, debt maturities, liquidity changes, competing trades, bytecode rounding or timing.

Extension: local WPLS and forked-WETH exits, with appreciation assumed

The completed [native-wrapper evidence study \(Appendix 41\)](#) now confirms the exact inherited WETH at `0xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2` withdraws native PLS. Together with canonical WPLS, these exact wrappers can be normalized one-for-one into PLS units before conversion costs. The generic label/valuation qualifications below concern unverified or different WETH-labelled assets; they do not negate this specific native-unit finding. Contract identities and available route liquidity remain distinct.

The holder's local exit can be denominated in PLS, WPLS or the **specified forked-WETH contract on PulseChain**. These are alternative local output assets; none requires Ethereum redemption. WPLS can be treated as one-for-one PLS only where its actual wrapping/redemption behavior and liquidity support that conversion. A forked-WETH label alone does not establish either Ethereum ETH value or one-for-one redemption into PLS. For that output, identify its exact contract, local swap inventory and its own valuation reference.

For any chosen local output asset j , let s_j be its disclosed dollar-reference value per unit. Receiving z_j units for q pDAI has reference execution value $s_j * z_j / q$. The target therefore requires τ / s_j output units per pDAI before accounting for trade-size effects and fees. PLS figures cannot be mechanically substituted for forked-WETH amounts without the relevant exchange rate and executable market path.

If appreciation is an explicit scenario assumption, a higher s_j lowers the number of output units needed for the same dollar-reference target. For example, a 1.5-fold increase changes τ / s_j to two thirds of its former level; a doubling halves it. A treasury holding that asset gains marked reference value, which can improve its modeled capacity to support subsequent exits. It receives no new token units from appreciation alone. Conversion depth, market inventory and obligations denominated in that asset still matter.

Conversely, if the pDAI/output-asset exchange ratio remains unchanged while the output asset appreciates, pDAI's dollar-reference price rises above its former level. A system specifically targeting one dollar would allow or induce the pDAI/output ratio to decrease. Selling available treasury pDAI into the market may then rebuild output-asset inventory, conditional on funded local demand and its actual price impact. A tighter dollar target and a permanently fixed token exchange ratio are different objectives.

The direction of the mechanical effect is clear: holding the pool state, sale size and other parameters fixed, increasing the reference raises the quoted reference value and weakly reduces the additional support purchase needed to meet a lower-bound target. It can eliminate that immediate purchase. It does not necessarily return the system exactly to one dollar: a lower-bound policy that buys only below target permits above-target quotes unless a separate inventory-constrained sell policy is modeled.

The claim that attaining the pDAI target **causes** PLS or forked WETH to appreciate is a network-demand hypothesis. It could be investigated through specified demand, liquidity and portfolio responses, but it is not implied by bridge accounting or the AMM equations. A model that assumes appreciation tests its consequences; it does not establish that causal feedback or predict the appreciation will occur. New appreciation scenarios must remain a separate explicit experiment batch so the previous six-scenario results stay frozen.

Independent check: selling retained pDAI after assumed appreciation

The same local model can include an inventory-constrained above-target sale. Use the saved `fee_funded` state immediately after its fifteenth holder sale: approximately 316,621.864633 pDAI and 3,165,715,793.709034 PLS in the pool, 259,579,401.553066 PLS remaining support treasury, and 698,378.135367 pDAI accumulated by the operator. This model treats that pDAI as unencumbered and available for sale. The reference shock is imposed; its occurrence is not predicted or derived from the earlier purchases.

For an operator sale of gross a pDAI, with fee factor h , the post-sale reserves are $x_1 = x + a$ and $y_1 = x * y / (x + h * a)$. Requiring the next holder's gross q sale to receive target average reference value τ yields:

$$h * a^2 + [x * (1 + h) + h^2 * q] * a + x * (x + h * q) - s * y * x * h / \tau = 0.$$

Use its nonnegative root only when the starting quote is above target and the operator owns enough free pDAI. PLS proceeds are $y * h * a / (x + h * a)$, transferred from pool inventory into the treasury. They are not newly created PLS.

Alternative reference shock	Operator pDAI sold	PLS returned to treasury	PLS treasury afterward	Next holder's output for 1,000 pDAI
1.5 times original reference	70,153.484465	572,788,016.391764	832,367,417.944829	6,666,666.666667 PLS
2 times original reference	130,138.650678	920,191,644.708543	1,179,771,046.261608	5,000,000 PLS

Each row starts independently from the same fifteenth-sale checkpoint; the two results are not cumulative or additive. Both satisfy the operator's inventory constraint, and the next holder receives \$1 reference value per pDAI under that row's assumed reference. The subsequent holder sale moves the price again; this is a checkpoint demonstration of the sell-above side of the policy, not continuous exact-price stabilization.

An independent 450-step bisection search at 110-digit precision agrees with both quadratic roots within $4e-104$; pDAI and PLS conservation pass at retained precision. The input checkpoint itself contains 80-digit values, so the extra arithmetic precision checks the formulas rather than adding state accuracy. The separate analytical-checkpoint batch preserves the original six-scenario experiment and its outputs.

Appendix 43: Finite pDAI exits into PLS: a fee-inclusive support model

Contents of this appendix

Result	299
Model and equations	299
Explicit experiment.....	300
Conservation and funding interpretation.....	301
Appreciation and the sell-above-target side	301
Reproduction and limits.....	302

Source record: docs/studies/analysis/pulsechain_local_pls_support_20260929.md.

Finite pDAI exits into PLS: a fee-inclusive support model

29 September 2026. Mathematical research only. The holder receives native PLS on PulseChain, or WPLS assumed exchangeable one-for-one with native PLS. No Ethereum exit, external-dollar redemption or cross-chain withdrawal is required by this model. The PLS dollar reference is only the unit used to measure the local result. Actual wrapper and historical trade evidence are separate from these illustrative reserves and trading fees.

Result

The conditional argument can be strengthened beyond a single price quote. In the explicit model below, an initial purchase plus a finite PLS support budget funds thirty successive local exits, each selling 1,000 pDAI for 10 million PLS. At the stipulated reference of \$0.0001 per PLS, each holder receives \$1 of locally valued PLS per pDAI. Both trade directions charge a 0.3% input fee retained in the pool. This is a constructed finite scenario, not an estimate of PulseChain market cost or a probability of success.

The same model also identifies failures when the budget is smaller, seller flow doubles, or the PLS reference falls. This makes the claim bounded and testable rather than assuming a permanent peg from one successful trade.

Model and equations

Let x and y be the pool's pDAI and PLS reserves. Let b be gross PLS spent buying pDAI; $g = 1 - \text{buy_fee}$. Actual post-purchase reserves are:

$$x1 = x \times y / (y + g \times b)$$

$$y1 = y + b.$$

The entire input remains in the pool; the fee-adjusted input determines output. Consequently, the product of actual reserves grows when a positive fee is retained. Reusing the fee-free constant-product terminal identity would be wrong. This is the conventional input-fee treatment described by the [Uniswap v2 primary white paper](#). The selected fee is illustrative, not a claim about PulseX, Curve or Balancer.

For a holder subsequently selling gross q pDAI, let $h = 1 - \text{sale_fee}$. The PLS output and dollar-reference execution value are:

$$z = y_1 \times h \times q / (x_1 + h \times q)$$

execution value = $s \times z / q$, where s is the PLS dollar reference.

For target τ , the sufficient initial input is the nonnegative solution of $A \times b^2 + B \times b + C = 0$, where:

$$\begin{aligned} A &= s \times g \\ B &= s \times y \times (1 + g) - \tau \times q \times g \\ C &= s \times y^2 - \tau \times q \times y - \tau \times x \times y / h \\ b &= (-B + \sqrt{B^2 - 4 \times A \times C}) / (2 \times A) \end{aligned}$$

Use $b = 0$ if the existing finite-sale quote already satisfies the target. The implementation uses equal buy/sale fees; the displayed derivation permits different fees. It uses 80-digit Decimal arithmetic, not EVM integer rounding.

Explicit experiment

Initial reserves are 1,000,000 pDAI and 1,000,000,000 PLS; initial reference is \$0.0001 per PLS. Target is \$1 of PLS per pDAI on the specified sale. The initial purchase establishes a quote for a 1,000-pDAI exit. It is separately funded; the support budget in the table is additional PLS inventory.

Before each of thirty seller events, the treasury computes the purchase needed for that event's trade size and reference. It executes that purchase only if its remaining PLS covers the full amount. Otherwise it makes no partial purchase, and the holder still sells at the resulting market quote. The policy is chosen to expose sufficiency and failure thresholds, not to optimize real execution.

No other trades, liquidity changes, debt payments or replenishment occur. Gas, wrapping and other transaction costs are funded separately and excluded. The model assumes timely intervention before each specified event; it does not prove a continuous price floor during arbitrary interleaved trading.

Scenario	Additional support PLS	Target exits out of 30	First below-target exit
No trading fee, 1,000 pDAI/ event, fixed reference	400,000,000	30	None
0.3% fees, 1,000 pDAI/ event, fixed reference	400,000,000	30	None
0.3% fees, no subsequent support	0	1	2
0.3% fees, smaller support budget	100,000,000	10	11
0.3% fees, PLS reference halves at event 16	400,000,000	15	16
0.3% fees, 2,000 pDAI/ event	400,000,000	20	21

For the 0.3%-fee, constant-reference, 1,000-pDAI scenario:

- Initial purchase: **2,175,295,195.262100 PLS**, receiving approximately **684,420.069032 pDAI**.
- Additional purchases before events 2-30: **290,871,239.738011 PLS**.

- Holders receive **300,000,000 PLS** in total for **30,000 pDAI**.
- Treasury support remaining: **109,128,760.261989 PLS**.
- Operator final pDAI inventory: approximately **713,333.206430 pDAI**.

There are 29 restoration purchases, because the initial purchase funds the first target quote. The calculation does not restore a fresh target quote after the thirtieth sale. The displayed remaining budget is unspent support inventory, not profit: substantial PLS was paid for pDAI, whose future sale is not modeled.

The fee-free control reproduces the earlier initial purchase of 2,167,281,613.012984 PLS and requires 290,000,000 additional PLS across the same 29 restoration purchases. Fees raise both acquisition and intervention costs.

Conservation and funding interpretation

At every event the experiment checks:

```
pool pDAI + operator pDAI
    = initial pool pDAI + cumulative seller pDAI input

pool PLS + remaining support PLS + cumulative holder PLS received
    = initial pool PLS + initial purchase PLS + initial support PLS
```

It also checks nonnegative cash and reserves, nondecreasing reserve product with retained fees, and achievement of the target whenever the calculated full intervention is funded. Local sellers exchange their pDAI for existing PLS. No bridge-out cash flow appears anywhere in these identities.

One of the paper's financing mechanisms could supply the initial purchase or support treasury. This model does not establish which deployed mechanism does so. Receipt sales, funded loans, eligible issuer trades or relayer delivery must supply **net spendable PLS**, or executable alternative purchasing assets, without counting settlement reserves twice. A loan-funded treasury also needs its debt maturity and repayment accounted for in any extended horizon.

A constant PLS/pDAI exchange ratio can satisfy a fixed PLS target even when the dollar reference falls. It cannot simultaneously preserve the same dollar-valued target without adjustment. The event-16 failure tests that distinction without requiring holders to sell PLS externally.

Appreciation and the sell-above-target side

A fresh batch, `pulsechain_local_pls_appreciation_20260929.json`, uses the same thirty-event model with a 1.5× or 2× reference increase at event 16. Both branches keep all thirty holder exits at or above the target while spending approximately 140,420,598.446934 additional support PLS. The buy-only policy stops buying when the quote exceeds the lower bound; it does not restore exact equality. Those successes therefore establish a local minimum execution value, not an exact peg after appreciation. The two branches share the same token flows because neither needs further buying within this finite horizon.

A separate sell-side checkpoint makes the two-sided intuition concrete. It starts after event 15 of the constant-reference, 0.3%-fee baseline, with 698,378.135367209278 pDAI owned by the operator and 259,579,401.553066 PLS of remaining support treasury. The operator is assumed to own that pDAI free of pledges or other commitments. There is no intervening trade or liquidity change.

Let a be the operator's gross pDAI sale, with input-fee factor h . Starting at this checkpoint's reserves x, y , the sale leaves:

$$x1 = x + a, y1 = x \times y / (x + h \times a).$$

For the next holder sale of q , restoring its execution value to τ requires:

$$(x + h \times a) \times (x + a + h \times q) = s \times h \times x \times y / T.$$

Solve the quadratic with coefficients:

$$\begin{aligned} A &= h \\ B &= x \times (1 + h) + h^2 \times q \\ C &= x \times (x + h \times q) - s \times h \times x \times y / T \\ a &= (-B + \sqrt{B^2 - 4 \times A \times C}) / (2 \times A) \end{aligned}$$

This branch applies when the existing holder quote is above target. The required sale must fit the operator's available pDAI. The operator receives $y \times h \times a / (x + h \times a)$ PLS from the pool; this changes ownership of existing PLS rather than creating native units.

Assumed reference	Operator pDAI sale	PLS received by operator	Next 1,000-pDAI holder's PLS output
\$0.00015 (+50%)	70,153.484465	572,788,016.391764	6,666,666.666667
\$0.0002 (+100%)	130,138.650678	920,191,644.708543	5,000,000

Both inventory checks pass; the next holder receives exactly \$1 of reference-valued PLS per pDAI. The operator's resulting PLS inventory is 832,367,417.944829 or 1,179,771,046.261608 respectively. These alternatives cannot be added together or treated as profits independent of the pDAI sold. The operator still retains approximately 628,224.650902 or 568,239.484689 pDAI.

The economic review independently confirmed both roots using 450 bisection iterations at 110-digit precision and checked conservation. This higher precision checks arithmetic, not additional precision in the original checkpoint. The checkpoint establishes one sell-above-target response, not stability under every future price path. No causal model here predicts that the peg raises PLS.

The separate manifest is `Research/experiments/pulsechain_local_pls_rebalance_20260929.json`; its source checkpoint and hash are embedded. Run the corresponding script with a new path:

```
python Research/scripts/model_local_pls_rebalance.py \
--manifest Research/experiments/pulsechain_local_pls_rebalance_20260929.json \
--output artifacts/pulsechain-local-pls-rebalance-reproduction
```

Recorded outputs are under `artifacts/pulsechain-local-pls-rebalance-20260929/`; the appreciation buy-only outputs are under `artifacts/pulsechain-local-pls-appreciation-20260929/`. Earlier batches and their script hashes remain unchanged.

Reproduction and limits

Run from the repository root with a fresh output directory:

```
python Research/scripts/model_local_pls_support.py \
--manifest Research/experiments/pulsechain_local_pls_support_20260929.json \
--output artifacts/pulsechain-local-pls-support-reproduction
```

The recorded batch is `artifacts/pulsechain-local-pls-support-20260929/`. It preserves inputs, per-event results and source/input/output hashes. It does not modify the older fee-free experiment. The script has no network access or signing capability. No chain simulation or transaction was performed.

This is a scenario result, not a statistical success rate. It does not cover all market venues, adversarial timing, new liquidity, native-PLS supply changes, LP exits, rebasing or fee-on-transfer tokens, oracle failures or liquidity-provider responses. The specified inventory, seller flow and reference path are the boundaries of the positive result. A stronger local-support claim must specify and evaluate those additional conditions rather than assume them away.

Appendix 44: Maker adapter state and the remaining pDAI routes

Contents of this appendix

Finding.....	304
Why a caged collateral adapter need not block exit.....	305
Route conclusions.....	305
Economic relevance	305
Verification and reproduction	306

Source record: docs/studies/analysis/pulsechain_maker_adapter_closure_20261004.md.

Maker adapter state and the remaining pDAI routes

4 October 2026. Getter and runtime-code reads only. No transactions, state overrides or execution simulations.

Finding

The USDC adapter ending 2199 is live at this checkpoint; Maker's Vat remains shut down.

Ordinary PSM conversion is therefore disabled. Existing savings-credit conversion and possible shutdown collateral settlement are separate routes with different rights and conditions.

The exact adapter is **AuthGemJoin5**, 0x0a59649758aa4d66e25f08dd01271e891fe52199. It is not DaiJoin, which is 0x9759a6ac90977b93b58547b4a71c78317f391a28.

At PulseChain chain 369, block **27,707,510**, hash

0x8a99d5ac2c73089e006d50765bbfaa5d92e6fb523fb6d5384a2c100794cfb63b, timestamp **4 October 2026, 01:39:15 UTC**:

Getter	Result and meaning
Vat.live()	0. Ordinary PSM operations encounter the shutdown check in Vat.frob.
USDC adapter live()	1. The adapter's own deposit-live gate is enabled; this does not bypass Vat shutdown.
DaiJoin.live()	1. Its exit-live gate remains enabled.
End.live()	0. Maker is in the shutdown regime; selected settlement functions have separate accounting conditions.
USDC adapter vat() / gem() / dec()	Exact inherited Vat, exact copied pUSDC and six decimals.
USDC adapter wards(PSM)	1. The PSM retains adapter deposit authorization; ordinary conversion still fails at Vat.
pDAI wards(DaiJoin)	1. DaiJoin retains token-mint authority.

Getter	Result and meaning
pUSDC held by the USDC adapter	113,692,361.975557 pUSDC. Physical custody is not every caller's redeemable balance.
Vat.gem(PSM-USDC-A, End)	0. No internal collateral is allocated to End at this location.

Why a caged collateral adapter need not block exit

The matched AuthGemJoin5 source checks `live == 1` in its authorized three-argument `join`. Its public `exit` has **no live check**: it debits the caller's internal collateral through `Vat.slip` and transfers the corresponding pUSDC. Consequently, a hypothetical adapter cage would block new deposits while retaining this exit path under its other conditions.

Those conditions include internal collateral credit, the adapter's Vat authorization, sufficient physical tokens and functioning token transfers. Exit does not make the entire adapter balance freely withdrawable. DaiJoin differs: its `exit` expressly requires its own `live == 1`, caller credit and the required Vat permission. The [public USDC adapter source](#) documents the design; the saved matched source and runtime checks establish the examined local implementation.

Route conclusions

Route	Conclusion
DOLA sale → pUSDC → ordinary PSM → newly minted pDAI	Historically executed; disabled by Vat shutdown at the fresh checkpoint, regardless of the adapter's live flag.
Owned sDAI/Pot credit → DaiJoin exit → pDAI	Historical post-shutdown execution is demonstrated. It converts existing credit and consumes the relevant claim; it is not unrestricted new credit or proof of continuing savings yield.
Process PSM collateral → End claim → USDC adapter exit	A source-supported conditional settlement proposal. Allocation and actual payout are unproven; the fresh End collateral getter remains zero. The prior prorated settlement rate is not a one-for-one conversion.
Existing authorized contract → Vat.suck → eligible DaiJoin exit	Separate authority-dependent possibility creating credit and corresponding debt. No usable group-controlled arbitrary issuance path is demonstrated.
Funded markets paying already existing pDAI	Separate market route, subject to actual liquidity and costs; does not require ordinary Maker PSM issuance.

The [arbitrage and shutdown study \(Appendix 25\)](#) retains actual trades, prior credit balances, source matches and proposed settlement arithmetic. This new checkpoint updates selected adapter flags, identities, custody and allocation only; it does not refresh all earlier claims or prove settlement execution.

Economic relevance

The recorded settlement rate is **0.000879277935257435670774324 pUSDC per pDAI**. It pays about **879.277935 pUSDC for one million pDAI**, subject to processing, eligibility and rounding. At equal accounting value per normalized pStable, this returns **0.0879278%** of input value before costs, a difference of approximately **99.9121%**. The issue is the low payout per claim, rather than the adapter's physical balance alone.

A fresh trade could be rational if the complete pDAI acquisition and settlement cost were below the pUSDC-equivalent payout, using executable market values. An already packed eligible claim may also have recoverable value without a new pDAI purchase. These are conditional economic cases, not observed use or proof of recurring capacity. For the stabilization thesis, retain this as a **low-priority legacy settlement possibility**, not a credible equal-parity funding engine at the recorded rate.

Verification and reproduction

Fourteen explicit getter/code jobs generated 23 read-only RPC records, including selector derivation and matching block-header rechecks. Every request returned without an RPC error. Fresh Vat, DaiJoin and USDC-adapter runtimes exactly equal the corresponding saved runtimes used in the earlier source audit. AuthGemJoin5 join/exit/cage and Vat slip/frob checks were inspected offline.

The explicit manifest is [pulsechain_maker_adapter_closure_20261004.json](#). Raw reads, decoded verification, frozen paper input/output and hashes are in `artifacts/pulsechain-maker-adapter-closure-20261004/`. Earlier batches are unchanged. Always use the explicit getter manifest with the probe; its broader discovery default includes operations outside this read-only scope.

Appendix 45: Inventory-supported local pricing and a proposed LP exit

Contents of this appendix

Proposed mechanism	307
What constitutes enough purchasing power	307
Dominant ownership and two-sided support	308
Standard V2 execution	308
Ethereum and BTC exit hypothesis	309
Present conclusion and reproduction	309

Source record: docs/studies/analysis/pulsechain_managed_peg_and_lp_exit_20260930.md.

Inventory-supported local pricing and a proposed LP exit

30 September 2026. Analytical refinement of the coordinated pDAI thesis. This study distinguishes scenario assumptions, algebraic consequences and unverified integrations. It adds no chain observations or transactions.

Proposed mechanism

Assume coordinated source issuance and accepted assertions, authorized Portal capacity, compatible conversion markets and a system with more destination aToken units than the corresponding pStable units outside its control. The system repeatedly converts recognized inputs, buys pDAI, pUSDC and pUSDT, retains inventory and offers inventory at a designated local dollar target. Dominant holdings and control of important liquidity positions reduce the quantity of outside inventory that can oppose its policy.

The proposed Ethereum exit is separate: ETH or BTC holders contribute actual ETH or WBTC to pools paired with a compatible pDAI representation. Users trade against those reserves at variable rates. The issuer need not promise direct one-dollar redemption. For analysis of that exit, a successful local peg can be granted as an assumption. That assumption is not evidence establishing the local peg mechanism itself.

What constitutes enough purchasing power

Let n be pDAI units outside system control that can become sellable during the supported horizon. Include relevant unlocks, third-party mint rights and bridge arrivals. Let R be the target reference asset and τ the desired number of R units per pDAI. A zero-fee, fixed-price full-buyback commitment needs at least $n * \tau$ spendable R units. Real execution also needs fees, transfer availability and adequate timing. For multiple stables, shared funding and competing claims must be consolidated once.

Let $c(A, n)$ be the net R output executable from owned aToken inventory A at the required scale, after costs and competing commitments. Receipt-financed coverage requires $c(A, n)$ plus other available R to cover $n * \tau$ and costs. The numerical inequality $A > n$ establishes coverage only after a compatible, adequately funded conversion supplies the required purchasing power. A quote for a small trade cannot establish the conversion of billions of units.

For example, 120 receipt units converted at an assumed net 0.5 R per receipt provide 60 R. They cannot purchase 100 outside units at 1 R each. At an assumed net 1 R per receipt, the same receipt quantity can cover that simplified buyback. These are specified conversion scenarios, not quotes for actual aDAI. Underlying redemption additionally depends on reserve cash and applicable withdrawal checks; wrapper sales depend on their own output reserves. [Verified receipt and cap analysis \(Appendix 49\)](#).

Repeated accepted issuance can renew purchasing inputs and avoid the need for new outside income at each intervention. Portal outstanding exposure still consumes destination headroom until backing. Income, purchases, existing holdings or valid mint authority over the exact underlying can fund backing. The [deferred campaign study \(Appendix 34\)](#) records these separate balances and the examined cap encoding.

Dominant ownership and two-sided support

If the system acquires all willing outside wallet holders through adequately funded offers, it can determine its subsequent asking price. This is a strong inventory-control result. A posted sale at 1 R constrains upward pricing when buyers can access it; a funded purchase at 1 R constrains downward pricing when sellers can access it. Fees, transaction costs and route restrictions produce an executable band around the target.

After inventory is sold back into circulation, holders can sell again. The policy must maintain coverage of that outstanding inventory. With no outside trades, an ask remains a quote; successful purchases and sales demonstrate execution. The system's token ownership and its share of LP entitlement must be counted separately. Burned LP shares cannot be withdrawn by the system, and ownership of tokens elsewhere does not confer ownership of pool reserves.

Under idealized assumptions of continuously accessible funded bid and ask, complete conversion coverage, adequate target inventory and transaction execution, the support policy can enforce its quoted band. The conditional proof concerns the funded policy and access, rather than token-unit surplus alone. On-chain scheduling every ten seconds leaves an interval in which other trades can change prices. A specification must identify which trade sizes receive support and whether support occurs within the trade or later.

Standard V2 execution

For an ordinary fee-free constant-product pool, let x be its pDAI reserve and y its R reserve. A sale of q pDAI receives:

```
R received = y * q / (x + q)
ending pDAI reserve = x + q
ending R reserve = x * y / (x + q)
ending marginal R/pDAI = x * y / (x + q)^2
```

With $x = y = 1,000,000$ and $q = 1,000,000$, the seller receives 500,000 R; the ending reserves are 2,000,000 pDAI and 500,000 R, implying a marginal ratio of 0.25. The pool began at 1 R per pDAI, yet neither this seller's average execution nor the final spot price stays at 1. Fees reduce output. Subsequent intervention can restore the ratio without changing the completed sale's proceeds. An external willingness to trade at the target can attract corrective arbitrage, with depth and costs determining the resulting band.

The actual Uniswap V2 pair checks that output is strictly smaller than the corresponding reserve and enforces the fee-adjusted reserve product. An ordinary finite-input swap therefore cannot purchase its complete output inventory. LP withdrawal is a different operation requiring the relevant LP ownership. V2

behavior is deterministic given reserves and input; the target-maintenance policy requires additional execution and inventory. [Pinned V2 pair, pricing and quote formulas](#). This algebra describes the standard design, not a bytecode audit of every PulseChain pool or an assumed custom redemption contract.

Ethereum and BTC exit hypothesis

An LP provides a market exit by paying from contributed counterasset reserves. It need not create an issuer redemption obligation or hold one dollar of collateral against each pDAI outstanding. Market access nevertheless depends on actual available ETH or WBTC and changes as swaps alter the reserves.

Locking or burning LP shares can prevent the providers from withdrawing their proportional position. It does not prevent traders from removing counterassets through valid swaps. In the fee-free model, if pDAI input equals its starting pool reserve, a seller removes half the initial ETH reserve. Input equal to nine times that pDAI reserve removes 90% of initial ETH. The remaining ETH is positive, but the quote deteriorates. Locking shares establishes persistence of liquidity entitlement, rather than fixed exit capacity or price.

ETH already committed to staking needs an actual liquidity or redemption route before it can fund an ETH pool. A staking receipt paired with pDAI is a different asset pair. The Ethereum representation must preserve legitimate transfer and bridge rights; WBTC also carries its own token and redemption dependencies. None of these proposed integrations was deployed or verified in this study.

With interoperable pDAI and available arbitrage, the implied external price is:

$$\text{USD per pDAI} = \text{ETH received per pDAI} * \text{USD per ETH}$$

Ignoring fees and market impact, a pDAI fixed at 1 USD exchanges for 1/2,000 ETH when ETH is 2,000 USD and for 1/4,000 ETH when ETH is 4,000 USD. Its ETH ratio changes while its dollar value stays constant. If an external pool implies another dollar value, connected traders have an incentive to buy on the cheaper side and sell on the dearer side. Bridges, fees, limits and delays can sustain differences. Fully connected markets cannot have arbitrarily independent prices while offering unrestricted conversion. [V2 market and arbitrage example](#).

Present conclusion and reproduction

The refined thesis is a managed local peg financed by accepted issuance, usable conversions and concentrated inventory, with a separately proposed LP market exit. A funded two-sided policy can be specified and proved under explicit coverage and execution assumptions. Receipt quantity and dominant ownership alone do not prove the required execution. Universal external redemption is not required to study that local policy; an independent dollar reference is required to distinguish dollar stability from internal parity.

The explicit scenario manifest is `Research/experiments/pulsechain_managed_peg_20260930.json`; its independently checked results are `artifacts/pulsechain-managed-peg-20260930/scenario-checks.json`. The formulas above use rational arithmetic and exclude fees, external flows, borrowing, controller replenishment and actual chain quotes. Scenario checks are illustrations of the stated predicates, not forecasts or an adversarial simulation of the whole proposed system. No blockchain calls were made.

Appendix 46: Optimism relays and Across governance on PulseChain

Contents of this appendix

The eight transactions, in chronological order.....	310
Synthetix has its own cross-chain administrative relay	311
The Aave execution does not demonstrate a new vote or messenger ownership	312
Old Optimism proofs can settle locally after the fork	312
The Across ownership proposal did execute through UMA governance.....	312
Consequence for the thesis and reproduction	313

Source record: docs/studies/analysis/pulsechain_optimism_relay_and_across_governance_20261001.md.

Optimism relays and Across governance on PulseChain

1 October 2026. All investigation is read-only. This study decodes eight examined transactions, compares relevant inherited and current state, and connects one proposed governance action to its previously verified execution. Addresses identify accounts and contracts, not people.

The records establish three different activities: an inherited Synthetix administrative message, later settlement of inherited Optimism withdrawals, and postfork control changes in Across. They do not establish that the same account owns every component.

The eight transactions, in chronological order

UTC date	Transaction	Recorded action	Meaning
4 May 2023	fd38...da9	Synthetix Safe calls its Ethereum owner relay, which sends an Optimism instruction to give an account Futures suspend/resume permission	A pre-fork Ethereum governance operation inherited by PulseChain
24 August 2024	385e...755	Optimism messenger proves a withdrawal and the copied standard bridge pays 0.02 native PLS	Successful local settlement using an April 2023 state commitment
6 October 2024	38b6...f93	Optimism DAI bridge releases 301,632.643171596086897 512 local DAI to Across HubPool	A direct Across/Optimism settlement connection using a May 2023 commitment
29 October 2024	3055...383	BAD9 sends 1 PLS to HubPool's loadEthForL2Calls()	Prefunding for future cross-chain fees; this transaction contains no L2 call

UTC date	Transaction	Recorded action	Meaning
14 June 2026	c5e7...7e2	feca proposes transferring HubPool ownership to itself through the Across Safe's OptimisticGovernor module	An UMA assertion-backed proposal; execution is a separate transaction
19 June 2026	9abe...56f	feca calls HubPool's owner-only <code>emergencyDeleteProposal()</code> and refunds a 0.45 copied-WETH bond	Administration of an Across rebalance bundle after acquiring ownership
2 July 2026	fae8...61b	feca configures an adapter and spoke for chain ID 88888	Direct evidence of Across configuration authority; no message is sent in this transaction
5 August 2026	4d84...c09	1967 executes Aave proposal 218, sending its queued MAI listing instruction toward Optimism	Permissionless execution of a queued proposal, with an outbound L1-to-L2 message

All eight saved receipts report success. The offline decoder checks transaction, receipt, trace and block-header identities and the transaction's inclusion in its recorded block. Failed optional subcalls in the native withdrawal are retained; its actual payment succeeds. This is one-provider evidence, with a separate Ethereum check of the pre-fork Synthetix transaction and inherited proof predicates.

Synthetix has its own cross-chain administrative relay

The first transaction is block **17,187,730**, before the inherited boundary **17,232,999**. An Ethereum RPC returns the same transaction and block. Its caller is `0x302d2451d9f47620374b54c521423bf0403916a2`, not 1967.

```
Synthetix Safe on L1
→ OwnerRelayOnEthereum
→ Optimism messenger sends instruction
→ designated OwnerRelayOnOptimism on L2
→ authorized L2 target
```

The decoded instruction targets `SystemStatus 0xe8c41be1a167314abaf2423b72bf8da826943ffd` and authorizes `0x6cd3f878852769e04a723a5f66ca7dd4d9e38a6c` to suspend and resume the Futures section. This is a specific permission, not a universal token-minting grant.

[SIP-167](#) describes this architecture. The preserved, integrity-checked `synthetix@2.88.1` source requires the L1 relay's owner to initiate instructions. The L2 relay requires both the designated messenger as caller and the correct cross-domain sender. Consequently, effective control of the Synthetix Safe could administer suitably owned L2 Synthetix contracts without owning the Optimism messenger itself. Current private-L2 delivery and ownership are not inspected.

At PulseChain block **27,684,339**, Synthetix's L1 relay owner remains Safe `0xeb3107117fead7de89cd14d463d340a2e6917769`; its nominee is zero and its resolver cache is valid. The Safe threshold is five of thirteen. Its sampled `isOwner(1967)` is false. Signer-key control, an undiscovered authorization route or private-L2 changes are not resolved by those getters.

The Aave execution does not demonstrate a new vote or messenger ownership

The August transaction calls `execute(218)` on Aave Governance V2. Published code permits execution once the governance and timelock conditions pass. The recorded path is governance → short executor → message adapter → Optimism `sendMessage`.

The inner instruction queues payload `0x1720d605289a189c5fbd360ca2307db5c6d3fd3fd` at Optimism executor `0x7d9103572be58ffe99dc390e8246f02dcae6f611`. Its bytes match the original proposal 218 execution report retained in the [Aave history study \(Appendix 2\)](#). The destination proposal concerned MAI listing on Aave V3 Optimism; destination execution on the private source environment is not shown here.

The inherited and current getters return the same **424,248.253753485267005365** votes for and zero against. Both return zero voting power for 1967 in this proposal. No vote-casting call occurs in the examined transaction. Its execution flag changes from false at the fork to true at the current checkpoint.

Calls to `AddressManager` `0xde1fcfb0851916ca5101820a69b13a4e276bd81f` are `STATICCALL getAddress(string)` registry reads. They do not update ownership or mappings. The registry and messenger report the separate Optimism Safe `0x9ba6e03d8b90de867373db8cf1a58d2f7f006b3a`, threshold five of seven. Its sampled `isOwner(1967)` is false. Calling a governance executor or passing through a messenger is different from controlling either contract.

Old Optimism proofs can settle locally after the fork

The two withdrawal proofs reference state commitments dated **10 April 2023** and **9 May 2023**. Calling the exact recorded `verifyStateCommitment` view calldata at the Ethereum fork block returns true for both. The April commitment was already outside the challenge period; the May commitment remained inside it at the fork and was mature by its October 2024 settlement. Message-success flags change from false at the inherited comparison to true at the current checkpoint.

The native withdrawal authenticates the L2 standard bridge `0x4200000000000000000000000000000000000000000000000000000000000000` and pays `0x5da70d2a1858a883b42d560edeb5a4ff4e7829fe`. A copied ETH balance/payment is native PLS on PulseChain. Optional failed `isUpgrading()` probes are tolerated by the published `ChugSplashProxy` logic; they are not Safe signer changes.

The DAI withdrawal authenticates L2 DAI bridge `0x467194771dae2967aef3ecbedd3bf9a310c76c65`. Source DAI is `0xda10009cbd5d07dd0cecc66161fc93d7c9000da1`; destination DAI is the existing local `0x6b175474e89094c44da98b954eedeac495271d0f`. The source user is Across's Optimism spoke `0x6f26bf09b1c792e3228e5467807a900a503c0281`, and the destination is its HubPool `0xc186fa914353c44b2e33ebe05f21846f1048beda`.

The trace and receipt show `transferFrom` releasing the exact DAI amount from local escrow to HubPool, returning true and emitting `Transfer`. This route delivers an existing usable pool token despite differing source/destination addresses. It releases escrow; it does not mint DAI.

These successful relays strengthen the destination-integration evidence. They also explain how postfork withdrawals coexist with the earlier finding of no new selected `StateCommitmentChain` batches: the supplied proofs were inherited. They do not prove that a newly asserted private-L2 balance has an accepted fresh destination commitment. Existing unspent claims are finite.

The Across ownership proposal did execute through UMA governance

The June proposal and [19 June execution](#) share both:

- Proposal hash `0xcfcabb729c0274e198ef3ab0556577c135c587f95ee22403a408ab0e1ec9b766`.

- Assertion ID 0xbc640aecfd6f681d85bc31e7a12abcf156f20616f6d96cdf8ae871e85f8fe54.

The proposal asks HubPool to transfer ownership from the Across Safe to feca. Its recorded challenge interval is **432,000 seconds, five days**, ending 19 June 2026 at 05:44:35 UTC. Execution follows at 05:46:05 UTC. The settlement event reports **undisputed** and **truthful**; the Safe module successfully calls HubPool and its ownership event records feca as new owner.

```
Proposal + bond → UMA assertion → five-day challenge interval
                        ↓ undisputed settlement
                Across Safe module executes
                        ↓
                HubPool owner becomes feca
```

There is a material conflict between the embedded rules and the executed action: the rules say **“oSnap should not be used to call the HubPool contract”**, yet HubPool is the target. They also require a unique approved proposal identifier; the recorded explanation is just `osnap`. The receipts establish on-chain acceptance and execution, not a valid off-chain Snapshot vote or compliance with those stated rules. The verified implementation submits a claim to the oracle rather than parsing the governance rules as an on-chain target allowlist. It permits proposal submission externally, subject to its bond and oracle/escalation settings.

The current module's liveness getter is **1,200,000 seconds**, and its collateral getter identifies the copied UMA token. Those are later settings, not the five-day period and copied-WETH bond used in this proposal. The earlier [Across control audit \(Appendix 3\)](#) separately proves that e121 acquired module ownership on 19 August 2026. UMA's optimistic assertion system and Optimism's rollup messenger are different systems.

The July owner-only call registers chain ID **88888**, adapter 0x3233f8c0db384a2a1d6a307de41a804dfe411eb6, and spoke 0x5c7bcd6e7de5423a257d81b442095a1a6ced35c5. The current getter matches. The sampled adapter source and spoke implementation are unverified; this study does not establish that 88888 is the private pARB network or that this configuration delivers fresh synths. The adapter's runtime cannot be treated as a standard bridge solely from its configured address.

The June emergency deletion returns a bundle bond to 0xf7bac63fc7ceacf0589f25454ecf5c2ce904997c. It is distinct from deleting an UMA governance proposal. BAD9's October 2024 fee deposit demonstrates funding of HubPool, not a cross-chain transaction in that receipt.

Consequence for the thesis and reproduction

Confirmed Across authority can support route configuration and treasury operations. Inherited Optimism claims can provide local inventory. Synthetix's separate owner relay supplies a credible administrative architecture, and its native synth bridge has a distinct issuer-specific destination mint path. A functioning new financing loop still needs accepted fresh source messages, exact usable outputs and destination liquidity; Across ownership does not grant Synthetix mint permissions.

Raw records, source responses, failures and derived results are in `artifacts/pulsechain-optimism-relay-votes-20261001/`. The explicit batch is `Research/experiments/pulsechain_optimism_relay_transactions_20261001.json`. Reproduce its eight-transaction decode with:

```
python Research/scripts/analyze_pulsechain_optimism_relays.py Research/experiments/
pulsechain_optimism_relay_transactions_20261001.json --output /tmp/optimism-transaction-analysis.json
```


The output path must be fresh. The command is offline. Current getter checkpoints are blocks **27,684,339** and **27,684,371**, with hashes retained in their raw records. Earlier ownership execution records remain under `artifacts/parb-across-control-20260929/`; `proposal-execution-match.json` verifies the matching event identifiers and records the rules conflict. No privileged action was performed or simulated.

Appendix 47: pDAI: profitable pool cycles and coordinated liquidity

Contents of this appendix

Finding.....	315
Model and accounting boundary	315
A complete supporting cycle	316
Counterexample: profitable rebalancing lowers the PLS exit.....	317
A two-cycle transmission from a FRAX sale	317
Frax issuance, lending and Convex	318
Aave and Balancer as liquidity connections	318
Cross-chain incentives are separate from principal transfer	319
Reinvestment and the combined accounts.....	319
Evidence status and reproduction	319

Source record: docs/studies/analysis/pulsechain_pool_cycles_20260929.md.

pDAI: profitable pool cycles and coordinated liquidity

29 September 2026. Illustrative mathematical experiments and published protocol architecture, alongside separately identified historical PulseChain evidence. No chain transactions, deployment simulation or new authority claims.

Finding

Connected markets can transmit demand into profitable pDAI purchases even when the initiating trade does not involve pDAI. The location of the price discrepancy and the closing trade determine the effect on pDAI's PLS execution price. A profitable route that buys pDAI as an intermediate asset can nevertheless lower that price when it sells pDAI into the final market.

Managed liquidity, authorized issuance, lending and reinvested income are coherent additions to this conditional architecture. Their contribution must be measured after repayment and consolidated costs. These mechanisms do not by themselves establish current pARB operation, universal token fungibility, common control of the venues, or a durable dollar peg.

Model and accounting boundary

Pool	Initial reserves	Pricing assumption
FRAX/pUSDC	100 million of each	Two-coin StableSwap-type, $A = 100$
pDAI/pUSDC	100 million of each	Two-coin StableSwap-type, $A = 100$
FRAX/PLS	10 million FRAX; 1 billion PLS	Constant product
pDAI/PLS	10 million pDAI; 1 billion PLS	Constant product
pUSDC/PLS	10 million pUSDC; 1 billion PLS	Constant product; used in cascade

These are invented reserves, not observations of those trading pairs on PulseChain. FRAX denotes the hypothetical accepted local asset, without an assumed redemption right or dollar value. The two-coin model is not a simulation of the deployed FRAX/3CRV metapool and its base-pool withdrawal mechanics.

All balances use normalized 18-decimal integer units, fixed unit exchange rates, 0.04% stable-pool output fees, 0.3% constant-product input fees and zero admin fees. Fees stay in the pools. Actual pUSDC has different raw precision; deployed metapools may use rate providers, virtual prices and different rounding rules.

For $n = 2$, $Ann = A \cdot n$, balances x_i and $s = \sum x_i$, the stable invariant is solved by integer iteration:

$$D_p = D \times \prod_i [D / (n \cdot x_i)]$$

$$D_{next} = (Ann \cdot S + n \cdot D_p) \cdot D / ((Ann-1) \cdot D + (n+1) \cdot D_p).$$

For a swap, keep D fixed, add the gross input, solve the corresponding output balance with the same invariant, and deduct the output fee. The script gives the precise sequence of integer divisions. Convergence is within one raw unit; there is no additional one-unit output subtraction. This reproduces the prior illustrative calculation; it is not a claim of byte-for-byte contract parity. Constant-product output is $y \cdot 997 \cdot a / (1000 \cdot x + 997 \cdot a)$, rounded down.

Four-leg cycles assume 3,000 PLS gas; three-leg cycles assume 2,000 PLS. Arbitrage capital is owned. Initiating trades are open positions with separately identified spending and receipts; their acquisition costs and gas are not modeled. Borrowing, bridge costs, ordering competition, withdrawals and rewards are excluded. Sizes are selected examples, not optimized trades. Each scenario resets the network; cycles within a cascade run sequentially on changed balances.

Exact conservation checks include pools, the initiating buyer, arbitrageurs and a gas recipient. A cycle's gross PLS gain equals the pools' aggregate PLS loss; non-PLS totals across pools are unchanged by a closed PLS cycle. No tokens are created or burned. Gas goes to the modeled recipient rather than disappearing.

A complete supporting cycle

An initiating buyer spends **200,000,000 PLS** buying FRAX in FRAX/PLS and receives **1,662,497.915624 FRAX**. A separate participant can then execute:

Leg	Amount received
Starting capital	20,000,000 PLS
PLS → pDAI	195,501.696178 pDAI
pDAI → pUSDC	195,419.712824 pUSDC
pUSDC → FRAX	195,337.765434 FRAX
FRAX → PLS	27,390,430.130599 PLS

Net profit after assumed gas is **7,387,430.130599 PLS**. The pDAI/PLS reserve-ratio price rises from 100 to **104.03388 PLS per pDAI**. More usefully, a subsequent 1,000-pDAI sale returns **103,711.232170 PLS**, up from **99,690.060901 PLS**, after trading fees and before that sale's gas. These are hypothetical probe quotes; the probe is not executed in the ledger.

The arbitrageur moves pDAI from its PLS pool into its stablecoin pool. Total pDAI across the network does not fall. Support of one execution route therefore does not establish permanent removal of circulating supply or an identical price improvement in every venue. No PLS dollar reference is assumed in this model.

Without the initiating purchase, the same four-leg trade loses **899,861.750741 PLS** after gas. Its mechanical price effect still exists if someone forces the loss-making trade, but the assumed incentive does not. The initiating buyer's expenditure, remaining FRAX inventory and funding costs must not be confused with the separate arbitrageur's realized profit.

Counterexample: profitable rebalancing lowers the PLS exit

Reset the network. An initiating buyer now spends **80 million pUSDC** buying FRAX in FRAX/pUSDC. The profitable closing route reverses:

Leg	Amount received
Starting capital	20,000,000 PLS
PLS → FRAX	195,501.696178 FRAX
FRAX → pUSDC	215,428.048068 pUSDC
pUSDC → pDAI	215,337.283789 pDAI
pDAI → PLS	21,017,891.409779 PLS

Net profit is **1,014,891.409779 PLS**, but the final trade sells pDAI into its PLS pool. Its reserve-ratio price falls to **95.83453599 PLS per pDAI**, and the 1,000-pDAI exit falls to **95,537.708062 PLS**. Buying pDAI along a route is therefore insufficient evidence of upward support in the local settlement market. This is an alternative market state, not additional capacity to add to the first example.

A two-cycle transmission from a FRAX sale

Reset again. Sell **80 million FRAX** into FRAX/pUSDC. A first arbitrageur closes PLS → pUSDC → FRAX → PLS, turning 20 million PLS into **21,026,564.138897 PLS**. After 2,000 PLS gas, profit is **1,024,564.138897 PLS**. This first cycle never buys pDAI, but it changes the pUSDC/PLS market.

A second arbitrageur then closes PLS → pDAI → pUSDC → PLS, turning 8 million PLS into **8,138,579.664172 PLS**. After gas, profit is **136,579.664172 PLS**. The pDAI/PLS reserve-ratio price reaches **101.6039808**, and the 1,000-pDAI exit becomes **101,288.989800 PLS**. Thus an input-token sale can transmit support through more than one profitable closing route.

This example assumes the initiating FRAX is already available and accepted. It does not model its source-chain issuance, collateral, bridge or acquisition costs. Calling it source-derived would require those additional proofs.

A sensitivity case supplies only **1 million FRAX** but forces the same trade sizes. The first cycle then loses **887,288.868760 PLS** after gas. The second still profits because the forced first trade changed pUSDC/PLS by the same input amount. Their combined result is a loss of **750,709.204588 PLS**. This is a diagnostic calculation, not a predicted voluntary sequence: an independent arbitrageur would not select that first loss-making trade on these assumptions. Neither the large case nor this control proves what other sizes or routes do.

Frax issuance, lending and Convex

Frax's [legacy Curve AMO description](#) connects FRAX and collateral deployment, Curve liquidity income and Convex staking. That specific AMO is now documented as deprecated; its architecture remains a relevant precedent, not a claim about current PulseChain operation.

The pinned [Convex_AMO_V2 source](#) selects FRAX/3CRV at `0xd632f22692FaC7611d2AA1C0D552930D43CAEd3B` and Convex Booster pool 32. It includes liquidity deposits, LP staking, withdrawals and reward claims with role restrictions. This matches a familiar pool address in the [historical Curve study \(Appendix 8\)](#); source-level wiring alone does not match a deployed AMO's bytecode, storage or authority.

The pinned [FraxAMOMinter source](#) requires an authorized caller, registered destination AMO, mint-cap headroom and a minimum collateral-ratio check for `mintFraxForAMO`. A price premium does not call this function automatically. These repository versions are pinned for reproduction, not identified as fork-time or deployed PulseChain implementations.

There are two different policy responses to FRAX demand. An authorized issuer can sell FRAX and allocate proceeds to pDAI purchases; or it can add FRAX liquidity to remove the premium. The latter can reduce the incentive for the supporting cycle. The same inventory and proceeds cannot count twice as independent support.

[Fraxlend pairs](#) lend a specified asset against specified collateral, with interest and liquidation conditions. Ordinary lending is not automatic FRAX issuance; an authorized AMO supplying issued FRAX is a separate mechanism. A financed candidate cycle is:

```
Eligible collateral → borrow FRAX → sell into a premium
→ trade through connected markets → recover enough FRAX
→ repay principal and interest → release collateral → retain surplus.
```

Every step needs executable quotes and available funding. This candidate has not been numerically demonstrated here. Borrowing FRAX, buying pDAI and retaining it is instead an open financed position. Its debt remains even if pDAI demand increases. Collateral appreciation and an assumed dollar price do not settle it.

Aave and Balancer as liquidity connections

[Balancer V2 boosted-pool documentation](#) describes nested linear pools connecting stablecoin cash and lending receipts. A candidate closed route is pDAI into pUSDC on Curve, through the USDC linear pool and containing stable pool, then through the DAI linear pool back to pDAI. It is profitable only if final pDAI exceeds starting pDAI and all costs. This candidate is architectural; no complete deployed-route quote is established.

The [pool-limit study \(Appendix 16\)](#) records actual local wrapper trades, but small available main-token output in the sampled pools. Replenishment requires recorded liquidity additions or successful withdrawals from lending positions. Large receipt balances and preminted pool shares are not spare cash. The shared Balancer Vault balance is not all allocated to any one pool.

The [Portal accounting study \(Appendix 17\)](#) separately records zero underlying cash at the sampled DAI, USDC and USDT aToken contracts, zero unbacked caps, and other supply constraints at block 27,659,393. These are dated destination observations. Ordinary debt repayment with underlying may restore cash; it does not also cancel Portal unbacked principal. A cycle using Portal must settle that distinct obligation and its fee. Aave lender interest, borrower debts and withdrawable underlying cannot be treated interchangeably.

Cross-chain incentives are separate from principal transfer

Curve's [cross-chain gauge repository](#) describes transmitting CRV emissions through chain-specific bridge wrappers. [Convex sidechain documentation](#) describes local tokenized staking positions and rewards; withdrawal returns the corresponding Curve LP token. Neither mechanism alone locks an arbitrary pARB LP token and issues an existing, accepted PulseChain LP token.

Reward transport can supply income while principal remains invested. To use that income on PulseChain still requires an operating source deployment, authenticated delivery, the exact local reward asset, and funded conversion. LP ownership, administrative control, gauge voting and reward custody are distinct rights. Staking through Convex does not establish control of other depositors' funds or universal bridge authority.

Reinvestment and the combined accounts

Assume all income controlled by the participating treasury is reinvested:

```
Attributable LP fees + collected lending income + realized reward proceeds  
→ treasury allocation → Aave liquidity, AMM inventory or pDAI purchases.
```

Income can strengthen reserves, but reinvestment is an allocation of resources, not a second receipt of those resources. Fees retained in LP positions already enter their value. Count them as available cash only after withdrawal or funded financing, and reduce the remaining position or recognize its financing debt. Protocol/admin fees and other LP holders' shares are not treasury income.

Fees and interest paid among commonly controlled positions cancel on consolidation. If the treasury owns the arbitrageur and the affected LP exposure, its trading gain must be reconciled with the LP inventory change. Outside trading fees, collected borrower payments and external demand for rewards can add resources; internal circulation alone cannot establish net growth.

For a treasury cash account in a stated settlement asset:

```
Closing cash = opening cash + external contributions + collected income  
              + realized asset sales + net borrowing  
              - purchases and deployments - debt service - operating costs.
```

Asset sales convert inventory to cash; borrowing raises liabilities; deposits replace cash with claims. None is income by itself. Evaluate total resources separately from immediate cash, using realizable asset values and all obligations. This study assumes full reinvestment as a policy, but does not assign an APY, forecast compounded returns, or demonstrate indefinite support.

Evidence status and reproduction

The [historical Curve record \(Appendix 8\)](#) verifies 820 selected events in successful receipts across four metapools, including an actual mUSD-to-DAI purchase. That establishes component trading behavior, not any of the modeled complete cycles or the economic origin of their hypothetical input. No selected configuration events in that bounded search is not evidence that balances never changed through swaps or liquidity movements.

The fresh explicit manifest is `Research/experiments/pulsechain_pool_cycles_20260929.json`; run from the checkout:

```
python3 Research/scripts/model_pulsechain_pool_cycles.py --manifest Research/experiments/
pulsechain_pool_cycles_20260929.json --output artifacts/pulsechain-pool-cycles-reproduction/results.json
```

Use a fresh output file: the command refuses to overwrite results. The verified reference run is `artifacts/pulsechain-pool-cycles-20260929/results.json` and records script/manifest hashes, full leg outputs, final reserves, probe quotes and exact conservation checks. All five scenarios reproduce the amounts above; the displayed six-decimal values are rounded, not raw token amounts.

Published-source snapshots and their URLs, retrieval time and SHA-256 hashes are in `artifacts/pulsechain-pool-cycles-20260929/sources/source-manifest.json`. The Frax repository is pinned to commit `30532c8cefcbf5c7efafcff4369261bd435a4859`. Documentation snapshots are evidence of the published architecture at retrieval, not historical deployed state. The anonymous release bundles these records and assigns appendix links.

A deployed proof would additionally require one consistent block/state for each venue; exact token mappings, rates and permissions; successful cash withdrawals; complete quotes including gas and debt settlement; and the resulting pDAI execution price at the stated size. A repeated-support claim further needs cash-flow evidence under adverse conditions. The new results establish finite profitable transmission and its directional limits within the declared model.

Appendix 48: PulseChain Portal backing, aDAI index and local arbitrage

Contents of this appendix

Scope and conclusion	321
1. Source backing and destination accounting	321
2. Five quantities that must remain distinct	322
3. Actual sampled markets and a bounded cycle	323
4. Independent current gates	323
What would resolve each check?	324
5. What can fit in one block?	324
6. Exact contracts and reproduction	325

Source record: docs/studies/analysis/pulsechain_portal_pricing_20260930.md.

PulseChain Portal backing, aDAI index and local arbitrage

Scope and conclusion

All observed state in this study is from **PulseChain, chain ID 369**, at block **27,673,486**, hash `0x2702410507da0c8e4bf66c752465d28cc977725a128b9eb405a21f99c29ce9b3`. No Ethereum state is substituted for the copied contracts. Requests were reads; no transactions were signed or broadcast and no configuration was changed.

Remote backing and local unbacked accounting are compatible. A source-backed bridge can still call destination `mintUnbacked`: the destination has issued its own aToken before receiving its own underlying. Aave's liquidity index does not multiply new issuance into extra principal. A static waDAI share represents multiple current aDAI units, which must be normalized before comparing prices.

Existing waDAI has a small quoted PulseX-to-Balancer spread at this block. The tested cycle is uneconomic even against the base transaction gas minimum at the sampled fee and spot conversion. New Portal issuance and wrapping also encounter distinct current configuration gates. These findings specify what would need to change for the proposed route; they do not disprove every possible coordinated route or establish an operating pARB source bridge.

1. Source backing and destination accounting

Suppose the operator controls a backed source Aave position. Locking or surrendering that position can secure a bridge's obligation. It does not put pDAI into the PulseChain destination reserve. If the destination operation creates its exact aDAI through `mintUnbacked`, it records a local advance until `backUnbacked` delivers underlying there. This remains true even with sound remote backing. The bridge must prevent the same source claim being spent twice.

This distinction is explicit in the upstream [deBridge Portal proposal](#): the proposed adapter locks liquidity on one chain, waits for finality, and calls `mintUnbacked` on the other. This is a design reference, not evidence that this adapter operates on PulseChain. Aave's [Portal architecture](#) provides a credit facility for integrated bridges; it is not itself a universal token transporter.

Destination operation	New Portal unbacked issuance?	Requirement
Authorized mintUnbacked	Yes	Bridge authorization, reserve validation and sufficient unbacked cap when called
Deliver pDAI and use ordinary Aave supply	No	Actual destination underlying and ordinary supply checks
Transfer existing destination aDAI	No	A funded holder or relayer and transferable receipts
Bridge a remote receipt into a separate representation	Not inherently	Exact mapping and a buyer, lender or adapter accepting that representation

The last three are alternatives to Portal credit, not hidden alternative Portal mint methods. A separate bridged receipt does not become the existing destination aDAI solely because its symbol matches. `backUnbacked` settles outstanding local advances; it is not an unrestricted way to mint aTokens for the payer.

2. Five quantities that must remain distinct

Quantity	PulseChain result	Meaning
DAI normalized liquidity index	9.984280942428067601829364415	Multiplier from scaled aDAI balance to current aDAI balance
aDAI displayed total supply	400,036,132.903002514218870593	Already includes the index
aDAI scaled total supply	40,066,594.200394977560121344	Internal accounting units before index multiplication
<code>waDAI.convertToAssets(1e18)</code>	9.956311649989486401	aDAI/pDAI accounting units represented by one static share
<code>waDAI.convertToShares(1e18)</code>	0.100438800547294637	Static shares for one current asset unit, as a conversion calculation

The aDAI implementation's `_mintScaled` divides a new principal amount by the current index before minting scaled units. Its displayed balance multiplies scaled units by that index. Ignoring rounding and intervening accrual:

`new scaled units = principal / index`

`new displayed aDAI = new scaled units × index = principal`

Therefore, minting 100 current aDAI does not create about 998 current aDAI. A permitted wrapper conversion would yield about 10.0438800547 waDAI shares. A price near ten pDAI for one waDAI share would then approximately match the principal represented; it would not establish a tenfold arbitrage profit.

The underlying DAI oracle returns **1.00009178** in its inherited USD accounting convention. The source's `latestRoundData.updatedAt` is **10 May 2023, 21:39:47 UTC**. That stale copied feed is not a current PulseChain dollar price. Direct oracle requests for these aDAI and waDAI addresses revert, and their configured price-source addresses are zero. The reserve's underlying valuation, receipt conversion and exchange price are separate observations.

3. Actual sampled markets and a bounded cycle

The exact V3 aDAI/pDAI PulseX V2 pair has stored reserves of approximately 28,276.614 aDAI and 16,326.380 pDAI: **0.577381 pDAI per aDAI** before fee and size effects. A one-aDAI input gives a reserve-only estimate of 0.575686 pDAI after the pair's 0.29% swap fee. The live aDAI balance is 28,276.789776, slightly above stored reserves because the receipt accrues; this estimate is not a complete swap simulation. These reads establish a discount in this exact pair, not a several-fold premium for this aDAI.

The waDAI/pDAI pair holds approximately 31.873622 waDAI and 81.139352 pDAI, giving **2.545658 pDAI per whole waDAI**. That is a premium to one pDAI token, but a discount to the share's approximately 9.956312-unit accounting claim.

An existing-wrapper cycle avoids creating a new wrapper position:

pDAI → buy existing waDAI on PulseX → sell waDAI into Balancer → pDAI

Step	Pinned calculation or read
Buy 0.005021940027364730 existing waDAI	0.012823345760635207 pDAI, reserve-based exact-output calculation
Sell that amount to the DAI linear pool	0.049999500004999931 pDAI, direct queryBatchSwap result
Difference before gas	0.037176154244364724 pDAI
Difference at sampled pDAI/WPLS spot ratio	6.80782566003938 PLS
Gas this difference could fund at sampled base fee	15,733.232 gas

That gas allowance is below the ordinary **21,000 gas transaction minimum**, before either swap, calldata or priority fees. Thus this tested standalone cycle is not profitable at the sampled base fee and spot conversion. A full executor transaction was not simulated. The calculation does not establish profitability of a larger or batched route, which would require its own quotes, incremental gas and closing balances.

The Balancer linear pool has 10.055365963093765 pDAI registered cash, but its lower-target fee accounting leaves only approximately **0.055466 pDAI** of additional output capacity on this simple direction at the sampled state. Queries using shares representing 0.01 and 0.05 aDAI succeed. Queries using shares representing 1 or 100 aDAI, and a query selling one whole waDAI, revert with `BAL#001`. A large share exchange rate is therefore not evidence of large executable output. Replenishing or restructuring liquidity would be a separate action. Earlier large historical flows do not measure present capacity.

4. Independent current gates

The destination DAI reserve has **unbacked mint cap 0, outstanding unbacked balance 0**, and **actual pDAI cash 0** at the pinned block. These are three different fields. A zero cap forbids positive `mintUnbacked` under that state; zero outstanding unbacked does not establish that no historical mint occurred.

The wrapper's pure conversions succeed, but `maxDeposit`, `maxMint` and `previewDeposit(1e18)` revert with arithmetic panic `0x11`. Its published current implementation computes the remaining supply capacity by subtracting supplied principal plus treasury accrual, measured at the stored index, from the supply cap.

Here that basis is approximately **450,597,451.148**, above the **338,000,000** supply cap. This checked subtraction explains the observed underflow. The wrapper applies this gate before both underlying and aToken deposit paths. Merely transferring existing aDAI to a prospective depositor does not bypass it.

The same wrapper reports `previewRedeem(1 wADAI) = 0`. Its redemption limit uses underlying Aave cash even when selecting the aToken output path. With no cash in the sampled reserve, the reviewed code does not support treating `redeemAsATokens` as an automatic escape from that gate. Existing wrapper tokens can still be exchanged in a funded market that accepts them.

These are current code-and-state conditions, not permanent impossibility claims. Different configurations, implementations or funded pools require their own validation. Raising only the unbacked mint cap would not address all the conditions above.

What would resolve each check?

An authorized Pool configurator can enable a positive unbacked mint allowance large enough for the outstanding balance and proposed issuance. The mint caller must independently hold the required bridge permission. Calling `backUnbacked` after a mint cannot cure a cap check that already reverted.

For the wrapper's subtraction, an authorized change could raise the reserve supply cap above its calculated supplied amount, with headroom for new issuance and accrual. This implementation treats a **zero supply cap as unlimited** and returns unlimited supply capacity before that subtraction. That differs from the **zero unbacked mint cap**, which prevents positive Portal issuance. Reducing outstanding supplied principal sufficiently could also restore capacity, but withdrawals require underlying liquidity and appropriate claim ownership; no such reduction was performed or assumed.

A separately authorized wrapper implementation change could handle exhausted capacity without arithmetic panic and reconsider its aToken-only deposit gate. Clamping the subtraction to zero alone would make the limit readable but still would not permit positive deposits. Whether a changed wrapper should admit existing aTokens independently of ordinary supply capacity is a design choice requiring review, not a property of the current implementation. This study has not established who controls these configuration or upgrade permissions.

After either configuration or code changes, the entire sequence needs fresh validation. Pool liquidity, receipt acceptance, conversion rounding and full backing remain independent requirements. A technical permission change does not create the underlying assets needed to settle the route.

5. What can fit in one block?

Scope	Timing conclusion
Existing local receipt purchase and resale	Can be composed into one PulseChain transaction when every call and repayment succeeds
Local <code>mintUnbacked</code> → <code>wrap</code> → <code>swap</code> → <code>backUnbacked</code>	Architecturally composable, but the sampled cap and wrapper gates prevent this proposed route under unchanged state
A pARB source lock followed by a PulseChain bridge delivery	Separate execution environments and message/finality requirements; not one shared atomic EVM transaction
Destination execution after an accepted message, or using a prefunded relay	Destination actions may be atomic; earlier source work and later settlement remain separate

Transactions merely placed in the same block are not automatically atomic with one another. The meaningful local guarantee is one transaction that reverts all its calls if any required step fails. Portal backing in that transaction needs the entire underlying principal and any supplied fee; the liquidity index cannot fill a settlement shortfall.

6. Exact contracts and reproduction

Contract	PulseChain address
pDAI	0x6b175474e89094c44da98b954eedeac495271d0f
V3 aDAI	0x018008bf33d285247a21d44e50697654f754e63
V3 waDAI	0x098256c06ab24f5655c5506a6488781bd711c14b
Aave V3 Pool	0x87870bca3f3fd6335c3f4ce8392d69350b4fa4e2
Aave oracle	0x54586be62e3c3580375ae3723c145253060ca0c2
DAI price source	0xaed0c38402a5d19df6e4c03f4e2dced6e29c1ee9
aDAI/pDAI PulseX pair	0xc1d1fdd3f2337e75a257a591f34a7644e0c4d06b
waDAI/pDAI PulseX pair	0xa73104df763f216e7e3a7cdc5b7acc4c00c31f4d
DAI Balancer linear pool	0x6667c6fa9f2b3fc1cc8d85320b62703d938e4385

Raw evidence is under `artifacts/pulsechain-portal-pricing-20260930/`: `discovery.json`, `details.json`, `oracle-discovery.json`, `quotes.json`, `feed.json` and `small-cycle.json`. These include requests, responses, chain ID, block hash and repeat block-hash checks. `analysis.json` records exact derived amounts and hashes of all six input files. The experiment manifests are `Research/experiments/pulsechain_portal_*_20260930.json`.

The current aDAI implementation address read from the proxy is `0x7effd7b47bfd17e52fb7559d3f924201b9dbff3d`; its explorer-published source is saved as `aDAI_implementation_source.json`. The current wrapper implementation is `0x715d73a88f2f0115d87cfe5e0f25d756b2f9679f`; its saved source is `artifacts/parb-validation-20260928/wav3_impl_source.json`. Published source was inspected, not independently recompiled. `pulsex_pair_source.json` documents the 29/10,000 swap fee. No V2 aDAI conclusion is inferred from failed V2 reserve calls or this bounded pair search.

Reproduce the derived figures without network access, choosing a fresh output:

```
python Research/scripts/analyze_pulsechain_portal_prices.py \
--evidence artifacts/pulsechain-portal-pricing-20260930 \
--output artifacts/pulsechain-portal-pricing-check/analysis.json
```

Related studies: [receipt and validator checks \(Appendix 49\)](#), [linear-pool limits \(Appendix 16\)](#), [Portal settlement accounting \(Appendix 17\)](#).

Appendix 49: Existing aTokens, new Portal issuance and validator access

Contents of this appendix

Cap, outstanding balance and event history are different evidence	326
A complete cycle inside one transaction or block.....	327
Restore the exact receipt route	327
Balancer's role on source and destination chains.....	328
Current validator-access finding	328
Reproduction references.....	329

Source record: docs/studies/analysis/pulsechain_receipt_and_validator_checks_20260930.md.

Existing aTokens, new Portal issuance and validator access

30 September 2026. Read-only interpretation at PulseChain block **27,673,362**, hash
0x5c20cef7102df1d2519b28fbca0a884876ed59ecb9c01fd6b4ff033e9a1f5886.

Cap, outstanding balance and event history are different evidence

Three different questions must not be collapsed into the statement that a bridge is closed:

Question	Evidence	What it establishes
What new unbacked issuance is allowed?	getConfiguration(asset) decoded through getUnbackedMintCap	A configured limit on outstanding unbacked issuance for that destination reserve
What unbacked amount is recorded now?	getReserveData(asset).unbacked	A current accounting balance, not the cumulative amount ever issued
Did issuance occur in a historical interval?	MintUnbacked logs at the specified address and block range	Occurrence or bounded absence of that event, not every transfer or bridge route

The fresh configuration reads for DAI, USDC, USDT, LINK and WBTC at the examined Aave V3 pool 0x87870bca3f3fd6335c3f4ce8392d69350b4fae2 each decode to **unbackedMintCap = 0**. Their recorded unbacked amounts also read zero. These were separately decoded fields, not an inference from missing logs. The implementation-slot read still identifies 0xf1cd4193bbc1ad4a23e833170f49d60f3d35a621, matching the saved verified source used for the predicate analysis.

The verified implementation's ReserveConfiguration places the 36-bit cap at bit 176. Its BridgeLogic.executeMintUnbacked adds the proposed positive amount to the reserve's outstanding unbacked balance and requires the result to be no greater than $\text{unbackedMintCap} \times 10^{\text{decimals}}$. With cap zero and outstanding balance zero, a positive new mintUnbacked cannot satisfy that predicate under this configuration. Supply validation and bridge-role authorization also apply. The cap is an outstanding-principal ceiling, not a count of mints or a cumulative lifetime transfer quota.

A complete cycle inside one transaction or block

With an authorized caller, compatible receipts, adequate pool output and sufficient unbacked-cap headroom **at the mint instruction**, this sequence can execute in one transaction:

```
mintUnbacked(M) → deposit aDAI into waDAI → swap waDAI for actual pDAI  
→ backUnbacked(M, fee) → retain only the surplus after settlement and costs.
```

The mint first increases recorded unbacked principal. Backing later reduces it and supplies actual underlying to the Aave reserve. If both complete in the same transaction, the ending unbacked balance can equal its starting value. If any uncaught call reverts, the transaction's earlier state changes and logs revert as well. Separate transactions within one block follow the same order of checks but do not have transaction-wide atomicity across the entire sequence.

The cap applies to the intermediate balance, not only the final net change. An actual zero cap at the mint instruction therefore cannot be bypassed by backing later in the transaction. Conversely, the block-end configuration read does not establish every earlier instruction's configuration. Authorized cap increases before a mint and reductions after backing could, in principle, leave a zero cap at block end. Determining whether that occurred requires ordered transactions, configuration events and, where needed, traces.

A successful cycle still emits `MintUnbacked` and `BackUnbacked`. Closing in one block does not erase those logs. A bounded event census can therefore test this possibility without relying on a nonzero end-of-block balance. The earlier event census and current configuration reads remain distinct.

Settlement also needs a complete cash account. If $M=100$ and the wrapper sale produces 99.8 pDAI, full backing needs an additional 0.2 pDAI, plus any supplied fee. If it produces 100.2 and the backing fee is zero, 100 can back the issue and 0.2 remains before other costs. Sending the 100 into Aave supports its reserve and the existing receipt claims; it is not destroyed, but cannot also be recorded as freely available pDAI elsewhere. A capped partial backing call can succeed while leaving residual unbacked principal, so transaction success alone is not proof of full settlement.

Separately, the earlier bounded event study found no `MintUnbacked` or `BackUnbacked` events for this V3 pool in its searched post-fork range through block 27,659,393. This fresh getter batch does not extend that event census. A zero current outstanding balance alone would not prove that no issuance occurred and was later backed. [Earlier event audit \(Appendix 2\)](#), [accounting and verified-source analysis \(Appendix 17\)](#).

Restore the exact receipt route

An existing aToken can be transferred without creating new unbacked issuance. Ordinary underlying-funded Aave supply, a transfer of an existing receipt, a gateway release from existing receipt inventory, and new Portal unbacked issuance are distinct ways of obtaining a destination holding. Each has its own checks. A zero unbacked-mint cap does not prohibit every one of them.

The destination sequence for the examined V3 DAI receipt is:

```
exact PulseChain V3 aDAI at 0x018008bfb33d285247a21d44e50697654f754e63  
→ receipt-deposit function of compatible waDAI vault  
→ waDAI at 0x098256c06ab24f5655c5506a6488781bd711c14b  
→ Balancer linear pool 0x6667c6fa9f2b3fc1cc8d85320b62703d938e4385  
→ available DAI or a supported nested pool route  
→ acquire pDAI/U and supply system-owned liquidity.
```

An aToken from a different chain or Aave version cannot be substituted merely because its symbol is aDAI. A bridge must deliver the exact accepted destination receipt, or an additional compatible conversion must do so. A remote receipt represented by a newly created token is not automatically the existing aToken.

There is successful public destination evidence:

Operation	Successful transaction
0.618019167905589973 waDAI → 5.200713616007117708 DAI	29 June 2026
6.679668 waUSDC → 43.641283 USDC	4 September 2026
Four USDT/waUSDT/BPT swaps, with 26,235,296.082510 gross USDT entering and the same gross USDT leaving	10 August 2026

The last transaction demonstrates large gross movement with nested LP claims; it does not show net creation of that quantity of USDT. These transactions establish functioning local conversions, not their inputs' source-chain provenance. The saved Portal experiments instead ran on a modified local fork with assumed permissions and changed configuration; their hashes must not be presented as public cross-chain transfers.

The [wrapper inventory \(Appendix 24\)](#) establishes exact DAI/USDC/USDT wrapper and Balancer relationships. V3 LINK and WBTC reserve existence is a separate observation. The complete aLINK/aWBTC bridge → wrapper → funded Balancer output route has not been established by this evidence set. Those are candidate extensions, not proven copies of the DAI integration.

Balancer's role on source and destination chains

Balancer's boosted-pool design combines local AMM liquidity with supported yield-bearing positions, including Aave receipts wrapped into static shares. Nested LP tokens can connect underlying assets through batch swaps. That can make the same principal useful for market access and lending yield, with withdrawal liquidity and claims accounted for. It is capital efficiency, not automatic borrowing or principal duplication. [Balancer V2 boosted pools](#).

An operating source-chain Balancer deployment could similarly receive source assets and issue LP shares. A compatible bridge or lender could accept a claim on those shares. This requires the actual source deployment, share custody, destination representation and accepted conversion. No universal Balancer mechanism transfers the principal of every L2 LP position into the existing PulseChain Vault.

Balancer V2 also offers local flash loans from available Vault balances. A profitable local cycle can borrow, trade, repay principal and applicable fee, then retain the surplus in one transaction. A delayed cross-chain withdrawal cannot normally serve as the later repayment leg of that same atomic loan. Collateralized leverage lasting longer requires a lender that accepts the specific collateral and bears liquidation and liquidity constraints. [Balancer V2 flash loans](#).

Current validator-access finding

The copied rollup at 0x5ef0d09d1e6204141b4d37530808ed19f60fba35 returns `validatorWhitelistDisabled() = true` at the same block. Reading its documented secondary implementation slot identifies user logic 0xa0ed0562629d45b88a34a342f20deb58c46c15ff. The explorer-published source for that implementation contains:

```
require(isValidator[msg.sender] || validatorWhitelistDisabled, "NOT_VALIDATOR");
```

The proxy's archived compilation includes an older user-logic source with a different modifier. That compilation bundle must not be mistaken for the current secondary implementation. The storage-slot read and implementation's own published source resolve the distinction. The source is explorer-verified; this study did not independently compile and bytecode-match it. [Current user-logic published source](#).

The current observation does not prove that a challenger has all required data, funds or functioning dependencies. It does show that the sampled validator allowlist condition is disabled. The stronger claim that changing a private RPC categorically excludes new challengers is not supported.

RPC access, chain data availability, contract admission and successful challenge execution are different layers. Nitro's deterministic execution and parent-chain batch publication can allow independently operated nodes when the required data and configuration are available. Missing private feed access can obstruct convenient operation without proving permanent impossibility of reconstruction. AnyTrust data-availability assumptions require their own assessment. No evidence here establishes who changed an endpoint, when, or whether all necessary data are unavailable. [Nitro architecture](#).

Reproduction references

The fixed-call manifest is `Research/experiments/pulsechain_liquidity_discovery_20260930.json`. Raw RPC records, reserve-field summaries, implementation-slot requests/responses and published-source responses are under `artifacts/pulsechain-liquidity-expansion-20260930/`. The batch records its block hash and verifies it again after collection. These are one-RPC observations. No live mint, transfer or challenge was submitted.

Appendix 50: Validation of the restorative pDAI thesis

Contents of this appendix

Finding.....	330
The strongest supported formulation	330
Claims a critical reader can test	331
Most valuable next evidence	331
Method and reproduction	332

Source record: docs/studies/analysis/pulsechain_restorative_review_20261004.md.

Validation of the restorative pDAI thesis

4 October 2026. Five model-based analysts, two review rounds. Saved evidence was inspected and revalidated offline. Blockchain work remained read-only; no new chain calls or execution simulations were performed.

Finding

FEI is optional and unusually easy to demonstrate. Its prominence reflects the clarity of its evidence, rather than proven strategic importance. Its public conversion links existing FEI to existing pDAI units. A route beginning with pDAI already needs that pDAI funded; the FEI step redistributes resources. Direct pDAI use, Synthetix recycling, compatible issuer conversions and other funded branches can serve complementary purposes.

All five final reviews accepted the conditional research account with no remaining priority revision in their checked scope. They retained coordinated intent to benefit pDAI and PulseChain, normalized eligible-stable accounting U, and distributed execution without an assumed central treasury or persistent local stockpile. Agreement establishes review convergence, not observed intent, hidden key custody, private-L2 deployment or sustained dollar capacity. The [maintained white paper \(main paper\)](#) contains the resulting explanation.

The strongest supported formulation

The network has three distinct jobs: obtain usable resources, convert and distribute accepted assets, and deliver the declared price service. Portal supplies destination credit with separate backing obligations; issuers and wrappers create compatible claims or coins; funded markets pay the counterassets. An independent executable dollar reference determines the payment quantities for the dollar policy.

The proposed corrective loop buys below the target, meets demand with target-price offers, and restores the inputs consumed. Funded conversions can transmit support through eligible connected markets within their costs, capacity and timing. Under accessible, funded bid/offer terms at declared sizes and deadlines, supported orders receive the declared band. This conditional result does not require every optional protocol to operate, nor imply that every standalone AMM quote stays at the target.

Target-price sales recover payment capital and recirculate pDAI. Backing payments, productive positions, venue refills and resale-support commitments compete for those resources across wallets. Shared 3pool or FRAXBP cash counts once. As parity improves, arbitrage spreads can shrink; recycled capital, opposite flows, new funded inputs and collectible net earnings remain possible replenishment sources. Future interest from retired debts cannot finance later backing.

Claims a critical reader can test

Question	Defensible claim and evidence boundary
Is FEI indispensable?	No. Exact pDAI delivery can serve offers or eligible reserves directly. FEI is a demonstrated optional relative-parity interface, with finite cash and continuing claims.
Does assigning U establish \$1?	U compares normalized eligible assets. Dollar-linked execution additionally needs an independently executable reference and funded payment quantities.
Does receipt surplus guarantee complete buyback?	Coverage requires permitted conversion into paid assets at the required scale, after costs and competing obligations, by the deadline. Nominal units alone do not establish coverage.
Does profitable arbitrage necessarily improve pDAI's price?	Complete closing flows determine direction. The observed profitable sDAI cycle slightly lowered its closing pool's PLS/pDAI ratio. Its net proceeds could finance separate support; that later use remains to be traced.
Does supply adjustment refill every pool?	Selling issued tokens consumes counterassets; deposits and ordinary supply create different claims. Reverse trades, funded redemption/contraction or new inputs restore the required side. Pool-specific rebase accounting remains necessary.
Does a successful local route prove fresh L2 funding?	It proves the observed local capability. Fresh L2 funding separately needs authenticated destination acceptance, exact output, payer and source obligations. Instructions can coordinate spending without delivering assets.
Does one unusable branch invalidate the architecture?	It excludes that branch's contribution at the observed checkpoint. A selected alternative must independently satisfy the same asset, authority, funding and refill requirements.
Is the research complete?	The discussed architecture and route families are covered. Exact integrated deployment, sustained paid capacity and intent attribution remain open evidence questions.

These distinctions strengthen the argument while retaining its favorable purpose assumption. Each proposed benefit can be tested through actual paid outcomes, useful reserve refills and reconciled claims rather than inferred from protocol names or nominal balances.

Most valuable next evidence

Follow one complete **funding → permitted use → acquisition or support → target-price circulation → restored capacity** sequence through whichever viable branch occurs. Do not predetermine FEI as the destination. Record exact tokens, executors, payers, net outputs, continuing claims, deadlines and the specific depleted resources replenished.

A local sequence can establish local service. The private-L2 extension needs a fresh authenticated destination entry; canonical Ethereum entry and exit require their own representation, custody and release checks. Repeated dollar-responsive execution should use the paper's predeclared finite benchmark, including failed requests and remaining next-order capacity. Broader flows, longer horizons and source outages need separately stated coverage.

Method and reproduction

The analyst roles were mechanism, evidence, economics, network completeness and first-reader clarity. First-round assessments used the same frozen paper separately. Second-round assessments checked the revised explanation and two narrow operation/interest wording corrections. Ten individual reports preserve findings and checked scopes. reviews shared models, environment and sources; no calibrated probability, exhaustive independent audit or actual human comprehension test is claimed.

The root reran two existing offline evidence verifiers: **2,111 equal-value checks** and **132 arbitrage/Maker checks** passed. These reconcile retained receipts, traces, amounts, debt and accounting; they do not update live state or cryptographically authenticate the chain. Source hashes, original link preservation, the opening assumptions and document structure were checked separately.

The explicit batch is [pulsechain_restorative_team_review_20261004.json](#). Frozen papers, ten assessments, source index, verification outputs and hashes are under `artifacts/pulsechain-restorative-team-review-20261004/`. The final reviewed freeze is `paper-final-review.md`, SHA-256 `4f1faf0655507e98668c7bcdeca671042b5c459eec8c1dd6b8337752a5d6e276`; the maintained paper also links this assessment. Earlier frozen evidence and research identities remain unchanged.

Reproduce the saved-evidence checks into new output files:

```
python Research/scripts/analyze_pulsechain_equal_value_routes.py \
  --input artifacts/pulsechain-equal-value-routes-20261003 \
  --output /tmp/pulsechain-restorative-equal-value.json
python Research/scripts/analyze_pulsechain_arbitrage_and_shutdown.py \
  --input artifacts/pulsechain-arbitrage-and-shutdown-20261003 \
  --prior artifacts/pulsechain-equal-value-routes-20261003 \
  --output /tmp/pulsechain-restorative-arbitrage.json
```

Appendix 51: End-to-end routes: issuance, arbitrage and settlement

Contents of this appendix

Scope and common test.....	333
Issuance authority before any route.....	334
1. Ordinary escrow return.....	335
2. Native-source custody and reverse representation.....	336
3. Token-specific burn/mint or shortfall issuance.....	337
4. Aave Portal: local advance with settlement.....	338
5. Relayer or inventory bridge.....	339
6. Collateralized lending or local issuance.....	340
7. Local market paths and nested lending receipts.....	341
8. LP, lending and reward income.....	342
Large sales: complete local correction.....	343
Large purchases: complete local correction.....	344
What L2 participation can earn.....	344

Source record: docs/studies/analysis/pulsechain_route_atlas_20260929.md.

End-to-end routes: issuance, arbitrage and settlement

29 September 2026. Conditional route classification for the pDAI thesis. Diagrams describe obligations and asset flows, not instructions to alter live contracts. Historical observations and hypothetical integrations are separated.

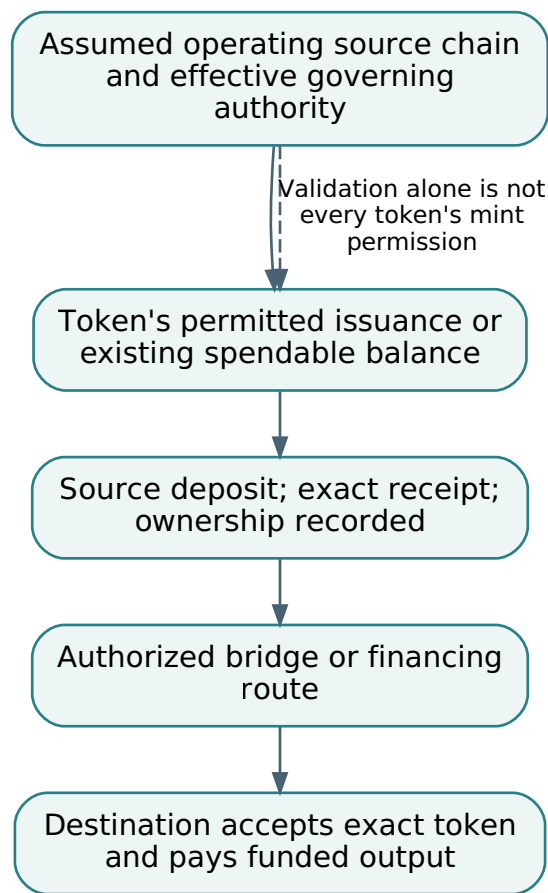
Scope and common test

There is no finite list of every possible future protocol or strategy. This atlas covers the accounting families in the maintained bridge, wrapper and liquidity inventory. A particular route is complete only when it identifies:

1. Source asset, execution environment, issuance rule and legitimate authority.
2. Receipt and destination token addresses, unit conversions and exact custody.
3. Message verification, timing, destination permission and capacity.
4. Funded output at every exchange or financing step.
5. The closing asset, all debt/settlement obligations and remaining inventory.
6. Net profit for each participant and the consolidated system, separately.
7. The effect on executable pDAI buying and selling at the stated size.

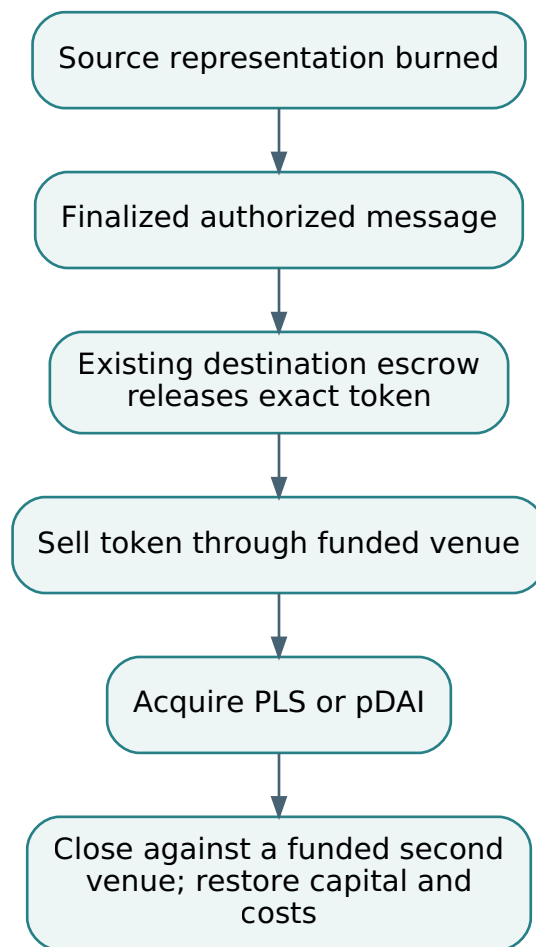
An arrow is conditional on these checks. Funding a pDAI position is not itself arbitrage: retaining pDAI leaves price exposure. A closed arbitrage cycle returns the starting asset with a surplus after costs and restores borrowed principal and collateral as specified. Market making may earn income over time without being risk-free or ending each transaction with restored inventory.

Issuance authority before any route



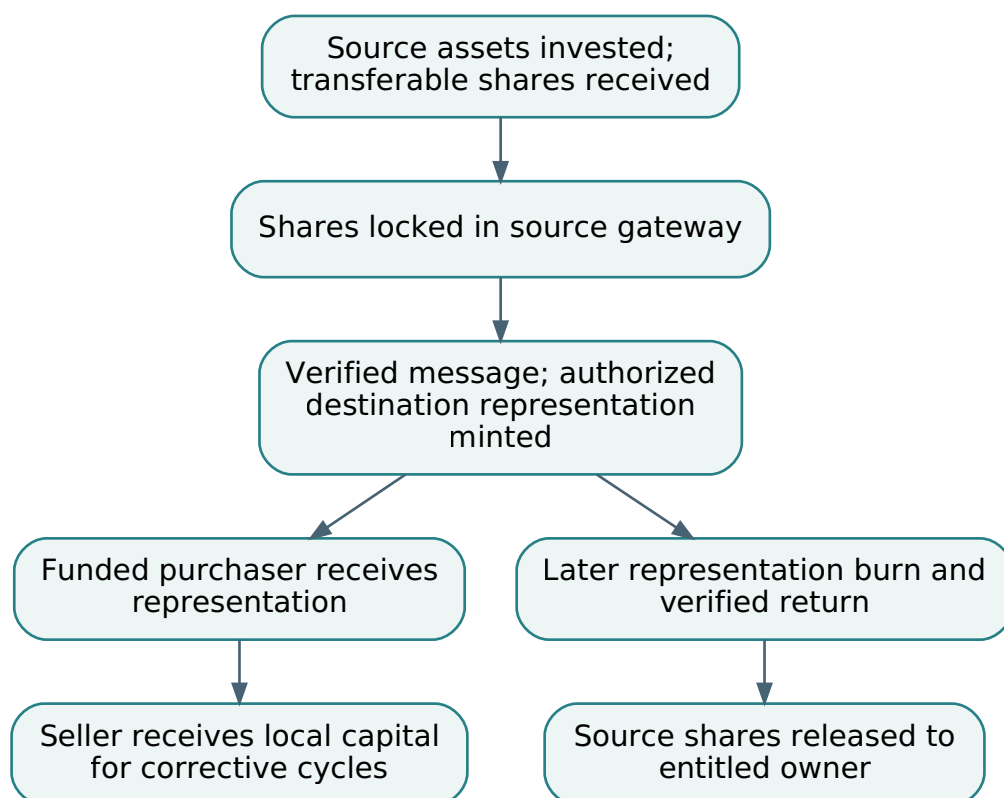
The strongest theoretical scenario can grant arbitrary source balances and relevant governance permissions. Even then, destination acceptance, message rules and output funding remain separate premises. An accepted accounting balance can fund a finite trade without establishing sound backing. Neither this assumption nor a copied L1 contract proves a functioning source L2. [Historical baseline \(Appendix 13\)](#), [cross-chain comparison \(Appendix 7\)](#).

1. Ordinary escrow return



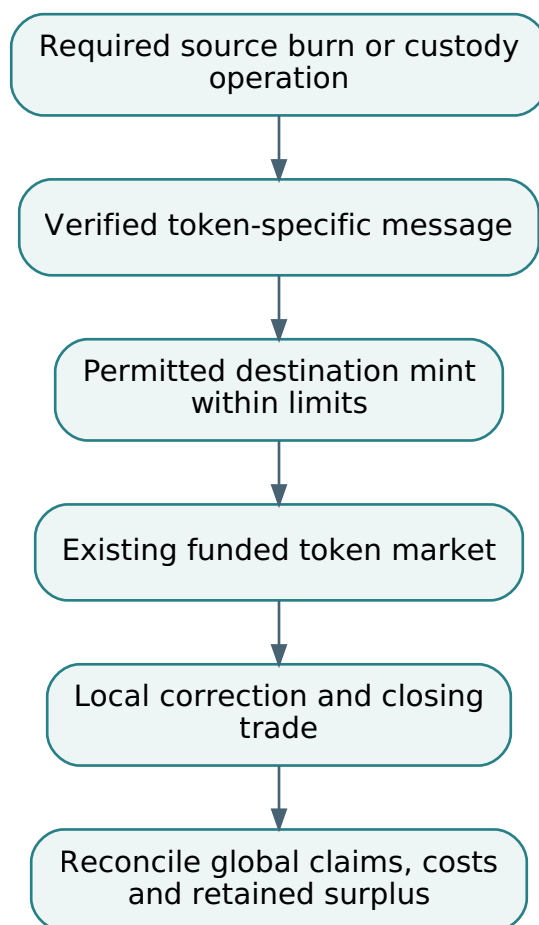
An empty escrow blocks the release unless that exact integration has an additional authorized source of tokens. Validator control does not turn a transfer call into minting. The audited standard gateway selections for OUSD, mUSD, eUSD and FRAX are not proof of native-source reverse minting of those existing tokens. Their individual destination issuer paths are separate. [Issuance inventory \(Appendix 4\)](#).

2. Native-source custody and reverse representation



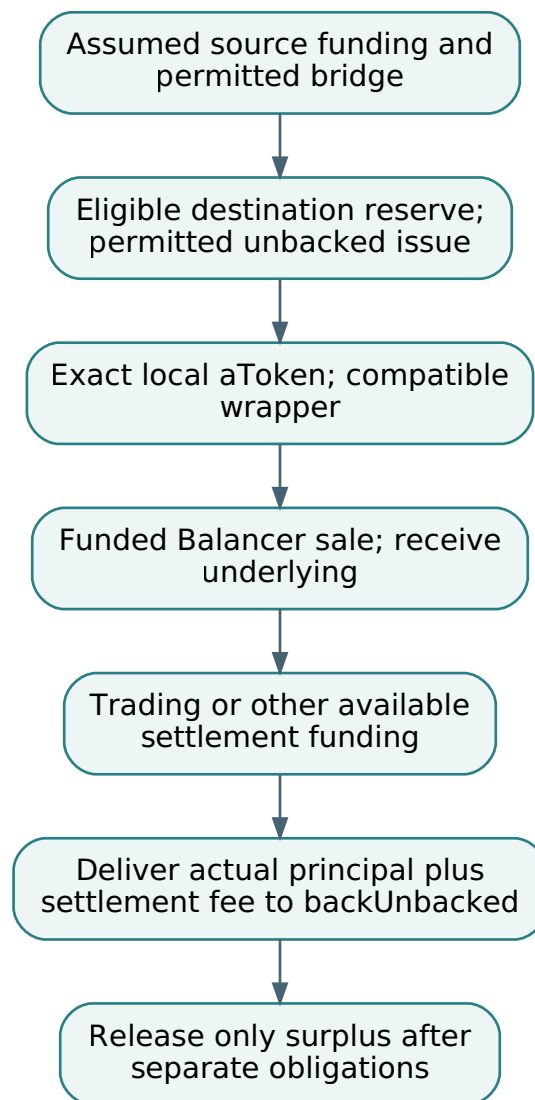
This mobilizes a claim while its underlying investment stays in place. The purchaser, not the original depositor, now owns that exclusive represented claim. Outright sale needs no repayment by the seller but transfers an asset; its proceeds are not automatically income. Reverse ARB is a historical architectural example, not proof that every stablecoin recognizes the same representation. [Bridge models](#) (Appendix 13).

3. Token-specific burn/mint or shortfall issuance



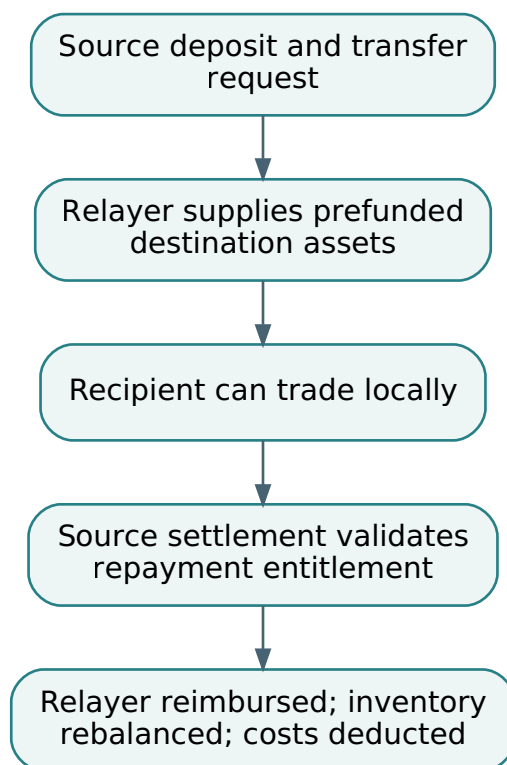
Destination ARB and GRT mint observations establish particular mechanisms; GRT's allowed escrow shortfall and ARB's reverse custody have different source accounting. Do not infer universal mint authority from either. LPT has a source-level shortfall branch, while the observed LPT withdrawal used escrow. [Specialized history \(Appendix 20\)](#), [GRT reconciliation \(Appendix 12\)](#).

4. Aave Portal: local advance with settlement



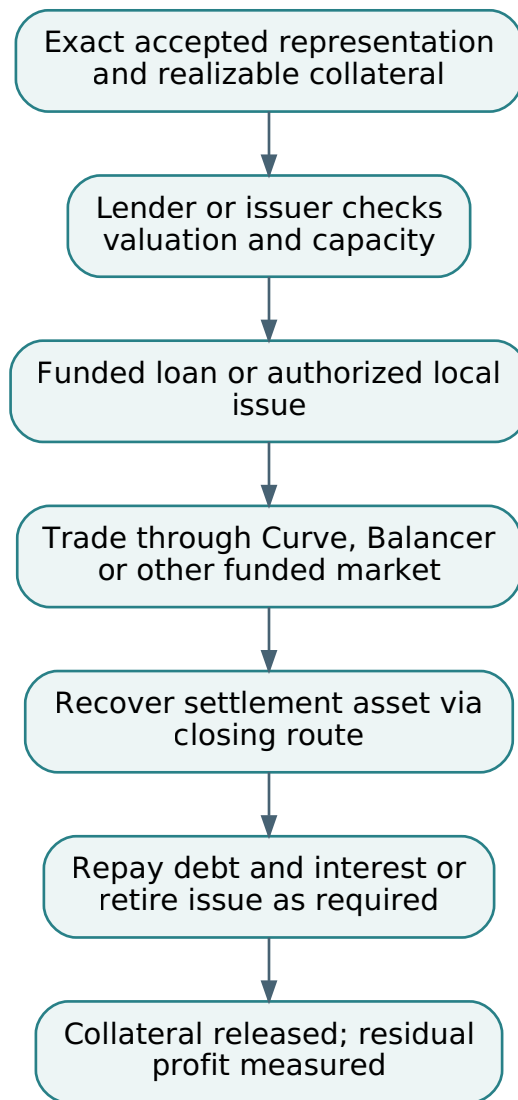
Issuing m and recovering q local underlying settles from those proceeds alone only when $q \geq m + F$, with settlement fee F . Ordinary loan repayment does not cancel the same unbacked balance. The audited stablecoin unbacked caps were zero. Historical local modified-fork experiments do not establish a public source-to-destination Portal cycle. [Portal ledger \(Appendix 17\)](#).

5. Relay or inventory bridge



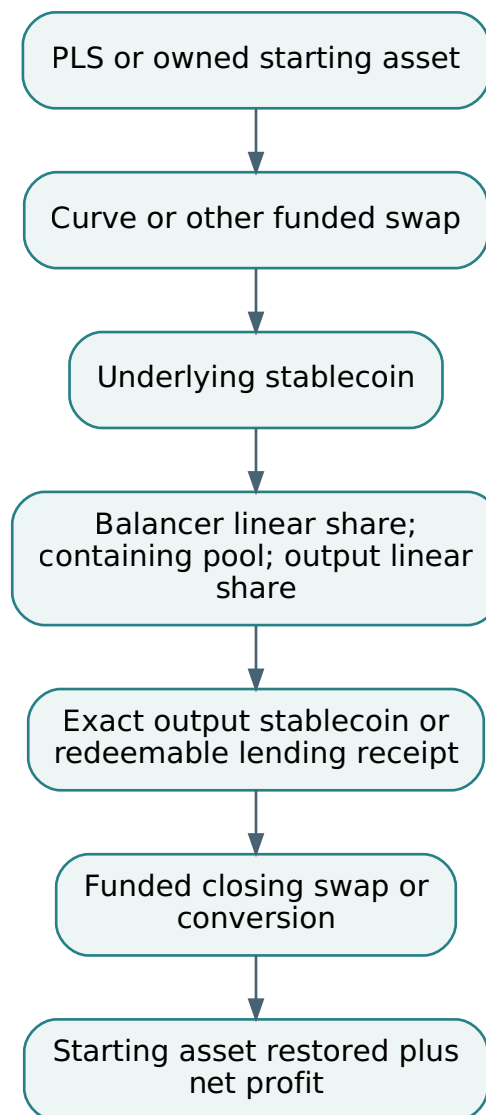
Relayer income is a fee for funding, execution and settlement risk. It is not new destination principal. Across is the studied example; a Frax-specific bridge/Ferry route would require its own deployed identity and settlement proof, not merely the shared label of a bridge. Bridge revenue can finance support only to the extent controlled, collected and not committed elsewhere. [Across comparison \(Appendix 7\)](#), [HubPool accounting \(Appendix 14\)](#).

6. Collateralized lending or local issuance



Fraxlend is an asset/collateral pair, while authorized FRAX AMO issuance is a separate capability. Reserve issuance requires the exact basket assets and usable conditions; a registered symbol is insufficient. Origin, mUSD and other issuer candidates need their own compatible collateral and redemption accounts. Borrowing to hold pDAI is a funded position, not a completed profitable cycle. [Reserve inventory \(Appendix 18\)](#), [Frax/Convex study \(Appendix 47\)](#).

7. Local market paths and nested lending receipts



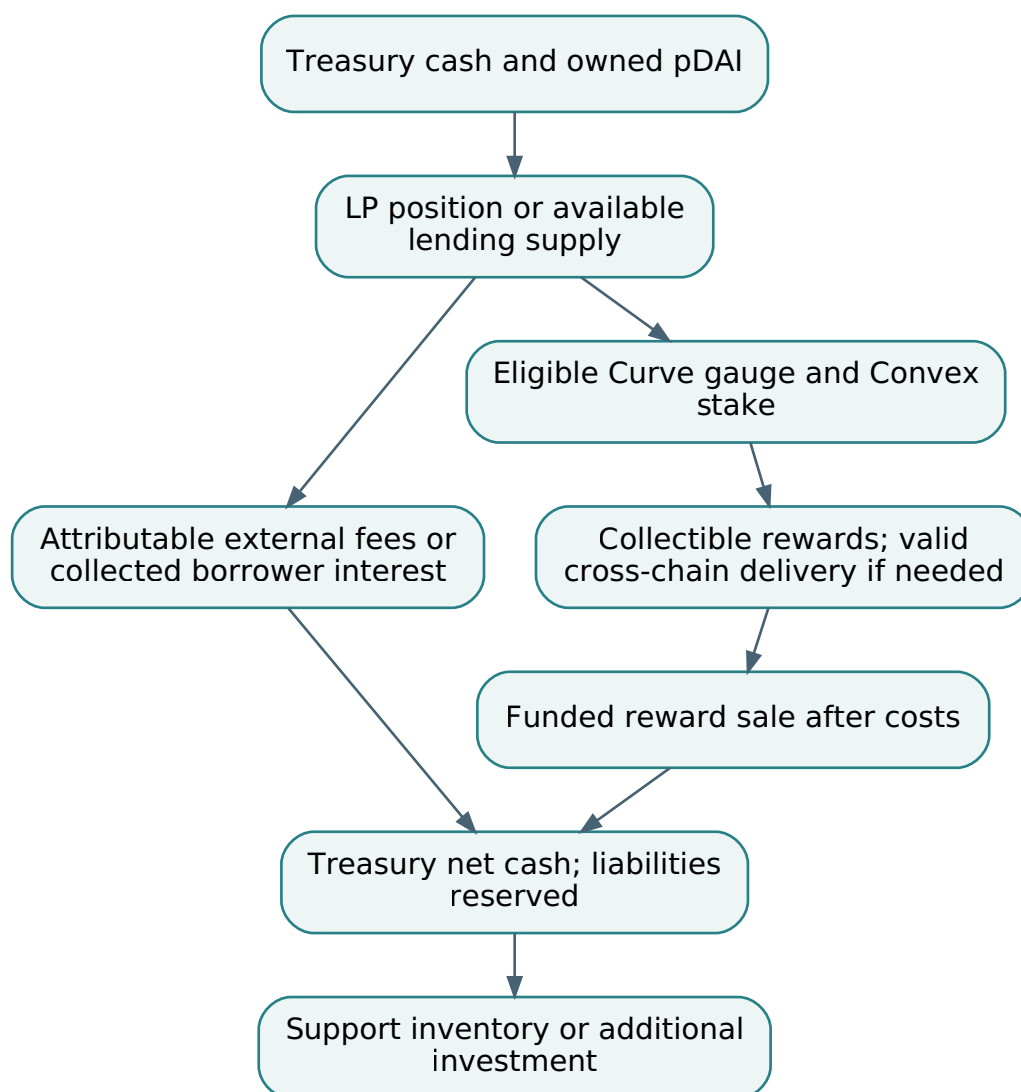
This includes direct pDAI/PLS trades, stablecoin triangulation, FRAX/3CRV underlying routes, other metapools, and Curve/Balancer discrepancies. Each additional pool, rate conversion and withdrawal needs an executable quote. Wrapped-to-main swaps consume main-token capacity. Moving the same asset through more wrappers does not multiply its reserves. [Wrapper inventory \(Appendix 24\)](#), [Pool accounting \(Appendix 16\)](#).

For the illustrative graph with FRAX, pUSDC, pDAI and PLS and its five named pools, the three simple cycles through PLS are:

```
PLS → pDAI → pUSDC → PLS
PLS → FRAX → pUSDC → PLS
PLS → pDAI → pUSDC → FRAX → PLS
```

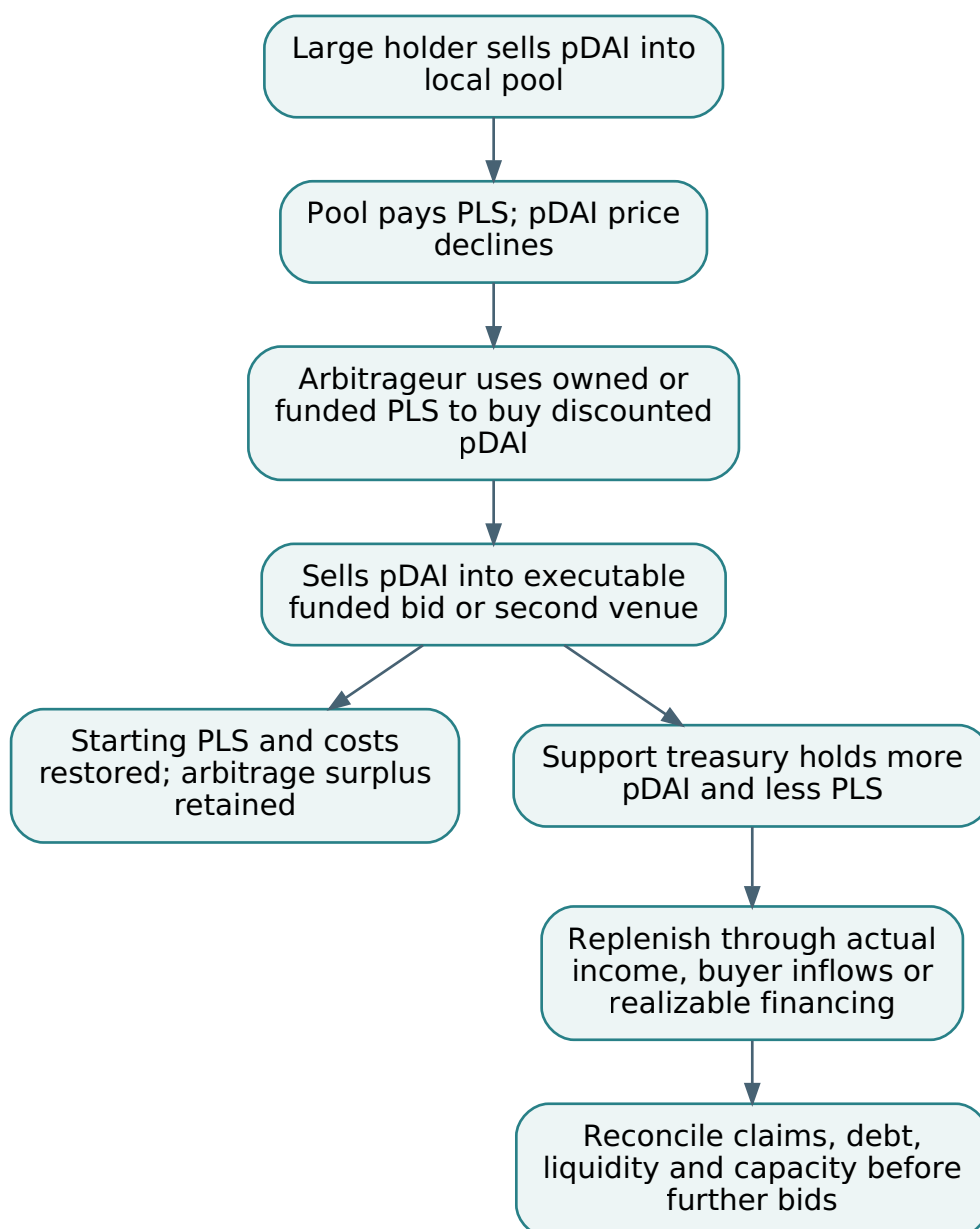
Each has a reverse orientation. Two-leg reversal in one pool is a separate fee-losing round trip. This enumerates topology, not profitable execution. Repeated or combined walks change state, share reserves and may impose gas, credit and timing costs; they cannot be valued by adding independent initial quotes. The [cycle experiment \(Appendix 47\)](#) tests selected sizes and states, not all strategies in every possible deployment.

8. LP, lending and reward income



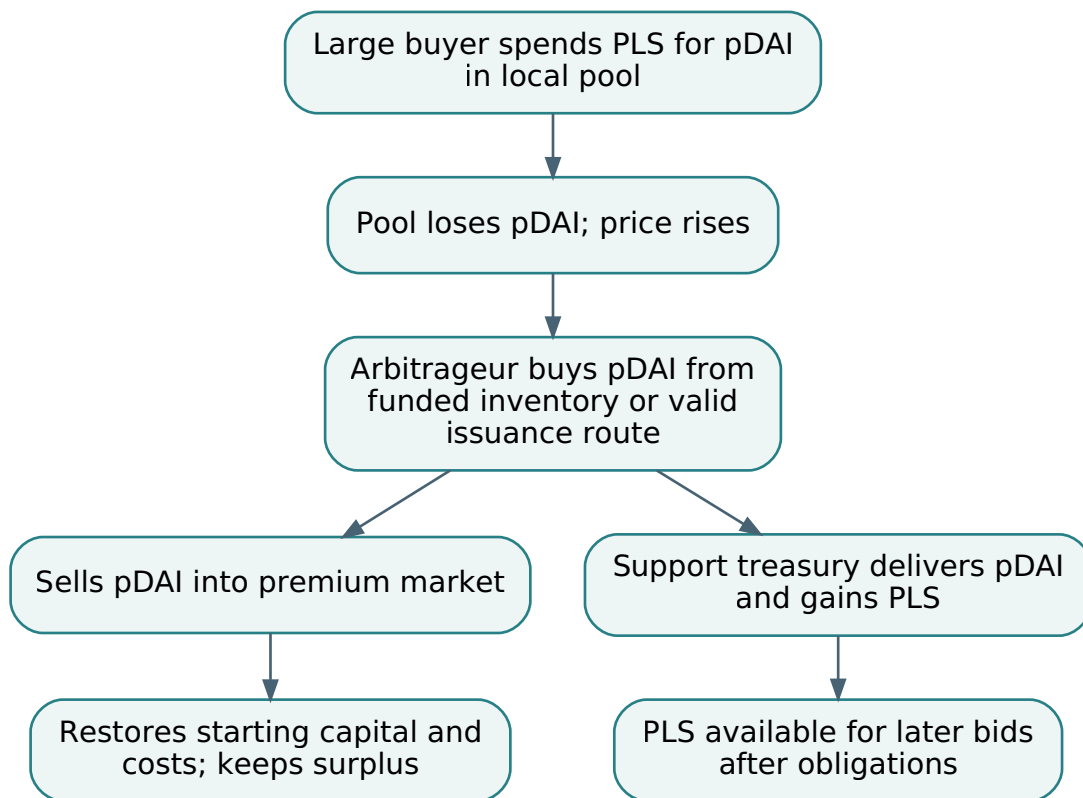
This is productive deployment and possible income, not necessarily arbitrage. Reward bridging does not transport interchangeable LP principal. A share in future interest is not presently withdrawable underlying. Each asset belongs in one consolidated account with its associated claims and obligations. [Frax/Convex and reinvestment \(Appendix 47\)](#).

Large sales: complete local correction



This closes the trader's account. It does not close the treasury's remaining pDAI exposure. A standing bid is one proposed explicit closing venue, not a historically established redemption facility. Without it, the complete alternate market route must supply equivalent funded demand. Source funding arriving after the seller needs payment cannot finance that earlier payment without an advance or existing buffer.

Large purchases: complete local correction



Available pDAI can come from retained inventory, a funded third-party sale, a permitted withdrawal of owned claims, an escrow release with inventory, or an exact-token issuance route that actually exists. Borrowing pDAI adds a later return obligation. Minting an unrelated stablecoin is not another branch for minting pDAI. Purchases return system-held pDAI to outside buyers, so accumulation need not be monotonic.

What L2 participation can earn

A source-chain operator earns only the revenue attached to its actual role: issuance revenue if any after obligations, lending income after funding and losses, bridge/relayer fees after reimbursement, owned LP income, or its own closed trading profit. An external arbitrageur's profit is not automatically L2 or treasury revenue. If the same system owns both sides, consolidate them; internal payments disappear and remaining asset, liability and outside-cash changes determine the result.

The route diagrams are composable. Every connection remains conditional on its own finite capacity, and mechanisms drawing on the same reserve are not independent sources of funding. Adding a future protocol requires adding its specific edge and settlement rules, not declaring the atlas universally complete.

Appendix 52: PulseChain Safe governance and rollup controls

Contents of this appendix

Scope and pinned evidence.....	345
What a Safe takeover would actually require	345
Verified GnosisDAO module execution and owner changes.....	346
Synthetix protocolDAO and Optimism remain separate	346
What the copied Optimism BondManager does.....	347
Arbitrum: verified administration transfers and open validator admission.....	347
Implication for the peg thesis.....	348
Reproduction and limits.....	349

Source record: docs/studies/analysis/pulsechain_safe_governance_and_rollup_controls_20261001.md.

PulseChain Safe governance and rollup controls

The copied GnosisDAO Safes and Arbitrum administration show substantial, verified postfork control changes. The inspected Synthetix protocolDAO and Optimism administrator Safes retain their inherited public configurations. These findings support a concrete module-based control pathway, but do not establish one group's control of every Safe, Synthetix council or messenger.

Scope and pinned evidence

All blockchain work was read-only: getters, runtime code, storage, logs, transaction receipts, block headers and historical traces. No transaction was signed or broadcast, and no administrative action was simulated.

Current reads use PulseChain chain 369, block **27,685,402**, hash

0x085116030d59ed608fabf0595a92b60a76e766282251553249a4d359c92db59d. Inherited comparisons use Ethereum block **17,232,999**, hash 0xe5667f9cafb115b1dcfc36b0b569a5601f01dfa000698140a1298ef4f87e2e37. Postfork event ranges start at block 17,233,000; rollup role deployment queries also include earlier March 2023 history. Evidence and gap retries are frozen in artifacts/pulsechain-safe-governance-rollup-20261001/.

What a Safe takeover would actually require

Safe owners normally need the configured signature threshold. An **enabled module** has a separate execution route. An oracle-backed module can therefore execute a transaction without assembling the ordinary owner signatures, provided its own acceptance conditions pass. This is why owner lists alone are insufficient. The [Safe module documentation](#) describes this authority; the deployed module's code supplies its actual conditions.

Controlling GnosisDAO, SafeDAO or a Zodiac factory does not automatically control unrelated Safes. A concrete link must exist through an enabled module, owner, delegated role, implementation or authenticated relay. Casting a governance vote through a Safe does not change its signing configuration.

Verified GnosisDAO module execution and owner changes

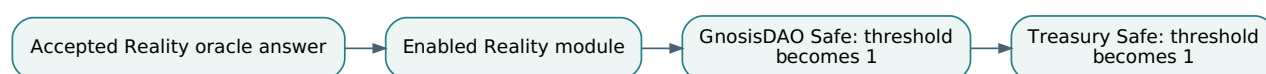
Two linked Safes matter:

Safe	Inherited configuration	Configuration at the current pin
GnosisDAO Safe 0x0da0c3e52c977ed3cbc641ff02dd271c3ed55afe	3 of 5; Reality module enabled	1 of 6; no enabled modules
Treasury/active-management Safe 0x849d52316331967b6ff1198e5e32a0eb168d039d	3 of 10	1 of 11

On **10 June 2026 at 22:13:15 UTC**, transaction [0xcbd5...867d](#) called the copied Reality module [0x0d70332ceb7f3c94b061cda48327891e3449a9e1](#). Its historical trace reads the oracle's accepted answer, bond and finalization state, then calls the Safe through `execTransactionFromModule`. The Safe adds [0xa8ed85b1d441da00270ecc6dc3600f3cba9d4a03](#) and reduces its threshold to one. Receipt-verified `AddedOwner`, `ChangedThreshold` and `ExecutionFromModuleSuccess` logs corroborate the trace.

Five and a half minutes later, [0xdfcd...7e93](#) disabled the Reality module. The new owner was then sufficient for ordinary Safe execution under the reduced threshold.

On **18 June 2026**, [0x9c53...2b37](#) used the GnosisDAO Safe, which was already an enabled module of the treasury Safe, to add the same address there and reduce the treasury threshold to one. Its receipt and six-call trace confirm the nested route.



Later history matters. In September, all five original GnosisDAO owners were removed; the added owner was replaced by [0x147b0ef96881ba4f8fcc18d50a14f1231f5021f5](#); the five original owners were then restored. Consequently, a present-day owner-list comparison conceals intermediate removals. At the pin, **each of six owners can authorize alone**. The treasury likewise has 11 eligible owners at threshold one. Neither Safe is presently exclusive to the newly added address.

The old Reality module's current owner/avatar/target remain the GnosisDAO Safe. Its inherited settings still specify a two-day question timeout, two-day cooldown, seven-day answer expiration and 100-token minimum bond. It is currently disabled at that Safe. A separate Zodiac Roles module remains enabled at the treasury; its four registered members match the inherited snapshot. Zero default roles do not prove that those members have no explicitly selected role permissions.

This proves changed execution authority and the route that changed it. It does **not** establish a legitimate Snapshot vote, malicious intent, stolen keys or common control with the five previously examined research wallets. Those attribution questions need additional evidence.

Synthetix protocolDAO and Optimism remain separate

The Synthetix owner is the **protocolDAO Safe** at [0xeb3107117fead7de89cd14d463d340a2e6917769](#), still **5 of 13**, nonce 2,884, with no enabled modules. The Optimism AddressManager owner is [0x9ba6e03d8b90de867373db8cf1a58d2f7f006b3a](#), still **5 of 7**, nonce 81, with no enabled modules. The selected postfork configuration-event census found no changes for either Safe; retries recovered initially unavailable ranges.

One Optimism signer is a nested Safe at `0x42d27eea1ad6e22af6284f609847cb3cd56b9c64`, still **2 of 2**, nonce 10, with no enabled modules. Its two owners and public configuration match the inherited state. Controlling that nested Safe would supply one parent signer, not the five required by the parent.

Inspected singleton/fallback runtime code matches inherited code. Guard/fallback storage was checked; the legacy Safe 1.1.1 does not implement the later 1.3 guard mechanism. Configuration equality does not establish who currently holds the private keys.

The [council study \(Appendix 68\)](#) expands this to Treasury, Ambassador, Grants and incentive Safes, the 17/22-seat distinction and the old L1 Spartan NFT. Their copied public controls also match the inherited comparison. Current private-L2 council membership and the Spartan bond-custody Safe remain unresolved.

Synthetix's own authenticated owner relay is an independent administration path: protocolDAO can send messages to suitably owned L2 contracts without controlling the global Optimism messenger. Earlier verified transactions demonstrate that architecture. They do not prove that permissionless callers executing an inherited proposal own the messenger.

What the copied Optimism BondManager does

The BondManager is `0xcd626e1328b41fcf24737f137bcd4ce0c32bc8d1`. Its locally verified implementation of `isCollateralized(address)` is a stub that returns whether the address equals the AddressManager's `0VM_Proposer`. It contains no functioning deposit-based admission system or elected validator committee.

At the pin, the registered proposer is `0x473300df21d047806a082244b417f96b32f13a33`. The BondManager returns true for that address and false for all five examined research wallets and the zero address. The proposer, sequencer and selected registry dependencies match inherited values; the selected AddressManager mapping/ownership event census found no postfork changes.

The StateCommitmentChain's append logic still checks this admission predicate. Its 12,592,000-second sequencer publishing window does not bypass BondManager authorization. The configured fraud-proof window is **604,800 seconds: seven days**. Depositing funds, electing SNX councillors or controlling an unrelated Safe does not satisfy this copied proposer check.

SCC/CTC storage containers are registered dependencies whose owner identifiers resolve through the same AddressManager, rather than independent hidden owner Safes. SCC/CTC counters match the inherited snapshot. Previously verified postfork withdrawals used inherited April/May 2023 roots; they do not establish a new privately asserted Optimism state root.

Arbitrum: verified administration transfers and open validator admission

The copied rollup at `0x5ef0d09d1e6204141b4d37530808ed19f60fba35` has the UpgradeExecutor at `0x3fffbadaf827559da092217e474760e2b2c3cedd` as owner. Its normal executor role still includes the L1 timelock `0xe6841d92b0c345144506576ec13ecf5103ac7f49` and the security-council Safe `0x3666a60ff589873ced457a9a8a0aa6f83d708767`. The security council remains **9 of 12**, with inherited owners, nonce 13 and no enabled modules.

However, normal-role equality omits the upgrade path:

Verified change	Evidence and resulting authority
24 February 2025	<code>0xd358...e6f0</code> executes through the L1 timelock and UpgradeExecutor. It reduces the minimum timelock delay from 259,200 seconds to one second and transfers proxy-admin ownership to <code>0xb8698cfffdaaf26ca21fe8aa560c18cc2b2f5996</code> .
19 June 2026	<code>0x8850...2953</code> changes the copied Inbox proxy's admin to <code>0x63a1c70df12ef3d3de17487179465c021de690f4</code> . The current admin slot confirms it.
19 July 2026	<code>0x1fb1...aeb5</code> changes the UpgradeExecutor proxy's admin itself to <code>0xb8698cfffdaaf26ca21fe8aa560c18cc2b2f5996</code> . Current storage and receipt-verified AdminChanged corroborate the trace.
1 September 2026	<code>0x7d7f...d5cc</code> routes another timelock execution through the executor and changes multiple proxy admins. The Bridge, SequencerInbox and ChallengeManager admin slots now contain <code>0xa0b347117a878da9942fa5c31b87c773a2969913</code> .

Both emphasized admin addresses have empty code at the pin. This is concrete externally owned address control of important upgrade paths. It can be more consequential than membership of the unchanged executor/security-council roles. It does not prove common ownership of all these addresses or that an upgrade has actually altered the currently inspected implementations. The older bridge ProxyAdmin's owner has subsequently changed again; its current getter returns `0x353e1d055ae5352d1b8c9d3f0ffef1e3a4b96294`, so the February recipient must not be described as its current owner.

The timelock's scheduling path authenticates the bridge and registered L2 timelock. Permissionless execution of a ready operation is not permission to schedule one. The original chain's governance authority and the copied system's accepted messaging therefore need separate treatment.

The validator whitelist changed from enabled to disabled between adjacent blocks **17,322,437 and 17,322,438**, on **21 May 2023**. The accepted transaction `0x90ca...272b` invokes `removeWhitelistAfterFork()` through a helper. The recorded selector matches a separately hashed signature. The deployed user logic permits this when the chain ID has changed: it is a public fork escape, not proof that its caller held the rollup admin key. Binary search located this transition; it is not an exhaustive proof that no other transitions occurred elsewhere.

There are 18 active staker addresses at the pin, compared with two in the inherited snapshot. Several later admin recipients also occur among those active stakers. This establishes address overlap, not 18 independent operators or exclusive validator control. Although the five original research wallets still return false from the stored `isValidator` list, the disabled whitelist removes that list as the admission barrier. Staking, assertion ancestry, timing and challenge conditions still apply.

Implication for the peg thesis

The evidence now supports a stronger statement: **module-based changes occurred in copied Gnosis governance, and externally owned addresses acquired key copied Arbitrum administration paths**. Those are concrete mechanisms for changing local authority. Across's previously verified oSnap execution provides another protocol-specific example.

The evidence still does not show that those routes control the inspected Synthetix protocolDAO or copied Optimism proposer. Arbitrum admission/upgrade authority, Optimism source-root acceptance, Synthetix collateralized issuance, destination synth minting and Curve liquidity are distinct dependencies. Bridged supply must pass the relevant route for the exact destination token. One verified control path does not supply every missing link.

Reproduction and limits

Getter manifests are `Research/experiments/pulsechain_safe_governance_*20261001.json`; history manifests include nested controls and rollup roles. `probe_pulsechain_routes.py` was always supplied explicit getter manifests. `collect_pulsechain_events.py` records query errors separately from receipt verification. Original failed records remain preserved; retries close the recorded root Safe, rollup-role and nested Reality query gaps. `pulsechain_safe_governance_followup_20261001.py` records missing receipts/headers and exact proxy-admin storage reads. `pulsechain_rollup_admission_boundary_20261001.py` records the historical adjacent-block whitelist comparison. Historical `trace_transaction` and `trace_block` results explain execution already recorded on-chain.

No source RPC was available for the private copied L2. Private-key custody, current private-L2 elections, Snapshot authorization for the Gnosis execution, the Spartan bond-custody Safe and common-control attribution remain unverified. These limits are specific remaining evidence requirements, not assumptions that the proposed architecture is impossible.

Appendix 53: Skeptical economic review — independent first round

Contents of this appendix

Verdict	350
What survived independent challenge	350
Ranked revisions and additions	351
Counterexamples to stronger readings	352
Independent arithmetic	353
Useful numerical witness to add.....	354
Falsifiability and open questions.....	354
Provisional epistemic judgments	354

Source record: docs/studies/analysis/pulsechain_skeptic_economics_round1_20260929.md.

Skeptical economic review — independent first round

29 September 2026. Read-only review of the maintained theoretical paper, Portal accounting, Across accounting, pool-capacity studies and the existing numerical batch. No main-paper edits, transactions, privileged calls or state-changing simulations. New arithmetic was evaluated independently with 80-digit Decimal precision; original experiment identities and outputs were preserved.

Verdict

The narrow conclusion survives: **a transferable, valuable source claim can finance a destination purchase when a funded counterparty accepts it, and a sufficient purchase can raise a specified market quote.** The paper does not establish the significantly stronger original thesis that validator/admin control and enormous nominal source balances generate that funding, or a self-financing route to a durable one-dollar pDAI price.

The present conclusion is technically defensible because the hardest economic premises are explicit. Its vulnerability is informational: assuming destination purchasing liquidity and usable backing can make the positive result read as “a funded purchase can be funded.” To become more informative, it needs a numerical source-to-destination balance-sheet witness and a clear distinction between what the bridge contributes and where spendable value originates.

What survived independent challenge

- Exact token identity, claim conservation, local backing and destination cash are separated correctly. The source receipt can remain invested while its ownership claim moves, provided the original holder cannot spend it again.
- The report correctly distinguishes a per-trade execution value from a marginal price, and both from all-holder redemption. The PLS dollar reference is explicitly independent of pDAI.
- Portal settlement and ordinary loan repayment are different. The documented accounting requirement $Q \geq M + F$ is correct when proceeds alone fund full backing and F is the actual fee paid. Current-source documentation is not substituted for the pinned historical implementation.

- The Portal companion discloses modified-fork tests that left residual unbacked balances instead of treating a test's "passed" label as full settlement.
- Current zero Portal capacity, zero sampled HubPool stable cash and minimal sampled wrapper output liquidity are disclosed. These contradict an assertion of immediate execution of those exact routes, not all future configurations.

Ranked revisions and additions

Rank	Issue	Required strengthening
Critical	Nominal collateral is not executable value	Define source token units, redemption rights, collectible assets, valuation discount and destination bid separately. "Fully collateralized" needs a unit and valuation basis. A receipt backed by a trillion self-issued units can be fully backed in units and have no funded buyer.
Critical	Generic possibility is stronger in wording than its economic content	State that the mathematical construction proves a market intervention, not the financing leg. Give one internally consistent numerical financing witness, explicitly stipulated, without presenting it as current deployed behavior.
Critical	No durable-target mechanism is proved	Specify a horizon, maximum seller/redemption flow, acceptable price band, intervention assets, replenishment source and withdrawal/liquidation terms. Otherwise "sustain" has no falsifiable size or duration.
High	Administrative control may weaken collateral credibility	A controller able to rewrite source balances can also change scarcity and claim enforceability. This does not logically force value to zero, but market value and haircuts cannot be inferred from control. State the assumed constraints, guarantees or willingness of funded buyers.
High	Purchases, retained inventory and settlement are conflated by readers	Add a consolidated before/after ledger for every actor: source owner, custodian, destination issuer, lender/relayer, AMM and purchaser. Show released claims as well as assets acquired.
High	Defending an above-target sale requires pDAI inventory; defending below requires buying assets	An instruction to buy below and sell above supplies neither inventory automatically. Revenue arises only if trades/financing actually produce it; changing PLS value changes the required ratio.

Rank	Issue	Required strengthening
High	Price support is not necessarily arbitrage	A profitable arbitrage needs enforceable conversion or simultaneous price differences after costs. A discretionary one-dollar aspiration does not itself make a purchase below one dollar profitable.
High	A debt payoff can release selling supply	Paying borrowers' debt may permit collateral withdrawals. Repaying illiquid reserves can make aTokens redeemable. Model those sellable claims instead of treating every pDAI sent to Aave as permanent supply removal.
Medium	Current facts are dispersed	Put a short route-specific blocker table beside the positive conclusion, including DAI/USDT supply caps already exceeded in the pinned state, zero Portal caps and quoted pool headroom. Do not imply changing one cap suffices.
Medium	A dollar-reference value can be a mark rather than liquid proceeds	State the reference venue/time/size and its own depth. A finite PLS quote remains a valid local target, but mark-valuing a very large PLS output does not prove it can be sold externally for that dollar amount.
Medium	No independence among route budgets	Keep shared-base-pool warning, and extend it to common collateral, relayer credit and common settlement reserves. Sum only separately spendable balances.
Medium	Claim hierarchy needs an exit stress case	Identify which party receives the ultimate collateral on redemption, whether collateral can be rehypothecated, and what happens if source withdrawals halt or receipts are diluted.

These are improvements to the interpretation and economic specification, not requests to construct administrative execution sequences.

Counterexamples to stronger readings

Perfect bridging without value. A source contract escrows one trillion newly issued units and a destination contract mints exactly matching receipts. All messages and token accounting succeed. There is no funded bid and no lender accepts the collateral. Destination purchasing power is zero. This disproves “valid lock/mint plus enormous units implies sufficient buying power.” It does not contradict the paper's explicit funded-acceptance premise.

Oracle acceptance without new aggregate wealth. A funded lender accepts a receipt at a declared value and lends its existing PLS. The borrower can spend that PLS. If liquidation cannot recover it, the lender bears the shortfall. Technical acceptance can redistribute existing purchasing power even where collateral is economically weak; it does not establish repeatable solvency. The report should permit this distinction rather than assume either immediate failure or creation of new economic resources.

Portal at par. Ignore fees and interest. Issue 100 aDAI, sell compatible wrapper claims for 100 pDAI, and back the full 100. The operator has no remaining pDAI profit; the AMM holds claims of 100 while Aave holds cash of 100. If the holder redeems those claims, that cash can become market inventory again. A completed cycle can move liquidity and affect prices, but it neither burns 100 pDAI nor creates 100 free units for repeated intervention. A discount or fee requires additional capital; a genuine premium may finance a surplus until available pricing opportunities change.

Across inventory exhaustion. Funded relayers deliver 10 million PLS locally and are paid in remote receipts. If nobody replenishes local PLS, total further local delivery stops after that inventory is spent even with unlimited remote receipts. This is consistent with healthy bookkeeping and fails the claim of unlimited destination replenishment from source-unit balances alone.

Competing market. Push one pDAI/PLS pool to one dollar while another deep market offers pDAI materially cheaper. Funded arbitrageurs can sell into the first pool. A one-pool construction does not specify the cost of raising the network's executable market, which includes inventory across accessible venues.

Circular collateral value. Suppose the source receipt ultimately redeems only into the same pDAI whose one-dollar value the intervention is intended to create. Marking that receipt at the hoped-for dollar target and borrowing against it assumes the valuation result. Such credit may still be mechanically available from a funded lender, but its solvency is exposed to the same pDAI decline. Use executable stressed realization values and identify who bears the loss; do not count an internal oracle mark as an independent source of support.

PLS repricing. Hold the pDAI/PLS quote fixed while the independent PLS dollar reference halves. pDAI's dollar-reference execution value halves. PLS-denominated intervention assets and a dollar target are not a fixed-value reserve system.

Independent arithmetic

Using the paper's inputs $x=1,000,000$, $y=1,000,000,000$, $s=0.0001$, $k=xy$ and $T=1$, A separate calculation obtained:

- Marginal-target input: 2,162,277,660.168379331998893544... PLS.
- Subsequent-sale target for $q=1,000$: 2,167,281,613.012984016139208948... PLS.
- That sale returns exactly 10,000,000 PLS within high-precision arithmetic.
- After that sale, the resulting marginal dollar-reference price is approximately 0.996842718386987, illustrating that the successful finite quote does not promise that subsequent trades enjoy the same price.

Both formulas and supplied numerical results are correct. The assumption of a fee-free pool is explicit, so absence of fees is not an arithmetic defect. The standard constant-product and marginal-price model is supported by the [Uniswap v2 primary white paper](#), which also explains that arbitrage links a pool's price to external markets. No real-world funding conclusion follows from that isolated calculation.

Useful numerical witness to add

A theoretical balance sheet could stipulate a transferable source claim with independently specified realization value; escrow it; issue an exclusive representation; have a funded destination buyer pay an explicit amount of PLS; and apply those PLS to the existing market example. The buyer's PLS must fall by exactly the purchaser's proceeds and the buyer must acquire the claim. Source investment stays put; ownership changes. This would establish composition without pretending that the bridge itself creates the PLS.

A lending variant must add the borrower's liability, lender receivable, liquidation haircut, cash funding and repayment/rollover assumption. A local stablecoin issuer variant must add the issuer's basket obligation and the funded market willing to exchange its output. Giving each mechanism an equivalent complete ledger is more useful than listing additional protocol names.

Falsifiability and open questions

A conditional claim is not falsified by a premise failing in today's deployment. It is weakened if its premises are mutually inconsistent, rely on the outcome being assumed, or omit a liability that defeats the claimed budget. Distinguish:

1. Can an exclusive source claim survive transfer and redemption without double allocation? A conflicting claim or uncontrolled dilution defeats that model.
2. What exact spendable asset pays the purchaser, who held it before, and what do they receive? No funded counterparty defeats that financing instance.
3. Are all obligations maturing before/during the purchase funded after costs? Check every chronological prefix, not only the final net balance. A negative intermediate cash balance defeats the sequence without explicitly funded credit. Cross-chain waiting periods cannot be presumed atomic; shared reserves cannot be paid out to two routes simultaneously.
4. What source assets or income ultimately service long-term debt? Unfunded maturity defeats repeatability even if today's purchase succeeds.
5. What seller-flow/horizon does the proposed budget cover? Selling beyond that bound defeats the stated price-support guarantee.
6. What happens to pDAI purchased for backing when receipt holders redeem or borrowers withdraw unlocked collateral? These flows can reverse supply removal.
7. Which independently observed PLS reference defines the target, and what trade size and duration does "one dollar" describe?

Provisional epistemic judgments

These are broad subjective judgments about support for explicit propositions, not calibrated market probabilities, frequencies from repeated experiments, or a forecast that any wallet will act. Logical accounting findings matter more than small numerical differences. reviews share a record and are not independent samples.

Proposition	Central judgment	Plausible judgment range
H1: Some authorized source receipt can finance a local purchase if it is valued and a counterparty is funded	97%	93-99%
H2: The paper establishes a complete route executable at the pinned deployed snapshot	2%	0-5%

Proposition	Central judgment	Plausible judgment range
H3: Validator/admin control and nominal copied collateral alone suffice for the target	1%	0-3%
H4: The paper establishes a durable one-dollar target under selling and redemption	1%	0-3%
H5: The finite AMM existence calculation is valid under its explicit assumptions	99.5%	99-100%

H2 and H4 ask whether the paper establishes a result. A low score is not the probability that a hidden system exists or that a future price move occurs. H3 has a direct zero-bid counterexample. The nonzero ranges acknowledge possible misreading of proposition wording, not a statistical estimate of on-chain fate.

Round two must challenge these findings after the team's first-round results are shared; this file preserves the independent first-round review.

Appendix 54: Skeptical economic review — second round

Contents of this appendix

Verdict on the revised argument	356
Attacking the shared objections	357
Assessment of the proposed 1,000-share witness.....	357
Remaining requirements for a stronger economic claim	358
Can meticulous operators mitigate the roadblocks?.....	358
Revised subjective judgments	359
Final revised-text check.....	359

Source record: docs/studies/analysis/pulsechain_skeptic_economics_round2_20260929.md.

Skeptical economic review — second round

29 September 2026. This round began only after reading the shared first-round synthesis and both the protocol and evidence reviews. It challenges the other reviews and this review's own objections. The maintained paper is not edited within this review. No public transactions or state-changing simulations occur.

Verdict on the revised argument

The proposed complete numerical witness strengthens **conditional composition**. It does not strengthen the claim that the audited contracts presently supply the required financing or that control of nominal source balances generates new aggregate purchasing resources. Those are distinct propositions.

One correction to overly restrictive skeptical language is essential: **independently valuable or solvent collateral is not necessary for every finite purchase**. A funded AMM, lender or counterparty can mechanically accept a weak claim and release actual purchasing assets. That can produce a purchase; the counterparty then bears claim, valuation and recovery risk. This does not require wrongdoing, and no motive conclusion follows. The relevant technical conditions are actual acceptance, valid transfers and funded output; economic solvency and repeatability require additional conditions.

The first-round “oracle acceptance without new aggregate wealth” counterexample already recognized this distinction, but “valuable source claim” in its opening verdict was unnecessarily restrictive. H1 now uses **accepted and funded**, not an intrinsic-value requirement. Comparisons with round-one scores must state that wording clarification.

Attacking the shared objections

“No independent value means no possible purchase” is false. A pre-funded pool can exchange its inventory under contractual pricing rules even if the incoming receipt's eventual recovery is poor. A buyer can also knowingly pay for a speculative claim. No independent dollar redemption right is necessary for the paper's PLS-reference objective. This rebuts a blanket impossibility claim while preserving the need to identify whose spendable inventory moves.

“No public source RPC means the theoretical architecture fails” is false. the source system can be stipulated for an existence construction. Its absence from the evidence restricts historical conclusions. The revised paper should continue treating source operation as a premise rather than an audited fact.

“Today's blocked Portal means all bridge paths fail” is false. Zero caps and supply-cap barriers apply to that exact pinned route and configuration. Other families can remain logically coherent, or a future parameterized scenario can be analyzed. Neither observation licenses a claim of current execution.

“All debt must be repaid before any purchase” is false. Financing can leave a long-term debt outstanding while supporting a present purchase. Immediate settlement and intermediate cash needs must be covered; longer obligations belong in solvency and repeatability analysis. A ledger should not make borrowing impossible by subtracting all future principal from present spending proceeds.

“A theoretical proof must prove actual market demand” confuses propositions. It can stipulate a funded buyer for a conditional existence result. What it cannot do is present that stipulated buyer as discovered evidence that the particular source balances can command this funding in actual markets.

“Every hypothetical can be rescued by another assumption” remains a real criticism. The answer is to freeze a complete specified example, show all its interfaces and balance changes, and state exactly what it demonstrates. An explicit example is more informative than an unbounded list of possible upgrades.

Assessment of the proposed 1,000-share witness

A consistent witness may start with 1,000 freely transferable source shares owned by the purchaser, a source protocol holding their invested assets, and a separate destination buyer holding at least 2,167,281,613.012984016139208948... PLS.

1. Source custody receives the 1,000 shares. The purchaser loses possession of them; the source protocol's invested assets do not move.
2. After the assumed valid message and authorized issuance, the purchaser receives exactly 1,000 destination representations. Custody supports that exclusive represented claim, not a second unrestricted source spending right.
3. The funded destination buyer pays the stipulated PLS amount and receives the 1,000 representations. Its PLS decreases by exactly the seller's proceeds.
4. The purchaser spends those PLS into the illustrative AMM and receives its approximately 684,271.838699 pDAI. The pool receives those PLS and loses that pDAI quantity. The buyer retains the source-position representation.

The original source investment remains where it was. Ownership and local cash have changed hands. This has no accounting contradiction and does not require assigning a dollar NAV to each source share. It is a stipulated trade, not a claim that anyone will buy those shares for that amount.

The buyer's starting PLS is the ultimate purchasing resource in this witness. Calling the bridge “capital mobilization” is reasonable because it makes a claim accessible to a destination buyer. Calling it generation of that PLS or evidence that the copied source balances are sufficient would be incorrect. The same price intervention could have been financed by a buyer accepting another asset; the L2-specific contribution is portable ownership and the assumed integration.

A lending or local-issuer variant is not proved merely by this outright-sale witness. Each needs its own liabilities, accepted assets, funded payout and maturity/redemption treatment. It remains valid to classify those as conditional families without claiming an equally complete numerical witness for every one.

Remaining requirements for a stronger economic claim

- Define a price-support horizon, sale size, band and seller/redemption envelope. A finite reserve can support a bounded scenario; no present analysis establishes support for arbitrary duration and flow.
- Check cash sufficiency at every sequence prefix and after any compulsory settlement. Cross-chain timing and one reserve shared by multiple routes matter.
- Treat the independent PLS reference as a valuation input with its own depth and variability. A local PLS output can satisfy the stated convention without proving that its entire marked dollar value can be realized elsewhere.
- Preserve distinction between quoted stablecoin ratios and dollar-reference purchasing power. Several copied assets trading at par with one another do not supply an independent one-dollar anchor.
- Stress source-share dilution, withdrawal blocks, claim discounts and common controller risk. These affect solvency and sustainable funding even when a first sale executes.
- Treat price-sensitive pDAI collateral as correlated support. Its hoped-for dollar value cannot independently prove the funding needed to create that value.
- Model claims returning to market after Aave redemption or collateral release. Deposits, repayments and escrow commitments do not permanently burn pDAI.
- Keep mint-capacity field widths, market inventory, actual buying budget, collectible assets and contract permission as separate measured quantities.

Can meticulous operators mitigate the roadblocks?

Yes, many are contingent configuration or financing problems. Assuming the relevant effective authority and a successfully executable governance process, permitted cap changes, supported collateral/oracle integrations and compatible implementation changes can enable transactions that fail in the fixed audited state. A credible funding commitment, fresh liquidity contribution, funded relayer or valid debt repayment can also address a cash shortage. These are possible mitigations, not observations that the named group has completed them.

Authority is component-specific and bounds matter: an unrelated immutable receiver or an issuer controlled elsewhere does not change because another protocol's governance votes. Upgrades and parameter changes may involve delay, coordination, new code risk and changed third-party exposure. Raising a limit alone neither supplies funds nor proves every other predicate now succeeds.

Do not turn the economic objection into the overbroad statement that governance can never create token cash. If authority actually extends to the exact token's issuer and that token permits issuance, authorized minting can create spendable units of that token. Changing an Aave cap or an oracle mark by itself does not perform that issuance. Creating units or claims does not guarantee their exchange into

the stipulated PLS budget, preserve the PLS reference price, or eliminate outstanding redemption obligations. Changes to native issuance or consensus would be a separate architecture, not a consequence of ordinary application governance.

Consequently there is no economic proof that all roadblocks are permanent. There is also no proof that meticulousness alone removes them. A finite accepted exchange is the easier proposition. Repeated solvent financing and a sustained price require actual resources, specified cash-flow bounds and parties willing or mechanically obligated to bear the resulting exposure. This distinction addresses remediability without supplying administrative execution instructions.

Revised subjective judgments

These assess the stated propositions, not future price or wallet behavior. Ranges express judgment sensitivity, not statistical confidence intervals. The reviews are correlated and their scores must not be multiplied.

Proposition	Central judgment	Judgment range
H1: Some authorized source receipt can finance a local purchase if accepted and funded	99%	97–100%
H2: The paper establishes a complete route executable at the pinned deployed snapshot	1%	0–3%
H3: Validator/admin control and nominal copied collateral alone suffice for the target	0.5%	0–2%
H4: The paper establishes durable one-dollar execution under selling and redemption	0.5%	0–2%
H5: The finite AMM existence mathematics is valid under its explicit assumptions	99.5%	99–100%
H6: Given effective relevant authority and executable governance, some identified configuration/integration roadblocks could be mitigated	99%	97–100%

H1 rises from 97% because the explicit ledger closes the previously underspecified financing handoff and because its wording no longer incorrectly suggests an intrinsic-value prerequisite. H2/H3/H4 receive slightly lower judgments after cross-review clarified the independent current barriers and economic counterexamples; this is not new negative market evidence. H5 is unchanged after the independent 80-digit recalculation. The low H2/H4 figures mean **not established by the paper**, not an estimated probability that a hidden or future route can exist.

Final revised-text check

The review examined the revised main text, including its numerical source-share sale ledger, exact-identity discussion, asynchronous custody model, additional Aave supply-cap barrier, intermediate cash conditions, circular-valuation warning, independent-reference qualification, finite-sale arithmetic and

proof-obligation table. These revisions address the material first-round objections. The ledger expressly excludes transaction costs and requires separately funded real costs, so the exact AMM budget is not silently spent twice on fees. No new accounting inconsistency was found.

The remaining limitation is substantive and now disclosed: the witness assumes the destination bid. It does not derive it from observed source collateral, control, or a demonstrated deployed route. Additional financing families remain conditional and need their own full ledgers for stronger proofs. The governance question is addressed above; H6 is new in round two and is not a before/after measurement of first-round confidence.

H2 is about the unchanged pinned deployed snapshot. Conceivable later governance or funding improvements do not turn that historical proposition into a proved one, but they can make a different future scenario executable. No forced consensus or future-price prediction follows from the judgments.

Final-text supplement: The review separately examined the added section 9, "Could governance remove the roadblocks?", after its insertion. Its remediability table, exact token-issuer qualification, distinction from native PLS issuance, and separate funding/solvency requirements accurately reflect this review. No correction is requested. Main-text SHA-256 at this check:

a9c49fb67d4c22b6d07e226471b52d39b770b3dc535075b6831590e2641f76d3.

Appendix 55: Skeptical evidence review, round 1

Contents of this appendix

Assessment	361
Representative checks performed	361
Strongest evidence supporting the limited thesis	362
Strongest objections to the stronger interpretation.....	363
Ranked clarifications and minimal proof obligations	364
Snapshot and reproducibility limits	365
Confidence judgments for cross-review comparison	365

Source record: docs/studies/analysis/pulsechain_skeptic_evidence_round1_20260929.md.

Skeptical evidence review, round 1

29 September 2026. Internal academic review of the conditional mechanism report and its evidence companion, without public transactions or state simulations. This review reuses earlier bridge studies: the present recalculation is an independent check of saved records, not an independent investigator or a second independent blockchain data provider. Origin/Morpho analysis was not reopened. The main paper was not edited.

Assessment

The paper supports a coherent conditional financing architecture and contains substantial evidence of particular destination-chain actions. It does **not** establish that the complete proposed source-to-pDAI route is executable in the pinned deployed state, or that such a route has established a durable dollar price. The main report generally states these distinctions correctly. Its most important residual risk is that readers mistake a large catalogue of true local observations for evidence connecting those observations into the proposed system.

The strongest positive evidence is the token-specific GRT mint: a mint call, zero-address token event, gateway event and exact supply increase agree. The strongest contrary evidence to an already configured Portal route is the stated zero unbacked caps, additionally constrained by actual reserve cash and supply limits. The strongest source-provenance warning is that the ARB execution's recorded source blocks and timestamp are not plausible chronology. None of these facts disproves every future or separately specified architecture.

Representative checks performed

The reviewed main-paper and evidence-companion file hashes, raw input hashes, and calculated results are saved in artifacts/pulsechain-skeptic-evidence-round1-20260929/. These identify the versions reviewed even if the papers change after round 1. The endpoint remains block 27,659,393, hash 0xc5e582cd902f4bffa3b1aa325b9b564fcd5d6dc0ffa77a03fcd105b65c6390275.

Check	Independently inspected or recomputed result	What this establishes
Four existing manifests	All 66 manifest entries checked match their saved SHA-256 values; some entries reference shared files	Internal file integrity against those manifests, not independent authenticity of the source RPC
Partitioned withdrawal records	Recounted 20 query responses, 106 event identities, 78 transaction hashes and ten gateway addresses from raw records	Agreement with the bounded returned-event census
Historical supply calls	Recomputed 106 predecessor/block pairs from 212 raw calls; only ARB and GRT have nonzero increases	Agreement with reported whole-block supply differences
GRT transaction	Receipt success; actual trace calls GraphToken.mint from the GRT gateway; minted raw amount 62875936675361278837595998 equals zero-address Transfer and supply delta	Actual destination issuance of 62,875,936.675361278837595998 GRT
ARB recorded message words	Raw transaction words decoded as L2 block, L1 block and timestamp all equal 10721873061364271811135909999083699 71006187625255, also the numeric gateway address	Those fields do not establish a plausible dated source-chain deposit
stkAAVE upgrade	Exact Upgraded event is present in the successful saved receipt, with matching transaction and block hash	A particular upgrade occurred; it does not establish an operating GHO issuer
Aave underlying cash	Raw DAI, USDC and USDT balanceOf responses are each zero at the endpoint	These specific reserves have no immediately held underlying cash in those reads
AMM existence example	Recomputed both formulas with 80-digit Decimal arithmetic; stored inputs agree within 5×10^{-61} PLS; modeled average sale value equals 1	The stipulated fee-free, fixed-reference existence calculation is mathematically correct

The GRT transaction is [0xf7ed5701...](#), block 27,434,949, hash [0xbc7a6608cbb7515daaa1464f02cf94fc162b9f89c8c7a0a2859d140c575dd4bb](#). The checked stkAAVE transaction is [0x61686aa7...](#), block 27,252,708, hash [0x322de2bd3690455a5956560adbfdaded946528b1e5d12cce43c3a5e653fa701a5](#). The checked ARB transaction is [0xeebae5bc...](#)

No sampled contradiction was found. This review did not independently replay all 78 traces, inspect every token's implementation, validate every receipt against a header receipt root, or repeat the entire audit against another archival node. It must not be cited as having done those things.

Strongest evidence supporting the limited thesis

1. Destination issuance is demonstrated, rather than inferred from a function name. ARB and GRT have actual destination mint histories, so a universal claim that L1 delivery always requires existing L1 token escrow is false for these integrations. GRT's executed amount is tied to its specific allowance, not an assumed infinite mint facility.

2. The historical reverse-gateway accounting is coherent: custody of a transferable invested receipt can support a destination representation while the underlying remains invested. It is not necessary to withdraw every underlying asset before transferring ownership of its receipt claim.
3. Exact local wrappers and matching pools, actual wrapper sales, an existing eUSD issuer and actual Curve trades establish meaningful destination components. They make a composition hypothesis worth examining, while not proving the components were joined by the claimed funding flow.
4. Specific governance and ownership events rebut the idea that nothing changed after the fork. The paper also correctly distinguishes an executed upgrade, a queued proposal, module ownership and human attribution.
5. The AMM example correctly distinguishes marginal price from the execution value of a finite sale. Under its assumptions, finite purchasing input can move the specified price without buying all pDAI supply.

Strongest objections to the stronger interpretation

Destination acceptance is not source collateral verification. An L1 trace proves which accepted call executed. It does not independently prove a real source lock or burn, the source application's balance sheet, uniqueness of the claim, or economic redemption. With assumed administrative control of the acceptance path, successful destination execution is particularly unsuitable as independent corroboration of what that same path asserted. The ARB metadata makes this limitation concrete. GRT is a strong destination example without being evidence of source investment remaining locked.

The fork supplies L1 state, not every operating L2 database. The paper now states this. Its operating pARB premise is permissible in a theoretical model, but it cannot simultaneously be counted as a retrospective finding. Repository availability before the fork also differs from deployed code and functioning cross-chain configuration at the fork.

Exact-token permissions prevent generalization. ARB's reverse bridge, GRT's limited issuer permission and LPT's external-minter branch are different. They do not issue the named existing stablecoins or Aave receipts. The named addresses' standard-gateway routes support an escrow interpretation. A newly created representation may be a perfectly coherent asset while still being unacceptable to the existing pool or issuer at issue.

The scenario assumes the economically decisive links. Source operation, usable assets, the required authority, compatible integration and actual purchasing liquidity are explicit premises. Once adequate financing and an executable market route are assumed, the existence of a purchasable quantity is unsurprising. The conditional paper demonstrates compatibility of selected accounting models, not discovery of endogenous funding or proof that nominal copied collateral commands the required market value. The finite AMM example is a local price theorem with exogenous PLS purchasing power; it is not a proof that the bridge generates that purchasing power.

Observations of disconnected steps are not a complete cash-flow record. A GRT mint, an mUSD swap, a debt repayment and a governance action by different addresses and dates cannot be composed retrospectively without tracing the assets, quantities, liabilities and intervening changes. Proposal execution does not establish that all necessary configuration changed; historical message emission does not establish counterpart execution. A queued inherited proposal need not be a new plan.

A price intervention does not establish a durable dollar commitment. A stated external PLS reference is a defensible measurement convention, but it adds an external valuation dependency. Cash needed for future sales, changing PLS value, creditor withdrawals and refinancing remains finite. Aave settlement cash backs aTokens; it is not independent free intervention capital. No reviewed record establishes an operational intervention rule, complete balance sheet or ability to sustain specified selling and redemption conditions.

Ranked clarifications and minimal proof obligations

Priority	Clarification	Minimum evidence needed for the stronger claim
1	Is the claim a conditional design, an executable endpoint route, or an observed historical operation?	One unambiguous claim label and a boundary between assumed and observed links
2	What exact source position backs the destination asset?	Source contract and state identifiers, an existing deposit/lock or burn record, corresponding message linkage, and supply/custody reconciliation; absence of a public RPC may require another verifiable source record
3	Which exact destination token and spending route is claimed?	Address-level identity, effective issuance authority, current implementation, actual required configuration and liquidity at a common time; registration alone is insufficient
4	Where did the purchase funds go and what obligations remain?	Linked historical asset movements through issuance, trading and settlement, with source receipts, local claims, borrowings and unbacked principal counted once
5	What does "\$1" mean and for how long?	Specified external reference, sale quantity, fees, horizon and seller/redemption scenario, plus a funded balance-sheet analysis; no finite test proves unlimited duration under arbitrary flows
6	What control claim is necessary?	Roles and successful authorized actions at the relevant time; human identity only if separately evidenced and needed for the claim
7	Can another researcher reproduce the review?	Frozen document version, scripts, manifests, exact query inputs, raw evidence access and archival-provider details; an independent node or cryptographic inclusion checks where stronger authenticity is asserted

For a theoretical existence claim, the missing source and integration facts can remain explicit assumptions. For an actual-history claim they must become observations. Those are different proof obligations; more examples of unrelated destination mints do not substitute for the missing link in either case.

Snapshot and reproducibility limits

The negative findings are bounded by selected addresses, signatures, intervals and the provider's returned history. Twenty successful partitions corroborate query consistency, not completeness of every possible call or nonstandard state change. Endpoint equality does not exclude intervening upgrades; the paper's recorded temporary gateway upgrade is a useful counterexample. Its disclosed pre-fork control-query gap must remain disclosed.

Hash matching detects changes relative to the saved manifests. A manifest created from the same RPC response does not independently authenticate the response. Traces are node-derived execution descriptions, while receipt logs and state are separately structured commitments; no independent commitment validation was performed in this round. Explorer verification is useful but is not this review's own source-to-runtime compilation check.

The raw evidence is intentionally under ignored artifacts. A checkout alone therefore does not reproduce the paper's exact archived observations unless the raw batches are also provided or all historical endpoints remain available. Publication should make the frozen evidence bundle available with its existing hashes and retain data-provider limitations. This is a reproducibility condition, not a reason to discard the positive transaction evidence.

Confidence judgments for cross-review comparison

These are subjective epistemic ranges about the propositions and the evidence presented, **not probabilities of future token prices**, and not statistically estimated confidence intervals. Shared records make reviews' judgments correlated. They should not be multiplied as independent evidence.

Proposition	Epistemic confidence	Basis
H1: Some authorized source receipt representation can finance a local purchase if accepted, valued and funded	95–99%	Coherent custody/claim accounting and ordinary financing logic; exact concrete integrations remain conditions
H2: This paper establishes a complete route executable at the pinned deployed snapshot	0–5%	The paper explicitly does not establish this; material source, permission, configuration, settlement and purchasing-power links remain open
H3: Validator/admin control and nominal copied collateral alone suffice for the target	0–1%	Authority does not supply acceptance, spendable counterpart assets or economically independent value
H4: This paper establishes a durable \$1 under selling/redemption	0–1%	No specified funded, time-bounded stress case or complete intervention ledger establishes that result
H5: The finite AMM existence mathematics is valid under its stated assumptions	99–100%	Direct independent high-precision recalculation agrees; application to actual markets is a separate unproved proposition

The original A/B/C framing corresponds to H1, H2 and H4 respectively. Very low H2/H4 support is not a forecast that future implementation or price support cannot occur. It means the present report does not establish those stronger propositions. Round 1 ends here pending the other reviews' findings.

Appendix 56: Skeptical evidence review, round 2

Contents of this appendix

Challenge to the skeptical objections	366
Views retained from round 1	367
Effect of the clarified proposition.....	367
Reproducibility and remaining disagreement	367
Inspection of the revised paper	368
Governance can change some roadblocks: evidence versus assumption	368
Final epistemic judgments.....	370

Source record: docs/studies/analysis/pulsechain_skeptic_evidence_round2_20260929.md.

Skeptical evidence review, round 2

29 September 2026. Internal second-round review after reading the shared synthesis and both protocol and economic first-round reports. This review reuses earlier bridge research. The checks are independent recalculations of shared saved records, not independent data-provider corroboration or external assurance. No transactions, privileged calls or state-changing simulations were performed. Origin/Morpho-specific analysis was not reopened.

Status: complete for the identified historical main-paper revision. The reviewed revision is identified by its SHA-256 in the round-2 evidence batch.

Challenge to the skeptical objections

The clarified H1 is correct: **an authorized source receipt can finance a local purchase if the exact claim is accepted and the output is funded**. Independent intrinsic value or demonstrated source solvency is not universally required for a finite contract execution. A configured AMM can transfer existing PLS for a claim without inspecting remote collateral. A funded lender can lend against a price its rules accept even if later liquidation cannot recover that value. The first transaction may succeed while a loss remains with the pool, lender or other claim holder.

The round-1 reference to economically usable backing must therefore not be read as an additional universal EVM precondition. Actual output assets and applicable contract predicates are the relevant execution conditions. Source custody, realizable backing and redemption matter when the claim being tested is that the system is backed, solvent, repeatable or able to support future prices. They are separate propositions from the successful transfer of a finite funded amount.

The economic review already makes this distinction in its funded-lender counterexample. Its opening phrase “valuable source claim” is unnecessarily restrictive unless “valuable” means only that an executable bid is available. The protocol review's no-bid counterexample defeats the unconditional inference from nominal units to purchasing power; it does not defeat a stipulated funded AMM exchange. This is a wording correction and a boundary on the objections, not new evidence that such a configured funding path currently exists.

Likewise, absence of an observed end-to-end transaction is not proof that no route could execute. It is appropriate evidence against the proposition that this paper has established that complete deployed route. A route can be proved executable by other sufficiently complete state and implementation evidence; an actual historical witness is one strong method, not the only conceivable method. The current record still leaves essential configuration and source links unresolved.

Views retained from round 1

- The GRT and ARB destination mint observations remain strong and token-specific. They show that pre-existing L1 token escrow is not always necessary for delivery through an authorized integration. They do not independently prove source custody, burns, investment or redemption.
- The raw ARB source-block and timestamp words remain implausible chronology. No amount of additional destination supply arithmetic converts those words into an independently verified source deposit.
- The selected router/gateway census remains bounded. Current default routes, historical routes and token issuer permissions are different facts. A source-backed new representation need not be the exact token already accepted by a destination pool or issuer.
- Specific governance events prove particular protocol actions. Queuing, execution, proxy authority, module authority and human control remain distinct. Historical association does not establish a coordinated stabilization plan.
- The 66 manifest checks, raw event recount, supply recalculation and numerical checks strengthen internal reproducibility. They do not make the common RPC independently authenticated, remove the disclosed pre-fork query gap, or establish source-chain history.
- An externally referenced finite PLS execution value is a valid stipulated target. It is not automatically executable external-dollar liquidation of all received PLS, and a successful one-time sale is not a durable peg.

Effect of the clarified proposition

H1 is deliberately existential and conditional. Its high support must be read with its conditions attached. A bridge can change the location and ownership of an exclusive claim, while a destination pool or lender supplies the spendable assets. This is a useful accounting result. It is not evidence that administrative control creates that funding or makes an arbitrary stablecoin a permitted issuer input.

The objective is not to demand solvency for every possible transaction. It is to avoid switching conclusions midway through the argument: an executable exchange proves an exchange; a backed representation requires source backing evidence; a self-financing cycle requires its liabilities and losses to reconcile; sustained support requires a specified horizon and funded response to the relevant flows.

Reproducibility and remaining disagreement

The review package should preserve the original and revised papers with hashes, identify the H1 wording change, publish the final proposition definitions beside any scores, and make the ignored raw evidence bundle available if external reproduction is intended. The shared records make the reviews correlated. Linear averaging summarizes their judgments; it is not new on-chain evidence or statistical verification of the thesis.

This review agrees with the other reviews' main distinction between conditional coherence and demonstrated deployment. This review rejects any stronger skeptical statement that a funded mechanical swap necessarily fails merely because the input's backing is economically weak. Conversely, This review does not regard such a swap's success as evidence that the backing is strong.

Inspection of the revised paper

The revision materially improves the claim boundary. The abstract now separates historical mechanisms, a stipulated financing ledger and a finite market theorem. It explicitly declines to infer funding from validator control or nominal copied balances and leaves the complete pinned deployment unestablished. The main text now distinguishes custody, solvency, redemption liquidity and purchasing capacity; adds the independent DAI/USDT supply-cap constraints; accounts for pending bridge rights and return; and specifies route-level rejection conditions.

The new ledger supplies the missing accounting connection without pretending it is an observed quote. Its investor commits 1,000 source shares, receives and sells 1,000 representations, then spends the counterparty's existing PLS. The counterparty loses exactly the buyer's PLS budget and gains the represented claim. Source custody retains 1,000 shares; the source vault's underlying and share obligations remain unchanged. The pDAI market gains the spent PLS and loses the pDAI acquired by the investor. No independent copy of the source underlying appears in this accounting.

This review checked the numerical connection against the existing independently recomputed finite-sale example: approximately 2,167,281,613.012984 PLS input acquires 684,271.838699 pDAI and leaves approximately 315,728.161301 pDAI in the modeled pool. Initial pDAI inventory equals final pool inventory plus the investor's purchase. The stipulated counterparty funds the sequence before the purchase; there is no negative intermediate PLS balance in the described order. The existing fee-free and separately funded transaction-cost qualifications remain necessary. The ledger is a conditional composition witness, not bytecode verification of an exact deployed composition.

The asynchronous identity is now expressly a simple lossless model and counts pending rights once. That repairs the overbroad implication that destination supply always equals committed escrow at every intermediate moment. Actual message finality, cancellation and release behavior still require the exact bridge implementation if a deployed claim is made.

The revised falsifier table appropriately tests a complete custody/redemption model. Its source-right and return requirements should not be reinterpreted as universal preconditions for a single local AMM trade. A local trade may execute before the economic weakness of a source claim is known; failure of redemption then defeats the backed/repeatable interpretation, not the fact that the trade occurred. The paper's explicit acceptance-versus-solvency distinction supports this reading.

Governance can change some roadblocks: evidence versus assumption

It is conceivable that careful operators with the necessary effective authority and an executable governance process could mitigate some identified limitations. A zero cap is a present parameter, not a theorem that every future configuration must retain it. Available inventory can change. Compatible integrations can be introduced where the actual control and interfaces permit them. Those statements do not prove that the named wallets possess all required authority, that every proposal can execute, or that changing a setting supplies new economic resources.

Observed issue or control fact	Conditional remediability	What the evidence does not establish
Aave v3 stablecoin Portal caps are zero; DAI/USDT also exceed supply caps	Parameters can differ in a configuration authorized through the relevant actual controls; each independent validation condition still matters	Aave v2 provider ownership or stkAAVE governance activity does not establish v3 bridge-role, ACL or cap authority

Observed issue or control fact	Conditional remediability	What the evidence does not establish
Bridge ProxyAdmin ownership and gateway implementation changes are recorded	Controlled upgradeable bridge components need not remain permanently at today's implementation	Authority over unrelated exact-token issuers, immutable contracts, every gateway or real source custody
ARB/GRT have token-specific issuer integrations	Other specific integrations are conceivable where their required issuer and bridge permissions exist	ARB/GRT permissions generalizing to DAI, USDC, USDT or arbitrary Aave receipts
Across HubPool/module ownership and a configured-spoke upgrade occurred	Some specific Across administrative paths have actually executed	Control of every UMA component, retained behavior of an unverified replacement, every relayer's inventory or future collectible settlement
Reserve Main is owned by a timelock; sampled role memberships are inherited and direct e121 checks are false	Governance-based changes can be considered if the actual proposal/voting/execution prerequisites are satisfied	The reviewed role history alone does not establish the claimed wallet's voting control or ability to complete that process
Sampled pools and HubPool have limited or zero output inventory	Accounted, funded incoming assets can change inventory; this need not always require protocol ownership	The existence, independence or affordability of the proposed replenishment capital
Reviewed Balancer targets and fees are configurable; main/wrapped token identities are immutable in that implementation	Some parameters may change under effective authorization; a different integration is a different case	Replacing the exact accepted token by changing an unrelated oracle, router or parameter
A plugin or oracle response prevents a particular issuer path	A relevant controlled configuration or compatible replacement could conceivably differ	Correct valuation, actual redemption, token issuer permission or funded market output being created by that change

These are classifications of possible authorized state differences, not a sequence for changing governance. A successful governance vote is only one part of an effective process: timelock/executor authority, the target's actual permissions and application validation determine the resulting capabilities. Observed execution proves its particular path and date; historical proposal participation does not prove the ability to amend every later target.

The final main-paper governance section correctly adds an issuer qualification: authorized control of the exact issuer can create additional token balances. That is different from changing a lender's oracle or borrowing limit, and does not itself guarantee an executable PLS bid. Native PLS issuance would require a separate consensus-level assumption, not ordinary application governance.

No accounting conservation requirement disappears through governance. An implementation change can change claims, issue authorized token units or redistribute existing assets, but this is a changed mechanism whose effects must be described, not evidence that the previous route already worked. Likewise, current insufficient cash is not permanent impossibility, while a claim to pay existing assets without any funded source remains incomplete under the stated accounting.

Final epistemic judgments

These central scores and ranges express subjective support for the exact propositions. They are neither calibrated event probabilities nor forecasts. Low H2/H4 scores concern what the current paper establishes; they do not estimate the probability of future governance changes or future price movements. H1 now means accepted and funded, without an intrinsic-value requirement. H6 is an existential claim about some roadblocks, not a universal guarantee.

ID	Proposition	Central score	Subjective range
H1	Some authorized source receipt can finance a local purchase if accepted and funded	99%	97-100%
H2	The paper establishes the complete proposed route executable at the pinned deployed snapshot	1%	0-3%
H3	Validator/admin control and nominal copied collateral alone suffice for the target	0%	0-1%
H4	The paper establishes durable \$1 execution value under selling and redemption	0%	0-1%
H5	The finite AMM existence mathematics is valid under its explicit assumptions	99.5%	99-100%
H6	Given effective relevant authority and an executable governance process, some identified configuration/integration roadblocks could be mitigated	99%	97-100%

The small ranges around near-logical propositions represent interpretive and review uncertainty, not frequencies. The governance clarification raises no conflict with low H2: a potentially changeable configuration can still be unestablished or unavailable at the specific audited snapshot. the round-2 assessment supports the revised conditional result while retaining all source provenance, exact-identity, funding, attribution and reproducibility limitations.

Appendix 57: Skeptical protocol review — round 1

Contents of this appendix

Verdict	371
Evidence directly inspected	372
Strongest support	372
Ranked corrections and additions	372
Critical: define the proposition without assuming its conclusion	372
Critical: distinguish three meanings of backing	373
High: qualify conservation for asynchronous execution	373
High: describe the reverse lifecycle	373
High: surface the independent Aave barriers	373
High: bound administrative hypotheticals	374
Medium: make rejection conditions readable	374
Counterexamples and falsifiers	374
Independent confidence judgments	374

Source record: docs/studies/analysis/pulsechain_skeptic_protocol_round1_20260929.md.

Skeptical protocol review — round 1

29 September 2026. Independent review of [pulsechain_dai_feasibility_20260928.md](#) (main paper) before receiving the other reviews' findings. Scope: contract composition, identity, bridge accounting, Portal and the difference between inherited, observed and hypothetical configurations. Read-only source and saved-evidence inspection; no new public transactions, administrative actions or modified-fork experiments. The review does not evaluate the previously excluded Origin/Morpho-specific route.

Verdict

The strongest defensible affirmative result is **conditional architectural coherence**: a transferable source receipt can be held in bridge custody while its authorized destination representation is exchanged for an existing local asset. The underlying can remain invested. A separately funded local exchange can then buy pDAI. Historical reverse-gateway source supplies real mechanics for the custody and mint steps.

The paper does **not** establish a complete deployed source-to-pDAI path at its pinned snapshot. It also does not establish that validator control, token-unit balances and administrative control produce enough economic buying power. Its explicit conditions generally acknowledge this. Nevertheless, the abstract and conclusion can be read as stronger than the evidence because they join component demonstrations to a separately funded AMM example. Their connection is an assumed funding and integration edge, not an observed or fully constructed end-to-end witness.

The revised thesis is consequently narrower than the original proposition. It has not shown that apparently unbacked copied balances become dollar-equivalent collateral by being attested, deposited and wrapped. It has shown that investment claims can be transported and financed **if** those claims receive usable financing or market demand. This narrower statement is meaningful, but the change in proposition should be explicit.

Evidence directly inspected

- Historical `L1ReverseCustomGateway.sol` and `L2ReverseCustomGateway.sol`, saved under `artifacts/parb-historical-bridge-models-20260929/`, pinned to commit `64cfba62f6dc1ce3b89869db341956f298810299`. The function bodies support L2 custody and L1 `bridgeMint`; the misleading historical header comment should not override those bodies. [Pinned L1 source](#).
- The saved explorer source for the deployed Aave V3 implementation, `artifacts/parb-debt-history-20260929/v3_implementation_source.json`. Its `BridgeLogic` calls `validateSupply`, enforces the unbacked cap, increments unbacked and mints the local aToken. Settlement reduces unbacked and transfers the actual reserve underlying to its aToken contract. [Explorer source](#).
- Saved reserve summary `artifacts/parb-loop-accounting-20260929/reserve_summary.json`: DAI/USDC/USDT cash, unbacked and unbacked caps are zero; DAI and USDT aToken supplies exceed their configured supply caps. These are observations at block 27,659,393, not observations of every later state.
- The wrapper inventory, Portal accounting, historical bridge and Reserve collateral studies cited by the main paper. These establish specific identities and limitations; this review did not independently repeat their chain calls.
- [ERC-4626](#): shares implement ERC-20, but nontransferable implementations are permitted. Conversion views describe idealized accounting; withdrawal limits and actual execution are separate. Its ordinary `mint` receives assets and does not constitute unrestricted bridge mint authority.

Strongest support

1. The reverse-gateway source is a genuine pre-fork architecture, not an invented modern bridge feature. The L2 gateway measures the increase in its token balance after custody transfer, while the L1 override invokes the paired token's mint method.
2. Existing ARB and GRT destination mints, as documented in the audit, refute a universal claim that every destination delivery must come from pre-existing L1 escrow. They do not establish source backing or universal token compatibility.
3. The exact V3 aToken/wrapper relationship supplies a coherent local Portal-to-wrapper interface. Portal's receipt can be an ordinary local aToken; it need not be a newly named remote representation. This is distinct from the generic reverse-gateway path.
4. An existing Reserve issuer can issue the exact token already listed in its Curve pool. That avoids the mistaken assumption that a bridged receipt itself must equal the pool coin. It still requires the actual active basket and valuation conditions to accept and receive the correct assets.
5. Nothing in these mechanisms logically requires the source investment to be sold merely because its transferable receipt enters escrow. The bridge can transport ownership of a claim without moving the underlying investment.

Ranked corrections and additions

Critical: define the proposition without assuming its conclusion

Split the result into three statements: historical interfaces exist; a specified hypothetical composition is internally consistent; sufficient purchasing assets can move a specified market to a target. Do not describe the third as proving the second's financing. Section 2's assumption of actual purchasing liquidity currently bears almost the whole economic burden.

A defensible theorem should identify initial holders and end holders of each asset, the actual financing asset, each exact interface, required cash and residual obligations. It need not claim a historical execution. Until that witness exists, use "architecture outline" or "conditional composition," not a proof of a complete deployed route.

Critical: distinguish three meanings of backing

Add explicit definitions:

- **Token-unit custody backing:** destination claims correspond to source receipts under bridge accounting.
- **Source protocol solvency:** the receipts can realize the claimed underlying under that protocol's obligations and withdrawal rules.
- **Economic purchasing capacity:** a counterparty or funded market provides an executable quantity of local assets for the claims.

The first does not establish the second or third. A trillion source units may be completely escrowed while receiving no financing. Source validators accepting an invented balance can establish neither independent economic demand nor realizable investments merely through acceptance. Even real source receipt units can face losses, unavailable redemptions or a market discount. This distinction strengthens the paper without requiring Ethereum dollar redemption.

High: qualify conservation for asynchronous execution

The main paper's simple settled one-to-one model is reasonable, but its companion states $D = E$ too generally. Between source lock and destination mint, escrow exceeds minted representations; between destination burn and source release, it also remains committed while the redemption is pending. For fixed units and no donations or losses, a useful accounting relation is:

`committed escrow = outstanding destination representations + pending issuance rights + pending redemption rights.`

This is a schematic accounting identity, not a claim those rights are all directly enumerable in the existing contracts. Include finalized-message processing, duplicate-message exclusion, cancellation rules and other destinations when applicable. Timing can change who has an enforceable claim without changing total collateral.

High: describe the reverse lifecycle

A convincing invested-receipt model needs more than entry. Describe which destination claim is surrendered on return, how the source receipt is released, and how that receipt can subsequently redeem underlying. Account for source withdrawal queues, frozen tokens, transferable ownership and share-rate changes. Otherwise “backed” can be mistaken for immediately redeemable.

For static shares, bridge the share count and leave the source conversion rate as a separate quantity. Do not assume destination collateral plugins automatically know the current rate or underlying liquidity. A functioning oracle reports a valuation; it does not deliver assets.

High: surface the independent Aave barriers

Main text mentions zero Portal caps but omits a second material observed barrier: DAI and USDT aToken supplies already exceed their configured supply caps. The deployed Portal path calls `validateSupply`, so enabling only an unbacked cap would not establish issuance availability. Also preserve role, reserve, amount and wrapper-transfer conditions. A V2 wrapper is not a V3 Portal wrapper.

The main Portal inequality is sound as a principal cash-flow condition. It should remain explicitly conditional on successful swaps and actual backing transfers. Ordinary repayment does not close the unbacked balance, and cash obtained by a wrapper sale is not retained treasury cash after being used to back that issuance.

High: bound administrative hypotheticals

Separate (a) already inherited exact configuration, (b) existing functionality under changed parameters, and (c) new adapters or changed implementation behavior. A new representation accepted by a new market is not proof that the old pools accept it. Full arbitrary code control would make a very broad existence claim nearly trivial; the useful theorem should restrict itself to named accounting models and compatible interfaces.

Medium: make rejection conditions readable

Add a compact route checklist: exact source unit; custody or burn operation; authorized destination issuer; exact output token; accepted market/collateral identity; funding asset and amount; settlement; return/redemption. Mark each as observed, historical source behavior, hypothetical or unresolved. Any broken edge blocks that particular composition, not every architecture in the paper.

Also replace the conclusion's awkward “selected the account as their plan” with “adopted this mechanism as their plan.”

Counterexamples and falsifiers

- **Accepted state, worthless receipt:** all bridge permissions and custody are valid, but no lender or buyer provides local purchasing assets. The broad validator-to-\$1 sufficiency claim fails while the custody bridge works correctly.
- **Correct symbol, wrong token:** a representation called aDAI reaches L1, but the Balancer wrapper accepts only its configured local Aave aToken. The proposed direct deposit fails; a separate integration would be needed.
- **Empty ordinary gateway:** accepted withdrawal plus zero escrow produces no ordinary stablecoin delivery. ARB/GRT-specific mint authority does not fix this token's transfer path.
- **Portal cap enabled, supply check fails:** the independent DAI/USDT supply-cap condition blocks fresh issuance at otherwise unchanged sampled state.
- **Local issuer recognizes a token only in its registry:** the active basket still requires other exact assets. Registration alone does not establish issuance from the new receipt.
- **Receipt sale at a discount:** wrapper proceeds are smaller than new Portal principal plus fees. Repetition accumulates an obligation unless another asset funds it.
- **Bridge completes, buying budget too small:** all contract transitions succeed but the funded output fails to move the stated market and trade size to the target.

A live receipt-to-pDAI route would require all its specified predicates simultaneously. A failing example falsifies a universal claim of sufficiency; it does not disprove an existential claim allowing different funding and integrations. A theorem so permissive that every failed predicate may simply be assumed away would have little explanatory power.

Independent confidence judgments

These are subjective ranges for the stated propositions, not calibrated frequencies, future-price forecasts or probabilities that an undisclosed network exists. They should not be multiplied across reviews sharing evidence.

Proposition	Subjective confidence in truth	Reason
H1: some authorized source receipt representation can finance a local purchase if valued and funded	96–99%	Real custody/mint interfaces and ordinary funded exchange compose without an accounting contradiction
H2: this paper establishes a complete route executable at its pinned deployed snapshot	0–2%	Missing source witness and integrations; explicit observed blockers in candidate paths
H3: validator/admin control and nominal copied collateral alone suffice for the target	0–2%	Counterexample with no executable funded demand is decisive
H4: this paper establishes durable \$1 under selling and redemption	0–1%	No sustained budget, market-response or redemption demonstration
H5: the finite AMM existence equations are valid under the stated idealized assumptions	97–99%	Algebra agrees with constant-product reserves; not a funding-path proof

H2 and H4 are judgments about what the paper establishes, not assertions that no such route or future price could exist. This review did not separately recompute every numerical digit of the AMM example. H1's high confidence is partly a consequence of its explicit funding and acceptance conditions; it should not be quoted as confidence in the original unconditional thesis.

Round 1 is complete. Awaiting shared findings before round 2.

Appendix 58: Skeptical protocol review — round 2

Contents of this appendix

Opinion changed: solvency is not necessary for a finite purchase	376
Objections that survive shared review	376
Contrary interpretation defended	377
Remaining proof boundaries	377
Final judgments after revised-paper check	378
Review of the strengthened paper	378
Can governance conceivably mitigate the roadblocks?	379
Final actual-text checkpoint	380

Source record: docs/studies/analysis/pulsechain_skeptic_protocol_round2_20260929.md.

Skeptical protocol review — round 2

29 September 2026. This round follows receipt of the shared first-round synthesis and both other reviews' first-round reports. It challenges those objections as well as the thesis. No public transactions, administrative changes or new execution experiments. The revised main paper was examined after revision; its added governance discussion is assessed separately below.

Opinion changed: solvency is not necessary for a finite purchase

The first-round distinction among custody coverage, source solvency and destination purchasing capacity remains useful, but those must not be described as three necessary predicates for every finite trade. A funded AMM, lender or other contract can mechanically deliver local assets for a claim whose backing has weak or even negligible realization value. The technically necessary fact for that exchange is successful exact-asset acceptance and funded output under the governing rules. Solvency determines who can ultimately redeem or absorb loss; it is not a universal precondition for successful finite spending.

Likewise, the initial funder need not be a new discretionary buyer or an external depositor entering after the bridge transaction. Existing liquidity providers may already have supplied the inventory that a configured market pays out. A criticism requiring fresh money to arrive at that moment would be too strong. Calling this a redistribution of existing purchasing assets is accurate; calling it technically impossible merely because the new claim lacks independent dollar backing is not.

This correction strengthens H1, not H3. The counterexample to H3 is a system without sufficient executable funded output, even when validator and administrative permissions and enormous nominal source balances are granted. A different system with funded acceptance can complete a finite purchase. Control and nominal balances alone do not select between them.

Objections that survive shared review

1. Contract-level identity and permission boundaries still matter. A newly minted representation does not automatically become the existing Aave aToken or another pool coin.

2. Aave Portal still requires its independent authorization, reserve predicates and settlement accounting. Zero cap and exceeded supply cap are separate observed barriers. Source custody does not itself perform the local underlying transfer.
3. A source bridge receipt can remain invested, but it must leave the original owner's unrestricted control while supporting an exclusive destination claim. Continuing investment is not continuing independent spendability.
4. The recorded ARB/GRT mint histories support specific destination issuance. They cannot verify a source lock/burn merely because the assumed controllers' destination contracts accepted a statement.
5. A fully specified hypothetical witness need not be historically observed. Conversely, a hypothetical witness cannot change the evidentiary status of the pinned deployment.
6. A finite successful purchase says little about successive sales, creditor redemptions or support duration. The economic review's distinction between intermediate funding and final netting is essential for a sequence that spans chains and time.

Contrary interpretation defended

The strongest affirmative interpretation is more than “money can buy tokens.” Reverse gateways preserve investment while transferring a receipt's claim across a domain boundary; specialized mint integrations can overcome a destination escrow shortage for authorized tokens; Portal can issue an existing local aToken before its reserve is backed; and Across can deliver funded destination assets before final cross-chain settlement. These are genuine timing, identity and custody transformations. They can be useful even though none creates the underlying destination inventory.

The strongest skeptical interpretation is that the paper has not connected these advantages to a quantified operational advantage or funded target on the actual network. A list of potentially useful transformations and a stipulated buyer still cannot establish that the named wallets have built the claimed system. Both interpretations are compatible. The report should retain the first without implying the second has been resolved.

Remaining proof boundaries

For a complete hypothetical composition, specify one receipt, one custody relation, one authorized destination representation, its funded purchaser, the exact locally spendable output, market input, all settlement obligations and final holders. This can be a clean symbolic or numerical model; it need not claim deployed addresses. It must not silently alternate between a generic token and an existing pool token.

For a deployed route, those abstract objects must be replaced with simultaneous contract identities, effective permissions, usable balances, mappings, source/message provenance and actual execution predicates at the relevant time. The existing audit leaves material links open. A parameter assumption and a new implementation assumption are different claims.

For return accounting, a settled $D = E$ identity is inadequate during in-flight messages. Committed custody can cover outstanding destination units plus exclusive pending issuance and return rights. This simplified relation assumes fixed share units and separately handles donations, losses, rebases, rounding and allocations to other destinations. A message must not appear twice in the accounting when its state changes.

For sustained support, a finite horizon, price band, trade size, seller/redemption envelope, reference-price path, intervention inventory, replenishment and loss allocation must be specified. A successful entry trade is not this test.

Final judgments after revised-paper check

These are subjective confidence judgments in narrowly worded propositions, not future-price forecasts or calibrated frequencies. H1's wording changes from “valued” in round 1 to “accepted and funded”; it does not require independently valuable backing. Shared evidence makes judgments correlated.

Proposition	Central judgment	Range
H1: Some authorized source receipt can finance a local purchase if accepted and funded	99%	97–100%
H2: The paper establishes a complete proposed route executable at the pinned deployed snapshot	0.5%	0–2%
H3: Validator/admin control and nominal copied collateral alone suffice for the target	0.5%	0–2%
H4: The paper establishes durable \$1 execution value under selling and redemption	0.5%	0–1%
H5: The finite AMM existence mathematics is valid under its explicit assumptions	99.5%	99–100%

Higher H5 confidence reflects the economic and evidence reviews' separately recalculated arithmetic. This review separately checked the algebra and assumptions but did not reproduce their full decimal calculation. Low H2/H4 scores concern what the paper establishes, not whether a hidden system or future price move exists. The small nonzero values express possible interpretation or review error rather than statistical odds about deterministic contract behavior.

Review of the strengthened paper

The revised abstract now separates three results: historical interfaces, a stipulated financing composition and a finite market existence result. Its conclusion no longer presents these as proof of an executable deployed route. The new 1,000-share ledger names the holder that gives up PLS and the holder that receives the exclusive investment claim. Its sequence is cash-feasible under the stipulated bid and timing, and it leaves underlying investment in place. No fresh independent collateral value is silently created.

The four-way distinction among custody, solvency, liquidity and buying power corrects the overstrong first-round wording. The asynchronous rights account, redemption discussion, source share-rate distinction and exact local-token requirements close the principal conceptual omissions. The independent Portal supply-cap condition is now visible. Intermediate funding, shared reserves, circular valuation, competing sellers and released collateral are included. These changes materially strengthen the narrow theoretical claim.

The financing witness is complete as a simplified **asset-flow ledger**, not as a contract-level executable proof. Its stipulated bid remains its strongest assumption. It says nothing about an actual market offering that bid or a lender sustainably accepting those claims. The paper acknowledges this, so the remaining issue is scope rather than an internal contradiction. No fatal contradiction was found in the revised conditional model.

One qualification remains to the positive interpretation: Portal, Across, direct receipt sales and local issuer routes are not economically interchangeable. The paper's fully specified example covers a direct receipt sale. The other families remain outlines with additional route-specific obligations. A reader must not transfer the direct-sale witness's complete ledger to those variants without adding their settlement or debt accounts.

Can governance conceivably mitigate the roadblocks?

Yes, some of them. A zero capacity, an uninstalled integration or an outdated supported configuration is often a contingent state, not a theorem of permanent impossibility. This conclusion assumes effective authority and a functioning authorization process. It does not depend on attributing unusual skill or meticulousness to any wallet group.

Roadblock class	Conditional remediation assessment	What remains after a governance decision
Existing adjustable parameters	Relevant roles may change caps, supported settings or flags within the implementation's permitted range	Every other predicate, amount, funding requirement and protocol bound still applies
Existing registration or basket integration	Authorized registration/configuration can potentially recognize a compatible token and its collateral semantics	Exact identity, active basket membership, compatible valuation, successful transfers and remaining basket components
Missing behavior or incompatible adapter	An upgradeable component may support a new implementation or adapter if its actual governance can authorize that change	A specified new integration and its accounting/compatibility; it is not inherited behavior simply because an upgrade is conceivable
Separate token authority or immutable receiving rules	Another system's administrator cannot automatically change these	The exact issuer's cooperation/authority or a different compatible route; a new market does not retroactively change an old pool coin
Empty inventory or settlement obligations	Governance may direct assets it controls, admit financed collateral or authorize funded borrowing	Actual transferable output assets, creditor liabilities, ownership and loss allocation remain necessary
Sustained selling and redemption	Policies can define intervention and replenish inventory where a real source exists	No vote by itself supplies unlimited liquid reserves or cancels another holder's independent claim

Aave is a clear parameter example. The separate supply and unbacked caps are configurable concepts in the V3 architecture. The [upstream PoolConfigurator source](#) includes separately authorized setters. This upstream source supports the architectural classification; it is not a fresh proof that the exact configurator and role chain at the pinned PulseChain snapshot provide those same reachable actions. The existing deployed `BridgeLogic` source establishes that both predicates matter. Relevant effective roles and configuration bounds must be checked for a deployment claim.

A successful governance path can also involve proposer eligibility, quorum, voting power, timelocks, executor authority, pause restrictions and cross-chain message delivery. These are feasibility conditions, not reasons to ask the research user for permission. Assumed ownership of one contract is not proof that the complete authorization path exists. Conversely, absence of demonstrated control does not disprove a scenario that explicitly grants effective relevant authority.

For the new proposition **H6 — given effective relevant authority and an executable governance process, some identified configuration/integration roadblocks could be mitigated** — the central confidence is **99%**, with a **97–100%** judgment range. This is an existential statement about some adjustable obstacles. It is compatible with low H2 because H2 concerns the unchanged fixed snapshot. It does not imply that every obstacle can be removed or that future funding is established.

The clean final separation is: **current blockage**, **conditional remediability**, **future funded execution**, and **durable support** are four different questions. This review supports conditional remediability strongly and leaves the latter two without a deployed or sustained-system proof.

Final actual-text checkpoint

Reviewed the final main-paper version with SHA-256

a9c49fb67d4c22b6d07e226471b52d39b770b3dc535075b6831590e2641f76d3, including its section 9 governance table and section 10 proof obligations. Also checked the companion bridge study's settled-state qualification and $E = D + I + R$ discussion. These satisfy the requested conservation correction.

Section 9 correctly adds a further nuance: authority over the exact token issuer may increase actual spendable token units. The objection is not that governance can never change balances. It is that authority over adjacent application settings does not confer unrelated issuer rights or establish a token's executable PLS exchange value. Native PLS issuance is explicitly outside the assumed application-governance powers. This avoids overclaiming either impossibility or financial sufficiency.

No material factual overreach or unresolved internal contradiction was found in this final text within this review's scope. The scores above, including H6, are final. The remaining limits are the explicitly stipulated bid/integration, unresolved deployed route, and absent sustained-support demonstration—not a contradiction in the conditional direct-sale construction.

Appendix 59: Two-round skeptical review of the PulseChain mechanism thesis

Contents of this appendix

Method and meaning of consensus	381
Round 1 shared findings	381
Questions explicitly sent into round 2	383
Round 2 result	383
Probabilistic opinion pool	384
Disposition of every material review issue	385
What the next stronger theoretical result would require	386

Source record: docs/studies/analysis/pulsechain_skeptical_review_20260929.md.

Two-round skeptical review of the PulseChain mechanism thesis

29 September 2026. Read-only internal review of the [conditional white paper \(main paper\)](#). Three review strands considered protocol composition, economics and evidence. This is model-assisted adversarial review, not independent external assurance. The evidence review reuses earlier bridge research; all three strands share underlying records. Agreement is therefore correlated.

The original paper is identified by SHA-256 9d78f09de3692d137ac1ae79af1551352a0794c76e4a7f31b9e08197140b64a4. The original text is preserved in the ignored review batch. The methods manifest is [Research/experiments/pulsechain_skeptical_review_20260929.json](#). No public transactions, deployments or configuration changes were made.

Method and meaning of consensus

Round 1 produced separate reviews before cross-comparison of their findings. This synthesis and all three reviews were then shared before round 2. Round 2 must challenge both the paper and the objections, retain dissent, and review the proposed strengthening. Review confidence is a subjective judgment about a precisely stated proposition. It is not a token-price forecast, a measured frequency or a calibrated statistical confidence interval.

For the probabilistic comparison, the final comparison uses an equal-weight linear opinion pool: average the three central judgments, using an interval midpoint only where the review supplies no central value. Report the full envelope of judgment ranges as disagreement/sensitivity, not as a confidence interval. Do not multiply the reviews' judgments as independent observations. Randomly resampling three correlated judgments would add numerical precision without adding evidence, so no such simulation is used.

Round 1 shared findings

The [protocol review \(Appendix 57\)](#), [economic review \(Appendix 53\)](#), and [evidence review \(Appendix 55\)](#) agree on the following distinction:

Historical bridge and protocol interfaces support a conditional architecture. The paper does not establish that the original validator/admin-control thesis supplies sufficient purchasing power, a complete deployed route, or a durable one-dollar price. The latter is an evidentiary conclusion, not a forecast that no future funded implementation can work.

Support that survived challenge:

- Reverse-gateway function bodies genuinely support source custody and authorized destination minting. The source investment need not be withdrawn while its transferable receipt changes ownership.
- The ARB and GRT records demonstrate specific destination issuance despite insufficient or absent destination token escrow. Those permissions do not generalize to every stablecoin or receipt.
- Exact Aave wrappers, existing issuer/pool identities and recorded swaps are useful component evidence. They do not trace a common financing operation.
- Independent high-precision recalculations agree with both AMM formulas.
- The evidence review matched all 66 sampled manifest entries, recounted 106 withdrawals in 78 transactions, and recomputed the 106 paired supply comparisons. Only the already identified ARB and GRT pairs increased supply. Hash checks demonstrate file consistency; they do not independently authenticate the original RPC responses.

Highest-priority weaknesses:

1. **Backed in units is not funded buying power.** Distinguish custody coverage, source solvency, actual redemption liquidity and an executable destination bid.
2. **The decisive financing premise is assumed.** The AMM calculation begins with spendable PLS; it does not derive those PLS from validator attestations.
3. **A full cash-flow witness is missing.** Name the initial funder, every transfer, each holder's final claim, intermediate cash needs and liabilities.
4. **Exact deployments remain unresolved.** Historical code, sampled deployed behavior, changed parameters and new integrations need separate labels.
5. **Aave has two independent sampled cap barriers.** Portal caps are zero; DAI/USDT supplies already exceed their supply caps. Raising one limit alone does not establish execution.
6. **Bridge entry needs an exit model.** Reconcile pending issuance/redemption, source receipt release, withdrawal conditions, rate changes and dilution.
7. **Dynamic support is unspecified.** Set a trade size, reference, price band, horizon, seller/redemption bound and source of replenishment.
8. **Repayment may restore sellable assets.** Deposits/escrow are not permanent burns, and debt repayment need not produce operator profit.
9. **Source provenance is missing.** A destination message accepted by the assumed controllers is not independent verification of the source assertion.
10. **Publication needs accessible evidence.** Ignored raw artifacts are not delivered by a source checkout; scripts and links alone may not reproduce historical provider responses.

Decisive counterexamples to stronger claims include a perfectly bridged receipt with no funded bid; an ordinary gateway with empty escrow; a nominally identical token rejected by the exact wrapper; Portal proceeds fully consumed by backing; finite Across inventory without replenishment; cheap competing pDAI markets; and a falling PLS reference. Circular valuation using the hoped-for pDAI price cannot independently establish the capital needed to reach that price.

Questions explicitly sent into round 2

The reviews must also attack an overly strong skeptical reading. A funded AMM or lender can mechanically exchange real assets for a claim even when the claim's source backing has weak economic value. A willing human buyer or independently valuable backing is not a universal technical prerequisite for a finite purchase. The required facts are executable acceptance and funded output. Losses, discounted liquidation, redemption and repeatability are separate questions. This does not authorize any administrative execution.

Round 2 asks whether a numerical custody/sale/purchase ledger can demonstrate conditional composition without smuggling in generated wealth; which objections are real logical failures versus unverified premises; and whether the revised paper cleanly separates a finite purchase from sustainable price support.

The five common propositions are:

ID	Proposition
H1	Some authorized source receipt can finance a local purchase if accepted and funded
H2	The paper establishes a complete proposed route executable at the pinned deployed snapshot
H3	Validator/admin control and nominal copied collateral alone suffice for the target
H4	The paper establishes durable \$1 execution value under selling and redemption
H5	The finite AMM existence mathematics is valid under its explicit assumptions

H1 deliberately does not assert that economic solvency is necessary for every technically executable transaction. Round 1's wording included "valued"; the round-2 clarification means executable acceptance, not independent intrinsic value. This wording change must accompany any comparison of scores.

Round 2 result

All three reviews inspected the revised paper, including the additional question about governance remediation. Their final text checkpoint is SHA-256

a9c49fb67d4c22b6d07e226471b52d39b770b3dc535075b6831590e2641f76d3. The separate final reviews are the [protocol review \(Appendix 58\)](#), [economic review \(Appendix 54\)](#), and [evidence review \(Appendix 56\)](#).

The affirmative theoretical result survived both attacks: a source receipt can remain invested, an exclusive authorized representation can be accepted for funded destination assets, and those assets can finance a finite pDAI purchase. The revised example conserves the 1,000 source shares and destination claims and identifies the counterparty supplying the approximately 2.167 billion PLS budget. The bid is stipulated rather than observed. This is a complete simplified accounting witness for outright sale, not bytecode validation of the named deployed route or a proof of every financing variant.

An overly strong objection was rejected: **economically sound collateral is not a universal technical prerequisite for a finite funded trade.** A pool or lender may mechanically accept a claim and pay existing assets. Its subsequent loss or redemption exposure does not undo the fact of that exchange. The revised paper now distinguishes execution, custody coverage, solvency, redemption liquidity, and sustainable support.

The governance challenge also survived: **some identified roadblocks could conceivably be mitigated under effective relevant authority.** Adjustable caps, supported collateral, valuation integrations, mappings and upgradeable behavior need not remain in their sampled state. Funding, repayments or offsetting flows can change inventory. Where authority includes the exact token issuer, authorized issuance may add spendable units; this does not itself guarantee their PLS buying power. Existing pool identities, unrelated issuer permissions and native-chain rules cannot be assumed adjustable through a neighboring application's votes.

These conclusions are compatible with the observed current blockers. A claim that an unchanged snapshot already works differs from a claim that a controlled, reconfigured and funded scenario could work. The evidence of particular control changes does not establish every required role, effective vote or executor path. The premise about meticulous operators is not needed as a substitute for those contract-specific conditions.

No review found a material internal contradiction in the revised stipulated direct-sale model. None accepted the stronger claims that nominal source balances and control alone supply the needed purchasing assets, that the audit establishes the complete deployed route, or that the paper proves durable support.

Probabilistic opinion pool

The table gives confidence **that each written proposition holds**, not the chance of pDAI reaching \$1. H2/H4 concern what this report establishes; their low values cannot be used as estimates of hidden systems or future prices. For H6, the proposition is: given effective relevant authority and an executable governance process, some identified configuration/integration roadblocks could be mitigated. It does not say every obstacle is removable.

Proposition	Protocol	Economics	Evidence	Equal-weight mean	Full judgment envelope
H1: Accepted, funded receipt can finance a purchase	99%	99%	99%	99%	97–100%
H2: Complete proposed route established at pinned deployment	0.5%	1%	1%	About 1%	0–3%
H3: Control and nominal copied collateral alone suffice	0.5%	0.5%	0%	Below 1%	0–2%
H4: Durable \$1 under selling/redemption established	0.5%	0.5%	0%	Below 1%	0–2%

Proposition	Protocol	Economics	Evidence	Equal-weight mean	Full judgment envelope
H5: Stipulated finite AMM mathematics is correct	99.5%	99.5%	99.5%	99.5%	99–100%
H6: Some governance/configuration barriers are remediable	99%	99%	99%	99%	97–100%

Near-zero nonzero judgments express possible reading/review error around nearly logical propositions, not measured randomness in deterministic contracts. The counterexamples and proof obligations carry more weight than the numerical scores. Shared source material and cross-review make these correlated opinions; unanimity is not independent experimental replication. There is no meaningful overall “probability the thesis is true” to derive by averaging these different claims. H1 changed wording and H6 was added in round 2, so score changes are not a controlled before/after confidence experiment.

Disposition of every material review issue

Issue	Result in the revised paper	What remains for a stronger claim
Scope of conclusion	Abstract/conclusion distinguish conditional composition, deployed execution and sustained support	A specific deployed or dynamic proof cannot be inferred from the abstract model
Custody versus value	Section 3 defines custody, solvency, withdrawal liquidity and purchasing capacity	Source valuation and realizable claims if economic backing is asserted
Mechanically accepted weak collateral	Sections 2–3 and conclusion explicitly allow finite funded exchange	Identify whose inventory is paid out and who retains losses or claims
Complete source-to-market ledger	Section 3 adds all parties to a numerical direct-sale witness	Actual receipt identity, quote and implementation; separate ledgers for loans, issuers, Portal and Across variants
Aave independent barriers	Sections 4 and 8 expose both unbacked caps and supply-cap constraints	Exact authorized configuration, issuance amount, liquidity and settlement predicates
Governance remediability	Section 9 classifies adjustable state, integrations, funding and immutable boundaries	Effective voting/execution authority, permissible bounds, timing and resulting interfaces
Authorized issuance nuance	Section 9 distinguishes exact-token mint authority from adjacent governance	Whether new units receive enough executable purchasing assets
Exact token identity	Sections 3, 4 and 10 retain address/interface requirements	Every receiving pool, wrapper and active basket must accept the actual output
ERC-4626 assumptions	Section 3 notes transfer restrictions and idealized conversions	Exact vault behavior; the standard's mint interface is not bridge authority

Issue	Result in the revised paper	What remains for a stronger claim
Asynchronous custody	Section 4 and bridge companion account for exclusive pending rights	Actual finality, cancellation, duplicate-message exclusion and release behavior
Source exit and dilution	Sections 2 and 4 add return lifecycle, rate changes and control assumptions	Source withdrawal limits, solvency, enforceability and tolerated administrative changes
Intermediate funding	Section 7 requires every step's asset balance to be available	Contract timing, fees, queues and any funded bridging credit
Shared resources	Sections 5 and 7 reject summing overlapping pools/collateral/credit	One combined executable route and balance-sheet calculation
Portal at-par profit claim	Sections 4 and 10 retain settlement inequality and counterexample	Any actual premium or separate funding after costs; ordinary debt repayment is distinct
Repaid assets can return to market	Section 7 accounts for withdrawal and unlocked collateral	Seller response and residual claims in a support scenario
Price support versus arbitrage	Section 7 separates desired target from executable profit	Enforceable conversion or actual net favorable price difference
Independent PLS reference	Section 1 distinguishes local reference valuation from external liquidation	Chosen venue/time/quantity/reference path for an actual claim
Finite trade versus durable peg	Sections 6–7 show post-sale price decline and bounded support conditions	Horizon, price band, seller/redemption envelope, costs, reserves and replenishment
Circular collateral valuation	Section 7 rejects using the desired pDAI price as independent funding proof	A recoverable funding basis or explicit funded-loss/credit scenario
Competing markets	Sections 5 and 7 include shared routes and arbitrage supply	Market-wide executable curves and new seller liquidity
Source provenance	Section 8 exposes destination/source distinction and implausible ARB chronology	Independently verifiable source state/message linkage for historical backing claims
Scope of ownership evidence	Sections 2 and 9 retain role-specific and attribution limits	Actual authority over each necessary target at the relevant time
Falsifiability	Section 10 gives route-specific failure conditions	Resolve each necessary premise rather than assume away every failed test
Reproducibility	Original/revised versions, reviews and methods are identified by hashes	Public evidence distribution and independent provider/commitment verification remain outstanding

What the next stronger theoretical result would require

For an informative hypothetical route, choose one exact financing family and specify compatible source and destination interfaces, every claim, the funded output curve, governance changes assumed, costs and timing. Prove its cash and custody ledgers close for the stated quantity. A hypothetical proof need not pretend an actual source RPC or observed historical execution exists.

For sustainable support, add a finite horizon, price band, seller/redemption envelope, PLS-reference changes, losses, maturities and replenishment rules. Then evaluate whether that specified system remains within its funded limits. There is no universal finite-reserve proof against arbitrary selling, and a stochastic scenario study would need explicitly justified distributions rather than numbers chosen to make the desired outcome likely.

For factual publication, provide access to the frozen raw evidence and the reproduction environment. A source checkout excludes ignored artifacts, so links to local paths alone do not make the audit independently reproducible. The review preserved the original provider limitations and the disclosed historical query gap; it did not perform a new external audit or establish participant intent.

Appendix 60: Stability bounds, two-sided correction and adverse tests

Contents of this appendix

1. What can be proved	388
2. The anchor and its capacity	388
What arbitrage does and does not bound	389
Finite resources disprove the unrestricted inference	389
3. Numerical assumptions and reproduction	389
4. Large selling: the complete account.....	390
5. Large buying: the opposite inventory obligation	391
6. Favorable recovery, income and supply concentration	391
7. Corrective feedback needs a timing rule	392
8. Reference, collateral and liabilities	392

Source record: docs/studies/analysis/pulsechain_stability_bounds_20260929.md.

Stability bounds, two-sided correction and adverse tests

29 September 2026. Mathematical analysis and explicitly invented markets. No deployed pDAI conversion facility, source-chain implementation or live execution is established by these experiments.

1. What can be proved

A path is a sequence of valid state transitions with exact asset identities, permissions, unit conversions and funded outputs. If each transition accepts the preceding output, every obligation due is funded, and the final state has the specified pDAI execution value, the composition reaches that value. This is a conditional execution proposition. It does not prove the assumptions or make its initial funding free. Source assets need only be accepted for a finite trade; their independent recoverability matters to losses and repeatability.

The [bridge models](#) (Appendix 13) establish possible exclusive representations; the [financing ledgers](#) (Appendix 42) show who pays for them; the [route atlas](#) (Appendix 51) identifies the separate transition families. A route which returns the starting asset with a positive net surplus and settles its prescribed obligations proves a profitable cycle for that participant. It does not prove a consolidated system profit if another owned account bears the corresponding loss.

2. The anchor and its capacity

Let $s > 0$ be a fixed PLS reference in dollars over the execution interval. A proposed facility bids $b = (1-\delta)/s$ PLS per pDAI and asks $a = (1+\delta)/s$, where $0 \leq \delta < 1$. It starts with spendable PLS R and deliverable pDAI H . Its currently quoted capacities, before other obligations and costs, are

$$\begin{aligned} q_{\text{sell_to_facility}} &\leq R/b \\ q_{\text{buy_from_facility}} &\leq H. \end{aligned}$$

These are units of pDAI. If a user sells q directly at the bid, the facility's PLS falls by bq and pDAI rises by q ; a purchase at the ask reverses these flows with aq . This explicitly supplies two-sided conversion while the quoted inventories remain available. The capacity is shared by all users, not promised independently to each simultaneous order.

With the PLS gas cost g , a direct sale nets reference value $1-\delta-sG/q$, and a direct purchase costs $1+\delta+sG/q$. Thus a tolerance ϵ including gas requires $\delta+sG/q \leq \epsilon$. Fixed gas makes arbitrarily small trades incompatible with a tight all-in band unless someone funds that cost. A minimum size or fee subsidy is part of the service definition.

This is a proposed funded conversion venue, not a pDAI token-contract redemption right found in the audit. It is one explicit way to supply the missing closing trade. A different complete market route can substitute if its executable quotes, timing and capacity provide the required terms.

What arbitrage does and does not bound

Let $c_m(q)$ be PLS paid to buy q in a market and $o_m(q)$ PLS received by selling q there, with market fees and impact included. With feasible funding, facility capacity and total extra cost G , the two cycles are profitable when

Buy market; sell facility:	$bq - c_m(q) - G > 0.$
Buy facility; sell market:	$o_m(q) - aq - G > 0.$

These are sufficient incentives at the stated size, not guarantees of inclusion or execution. Once no such profitable cycle remains at a tested size, $c_m(q) \geq bq-G$ and $o_m(q) \leq aq+G$. Notice what is not implied: these cross-side bounds do not force every AMM sale quote above the facility bid, or every AMM purchase quote below the ask. A market can have a wide spread with no profitable correction at that size. Users need access to the facility or another best-execution route. A finite profitable correction also need not eliminate all remaining opportunities at other sizes.

Finite resources disprove the unrestricted inference

With no intervening inflows or borrowing, a facility cannot pay more than R PLS, nor deliver more pDAI than it holds. Multiple L2 token mints do not change that identity. Even an accepting constant-product market with input reserve x and output reserve y pays only $ya/(x+a) < y$ for finite input a in a fee-free example; arbitrarily many input units do not create additional output inventory. Fees do not remove this bound.

This is not a claim that an exact conversion rate is impossible under every specification. A fully funded service can quote one price for a bounded supply and funded costs. It refutes the stronger inference that controlled validators, nominal balances and arbitrage alone guarantee both directions for unrestricted flow, time and adverse states. The claimed range must match actual resources.

3. Numerical assumptions and reproduction

The fresh manifest is `Research/experiments/pulsechain_stability_review_20260929.json`. The calculation uses 70-digit Decimal arithmetic and checks token conservation with relative tolerance 10^{-45} . This is a mathematical model, not integer EVM parity. Exact raw-token integer cycles remain in the separate [pool-cycle experiment](#) (Appendix 47).

The initial AMM has **1,000,000 pDAI** and **10,000,000,000 PLS**, charges 0.3% on input, and starts at a one-dollar reserve-ratio price using $s = \$0.0001$. The treasury separately holds **300,000 pDAI** and **3,000,000,000 PLS**. Its proposed bid is **9,900 PLS** and ask **10,100 PLS** per pDAI. The arbitrageur owns sufficient working capital and pays **3,000 PLS** gas per correction. There are no competing trades during the correction.

An optional source funding witness stipulates that 1,000 exclusive source receipts are locked, 1,000 destination representations are issued and a buyer pays 3 billion already-owned PLS for them. That finances the starting treasury cash without withdrawing the source investment. This is an accounting witness, not a measured valuation, demonstrated buyer or new source-chain simulation. The 300,000 treasury pDAI is separately preowned.

Each market scenario starts from those balances, except the named unfunded controls. Repeated-pressure scenarios apply eight successive equal shocks. Outside sellers are assumed to own the required total pDAI at the beginning; buyers have the required PLS. No new pDAI is minted. The ledger includes the pool, treasury, shock participant, arbitrageur and gas recipient.

The profit-maximizing single corrective trade can be solved analytically. Write current pool reserves as x pDAI and y PLS and effective input fraction $g = 0.997$. After a sale shock, the arbitrageur spends

$$z_PLS = \max(0, [\sqrt{b*x*g*y} - y]/g)$$

buying pDAI to sell at the facility bid. Clip the output to R/b . After a buying shock, it buys pDAI at the facility ask and sells it to the AMM:

$$z_pDAI = \max(0, [\sqrt{y*g*x/a} - x]/g), \text{ clipped to } H.$$

The formulas differentiate gross cycle surplus; a fixed gas charge leaves the interior optimum unchanged but can make execution unprofitable. No correction runs unless gross surplus exceeds gas. Debt-financed capital would need its own cost and repayment account; none is assumed here.

Reproduce into a fresh file:

```
python3 Research/scripts/model_pulsechain_stability.py --manifest Research/experiments/
pulsechain_stability_review_20260929.json --output artifacts/pulsechain-stability-reproduction/results.json
```

The reference output is `artifacts/pulsechain-stability-review-20260929/results.json`. It records both input hashes, complete balances, cycle profits, market probes, capacity controls, supply-depth examples, net-income example, controller tests and all six simple cycle orientations in the specified four-asset graph.

4. Large selling: the complete account

A forced sale of **200,000 pDAI** into the AMM pays **1,662,497,915.624479 PLS** to its seller. The reserve-ratio reference price falls from \$1 to **\$0.694792**. The arbitrageur then spends **1,604,735,136.488911 PLS** to acquire **193,199.495917 pDAI**, sells that pDAI to the facility for **1,912,675,009.573756 PLS**, and keeps **307,936,873.084846 PLS** after gas.

The treasury now holds **493,199.495917 pDAI** and **1,087,324,990.426244 PLS**. It has accumulated pDAI and spent reserves. The arbitrageur's profit is not treasury income. The pool's resulting marginal reference price is **\$0.987508**. A 1,000-pDAI sale directly to the facility still pays \$0.99 per pDAI before gas, while that sale into the AMM averages **\$0.983572** before gas. The venues are not interchangeable promises.

The original seller received an average **\$0.831249** per pDAI before gas. Later correction does not repair that earlier execution. This is a deliberately specified AMM shock, not a prediction that a best-execution user would select it when a better funded direct bid is accessible. The distinction matters: post-trade recovery and guaranteed in-band execution are different services.

A second equal AMM sale consumes the remaining facility PLS during partial correction, leaving a marginal price of **\$0.832479**. After that, further sales receive no facility-funded correction despite the treasury owning more pDAI. The eight-shock stress finishes near **\$0.190264** in this model. These are finite initial inventories with no income or new funding, not a forecast. The unfunded-bid control performs no correction after the first shock.

The optimum was checked independently against neighboring trade sizes and its closed surplus. Every step reconciles pool, treasury, outside seller, arbitrageur and gas-recipient token movements. Extra source balances alone change none of these cash flows; a separately funded sale or financing would have to replenish PLS before another bid can pay.

5. Large buying: the opposite inventory obligation

Reset the initial state. A buyer spends **2,000,000,000 PLS** into the AMM, receiving **166,249.791562 pDAI** and moving its marginal price to **\$1.43928**. The arbitrageur buys **160,523.354012 pDAI** from the facility for **1,621,285,875.520206 PLS**, then sells it into the premium AMM for **1,932,498,372.003088 PLS**. After gas it earns **311,209,496.482882 PLS**.

The treasury now has **139,476.645988 pDAI** and **4,621,285,875.520206 PLS**. The marginal AMM price is **\$1.012548**. Its 1,000-pDAI purchase quote is **\$1.016618** per pDAI before gas, while the facility still asks \$1.01. Again, access to the better route matters. The initial AMM buyer paid about **\$1.203009** per pDAI before gas; later correction does not refund that price impact.

A second equal buying shock exhausts the facility's remaining pDAI during partial correction, despite its growing PLS reserve. Its marginal price remains **\$1.066216**; after eight shocks without replenishment it reaches about **\$4.968517**. The unfunded-ask control cannot correct the first shock. The analytical buy optimum and token conservation were checked as for selling.

Buying pressure can therefore fund later bids, while selling pressure can replenish inventory for later asks. This is a useful two-sided interaction, not proof that flows arrive in the required order or that all acquired inventory will be resold at a gain. Additional pDAI delivery requires exact-token stock, a valid withdrawal or issuance route, or borrowing with a return obligation. New FRAX issuance cannot be silently substituted for it.

6. Favorable recovery, income and supply concentration

A separate explicit batch, `Research/experiments/pulsechain_stability_recovery_20260929.json`, starts with the actual modeled pool and treasury left by the large-sale case. Its funding witness records the preceding output hash. A subsequent outside purchase of **2 billion PLS** creates another profitable correction: the treasury sells **156,140.782347 pDAI**, receives **1,577,021,901.705877 PLS**, and ends with **337,058.713569 pDAI** and **2,664,346,892.132120 PLS**. The arbitrageur earns **292,741,760.017276 PLS** after gas. The cash reserve has substantially recovered, while it remains below its original 3-billion-PLS level and inventory remains above its original 300,000 pDAI. Their combined value requires a stated valuation; the cash change alone is not total profit.

This shows the useful alternation rather than assuming it. Outside funded buying provides the cash; the timing and occurrence of that buying are premises. Reproduce with the same script, the recovery manifest and a fresh output file. The reference result is `artifacts/pulsechain-stability-review-20260929/recovery-results.json`.

The original batch also reproduces the accumulation appendix's three exact price-impact cases: halved selling at retained depth improves execution; halved selling with 90% less depth worsens it. A separate income calculation produces **\$240,000** gross attributable external LP fees, **\$40,000** collected interest and **\$80,000** funding/operating costs. Net income is **\$200,000** without losses, but **−\$50,000** with **\$250,000** realized losses. The favorable case is in the recovery batch; neither injects this income into the market simulation. Generation, collection and the time available for reinvestment must be separately specified before claiming further support capacity.

The [accumulation study \(Appendix 65\)](#) distinguishes physical token balances, beneficial claims, debt, external wallet float and potential selling through borrowing or withdrawal. Concentration can reduce one seller cohort's inventory without permanently removing assets from trading.

7. Corrective feedback needs a timing rule

The favorable economic feedback is not itself a control law. To expose the role of response strength and delay, consider a deliberately simplified local model. Let e_t be deviation from the target and k the proportional response, with unit-normalized price impact and no new shock or capacity saturation:

Immediate correction: $e_{t+1} = (1-k)e_t$.
One-step delay: $e_{t+1} = e_t - k e_{t-1}$.

Immediate correction contracts when $0 < k < 2$. With one-step delay the characteristic equation is $\lambda^2 - \lambda + k = 0$; both roots lie inside the unit circle exactly when $0 < k < 1$. For $k > 1/4$, their magnitude is $\sqrt{k}$. Thus $k=1.2$ is stable without delay but unstable with this delay. Two independently chosen delayed controllers of gain 0.6, acting on the same error, have combined gain 1.2; individually stable policies need not be stable together.

The batch checks these roots and 40-step numerical sequences. Divergent values outside the initial neighborhood are mathematical diagnostics, not forecast pDAI prices. This linear toy omits AMM curvature, saturation, noise, arbitrage competition and state-dependent liquidity. It refutes the assertion that stronger or more coordinated responses must always improve stability; it does not calibrate an actual controller.

A production proposal would need explicit quote updates, inventory budgets, response limits, shared commitments, oracle validity and finality assumptions. Remote funds cannot serve an immediate quote until delivered or credibly advanced. Under an outage or exhausted capacity the service must stop promising unavailable execution; that is a failure of an unconditional peg claim, even if the bounded service behaves as specified. No controller implementation is claimed in this paper.

8. Reference, collateral and liabilities

The original 3-billion-PLS reserve supports at most **303,030.303030 pDAI** at the \$0.99 reference bid and $s=\$0.0001$, before costs and other commitments. If PLS halves in reference value, the same reserve's capacity halves to **151,515.151515 pDAI**. At $s=\$0.00001$, capacity is **30,303.030303 pDAI**. These are stressed quote budgets, not simultaneous commitments to be added.

If a controller quotes using s_{hat} while the relevant true reference is s , its nominal \$0.99 payout has true reference value $0.99*s/s_{\text{hat}}$ before costs. An erroneous twofold overstatement of PLS value therefore pays only \$0.495 of that reference per pDAI. The reference cannot be validated merely by a market whose target price it is supposed to support.

Collateral haircut, borrowing limits and maturity matter as much as its marked balance. Realizable claim proceeds L , support outflows w , costs c and debt payments due D_{due} must satisfy $R + L \geq W + C + D_{\text{due}}$ before the relevant payments. A later expected return does not repair an earlier shortfall. Future

principal does not all have to be repaid immediately; it remains a later obligation with its own liquidity test. Internal claims cancel on consolidation. Minting claims on self-issued balances can produce accepted trades, but marking those claims at the target cannot prove the resources required to defend it.

Appendix 61: Ten-pass review: method, findings and claim boundaries

Contents of this appendix

Result	394
Review method and questions.....	394
Claim-to-evidence map	395
Numerical and historical verification	396
Reproduction references.....	396
What a deployed proof would still require	397

Source record: docs/studies/analysis/pulsechain_stability_review_20260929.md.

Ten-pass review: method, findings and claim boundaries

29 September 2026. Ten sequential read-challenge-check-rewrite passes over the maintained pDAI thesis, with a separate model-assisted examination of supply accumulation. This is internal analytical review, not ten independent reviews, external assurance, a novelty determination or a certificate of perfection.

Result

The main argument is **1,669 whitespace-delimited words**, compared with 3,106 at the start, while detailed routes, two-sided examples, accumulation accounting and stress conditions are in linked appendices. The surviving claim is a bounded composition: accepted exclusive claims can access funded local markets; profitable cycles can transmit demand; and maintained liquidity with retained net outside income can reinforce later support.

The stronger proposition that controlled validators, unlimited nominal L2 balances and arbitrage alone imply perfect pDAI stability does not survive. Finite output inventories, two-sided obligations, incompatible token identities, reference error, delayed funding and controller overshoot give explicit failure conditions. These do not refute the conditional mechanism. They define the conditions a working system would have to satisfy.

Review method and questions

Each pass reconsidered the current manuscript, challenged a specific inference, checked its mathematical or evidence basis, and changed the main text. A machine-readable record retains the input/output manuscript hashes and word counts for ten distinct revisions; it does not substitute for the supporting arguments. The sequence is an analytical method, not ten votes for the thesis.

Pass	Question tested	Finding retained
1	Does control of issuance prove funded stability?	Authority, accepted claims and output funding are distinct premises.

Pass	Question tested	Finding retained
2	Does each route deliver the exact usable asset and settle?	Eight accounting families require separate permissions, inventory and closing obligations.
3	Does relative-price arbitrage define the dollar target?	A funded anchor or equivalent closing route is needed; costs and capacity constrain execution.
4	Does large selling produce a complete profitable cycle?	The modeled trader profits; treasury pDAI grows while PLS declines and eventually exhausts.
5	Does large buying restore the other side?	The modeled trader profits and treasury PLS rises; pDAI inventory can exhaust instead.
6	Does accumulation create favorable reflexivity?	It can, with maintained depth and retained net outside income; lending and LP holdings remain accessible through their rules.
7	Is stronger coordinated correction always stabilizing?	Delay and combined response strength can turn correction into instability.
8	Which claims are observed on-chain?	Saved receipts confirm component trades and particular mints; the complete proposed system remains unverified.
9	Do the bounds withstand adverse examples and reproduction?	Repeated flows, empty reserves, reduced depth, losses and reference error refute the unrestricted claim; batches reproduce.
10	Can the argument be simpler without losing its tests?	The main paper states the mechanism and scope; proofs, examples and full route diagrams remain traceable in appendices.

Claim-to-evidence map

Material claim	Status and support
A represented receipt can remain invested at source	Conditional custody/accounting proposition; historical bridge models (Appendix 13) .
Each named stablecoin already has unrestricted reverse minting	Not established; exact queried routes and token-specific capabilities in the issuance inventory (Appendix 4) .
Indirect pDAI-supporting arbitrage exists	Finite numerical witnesses in the pool-cycle study (Appendix 47) , with an opposite-direction counterexample.
A funded bid and ask supply bounded two-sided conversion	Proposed design and arithmetic conditions in the stability bounds (Appendix 60) ; not an audited deployed facility.
Large buying after selling can restore support cash	Dependent recovery experiment using the prior scenario's actual ending balances; stability bounds (Appendix 60) .

Material claim	Status and support
Accumulation can reduce later sell pressure	Conditional comparison at maintained depth; supply accumulation (Appendix 65) . Reduced depth reverses the example.
More ownership necessarily means more spendable income	False without activity, attribution and net-cost conditions; the same accumulation study separates fees, claims, cash and losses.
Demand inevitably starts at \$1 after sellers leave	Unverified behavioral hypothesis. Timing, size, persistence and return of selling require evidence.
Curve, Convex, Frax, Aave and Balancer can be financially connected	Published architecture and selected local component evidence; pool-cycle study (Appendix 47) , route atlas (Appendix 51) .
Controlling validators makes the system decentralized	Does not follow. Operational dependence, entry, governance and failure of the controlling group require their own specification.
Current PulseChain records prove a perpetual peg	Not established by the evidence audit (Appendix 32) .

Numerical and historical verification

The original pool-cycle batch was reproduced byte-for-byte, including its integer conservation checks. The new stability and dependent recovery batches also reproduced byte-for-byte. They use 70-digit Decimal arithmetic and relative conservation tolerance 10^{-45} ; they are not EVM execution tests. The two analytical correction optima were separately checked against neighboring trade sizes and closed surplus formulas. Supply-depth examples were compared with exact fractions, and local controller stability with characteristic roots.

The new tests include large selling, large buying, eight-shock one-sided stress, zero-bid and zero-ask capacity, buying after selling, PLS-reference declines, retained versus withdrawn depth, positive gross fees with either positive net income or net loss, and delayed corrective responses. Some are deterministic scenario calculations and others elementary inequalities. They do not sample all market states, prove global dynamic stability or forecast adoption.

An offline recheck matched **820 saved Curve events** to logs in their successful receipts, in **820 distinct transactions**. It also rechecked exact input/output transfers in transaction

0x7f61b4042a71edb340898e3ad46abc01c4d8ddf7197594d077cbfb426c67a5f1, at block 27,264,036:

13,489.157023144354307302 mUSD into the pool and **564.501482257993107424 DAI** to the examined wallet. The saved reserve summary confirms zero cash and unbacked caps for the three sampled stablecoin reserves. This is a check of retained records, not a new chain-wide query or independent node consensus. The older appendices retain their own original audit scope.

The repository research suite completed with **63 tests and 82 subtests passed** using `./scripts/test.sh --research`. Those regression tests cover the research codebase; the explicit numerical checks above validate the new model results. Passing either is not proof that the proposed financial system is deployed.

Reproduction references

The maintained inputs are:

- `Research/experiments/pulsechain_pool_cycles_20260929.json`, using `Research/scripts/model_pulsechain_pool_cycles.py`.

- Research/experiments/pulsechain_stability_review_20260929.json, using Research/scripts/model_pulsechain_stability.py.
- Research/experiments/pulsechain_stability_recovery_20260929.json, using the same stability script and the preceding final-state dependency.

Fresh output commands and reference locations are in the two numerical studies. The outputs include script and input hashes. The scoped ignored directory artifacts/pulsechain-stability-review-20260929/ contains results.json, recovery-results.json, adverse-checks.json, historical-evidence-check.json and review-checkpoints.json. Source snapshots and retrieval hashes are under its sources/ directory. These are working evidence, not duplicate prose reports.

The anonymous release supplies the concise thesis, consolidated appendices, maintained inputs, scripts and scoped evidence, with an integrity manifest. Public identifiers and historical hashes retain their meaning; release hashes identify the supplied sanitized bytes. Prior packages and frozen experiments remain intact.

What a deployed proof would still require

One must specify the actual controllers and loss-bearing parties; operating source chains; exact token and contract mappings; valid message paths and finality; source custody and destination liabilities; funded bids, asks and withdrawals; a reference policy; flow, timing and collateral bounds; and a tested rule for reinvestment and adverse conditions. Historical execution must then connect those components in the same actual account.

A mathematical guarantee can quantify a declared model and bounded disturbance set. No finite collection of examples or repeated AI agreement certifies all possible future economic behavior. The purpose of this review is to expose those quantifiers and make the positive mechanism testable, not to replace missing premises with rhetorical certainty.

Appendix 62: Routes and operating conditions for a stable pDAI

Contents of this appendix

Scope and definitions	398
Financing routes	398
Markets and maintenance routes	401
A simple proposed operating rule.....	402
Measure capacity without counting resources twice	403
Authority and change path.....	404
Reproduce without altering frozen results	404
Stress cases and completeness boundary.....	405

Source record: docs/studies/analysis/pulsechain_stability_route_design_20260930.md.

Routes and operating conditions for a stable pDAI

30 September 2026. Study of the coordinated proposal. This is a synthesis of saved evidence and offline design reasoning; no new blockchain calls or transactions were made for this edition.

Scope and definitions

This study enumerates every accounting route family identified in the recovered research. It is not an inventory of every possible future deployment, an exhaustive pARB application catalogue or a claim that every route currently operates. The destination Curve pool census covers 619 distinct pools returned by eight responsive roots at PulseChain block 27,675,130, hash

0xdd12f23ad571882b63fd333c5d3909a45a05fe6f0a3c2b273cb7e1e6b0917954. Other supporting studies retain their own block identities. Historical observations are not synchronized into a fictional single live state.

“Observed” below means a saved destination getter, source condition or balance supports the narrow statement. “Conditional” means the complete route depends on unresolved predicates. “Proposed” means a design rather than a demonstrated deployment. Contract identities, rights and obligations survive consolidation; claims between controlled entities are eliminated when calculating net resources.

Financing routes

ID	Inputs → output	Role and boundary	Evidence/status
F1	Authorized source stable inputs → L2 OUSD/eUSD/mUSD → bridged representation → accepted local collateral → existing local stable	Central issuer route. Source issuance, source issuer deployment, bridge mapping, plugin/accounting and local limits must all work.	Proposed end to end; local issuer predicates examined in the issuer network (Appendix 38).

ID	Inputs → output	Role and boundary	Evidence/status
F2	Source stable inputs → transferable lending/vault/ staking receipt → bridge → local issuer	Direct receipt variant of F1. A source position must have rights that the destination collateral adapter can value and realize.	Conditional. Historical Aave wrappers and Curve/GMX baselines are bounded; not evidence for every source protocol. Native baseline (Appendix 15) , wrapper inventory (Appendix 24) .
F3	Source assets or positions → bridge → accepted LP-share collateral → local issuer	LP collateral variant. Pairing alone is not issuer acceptance; nested assets, fees, rates and withdrawal limitations must be tracked.	Proposed for the new asset; existing plugins and issuer accounting require inspection. Reserve collateral (Appendix 18) .
F4	Owned eligible local collateral → local OUSD/ eUSD/mUSD or other issuer output	Avoids source-chain and bridge dependence for that branch. Output is a liability or collateral-backed claim under the issuer's rules.	Ordinary capability examined; complete current mintability remains conditional. Corrected mUSD getter quotes are not execution. Issuer network (Appendix 38) .
F5	Effective minter/pool/credit authority → existing DOLA/ XAI/aUSD/FRAX or other local token	Shorter route when authority exists. Credit, collateral and ceilings differ; minted units are not verified dollar proceeds.	Narrow source capabilities observed; common control and some controller execution remain open. Authority map (Appendix 29) .
F6	Authorized Portal unbacked mint → aToken → compatible wrapper → market proceeds	Temporarily separates claim issuance from underlying provision. Wrapping itself obtains no counter-asset; a sale needs a funded buyer or available pool output.	All 17 sampled unbacked caps are zero. Both issuance and backing require a Bridge caller. Mint also checks active/unfrozen/ unpaused status and ordinary supply headroom including treasury accrual; backing transfers matching underlying plus supplied fee using allowance. Saved waDAI deposits encounter a separate supply-headroom obstruction. Changes and headroom required. Portal pricing (Appendix 48) , deferred study (Appendix 34) .
F7	Valid local collateral → borrow existing reserve cash or mint collateralized debt asset	Supplies tradeable assets with debt service and liquidation exposure. Loans do not automatically create reserve cash. LUSD is the immutable/native-collateral example; Silo/XAI requires its particular credit-market conditions.	General contract family identified; no universal current loan capacity certified. Local financing (Appendix 42) , issuer network (Appendix 38) .

ID	Inputs → output	Role and boundary	Evidence/status
F8	Matching caller-owned aTokens → borrower debt repayment → valid collateral withdrawal	Can release owned collateral for later use. Debt in each token is separate; execution rights, health and cash govern release. It is not Portal settlement.	V3 account debt measured; V2 unresolved, account control not proved. Borrower positions (Appendix 21) , deferred study (Appendix 34) .
F9	Funded source transfer → standard gateway → destination escrow release	Moves an existing asset. Source minting does not establish destination escrow capacity.	Bounded mappings/escrow examined. Gateway census (Appendix 11) , bridge inventory (Appendix 4) .
F10	Supported source deposit → Across relayer liquidity → destination asset → later reimbursement	Earlier access to existing destination inventory. Supported addresses, chain configuration, fees, relayer funds and settlement determine capacity.	Saved Hub/controller examined; arbitrary new wrapper support not proved. Across control (Appendix 3) , accounting (Appendix 14) .
F11	Source-native/reverse/special gateway → token-authorized destination mint	Different from escrow. A token-specific GRT shortfall mint is evidence for that token, not universal stablecoin minting. Exclusive source custody/ burn where required, authenticated messages, exact destination authority and limits must hold.	Historical token-specific reconciliation. GRT study (Appendix 12) , specialized history (Appendix 20) .
F12	Owned PLS/other treasury, outside contributions or funded new buyers → market conversion → spendable assets	Acquires actual proceeds. Appreciation helps only when realizable without defeating commitments; outside funding is an assumption until supplied.	Saved local markets, conditional funding. Copied WETH can wrap PLS. Local evidence (Appendix 41) .
F13	Correctly owned resources → outside lending interest, staking income, LP fees, protocol revenue or sold rewards	Replenishment and gradual settlement after costs. A reward price is not a sale; a claim may be encumbered or illiquid.	Potential income and modeled cycles, not guaranteed future rates. Pool cycles (Appendix 47) , deferred study (Appendix 34) .
F14	Funded inventory → price differences across venues → realized net arbitrage proceeds	Can improve execution or earn external transfers. Must include all legs, fees, gas, slippage and residual exposure; repeated controlled trades are not consolidated income.	Conditional opportunity; no unbounded profit claim. Route atlas (Appendix 51) .

F9/F11 also cover researched Omnibridge representations and Polygon mintable-predicate examples. They use their own authentication and token permissions; they are not automatically pARB routes. See the [bridge comparison \(Appendix 7\)](#).

F2/F3 include transferable vault shares or position claims, including NFTs where a compatible buyer, lender or adapter makes them usable. F7/F14 may use flash credit for a local cycle repaying principal and fees in the same transaction. Delayed bridge reimbursement cannot repay that same atomic loan afterward. See the [receipt and validator study \(Appendix 49\)](#). These are variants, not new independent budgets.

A source protocol name does not make a route viable. The next source inventory must record chain/RPC, contract address, active implementation, deposit token, receipt token, redeemability, beneficiary rights, oracle, limits, and supported bridge output. The issuer destination then needs an exact collateral address and functioning acceptance path. Until that inventory exists, F1-F3 are architectural routes, not a verified list of all L2 protocols.

Markets and maintenance routes

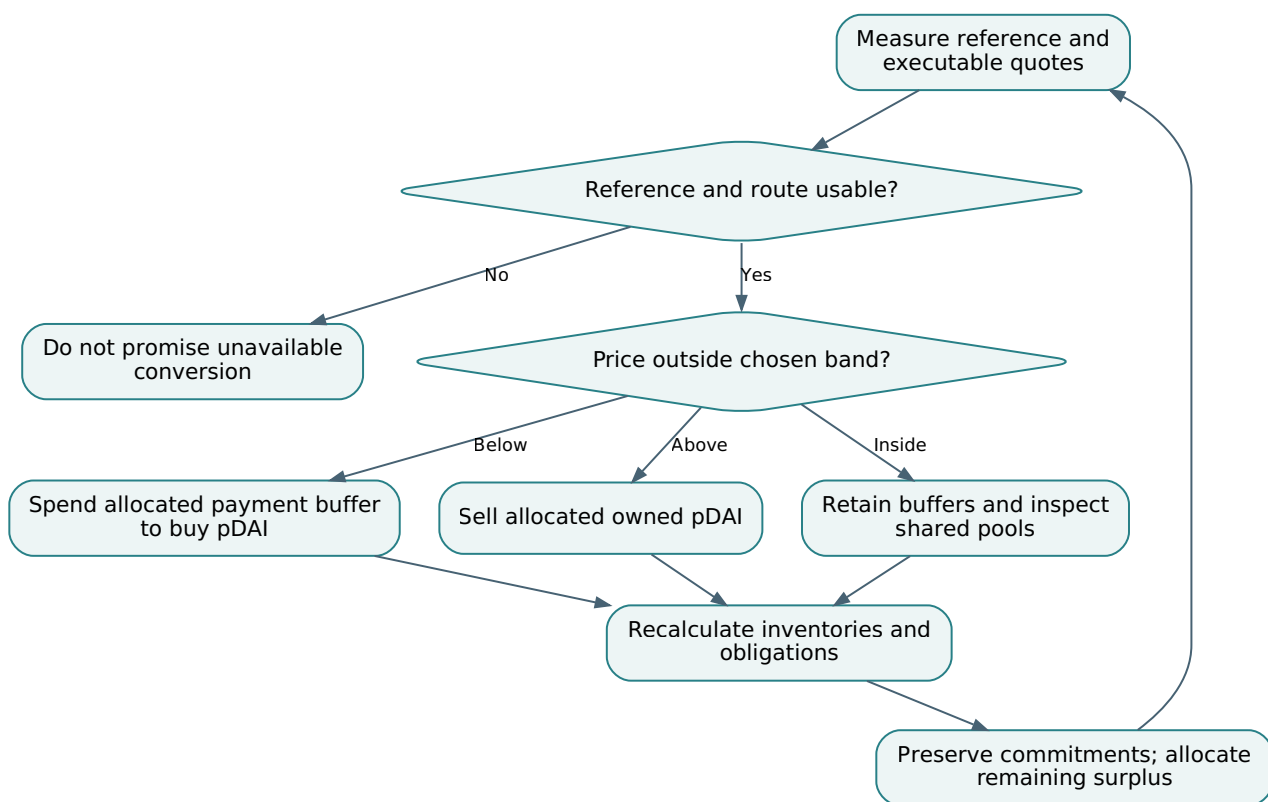
Financing creates or moves inputs. Markets exchange them. Reuse of a market does not add another independent financing source.

ID	Operation	What it accomplishes	Limits and evidence
M1	Existing issuer token → Curve metapool/base pool → pDAI/pUSDC/pUSDT	Acquires base assets by swap. Exact 3CRV and FRAXBP identities are followed through separate graphs.	Shared reserves, current quotes, rates, fees and coin-list identity. Issuer network (Appendix 38) .
M2	Eligible assets → Curve/Balancer/V2 deposit → LP share; later withdrawal	Supplies liquidity or invests surplus. Does not acquire the same output as a swap. Underlying accounting determines exposure.	Pool-specific linear limits and share withdrawal. Pool limits (Appendix 16) , liquidity growth (Appendix 40) .
M3	Counter-asset → Atropa/native/other local pairs → pDAI or neighboring asset	Additional routes, intermediates and fee opportunities.	A large nominal reserve need not have independent dollar value. Burned LP shares cannot fund treasury withdrawals. Atropa (Appendix 27) .
M4	Payment inventory → below-target pDAI; owned pDAI → above-target payment inventory	Two-sided managed band. This is the baseline proposal for V2 venues.	Finite buffers, size/horizon, executable fees, intervening transactions and response time. Managed peg (Appendix 45) , bounds (Appendix 60) .
M5	Inventory → alternative pool depth, concentrated liquidity or fixed-rate conversion	Optional tighter execution. Plain treasury conversion can exchange owned tokens; minting pDAI requires its own authority.	Proposed alternatives, not established deployments. Liquidity range, outside flow and resource limits remain.
M6	Existing local pDAI → PLS/native market exit	Allows local users to receive a native asset at the market rate.	Native purchasing power varies; wrapped PLS does not become Ethereum ETH. Local evidence (Appendix 41) .

ID	Operation	What it accomplishes	Limits and evidence
M7	Authorized external pDAI representation → funded Ethereum ETH/WBTC LP exit	Proposed external market conversion, independent of a dollar redemption commitment.	Representation, bridge rights, genuine external reserves, fees and capacity unproved. LP exit (Appendix 45) .

A funded fixed-rate converter is conceptually distinct from V2 reserve-ratio pricing. Maker's historical [PSM implementation](#) supplies a documented example with collateral exchange and mint/burn permissions. No copied PSM route, pDAI mint authority or new converter deployment is established here. An inventory-only converter would need both owned pDAI and payment assets; its quotes cease when either side is exhausted.

A simple proposed operating rule



This is a controller specification, not deployed code. An operator submits actions; contracts execute submitted transactions. Preserve support and settlement commitments before optional yield. The reference must be defined before any “below” or “above” comparison. A local reference requires executable parity; a dollar reference also requires independent economic validation. For dollar parity, payment units per pDAI equal one dollar divided by the reliably measured dollar price of the payment asset, before costs. Revalue coverage when that price changes. Fixed local parity is a separate promise.

Time-weighted observations measure a market history and cannot make a frozen accounting number payable. The [Uniswap oracle documentation](#) describes cumulative observation mechanics, not a dollar guarantee.

The proposed ten-second cadence is an allocation schedule, not a promise of inclusion in every block or a guarantee that an old quote remains executable. Each eligible action must respect current funds, size limits, fees and settlement commitments. Record skipped slots explicitly rather than treating unspent allocations as completed purchases. Transaction ordering and competition can worsen realized execution even when a prior quote was favorable.

Below target, buy with the least costly usable route within the allocated budget. Above target, sell owned transferable pDAI; issuer minting of OUSD or eUSD does not itself mint pDAI. Inside the band, avoid spending simply because a slot arrived. Use net surplus to relieve shared-pool shortages, earn collectible income or reduce liabilities according to priorities defined in advance.

Measure capacity without counting resources twice

Define a horizon H , a payment asset, a permitted execution band and the maximum cumulative outside selling and buying flow to be supported. The commitment must address repeated trades, not just one marginal quote. Define permitted outages and the response time during them.

Report:

- **Execution:** net payment proceeds or purchase cost for a fixed ladder of sizes, including an illustrative 50m-pDAI case where relevant.
- **Price:** reference deviation, observation window and target-restoration time.
- **Immediate liquidity:** owned immediately deliverable payment units and pDAI; encumbered and delayed resources listed separately.
- **Solvency:** realizable consolidated assets versus enforceable outside liabilities; show stressed recoveries separately. System-owned pDAI at its defended price is not independent counterasset backing.
- **Obligations:** borrower debt, Portal unbacked principal, issuer collateral promises, bridge reimbursement and existing LP/credit commitments in their own denominations.
- **Income:** collectible outside proceeds less all costs and losses; internal transfers eliminated.
- **Exposure:** outside float plus credible future mints, unlocks, borrowing and bridge arrivals over H .

Do not add an LP share's value to its underlying assets, a wrapped receipt to the same aToken, or a Reserve/Origin/mStable output to its collateral as independent capital. Buying pDAI changes portfolio composition; it does not automatically create profit. Paying an external user removes payment capacity. New buyer inflows replenish it only when actually received.

The arithmetic $\text{deliverable payment units} / \text{promised payment units over } H$ is a useful fixed-rate coverage indicator under matching units and explicit netting assumptions. It does not establish a V2 slippage bound. V2 needs reserves, fee and ordering calculations; a yield forecast does not count as immediately deliverable reserves. Saved [stability \(Appendix 60\)](#) and [liquidity \(Appendix 40\)](#) models retain their toy assumptions and are not reclassified as live capacity.

Authority and change path

Branch	Saved authority finding	Remaining proof
Portal	ACL manager 0xc2aacf6553d20d1e9d78e365aaba8032af9c85b0; short executor 0xee56e2b3d491590b5b31738cc34d5232f378a8d5; saved admin 0xec568fffba86c094cf06b22134b23074dfe2252c	Accepted admin control, required roles, positive caps, all execution and market predicates. Saved pending admin 0xa30190b96faee0080144aa0b7645081fcbf49e6f may immediately accept if it controls the caller and state is unchanged; subsequent newly queued actions require at least 86,400 seconds and grace-period checks. Pending nomination is not accepted control.
Origin	Governor timelock 0xc35918cde7233f2dd33fa41ae3cb6ae0e42e0e69f; unverified delegated admin and price provider	Effective proposal/execution rights and compatible full acceptance path. Two-day minDelay is only one clock; proposer/executor role holders remain uncensured.
Reserve	Saved Main OWNER timelock 0xc8ee187a5e5c9dc9b42414ddf861ffc615446a2c; proposer Governor 0x7e880d8bd9c9612d6a9759f96acd23df4a4650e6	Effective voting/execution, compatible collateral plugin, sound basket after refresh and issuer capacity. Saved voting delay 14,400 blocks, voting period 21,600 blocks and three-day timelock are distinct; block duration is chain-dependent.
mStable	Nexus governor Safe 0xf6ff1f7fceb2ce6d26687eaab5988b445d0b94a2, 3-of-5; proxy admin 0x5c8eb57b44c1c6391fc7a8a0cf44d26896f92386 reports seven-day delay	Actual signer control, complete upgrade path and compatible implementation. Safe owners are in the issuer study.
Other stablecoins	Token-specific operators, owners, pools or minters identified where getters/source allow	Controller-level effective execution and economic acceptance; holding a governance token alone is insufficient.
Bridges and borrower	Exact route-specific permissions and account rights	Common coordination, relayer capacity and lawful execution rights are not established by token balances.

The [claims and authority map \(Appendix 29\)](#) is the detailed register. This table does not certify which person controls an address. No changes were attempted, no keys accessed and no state-changing simulations executed.

Reproduce without altering frozen results

The offline issuer analyzer rewrites catalogues in its `--folder` directory. Copy the saved issuer evidence to a fresh explicit artifact batch before rerunning it; never point a reproduction run at the frozen original. Retain corrected inputs, excluded malformed batches and their recorded hashes. The [issuer study \(Appendix 38\)](#) gives the decoder command and input identity. Published getter/source statements and proposed controller rules remain distinct.

Stress cases and completeness boundary

Evaluate sustained selling, sustained buying, shared-counterasset depeg, stale valuation, failed collateral redemption, bridge outage, missing operator responses, fees exceeding income, source issuance increases and scarce base-pool inventory. A frozen price feed is a failure of economic measurement even when the contract accepts it. A pause can preserve remaining resources while making conversion unavailable; availability must be part of the promised service.

The specific stress magnitudes, target band, H, funded balances and loss policy are not yet chosen. No new live-capacity test was performed for this synthesis. A complete economic validation would need those quantities, all applicable authority and bridge predicates, and independent review of the deployed implementations. Agreement between reviews can improve this document and find contradictions. It cannot validate an unknown deployment, turn a conditional forecast into measured income, or prove flawlessness.

Compare out-of-band execution, unavailable conversion time and settlement shortfalls at the same budget, sizes and horizon. No optimal result or statistical confidence has been measured.

The simplest candidate is a named reference, one or a few functioning funding routes, both inventories, sufficient accessible depth and an explicit response rule. Additional mechanisms should earn their place by adding measured capacity or independent resilience.

Appendix 63: StakeWise proxy changes and Curve asset routes

Contents of this appendix

Contracts and checkpoint.....	406
What the three supplied transactions establish.....	407
The deposit pool was separately swept.....	408
Observed Curve activity	408
What Curve accepts and how it prices trades	409
Where an L2 asset could fit	409
Evidence and reproduction.....	410

Source record: docs/studies/analysis/pulsechain_stakewise_proxy_routes_20261002.md.

StakeWise proxy changes and Curve asset routes

2 October 2026. All blockchain work is read-only. This study separates the StakeWise deposit pool, the sETH2 token proxy and the Curve trading pools. The controller demonstrably installed custom minting implementations, then later pointed sETH2 at incompatible Pool code. The refreshed destination contracts do not establish an operating pARB bridge or an Arbitrum price feed.

Contracts and checkpoint

The refreshed PulseChain checkpoint is block **27,690,169**, hash 0x9805ab8cd6142ff6e69b82049318440b925e0c368880008809ac7c3f16943e70. The inherited comparison uses block 17,232,999. Historical event collection begins at 17,233,000.

Component	Address	Finding at the refreshed checkpoint
sETH2 token proxy	0xfe2e637202056d30016725477c5da089ab0a043a	Implementation is 0xe68e649862f7036094f1e4ed5d69a738acde666f; ERC20 supply, decimals, metadata and sampled balance getters revert.
StakeWise deposit pool	0xc874b064f465bdd6411d45734b56fac750cda29a	Implementation is 0x2cedcb52a40d28b8571f63aff9381c1f7d8711fa; native balance is zero.
ProxyAdmin	0x3eb0175dcd67d3ab139aa03165e24aa2188a4c22	Admin of both proxies; owned by StakeWise Safe 0x144a98cb1cddb23610501fe6108858d9b7d24934.
Curve pair A	0xd1011b637f979a5d9093df1b32e7736c289024f5	Coins are sETH2 and local stETH. LP supply and direct stETH balance are zero.

Component	Address	Finding at the refreshed checkpoint
Curve pair B	0xe95e4c2dac312f31dc605533d5a4d0af42579308	Same coins; direct stETH balance is 9.099824936700059818. Balance and quote methods that depend on sETH2 revert.

Both Curve pools are minimal proxies pointing to 0x24d937143d3f5cf04c72ba112735151a8cae2262. Its PulseChain runtime exactly matches the deployed bytecode in the saved, verified Ethereum explorer page. The same comparison confirms that the PulseChain runtime at E68 is the verified StakeWise **Pool**, not StakedEthToken. These are exact byte comparisons, rather than names inferred from an explorer label.

The earlier Safe census found a change from 4-of-7 to 1-of-8, with 0xa8ed85b1d441da00270ecc6dc3600f3cba9d4a03 both an owner and an enabled module; the original owners remained. The refreshed ProxyAdmin ownership and admin-slot reads independently establish the downstream upgrade path. They do not establish exclusive human control or ownership of Curve LP shares.

What the three supplied transactions establish

MINTER_ROLE means permission to create new token units. It is a named access-control role stored by a contract, not collateral, a token or a bridge. A mint function may require membership of that role and any additional conditions its implementation imposes. Granting the role does not itself move an asset across chains or prove backing. The role identifier is a hash of its name; its role members are the addresses allowed to use it.

UTC time	Transaction	Successful contract changes
20 June 2026 10:42:55	0xdbc70e7a30cd054d7ceb8bcc4eb558b827266de5608bf5e13c3efff551f51a9b	The Safe upgraded rETH2 to 0x664ef6c0869d6fbfa9d64321c28553870793980b and sETH2 to 0x0257e3c97a4e96fbb68e7c25fdab16eda5e3b2cc. Both upgrades called <code>initializeMinter(address)</code> with a8ed.
8 July 2026 23:05:55	0xfe87b49c78faeec0d0e62aef26484d42c21144dd8f83a796b25f21bcff655b7c	The Safe upgraded SWISE to 0x9ce408d497f42dd56216a55bc0e85c1dc9ba8c7c and called the same minter initializer for a8ed.
14 September 2026 21:31:35	0x37a58e1b8c632a40bcd42c209747963ff191cb5a547e9c2a311205b9710d8ec4	The Safe changed all three token implementations: rETH2 and SWISE returned to their inherited implementations; sETH2 was explicitly pointed to E68 Pool code.

The first two receipts contain `RoleGranted` events whose role identifier equals Keccak-256 of `MINTER_ROLE`. Getter reads immediately after the June transaction confirm a8ed as the single member of that role on both sETH2 and rETH2, and confirm functioning sETH2 ERC20 getters at that historical checkpoint. The custom implementation bytecodes contain ERC20 and `mint(address,uint256)` dispatch entries and minter-specific checks. The custom source is unverified, so selector presence alone is not a complete reconstruction of the minting path, collateral rules or caps.

The role was followed by actual issuance. Three receipt-verified a8ed transactions called the sETH2 proxy's `mint(address,uint256,address)` selector 0xd4d1513, creating **1,000,000**, **1,000,000,000,000** and **2,000,000,000,000** sETH2 for a8ed. The transactions are

0x160bc7b4716036f34c73b42b609347a6a0ccafba0c28ecf4c2e27df7adb87176,

0xc15b9e1bdd1c4607d39a1abf74ee8df669e6fb5f2809937fcb2253ad7b66bb37 and 0xa895ca0985edafbf173f759dbee2b1183bb973400db0055a9efc234891aa5e9c. Their successful historical traces show direct local calls from a8ed, delegated custom token logic, and rETH2 checkpoint calls. They show no incoming L2 messenger, bridge call or collateral-token transfer in those mint transactions. An off-chain financing arrangement or separately held backing is not excluded. The amounts are nominal sETH2 units, not verified ETH or dollar backing.

The third transaction directly explains the current sETH2 failure. Its inherited implementation was 0x41bcac23e4db058d8d7aabe2fccdae5f01fe647a. It is not a case of confusing the token proxy with the separate deposit-pool proxy. Pool code has staking and validator functions, but lacks the ERC20 entry points expected at the token address. The evidence establishes the explicit incompatible implementation selection; it does not establish the operator's reason for choosing it.

The deposit pool was separately swept

On 19 June 2026 at 06:09:25 UTC, transaction

0x05f045f7ab0bf56a2d3c728a63c45342db5a7bb88783eb14cc5088c8f36e45ea upgraded the deposit pool to 2ced through the Reality module, Safe and ProxyAdmin. The new runtime is 315 bytes. Inspection shows a single dispatched `sweep(address)` function using the proxy's entire native balance as payment value, with no caller authorization check in that implementation. It contains no ERC20 collateral intake, token issuance or oracle integration.

At 06:11:15 UTC, **transaction index 0** of block 26,822,460,

0xbcd4d20a304c487b23c6035afeac88acd84c7ff029135c0366b3b8be72ac8ad4, caused the deposit pool to transfer **167,165,746.693072211927428963 PLS** to 0x547a396e22f06735aad0e54820ae10137e5ae792. The transaction succeeded; the internal transfer value and before/after native balances corroborate the amount.

The previously identified Reality-module `sweep167` call, transaction

0x8cd90d9cac79ce6b4e1a38fe53fdacd264fb7b737b084e3abd29c534a25e7390, was **index 14 in that same block**. It called the pool with recipient 0x7563550940b136ddcd553a279cf025f704e7cf5b, but transferred **zero**, because the pool was already empty. Its replayed state difference does not show a pool-balance change. The evidence distinguishes the actual recipient from the later proposed recipient; common ownership and transaction intent are unverified. These are native PLS amounts, not ETH amounts or dollar valuations.

This sweep is a deposit-pool cash withdrawal. Curve liquidity changes require separate pool-specific evidence.

Observed Curve activity

The selected postfork Curve event queries completed without errors for both exact pool addresses and the matched implementation's five liquidity/exchange signatures. Pair A returned no selected activity. Its inherited LP supply and both token balances were already zero, so its present emptiness does not establish a postfork withdrawal. Pair B returned three deposits and three swaps, with no matching LP withdrawal events. Its inherited direct stETH balance was 0.099824936700059819, compared with about 9.10 now. The broader token-mint census has eight query errors in other ranges; it is not a complete token issuance census. A separate error-free sETH2 query over the custom implementation period returned the three direct mints above, all verified against receipts and historical traces.

On 2 August 2026, address 0x0c0330d30038ca28b99c1d1f1e92bf3e287c8f17 deposited

210,411,070.397572542914872665 sETH2 and zero stETH into pair B in transaction

0x792818f827a143bc11584f2698f47553152f28eca4bce3f41d021f000cf3552e. Its trace confirms Curve used the existing sETH2 address and delegated to the custom 0257 implementation. This proves that large quantities of

the token could enter Curve under the changed implementation. It does not establish that those deposited units came from the three a8ed mints, or that the depositor and minter share a human controller.

The subsequent three swaps went **from stETH into sETH2**: 1 stETH purchased about 149.87 million sETH2; 3 purchased about 28.99 million; 5 purchased about 10.18 million. This observed direction supplied stETH to the pool. It is not evidence of those trades extracting stETH to finance pDAI purchases. The enormous unit disparity is consistent with pool imbalance and is incompatible with treating this history as execution at one ETH per sETH2.

What Curve accepts and how it prices trades

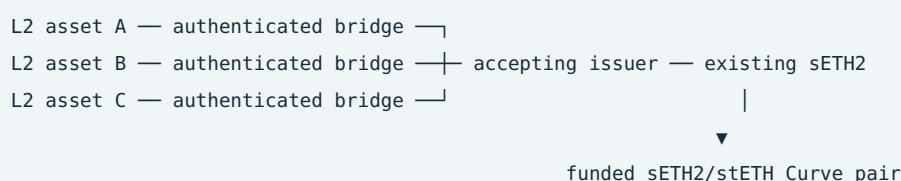
The matched Curve implementation sets two coin addresses and rate multipliers at initialization. It reads each configured token's `balanceOf(pool)` for balances and computes quotes from those reserves, amplification and fees. Transfers use the configured token addresses. The inspected implementation has **no external price-feed input or Arbitrum oracle hook**. Initialization is guarded against repetition; its ABI exposes no coin replacement setter.

Consequently, sending an unrelated bridged ERC20 to a pool does not count it as sETH2. Matching a name, symbol or claimed price is insufficient. Changing the StakeWise deposit pool does not change Curve's coin list. Changing the logic at the existing sETH2 token address could preserve Curve's accepted address, provided the replacement correctly implements balances, transfers, storage and issuance.

The installed Pool implementation does not expand a Curve coin slot into a list of collateral assets. It currently prevents the token reads on which both Curve pairs depend. Restoring working token logic would be an additional authorized change, not a function of the current Pool code.

Where an L2 asset could fit

An operator could design an issuer that accepts several bridged assets and creates one fungible sETH2 claim at the existing token address:



This is a proposed adapter route. Curve would still have one sETH2 coin and one stETH coin. The multiple underlying assets would be accounted for by the issuer, with its valuation, mint limits, redemption policy and obligations. A token that cannot be authenticated or valued could still be admitted by a permissive custom issuer, but that admission alone would not create backing or independent purchasing power.

An authenticated cross-chain message could alternatively authorize minting at the local token address. The historical minter changes establish local issuance permissions; they do not identify a pARB messenger, gateway mint function, source deployment or successful message carrying an L2 asset into sETH2. A controller with local mint authority may not need an L2 transaction to issue tokens, depending on the implementation's additional checks.

At the refreshed checkpoint, the copied Arbitrum router selects its **standard ERC20 gateway** for sETH2. That gateway computes source counterpart `0xc0f8bc165b23b3c91248bde174480d536fc7a8c5`; the custom gateway's sETH2 mapping is zero. A calculated counterpart is not evidence of source deployment.

Ordinary standard-gateway settlement releases the existing destination token from escrow; this does not establish a new direct bridge-mint route. The destination sETH2 balance getter currently reverts, so its escrow quantity cannot be certified by that getter.

An oracle could inform a future issuer's conversion rate, but it would not directly set this Curve pool's executable price. One dollar of stable collateral and one ETH-denominated sETH2 claim are different units; valuation, buffers and redemption must handle that difference. Swaps obtain only the counterasset inventory actually available. At this checkpoint pair A has no LP supply or stETH, pair B has about 9.10 local stETH, and the sETH2-dependent quote path is broken. These observations do not support a currently operating large financing route.

Evidence and reproduction

Raw requests, responses, historical traces and source snapshots are under `artifacts/pulsechain-stakewise-proxy-20261002/`. The canonical upstream identities are [StakeWise's published deployment list](#). Exact runtime comparisons use the saved verified explorer pages for [E68 Pool](#) and [Curve's shared implementation](#).

Explicit manifests under `Research/experiments/` are `pulsechain_stakewise_proxy_routes_20261002.json`, `pulsechain_stakewise_proxy_history_20261002.json`, `pulsechain_stakewise_trace_reads_20261002.json`, `pulsechain_stakewise_custom_minter_getters_20261002.json`, `pulsechain_stakewise_curve_activity_20261002.json` and `pulsechain_stakewise_curve_inherited_20261002.json`. Getter collection uses `Research/scripts/probe_pulsechain_routes.py --calls <manifest> --block <checkpoint> --output <new-file>`. Event collection uses `Research/scripts/collect_pulsechain_events.py --manifest <manifest> --output <new-file>`. Historical minter getters use block `0x1996d79`; the fork-boundary getters use `0x106f467` and are saved in `curve-fork-boundary.json`. The earlier `curve-inherited.json` actually sampled block 17,233,447 and is excluded from fork-boundary comparisons. Frozen evidence is not overwritten. No mint, transfer, upgrade or setter was simulated or broadcast.

Appendix 64: StakeWise vaults, Index Coop and reward authority on PulseChain

Contents of this appendix

dsETH accepts the existing sETH2 identity.....	411
Is dsETH already controlled by the group?	412
wsETH2, Opium and Babylon.....	414
Original reward oracle and distributor	414
Evidence and reproduction	415

Source record: docs/studies/analysis/pulsechain_stakewise_vault_integrations_20261002.md.

StakeWise vaults, Index Coop and reward authority on PulseChain

2 October 2026. Read-only evidence at PulseChain block **27,690,565**, hash

0x8eafcc2cbd9f3c7fd00a499ba4173ba098bc207762d553543f998be5bac03334, timestamp **01:24:15 UTC**. Inherited-state comparisons use block **17,232,999**. No transactions, signatures or setter simulations were submitted.

The strongest additional integration is dsETH: it already holds exact sETH2 and has an installed rETH2 reinvestment extension. Group control of dsETH is not established. Opium and Babylon add options and investment-strategy routes. The original StakeWise oracle and distributor also have an administrator already linked to the controlled StakeWise Safe. These findings extend the local network; none identifies a working L2 bridge.

dsETH accepts the existing sETH2 identity

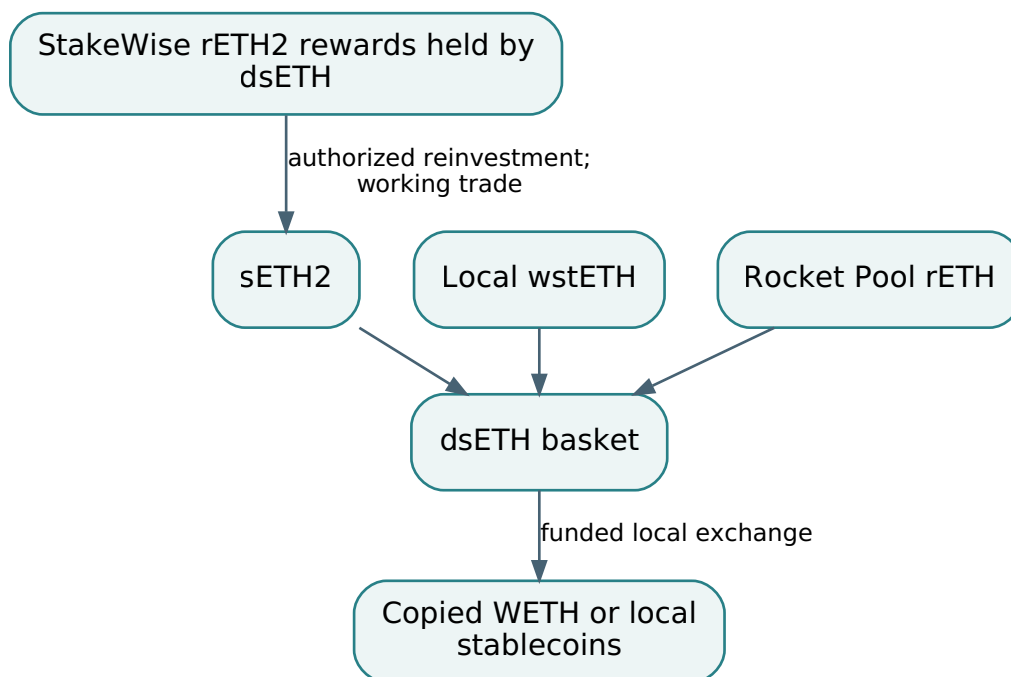
The copied Index Coop token is **dsETH**, 0x341c05c0e9b33c0e38d64de76516b2ce970bb3be. Its current components are:

Component	Exact token address	Meaning
sETH2	0xfe2e637202056d30016725477c5da089a b0a043a	Direct StakeWise principal component
wstETH	0x7f39c581f595b53c5cb19bd0b3f8da6c9 35e2ca0	Wrapper for local Lido stETH
rETH	0xae78736cd615f374d3085123a210448e7 4fc6393	Rocket Pool token; different from StakeWise rETH2

This direct composition agrees with [Index Coop's January 2023 explanation](#). Its earlier launch proposal discussed wrapping StakeWise exposure; the observed PulseChain basket directly contains sETH2, not wsETH2. Minting dsETH normally requires the proportional amounts of **all three** components. Supplying only newly minted sETH2 does not satisfy that basket.

The installed extension 0x1fcf74d4d0335095c02e1ca78ec55339c2ec927b exactly matches verified Ethereum **StakeWiseReinvestmentExtension** runtime. Its immutable getters identify this dsETH token, exact sETH2 and exact rETH2. Its configured trade is **rETH2 → sETH2**, through UniswapV3ExchangeAdapterV2, at fee tier 500, or 0.05%. It absorbs rETH2 already held by the basket and trades it into principal. `reinvest`

requires an allowed caller; `anyoneCallable` is false, and the three sampled research wallets are not allowed. The operator can manage that caller list. This is a concrete reward-reinvestment integration, not arbitrary collateral minting.



At the checkpoint, dsETH supply is **776.8495483**. Two canonical Uniswap V3 dsETH/copied-WETH pools hold approximately **83.0922** and **827.0058** WETH units on the other side. Copied WETH wraps native PLS on PulseChain. The checked dsETH/pUSDC pool has zero active liquidity and only **0.000318 pUSDC**. Those balances bound inventory; they are not executable trade quotes or Ethereum redemption rights.

The dsETH issuance-quantity getter currently reverts. Its redemption-quantity getter returns nominal component amounts, but this does not establish that redemption transfers succeed: sETH2's current implementation fails ordinary ERC20 getters. Restoring token functionality is necessary for the affected basket operations.

Is dsETH already controlled by the group?

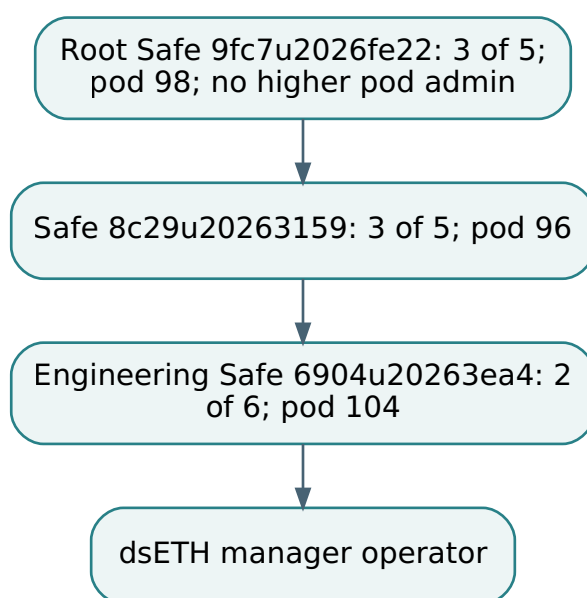
Not confirmed. Its manager is `0xbb6134ba82192e0ab23de846f1cae7aa9ae383d5`, an exact runtime match to verified **BaseManagerV2**. Its operator is the Index Coop Engineering Safe, `0x6904110f17fed2162a11b5fa66b188d801443ea4`; its methodologist is the separate contract `0x8648fefb23b5576c6329cb3514e5cdb54650b0aa`. The methodologist's authority was identified through the manager getter; its internal authorization was not resolved by verified source here.

The Engineering Safe remains **2-of-6**, with the same owners, nonce, modules, implementation address, guard and fallback handler as the fork. Its enabled modules exactly match verified Ethereum runtime:

Module	Permission relevant to this investigation
Orca ControllerV1, <code>0x4c98af741e352c6551bff9509b3f8ca9dd4e6397</code>	Connects ERC1155 membership to Safe signers; an authorized pod administrator can mint or burn memberships and alter signer membership

Module	Permission relevant to this investigation
AllowanceModule, 0xcfbfac74c26f8647cbdb8c5caf80bb5b32e43134	Authorizes bounded native/token transfers for registered delegates; does not expose arbitrary manager calls to every delegate

Thus unchanged Safe owners alone would be an incomplete check. The actual Orca administrator chain was followed:



The full root address is 0x9fc7e174e3ad480f3027814f65222726e2f4fe22; the intermediate address is 0x8c2907b46b386401c2e46903ede0c87d23503159. Their sampled owners, thresholds, enabled modules and Safe storage match the inherited state. The parent administrator getters also match. The completed selected post-fork authority-event search returns no events, including after repairing its four membership-controller query gaps. None of these observed signer lists identifies the sampled **1967**, **e121** or **a8ed** wallets. Those three wallets each hold zero dsETH at the checkpoint; that is not a complete group holdings census.

The controller's global owner is an Orca PermissionManager, 0x922a37bb4b8a155a916c15c69345731613381870. Its complete selected role history leaves administrator Safe 0x2c21843f215d17736045d39bad86b68254793193, also **3-of-5**. Its parent pod administrator is another **3-of-5** Safe, 0x945da540ef13bf67dc70ec1670d5352c0c6c8e8f, with no higher pod administrator. These controls likewise match inherited snapshots. The PermissionManager's role permits calls as the global contract owner; that is distinct from the per-pod administrator permission required to change existing pod memberships.

The operator can add extensions and manage unprotected modules through BaseManagerV2. Consequently, verified control of the Engineering Safe or its membership administrator could open a portfolio-management route without first replacing the manager. Replacing the manager or altering protected-module authority requires the source's mutual operator/methodologist approval. The protected module currently listed is 0x165edf07bb61904f47800e13f5120e64c4b9a186.

Classify dsETH as an **existing integration with unproven group control**. A demonstrated authorized execution or evidence linking sufficient live signers or pod-administrator authority to the group would change that classification. ERC20 holdings alone would not confer manager authority.

wsETH2, Opium and Babylon

wsETH2, `0x5da21d9e63f1ea13d34e48b7223bcc97e3ecd687`, is a separate single-component SetToken: one nominal sETH2 per basket unit. Its supply is only **0.5439366**. Its manager has no extensions, is uninitialized, and names the Engineering Safe as both operator and methodologist. Its issuance-quantity getter also reverts. It is an existing wrapper identity, with limited observed use, not an independent ETH reserve or the current dsETH input.

Opium: [StakeWise's June 2022 proposal](#) names three reward-routing contracts. Current code and getters confirm the seller pool `0x3ee101bf969fac08be892c737d2969b3db38d2b8`, named **Opium Weekly Turbo sETH2 Seller**, whose underlying is exact sETH2. Its LP supply is about **995.3274**. The related `0xd60f0a53c7e97f78fe4ac9013f5749920c601494` exactly matches verified Opium **Core** runtime; it should not be counted as another seller vault. The third address is `0xe47b867b2b5b21a2022068c9ef1293783864b274`.

Receipt-verified pre-fork events and current flags show all three have StakeWise reward re-routing enabled. Such re-routing moves staking rewards toward the Merkle distributor; it does not give that distributor governance of the vault. Options can produce funded premiums, but also create settlement obligations. Current sETH2 balance calls fail, so usable collateral capacity is unresolved.

Babylon Finance: integration `0x6a2340d167da153f5450ab49073f5155baa56e36` matches the [official deployment artifact](#) after filling its three immutable controller placeholders with `0xd4a5b5fcb561daf3adf86f8477555b92fba43b5f`. Its [source](#) stakes native assets through the StakeWise deposit pool. Its exit converts rETH2 to sETH2 through fee tier 500, then sETH2 to copied WETH through fee tier 3000, and unwraps WETH. The rETH2 leg uses a hard-coded minimum of 98% of nominal input, which can obstruct a depegged trade.

This establishes a deployed investment-strategy connection. It does not prove a funded active garden, control of Babylon's controller, or acceptance of arbitrary bridged ERC20s. The upgraded StakeWise deposit pool and broken sETH2 currently interrupt the original strategy route.

The checked canonical Uniswap V3 factory has one exact rETH2 pool, paired with sETH2 at `0xa9ffb27d36901f87f1d0f20773f7072e38c5bfbba`. The three checked V2 factories return **76 unique rETH2 pairs**: 2 Uniswap V2, 8 PulseX V1 and 66 PulseX V2. A direct rETH2/WPLS pair holds about **323,023.64 WPLS** against approximately **1.923 billion rETH2**. That is a local liquidity connection; it does not value billions of tokens at Ethereum ETH prices. The full derived pool inventory remains in the evidence directory.

The [Yearn discussion](#) is a strategy proposal, not sufficient evidence of a live PulseChain vault. StakeWise V3/osETH designs launched after the May 2023 fork require separate PulseChain deployment evidence before inclusion.

Original reward oracle and distributor

The original Merkle distributor is `0xa3f21010e8b9a3930996c8849df38f9ca3647c20`; its oracle contract is `0x8a887282e67ff41d36c0b7537eab035291461acd`. Their current implementation runtimes exactly match verified Ethereum source. Both name the already-controlled StakeWise Safe `0x144a98cb1cd8b23610501fe6108858d9b7d24934` as their sole enumerated default administrator.

This administrator can change the local oracle roster. The current roster contains **five** oracle members, with reward updates requiring signatures from **more than two thirds**, therefore at least **four** members. Neither a8ed nor e121 is currently an oracle member. These are local StakeWise reward/validator-report signers, not Arbitrum or Optimism rollup validators.

The oracle, rather than an arbitrary admin caller, supplies new distributor roots in the matched implementation. A valid claim transfers existing funded ERC20s, or invokes the rETH2 claim accounting against allocated distributor rewards. It does not freely mint arbitrary stablecoins. Local reward attestations also do not establish Ethereum beacon income or bridge backing.

Three successful direct rETH2 mint transactions parallel the sETH2 issuance already studied: [one million](#), [one trillion](#) and [two trillion](#). Their traces are local a8ed calls through the custom reward-token implementation, with no bridge or collateral-transfer call. Current nominal total rewards/supply is **3,000,001,005,828.3261 rETH2**, but a8ed's current balance getter returns zero. Historical issuance must not be presented as proof that a8ed still holds that amount in transferable form.

Evidence and reproduction

Frozen requests, responses, receipts, traces, source extracts and upstream commit references are under `artifacts/pulsechain-stakewise-integrations-20261002/`. Explicit manifests are under `Research/experiments/`. Getter batches use `Research/scripts/probe_pulsechain_routes.py --block <pinned-block> --calls <manifest> --output <new-file>`; event batches use `Research/scripts/collect_pulsechain_events.py --manifest <manifest> --output <new-file>`. Always use the explicit getter manifest; reproductions require fresh output names.

Runtime matching is against deployed bytecode, not token names. Babylon's constructor immutables are resolved explicitly. V2 and V3 listing retries repair the original factory-query gaps. Positive mint receipts and traces are verified, while some older rETH2 mint-query ranges still fail across retry batches; the observed three mints are not claimed to be a complete issuance census. Negative event conclusions are scoped to the exact queried events, contracts, ranges and completed RPC responses. Getter equality does not identify private-key holders or exclude off-chain coordination.

`python Research/scripts/verify_pulsechain_stakewise_integrations.py --evidence artifacts/pulsechain-stakewise-integrations-20261002` passed **168 offline evidence checks**. These cover snapshot equality, exact deployed-runtime matches, resolved Babylon immutables, repaired factory and Index membership event ranges, and mint receipt/trace/header consistency. The run saves `verification.json` and a SHA-256 inventory of **399 files**. They verify saved evidence consistency, while retaining RPC and explorer trust assumptions.

The [earlier lending study \(Appendix 39\)](#) records Fuse, Silo, Aave, Compound, Balancer and Lido routes and their separate capacity constraints. This study adds basket management, options, investment strategies and reward authority to that network.

Appendix 65: pDAI accumulation, income and the stability feedback

Contents of this appendix

The strongest defensible claim.....	416
Phases and the two directions	417
Count tokens, economic claims and cash separately	418
Less selling helps only relative to available depth.....	418
Income can renew support; moving capital does not earn it.....	419
Conditions that distinguish reinforcement from fragility	420
Verification and source boundary	420

Source record: docs/studies/analysis/pulsechain_supply_accumulation_20260929.md.

pDAI accumulation, income and the stability feedback

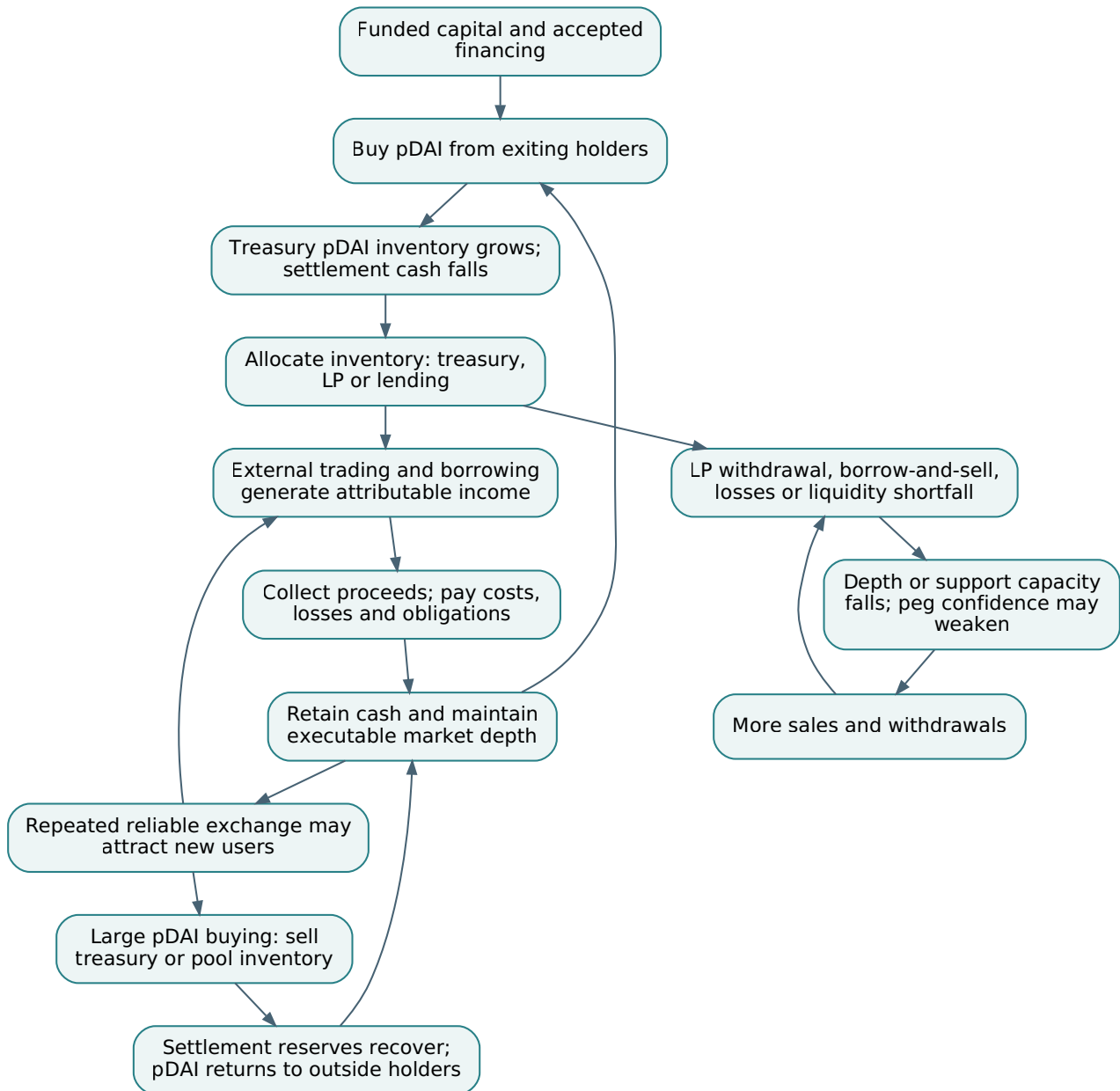
29 September 2026. Conditional economic analysis. Numerical examples are invented, independently calculated scenarios, not observed PulseChain balances, fees, ownership or future demand. The proposed coordinating treasury includes only positions whose beneficial ownership and obligations are actually shared.

The strongest defensible claim

A funded system can acquire pDAI from exiting holders, retain part as treasury inventory, provide trading liquidity with part, and supply part to lending. If outside users keep trading and borrowing, attributable income can help pay for subsequent support. If executable liquidity is maintained while prospective sales shrink, a given remaining seller cohort has less price impact. Reliable exchange could then attract payment, saving and collateral demand, strengthening the resources available for correction.

This is a coherent favorable feedback, conditional on funded buyers, retained liquidity, external activity, sound credit and renewed demand. The accumulation of pDAI is one possible result of support; it does not establish those conditions. Ownership of more pDAI alone neither earns fees nor creates the PLS needed to purchase further pDAI. The economic commitment is to maintain a usable market, which requires inventory on both sides.

Phases and the two directions



During large selling, purchases transfer pDAI toward the treasury while PLS or another funded buying asset transfers outward. Pool liquidity and timely intervention determine the achieved execution price. Acquiring a large fraction of a finite seller cohort can reduce that cohort's remaining inventory. The purchase price and cash spent still have to be financed.

During large buying, buyers pay the settlement asset and receive pDAI from pools or treasury sales. Supplying that inventory can contain the premium and rebuild settlement reserves, but it transfers pDAI outward again. Eventually inventory, permitted pDAI issuance or other compatible delivery must meet further buying if the chosen price band is to hold. Permission to mint some L2 token does not itself replenish the exact pDAI token demanded by the buyer.

The proposed transition from early selling to post-target adoption is a scenario, not an observed schedule. Demand may arrive before, at or after one dollar, or fail to arrive. Reaching one dollar once is weaker evidence of useful stability than repeated execution through adverse flows. Former sellers can return as buyers or borrowers; new owners can later sell. A lasting system therefore needs a circulation policy, rather than a premise that sellers are permanently exhausted.

Count tokens, economic claims and cash separately

For a simple disjoint classification of actual pDAI balances at one instant:

$$S = T + \sum P_i + \sum C_j + E + O.$$

s is actual token supply; T is directly held treasury pDAI; P_i is pDAI in pool i ; C_j is unborrowed pDAI cash in lending reserve j ; E is pDAI in outside wallets; and O covers all remaining custodians. Assign each balance to exactly one category. Borrowed pDAI appears where it currently resides, including E , another pool, or another lending reserve. Receipt tokens are claims on these balances and debts; they are not additional units of s .

In a simple pro-rata LP, the treasury's pDAI exposure is $\alpha_i P_i$, where α_i is its beneficial pool share. Its LP position also contains the corresponding share of the other assets. Pool administration or gauge voting is not ownership of the other LPs' inventory. Concentrated, nested and staked positions require their actual position-specific withdrawal or conversion rules; unwrap claim chains once before adding their underlying exposure.

Let L_j be a treasury lending claim measured in contractual pDAI units and D its pDAI-denominated debt. A simplified contractual exposure measure is

$$H = T + \sum \alpha_i P_i + \sum L_j - D.$$

This is net exposure, not a count of physically sequestered tokens or immediate cash. L_j can be supported by pool cash plus borrower receivables, can accrue interest, and can suffer loss. Do not add both L_j and its underlying lending cash to H . Debt in PLS or another asset remains a separate liability in that asset; it cannot be omitted merely because D is denominated in pDAI. If the system contains both lender and borrower, their internal claims and debt cancel. Pledges and withdrawal restrictions further reduce immediately usable inventory.

Define narrow external wallet float as $F_{\text{wallet}} = E$. Broader potential selling over a horizon also depends on LP withdrawals, permitted borrowing, claim redemption, treasury distributions and transactions bringing tokens back to outside holders. These are executable paths, not independent token supplies to sum without time order. In particular, $s - H$ is not a valid general measure of sellable supply.

Supplying pDAI to an AMM makes it available for buyers to acquire. Supplying it to an enabled lending reserve can allow borrowers to obtain it and sell it. [Aave's borrowing documentation](#) describes collateralized borrowing and delivery to the borrower; its [withdrawal documentation](#) conditions withdrawals on available underlying and collateral constraints. These are architectural facts, not proof that a particular PulseChain reserve currently enables those operations. The existing [Portal accounting study \(Appendix 17\)](#) supplies the dated local constraints.

Less selling helps only relative to available depth

For a fee-free constant-product pool with x pDAI, settlement reserve y , and settlement reference s dollars per unit, a sale of q pDAI receives $yq/(x+q)$ settlement units. Starting at marginal reference price $sy/x = 1$,

Average sale value = $1 / (1 + q/x)$.
 Final marginal price = $1 / (1 + q/x)^2$.

The governing ratio is sell size to available pDAI depth at the stated price, not the treasury's fraction of total supply. Both sides of the pool must remain funded. The following scenarios each start at one dollar; they are alternative states, not consecutive transactions.

Scenario	Pool pDAI x	Settlement reserve value	Sale q	Average sale value	Final marginal price
Initial selling	1,000,000	\$1,000,000	100,000	\$0.909091	\$0.826446
Selling halves; depth retained	1,000,000	\$1,000,000	50,000	\$0.952381	\$0.907029
Selling halves; depth falls 90%	100,000	\$100,000	50,000	\$0.666667	\$0.444444

The last two cases have identical reduced sell size. Retaining liquidity makes the remaining sellers less influential; losing liquidity reverses the benefit. These examples exclude fees, active support, routing and price-reference moves. They establish the comparative mechanism, not an exact peg or predictions for Curve's amplified invariant. The [local support study \(Appendix 43\)](#) separately tests fee-inclusive funded intervention. The [pool-cycle study \(Appendix 47\)](#) models connected stable and constant-product pools and demonstrates both supporting and opposing cycles.

Income can renew support; moving capital does not earn it

For a fixed illustrative fee and constant beneficial liquidity share:

Attributable external LP fees = outside traded notional × LP fee × owned share.

Use the actual LP fee after any protocol allocation and the share active during each trade. Larger treasury ownership can increase its share of a given fee pool, but cannot increase that share beyond the income actually earned. More pDAI parked in an idle wallet earns no trading fee. More liquidity at fixed external volume need not produce more total fees. Lending income likewise needs borrowers and collectible payments; outstanding interest is not necessarily cash. [Uniswap's fee documentation](#) distinguishes LP allocations, protocol fees and the treatment of accrued fees across versions. Actual destination fee settings require their own evidence.

For example, assume \$100 million of external volume, a 0.30% fee entirely allocated to LPs, and an 80% treasury share. Gross attributable fees are \$240,000. Add \$40,000 of collected external interest; subtract \$60,000 of funding cost, \$20,000 of operating cost and \$250,000 of realized inventory or credit loss. Net resources decline by \$50,000 despite substantial fee income. With the same external activity and zero realized loss, net income would instead be positive \$200,000. The separate recovery batch verifies that favorable comparison; no such income is injected into its trading simulation. The numbers are illustrative and assume all listed receipts and costs are realized; they are not a forecast or an inferred PulseChain fee schedule.

An owned arbitrageur paying fees to an owned LP transfers resources within the same system. Consolidate both positions, including inventory changes. Other LPs' shares, protocol fees and outside arbitrage gains leave that boundary. Selling newly issued reward tokens produces support cash only when a funded buyer accepts them; their face amount is not income in PLS. LP fees already included in position value cannot also count as a second cash asset.

For each settlement asset, keep the usable cash account:

```
Closing cash = opening cash + external contributions + collected income
               + funded asset sales + borrowing proceeds
               - purchases and deployments - debt service - operating payments.
```

Borrowing creates debt; deploying cash creates an investment; buying pDAI changes the asset mix. None alone demonstrates a profit or loss. The separate balance sheet values realizable assets against every external liability. Marking treasury pDAI at one dollar cannot establish the external cash available to defend that same dollar valuation. Liquidation values and settlement timing matter, particularly when support and collateral decline together. The [financing study \(Appendix 42\)](#) supplies complete claim, liability and per-step liquidity accounts.

Conditions that distinguish reinforcement from fragility

The proposed favorable feedback is strongest when ownership shifts to patient, unlevered capital; funded depth persists; external usage produces positive net income; and credible two-sided execution attracts additional voluntary demand. Income can accumulate during quieter periods and fund later deficits. It need not cover every intervention contemporaneously, provided prior reserves or committed funding are available before payment is due.

The same connections can reverse. Lower confidence can cause withdrawals; withdrawals reduce depth; price impact and collateral losses can increase; liquidations, borrowing and treasury debt service can then return pDAI to the market. A larger system share makes the operator more exposed to these losses and may make continued support more dependent on that operator's discretion. Common ownership can coordinate allocation but does not remove these balances or prove decentralized governance.

The required empirical record is therefore a time series of beneficial ownership, unencumbered settlement cash, executable depth, outside volume and income, withdrawals, debts, realized losses and two-sided quotes. A shrinking wallet balance alone cannot establish shrinking sell capacity. Rising fees alone cannot establish reserve growth. A price touching one dollar cannot establish post-target adoption. Those are distinct, falsifiable claims.

Verification and source boundary

The price-impact table follows directly from the displayed constant-product equations. Exact fractions $x/(x+q)$ and their squares were independently checked with Python's `fractions.Fraction`; the fee example uses the displayed arithmetic. Displayed prices are rounded to six decimals. These analytical comparisons create no new chain observation and do not change any frozen experiment.

The new stability batch incorporates these depth and net-income examples. Its manifest is `Research/experiments/pulsechain_stability_review_20260929.json`; the reference output is `artifacts/pulsechain-stability-review-20260929/results.json`. Reproduce into a fresh output path from the checkout:

```
python3 Research/scripts/model_pulsechain_stability.py --manifest Research/experiments/
pulsechain_stability_review_20260929.json --output artifacts/pulsechain-stability-reproduction/results.json
```

The [stability-bounds study \(Appendix 60\)](#) records the batch's full assumptions and verification. The completed batch reproduces all three depth cases and the net-income example.

The linked Aave and Uniswap primary documentation was checked on 29 September 2026. It supports only the protocol architecture attributed to it. The local studies preserve their own pinned historical evidence and reproduction records; their results do not identify common ownership of all venues or prove this proposed adoption path. The feedback is a testable conditional explanation of how coordinated capital could strengthen stability, with its failure conditions included in the same account.

Appendix 66: sUSD: existing pool connections and issuer-specific bridge minting

Contents of this appendix

What was already covered, and what this pass adds	422
Existing L1 connections	423
Two different bridge paths	423
pARB standard gateway: escrow return	423
Synthetix synth bridge: authenticated issuer minting	424
What this means for the pDAI thesis	425
The source-side financing loop	425
The Arbitrum version distinction	426
How the connected assets are reached	426
Timing, scale and the return route	427
Reproduction	428

Source record: docs/studies/analysis/pulsechain_susd_route_20261001.md.

sUSD: existing pool connections and issuer-specific bridge minting

The subsequent [integration and wallet audit \(Appendix 69\)](#) verifies five historical local DAI-wrapper mints by e121 and the wider integration catalogue. The [relay audit \(Appendix 46\)](#) verifies inherited Optimism withdrawals settled after the fork. Fresh private-L2 synth issuance and messaging remain unverified.

1 October 2026. Read-only extension of the [integrated network study \(Appendix 37\)](#). No mint, burn, relay, administrative action or state-changing simulation was executed.

The later [control and storage audit \(Appendix 67\)](#) confirms the copied seven-day finality setting, compares Safe signers/modules/implementations to inherited state, and identifies the local DAI/sUSD wrapper with reopened capacity. Its later findings govern those questions; the earlier checkpoint evidence below retains its original scope.

What was already covered, and what this pass adds

sUSD was included as a Synthetix monetary family, a candidate collateral/debt supply route and an accepted input in the sampled mStable mUSD basket. It had not received a dedicated audit of its synth bridge. This pass identifies its six inherited Curve connections and checks the copied Synthetix bridge/resolver configuration separately from the pARB escrow gateway.

The destination identity criterion is straightforward: **the token ultimately delivered to PulseChain must be the existing token accepted by the intended pool or issuer.** L2 and L1 addresses need not match. Minting on PulseChain is one way to obtain that token; releasing escrow or delivering existing inventory are others. A new bridged wrapper must first be converted into an accepted token or become explicitly supported collateral/pool inventory.

Existing L1 connections

The pool-facing PulseChain sUSD address is 0x57ab1ec28d129707052df4df418d58a2d46d5f51. The inherited census at Ethereum block **17,232,999**, the verified matching PulseChain inherited boundary of **10 May 2023**, returns the following six connections. This is pool membership, not measured current usable balances or trade capacity.

Pool	Exact pool address	Direct coins / underlying connection
Four-coin sUSD pool	0xa5407eae9ba41422680e2e00537571bcc53efbfd	DAI, USDC, USDT and sUSD directly
sUSD / 3CRV metapool	0xd05ce4ab1f4fb0c0e1b65ebe3ed7f2dcfc6ccf20	sUSD and 3CRV; underlying DAI, USDC and USDT
sUSD / FRAXBP metapool	0xe3c190c57b5959ae62efe3b6797058b76ba2f5ef	sUSD and crvFRAX; underlying FRAX and USDC
Aave sUSD pool	0xeb16ae0052ed37f479f7fe63849198df1765a733	aDAI and aSUSD; underlying DAI and sUSD
ibEUR / sUSD	0x1f71f05cf491595652378fe94b7820344a551b8e	ibEUR and sUSD; FX valuation required
sUSD / UST / MIM	0x08eaf78d40abfa6c341f05692eb48edca425ce04	sUSD, legacy UST and MIM; membership does not establish sound counter-assets

The sampled mUSD basket also accepts this existing local sUSD alongside USDC, DAI and USDT, subject to its weight limits and working issuance. That adds a candidate path from sUSD to mUSD and then mUSD/3CRV. It does not make mUSD exclusively sUSD-backed or remove other basket constraints. See the [issuer implementation and quote audit \(Appendix 38\)](#).

```
Existing PulseChain sUSD —————┐ four-coin pool — pDAI / pUSDC / pUSDT
                                   └─ sUSD/3CRV — 3pool underlying
                                   └─ sUSD/FRAXBP — FRAX / pUSDC
                                   └─ eligible Aave conversion — DAI/sUSD route
                                   └─ accepted mUSD basket — mUSD/3CRV
```

These are candidate conversions with separately available liquidity and issuance limits. The Aave route must use eligible receipts/underlying interfaces; it is not an oracle-driven automatic exchange.

Two different bridge paths

pARB standard gateway: escrow return

At the earlier integrated-network checkpoint, PulseChain block **27,681,116**, the pARB router returns standard gateway 0xa3a7b6f88361f48403514059f1f16c8e78d60eec for sUSD. That gateway calculates L2 counterpart 0xa970af1a584579b618be4d69ad6f73459d112f95. A source deployment at that calculated address was not verified. The gateway's selected local sUSD balance was **zero**.

The saved standard gateway implementation releases escrow. This branch can return the existing local sUSD token when properly funded and authenticated; its calculated mapping does not confer fresh sUSD mint rights. It should not be confused with the Synthetix-specific bridge below.

Synthetix synth bridge: authenticated issuer minting

Synthetix's historical [SIP-229](#), marked implemented, describes burning a synth on one layer and minting it through the Issuer on the other after authenticated cross-chain messaging. It initially enables sUSD and concerns Ethereum/Optimism. That is an existing issuer-specific architecture closely aligned with the proposed direct-L1-output route, rather than a generic escrow release.

Fresh destination getters are fixed at PulseChain **chain 369, block 27,683,120**, hash `0x9c1323a0bf94a1d0f38684c98891a7cad2f33fd09c45ca5aca16db76a2cd916c`. They establish:

Component or predicate	Returned value
Existing pool token proxy target()	<code>0x10a5f7d9d65bcc2734763444d4940a31b109275f</code>
Synth currency key	sUSD
Synth resolver	<code>0x4e3b31eb0e5cb73641ee1e65e7dcefe520ba3ef2</code>
Resolver Issuer	<code>0xca68a3d663483515a9d434e854ab59a41b3a523c</code>
Issuer synths (sUSD)	Same <code>0x10a5...275f</code> implementation behind the pool-facing proxy
Resolver SynthetixBridgeToOptimism	<code>0x39ea01a0298c315d149a490e34b59dbf2ec7e48f</code>
Bridge initiationActive()	True
crossChainSynthTransferEnabled(sUSD)	1
System / issuance / sUSD suspension	Each false in the sampled getter
Bridge / Issuer / synth isResolverCached()	Each true
Resolver ext:Messenger	<code>0x25ace71c97b33cc4729cf772ae268934f7ab5fa1</code>
Resolver ovm:SynthetixBridgeToBase	<code>0x136b1ec699c62b0606854056f02dc7bb80482d63</code>
Synth, Issuer and bridge owner getters	<code>0xeb3107117fead7de89cd14d463d340a2e6917769</code>

The unprefixed resolver key `SynthetixBridgeToBase` returns zero; the bridge source actually uses the prefixed `ovm:SynthetixBridgeToBase` key, whose getter succeeds. A zero result for an incorrect key must not be interpreted as absence of the configured counterpart.

The verified copied [SynthetixBridgeToOptimism source](#) contains `finalizeSynthTransfer`. It requires the caller to be its messenger and the messenger's cross-domain sender to be its configured counterpart. The function updates its received-transfer accounting and calls the Issuer's `issueSynthsWithoutDebt(currencyKey, destination, amount)`. That method name describes the destination issuance operation; it does not mean the consolidated source debt/backing disappears.

The [verified synth implementation](#) is `MultiCollateralSynth`. Its issue/burn functions restrict callers to authorized internal contracts and update token-state balances and supply. The new reads confirm that the Issuer's sUSD mapping identifies this exact implementation, not an arbitrary newly deployed wrapper.

The sampled Issuer explorer response contains no verified source body. Full Issuer authorization, debt-accounting dependencies, copied messenger finality/authentication, source-chain operation and complete end-to-end transfer execution remain unresolved. The enabled setting and unsuspended

destination are useful local predicates; they do not prove a live funded cross-chain route. Upstream architecture and verified bridge source are stronger evidence than a name or deterministic counterpart calculation, but they are not a successful transfer receipt.

What this means for the pDAI thesis

sUSD is a useful additional network junction because its existing address reaches both major Curve bases, the direct four-coin pool and mUSD's collateral basket. If an authorized source/bridge/Issuer path creates or delivers that existing sUSD, ordinary permitted trades could use those connections without changing their coin lists. Correct pool output identity is the essential integration condition.

There are three candidate ways to supply it: acquire existing local sUSD; mint through Synthetix's valid local collateral/debt mechanism; or use an authenticated issuer-specific synth transfer. The native synth transfer design uses the copied **Optimism** messenger/counterpart, while the earlier escrow mapping belongs to **pARB**. A pARB assertion or arbitrary L2 mint is not automatically accepted by the Optimism messenger or authorized Synthetix Issuer. A compatible source environment or properly authorized integration remains necessary.

Unlike OUSD's eligible-holder yield rebase or DOLA's Fed venue-management design, sUSD's ordinary synth supply changes through permitted issuance/burning and debt/transfer accounting. Adding sUSD to a Curve pool does not automatically expand or contract its balance to restore \$1. The useful supply buffer is the functioning authorized issuance/retirement route plus inventory management.

For example, supplying existing sUSD to a funded Curve exchange could acquire cheap pDAI. Conversely, accumulated pDAI could buy discounted sUSD for debt retirement or another eligible conversion. This could support both local markets when directions, fees, available inventory and obligations are accounted for. Continuously selling newly supplied sUSD can consume the pool's pStables and weaken sUSD's own quote; the broader controller still needs a return and contraction policy.

The synth's inherited/current token-owner comparison matches the same `0xeb31...7769` address. Fresh Issuer/bridge owner getters also identify that address. These are contract authority observations, not proof that the proposed group controls it or that Synthetix governance was taken over. Current upstream reforms are not automatically installed on the copied chain; this study evaluates the copied contracts and records the historical architecture separately.

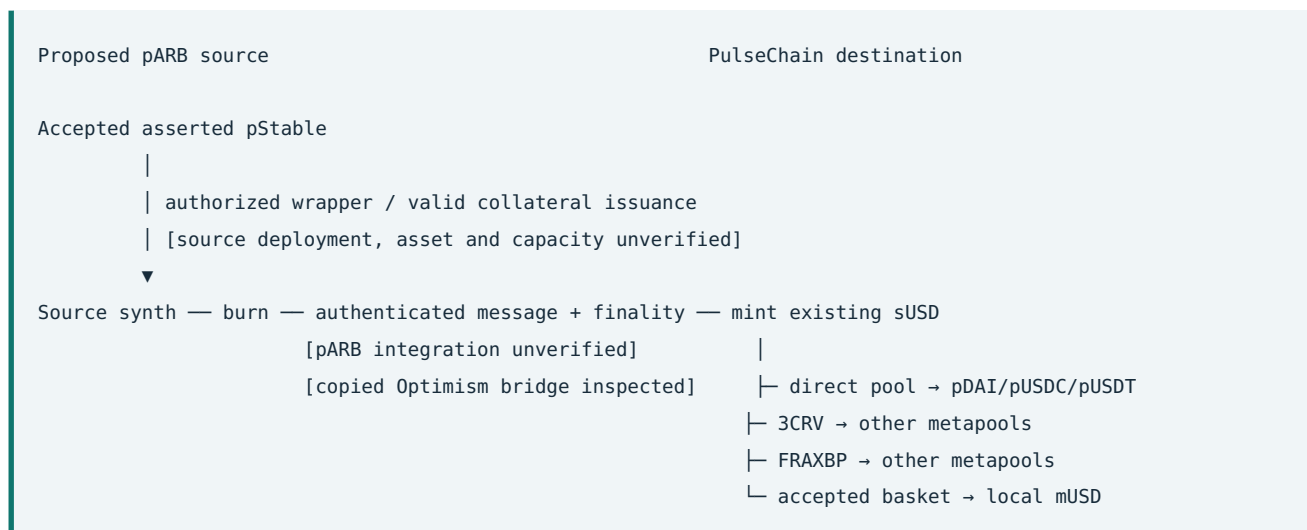
The source-side financing loop

The proposed mechanism closely matches a documented Synthetix route. [SCCP-264](#), created 16 December 2022 and marked implemented, specifies Ethereum/Optimism DAI wrappers whose output is sUSD. [SCCP-2011](#), created 26 May 2023 and marked implemented, explicitly describes starting with DAI on Optimism, minting sUSD in the wrapper, bridging it to Ethereum, and burning for DAI or selling where sUSD trades at a premium. Its parameter changes are later than the inherited 10 May boundary and must not be imported into PulseChain without checking the copied deployment.

The relevant input operation is **deposit accepted collateral into an authorized synth wrapper**, rather than generic staking in a pool. [SIP-182](#) describes a factory that supports approved ERC20/synth pairs, with per-wrapper maximum token amounts and mint/burn fees. Classic SNX staking issues debt against SNX. V3 collateral delegation and spot wrappers have different accounting and outputs. LP deposits into Curve alone acquire LP shares and do not authorize synth issuance.

The already-saved verified MultiCollateralSynth implementation has a material additional authorization branch: its `_isInternalContract` override accepts registered CollateralManager contracts, factory-recognized wrappers and the EtherWrapper, as well as its base internal contracts. This establishes that

the inherited destination synth design supports recognized collateral wrappers. It does not identify an operational pARB wrapper, accepted pARB collateral address, available cap or source oracle. No new state-changing call was used to examine it.



An assertion accepted by a rollup establishes state/message acceptance under that rollup's rules. It is not by itself permission to mint any ERC20. Even assuming spendable pStable balances exist on pARB, a wrapper must accept their exact source token address, value them under its own rules and have issuance capacity. The resulting synth must also have a valid route to the destination bridge. The copied bridge authenticates its configured Optimism messenger/counterpart. An unrelated pARB gateway or validator assertion does not satisfy that check automatically.

The Arbitrum version distinction

[SIP-367](#), created 12 March 2024 and marked implemented, describes Synthetix V3 on Arbitrum: collateral-backed issuance of an Arbitrum-native stablecoin, proposed name USDh, and separate USDC/sUSDC and DAI/sDAI spot wrappers. Its configured issuance ratios and caps are not evidence for the copied V2 sUSD system. The later [SCCP-336](#) describes changing Arbitrum's collateral set and disabling its DAI wrapper, further illustrating that accepted assets are configuration-dependent.

Therefore, “Synthetix exists on Arbitrum” is not enough to prove the desired bridge output. If a pARB installation uses V3, its output needs a verified interconversion or authorized transfer integration that ends in existing PulseChain sUSD. If it instead deploys a compatible V2 source, its wrapper and accepted messenger still need to be identified. Neither source installation has been established by this research.

How the connected assets are reached

The inherited pool catalogue establishes these local market connections:

Connection	What it permits in principle	What must be checked separately
sUSD / DAI / USDC / USDT	Exchange existing sUSD for the three local base stables	Current pool operation, reserves, fees and trade size
sUSD / 3CRV and OUSD / 3CRV	Route through 3CRV/3pool to OUSD or the base stables	Swap/deposit/withdraw interfaces, distinct pool inventory
sUSD / FRAXBP and eUSD / FRAXBP	Route through the shared FRAX/USDC base	Usable reserves; Reserve issuer collateral acceptance if minting eUSD

Connection	What it permits in principle	What must be checked separately
MIM / 3CRV, MIM / FRAXBP and sUSD / UST / MIM	Several market paths to MIM	Existing MIM supply/liquidity; legacy UST is not sound dollar backing by assumption
V2 aDAI / aSUSD	A lending-receipt path between local DAI and sUSD	Aave V2 deposit/redemption and pool-specific interfaces; not V3 Portal aDAI
mUSD basket accepts sUSD	Candidate issuance of existing local mUSD from eligible sUSD	Weight limits, fees, collateral status and complete mint success

The exact OUSD/3CRV pool is `0x87650d7bbfc3a9f10587d7778206671719d9910d`; the sampled Reserve eUSD/FRAXBP pool is `0xaeda92e6a3b1028edc139a4ae56ec881f3064d4f`. MIM/3CRV is

`0x5a6a4d54456819380173272a5e8e9b9904bdf41b` and MIM/FRAXBP is `0xb3bc1833ac51aacea92acd551fbe1ab7edc59edf`.

Catalogue entries are preserved in the financing batch alongside their source catalogue reference.

The aDAI receipt in the sUSD/Aave pool is V2 `0x028171bca77440897b824ca71d1c56cac55b68a3`; its aSUSD counterpart is `0x6c5024cd4f8a59110119c56f8933403a539555eb`. V3 aDAI `0x018008bfb33d285247a21d44e50697654f754e63` belongs to a different route. Identifying both as “aDAI” would falsely merge the two systems.

A shared base is a market connection, not direct issuer backing. Buying OUSD or eUSD through their metapools is distinct from minting them. OUSD's sampled issuer accepts DAI/USDC/USDT; Reserve eUSD requires its eligible collateral basket; MIM requires its own authorized issuance route. The system could nevertheless use sUSD to reach their existing markets without first minting each issuer's stablecoin or changing each Curve coin list. This makes sUSD a potentially simpler first distribution route than separately installing many bridged wrappers.

Timing, scale and the return route

Synthetix's [1 March 2023 announcement](#) confirms the original native Optimism-to-Ethereum sUSD transfer had a seven-day wait. It distinguishes that native transfer from a faster aggregator alternative. The copied PulseChain route's actual challenge/finality period, relaying and source operation remain unverified. A liquidity provider can front destination tokens sooner only if it has inventory and accepts the settlement risk; it does not remove native settlement.

For purchases every ten seconds, a seven-day replenishment interval contains **60,480 purchase opportunities**. If each purchase spends q sUSD, the corresponding flow consumes $60,480 \times q$ sUSD. A 1,000-sUSD purchase uses **60.48 million sUSD** across the interval, before gas, fees, delays and additional reserves. This is a working-inventory illustration, not a measured capacity estimate. A large initial bridge transfer can settle first and fund gradual purchases later; 50 billion tokens need not enter an LP together. Issuance volume, bridging volume, swap size and LP inventory are four different controls.

The controller also needs the reverse operation. It can acquire pDAI when cheap, then later use inventory to buy discounted sUSD, retire source synth obligations or redeem eligible collateral. Source wrappers can provide a mint/burn supply buffer within their capacity and custody rules. Minting more sUSD and selling it repeatedly without a return flow fills destination pools with sUSD and removes their counter-assets. This supports pDAI temporarily while potentially weakening sUSD. Multiple metapools sharing 3CRV or FRAXBP do not multiply the same underlying reserves.

For a collateral wrapper, the synth claim is linked to deposited collateral and its accepted redemption route; its accounting is not identical to a new SNX staker loan. For the native transfer, source burning and destination minting relocate the claim; they do not duplicate it or create external dollar backing. Token balances, source collateral, in-transit amounts and obligations must each be counted once. The absence of a new destination SNX debt allocation is not the absence of a system-wide claim.

The thesis can therefore state: **a verified collateral-to-synth source and authenticated existing-token bridge could turn L2 balances into usable L1 Curve inventory.** The upstream Optimism precedent and the inspected destination bridge support that architecture. The complete pARB instance remains contingent on identified source collateral acceptance, compatible synth output, accepted messaging and working finality, followed by measured destination liquidity. Evidence for these specific connections would resolve the present uncertainty; further token names or governance assumptions would not.

Reproduction

Raw getter/source records are preserved under `artifacts/pulsechain-susd-route-20261001/`. Explicit getter-only manifests are `Research/experiments/pulsechain_susd_{proxy,implementation,resolver,controls,bridge_path}_20261001.json`. Use `Research/scripts/probe_pulsechain_routes.py --calls` with a retaining RPC and the recorded block for reproduction. Source responses are stored unchanged, including the unverified Issuer response. Hashes and selected older pool/bridge inputs appear in the batch manifest. Production and research code were not changed; the previous 63-test/82-subtest result remains the last code-validation result and was not rerun for this documentation/getter extension.

The financing extension uses explicit batch `artifacts/pulsechain-susd-financing-20261001/` and manifest `Research/experiments/pulsechain_susd_financing_20261001.json`. It saves eight primary-source documents, the Synthetix SIP repository commit, hashes, connected pool rows and the earlier archive hashes. It makes no new blockchain calls. The historical timing calculation uses only 604,800 seconds divided by a ten-second interval; it is not evidence of a seven-day PulseChain delay or a funded 60.48-million-sUSD route.

Appendix 67: Synthetix control, copied Optimism messaging and purchased-stable custody

Contents of this appendix

Findings	429
Evidence checkpoints and method	430
Who can administer the copied systems?	430
What the admin functions permit	431
Destination messaging is a distinct proof system	432
The actual local wrappers.....	432
What raises or lowers capacity?	433
Purchased pDAI can replenish local synth inventory	433
Where the other purchased stables could be held	434
Reproduction and limits	435

Source record: docs/studies/analysis/pulsechain_synthetix_control_and_storage_20261001.md.

Synthetix control, copied Optimism messaging and purchased-stable custody

1 October 2026. All blockchain operations in this study are public reads: getters, runtime bytecode, transaction counts, logs, receipts and headers. No mint, burn, administrative call, signing or state-changing simulation was performed. The linked L2 is reported to have a private RPC and is not inspected. Conclusions about its deployment or operators remain conditional.

Findings

The later [Safe and rollup audit \(Appendix 52\)](#) extends these reads to block 27,685,402, verifies the Optimism BondManager's proposer-only admission and finds distinct GnosisDAO and Arbitrum control changes. The [council study \(Appendix 68\)](#) adds protocolDAO, Treasury, Ambassador and Grants custody checks and separates staker elections from multisig execution authority. Those findings do not change this study's result for the sampled Synthetix and Optimism Safes.

The selected public PulseChain controls do **not establish a Synthetix or Optimism takeover**. The two controlling Safes retain inherited owners, thresholds, modules, implementations and transaction counters. The copied Optimism registry and state-batch counters also match inherited state. A complete bounded census of selected control events returns no postfork events for the two Safes, Synthetix resolver or Optimism StateCommitmentChain. This does not identify who privately possesses existing keys.

A concrete local DAI/sUSD wrapper does exist. Its cap is unchanged; its DAI reserves have declined since the inherited snapshot, reopening capacity. The later [integration and wallet audit \(Appendix 69\)](#) verifies five successful historical e121 mints, establishing operation at their recorded amounts. That evidence advances the local recycling route beyond the getter-only checks below; it does not establish source-L2 issuance or new governance control.

The [relay transaction audit \(Appendix 46\)](#) also verifies two postfork withdrawals using inherited Optimism commitments. The empty selected state-batch census below concerns new batches, not every messenger relay or token settlement.

The wrapper can hold bought pDAI as synth collateral. That changes the financial claim: the collateral supports eligible synth redemption, including by other holders. It is not exclusively retained treasury cash or an interest-paying deposit account. Other acquired stables can be held directly, placed in eligible earning positions or used in supported issuer baskets, with separate custody and liquidity accounting.

Evidence checkpoints and method

The inherited reference is Ethereum chain 1 at block **17,232,999**, hash `0xe5667f9cafb115b1dcfc36b0b569a5601f01dfa000698140a1298ef4f87e2e37`, matching the preserved PulseChain boundary of 10 May 2023. Archive getters and code reads use <https://eth.drpc.org> with single requests.

Primary public-control reads are PulseChain chain 369 at block **27,683,800**, hash `0xb3744382d2a06dac8856f1fefe4651dd7f5a9c508f420ad97b6683636c7806a3`. Wrapper/predicate and nested-Safe reads use a separate later checkpoint, block **27,683,934**, hash `0x29fed31878085918ed1376c5c28bd7db41f4384a0966238e01401b3c5339b496`. These are explicitly different checkpoints. Older attempted wrapper reads hit pruned state and are retained as failed reads, not zeros.

The historical Synthetix package `synthetix@2.88.1`, published **3 May 2023**, supplies primary upstream source and deployment identities before the inherited boundary. Its downloaded tarball matches the registry's SHA-1 and SHA-512 integrity values. Selected files and SHA-256 records are retained. This is an upstream source/address association and inherited/current runtime comparison, not an independent Solidity compiler reproduction. Exact wrapper and messenger sources were also fetched from the PulseChain explorer; the selected Issuer, factory, resolver implementation and Safe singleton explorer responses lack verified source bodies.

Who can administer the copied systems?

Control	Current observed authority	Inherited comparison
Synthetix owner Safe	<code>0xeb3107117fead7de89cd14d463d340a2e6917769</code> , threshold 5, 13 owners	Owner list, threshold, singleton and nonce 2,884 match
Synthetix Safe modules	Empty getter and empty paginated module list ending at the sentinel	Same inherited values
Optimism AddressManager	<code>0xde1fcfb0851916ca5101820a69b13a4e276bd81f</code>	Registry entries and owner match
Optimism manager/messenger owner	<code>0x9ba6e03d8b90de867373db8cf1a58d2f7f006b3a</code> , threshold 5, 7 owners	Owners, threshold, singleton, nonce 81 , empty modules match
Contract signer inside Optimism owner Safe	<code>0x42d27eea1ad6e22af6284f609847cb3cd56b9c64</code> , 2-of-2 Safe	Owners, threshold, singleton, nonce 10 , modules match
Synthetix SNX/sUSD proxy targets	SNX <code>0xd0da9cbea9c3852c5d63a95f9abcc4f6ea0f9032</code> ; sUSD <code>0x10a5f7d9d65bcc2734763444d4940a31b109275f</code>	Both targets match

All 13 Synthetix signer addresses return empty runtime code at the later checkpoint. Six of the seven Optimism signer addresses do too; the seventh is the nested Safe above. The sampled control structure therefore has no observed replacement by a token-voting owner contract or enabled voting module. This is a claim about selected on-chain execution authority; upstream council elections and off-chain voting instructions are different mechanisms.

The selected controller/address values identify the required contracts. They do not prove common human control, stolen or transferred keys, ownership of any signer by the proposed group, or current private-L2 administration. An unchanged signer can change private custody without an on-chain ownership event. Consequently the study neither confirms that allegation nor categorically rules out private key control.

The control-event census spans **17,233,000–27,683,800**. It makes **840 contiguous, nonoverlapping queries**, each at most 50,000 blocks, across the Synthetix Safe, Optimism owner Safe, Synthetix resolver and StateCommitmentChain. Selected events cover Safe owner/threshold/module/singleton/execution changes, resolver imports/ownership changes and state-batch append/deletion. It returns **zero selected events, zero query errors and zero verification gaps**. This bounded census is not every event of every Synthetix component. Earlier broad queries and attempts using unsupported address-list filters did not complete; they are not part of this successful census.

Seventeen runtime comparisons cover SNX/sUSD proxies, Synthetix, Issuer, DebtCache, SystemSettings, SystemStatus, WrapperFactory, EtherWrapper, ExchangeRates, TokenState, FlexibleStorage, CollateralManager, both Safe singletons, messenger implementation and StateCommitmentChain. All match inherited bytes. Both subsequently discovered factory-wrapper runtimes also match: **19 matching runtime comparisons** in total. The protocol comparison has 68 identical results of 69 reads; the remaining getter is the changed debt-cache amount. Its cache timestamp remains 10 May 2023 and its stale flag is true in both compared reads. Inventory/accounting changes must not be conflated with administrative upgrades.

What the admin functions permit

Administrative change	Source-backed authority path	Observed status
Replace SNX/sUSD proxy implementation	Proxy owner can change its target	Sampled targets and owner remain inherited
Change Synthetix component addresses	AddressResolver owner imports address mappings; dependent caches must match	Selected mappings/caches match; selected resolver census has no postfork changes
Add a collateral wrapper	WrapperFactory owner creates the token/synth pair and registers it	Selected factory has two recognized children
Change wrapper capacity	SystemSettings owner sets <code>setWrapperMaxTokenAmount(address,uint256)</code>	Sampled DAI and WETH caps match inherited settings
Change wrapper conversion fees	SystemSettings owner sets wrapper mint/burn fee rates	Sampled fees match inherited settings
Transfer Synthetix contract ownership	Current owner nominates; nominee accepts	Sampled nominated owners are zero
Change Optimism registry entries	AddressManager owner changes registered implementation/dependency addresses	Selected values match inherited state

Administrative change	Source-backed authority path	Observed status
Pause or control message acceptance	Messenger's owner controls its published pause/block/allow functions	Sampled messenger is unpaused; proof/counterpart requirements remain

No administrative transaction was constructed or simulated. The table maps ordinary governance capabilities, not proof that the proposed operator can exercise them. The current Synthetix authorization requires the existing Safe's approval rule; the separate Optimism registry uses its own Safe. A move to a voting controller would need an actual authorized ownership, implementation or module change, none established by the selected evidence.

Destination messaging is a distinct proof system

The copied messenger proxy is `0x25ace71c97b33cc4729cf772ae268934f7ab5fa1`; its manager resolves implementation `0xd9166833ff12a5f900ccfbf2c8b62a90f1ca1fd5`. Its verified legacy source verifies both the accepted state commitment and message storage proof. StateCommitmentChain is `0xbe5dab4a2e9cd0f27300db4ab94bee3a233aeb19`.

The StateCommitmentChain getter sets `FRAUD_PROOF_WINDOW = 604800`, confirming the copied configuration's **seven-day** rule. Its `getTotalBatches`, `getTotalElements` and last sequencer timestamp match inherited state; the last sequencer timestamp is **10 May 2023, 22:33:35 UTC**. The selected postfork append/deletion census is empty. This confirms the configured delay and absence of selected new accepted batches through the checkpoint, rather than a live private-L2 withdrawal taking exactly seven days end to end.

The sUSD bridge additionally checks its messenger and configured source counterpart `0x136b1ec699c62b0606854056f02dc7bb80482d63`. The historical package identifies that counterpart as Optimism's SynthetixBridgeToBase. Its native source burn/destination mint architecture is established in [SIP-229](#).

Historical upstream Issuer source supplies a further authorization detail missing from the explorer response: `issueSynthsWithoutDebt` restricts callers to resolver-selected trusted bridges or FeePool, issues the selected synth and updates cached debt. The recorded destination bridge is one of the relevant resolver-selected identities. This supports the authorized issuer route; it does not execute a transfer or certify the private source's debt state.

Arbitrum's rollup/outbox assertion system and legacy Optimism's state/message-proof system serve a similar messaging purpose but have distinct contracts, authenticated senders, commitments and proof formats. Owning one system does not make its assertions pass the other system automatically. A compatible source deployment or authorized integration could connect them. The currently sampled public Optimism configuration does not establish that connection, and no fresh postfork accepted roots were observed in the selected component. A private RPC change alone does not establish a change in the public destination's accepted roots or counterpart.

The actual local wrappers

The selected WrapperFactory's contract nonce is **3** both at the inherited boundary and at the public-control checkpoint. Its published source creates Wrapper children with ordinary contract creation. The two candidate child addresses were derived from the factory's address/nonces, then confirmed by token, currency-key and factory `isWrapper` getters. Address derivation alone was not treated as proof of registration.

Local wrapper	Input / output	Cap	Reserves	Spare capacity	Mint / burn fees
0xe01698760ec750 f5f1603ce84c148b ab99cf1a74	Existing pDAI / sUSD	20,000,000 DAI	5,352,484.69872 3 DAI	14,647,515.3012 77 DAI	0.25% / 0.05%
0xcea392596f1ab7 f1d6f8f241967094 ca519e6129	Copied WETH / sETH	20,000 WETH	22,009.418071 WETH	0	0.10% / 0.50%

Both wrappers are factory-recognized, unpaused and resolver-cached at the later checkpoint. Their caps and fees match inherited values. DAI reserves were exactly 20 million at the inherited boundary; their decline reopened capacity without a cap increase. The WETH wrapper's raw balance increased above its cap; the transfer provenance was not established. Copied WETH must not be equated with externally redeemable Ethereum ETH.

An older EtherWrapper also exists at 0xc1aae9d18bbe386b102435a8632c8063d31e747c, with sampled maximum/capacity zero. The historical package names NativeEtherWrapper 0x7a3d898b717e50a96fd8b232e9d15f0a547a7eeb, an adapter to the older wrapper rather than an independent stablecoin collateral route. The two child contracts are the recognized candidates discovered from this selected factory. The study does not inventory every custom deployment or every registered CollateralManager loan contract.

Historical Optimism also had DAI and LUSD/sUSD wrapper routes; see [SCCP-264](#) and [SIP-189](#). That historical fact is separate from the uninspected private L2's current contracts. No USDC, USDT, mUSD, FRAX or MIM child was found in the selected local factory. Its owner-controlled factory design can add approved token/synth pairs; those inputs are not currently authorized merely because they are stablecoins or Curve coins.

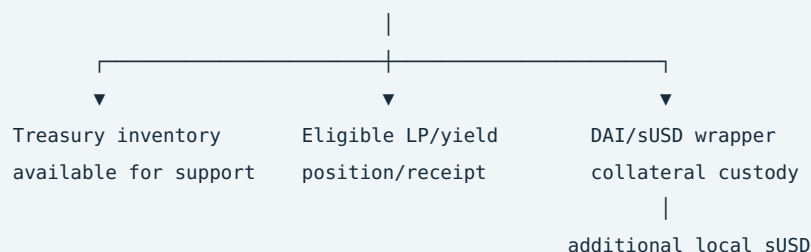
What raises or lowers capacity?

The verified wrapper source defines `capacity = max(maxTokenAmount - token.balanceOf(wrapper), 0)`. Governance sets `maxTokenAmount`; user flows change the balance. Deposits consume spare capacity. Eligible burn/redemption releases collateral and reopens capacity. Direct transfers into the wrapper also consume capacity and do not themselves issue a depositor receipt. Lowering the cap below existing balances makes spare capacity zero; it does not by itself remove collateral or rewrite outstanding synth claims. Pausing and valuation rules are separate predicates.

DAI-wrapper minting checks the sUSD currency key rather than a live market price of pDAI. The sampled `rateIsValid(sUSD)` returns false; system/issuance suspension getters are clear. Published `ExchangeRates` source treats sUSD as the unit-of-account exception. The stale general debt cache and other stale synth feeds therefore cannot be promoted into proof that this particular wrapper's mint is disabled. Conversely, the successful selected getters do not certify a complete funded mint: token allowances, transfers and all dependent calls still belong to execution. No such execution was tested.

Purchased pDAI can replenish local synth inventory

Source synth → authenticated transfer → local sUSD → buy pDAI



For a simple fee illustration, 100 pDAI deposited into the sampled DAI wrapper produces about **99.75 sUSD** for the depositor; mint fees support the protocol's fee distribution. Burning 100 sUSD through an eligible funded wrapper redemption yields about **99.95 pDAI**. These figures come from the recorded positive fee rates and source mechanics, not a submitted quote or transaction.

If sUSD is a spendable counter-asset and pDAI can be bought for less than roughly 0.9975 sUSD per pDAI after trade costs, this conversion can create an arbitrage incentive to buy pDAI and deposit it. The opposite conversion offers a ceiling near 1.0005 sUSD per pDAI when eligible redemption inventory is available. This is a bounded relationship **against sUSD**, not an independently proved external US-dollar peg. Market execution, inventory, fee changes and deposit/redemption conditions determine whether the theoretical band is accessible.

The operator need not unwrap source synths to use this branch. Purchased local pDAI itself enters the local wrapper; newly issued sUSD can return to the market. This is a potentially simpler local recycling mechanism than repeatedly withdrawing source collateral. It remains limited by the sampled **14.65-million-DAI spare capacity**, rather than providing demonstrated capacity for 50 billion tokens.

The deposited pDAI supports outstanding synth claims. Once the new sUSD is sold, other eligible synth holders may acquire and burn sUSD to withdraw available pDAI. Thus depositing all acquired pDAI does not preserve exclusive ownership or permanent removal from circulation. Directly donating pDAI to a wrapper does not confer a private withdrawal right. The wrapper has no ordinary deposit-interest mechanism: its role is collateral custody and issuance/redemption, while fees flow under Synthetix's own fee system.

Where the other purchased stables could be held

Destination	Relevant assets / connection	Financial claim and limitation
Operator-controlled treasury	Ordinary transferable pDAI/pUSDC/pUSDT/mUSD/FRAX/MIM and other coins	Direct inventory; requires actual custody, not Synthetix governance
Existing Curve LP	Exact coins supported by that pool or its underlying interfaces	LP claim and fees; pool reserves remain available to swaps/redemptions
Eligible Aave supply	Exact accepted local underlying; V2/V3 routes differ	aToken claim; reserve status, code, caps and liquidity need matching
Origin OUSD issuer / eligible strategies	Sampled local DAI/USDC/USDT acceptance	OUSD claim; vault/strategy rules, eligibility and withdrawal liquidity apply
mStable basket / savings	Sampled DAI/USDC/USDT/sUSD basket; eligible mUSD savings route	mUSD or savings claim; weights, caps and working calls remain required
Reserve eUSD backing	Eligible local collateral receipt basket from the issuer audit	RToken claim; generic FRAX/MIM ownership is not basket acceptance
Synthetix additional wrappers	Potential new approved underlying/synth pairs	Requires demonstrated factory/settings authority and a deployed recognized wrapper

These are custody/financing route families, not recommendations or certification of present yields. The [issuer study \(Appendix 38\)](#) supplies the exact sampled collateral rules. Merely placing assets in another contract does not preserve spendable treasury ownership; the receipt, encumbrance and withdrawal rule must be tracked. Freely available peg-support inventory, synth collateral and earning positions need separate balances to avoid counting the same assets twice.

Reproduction and limits

Evidence is frozen in `artifacts/pulsechain-susd-authority-custody-20261001/`. Explicit getter manifests start with `Research/experiments/pulsechain_susd_authority_`; the successful selected control-history manifest is `pulsechain_susd_authority_control_history_20261001.json`. Getter reproduction uses `Research/scripts/probe_pulsechain_routes.py --calls`; older PulseChain state requires a retaining endpoint. Event reproduction uses `Research/scripts/collect_pulsechain_events.py --manifest`, with 50,000-block partitions for the sampled public endpoint. Historical Ethereum getters use `--chain-id 1 --single-requests` and preencoded data.

Failed broad log requests, unavailable inherited receipts, Ethereum free-plan historical log refusals, a failed archive endpoint and pruned wrapper-state reads are retained or recorded as collection limitations. None is interpreted as zero activity or zero balance. The successful control census is separately validated for complete, contiguous selected ranges. Private source RPC access, private key ownership, a working pARB-to-Optimism integration and complete mint/bridge execution remain unresolved.

The event collector now supports bounded ranges, explicit chain identity and preservation of unavailable receipt records as unverified evidence. It uses the same public research User-Agent as the getter probe. A representative test checks successful receipt inclusion and the missing-receipt case without treating incomplete evidence as verified. `./scripts/test.sh --research` passed **65 tests and 82 subtests** after these changes. Earlier research archives remain frozen; the new release has its own hashes and validation record.

Appendix 68: Synthetix councils and their copied PulseChain authority

Contents of this appendix

Election rules at the fork	436
Who can implement a protocol change?	437
Copied council Safes checked on PulseChain	437
The old Spartan NFT on PulseChain	438
Reproduction and research implication	439

Source record: docs/studies/analysis/pulsechain_synthetix_councils_20261001.md.

Synthetix councils and their copied PulseChain authority

SNX stakers elect the Spartan Council. That establishes how Synthetix selected its policy representatives; it does not establish a currently operating PulseChain council or automatically replace the signers controlling its copied contracts. The published May 2023 council names, copied L1 membership NFT and protocolDAO signing authority are three different records.

Election rules at the fork

The [April 2023 election announcement](#) specifies eight Spartan, four Treasury, five Ambassador and five Grants seats. **Seventeen** is the subtotal for Spartan, Treasury and Ambassador; **22** includes Grants. The epoch began on 30 April 2023. Nominations and voting occurred on **Optimism**, with L1 voting power incorporated through a Merkle tree. All council election modules used quadratic weighting except Treasury, which used linear weighting.

The pre-fork [governance documentation](#) describes voting power derived from debt ownership and a staking snapshot before elections. Holding SNX or purchasing sUSD does not by itself establish eligible voting power. [SIP-207](#) documents the transition to on-chain Optimism elections and NFT membership.

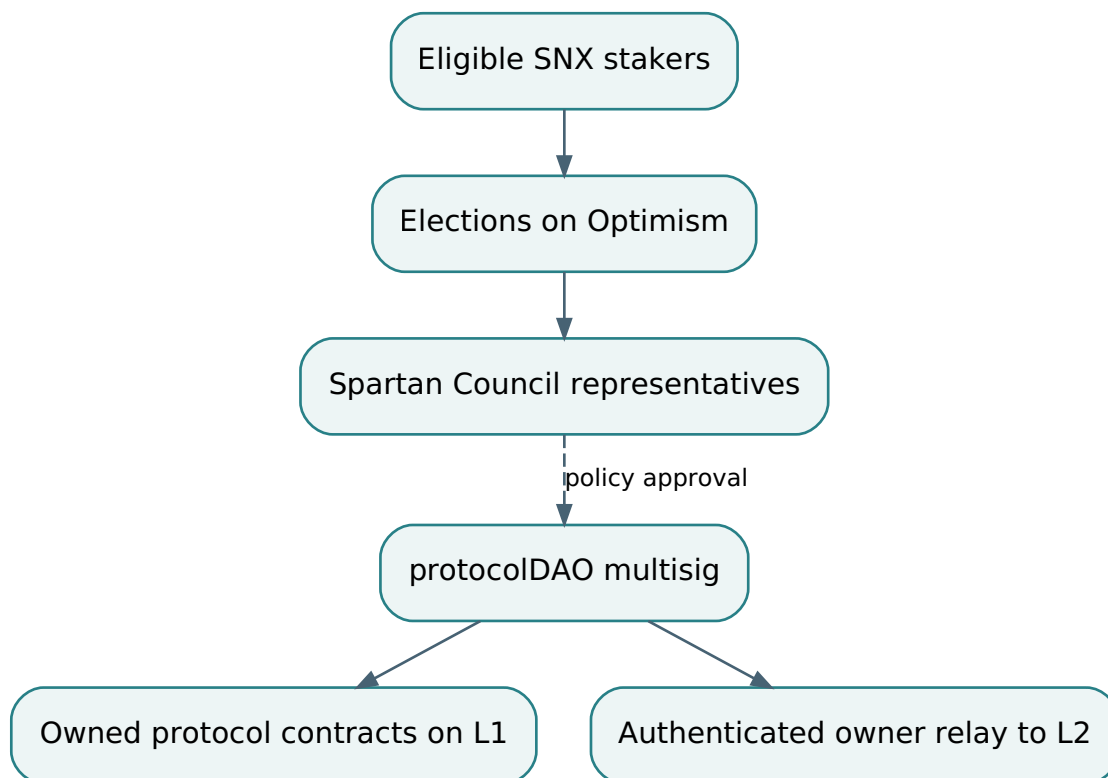
The [published May results](#) identify:

Council	Published members for the epoch
Spartan	Adam, Burt, Cavalier, cyberduck, Danijel, dsacks, Jackson, ml_sudo
Treasury	Carrot Anderson, Elliptic-crv, Kain, SpartanGlory
Ambassador	GUNBOATS, Kevin, mastermojo, Matt, Westie
Grants	ALEXANDER, CT, JVK, sunny, synthquest

These are historical Synthetix results, **not verified current PulseChain officeholders**. An Ethereum state fork does not automatically copy the separate Optimism election state. No authenticated current election/membership record from the private copied L2 was available to this study. Later Ethereum/Optimism governance reorganizations also cannot be assumed to have propagated to PulseChain.

Who can implement a protocol change?

The Spartan Council approves policy proposals. The protocolDAO implements approved changes through its contract ownership and multisig. The pre-fork [Spartan documentation](#) also describes security bonds deposited by protocolDAO signers and held under Spartan control. The exact copied bond-custody Safe was not established here; that remains a specific unresolved control dependency.



The dotted arrow represents the governance process, not proof that an election result can force a transaction from the multisig. The [Safe voting tutorial](#) lets a staked Safe cast a vote. Voting through a Safe does not remove its owners, change its threshold or give one elected representative the power of all its signers.

Copied council Safes checked on PulseChain

Reads are pinned to chain 369, block **27,685,402**, hash

0x085116030d59ed608fabf0595a92b60a76e766282251553249a4d359c92db59d. Comparisons use Ethereum block

17,232,999, hash 0xe5667f9cafb115b1dcfc36b0b569a5601f01dfa000698140a1298ef4f87e2e37.

Contract / association	Address	Copied threshold	Finding
protocolDAO / Synthetix owner	0xeb3107117fead7de89cd14d463d340a2e6917769	5 of 13	Owners, threshold, nonce and enabled modules match inherited state
Treasury Council candidate	0x99f4176ee457afedffcb1839c7ab7a030a5e4a92	2 of 4	Owners, threshold, nonce and enabled modules match inherited state
Treasury incentive Safe	0x579b66d0a7c48eee63b3bd2bca17bf0fa0f0787b	2 of 4	Same comparison result

Contract / association	Address	Copied threshold	Finding
Ambassador Council	0x46abfe1c972fca43766d6ad70e1c1df72f4bb4d1	2 of 5	Same comparison result
Grants Council candidate	0xeb9a82736cc030fc4a4cd4b53e9b2c67e153208d	3 of 5	Same comparison result

All five have empty enabled-module lists. Council Safe guard/fallback slots and proxy runtime also match the inherited comparison. The 16 distinct signers of the four additional council/incentive Safes have empty code at both pins; no nested contract signer was found among them. The protocolDAO signers were checked in the wider [control audit \(Appendix 52\)](#). Equal public configuration cannot prove unchanged private-key custody.

Associations are deliberately qualified: the Treasury and Grants candidates use [Treasury](#) and [Grants](#) explorer labels. The incentive address is specified by [SIP-294](#). The Ambassador address was published by an Ambassador representative in the [January 2023 delegate platform](#). These records locate relevant custody contracts; they do not map every signer to a named May council member.

The Treasury candidate's `masterCopy()` getter reverted at both pins. This is not evidence that its implementation is absent; slot-zero storage was separately read. Unsupported getters are retained as errors, never decoded as zero-valued answers.

The old Spartan NFT on PulseChain

The [March 2021 Synthetix announcement](#) identifies the L1 Spartan Council NFT at `0x023c66b7e13d30a3c46aa433fd2829763d5817c5`. On PulseChain it still returns eight tokens and the administrative owner `0xde910777c787903f78c89e7a0bf7f4c435cbb1fe`, matching the inherited comparison.

The eight copied membership wallets are:

Token	Membership wallet
1	0xcc2e5565abfc3b457e284d84e87a56583833afef
2	0x37ffbbee75b446b2854efff46736642be7a70a95
3	0xdf09b6bb09fdee5f8d4c17c6642f0a54d6a7654a
4	0x42f9134e9d3bf7eee1f8a5ac2a4328b059e7468c
5	0x656b7f17933ee35058d1beb8b6c65b580e799440
6	0x8be60fe9f7c8d940d8da9d5ddd0d8e0c15a4288b
7	0xa41228de09fd143727d6337585c6b02c698146bc
8	0xbc3668d2aafa53ed5e5134ba13ec74ea195d000

Membership addresses and metadata match the inherited comparison. All metadata fields return the same IPFS content identifier, so they do not identify eight individual people. This is the older L1 membership register, which predates the Optimism election system. Its eight wallets must not be presented as the May 2023 winners or a verified current PulseChain council.

Reproduction and research implication

Evidence is in artifacts/pulsechain-safe-governance-rollup-20261001/: council-safes-current.json, council-safes-inherited.json, legacy-council-tokens.json, council-members-current.json and council-members-inherited.json. Explicit manifests under Research/experiments/pulsechain_synthetic_council*20261001.json contain only getters, code reads and storage reads. Historical sources and commit references are retained in the same evidence batch. No transaction was signed, broadcast or simulated as an administrative action.

The peg thesis can include control of SNX voting power as a potential route to elected representation. It cannot treat that as verified control of the copied protocolDAO, fresh destination messages or unlimited synth issuance. Those require separate evidence of effective contract authority, source issuance, accepted destination messaging and applicable collateral/cap conditions. Current PulseChain Spartan officeholders and the copied Spartan bond-custody Safe remain unverified.

Appendix 69: Synthetix integrations and observed research-wallet activity

Contents of this appendix

What was examined	440
Historical integrations relevant to the thesis	440
Other links, proposals and data services	442
All eleven historical local direct-integration addresses	443
What the five wallets actually did	444
The wrapper route is demonstrated, not just inferred from getters	445
UMA, Across and the simpler integrated thesis	445
Reproduction	446

Source record: docs/studies/analysis/pulsechain_synthetix_integrations_20261001.md.

Synthetix integrations and observed research-wallet activity

1 October 2026. Blockchain work remains read-only. This inventory covers the official pre-fork ecosystem catalogue, selected historical integration reports and proposals, the local DirectIntegrationManager event history, and SNX/sUSD transfers involving five examined wallets. It distinguishes historical integration, current local configuration, observed use and administrative control.

The strongest new finding is **actual use of the local DAI/sUSD wrapper**. Five successful transactions from e121 deposited **136,554.715815649876218232 pDAI** in total and issued

136,213.329026110751527686 sUSD to that wallet after its 25-basis-point mint fee. This proves a working local collateral-to-synth route at those historical executions. It does not prove fresh private-L2 issuance or control of Synthetix governance.

What was examined

The official Gitbook repository is pinned to commit 2de4381a605d7a7b9f03d08164a0f29b7ea629b7, dated **8 May 2023**. All 19 selected integration/ecosystem Markdown files are saved. The SIP repository is pinned to 3b6b70f137cd6b0a23c39daceaa76f80cdbc520a, dated **9 May 2023**. A complete archive search found 31 proposals with titles matching the recorded integration/bridge/wrapper/ecosystem terms; their pre-fork statuses are retained. A second full-archive search associates all eleven locally discovered integration addresses with their published SIP/SCCP records.

This is a complete inventory of those defined source sets and the selected manager's queried event history. Permissionless token use has no universal registry: it would be misleading to call this proof of every application ever interacting with SNX or a synth. Private-L2 installations and every intervening wallet are outside the accessible scope.

Historical integrations relevant to the thesis

The functions below are documented historical connections. Their presence in a source report does not certify a presently working PulseChain or private-L2 route.

Protocol or service	Connection to Synthetix	Potential contribution
Curve	sUSD/stable pools and cross-asset synth routing	Existing local pool coins and swaps reaching the major stablecoin bases
1inch, including Fusion	Aggregated cross-asset atomic trades; special integration parameters	Routes from ordinary tokens through synths and back to ordinary tokens
CoW/Otex	Direct atomic integration in SIP-296	Another authorized exchange route with configured terms
0x	Synth/atomic trading reported in Q3 2022	Aggregated trade access; not a separate mint authority
Kwenta	Spot and perpetual trading frontend	Traders and fee generation for the Synthetix liquidity system
Decentrex	Perpetual trading frontend	Another interface to the same underlying market system
Lyra	sUSD settlement/collateral and synth hedging	Options liquidity and fees, with market risk
Polynomial	Lyra options vaults, Synthetix perps and basis strategies	Managed earning/hedging positions, subject to strategy risk
Unidex	Reported Perps integration by Q1 2023	Trading access; source report alone does not establish copied deployment
dHEDGE and Toros	Managed synth portfolios and debt-mirror strategies	Transferable investment claims and debt exposure management
Aave	SNX/sUSD lending and borrowing; local V2 aSUSD/aDAI connection	Lending receipts, eligible credit and collateral recycling
mStable	Sampled local mUSD basket accepts existing sUSD	A candidate issuance path from accepted sUSD to existing mUSD
Beefy	Reported SNX and sUSD vaults	Yield strategy receipts; not synth issuance permission
Granary	Reported SNX and sUSD lending integrations	Lending/borrowing positions
Sonne	Reported SNX integration	SNX lending rather than automatic sUSD minting
Silo	Reported SNX and sUSD integrations	Isolated lending positions; exact market/debt token matters
Velodrome	SNX/sUSD, USDC/sUSD and WETH/sETH markets and incentives	L2 liquidity distribution and ordinary market conversions
Uniswap V3	Token liquidity and atomic-pricing reference	Prices and trading; oracle dependencies must remain valid
Gelato / G-UNI / Sorbet	Automated claims and managed WETH/SNX liquidity	Operational automation and LP receipts, within delegated permissions
Yearn / historical iearn	Historical Curve/sUSD liquidity relationship; local ySUSD lending wrapper	Yield/lending shares rather than new freely backed stable supply

Protocol or service	Connection to Synthetix	Potential contribution
Convex	Curve LP incentives and liquidity governance relationships	Incentivized liquidity; governance voting is distinct from issuer ownership
Native Optimism/Synthetix bridges	SNX escrow transfer and authorized synth burn/mint transfer	A specific cross-chain asset path with authenticated messages
Hop	Reported SNX/sUSD fast-bridge support	Destination liquidity in advance of native settlement
Socket	Bridging aggregation integrated into the Synthetix UI	Route selection; Across appears in Socket's official bridge enumeration
Thales / Overtime / Exotic Markets	Synthetix ecosystem origin and sUSD-denominated positional markets	Additional uses and demand for sUSD; not a general synth minting grant
Aelin	Community-origin capital-raising protocol and SNX-holder governance/distribution links	Capital coordination; distinct from synth collateral issuance
Chainlink	Synth price feeds and atomic-pricing dependencies	Required valuation infrastructure; no funded market bid is created by an oracle

Primary evidence for the catalogue and exchange integrations is the [pinned official ecosystem](#), [SIP-288](#), [SIP-296](#), and the [Q3 2022 report](#). The [Q4 2022 report](#) identifies Beefy, Granary, Sonne, Hop and Silo; the [Q1 2023 report](#) identifies Unidex and distinguishes Mux/Nested work still in progress.

The [2022 ecosystem guide](#) records Thales, Aelin and Velodrome connections. The [2022 milestone review](#) records debt hedging and Gelato claims. [Aave's 2020 integration announcement](#) and the [Curve/learn liquidity trial](#) establish earlier links. The local mStable and pool connections are verified separately in the [issuer study](#) (Appendix 38) and [sUSD route study](#) (Appendix 66).

Other links, proposals and data services

The historical [volume program](#) names SNX.Link, Pillar Wallet, ParaSwap, dHEDGE, Kwenta and Solidefi. This is frontend/referral history, not certification of present liquidity or administrative privileges.

The Q2/Q4 reports also record governance participation or outreach involving Euler, Gearbox, Across, Alchemix, SushiSwap, Dodo, MetaMask, Matcha, Zapper, Zerion, Bankless Academy and EthernautDAO. Outreach is not an implemented token route. Mux and Nested are explicitly in progress in the selected Q1 2023 source. Fixed Forex's special direct-integration proposal **SIP-292 is rejected** at the pinned boundary, as are the proposed special Curve integration in SIP-268 and peg-balancing integration in SIP-2007. Existing ordinary markets must not be confused with these unimplemented proposed privileges.

Data/tooling connections in the pinned Gitbook include The Graph subgraphs, DefiLlama and CryptoFees/CryptoStats adapters, and Kwenta's Python SDK. Tracking codes attribute trading activity and revenue; they do not appoint their users as owners.

All eleven historical local direct-integration addresses

The local manager is `0x5f41718459955217df6a02aa8f5c19614ecfa6e7`. At block **15,000,000**, the recorded code getter is empty, preceding its deployment. The selected history through block **27,684,371** covers **254 contiguous 50,000-block-or-smaller queries**. It returns **94 IntegrationParametersSet**, **two OwnerChanged** and **one OwnerNominated** events, all receipt verified, with no query errors or verification gaps. **None is postfork.**

Current getters at block **27,684,469**, hash

`0xda9cafaaa01d89769eaff468f4a3c5e9b61c4cc5009ad369f6d2e4766352c3c5`, compare every address's sUSD/sETH/sBTC settings to the zero-address default:

Address	Published integration association	Current returned terms
<code>0xef6fa3307af6ab6ddcc4826c4945041dd5c18d4b</code>	1inch, SIP-288 / SCCP-267	Defaults for all three keys
<code>0x3b17056cc4439c61cea41fe1c9f517af75a978f7</code>	1inch, SIP-288 / SCCP-267	Defaults
<code>0xf021f084477242fe6835c67234b4345de4db19e1</code>	1inch, SIP-288 / SCCP-267	Defaults
<code>0x53222470cdcfb8081c0e3a50fd106f0d69e63f20</code>	1inch, SIP-288 / SCCP-267	Defaults
<code>0xcc0333574bac61e0d891053083465ef68f544d2e</code>	1inch, SCCP-267	Defaults
<code>0x15a3c296eae49734bc467f95815f6dea9a3de233</code>	1inch, SCCP-267	Defaults
<code>0xa77c88abcaa770c54a6cfbfd0c586a475537bbc1</code>	1inch Fusion, SCCP-274; reverted by SCCP-276	Defaults
<code>0x004f4b64c0a206f2d979229e28d322d1c50864a3</code>	Otex/CoW, SIP-296	Overrides; 10-million atomic volume cap
<code>0xdecf04ca46d287b3cb99692ad512e568ba276140</code>	1inch Fusion, SCCP-282	Overrides; 10-million atomic volume cap
<code>0x609262f06dc1967ee0c475991b2030cca4233741</code>	1inch Fusion, SCCP-292	Overrides; 15-million atomic volume cap
<code>0x62c9c2ace3e86eee3016c7e73df70edd8a45ace9</code>	1inch Fusion, SCCP-299	Overrides; 15-million atomic volume cap

These are **exchange volume settings**, not wrapper collateral caps or permission to mint arbitrary tokens. Four addresses retain differences from default; historical configuration does not establish a continuing benefit for the other seven. The `1967` substring in `0x6092...3741` is not the research wallet ending in 1967.

[SIP-267](#) and the integrity-checked pre-fork npm source describe owner-controlled parameter overrides. The current owner getter remains the Synthetix Safe. The manager and eleven integration explorer responses lack verified source bodies, so address attribution rests on published proposals, saved events and current getter results, not independent compilation. All eleven have nonempty sampled runtime.

Five research wallets—1967, feca, BAD9, e121 and a301—receive the default exchange parameters for all three tested keys. This does not prevent them from trading through an integration contract. Sampled sUSD rate validity is clear, but sETH and sBTC are **rate-invalid**. Consequently historical atomic ETH/BTC routes cannot be promoted into verified present operation.

Optimism has distinct direct integrations in implemented [SIP-2006](#) for Lyra (`0xbfa31380ed380ceb325153ea08f296a45a489108`) and [SIP-2008](#) for Polynomial (`0x3349de7822aa05f857e92d167b264809419dc620`). Those historical L2 addresses are not entries in this local L1 census; their current private-L2 state is unverified.

What the five wallets actually did

Filtered SNX and sUSD Transfer queries cover **15,000,000–27,684,469**, in 1,016 contiguous bounded requests. Incoming and outgoing filters cover all five wallets. The result has **105 distinct receipt-verified events**, no query errors and no verification gaps.

Wallet	sUSD events / transactions	Observed UTC span	SNX Transfer events in this selected range
e121: <code>0x98c89980d1eae247ada3636ae27648f78e66e121</code>	94 / 90	6 April 2025–8 September 2026	0
1967: <code>0xfdb32bcf68bb0ced033200373384a619c71967</code>	11 / 10	19 June–17 July 2026	0
feca: <code>0xfeca7291cebe75789ffa707754db259187b1d140</code>	0 / 0	None returned	0
BAD9: <code>0xbad9dff1026e59ea4c39cc3398aa5eb6f6f229ac</code>	0 / 0	None returned	0
a301: <code>0xa30190b96faee0080144aa0b7645081fcbf49e6f</code>	0 / 0	None returned	0

All 105 events occur in transactions initiated by the corresponding e121 or 1967 wallet. This establishes active sUSD use over approximately seventeen months for e121. Zero direct SNX transfers in this selected set does not exclude escrow holdings, other group wallets, intermediate contracts or older activity. Common human ownership of the five accounts is not established.

Observed target/source associations include:

- Existing Curve four-coin sUSD pool `0xa5407eae9ba41422680e2e00537571bcc53efbfd`, used by e121 and 1967.
- Local DAI/sUSD Wrapper `0xe01698760ec750f5f1603ce84c148bab99cf1a74`, used for five e121 mints on **4–7 September 2026**.
- Synthetix SynthRedeemer `0xe533139af961c9747356d947838c98451015e234`, including e121's first returned activity.
- PulseXSwapRouter `0xda9aba4eacf54e0273f56dffee6b8f1e20b23bba`, used by both wallets.
- A verified Balancer BPool `0x3029c3763d4b61b255b5eea5a506cb2f080487a7`, Mooniswap `0x2d5786b5cf47627f5c58e2026b80d7d56a296cea`, and PiteasRouter `0x6bf228eb7f8ad948d37ded07e595efddfaaf88a6`, appearing in e121 activity.

- Historical learn ySUSD at 0xf61718057901f84c4eec4339ef8f0d86d2b45600 and bZx LoanToken at 0x49f4592e641820e928f9919ef4abd92a719b4b49, appearing in 1967 activity.

The ySUSD source names existing local sUSD as its underlying and contains lending-allocation interfaces. This demonstrates a local receipt/custody option; it does not certify current yield or every configured downstream lender. The LoanToken transaction calls `mint(address,uint256)` but transfers **999 existing sUSD from 1967 into the lending contract**. It mints a lending receipt, not fresh sUSD. Symbolic function names alone would give the wrong interpretation.

Twenty-four distinct transaction targets are retained in `activity-selected-source-and-trace-index.json`, including unresolved source responses. Unverified targets are not assigned speculative protocol names. A token transfer involving a router or vault demonstrates that transaction's asset movement; it is not proof that the wallet governs the router or vault.

The wrapper route is demonstrated, not just inferred from getters

Transaction [181c...0fca](#) on **4 September 2026** deposits **30,000 pDAI** and issues **29,925 sUSD** to e121. Its successful receipt and 32-call trace agree with the exact existing wrapper and synth identities. The other four verified deposits are approximately 12,440.445, 41,656.709, 3,973.582 and 48,483.980 pDAI. Full precision stays in the evidence files.

Owned pDAI

- existing local DAI wrapper
- existing local sUSD, after mint fee
- funded Curve / other market
- pDAI, pUSDC, pUSD or another accepted market asset

The first two arrows are proved by the historical mints. Curve trading and other sUSD use are also observed in separate transactions, but this study does not claim to have reconciled every transaction into one profitable uninterrupted loop. Actual profit, current remaining capacity and outside-dollar convertibility require their own measurements.

Deposited pDAI becomes redeemable synth collateral. If the operator sells the synth, it cannot continue counting that collateral as free, exclusively controlled peg inventory. The prior [custody audit \(Appendix 67\)](#) supplies the recorded 20-million-pDAI cap and 14.648-million spare-capacity checkpoint. A successful 30,000-pDAI mint establishes operation at that historical size; it does not establish 50-billion-token capacity.

UMA, Across and the simpler integrated thesis

There are several different links:

1. **Across ↔ Optimism asset settlement:** the examined October 2024 receipt releases existing local DAI to Across HubPool using an inherited Optimism proof. This is an actual canonical settlement connection.
2. **Across ↔ UMA governance:** the June 2026 proposal and execution share an assertion ID and transfer HubPool ownership through its Safe module. The recorded rules exclude HubPool calls; on-chain settlement is not evidence of a valid Snapshot vote. This authority is distinct from Synthetix administration.
3. **Synthetix UI ↔ Socket ↔ possible Across routing:** [Synthetix's bridge announcement](#) and [Socket's bridge enumeration](#) support an aggregator relationship. They do not prove a presently funded copied route or an Across permission to issue synths. Synthetix's Q4 2022 report also records Across governance-related contact.

4. **Synthetix ecosystem ↔ UMA discussion:** the [Q2 2022 report](#) records a joint Thales/UMA event. That supports historical ecosystem contact; it does not establish that UMA authenticates Synthetix's native synth bridge.

The **UMA perpetual market listed in September 2023** and **Chainlink CCIP teleporters launched in July 2023** are later developments. They are not inherited May 2023 capabilities. A tradable UMA market is also not an UMA-governed Synthetix issuer. Modern public-network upgrades must be demonstrated on the copied/private deployment before entering the operative thesis.

The [relay and governance study \(Appendix 46\)](#) retains all eight examined transactions, the confirmed ownership execution, and proof dates. Synthetix's own owner relay provides a legitimate separate way for an authorized L1 Safe to administer appropriately owned L2 contracts. The selected public records still do not prove that the proposed wallet group controls that Safe or the global Optimism messenger.

For peg support, **sUSD can be one distribution hub:** exact local sUSD → existing Curve bases → the local stablecoin markets, with Aave and mStable as additional eligible conversions. Purchased pDAI can return to the demonstrated local wrapper. That reduces the number of issuer changes needed for a first route. OUSD/eUSD/MIM market access through shared bases is separate from permission to mint those issuers' coins. Source-side financing on the inaccessible pARB deployment remains an additional, unverified branch.

Reproduction

Evidence is under `artifacts/pulsechain-synthetix-integrations-20261001/`. Explicit manifests are `Research/experiments/pulsechain_synthetix_integration_events_20261001.json`, `pulsechain_synthetix_integrations_getters_20261001.json`, `pulsechain_synthetix_registered_integrations_20261001.json`, `pulsechain_synthetix_integration_runtime_20261001.json` and `pulsechain_synthetix_wallet_activity_20261001.json`.

```
python Research/scripts/analyze_pulsechain_synthetix_integrations.py artifacts/pulsechain-synthetix-integrations-20261001 --output /tmp/synthetix-integration-analysis.json
```

Use a fresh output path. The command is offline and verifies selected query coverage, receipt-verified event status, all eleven returned integration settings, wallet event counts, and the five saved mint traces/deposits/outputs. Getter reproduction must specify `--calls`; event reproduction uses the explicit manifest. No state-changing simulation, account impersonation, signature or broadcast is part of the method.

Appendix 70: The integrated pDAI accumulation-and-peg thesis

Contents of this appendix

The mechanism they identified	447
What the additional points changed	448
Review results and dissent	449
Reproduction and preservation	450

Source record: docs/studies/analysis/pulsechain_thesis_consensus_20261003.md.

The integrated pDAI accumulation-and-peg thesis

3 October 2026. Five model analysts, three review rounds, frozen evidence. All blockchain work remains read-only.

The strongest integrated thesis is a coordinated reserve manager built around pDAI: use selected issuer and collateral permissions to create existing pool-facing tokens, convert them through funded markets, accumulate pDAI and neighboring stables, retain both payment and sale inventory, and replenish the resources consumed by support. A private L2 could provide collateral and coordinate execution. PLS proceeds could be redeployed through less attributable wallets before buying pDAI.

All five analysts support this **conditional architecture**, including assessment of the L2 branch before a fresh transfer occurs. They do not conclude that the saved record proves one operator is already maintaining a dollar peg. Their agreement is a convergence of reasoning on shared evidence, not a calibrated probability of hidden intent or a guarantee of perpetual stability.

The [maintained white paper \(main paper\)](#) incorporates the result. This study records the method, judgments, dissent and reproduction references; it does not duplicate the full contract appendix.

The mechanism they identified

The comparison rule is **one normalized eligible pStable = one accounting unit, U**. A cheap eligible token can enter an accepting issuer and produce a scarce pool coin. The reverse route buys a cheap issuer coin and redeems its funded underlying. These conversions move resources across markets and change issuer supply. Receipt tokens and LP shares require exchange-rate normalization.

```
Proposed L2 vault receipt
    ↓
Authenticated destination representation
    ↓
Accepted collateral plugin / active issuer basket
    ↓
Existing local issuer coin ← permitted local issuance
    ↓
Funded issuer / pool pays pDAI, stables or PLS
    ↓
Spendable inventory + owned liquid capital
```

↓
Optional private PLS redeployment → gradual purchases
↓
Retained payment assets + retained pDAI
↓
Buy below target / sell above target → restore capacity

Authorized L2 instructions can direct the local policy separately.

A specific prospective branch uses a liquid L2 receipt, an authenticated destination representation, a compatible Reserve collateral plugin and activated basket, and **existing Reserve eUSD**. That eUSD enters its **FRAXBP** pool; funded pUSDC output can reach pDAI or PLS through further permitted markets. The new representation stays behind the issuer, so this branch need not replace Curve's coin list. Origin OUSD and mUSD use 3CRV and have different collateral rules. Source and destination addresses need not match; the final accepted local identity matters.

This recipe combines demonstrated local components and specific authority paths with still-unverified source interfaces. The smallest missing connection is an exact authenticated output that a functioning local issuer accepts under effective governance. Source receipt rights, gateway mapping/minter, collateral plugin, basket activation, pricing and counterasset funding remain separate requirements. Copied Arbitrum administration does not confer Optimism/Synthetix permissions. The [issuer audit \(Appendix 38\)](#) and [control audit \(Appendix 52\)](#) identify those distinctions.

The L2 can also coordinate local trades without importing new principal through authorized keeper keys or a destination executor. Attestations authorize acceptance of state or delivery; they are distinct from financial issuance. Neither alone supplies the resources that funded markets pay. Zero Portal caps are changeable gates under sufficient authority, alongside other checks. The broken current sETH2 implementation and disabled ordinary Maker PSM remain specific route constraints.

Accumulation reduces competing sell inventory and builds offer inventory. Dollar support additionally requires funded bids and asks at payment quantities tied to an independent, executable dollar valuation. PLS, ETH or BTC may provide variable-price payment assets without universal fixed Ethereum redemption. Common U accounting does not itself fix that dollar scale.

Every support trade need not profit. The consolidated portfolio must fund its policy and obligations. Pledged collateral, receipts and free trading inventory cannot be counted repeatedly. Promised in-band execution needs funding or intervention before or within the supported trade; correcting a quote afterward cannot repair completed proceeds. Define trade size, costs, response time, cumulative flow horizon and adverse reference/liquidity bounds. See [managed execution and exits \(Appendix 45\)](#).

What the additional points changed

PLS privacy: all analysts received the premise that native PLS is the practical privacy staging asset on PulseChain. A visible PLS exit establishes immediate conversion, not final extraction. It can be followed by obscured accumulation; it can also be followed by disposal. The review no longer treats a missing direct proceeds-to-purchase trail as decisive against the thesis. Active deployment, anonymity depth and relevant use remain unverified. The original [Tornado ERC20 implementation](#) shows that native-only mixing is not a general technical requirement; this generic source does not establish a working PulseChain token mixer.

Prospective L2: the analysts assessed the full source-to-issuer-to-market route, rather than substituting a local-only minimum or demanding a completed transfer before discussing plausibility. Identifiable upgrade, collateral and existing-token interfaces make it a serious construction to investigate. That judgment does not promote the specified interfaces into installed deployments.

Three-year ratio: the user reports increasing **PLS per pDAI**. If reconstructed, that supports relative strength and is compatible with persistent demand or sellable-supply contraction, including accumulation in other wallets. The frozen record contains no independently verified three-year series. PLS weakness, liquidity changes and independent buyers remain alternatives. Dollars per pDAI equal PLS per pDAI multiplied by dollars per PLS. The ratio alone identifies neither source funding nor dollar appreciation. A short-run adverse trade in one pool does not refute the reported long-run trend.

Obscured ownership still permits measurement of exact-token purchases, issuer custody, pool reserves, executable bid/offer depth and replenishment. Those effects need not identify a single human controller.

Review results and dissent

The five roles were mechanism, economics, routes, attribution and clarity. Round 1 used fresh contexts and the frozen paper/evidence without peer reports. Round 2 challenged a shared synthesis with all three user steering points and the five initial assessments. Round 3 challenged the revised integrated thesis using the second-round assessments. Later rounds were informed rereading by the same reviews, not independent new samples or actual human comprehension tests.

The final judgments were:

Fixed assessment	Final judgments
A — Coherent conditional financing/arbitrage architecture	5 supported
B — Two saved positive gas-adjusted closed local trades	5 supported
C — Saved evidence proves one current coordinated dollar-peg campaign	5 unsupported
D — Equal local U mechanically establishes a dollar anchor	5 unsupported
E — Funded permissions and an external reference make a bounded band testable	5 supported
F — Saved current state proves fresh private-L2 or Portal funding	5 unsupported
G — L2 financing/orchestration can compose under the specified interfaces and obligations	5 supported
H — A PLS exit alone does not establish final extraction or exclude accumulation	5 supported
I — A verified rising ratio fits accumulation without identifying buyer/funding/dollar gain	5 supported

“Unsupported” means not established by the saved evidence, rather than impossible. A/E/G support qualified designs; H/I support qualified reasoning, not a mixer deployment or verified price series. Initial A/E label differences were resolved by reading their explicitly conditional wording; no new operating evidence caused that change. Raw round-1 counts remain preserved.

Two reviews accepted the final draft as written. Three accepted its substantive architecture while requesting narrow diagram, verification-language or execution-promise corrections. Those corrections were incorporated: separate receipt, representation, issuer acceptance and existing output; identify the payer and separate orchestration; mark unverified interfaces; distinguish supported execution from later quote restoration. No unresolved contradiction was identified in the conditional architecture. This does not justify calling deployment or every possible market scenario flawless.

The remaining substantive dissent concerns interpretation. The same apparatus can support a peg, monetize inherited resources, or run ordinary arbitrage. The saved mUSD and sDAI operations gain **200,053.199510 PLS** and **110.687385 PLS** respectively after gas, but their closing assets do not establish subsequent retained support. The sDAI operation sells all its minted pDAI. These are compatible with later purchases elsewhere and do not prove them. Exact reconciliation is in the [closed-trade study \(Appendix 25\)](#).

The most valuable next design evidence specifies the source-to-accepted-destination interface with controllers and resource obligations. The most valuable behavioral evidence measures net retained inventory and repeated dollar-responsive two-way support, followed by restored issuer/payment capacity. Reconstructing the reported price/depth history would test whether accumulation explains its trajectory. These tests remain useful without reconstructing every private wallet link.

Reproduction and preservation

The explicit experiment record is `Research/experiments/pulsechain_thesis_consensus_20261003.json`. Frozen inputs and all fifteen Markdown/JSON assessments are in `ignored artifacts/pulsechain-thesis-consensus-20261003/`. The batch includes all 69 prior study snapshots, the initial paper, round-specific synthesis inputs, user steering, raw verdict counts, final corrections, generic privacy source snapshots and hash inventories. No original archive or historical evidence identity was overwritten.

The initial paper hash is `3f64f670beec3a09b9301cbc8a3942cde3c46fe15dc523ed83f89ebf6f586c93`. All 70 initial input hashes match. Existing funding, route and shutdown inventories were rechecked: 80, 134 and 33 recorded hash entries respectively matched. The closed-loop verifier reproduced 132 checks from saved inputs; economics and routes independently reproduced it. No new blockchain calls were needed for this consensus review, and no application code changed.

```
python Research/scripts/analyze_pulsechain_arbitrage_and_shutdown.py \
--input artifacts/pulsechain-arbitrage-and-shutdown-20261003 \
--prior artifacts/pulsechain-equal-value-routes-20261003 \
--output /tmp/pulsechain-consensus-loop-reproduction-20261003.json
```

The final local verification checks assessment schemas, source-reference hashes, frozen review inputs, saved outputs and maintained-paper links. The original private recovery records, private continuity notes, deferred-settlement discussion and research ZIP remain preserved under `private-recovery-records`.

Appendix 71: PulseChain thesis: three-round comprehension and evidence study

Contents of this appendix

Method and scope	451
Findings	451
Evidence and reproduction	452
Remaining proof requirements	453

Source record: docs/studies/analysis/pulsechain_three_round_review_20260930.md.

PulseChain thesis: three-round comprehension and evidence study

30 September 2026. The [reviewed edition](#) describes a conditional price-support mechanism. This study records that edition's review method, findings and reproduction references. Its frozen inputs identify the exact text assessed; subsequent editorial revisions of the [maintained thesis \(main paper\)](#) are outside these three rounds. All blockchain work was read-only.

Method and scope

Five AI reviews assessed protocol accounting, bridges and token delivery, economics, clarity, and evidence reproduction. Each completed three rounds. Each report included five simulated audience perspectives: an Aave/Solidity engineer, a DeFi liquidity provider, a finance professional without blockchain experience, a general reader without blockchain experience, and a beginner familiar with wallets. That is **15 reports and 75 simulated audience assessments**. No human participants were recruited; these results cannot measure actual reader comprehension or establish universal understanding.

First-round reviews received the frozen paper without prior conversation and recorded a teach-back before consulting evidence or other reports. Later rounds were informed rereadings of revised inputs by the same reviews. Shared records and AI-generated reader responses limit independence. Each assessment asked what changes hands, what finances purchases, what settlement requires, which callers have authority, and what supports a dollar quote. Reports included likely errors, questions, answers and concrete edits, rather than invented human pass scores.

The paper was revised after every round. Frozen inputs, reports and final documents are retained in the review batch; their SHA-256 hashes and word counts are recorded in [the experiment manifest](#). The first input contains 6,099 whitespace-delimited words; the final thesis contains 2,994. Three diagrams show the purchase cycle, separate Portal/debt allocations, and stablecoin pool connections. Their syntax and financial meaning were reviewed; no rendered-image compatibility test was performed.

Findings

The reviews found a coherent conditional mechanism: repeated accepted inputs can finance local accumulation when an exact-token delivery and funded conversion remain usable. Retained inventory can meet buying demand, while executable bids can support selling. Holding most tokens strengthens

influence but does not itself establish dollar coverage. A fluctuating payment asset requires changing payment quantities and revaluing executable coverage against inventory sellable over the declared horizon.

The most consequential accounting distinction is between trade receipts, receipts burned against the borrower's debt, and outstanding Portal principal. One unit cannot serve both allocations. Debt repayment does not create reserve cash or settle Portal exposure. The inspected backing method also requires a currently Bridge-authorized caller, matching underlying, allowance and supplied fee. Income is one possible settlement source; its modeled multi-year horizon is not a protocol maturity. Acquired assets, outside earnings, expenses and settlement commitments must be accounted for without counting nested receipts or controlled-account fees twice.

The authority checks distinguish immediate acceptance of the observed executor nomination from the subsequent one-day configuration queue. They identify the borrower's reported owner and the HubPool owner, but do not prove arbitrary borrower execution, common private-key control or every rollup-owner role holder. Published UpgradeExecutor delegatecall authority is broader than ordinary role grants. Portal permission does not confer issuer rights over underlying tokens. The map lists observed addresses, required changes and unresolved controls.

Audience simulations repeatedly exposed confusion between a receipt and cash, LP locking and protection from swaps, a restored marginal price and a completed seller's proceeds, and token parity and dollar value. The revised examples and diagrams make those distinctions explicit. They preserve the adverse network result: repeated local support pushes the surrounding modeled U asset down to 0.694741 rather than keeping the whole network at one dollar.

Evidence and reproduction

The reproduction review reran four existing offline calculations and obtained JSON objects identical to the preserved results: liquidity growth, coordinated network support, deferred campaign/debt accounting, and Portal pricing. The [first reproduction report](#) identifies each script and result file. The economics review separately checked key V2, StableSwap, cadence and yield calculations. These checks reproduce arithmetic from saved evidence; they do not execute the proposed campaign.

Three additional fixed-block authority batches contain getters, contract-code reads and storage reads. Published source and proxy slots support the authority map, with explicit limits: not every dependency was independently compiled and matched to runtime bytecode. Older Atropa, pricing and validator snapshots retain their own block numbers. Incorrect raw getter labels and initial wrong-signature Across responses are identified in the reports and excluded from the relevant claims; frozen records are preserved.

Review strand	Round 1	Round 2	Round 3
Protocol	Report	Report	Report
Bridges	Report	Report	Report
Economics	Report	Report	Report
Clarity	Report	Report	Report
Reproduction	Report	Report	Report

Remaining proof requirements

The operating source environment, exact integrations, campaign-scale output, some account execution and issuer rights, dollar reference, funded support policy, and Ethereum exit remain unverified or unspecified. The final reports support the conditional explanation after their small clarifications; they do not certify a flawless deployment, perpetual peg or real-human comprehension. Final copy edits were applied after round three and checked locally; they were not presented as an additional blind review round. Earlier releases remain frozen.

Appendix 72: Twelve-point follow-up: local PLS exits and financing conditions

Contents of this appendix

Findings for each examined point.....	454
What is now positively established within scope	456
Numerical results and limits	456
Methods, validation and unfinished scope.....	456
Supporting studies	457

Source record: docs/studies/analysis/pulsechain_twelve_point_resolution_20260929.md.

Twelve-point follow-up: local PLS exits and financing conditions

29 September 2026. Follow-up to the skeptical review and the clarification that redemption means a local pDAI exit into PLS, WPLS or the exact inherited native-wrapping WETH contract. No Ethereum bridge-out or external-dollar payout is required. The dollar reference remains a measurement convention for local execution. Governance and operator control are scenario assumptions where not independently established.

The maintained [white paper \(main paper\)](#) incorporates the completed findings. The financing, mathematical and local-evidence studies below contain detailed derivations and records. This follow-up strengthens the conditional argument without claiming that every deployed financing path is proved executable.

Findings for each examined point

Point	Work completed and conclusion	Remaining boundary
1. Meanings of backing	Defined source custody, source solvency, withdrawal timing, funded acceptance and the holder's local PLS exit separately. A mechanically accepted weak claim can finance a finite purchase.	Solvency is still necessary for a claim of solvent ongoing financing; it is not a universal precondition for a local swap.
2. Funding source	Added full outright-sale, borrowing, issuer and relayer ledgers identifying who pays existing local assets, who receives each claim, and when liabilities are due.	Exact executable bids/loan terms for the proposed deployed source receipts remain stipulated.
3. Exact identities	Identified a historical copied-DAI/WPLS pool and native conversion for canonical WPLS and inherited WETH. Existing Aave receipt/wrapper identities remain defined by their own contracts.	Economic native-unit equivalence does not make different token addresses interchangeable inside pools or collateral adapters.

Point	Work completed and conclusion	Remaining boundary
4. Governance versus current state	Retained explicit classification of adjustable settings, changed integrations and new implementations. Effective authorized governance can conceivably remove some current blockers.	A new detailed permission-path extension did not reach completed review; no new exact-role readiness claim is promoted from it.
5. Aave cap barriers	Retained verified earlier distinctions between zero Portal capacity and independent DAI/USDT supply-cap constraints. Each subsequent wrapper operation must also pass its own limits at its execution state.	The additional detailed deposit-limit/permission review remains incomplete; raising a single cap is not treated as proof of an executable full route.
6. Portal and ordinary debt	Closed a 100-unit Portal example with 99.9 market proceeds plus 0.1 contributed underlying; ordinary repayment remains separate. Distinguished underlying-funded and aToken-funded debt repayment, including a conditional discounted-claim benefit to an eligible borrower.	Full settlement consumes its assigned cash. Third-party payment does not transfer another borrower's collateral, and no historical arbitrage profit is asserted.
7. Pending transfers and return	Retained settled-state versus in-flight custody accounting and distinguished optional source return from the pDAI holder's local exit.	Exact cancellation/finality/return rights remain implementation-specific when source backing is claimed. None is silently imposed as an Ethereum exit requirement.
8. Intermediate funding	Added per-actor, per-asset cash constraints at every step, not merely at the end. Loan maturities and operating costs belong at their actual due times.	No instantaneous cross-chain financing or future receivable is counted as current local cash without a funded advance.
9. Shared resources	Consolidated source claims, collateral pledges, relay inventory, pool cash and native-wrapper backing. Funds may be reused after actual return, but not independently counted twice.	A combined deployed-route budget still needs state-dependent execution curves and exclusive capital allocations.
10. Assets returning to market	Modeled holder exits separately from Aave withdrawals, released collateral, issuer redemption and source returns. Added an inventory-constrained sell-above-target example that replenishes PLS under assumed appreciation.	Deposits/repayments are not permanent burns. Selling retained pDAI uses inventory and may affect markets; resulting PLS is not costless profit.
11. Bounded support	Evaluated six thirty-event fee-inclusive scenarios, two appreciation scenarios, and two alternative sell-side checkpoints. Independently checked the six original scenarios and sell-side roots.	Results require the stated funded inventory, timing and reference path. They do not calibrate real selling pressure or prove continuous support under arbitrary flows.

Point	Work completed and conclusion	Remaining boundary
12. Evidence and reproduction	Verified one historical pDAI-to-WPLS branch followed by aggregate native payout, pinned local-pool reserves, and exact native-wrapper behavior. Prepared selected evidence and explicit mathematical batches with hashes.	Historical observations are not live quotes or source-financing provenance. Selected publication evidence is not the complete audit archive or independent node authentication.

What is now positively established within scope

The holder-side exit is no longer only an abstract example: a historical local swap and native withdrawal sequence has been reconciled. Its source financing is not attributed to the thesis. The exact inherited WETH and canonical WPLS wrappers provide native PLS conversion mechanics, so the theory need not prove an Ethereum redemption process to use these as local exit forms.

The financial argument now includes more than an outright sale. Borrowing, eligible issuance, Portal settlement and relayer inventory exchange have complete illustrative ledgers. None requires free creation of local purchasing assets. Each identifies the provider of the inventory and resulting exposure. This is a positive conditional composition result, rather than a list of unconnected protocol names.

The dynamic argument now includes a funded sequence of local exits with trading fees and a separate sale of retained pDAI when an assumed PLS appreciation moves the local dollar-reference quote above target. It shows how a two-sided policy could use existing inventories, while exposing budget and reference-price failure cases. It does not prove that establishing a peg causes PLS appreciation.

Numerical results and limits

At the illustrative 0.3% fee and starting reserves, the initial purchase costs 2,175,295,195.262100 PLS. With 400 million additional support PLS, thirty successive 1,000-pDAI exits each receive 10 million PLS at the fixed \$0.0001 reference. The 29 intermediate purchases cost 290,871,239.738011 PLS. These are local PLS payments; there is no bridge-out cash flow.

If the reference instead rises at event 16, fewer PLS are needed per dollar of pDAI. Under an explicitly separate sell-side policy, a 50% rise permits the modeled operator to exchange 70,153.484465 retained pDAI for 572,788,016.391764 PLS while restoring the next 1,000-pDAI quote to the target. Doubling the reference gives a different feasible branch: 130,138.650678 pDAI sold for 920,191,644.708543 PLS. The branches are alternatives, not additive proceeds.

These positive results are financed scenarios. They do not remove exact-token compatibility, role and cap checks, amount-specific liquidity, fees, timing, debt obligations or counterparty responses. A local PLS target changes the required exit asset; it does not eliminate the funding question.

Methods, validation and unfinished scope

The mathematical models are offline Decimal calculations. No transaction, governance vote, deployment or state-changing simulation was performed. The economic review independently recomputed all 180 events in the initial six scenarios using bisection rather than the quadratic solver; conservation and results agreed. It separately checked both sell-side roots and inventory bounds. The two buy-only appreciation branches use the same validated solver and run its per-event conservation and cash checks.

An automated review restriction left the contract-permission extension incomplete. That subreview was not treated as completed verification. Its unfinished detailed role and wrapper-limit work is not used to claim deployed readiness. Completed earlier protocol findings remain cited; the independent financing and local-exit work continued within its own scope.

The research suite passed 63 tests and 82 subtests after both mathematical scripts were added. The sell-side checkpoint additionally has independent formula/numerical checks and asserts token conservation and inventory limits. Document and artifact checks are recorded with the final selected publication bundle; the old experiments retain their original identities.

Supporting studies

- [Financing families, debt, settlement and local exit accounting \(Appendix 42\).](#)
 - [Fee-inclusive support, appreciation and sell-side equations \(Appendix 43\).](#)
 - [Historical local exit and exact native-wrapper evidence \(Appendix 41\).](#)
 - [Earlier Portal accounting and sampled reserve constraints \(Appendix 17\).](#)
 - [Original two-round skeptical review \(Appendix 59\).](#)
-

Appendix 73: Completeness and clarity of the pDAI white paper

Contents of this appendix

Four meanings of complete	458
What improved	459
A finite benchmark and its limits	459
A focused reference identity checkpoint	460
The evidence that would change the conclusion	460
Method and reproduction	461

Source record: docs/studies/analysis/pulsechain_whitepaper_completeness_20261003.md.

Completeness and clarity of the pDAI white paper

3 October 2026. Five fresh-context analysts followed by two shared-draft challenge rounds. Blockchain work remains read-only.

The paper now states one thesis: **distributed wallets could use private-L2 funding, accepted local issuance and common-unit arbitrage to acquire pDAI and neighboring stables, manage supply, and fund a separately specified two-sided dollar policy with timely replenishment.** It preserves just-in-time funding without assuming a central treasury or persistent local stockpile.

All five final reviews accepted the revised conditional research account with no blocking corrections. This agreement establishes review convergence, not proof of private source operation, common control, available capacity or intended purpose. The [maintained white paper \(main paper\)](#) is the resulting document.

Four meanings of complete

Question	Finding
Does the paper cover the discussed mechanisms?	Substantially complete: private L2, exact destination acceptance, local issuance/conversion, common U, distributed acquisitions, Curve bases, supply buffers, native financing, debt/backing, clocks, exits and intent are present directly or through the retained appendix/studies.
Does it specify a reviewable mechanism?	Yes as a conditional architecture, an exact local route register and a finite research benchmark. Actual source endpoints, executors, available funding and replenishment commitments remain unverified deployment fields.
Does it prove a deployed dollar-peg campaign?	No. Local components and two profitable closed trades are demonstrated; the complete fresh-source, repeated-support and restoration sequence is not.

Question	Finding
Does it establish the participants' intent?	No. Coordination, accumulation and a dollar objective remain interpretations that must be compared against independent arbitrage, monetization and replacement inflows.

Topic coverage is different from an executable deployment or certainty about intent. A research-selected test can make the proposition falsifiable without inventing the participants' actual configuration. Optional issuers, Portal, rebases, native yield and external LPs are alternative capacity branches; universal governance control is not a necessary premise.

What improved

The original main body was 6,762 words. The revised body is 1,916 words, about 72% shorter; detailed material remains in the appendix and existing studies. All 43 distinct local study-link targets from the original paper survive. The initial paper is frozen in the review batch rather than overwriting historical evidence.

The rewrite organizes the argument around five jobs: source funding, destination acceptance, funded conversion, price-sensitive trading and replenishment. It distinguishes profitable arbitrage, net acquisition, common-U relative parity, bounded dollar execution and intent. Each job needs the exact asset, effective executor, payer, resulting claims and deadline.

Residual buffer language was replaced with collectively executable inputs and permitted just-in-time delivery. Dollar-reference and restoration decisions were promoted from scattered caveats to explicit operating requirements. Copied Arbitrum/Optimism gateways, copied Across and the canonical PulseChain-Ethereum capital boundary are named separately. The core accounting rule remains one normalized eligible stable unit = U, with token admission, conversion limits and suspension conditions required.

The appendix adds a minimal direct-delivery/FEI route register. It names existing pDAI, the public FEI module, existing FEI and its Curve pool, while explicitly leaving the private source and conversion into a distinct dollar payment token open. A longer receipt/Reserve branch remains a separate construction.

A finite benchmark and its limits

The proposed benchmark uses exactly 1,000 pDAI per request, an all-in 1% band around \$1, and a 24-hour horizon. It predeclares 100 requests on each side in one-sided batches at six-minute intervals. Refused or expired eligible requests remain in the completion denominator; only counting successful trades would conceal a funding shortfall.

Quotes record a block, binding acceptance and expiry at b+2. Accepted requests must settle on time; the independently executable dollar reference is measured at settlement. Quote expiry is not a guarantee of transaction inclusion. Inputs must arrive before or within execution. Restoring a later quote cannot repair completed proceeds.

Within an hour after execution, the test requires another 1,000-pDAI order's deliverable capacity on both sides plus already accepted obligations. Remaining requests need an exact-resource/deadline ledger. Measuring next-order capacity even after the final request prevents restoration from becoming vacuous. A 1% payment-reference decline is inside the stated test envelope; larger shocks and greater flows are not silently advertised as covered.

These are research choices, not deployed contracts or promises attributed to the wallet group. The test is a specification for historical observation or separately labelled offline economic evaluation; this investigation did not submit orders or run blockchain execution simulations. Its payment candidate, actual resources, source obligations and complete service still need evidence.

A focused reference identity checkpoint

The selected payment candidate is PulseChain token 0x15d38573d2feeb82e7ad5187ab8c1d52810b1f07. At chain 369, block **27,699,585**, hash 0xd8f0bc5ca2e16f0660c45d5595a6c45712befe5a6de8f8a5830cd9accc433995, getters confirm:

- Name: USD Coin from Ethereum; symbol USDC; six decimals.
- Token implementation: 0x539a69de74e9ed69f7f909fa935d05b8caba11.
- Owner and bridge contract: 0x4fd0aaa7506f3d9cb8274bdb946ec42a1b8751ef.
- Mediator implementation: 0xe64b29deb76616ff5a39e48ea33352dff62a72e4.
- Configured foreign token: 0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48.
- Configured counterpart: 0x1715a3e4a142d8b698131108995174f37aeba10d.

Copied pUSDC at A0b on PulseChain is a different token from the candidate representation. Chain identity is essential. Proxy/implementation source responses and verified getter ABI signatures are saved with the checkpoint. Four successful batches share the same pinned header and each rechecks it.

This establishes local identity, bridge authorization and configured mappings. It does not audit Ethereum custody, destination claims, actual release, external valuation, pDAI/payment execution, benchmark capacity, private-L2 connectivity or group control. The raw responses do not substitute for those proofs.

The evidence that would change the conclusion

Three jobs provide more information than additional review agreement:

1. **Exact source/conversion and post-use restoration.** Record source rights, fresh authenticated output, destination acceptance, executor, payer, claims and resulting state. Follow one known local cycle through an actual refill. Failed acceptance, empty payment resources or missing refill exclude that route's tested capacity; a view quote or new nominal claim cannot replace the paid asset.
2. **Canonical bridge and distributed acquisition reconstruction.** Reconcile per-asset custody, destination claims, completed deposits/releases, pending obligations and price revaluation. Compare aggregate purchases, sales, issuance, LP changes and custody without requiring one visible wallet. Flat marked TVL without replacement quantities weakens the refill interpretation; churn without net acquisition weakens the accumulation interpretation.
3. **Repeated dollar-responsive support.** Observe predeclared eligible requests, all-in outcomes, funding deadlines and restored capacity under the benchmark's limits. Missed accepted commitments, out-of-band outcomes, unaccounted obligations or failed refill reject that specified service. Independent arbitrage can produce support effects; attributing coordination or peg intent remains a separate question.

One route or benchmark failing does not disprove every possible architecture. One passing episode does not establish perpetual support. Those boundaries keep the next work informative rather than turning every outcome into confirmation.

Method and reproduction

The roles were mechanism, evidence, completeness, economics and clarity. Round 1 used fresh contexts and the frozen paper without peer reports. Rounds 2 and 3 challenged shared revised papers and preceding assessments. Fifteen individual Markdown/JSON reports are preserved. No actual human comprehension test or calibrated probability was claimed.

The explicit experiment is `Research/experiments/pulsechain_whitepaper_completeness_20261003.json`. Frozen papers, the 71-study initial index, assessments, source responses and checkpoint are in ignored artifacts/`pulsechain-whitepaper-completeness-20261003/`. Read-only reference probes have explicit getter manifests named `pulsechain_dollar_reference_token_20261003.json`, `pulsechain_dollar_reference_identity_20261003.json`, `pulsechain_dollar_reference_mediator_20261003.json` and `pulsechain_dollar_reference_mapping_20261003.json` under `Research/experiments/`.

Reproduction checks validate frozen hashes, report schemas, getter ABI/view signatures, pinned headers, decoded identities, study-link preservation and saved documents. All prior archive, recovery and historical research identities remain unchanged. Application code was not modified.

Working archive and evidence directory

The research archive is supplied separately as five ordinary ZIP volumes on the private download page. Together they contain 3,560 working files, editable report text and an integrity manifest. Download all five and extract them into the same new folder. This browser edition supplies its archive separately to fit the file attachment limit.

After extraction, paths below are relative to the extracted archive. Current synthesis is in `whitepaper.md`; `thesis.md` contains the paper before the 73 appended studies. Read `release-manifest.json` for scope, file hashes and incomplete work.

Historical drafts retain their dates and can be superseded by later corrections. Model-based reviews share sources and do not constitute an independent human audit. Private conversation exports and recovery records are excluded. Public contract identifiers and third-party source attribution are retained.

Released-file hashes describe the supplied, de-identified bytes. Historical embedded hashes continue to identify their original pre-redaction records. Extract into a new research folder. Reproduction code is reference material; this release does not authorize blockchain execution.

working-files/Research/experiments/parb_gateway_control_events_20260929.json

SHA-256 1944d56521a1061fd9f6c7979a27fee25139df60eeaf2400207029a2ef33998f

working-files/Research/experiments/parb_gateway_control_getters_20260929.json

SHA-256 c49d95f3797862704541b310fa2fd74d848a3651e2a3e113a6af03cf687a1628

working-files/Research/experiments/parb_gateway_control_history_20260929.json

SHA-256 cb4967291bccdc8b4d1eb5ce4ed23543d801f1551cce92beb04b05a91f564df0

working-files/Research/experiments/parb_gateway_specialized_controls_20260929.json

SHA-256 759df8e5850d4b6913f5b9ab7699addee9217b0eaa4be68cf105bb466bbf6ce1a

working-files/Research/experiments/parb_grt_allowance_events_20260929.json

SHA-256 00f1330776bb85cf56757fa22e3c045442dec75ba4a3e633d2a77787f518b3ec

working-files/Research/experiments/parb_grt_historical_getters_20260929.json

SHA-256 ce4999da95cfd91080361752f24dc639213a73514535b96fdf69c54a2f7889e1

working-files/Research/experiments/parb_grt_withdrawal_events_20260929.json

SHA-256 175d8825ccb24e884625edcdccf36a46c1e8d690600207023ce607684adeafbc

working-files/Research/experiments/pulsechain_maker_adapter_closure_20261004.json

SHA-256 700fcc39a78885e5ed3b7cda54c3dcaaeda2878a7625250ee1fbbdb7a3ffa428

working-files/Research/experiments/pulsechain_restorative_team_review_20261004.json

SHA-256 6f1fe2629992dc4ec056f27ea91bbd3320463a8e3fd99286218ad9dcb2c2c168

working-files/Research/experiments/pulsechain_three_round_review_20260930.json

SHA-256 9a504fb68657ea5eb2173f6ff11c32ae48248f65f9c21086cde89f1ac99af114

working-files/artifacts/pulsechain-integrated-network-20261001/all-pool-token-identities.csv

SHA-256 b5a79f1130b579df24724cbeaf6cf9b5eb808f0e30b4c72501cbbc719c9d383e

working-files/artifacts/pulsechain-integrated-network-20261001/authority-comparisons.csv

SHA-256 2212f812e74397958797be6fddbae54b5ceca89c07f7e26e9189204c7953348e

working-files/artifacts/pulsechain-integrated-network-20261001/bridge-token-identities.csv

SHA-256 6ca283506f329cbc2389d5e9443479ed30a57e4bcf7155761e7a6ddf4141ad5c

working-files/artifacts/pulsechain-integrated-network-20261001/pool-catalogue.csv

SHA-256 efec4b91f367869306ef3b236cea00cc217dd66e9755f39e8e77899a70ab16c5

working-files/artifacts/pulsechain-issuer-collateral-20260930/issuer-candidates.csv

SHA-256 74f1581538438dc62a6bee094fa59f37cdada32292856cb7a914a4ffc8f437b9

working-files/artifacts/pulsechain-issuer-collateral-20260930/issuer-candidates.json

SHA-256 b86488f4aa5e52c90ce4a1561ee809ef90dfcefe1a852ffcab983643da915319

working-files/artifacts/pulsechain-issuer-collateral-20260930/pool-catalogue.csv

SHA-256 b408c0ad795184b3b0c4cef7d5d676e7388365908a098000896a1d3031325e9e

working-files/artifacts/pulsechain-issuer-collateral-20260930/pool-catalogue.json

SHA-256 849c17ceadd3342ca97da01356f852c354c042fa57743c99f38038ef6d7d9fa2

working-files/artifacts/pulsechain-issuer-collateral-20260930/token-catalogue.json

SHA-256 fdbc59b67cc45853843a7071ba0fa149c726b2a44510a25976d1185b175a892e

working-files/artifacts/pulsechain-three-round-review-20260930/final-thesis.md

SHA-256 fba94b3b31c9ee94263ae8fb35df7b27c02404130266e04a2a22b1037871d036

working-files/artifacts/pulsechain-three-round-review-20260930/round1-bridges.md

SHA-256 f228072abc4eda3606bb867779a4945b5c32f2569bc8133e1ca540b0ec0ef024

working-files/artifacts/pulsechain-three-round-review-20260930/round1-clarity.md

SHA-256 c07fbbd7431c619e6165d29a186d7ce55b369a779474ab56452f2dc0f94f5fc5

working-files/artifacts/pulsechain-three-round-review-20260930/round1-economics.md

SHA-256 a6c2178384d5f9637769bb011ee924da8b8467bb59b350fe85a7da0dcc250313

working-files/artifacts/pulsechain-three-round-review-20260930/round1-protocol.md

SHA-256 9b55331ce17f10ad4b9abf5a40acad2256f9995698e8fc5abe2f9fcb5d66911f

working-files/artifacts/pulsechain-three-round-review-20260930/round1-reproduction.md

SHA-256 db784aeeebf98255636e2013144cb4e089635524a9daa7b40c70f0885079e49a

working-files/artifacts/pulsechain-three-round-review-20260930/round2-bridges.md

SHA-256 87f0610db484e5cee9723bdf0aab8cc30af9e4328acb0e816ff762b9f790f775

working-files/artifacts/pulsechain-three-round-review-20260930/round2-clarity.md

SHA-256 15a8471d483aa886a925e0004654fa40b3e982cde8467e5e65f4efacc9a9f5bf

working-files/artifacts/pulsechain-three-round-review-20260930/round2-economics.md

SHA-256 15416b304ec7db572ad9f417b72965bad92e809c69434c3d6371556cdc875a0d

working-files/artifacts/pulsechain-three-round-review-20260930/round2-protocol.md

SHA-256 a5eeb48f9d1ea5439ce8583ff1c0ff17ad78a74b441bb4fc04637dc551bcf533

working-files/artifacts/pulsechain-three-round-review-20260930/round2-reproduction.md

SHA-256 2bdadcde71bde6d69dbef91df870b40deabd705716dcea5fa22811d1fd5ef0

working-files/artifacts/pulsechain-three-round-review-20260930/round3-bridges.md

SHA-256 01432292bf09a8b196ff8b98cea9087328eafc1341fb64a8e6c5578066014350

working-files/artifacts/pulsechain-three-round-review-20260930/round3-clarity.md

SHA-256 7d4e4c2f35672f3f80f3de761e99d90e39c877d6edad98b9ca40f5192b7f79bc

working-files/artifacts/pulsechain-three-round-review-20260930/round3-economics.md

SHA-256 751ffb163e27fce0340d599a36b3c1f70a4ab32268a2b393540247f26bd3e6ee

working-files/artifacts/pulsechain-three-round-review-20260930/round3-protocol.md

SHA-256 fa8804522eb71e24c992f6921ac62e7799aadb725de631f37031a5400fd3d8fd

working-files/artifacts/pulsechain-three-round-review-20260930/round3-reproduction.md

SHA-256 693a1e684659a22b30a06b03f7f3974e903995ebabbbc0e6d52f1fbd3b2e4deb